

简介

Ceph 存储集群至少需要 1 个 Ceph Monitor 和 2 个 OSD 守护进程

运行 Ceph 文件系统客户端时，则必须要有元数据服务器（Metadata Server）。



- **Ceph OSDs:** *Ceph OSD 守护进程*（Ceph OSD）的功能：存储数据，处理数据的复制、恢复、回填、再均衡，并通过检查其他 OSD 守护进程的心跳来向 Ceph Monitors 提供一些监控信息。当 Ceph 存储集群设定为有 2 个副本时，至少需要 2 个 OSD 守护进程，集群才能达到 `active+clean` 状态（Ceph 默认有 3 个副本，但你可以调整副本数）。
- **Monitors:** *Ceph Monitor* 维护着展示集群状态的各种图表，包括监视器映射、OSD 映射、归置组（PG）映射、和 CRUSH 映射。Ceph 保存着发生在 Monitors、OSD 和 PG 上的每一次状态变更的历史信息（称为 epoch）。
- **MDSs:** *Ceph 元数据服务器*（MDS）为 *Ceph 文件系统* 存储元数据（也就是说，Ceph 块设备和 Ceph 对象存储不使用 MDS）。元数据服务器使得 POSIX 文件系统的用户们，可以在不对 Ceph 存储集群造成负担的前提下，执行诸如 `ls`、`find` 等基本命令。

Ceph 把客户端数据保存为存储池内的对象。通过使用 CRUSH 算法，Ceph 可以计算出哪个归置组（PG）应该持有指定的对象(Object)，然后进一步计算出哪个 OSD 守护进程持有该归置组。CRUSH 算法使得 Ceph 存储集群能够动态地伸缩、再均衡和修复。

硬件推荐

Ceph 为普通硬件设计，这可使构建、维护 PB 级数据集群的费用相对低廉。规划集群硬件时，需要均衡几方面的因素，包括区域失效和潜在的性能问题。硬件规划要包含把使用 Ceph 集群的 Ceph 守护进程和其他进程恰当分布。通常，我们推荐在一台机器上只运行一种类型的守护进程。我们推荐把使用数据集群的进程（如 OpenStack、CloudStack 等）安装在别的机器上。

Tip

关于 Ceph 的高品质博客文章也值得参考，比如 [Ceph Write Throughput 1](#)、[Ceph Write Throughput 2](#)、[Argonaut v. Bobtail Performance Preview](#)、[Bobtail Performance - I/O Scheduler Comparison](#)。

CPU

Ceph（MDS）元数据服务器对 CPU 敏感，它会动态地重分布它们的负载，所以你的元数据服务器应该有足够的处理能力（如 4 核或更强悍的 CPU）。Ceph 的 OSD 运行着 RADOS 服务、用 CRUSH 计算数据存放位置、复制数据、维护它自己的集群运行图副本，因此 OSD 需要一定的处理能力（如双核 CPU）。监视器只简单地维护着集群运行图的副本，因此对 CPU 不敏感；但必须考虑机器以后是否还会运行 Ceph 监视器以外的 CPU 密集型任务。例如，如果服务器以后要运行用于计算的虚拟机（如 OpenStack Nova），你就要确保给 Ceph 进程保留了足够的处理能力，所以我们推荐在其他机器上运行 CPU 密集型任务。

RAM 内存

元数据服务器和监视器必须可以尽快地提供它们的数据，所以他们应该有足够的内存，至少每进程 1GB。OSD 的日常运行不需要那么多内存（如每进程 500MB）差不多了；然而在恢复期间它们占用内存比较大（如每进程每 TB 数据需要约 1GB 内存）。通常内存越多越好。

数据存储

要谨慎地规划数据存储配置，因为其间涉及明显的成本和性能折衷。来自操作系统的并行操作和到单个硬盘的多个守护进程并发读、写请求操作会极大地降低性能。文件系统局限性也要考虑：btrfs 尚未稳定到可以用于生产环境的程度，但它可以同时记日志并写入数据，而 xfs 和 ext4 却不能。

Important

因为 Ceph 发送 ACK 前必须把所有数据写入日志（至少对 xfs 和 ext4 来说是），因此均衡日志和 OSD 性能相当重要。

硬盘驱动器

OSD 应该有足够的空间用于存储对象数据。考虑到大硬盘的每 GB 成本，我们建议用容量大于 1TB 的硬盘。建议用 GB 数除以硬盘价格来计算每 GB 成本，因为较大的硬盘通常会对每 GB 成本有较大影响，例如，单价为 \$75 的 1TB 硬盘其每 GB 价格为 \$0.07（ $\$75/1024=0.0732$ ），又如单价为 \$150 的 3TB 硬盘其每 GB 价格为 \$0.05

($\$150/3072=0.0488$) , 这样使用 1TB 硬盘会增加 40% 的每 GB 价格, 它将表现为较低的经济性。另外, 单个驱动器容量越大, 其对应的 OSD 所需内存就越大, 特别是在重均衡、回填、恢复期间。根据经验, **1TB 的存储空间大约需要 1GB 内存。**

Tip

不顾分区而在单个硬盘上运行多个 OSD, 这样不明智!

Tip

不顾分区而在运行了 OSD 的硬盘上同时运行监视器或元数据服务器也不明智!

存储驱动器受限于寻道时间、访问时间、读写时间、还有总吞吐量, 这些物理局限性影响着整体系统性能, 尤其在系统恢复期间。因此我们推荐独立的驱动器用于安装操作系统和软件, 另外每个 OSD 守护进程占用一个驱动器。大多数 “slow OSD” 问题的起因都是在相同的硬盘上运行了操作系统、多个 OSD、和/或多个日志文件。鉴于解决性能问题的成本差不多会超过另外增加磁盘驱动器, 你应该在设计时就避免增加 OSD 存储驱动器的负担来提升性能。

Ceph 允许你在每块硬盘驱动器上运行多个 OSD, 但这会导致资源竞争并降低总体吞吐量; Ceph 也允许把日志和对象数据存储在同一驱动器上, 但这会增加记录写日志并回应客户端的延时, 因为 Ceph 必须先写入日志才会回应确认了写动作。btrfs 文件系统能同时写入日志数据和对象数据, xfs 和 ext4 却不能。

Ceph 最佳实践指示, 你应该分别在单独的硬盘运行操作系统、OSD 数据和 OSD 日志。

固态硬盘

一种提升性能的方法是使用固态硬盘 (SSD) 来降低随机访问时间和读延时, 同时增加吞吐量。SSD 和硬盘相比每 GB 成本通常要高 10 倍以上, 但访问时间至少比硬盘快 100 倍。

SSD 没有可移动机械部件, 所以不存在和硬盘一样的局限性。但 SSD 也有局限性, 评估 SSD 时, 顺序读写性能很重要, 在为多个 OSD 存储日志时, 有着 400MB/s 顺序读写吞吐量的 SSD 其性能远高于 120MB/s 的。

Important

我们建议发掘 SSD 的用法来提升性能。然而在大量投入 SSD 前, 我们强烈建议核实 SSD 的性能指标, 并在测试环境下衡量性能。

正因为 SSD 没有移动机械部件，所以它很适合 Ceph 里不需要太多存储空间的地方。相对廉价的 SSD 很诱人，慎用！可接受的 IOPS 指标对选择用于 Ceph 的 SSD 还不够，用于日志和 SSD 时还有几个重要考量：

- **写密集语义：** 记日志涉及写密集语义，所以你要确保选用的 SSD 写入性能和硬盘相当或好于硬盘。廉价 SSD 可能在加速访问的同时引入写延时，有时候高性能硬盘的写入速度可以和便宜 SSD 相媲美。
- **顺序写入：** 在一个 SSD 上为多个 OSD 存储多个日志时也必须考虑 SSD 的顺序写入极限，因为它们要同时处理多个 OSD 日志的写入请求。
- **分区对齐：** 采用了 SSD 的一个常见问题是人们喜欢分区，却常常忽略了分区对齐，这会导致 SSD 的数据传输速率慢很多，所以请确保分区对齐了。

SSD 用于对象存储太昂贵了，但是把 OSD 的日志存到 SSD、把对象数据存储到独立的硬盘可以明显提升性能。osd journal 选项的默认值是 `/var/lib/ceph/osd/$cluster-$id/journal`，你可以把它挂载到一个 SSD 或 SSD 分区，这样它就不再是和对象数据一样存储在同一个硬盘上的文件了。

提升 CephFS 文件系统性能的一种方法是从 CephFS 文件内容里分离出元数据。Ceph 提供了默认的 metadata 存储池来存储 CephFS 元数据，所以你不需要给 CephFS 元数据创建存储池，但可以给它创建一个仅指向某主机 SSD 的 CRUSH 运行图。详见[给存储池指定 OSD](#)。

控制器

硬盘控制器对写吞吐量也有显著影响，要谨慎地选择，以免产生性能瓶颈。

Tip

Ceph blog 通常是优秀的 Ceph 性能问题来源，见 [Ceph Write Throughput 1](#) 和 [Ceph Write Throughput 2](#)。

其他注意事项

你可以在同一主机上运行多个 OSD，但要确保 OSD 硬盘总吞吐量不超过为客户端提供读写服务所需的网络带宽；还要考虑集群在每台主机上所存储的数据占总体的百分比，如果一台主机所占百分比太大而它挂了，就可能导致诸如超过 full ratio 的问题，此问题会使 Ceph 中止运作以防数据丢失。

如果每台主机运行多个 OSD，也得保证内核是最新的。参阅[操作系统推荐](#)里关于 glibc 和 syncfs(2) 的部分，确保硬件性能可达期望值。

OSD 数量较多（如 20 个以上）的主机会派生出大量线程，尤其是在恢复和重均衡期间。很多 Linux 内核默认的最大线程数较小（如 32k 个），如果您遇到了这类问题，可以把 `kernel.pid_max` 值调高些。理论最大值是 4194303。例如把下列这行加入 `/etc/sysctl.conf` 文件：

```
kernel.pid_max = 4194303
```

网络

建议每台机器最少两个千兆网卡，现在大多数机械硬盘都能达到大概 100MB/s 的吞吐量，网卡应该能处理所有 OSD 硬盘总吞吐量，**所以推荐最少两个千兆网卡，分别用于公网（前端）和集群网络（后端）**。集群网络（最好别连接到国际互联网）用于处理由数据复制产生的额外负载，而且可防止拒绝服务攻击，拒绝服务攻击会干扰数据归置组，使之在 OSD 数据复制时不能回到 `active + clean` 状态。**请考虑部署万兆网卡**。通过 1Gbps 网络复制 1TB 数据耗时 3 小时，而 3TB（典型配置）需要 9 小时，相比之下，如果使用 10Gbps 复制时间可分别缩减到 20 分钟和 1 小时。**在一个 PB 级集群中，OSD 磁盘失败是常态，而非异常**；在性价比合理的前提下，系统管理员想让 PG 尽快从 `degraded`（降级）状态恢复到 `active + clean` 状态。另外，一些部署工具（如 Dell 的 Crowbar）部署了 5 个不同的网络，但使用了 VLAN 以提高网络和硬件可管理性。VLAN 使用 802.1q 协议，还需要采用支持 VLAN 功能的网卡和交换机，增加的硬件成本可用节省的运营（网络安装、维护）成本抵消。使用 VLAN 来处理集群和计算栈（如 OpenStack、CloudStack 等等）之间的 VM 流量时，采用 10G 网卡仍然值得。**每个网络的机架路由器到核心路由器应该有更大的带宽，如 40Gbps 到 100Gbps。**

服务器应配置底板管理控制器（Baseboard Management Controller, BMC），管理和部署工具也应该大规模使用 BMC，**所以请考虑带外网络管理的成本/效益平衡**，此程序管理着 SSH 访问、VM 映像上传、操作系统安装、端口管理、等等，会徒增网络负载。运营 3 个网络有点过分，但是每条流量路径都指示了部署一个大型数据集群前要仔细考虑的潜能力、吞吐量、性能瓶颈。

故障域

故障域指任何导致不能访问一个或多个 OSD 的故障，可以是主机上停止的进程、硬盘故障、操作系统崩溃、有问题的网卡、损坏的电源、断网、断电等等。规划硬件需求时，要在多个需求间寻求平衡点，像付出很多努力减少故障域带来的成本削减、隔离每个潜在故障域增加的成本。

最低硬件推荐

Ceph 可以运行在廉价的普通硬件上，小型生产集群和开发集群可以在一般的硬件上。

进程	条件	最低建议
ceph-osd	Processor	• 1x 64-bit AMD-64 • 1x 32-bit ARM dual-core or better • 1x i386 dual-core
	RAM	~1GB for 1TB of storage per daemon
	Volume Storage	1x storage drive per daemon
	Journal	1x SSD partition per daemon (optional)
	Network	2x 1GB Ethernet NICs
ceph-mon	Processor	• 1x 64-bit AMD-64/i386 • 1x 32-bit ARM dual-core or better • 1x i386 dual-core
	RAM	1 GB per daemon
	Disk Space	10 GB per daemon
	Network	2x 1GB Ethernet NICs
ceph-mds	Processor	• 1x 64-bit AMD-64 quad-core • 1x 32-bit ARM quad-core • 1x i386 quad-core
	RAM	1 GB minimum per daemon
	Disk Space	1 MB per daemon
	Network	2x 1GB Ethernet NICs

Tip

如果在只有一块硬盘的机器上运行 OSD，要把数据和操作系统分别放到不同分区；一般来说，我们推荐操作系统和数据分别使用不同的硬盘。

生产集群实例

PB 级生产集群也可以使用普通硬件，但应该配备更多内存、CPU 和数据存储空间来解决流量压力。

DELL 实例

一个最新（2012）的 Ceph 集群项目使用了 2 个相当强悍的 OSD 硬件配置，和稍逊的监视器配置。

Configuration	Criteria	Minimum Recommended
Dell PE R510	Processor	2x 64-bit quad-core Xeon CPUs
	RAM	16 GB
	Volume Storage	8x 2TB drives. 1 OS, 7 Storage
	Client Network	2x 1GB Ethernet NICs
	OSD Network	2x 1GB Ethernet NICs
	Mgmt. Network	2x 1GB Ethernet NICs
Dell PE R515	Processor	1x hex-core Opteron CPU
	RAM	16 GB

Configuration	Criteria	Minimum Recommended
	Volume Storage	12x 3TB drives. Storage
	OS Storage	1x 500GB drive. Operating System.
	Client Network	2x 1GB Ethernet NICs
	OSD Network	2x 1GB Ethernet NICs
	Mgmt. Network	2x 1GB Ethernet NICs

推荐操作系统

CEPH 依赖

按常规来说，我们建议在较新的 Linux 发行版上部署 Ceph ；同样，要选择长期支持的版本。

LINUX 内核

- **Ceph 内核态客户端**

当前我们推荐：

- 4.1.4 or later
- 3.16.3 or later (rbd deadlock regression in 3.16.[0-2])
- *NOT* v3.15.* (rbd deadlock regression)
- 3.14.*

如果您坚持用很旧的，可以考虑这些：

- 3.10.*

firefly (CRUSH_TUNABLES3) 这个版本的可选选项到 3.15 版才开始支持。详情见 [CRUSH 可调值](#)。

- **B-tree 文件系统 (Btrfs)**

如果您想在 `btrfs` 上运行 Ceph，我们推荐使用一个最新的 Linux 内核 (3.14 或更新)。

系统平台

下面的表格展示了 Ceph 需求和各种 Linux 发行版的对应关系。一般来说，Ceph 对内核和系统初始化阶段的依赖很少 (如 `sysvinit`、`upstart`、`systemd`)。

INFERNALIS (9.1.0)

Distro	Release	Code Name	Kernel	Notes	Testing
CentOS	7	N/A	linux-3.10.0		B, I, C
Debian	8.0	Jessie	linux-3.16.0	1, 2	B, I
Fedora	22	N/A	linux-3.14.0		B, I
RHEL	7	Maipo	linux-3.10.0		B, I
Ubuntu	14.04	Trusty Tahr	linux-3.13.0		B, I, C

HAMMER (0.94)

Distro	Release	Code Name	Kernel	Notes	Testing
--------	---------	-----------	--------	-------	---------

Distro	Release	Code Name	Kernel	Notes	Testing
CentOS	6	N/A	linux-2.6.32	1, 2	
CentOS	7	N/A	linux-3.10.0		B, I, C
Debian	7.0	Wheezy	linux-3.2.0	1, 2	
Ubuntu	12.04	Precise Pangolin	linux-3.2.0	1, 2	
Ubuntu	14.04	Trusty Tahr	linux-3.13.0		B, I, C

FIREFLY (0.80)

Distro	Release	Code Name	Kernel	Notes	Testing
CentOS	6	N/A	linux-2.6.32	1, 2	B, I
CentOS	7	N/A	linux-3.10.0		B
Debian	6.0	Squeeze	linux-2.6.32	1, 2, 3	B
Debian	7.0	Wheezy	linux-3.2.0	1, 2	B

Distro	Release	Code Name	Kernel	Notes	Testing
Fedora	19	Schrödinger's Cat	linux-3.10.0		B
Fedora	20	Heisenbug	linux-3.14.0		B
RHEL	6	Santiago	linux-2.6.32	1, 2	B, I, C
RHEL	7	Maipo	linux-3.10.0		B, I, C
Ubuntu	12.04	Precise Pangolin	linux-3.2.0	1, 2	B, I, C
Ubuntu	14.04	Trusty Tahr	linux-3.13.0		B, I, C

附注

- **1:** 默认内核 `btrfs` 版本较老，不推荐用于 `ceph-osd` 存储节点；要升级到推荐的内核，或者改用 `xfs`、`ext4`。
- **2:** 默认内核带的 `Ceph` 客户端较老，不推荐做内核空间客户端（内核 `RBD` 或 `Ceph` 文件系统），请升级到推荐内核。
- **3:** 默认内核或已安装的 `glibc` 版本若不支持 `syncfs(2)` 系统调用，同一台机器上使用 `xfs` 或 `ext4` 的 `ceph-osd` 守护进程性能不会如愿。

测试版

- **B:** 我们会为此平台构建发布包。对其中的某些平台，可能也会持续地编译所有分支、做基本单元测试。
- **I:** 我们在这个平台上做基本的安装和功能测试。
- **C:** 我们在这个平台上持续地做全面的功能、退化、压力测试，包括开发分支、预发布版本、正式发布版本。

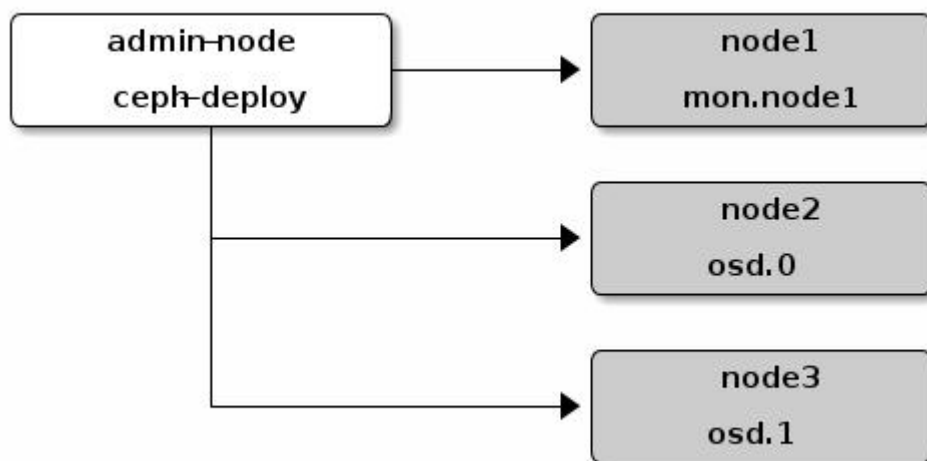
安装（快速）

预检

New in version 0.60.

我们建议安装一个 `ceph-deploy` 管理节点和一个三节点的 **Ceph 存储集群** 来研究 Ceph 的基本特性。这篇**预检**会帮你准备一个 `ceph-deploy` 管理节点、以及三个 Ceph 节点（或虚拟机），以此构成 Ceph 存储集群。在进行下一步之前，请参见**操作系统推荐**以确认你安装了合适的 Linux 发行版。如果你在整个生产集群中只部署了单一 Linux 发行版的同一版本，那么在排查生产环境中遇到的问题时就会容易一点。

在下面的描述中 **节点**代表一台机器。



安装 CEPH 部署工具

把 Ceph 仓库添加到 `ceph-deploy` 管理节点，然后安装 `ceph-deploy`。

高级包管理工具（APT）

在 Debian 和 Ubuntu 发行版上，执行下列步骤：

1. 添加 release key：

```
wget -q -O-  
'https://download.ceph.com/keys/release.asc' | sudo  
apt-key add -
```

2. 添加 Ceph 软件包源，用 Ceph 稳定版
(如 `cuttlefish`、`dumpling`、`emperor`、`firefly` 等等) 替换
掉 `{ceph-stable-release}`。例如：

```
echo deb http://download.ceph.com/debian-{ceph-  
stable-release}/ $(lsb_release -sc) main | sudo tee  
/etc/apt/sources.list.d/ceph.list
```

3. 更新你的仓库，并安装 `ceph-deploy`：

```
sudo apt-get update && sudo apt-get install ceph-  
deploy
```

Note

你也可以从欧洲镜像 `eu.ceph.com` 下载软件包，只需把 `http://ceph.com/` 替换成 `http://eu.ceph.com/` 即可。

[红帽包管理工具 \(RPM\)](#)

在 Red Hat (`rhel6`、`rhel7`)、CentOS (`el6`、`el7`) 和 Fedora 19-20 (`f19` - `f20`) 上执行下列步骤：

1. 在 RHEL7 上，用 `subscription-manager` 注册你的目标机器，确认你的订阅，并启用安装依赖包的“Extras”软件仓库。例如：

```
sudo subscription-manager repos --enable=rhel-7-  
server-extras-rpms
```

2. 在 RHEL6 上，安装并启用 Extra Packages for Enterprise Linux (EPEL) 软件仓库。
请查阅 [EPEL wiki](#) 获取更多信息。
3. 在 CentOS 上，可以执行下列命令：

```
sudo yum install -y yum-utils && sudo yum-config-  
manager --add-repo  
https://dl.fedoraproject.org/pub/epel/7/x86_64/ &&  
sudo yum install --nogpgcheck -y epel-release &&  
sudo rpm --import /etc/pki/rpm-gpg/RPM-GPG-KEY-EPEL-  
7 && sudo rm /etc/yum.repos.d/dl.fedoraproject.org*
```

4. 把软件包源加入软件仓库。用文本编辑器创建一个 YUM (Yellowdog Updater, Modified) 库文件，其路径为 `/etc/yum.repos.d/ceph.repo`。例如：

```
sudo vim /etc/yum.repos.d/ceph.repo
```

把如下内容粘贴进去，用 Ceph 的最新主稳定版名字替换 `{ceph-stable-release}`（如 `firefly`），用你的 Linux 发行版名字替换 `{distro}`（如 `el6` 为 CentOS 6、`el7` 为 CentOS 7、`rhel6` 为 Red Hat 6.5、`rhel7` 为 Red Hat 7、`fc19` 是 Fedora 19、`fc20` 是 Fedora 20）。最后保存到 `/etc/yum.repos.d/ceph.repo` 文件中。

```
[ceph-noarch]  
  
name=Ceph noarch packages  
  
baseurl=http://download.ceph.com/rpm-{ceph-  
release}/{distro}/noarch  
  
enabled=1  
  
gpgcheck=1  
  
type=rpm-md  
  
gpgkey=https://download.ceph.com/keys/release.asc
```

5. 更新软件库并安装 `ceph-deploy`：

```
sudo yum update && sudo yum install ceph-deploy
```

Note

你也可以从欧洲镜像 eu.ceph.com 下载软件包，只需把 <http://ceph.com/> 替换成 <http://eu.ceph.com/> 即可。

CEPH 节点安装

你的管理节点必须能够通过 SSH 无密码地访问各 Ceph 节点。如果 `ceph-deploy` 以某个普通用户登录，那么这个用户必须有无密码使用 `sudo` 的权限。

安装 NTP

我们建议在所有 Ceph 节点上安装 NTP 服务（特别是 Ceph Monitor 节点），以免因时钟漂移导致故障，详情见[时钟](#)。

在 CentOS / RHEL 上，执行：

```
sudo yum install ntp ntpdate ntp-doc
```

在 Debian / Ubuntu 上，执行：

```
sudo apt-get install ntp
```

确保在各 Ceph 节点上启动了 NTP 服务，并且要使用同一个 NTP 服务器，详情见[NTP](#)。

安装 SSH 服务器

在**所有** Ceph 节点上执行如下步骤：

1. 在各 Ceph 节点安装 SSH 服务器（如果还没有）：(debain)

```
sudo apt-get install openssh-server
```

或者(centos)

```
sudo yum install openssh-server
```

2. 确保**所有** Ceph 节点上的 SSH 服务器都在运行。

创建部署 Ceph 的用户

ceph-deploy 工具必须以普通用户登录 Ceph 节点，且此用户拥有无密码使用 **sudo** 的权限，因为它需要在安装软件及配置文件的过程中，不必输入密码。

较新版的 **ceph-deploy** 支持用 **--username** 选项提供可无密码使用 **sudo** 的用户名（包括 **root**，虽然**不建议**这样做）。使用 **ceph-deploy --username {username}** 命令时，指定的用户必须能够通过无密码 SSH 连接到 Ceph 节点，因为 **ceph-deploy** 中途不会提示输入密码。

我们建议在集群内的**所有** Ceph 节点上给 **ceph-deploy** 创建一个特定的用户，但**不要用**“**ceph**”这个名字。全集群统一的用户名可简化操作（非必需），然而你应该避免使用知名用户名，因为黑客们会用它做暴力破解（如 **root**、**admin**、**{productname}**）。后续步骤描述了如何创建无 **sudo** 密码的用户，你要用自己取的名字替换 **{username}**。

Note

从 **Infernalis 版**起，用户名“**ceph**”保留给了 Ceph 守护进程。如果 Ceph 节点上已经有了“**ceph**”用户，升级前必须先删掉这个用户。

1. 在各 Ceph 节点创建新用户。

```
ssh user@ceph-server

sudo useradd -d /home/{username} -m {username}

sudo passwd {username}
```

2. 确保各 Ceph 节点上新创建的用户都有 **sudo** 权限。

```
echo "{username} ALL = (root) NOPASSWD:ALL" | sudo
tee /etc/sudoers.d/{username}
```



```
sudo chmod 0440 /etc/sudoers.d/{username}
```

允许无密码SSH 登录

正因为 `ceph-deploy` 不支持输入密码，你必须在管理节点上生成 SSH 密钥并把其公钥分发到各 Ceph 节点。`ceph-deploy` 会尝试给初始 monitors 生成 SSH 密钥对。

1. 生成 SSH 密钥对，但不要使用 `sudo` 或 `root` 用户。提示“Enter passphrase”时，直接回车，口令即为空：

```
# ssh-keygen

Generating public/private key pair.

Enter file in which to save the key (/ceph-admin/.ssh/id_rsa):

Enter passphrase (empty for no passphrase):

Enter same passphrase again:

Your identification has been saved in /ceph-admin/.ssh/id_rsa.

Your public key has been saved in /ceph-admin/.ssh/id_rsa.pub.
```

2. 把公钥拷贝到各 Ceph 节点，把下列命令中的 `{username}` 替换成前面[创建部署 Ceph 的用户](#)里的用户名。

```
ssh-copy-id {username}@node1

ssh-copy-id {username}@node2

ssh-copy-id {username}@node3
```

3. **（推荐做法）**修改 `ceph-deploy` 管理节点上的 `~/.ssh/config` 文件，这样 `ceph-deploy` 就能用你所建的用户名登录 Ceph 节点了，而无需每次执行 `ceph-deploy` 都要指定 `--username {username}`。这样做同时也简化了 `ssh` 和 `scp` 的用法。把 `{username}` 替换成你创建的用户名。

```
Host node1

    Hostname node1

    User {username}

Host node2

    Hostname node2

    User {username}

Host node3

    Hostname node3

    User {username}
```

引导时联网

Ceph 的各 OSD 进程通过网络互联并向 Monitors 上报自己的状态。如果网络默认为 `off`，那么 Ceph 集群在启动时就不能上线，直到你打开网络。

某些发行版（如 CentOS）默认关闭网络接口。所以需要确保网卡在系统启动时都能启动，这样 Ceph 守护进程才能通过网络通信。例如，在 Red Hat 和 CentOS 上，需进入 `/etc/sysconfig/network-scripts` 目录并确保 `ifcfg-{iface}` 文件中的 `ONBOOT` 设置成了 `yes`。

确保联通性

用 `ping` 短主机名（`hostname -s`）的方式确认网络联通性。解决掉可能存在的主机名解析问题。

Note

主机名应该解析为网络 IP 地址，而非回环接口 IP 地址（即主机名应该解析成非 127.0.0.1 的 IP 地址）。如果你的管理节点同时也是一个 Ceph 节点，也要确认它能正确解析自己的主机名和 IP 地址（即非回环 IP 地址）。

开放所需端口

Ceph Monitors 之间默认使用 6789 端口通信，OSD 之间默认用 6800:7300 这个范围内的端口通信。详情见[网络配置参考](#)。Ceph OSD 能利用多个网络连接进行与客户端、monitors、其他 OSD 间的复制和心跳的通信。

某些发行版（如 RHEL）的默认防火墙配置非常严格，你可能需要调整防火墙，允许相应的入站请求，这样客户端才能与 Ceph 节点上的守护进程通信。

对于 RHEL 7 上的 `firewalld`，要对公共域开放 Ceph Monitors 使用的 6789 端口和 OSD 使用的 6800:7300 端口范围，并且要配置为永久规则，这样重启后规则仍有效。例如：

```
sudo firewall-cmd --zone=public --add-port=6789/tcp --permanent
```

若使用 `iptables`，要开放 Ceph Monitors 使用的 6789 端口和 OSD 使用的 6800:7300 端口范围，命令如下：

```
sudo iptables -A INPUT -i {iface} -p tcp -s {ip-address}/{netmask} --dport 6789 -j ACCEPT
```

在每个节点上配置好 `iptables` 之后要一定要保存，这样重启之后才依然有效。例如：

```
/sbin/service iptables save
```

终端 (TTY)

在 CentOS 和 RHEL 上执行 `ceph-deploy` 命令时可能会报错。如果你的 Ceph 节点默认设置了 `requiretty`，执行 `sudo visudo` 禁用它，并找到 `Defaults requiretty` 选

项，把它改为 `Defaults:ceph !requiretty` 或者直接注释掉，这样 `ceph-deploy` 就可以用之前创建的用户（[创建部署 Ceph 的用户](#)）连接了。

Note

编辑配置文件 `/etc/sudoers` 时，必须用 `sudo visudo` 而不是文本编辑器。

SELINUX

在 CentOS 和 RHEL 上，SELinux 默认为 `Enforcing` 开启状态。为简化安装，我们建议把 SELinux 设置为 `Permissive` 或者完全禁用，也就是在加固系统配置前先确保集群的安装、配置没问题。用下列命令把 SELinux 设置为 `Permissive`：

```
sudo setenforce 0
```

要使 SELinux 配置永久生效（如果它的确是问题根源），需修改其配置文件 `/etc/selinux/config`。

优先级/首选项

确保你的包管理器安装了优先级/首选项包且已启用。在 CentOS 上你也许得安装 EPEL，在 RHEL 上你也许得启用可选软件库。

```
sudo yum install yum-plugin-priorities
```

比如在 RHEL 7 服务器上，可用下列命令安装 `yum-plugin-priorities` 并启用 `rhel-7-server-optional-rpms` 软件库：

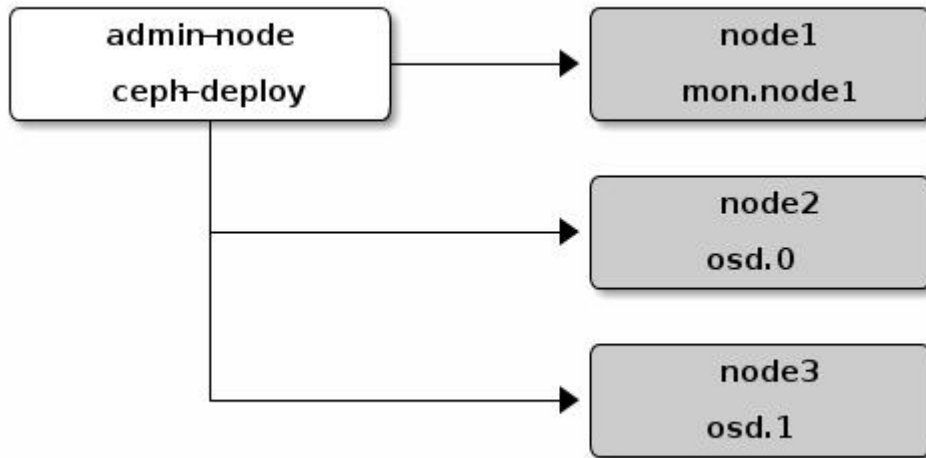
```
sudo yum install yum-plugin-priorities --enablerepo=rhel-7-server-optional-rpms
```

总结

快速入门的预检部分到此结束，请继续[存储集群快速入门](#)。

存储集群快速入门

如果你还没完成**预检**，请先做完。本篇**快速入门**用 `ceph-deploy` 从管理节点建立一个 **Ceph 存储集群**，该集群包含三个节点，以此探索 Ceph 的功能。



第一次练习时，我们创建一个 Ceph 存储集群，它有一个 Monitor 和两个 OSD 守护进程。一旦集群达到 `active + clean` 状态，再扩展它：增加第三个 OSD、增加元数据服务器和两个 Ceph Monitors。为获得最佳体验，先在管理节点上创建一个目录，用于保存 `ceph-deploy` 生成的配置文件和密钥对。

```
mkdir my-cluster  
  
cd my-cluster
```

`ceph-deploy` 会把文件输出到当前目录，所以请确保在此目录下执行 `ceph-deploy`。

Important

如果你是用另一普通用户登录的，不要用 `sudo` 或在 `root` 身份运行 `ceph-deploy`，因为它不会在远程主机上调用所需的 `sudo` 命令。

禁用 requiretty

在某些发行版（如 CentOS）上，执行 `ceph-deploy` 命令时，如果你的 Ceph 节点默认设置了 `requiretty` 那就会遇到报错。可以这样禁用此功能：执行 `sudo visudo`，找到 `Defaults requiretty` 选项，把它改为 `Defaults:ceph !requiretty`，这样 `ceph-deploy` 就能用 `ceph` 用户登录并使用 `sudo` 了。

创建集群

如果在某些地方碰到麻烦，想从头再来，可以用下列命令清除配置：

```
ceph-deploy purgedata {ceph-node} [{ceph-node}]  
  
ceph-deploy forgetkeys
```

用下列命令可以连 Ceph 安装包一起清除：

```
ceph-deploy purge {ceph-node} [{ceph-node}]
```

如果执行了 `purge`，你必须重新安装 Ceph。

在管理节点上，进入刚创建的放置配置文件的目录，用 `ceph-deploy` 执行如下步骤。

1. 创建集群。

```
ceph-deploy new {initial-monitor-node(s)}
```

例如：

```
ceph-deploy new node1
```

在当前目录下用 `ls` 和 `cat` 检查 `ceph-deploy` 的输出，应该有一个 Ceph 配置文件、一个 monitor 密钥环和一个日志文件。详情见 [ceph-deploy new -h](#)。

2. 把 Ceph 配置文件里的默认副本数从 3 改成 2，这样只有两个 OSD 也可以达到 `active + clean` 状态。把下面这行加入 `[global]` 段：

```
osd pool default size = 2
```

3. 如果你有多个网卡，可以把 `public network` 写入 Ceph 配置文件的 `[global]` 段下。详情见[网络配置参考](#)。

```
public network = {ip-address}/{netmask}
```

4. 安装 Ceph。

```
ceph-deploy install {ceph-node} [{ceph-node} ...]
```

例如：

```
ceph-deploy install admin-node node1 node2 node3
```

`ceph-deploy` 将在各节点安装 Ceph。 **注：**如果你执行过 `ceph-deploy purge`，你必须重新执行这一步来安装 Ceph。

5. 配置初始 monitor(s)、并收集所有密钥：

```
ceph-deploy mon create-initial
```

完成上述操作后，当前目录里应该会出现这些密钥环：

- `{cluster-name}.client.admin.keyring`
- `{cluster-name}.bootstrap-osd.keyring`
- `{cluster-name}.bootstrap-mds.keyring`
- `{cluster-name}.bootstrap-rgw.keyring`

Note

只有在安装 Hammer 或更高版时才会创建 `bootstrap-rgw` 密钥环。

Note

如果此步失败并输出类似于如下信息 “Unable to find /etc/ceph/ceph.client.admin.keyring” , 请确认 ceph.conf 中为 monitor 指定的 IP 是 Public IP , 而不是 Private IP。

1. 添加两个 OSD 。为了快速地安装，这篇快速入门把目录而非整个硬盘用于 OSD 守护进程。如何为 OSD 及其日志使用独立硬盘或分区，请参考 [ceph-deploy osd](#) 。登录到 Ceph 节点、并给 OSD 守护进程创建一个目录。

```
ssh node2
sudo mkdir /var/local/osd0
exit

ssh node3
sudo mkdir /var/local/osd1
exit
```

然后，从管理节点执行 `ceph-deploy` 来准备 OSD 。

```
ceph-deploy osd prepare {ceph-node}:/path/to/directory
```

例如：

```
ceph-deploy osd prepare node2:/var/local/osd0 node3:/var/local/osd1
```

最后，激活 OSD 。

```
ceph-deploy osd activate {ceph-node}:/path/to/directory
```

例如：

```
ceph-deploy osd activate node2:/var/local/osd0 node3:/var/local/osd1
```

2. 用 `ceph-deploy` 把配置文件和 admin 密钥拷贝到管理节点和 Ceph 节点，这样你每次执行 Ceph 命令行时就无需指定 monitor 地址和 `ceph.client.admin.keyring` 了。


```
ceph-deploy admin {admin-node} {ceph-node}
```

例如：

```
ceph-deploy admin admin-node node1 node2 node3
```

`ceph-deploy` 和本地管理主机 (`admin-node`) 通信时，必须通过主机名可达。必要时可修改 `/etc/hosts`，加入管理主机的名字。

3. 确保你对 `ceph.client.admin.keyring` 有正确的操作权限。

```
sudo chmod +r /etc/ceph/ceph.client.admin.keyring
```

4. 检查集群的健康状况。

```
ceph health
```

等 peering 完成后，集群应该达到 `active + clean` 状态。

操作集群

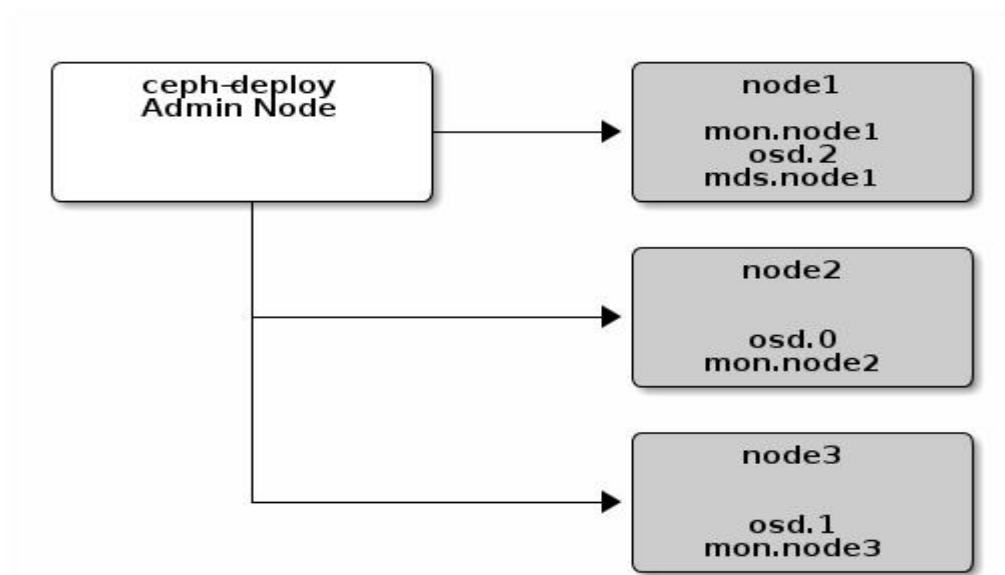
用 `ceph-deploy` 部署完成后它会自动启动集群。要在 Debian/Ubuntu 发行版下操作集群守护进程，参见[用 Upstart 运行 Ceph](#)；要在 CentOS、Red Hat、Fedora 和 SLES 下操作集群守护进程，参见[用 sysvinit 运行 Ceph](#)。

关于 peering 和集群健康状况请参见[监控集群](#)；关于 OSD 守护进程和归置组 (placement group) 健康状况参见[监控 OSD 和归置组](#)；关于用户管理请参见[用户管理](#)。

Ceph 集群部署完成后，你可以尝试一下管理功能、`rados` 对象存储命令，之后可以继续快速入门手册，了解 Ceph 块设备、Ceph 文件系统和 Ceph 对象网关。

扩展集群（扩容）

一个基本的集群启动并开始运行后，下一步就是扩展集群。在 `node1` 上添加一个 OSD 守护进程和一个元数据服务器。然后分别在 `node2` 和 `node3` 上添加 Ceph Monitor，以形成 Monitors 的法定人数。



添加 OSD

你运行的这个三节点集群只是用于演示的，把 OSD 添加到 monitor 节点就行。

```
ssh node1
sudo mkdir /var/local/osd2
exit
```

然后，从 `ceph-deploy` 节点准备 OSD。

```
ceph-deploy osd prepare {ceph-node}:/path/to/directory
```

例如：

```
ceph-deploy osd prepare node1:/var/local/osd2
```

最后，激活 OSD 。

```
ceph-deploy osd activate {ceph-node}:/path/to/directory
```

例如：

```
ceph-deploy osd activate node1:/var/local/osd2
```

一旦你新加了 OSD ， Ceph 集群就开始重均衡，把归置组迁移到新 OSD 。可以用下面的 `ceph` 命令观察此过程：

```
ceph -w
```

你应该能看到归置组状态从 `active + clean` 变为 `active` ，还有一些降级的对象；迁移完成后又会回到 `active + clean` 状态（ `Control-C` 退出 ）。

添加元数据服务器

至少需要一个元数据服务器才能使用 CephFS ，执行下列命令创建元数据服务器：

```
ceph-deploy mds create {ceph-node}
```

例如：

```
ceph-deploy mds create node1
```

Note

当前生产环境下的 Ceph 只能运行一个元数据服务器。你可以配置多个，但现在我们还不会为多个元数据服务器的集群提供商业支持。

添加 RGW 例程

要使用 Ceph 的 *Ceph 对象网关* 组件，必须部署 *RGW* 例程。用下列方法创建新 RGW 例程：

```
ceph-deploy rgw create {gateway-node}
```

例如：

```
ceph-deploy rgw create node1
```

Note

这个功能是从 **Hammer** 版和 `ceph-deploy v1.5.23` 才开始有的。

RGW 例程默认会监听 7480 端口，可以更改该节点 `ceph.conf` 内与 **RGW** 相关的配置，如下：

```
[client]

rgw frontends = civetweb port=80
```

用的是 IPv6 地址的话：

```
[client]

rgw frontends = civetweb port=[::]:80
```

添加 MONITORS

Ceph 存储集群需要至少一个 Monitor 才能运行。为达到高可用，典型的 Ceph 存储集群会运行多个 Monitors，这样在单个 Monitor 失败时不会影响 Ceph 存储集群的可用性。Ceph 使用 PASOX 算法，此算法要求有多半 monitors（即 1、2:3、3:4、3:5、4:6 等）形成法定人数。

新增两个监视器到 Ceph 集群。

```
ceph-deploy mon add {ceph-node}
```

例如：

```
ceph-deploy mon add node2 node3
```

新增 Monitor 后，Ceph 会自动开始同步并形成法定人数。你可以用下面的命令检查法定人数状态：

```
ceph quorum_status --format json-pretty
```

Tip

当你的 Ceph 集群运行着多个 monitor 时，各 monitor 主机上都**应该**配置 NTP，而且要确保这些 monitor 位于 NTP 服务的同一级。

存入/检出对象数据

要把对象存入 Ceph 存储集群，客户端必须做到：

1. 指定对象名
2. 指定**存储池**

Ceph 客户端检出最新集群运行图，用 CRUSH 算法计算出如何把对象映射到**归置组**，然后动态地计算如何把归置组分配到 OSD。要定位对象，只需要对象名和存储池名字即可，例如：

```
ceph osd map {poolname} {object-name}
```

练习：定位某个对象

作为练习，我们先创建一个对象，用 `rados put` 命令加上对象名、一个有数据的测试文件路径、并指定存储池。例如：

```
echo {Test-data} > testfile.txt

rados put {object-name} {file-path} --pool=data
```

如：

```
rados put test-object-1 testfile.txt --pool=data
```

为确认 Ceph 存储集群存储了此对象，可执行：

```
rados -p data ls
```

现在，定位对象：

```
ceph osd map {pool-name} {object-name}
```

如：

```
ceph osd map data test-object-1
```

Ceph 应该会输出对象的位置，例如：

```
osdmap e537 pool 'data' (0) object 'test-object-1' -> pg  
0.d1743484 (0.4) -> up [1,0] acting [1,0]
```

用 ``rados rm`` 命令可删除此测试对象，例如：

```
rados rm test-object-1 --pool=data
```

随着集群的运行，对象位置可能会动态改变。Ceph 有动态均衡机制，无需手动干预即可完成。

块设备快速入门

要实践本手册，你必须先完成[存储集群快速入门](#)，并确保 **Ceph 存储集群**处于 `active + clean` 状态，这样才能使用 **Ceph 块设备**。

Note

Ceph 块设备也叫 **RBD** 或 **RADOS** 块设备。



你可以在虚拟机上运行 `ceph-client` 节点，但是不能在与 Ceph 存储集群（除非它们也用 VM）相同的物理节点上执行下列步骤。详情见 [FAQ](#)。

安装 CEPH

1. 确认你使用了合适的内核版本，详情见[操作系统推荐](#)。

```
lsb_release -a

uname -r
```

2. 在管理节点上，通过 `ceph-deploy` 把 Ceph 安装到 `ceph-client` 节点。

```
ceph-deploy install ceph-client
```

3. 在管理节点上，用 `ceph-deploy` 把 Ceph 配置文件和 `ceph.client.admin.keyring` 拷贝到 `ceph-client`。

```
ceph-deploy admin ceph-client
```

`ceph-deploy` 工具会把密钥环复制到 `/etc/ceph` 目录，要确保此密钥环文件有读权限

（如 `sudo chmod +r/etc/ceph/ceph.client.admin.keyring`）。

配置块设备

1. 在 `ceph-client` 节点上创建一个块设备 image。

```
rbd create foo --size 4096 [-m {mon-IP}] [-k
/path/to/ceph.client.admin.keyring]
```

2. 在 `ceph-client` 节点上，把 image 映射为块设备。

```
sudo rbd map foo --name client.admin [-m {mon-IP}]
[-k /path/to/ceph.client.admin.keyring]
```

3. 在 `ceph-client` 节点上，创建文件系统后就可以使用块设备了。

```
sudo mkfs.ext4 -m0 /dev/rbd/rbd/foo
```

此命令可能耗时较长。

4. 在 `ceph-client` 节点上挂载此文件系统。

```
sudo mkdir /mnt/ceph-block-device

sudo mount /dev/rbd/rbd/foo /mnt/ceph-block-device

cd /mnt/ceph-block-device
```

详情见[块设备](#)。

CEPH 文件系统快速入门

开始实践 [Ceph 文件系统](#) 入门手册前，必须先完成[存储集群快速入门](#)。在管理节点上完成此入门。

准备工作

1. 确认你使用了合适的内核版本，详情见[操作系统推荐](#)。

```
lsb_release -a
```



```
uname -r
```

2. 在管理节点上，通过 `ceph-deploy` 把 Ceph 安装到 `ceph-client` 节点上。

```
ceph-deploy install ceph-client
```

3. 确保 **Ceph 存储集群** 在运行，且处于 `active + clean` 状态。同时，确保至少有一个 **Ceph 元数据服务器** 在运行。

```
ceph -s [-m {monitor-ip-address}] [-k  
{path/to/ceph.client.admin.keyring}]
```

创建文件系统

虽然已创建了元数据服务器（[存储集群快速入门](#)），但如果你没有创建存储池和文件系统，它是不会变为活动状态的。参见 [创建 Ceph 文件系统](#)。

```
ceph osd pool create cephfs_data <pg_num>  
  
ceph osd pool create cephfs_metadata <pg_num>  
  
ceph fs new <fs_name> cephfs_metadata cephfs_data
```

创建密钥文件

Ceph 存储集群默认启用认证，你应该有个包含密钥的配置文件（但不是密钥环本身）。用下述方法获取某一用户的密钥：

1. 在密钥环文件中找到与某用户对应的密钥，例如：

```
cat ceph.client.admin.keyring
```

2. 找到用于挂载 Ceph 文件系统的用户，复制其密钥。大概看起来如下所示：

```
[client.admin]
```

```
key = AQCj2YpRiAe6CxAA7/ETt7Hcl9IxyYciVs47w==
```

3. 打开文本编辑器。
4. 把密钥粘贴进去，大概像这样：

```
AQCj2YpRiAe6CxAA7/ETt7Hcl9IxyYciVs47w==
```

5. 保存文件，并把其用户名 `name` 作为一个属性（如 `admin.secret`）。
6. 确保此文件对用户有合适的权限，但对其他用户不可见。

内核驱动

把 Ceph FS 挂载为内核驱动。

```
sudo mkdir /mnt/mycephfs

sudo mount -t ceph {ip-address-of-monitor}:6789:/
/mnt/mycephfs
```

Ceph 存储集群默认需要认证，所以挂载时需要指定用户名 `name` 和[创建密钥文件](#)一节中创建的密钥文件 `secretfile`，例如：

```
sudo mount -t ceph 192.168.0.1:6789:/ /mnt/mycephfs -o
name=admin,secretfile=admin.secret
```

Note

从管理节点而非服务器节点挂载 Ceph FS 文件系统，详情见 [FAQ](#)。

用户空间文件系统（FUSE）

把 Ceph FS 挂载为用户空间文件系统（FUSE）。

```
sudo mkdir ~/mycephfs

sudo ceph-fuse -m {ip-address-of-monitor}:6789 ~/mycephfs
```

Ceph 存储集群默认要求认证，需指定相应的密钥环文件，除非它在默认位置（即 `/etc/ceph`）：

```
sudo ceph-fuse -k ./ceph.client.admin.keyring -m
192.168.0.1:6789 ~/mycephfs
```

附加信息

附加信息见 [Ceph FS](#)。Ceph FS 还不像 Ceph 块设备和 Ceph 对象存储那么稳定，如果遇到问题请参考[故障排除](#)。

CEPH 对象存储快速入门

从 firefly (v0.80) 起，Ceph 存储集群显著地简化了 Ceph 对象网关的安装和配置。网关守护进程内嵌了 Civetweb，无需额外安装 web 服务器或配置 FastCGI。此外，可以直接使用 `ceph-deploy` 来安装网关软件包、生成密钥、配置数据目录以及创建一个网关实例。

Tip

Civetweb 默认使用 7480 端口。要么直接打开 7480 端口，要么在你的 Ceph 配置文件中设置首选端口（例如 80 端口）。

要使用 Ceph 对象网关，请执行下列步骤：

安装 CEPH 对象网关

1. 在 `client-node` 上执行预安装步骤。如果你打算使用 Civetweb 的默认端口 7480，必须通过 `firewall-cmd` 或 `iptables` 来打开它。详情见[预检](#)。
2. 从管理节点的工作目录，在 `client-node` 上安装 Ceph 对象网关软件包。例如：

```
ceph-deploy install --rgw <client-node> [<client-
node> ...]
```

新建 CEPH 对象网关实例

从管理节点的工作目录，在 `client-node` 上新建一个 Ceph 对象网关实例。例如：

```
ceph-deploy rgw create
```

一旦网关开始运行，你就可以通过 7480 端口来访问它（比如 `http://client-node:7480`）。

配置 CEPH 对象网关实例

1. 通过修改 Ceph 配置文件可以更改默认端口（比如改成 80）。增加名为 `[client.rgw.<client-node>]` 的小节，把 `<client-node>` 替换成你自己 Ceph 客户端节点的短名称（即 `hostname -s` 的输出）。例如，你的节点名就是 `client-node`，在 `[global]` 节后增加一个类似于下面的小节：

```
[client.rgw.client-node]

rgw_frontends = "civetweb port=80"
```

Note

确保在 `rgw_frontends` 键值对的 `port=<port-number>` 中没有空格。

Important

如果你打算使用 80 端口，确保 Apache 服务器没有在使用该端口，否则会 and Civetweb 冲突。出现这种情况时我们建议移除 Apache 服务。

2. 为了使新端口的设置生效，需要重启 Ceph 对象网关。在 RHEL 7 和 Fedora 上，执行：

```
sudo systemctl restart ceph-radosgw.service
```

在 RHEL 6 和 Ubuntu 上，执行：

```
sudo service radosgw restart id=rgw.<short-hostname>
```

3. 最后，检查节点的防火墙，确保你所选用的端口（例如 80 端口）处于开放状态。如果没有，把该端口加入放行规则并重载防火墙的配置。例如：

```
sudo firewall-cmd --list-all sudo firewall-cmd --  
zone=public --add-port 80/tcp --permanent  
  
sudo firewall-cmd --reload
```

关于使用 `firewall-cmd` 或 `iptables` 配置防火墙的详细信息，请参阅[预检](#)。

你应该可以生成一个未授权的请求，并收到应答。例如，一个如下不带参数的请求：

```
http://<client-node>:80
```

应该收到这样的应答：

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<ListAllMyBucketsResult  
  xmlns="http://s3.amazonaws.com/doc/2006-03-01/">  
  
  <Owner>  
  
    <ID>anonymous</ID>  
  
    <DisplayName></DisplayName>  
  
  </Owner>  
  
  <Buckets>  
  
  </Buckets>
```

```
</ListAllMyBucketsResult>
```

更多管理和 API 细节请参阅 [Ceph 对象网关的配置](#) 指南。

安装（手动-略）

获取软件

获取 Ceph 软件的方法有多种，最简单、通用的[获取软件包](#)方法是添加软件源之后通过包管理工具（像 APT、YUM）操作；也可以直接从 Ceph 仓库下载预编译软件包；最后，你可以下载源码包或克隆 Ceph 源码库、并自行编译。

- [获取二进制包](#)
- [获取源码包](#)
- [克隆源码](#)
- [构建 Ceph](#)

获取二进制包

要安装 Ceph 及其依赖软件，你需要参考本手册从 Ceph 软件库下载，然后继续看[安装 Ceph 对象存储](#)。

获取软件包

有两种方法获取软件包：

- **增加源：** 增加源是获取二进制包的最简方法，因为多数情况下包管理工具都能自动下载、并解决依赖关系。然而，这种方法要求各 [Ceph 节点](#)都能连接互联网。
- **手动下载：** 如果你的环境不允许 [Ceph 节点](#)访问互联网，手动下载软件包安装 Ceph 也不复杂。

准备工作

所有 Ceph 部署都需要 Ceph 软件包（除非是开发），你应该安装相应的密钥和推荐的软件包。

- **密钥：（推荐）** 不管你是用仓库还是手动下载，你都需要用密钥校验软件包。如果你没有密钥，就会收到安全警告。有两个密钥：一个用于发布（常用）、一个用于开发（仅适用于程序员和 QA），请按需选择，详情见[安装密钥](#)。
- **Ceph：（必要）** 所有 Ceph 部署都需要 Ceph 发布的软件包，除非你部署开发版软件包（仅有开发版、QA、和尖端部署）。详情见[添加 Ceph 库](#)。
- **Ceph Development：（可选）** 如果你在做 Ceph 开发、为 Ceph 做构建测试、或者急需开发版中的尖端功能，可以安装开发版软件包，详情见[添加 Ceph 开发库](#)。
- **Apache/FastCGI：（可选）** 如果你想部署 **Ceph 对象存储** 服务，那么必须安装 Apache 和 FastCGI。Ceph 库提供的 Apache 和 FastCGI 二进制包和来自 Apache 的是一样的，但它打开了 100-continue 支持。如果你想启用 **Ceph 对象网关**、且支持 100-continue，那必须从 Ceph 库下载 Apache/FastCGI 软件包。详情见[添加 Apache/CGI 源](#)。

如果你想手动下载二进制包，请参考[下载软件包](#)。

安装密钥

把密钥加入你系统的可信密钥列表内，以消除安全告警。对主要发行版

（如 `dumpling`、`emperor`、`firefly`）和开发版（如 `release-name-rc1`、`release-name-rc2`）应该用 `release.asc` 密钥；开发中的测试版应使用 `autobuild.asc` 密钥（开发者和 QA）。

APT

执行下列命令安装 `release.asc` 内的密钥：

```
wget -q -O- 'https://download.ceph.com/keys/release.asc'
| sudo apt-key add -
```

执行下列命令安装 `autobuild.asc` 密钥（仅适用于 QA 和开发者）：

```
wget -q -O-
'https://download.ceph.com/keys/autobuild.asc' | sudo
apt-key add -
```

RPM

执行下列命令安装 `release.asc` 密钥：

```
sudo rpm --import  
'https://download.ceph.com/keys/release.asc'
```

执行下列命令安装 `autobuild.asc` 密钥（仅对 QA 和开发者）：

```
sudo rpm --import  
'https://download.ceph.com/keys/autobuild.asc'
```

镜像

为提升用户体验，世界各地有多个 Ceph 镜像。

这些镜像位于如下地点：

- **EU:** <http://eu.ceph.com/>
- **AU:** <http://au.ceph.com/>

你可以把所有的 `ceph.com` URL 替换成任意镜像，例如：

<http://download.ceph.com/debian-hammer>

可以改成：

<http://eu.ceph.com/debian-hammer>

添加 CEPH 库

发布库用 `release.asc` 公钥校验软件包。要通过 APT 或 YUM 安装 Ceph 二进制包，必须先配置库。

适合 Debian/Ubuntu 的包位于：

```
http://download.ceph.com/debian-{release-name}
```


适合 CentOS/RHEL 和其他发行版（通过 YUM 安装）的包位于：

```
http://download.ceph.com/rpm-{release-name}
```

Ceph 的主要发布包括：

- **Hammer:** 是最新的，也是 Ceph 的第八个重要发布。这些包适合于生产环境，重要的缺陷修正会移植回来、并在必要时发布修正版。
- **Giant:** 是 Ceph 的第六个重要发布。这些包适合于生产环境，重要的缺陷修正会移植回来、并在必要时发布修正版。
- **Firefly:** 是 Ceph 的第六个重要发布。这些包适合于生产环境，重要的缺陷修正会移植回来、并在必要时发布修正版。
- **Emperor:** 是 Ceph 的第五个重要发布。这些包比较老了，而且已不再维护了，所以我们建议升级到 Firefly。
- **Argonaut, Bobtail, Cuttlefish, Dumpling:** 这些是 Ceph 的前 4 个主要发布，它们都很老了，且不再维护，所以请升级到近期版本。

Tip

对国际用户来说，在全球有很多镜像可用。请参考 [镜像](#)。

DEBIAN 二进制包

把 Ceph 库加入系统级 APT 源列表。在较新版本的 Debian/Ubuntu 上，用命令 `lsb_release -sc` 可获取短代码名，然后用它替换下列命令里的 {codename}。

```
sudo apt-add-repository 'deb
http://download.ceph.com/debian-firefly/ {codename} main'
```

对于早期 Linux 发行版，你可以执行下列命令：

```
echo deb http://download.ceph.com/debian-firefly/
$(lsb_release -sc) main | sudo tee
/etc/apt/sources.list.d/ceph.list
```

对于早期 Ceph 发布，可用 Ceph 发布名替换 {release-name}。用命令 `lsb_release -sc` 可获取短代码名，然后用它替换下列命令里的 {codename}。

```
sudo apt-add-repository 'deb
http://download.ceph.com/debian-{release-name}/ {codename}
main'
```

对较老的 Linux 发行版，用发布名替换 {release-name}。

```
echo deb http://download.ceph.com/debian-{release-name}/
$(lsb_release -sc) main | sudo tee
/etc/apt/sources.list.d/ceph.list
```

要在 ARM 处理器上运行 Ceph 的话，需要 Google 的内存剖析工具 (`google-perftools`)，Ceph 库里有：<http://download.ceph.com/packages/google-perftools/debian>。

```
echo deb http://download.ceph.com/packages/google-
perftools/debian $(lsb_release -sc) main | sudo tee
/etc/apt/sources.list.d/google-perftools.list
```

对于开发版，把我们的软件库加入 APT 源。这里 [Debian 测试版软件库](#) 是已支持的 Debian/Ubuntu 列表。

```
echo deb http://download.ceph.com/debian-testing/
$(lsb_release -sc) main | sudo tee
/etc/apt/sources.list.d/ceph.list
```

Tip

对国际用户来说，在全球有很多镜像可用。请参考 [镜像](#)。

RPM 二进制包

对于主要发布，你可以在 `/etc/yum.repos.d/` 目录下新增一个 Ceph 库：创建 `ceph.repo`。在下例中，需要用 Ceph 主要发布名（如 `dumpling`、`emperor`）替换 `{ceph-release}`、用 Linux 发行版名（`el6`、`rhel6` 等）替换 `{distro}`。你可以到 <http://download.ceph.com/rpm-{ceph-release}/> 看看 Ceph 支持哪些发行版。有些 Ceph 包（如 EPEL）必须优先于标准包，所以你必须确保设置了 `priority=2`。

```
[ceph]

name=Ceph packages for $basearch

baseurl=http://download.ceph.com/rpm-{ceph-release}/{distro}/$basearch

enabled=1

priority=2

gpgcheck=1

type=rpm-md

gpgkey=https://download.ceph.com/keys/release.asc


[ceph-noarch]

name=Ceph noarch packages

baseurl=http://download.ceph.com/rpm-{ceph-release}/{distro}/noarch

enabled=1

priority=2

gpgcheck=1
```

```
type=rpm-md

gpgkey=https://download.ceph.com/keys/release.asc


[ceph-source]

name=Ceph source packages

baseurl=http://download.ceph.com/rpm-{ceph-
release}/{distro}/SRPMS

enabled=0

priority=2

gpgcheck=1

type=rpm-md

gpgkey=https://download.ceph.com/keys/release.asc
```

如果想用开发版，你也可以用相应配置：

```
[ceph]

name=Ceph packages for $basearch/$releasever

baseurl=http://download.ceph.com/rpm-
testing/{distro}/$basearch

enabled=1

priority=2

gpgcheck=1
```

type=rpm-md

gpgkey=https://download.ceph.com/keys/release.asc

[ceph-noarch]

name=Ceph noarch packages

baseurl=http://download.ceph.com/rpm-testing/{distro}/noarch

enabled=1

priority=2

gpgcheck=1

type=rpm-md

gpgkey=https://download.ceph.com/keys/release.asc

[ceph-source]

name=Ceph source packages

baseurl=http://download.ceph.com/rpm-testing/{distro}/SRPMS

enabled=0

priority=2

gpgcheck=1

type=rpm-md

```
gpgkey=https://download.ceph.com/keys/release.asc
```

对于某些包，你可以通过名字直接下载。按照我们的开发进度，每 3-4 周会发布一次。这些包的变动比主要发布频繁，开发版会迅速地集成新功能，然而这些新功能需要几周时间的质检才会发布。

软件库包会把软件库的具体配置安装到本机，以便 `yum` 或 `up2date` 使用。

把 `{distro}` 替换成你的 Linux 发行版名字，把 `{release}` 换成 Ceph 的某个发布名。

```
su -c 'rpm -Uvh  
http://download.ceph.com/rpms/{distro}/x86_64/ceph-  
{release}.el6.noarch.rpm'
```

你可以从这个地址直接下载 RPM：

```
http://download.ceph.com/rpm-testing
```

Tip

对国际用户来说，在全球有很多镜像可用。请参考 [镜像](#)。

添加 CEPH 开发库

开发库用 `autobuild.asc` 密钥校验软件包。如果你在参与 Ceph 开发，想要部署并测试某个分支，确保先删除（或禁用）主要版本库的配置文件。

DEBIAN 二进制包

我们自动为 Debian 和 Ubuntu 构建 Ceph 当前分支的二进制包，这些包只适合开发者和质检人员。

把此仓库加入 APT 源，用你要测试的分支名（如 `chef-3`、`wip-hack`、`master`）替换 `{BRANCH}`。我们所构建的完整分支列表在 [the gitbuilder page](#)。

```
echo deb http://gitbuilder.ceph.com/ceph-deb-
$(lsb_release -sc)-x86_64-basic/ref/{BRANCH}
$(lsb_release -sc) main | sudo tee
/etc/apt/sources.list.d/ceph.list
```

RPM 二进制包

对于当前开发分支，你可以在 `/etc/yum.repos.d/` 目录下创建 `ceph.repo` 文件，内容如下，用你的 Linux 发行版名字（`centos6`、`rhel6` 等）替换 `{distro}`、用你想安装的分支名替换 `{branch}`。

```
[ceph-source]

name=Ceph source packages

baseurl=http://gitbuilder.ceph.com/ceph-rpm-{distro}-
x86_64-basic/ref/{branch}/SRPMS

enabled=0

gpgcheck=1

type=rpm-md

gpgkey=https://download.ceph.com/keys/autobuild.asc
```

你可以到 <http://gitbuilder.ceph.com> 查看 Ceph 支持哪些发行版。

添加 APACHE/CGI 源

Ceph 对象存储与普通的 Apache 和 FastCGI 库对接，只是 Ceph 要求 Apache 和 FastCGI 支持 100-continue 功能。请配置相应的软件库，以使用对应的 Apache 和 FastCGI 包。

DEBIAN 二进制包

如果想要 100-continue 功能，请把我们的源加入 APT 源列表。

```
echo deb http://gitbuilder.ceph.com/apache2-deb-
$(lsb_release -sc)-x86_64-basic/ref/master $(lsb_release
-sc) main | sudo tee /etc/apt/sources.list.d/ceph-
apache.list

echo deb http://gitbuilder.ceph.com/libapache-mod-
fastcgi-deb-$(lsb_release -sc)-x86_64-basic/ref/master
$(lsb_release -sc) main | sudo tee
/etc/apt/sources.list.d/ceph-fastcgi.list
```

RPM 二进制包

你可以在 `/etc/yum.repos.d/` 目录下创建 `ceph-apache.repo` 文件，内容如下，用你的 Linux 发行版名字（如 `el6`、`rhel6`）替换 `{distro}`，<http://gitbuilder.ceph.com> 列出了支持的发行版。

```
[apache2-ceph-noarch]

name=Apache noarch packages for Ceph

baseurl=http://gitbuilder.ceph.com/apache2-rpm-{distro}-
x86_64-basic/ref/master

enabled=1

priority=2

gpgcheck=1

type=rpm-md

gpgkey=https://download.ceph.com/keys/autobuild.asc

[apache2-ceph-source]

name=Apache source packages for Ceph
```



```
baseurl=http://gitbuilder.ceph.com/apache2-rpm-{distro}-x86_64-basic/ref/master

enabled=0

priority=2

gpgcheck=1

type=rpm-md

gpgkey=https://download.ceph.com/keys/autobuild.asc
```

仿照前述步骤创建 `ceph-fastcgi.repo` 文件。

```
[fastcgi-ceph-basearch]

name=FastCGI basearch packages for Ceph

baseurl=http://gitbuilder.ceph.com/mod_fastcgi-rpm-{distro}-x86_64-basic/ref/master

enabled=1

priority=2

gpgcheck=1

type=rpm-md

gpgkey=https://download.ceph.com/keys/autobuild.asc


[fastcgi-ceph-noarch]

name=FastCGI noarch packages for Ceph
```

```
baseurl=http://gitbuilder.ceph.com/mod_fastcgi-rpm-
{distro}-x86_64-basic/ref/master

enabled=1

priority=2

gpgcheck=1

type=rpm-md

gpgkey=https://download.ceph.com/keys/autobuild.asc


[fastcgi-ceph-source]

name=FastCGI source packages for Ceph

baseurl=http://gitbuilder.ceph.com/mod_fastcgi-rpm-
{distro}-x86_64-basic/ref/master

enabled=0

priority=2

gpgcheck=1

type=rpm-md

gpgkey=https://download.ceph.com/keys/autobuild.asc
```

下载软件包

如果你位于防火墙之内，不能访问互联网，那你必须先下载齐所需软件包（镜像所有依赖）。

DEBIAN 二进制包

Ceph 依赖这些第三方库。

- libaio1
- libsnappy1
- libcurl3
- curl
- libgoogle-perftools4
- google-perftools
- libleveldb1

这个软件库包会装好所需的 apt 软件库的配置文件。需用最新 Ceph 发布替换掉 {release}、用最新 Ceph 版本号替换 {version}、用自己的 Linux 发行版代号替换 {distro}、用自己的 CPU 架构替换 {arch}。

```
wget -q http://download.ceph.com/debian-
{release}/pool/main/c/ceph/ceph_{version}{distro}_{arch}.
deb
```

RPM 二进制包

Ceph 依赖一些第三方库。执行下列命令添加 EPEL 库：

```
su -c 'rpm -Uvh http://download.ceph.com/rpm-
firefly/{distro}/noarch/ceph-
{version}.{distro}.noarch.rpm'
```

Ceph 依赖下列包：

- snappy
- leveldb
- gdisk
- python-argparse
- gperftools-lib

当前，我们为这些平台 RHEL/CentOS6 (el6)、Fedora 18 和 19 (f18 和 f19)、OpenSUSE 12.2 (opensuse12.2) 和 SLES (sles11) 分别构建二进制包，仓库包会在本地系统上装好 Ceph 库配置文件，这样 yum 或 up2date 就可以使用这些配置文件自动安装了。用自己的发行版名字替换 {distro}。

```
su -c 'rpm -Uvh http://download.ceph.com/rpm-  
firefly/{distro}/noarch/ceph-  
{version}.{distro}.noarch.rpm'
```

例如，对于 CentOS 6 (el6)：

```
su -c 'rpm -Uvh http://download.ceph.com/rpm-  
firefly/el6/noarch/ceph-release-1-0.el6.noarch.rpm'
```

你可以从这里直接下载 RPM 包：

```
http://download.ceph.com/rpm-firefly
```

对较老的 Ceph 发布，用 Ceph 发布名替换 {release-name}，你可以执行 `lsb_release -sc` 命令获取发行版代号。

```
su -c 'rpm -Uvh http://download.ceph.com/rpm-{release-  
name}/{distro}/noarch/ceph-{version}.{distro}.noarch.rpm'
```

下载 CEPH 源码包

随着 Ceph 开发的推进，Ceph 团队会不断发布源码，你可以在以下这些地方下载发布的源码：

[Ceph 发布的源码包](#)

[Ceph 发布的源码包（欧洲镜像）](#)

克隆 CEPH 源码库

你可以去[位于 github 的 Ceph 源码库](#)克隆某个 Ceph 分支，先选择一个分支（默认是 master），然后点击 **Download ZIP** 按钮。

要克隆整个 git 源码库，你得先安装、配置 git。

安装 GIT

在 Debian/Ubuntu 上执行下列命令安装 git：

```
sudo apt-get install git
```

在 CentOS/RHEL 上执行下列命令安装 git：

```
sudo yum install git
```

相应地，你必须有 github 帐户。如果你还没有，去 github.com 注册一个，然后按照[设置 Git](#) 指引配置 git。

添加 SSH 密钥（可选）

如果你计划向 Ceph 贡献代码、或者想通过 SSH 克隆

（ `git@github.com:ceph/ceph.git` ），你必须生成一个 SSH 密钥对。

Tip

如果你只是想克隆，不需要 SSH 密钥也可用 `git clone --recursive https://github.com/ceph/ceph.git` 克隆。

执行如下命令生成 SSH 密钥对用于 github：

```
ssh-keygen
```

把此密钥对的公钥加入 github 帐户（本例假设用了默认路径）：

```
cat .ssh/id_rsa.pub
```

复制公钥。

进入 `github` 帐户，点击“Account Settings”（即 `tools` 图标），然后点击导航条左边的“SSH Keys”。

点击“SSH Keys”列表里的“Add SSH key”，给密钥起个名字，把复制的公钥粘帖进去，最后点击“Add key”按钮。

克隆源码

执行下列命令克隆源码库：

```
git clone --recursive https://github.com/ceph/ceph.git
```

`git clone` 完成后，你应该已经得到了一份完整的 Ceph 源码库。

Tip

确保你获取到的源码库之内的各子模块都是最新的，运行 `git status` 确认。

```
cd ceph  
  
git status
```

如果你的子模块过时了，运行：

```
git submodule update --force --init --recursive
```

选择分支

克隆完源码和子模块后，你的源码库将默认位于 `master` 分支上，这是个不稳定开发分支，你也可以切换到其他分支上。

- `master`: 不稳定开发分支；
- `stable`: 缺陷修正分支；
- `next`: 发布候选分支。

```
git checkout master
```

构建 CEPH

你可以下载 Ceph 源码并自行构建。首先，你得准备开发环境、编译 Ceph、然后安装到用户区或者构建二进制包并安装。

构建依赖

Tip

对照本段检查下你的 Linux/Unix 发行版是否满足这些依赖。

构建 Ceph 源码前，你得先安装几个库和工具：

```
./install-deps.sh
```

Note

在某些支持 Google 内存剖析工具的发行版上，名字未必如此（如 `libgoogle-perftools4`）。

构建 CEPH

Ceph 用 `automake` 和 `configure` 脚本简化构建过程。先进入刚克隆的 Ceph 源码库，执行下列命令开始构建：

```
cd ceph

./autogen.sh

./configure

make
```

超线程

你可以根据自己的硬件配置情况用 `make -j` 并行编译，比如在双核处理器上用 `make -j 4` 可能会编译得快些。

参考[安装自构建软件](#)把构建好的软件安装到用户区。

构建 Ceph 安装包

要构建安装包，你必须克隆 [Ceph](#) 源码库。用 `dpkg-buildpackage` 基于最新代码为 Debian/Ubuntu 创建安装包；用 `rpmbuild` 为 RPM 包管理器创建安装包。

Tip

在多核 CPU 上构建时，用参数 `-j`、再加上核心数的 2 倍数，例如在双核处理器上用 `-j 4` 来加速构建。

高级打包工具（APT）

要为 Debian/Ubuntu 创建 `.deb` 安装包，先要克隆 Ceph 源码库、安装好必要的[构建依赖](#)和 `debhelper`。

```
sudo apt-get install debhelper
```

装好 `debhelper` 之后就可以开始构建安装包了：

```
sudo dpkg-buildpackage
```

在多核处理器上可以用 `-j` 加快构建速度。

RPM 包管理器

要创建 `.rpm` 包，先得克隆 [Ceph](#) 源码库、安装必要的[构建依赖](#)、安装好 `rpm-build` 和 `rpmdevtools`：


```
yum install rpm-build rpmdevtools
```

安装完这些工具后，设置 RPM 编译环境：

```
rpmdev-setuptree
```

下载源码包，编译 RPM 时需要：

```
wget -P ~/rpmbuild/SOURCES/  
http://ceph.com/download/ceph-<version>.tar.bz2
```

或者从欧洲镜像下载：

```
wget -P ~/rpmbuild/SOURCES/  
http://eu.ceph.com/download/ceph-<version>.tar.bz2
```

提取规范文件：

```
tar --strip-components=1 -C ~/rpmbuild/SPECS/ --no-  
anchored -xvjf ~/rpmbuild/SOURCES/ceph-<version>.tar.bz2  
"ceph.spec"
```

开始构建 RPM 包：

```
rpmbuild -ba ~/rpmbuild/SPECS/ceph.spec
```

在多核处理器上可以用 `-j` 加快构建速度。

安装软件

获取到（或者软件库里有）Ceph 软件包之后，安装很简单。要在集群内的各节点安装，你可以用 `ceph-deploy`、或者包管理工具。如果你想安装 Ceph 对象网关、或 QEMU，那么对于使用 `yum` 的发行版，你应该设置软件库优先级。

- [安装 ceph-deploy](#)
- [安装 Ceph 存储集群](#)
- [安装 Ceph 对象网关](#)
- [作为虚拟化的块设备](#)

安装 CEPH-DEPLOY

`ceph-deploy` 工具可用来装配起或拆除 Ceph 集群，方便了开发、测试和概念验证项目。

APT

要用 `apt` 安装 `ceph-deploy`，用此命令：

```
sudo apt-get update && sudo apt-get install ceph-deploy
```

RPM

要用 `yum` 安装 `ceph-deploy`，用此命令：

```
sudo yum install ceph-deploy
```

安装 CEPH 存储集群

本指南说明了如何手动安装 Ceph 软件包，此方法只适用于那些没采用部署工具（如 `ceph-deploy`、`chef`、`juju` 等）的用户。

Tip

你也可以用 `ceph-deploy` 安装 Ceph 软件包，也许它更方便，因为只需一个命令就可以把 `ceph` 安装到多台主机。

用APT 安装

只要把正式版或开发版软件包源加入了 APT，你就可以更新 APT 数据库并安装 Ceph 了：

```
sudo apt-get update && sudo apt-get install ceph ceph-mds
```

用RPM 安装

要用 RPM 安装 Ceph，可按如下步骤进行：

1. 安装 `yum-plugin-priorities`。

```
sudo yum install yum-plugin-priorities
```

2. 确认 `/etc/yum/pluginconf.d/priorities.conf` 文件存在。
3. 确认 `priorities.conf` 里面打开了插件支持。

```
4. [main]
```

```
5. enabled = 1
```

6. 确认你的 YUM `ceph.repo` 库文件条目包含 `priority=2`，详情见[获取软件包](#)：

```
7. [ceph]
```

```
8. name=Ceph packages for $basearch
```

```
9. baseurl=http://download.ceph.com/rpm-{ceph-  
  release}/{distro}/$basearch
```

```
10.     enabled=1
```

```
11.     priority=2
```

```
12.     gpgcheck=1
```

```
13.     type=rpm-md

14.     gpgkey=https://download.ceph.com/keys/release.
      asc

15.

16.     [ceph-noarch]

17.     name=Ceph noarch packages

18.     baseurl=http://download.ceph.com/rpm-{ceph-
      release}/{distro}/noarch

19.     enabled=1

20.     priority=2

21.     gpgcheck=1

22.     type=rpm-md

23.     gpgkey=https://download.ceph.com/keys/release.
      asc

24.

25.     [ceph-source]

26.     name=Ceph source packages

27.     baseurl=http://download.ceph.com/rpm-{ceph-
      release}/{distro}/SRPMS

28.     enabled=0

29.     priority=2

30.     gpgcheck=1
```

```
31.         type=rpm-md
```

```
gpgkey=https://download.ceph.com/keys/release.asc
```

32. 安装依赖的软件包：

```
sudo yum install snappy leveldb gdisk python-  
argparse gperftools-libs
```

成功添加正式版或开发版软件包的库文件之后，或把 `ceph.repo` 文件放入 `/etc/yum.repos.d` 之后，你就可以安装 Ceph 软件包了。

```
sudo yum install ceph
```

从源码安装

如果你是从源码构建的 Ceph，可以用下面的命令安装到用户区：

```
sudo make install
```

如果你是本地安装的，`make` 会把可执行文件放到 `usr/local/bin` 里面。你可以把 Ceph 配置文件放到 `usr/local/bin` 目录下，这样就能从这个目录运行 Ceph 了。

安装 CEPH 对象网关

自从 *firefly* (v0.80) 版本开始，Ceph 对象网关运行在 Civetweb 上（已经集成进守护进程 `ceph-radosgw`），而不再是 Apache 和 FastCGI 之上。使用 Civetweb 简化了 Ceph 对象网关的安装和配置。

Note

要提供 Ceph 网关服务，你得有一个正常运行的 Ceph 集群，且网关主机能访问存储集群的公共网络。

Note

在 0.80 版本中，Ceph 对象网关不再支持 SSL。你可以设置一个支持 SSL 的反向代理服务器来将 HTTPS 请求转为 HTTP 请求发给 CivetWeb。

执行安装前的准备工作

首先进行 [环境检查](#)并在你的 Ceph 对象网关节点上执行安装前的准备工作。特别的，你需要禁用部署 Ceph 集群所用用户的 `requiretty`，同时设置 SELinux 为 `Permissive` 以及 Ceph 部署用户使用 `sudo` 时无需密码。对于 Ceph 对象网关，在生产环境下你需要开启 Civetweb 所使用的端口。

Note

Civetweb 默认运行在 `7480` 端口上。

安装 Ceph 对象网关

在你的管理节点的工作目录下，给 Ceph 对象网关节点安装 Ceph 对象所需的软件包。例如：

```
ceph-deploy install --rgw <gateway-node1> [<gateway-node2> ...]
```

`ceph-common` 包是它的一个依赖性，所以 `ceph-deploy` 也将安装这个包。`ceph` 的命令行工具就会为管理员准备好。为了让你的 Ceph 对象网关节点成为管理节点，可以在管理节点的工作目录下执行以下命令：

```
ceph-deploy admin <node-name>
```

新建网关实例

在你的管理节点的工作目录下，使用命令在 Ceph 对象网关节点上新建一个 Ceph 对象网关实例。举例如下：

```
ceph-deploy rgw create <gateway-node1>
```

在网关服务成功运行后，你可以使用未经授权的请求来访问端口 7480，就像这样：

```
http://client-node:7480
```

如果网关实例工作正常，你接收到的返回信息大概如下所示：

```
<?xml version="1.0" encoding="UTF-8"?>

<ListAllMyBucketsResult
  xmlns="http://s3.amazonaws.com/doc/2006-03-01/">

  <Owner>

    <ID>anonymous</ID>

    <DisplayName></DisplayName>

  </Owner>

  <Buckets>

  </Buckets>

</ListAllMyBucketsResult>
```

在任何时候如果你遇到麻烦，而你也想重新来一次，执行下面的命令来清除配置：

```
ceph-deploy purge <gateway-node1> [<gateway-node2>]
```

```
ceph-deploy purgedata <gateway-node1> [<gateway-node2>]
```

如果你执行了 `purge`, 你必须重新安装 Ceph.

修改默认端口

Civetweb 默认运行在端口 7480 之上。如果想修改这个默认端口 (比如使用端口 80), 修改你的管理节点的工作目录下的 Ceph 配置文件。添加一节, 以 `[client.rgw.<gateway-node>]` 作为名字, 使用你的 Ceph 对象网关节点的短主机名替换其中的 `<gateway-node>` (如, `hostname -s`).

Note

在 0.94 版本中, Ceph 对象网关不再支持 SSL。你可以设置一个支持 SSL 的反向代理服务器来将 HTTPS 请求转为 HTTP 请求发给 CivetWeb。

比如, 如果你的主机名是 `gateway-node1`, 在 `[global]` 节后添加的节名如下:

```
[client.rgw.gateway-node1]

rgw_frontends = "civetweb port=80"
```

Note

请确保在 `rgw_frontends` 的键值对中 `port=<port-number>` 内部没有红格。节名 `[client.rgw.gateway-node1]` 表示接下来的 Ceph 配置文件配置了一个 Ceph 存储集群的客户端, 这个客户端的类型是 Ceph 对象网关存储 (比如, `rgw`), 同时实例名是 `gateway-node1`.

将该配置文件推送到你的 Ceph 对象网关节点(也包括其他 Ceph 节点):

```
ceph-deploy --overwrite-conf config push <gateway-node>
[<other-nodes>]
```


为了使新配置的端口生效，需要重启 Ceph 对象网关：

```
sudo systemctl restart ceph-radosgw.service
```

最后，需要确保你选择的端口在节点的防火墙配置中是开放的（比如，端口 80）。如果它不是开放的，将它设为开放并重启防火墙。如果你用的是 `firewalld`，执行下面的命令：

```
sudo firewall-cmd --list-all

sudo firewall-cmd --zone=public --add-port 80/tcp --
permanent

sudo firewall-cmd --reload
```

如果你使用 `iptables`，执行：

```
sudo iptables --list

sudo iptables -I INPUT 1 -i <iface> -p tcp -s <ip-
address>/<netmask> --dport 80 -j ACCEPT
```

替换其中的 `<iface>` and `<ip-address>/<netmask>` 为你的 Ceph 对象网关节点的实
际值。

一旦你完成了 `iptables` 的配置，请确保你已经将本次更改持久化，这样它在你的 Ceph 对
象网关节点重启也依然生效。持久化请执行：

```
sudo apt-get install iptables-persistent
```

会有一个终端界面弹出。选择 `yes` 来保存当前 IPv4 的 `iptables` 规则到文
件 `/etc/iptables/rules.v4` 中，当前的 IPv6 `iptables` 规则到文件
`/etc/iptables/rules.v6` 中。

在前面一步设置的 IPv4 iptables 规则将会被写入文件 `/etc/iptables/rules.v4`，这样就实现了持久化，重启也依然生效。

如果你在安装 `iptables-persistent` 后有新增了新的 IPv4 iptables 规则，你需要使用 `root` 执行下面的命令：

```
iptables-save > /etc/iptables/rules.v4
```

从 APACHE 迁移到 CIVETWEB

如果你在 0.80 或以上版本的 Ceph 存储上使用 Apache 和 FastCGI 来构建 Ceph 对象网关，实际上你已经运行了 Civetweb -它随着 `ceph-radosgw` 进程启动而启动，默认情况下运行在 7480 端口，因此它不会跟基于 Apache 和 FastCGI 安装的以及其他常用的 web 服务所使用的端口冲突。迁移到使用 Civetweb 首先需要删除您的 Apache。然后，您必须从 Ceph 配置文件中删除 Apache 和 FastCGI 的设置并重置 `rgw_frontends` 到 Civetweb。

回顾使用 `ceph-deploy` 来安装 Ceph 对象网关的描述，注意到它的配置文件只有一个名为 `rgw_frontends` 的设置(这是假设你选择改变默认端口)。`ceph-deploy` 会生成对象网关的数据目录和 keyring，并将 keyring 放在 `/var/lib/ceph/radosgw/{rgw-
instance}` 目录下。从这个目录上看这个守护进程是在默认位置，然而你也可以在 Ceph 配置文件中指定不同的位置。因为你已经有密钥和数据目录，因此如果你使用的是默认路径之外的其他目录，你就需要在 Ceph 配置文件中维护这些路径。

一个典型的基于 Apache 部署的 Ceph 对象网关配置文件类似下面的样例：

在 Red Hat Enterprise Linux 系统上：

```
[client.radosgw.gateway-node1]

host = {hostname}

keyring = /etc/ceph/ceph.client.radosgw.keyring

rgw socket path = ""

log file = /var/log/radosgw/client.radosgw.gateway-  
node1.log
```

```
rgw frontends = fastcgi socket\_port=9000
socket\_host=0.0.0.0

rgw print continue = false
```

在 Ubuntu 系统上:

```
[client.radosgw.gateway-node]

host = {hostname}

keyring = /etc/ceph/ceph.client.radosgw.keyring

rgw socket path =
/var/run/ceph/ceph.radosgw.gateway.fastcgi.sock

log file = /var/log/radosgw/client.radosgw.gateway-
node1.log
```

为了能够使用 Civetweb，需要做一点修改，简单地删除 Apache 特有的设置如 `rgw_socket_path` 和 `rgw_print_continue`。然后，改变 `rgw_frontends` 的设置来指向 Civetweb 而不是 Apache FastCGI 作为前端，并指定您想要使用的端口号。例如:

```
[client.radosgw.gateway-node1]

host = {hostname}

keyring = /etc/ceph/ceph.client.radosgw.keyring

log file = /var/log/radosgw/client.radosgw.gateway-
node1.log

rgw_frontends = civetweb port=80
```

最后，重启 Ceph 对象网关。在 Red Hat Enterprise Linux 请执行:

```
sudo systemctl restart ceph-radosgw.service
```

在 Ubuntu 上执行:

```
sudo service radosgw restart id=rgw.<short-hostname>
```

如果你使用的端口还没有在防护墙端放开，你还需要在你的防火墙开放该端口。

配置 *BUCKET SHARDING*

Ceph 对象网关在 `index_pool` 中存储 bucket 的索引数据，默认情况下是资源池 `.rgw.buckets.index`。有时用户喜欢把很多对象(几十万到上百万的对象)存放到同一个 bucket 中。如果你不使用网关的管理接口来为每个 bucket 的最大对象数设置配额，那么当一旦用户存放大量的对象到一个 bucket 中时，bucket 索引的性能会呈现明显的下降。

在 0.94 版本的 Ceph 中，您可以给 bucket 索引进行分片，这样在你允许 bucket 中有大量对象时，能够有助于防止出现性能瓶颈。设置项的

“`rgw_override_bucket_index_max_shards`”允许您设置一个 bucket 的最大分片数。它的默认值为 `0`，这意味着 bucket 索引分片功能在默认情况下情况下是关闭的。

开启 bucket 的索引分片功能，只需

给 `rgw_override_bucket_index_max_shards` 设置一个大于 `0` 的值。

简单的配置，只需要在 Ceph 配置文件中加

入 `rgw_override_bucket_index_max_shards`。将其添加在 `[global]` 部分来设置一个系统层面生效的值。你也可以在 Ceph 配置文件中将它设置为某一个实例生效。

一旦你在 Ceph 配置文件中修改了你的 bucket 分片设置，你需要重启网关。在 Red Hat Enterprise Linux 请执行:

```
sudo systemctl restart ceph-radosgw.service
```

在 Ubuntu 上请执行:

```
sudo service radosgw restart id=rgw.<short-hostname>
```

对于异地场景的配置而言，为了灾备每一个 zone 都有一个不同的 `index_pool` 设置。为了保持这个参数在一个 region 的所有 zone 中保持一致，你可以在 region 的网关配置中指定 `rgw_override_bucket_index_max_shards`。举例如下：

```
radosgw-admin region get > region.json
```

打开 `region.json` 的文件，为每一个 zone 编辑 `bucket_index_max_shards` 的设置。保存 `region.json` 文件并重置 region。举例如下：

```
radosgw-admin region set < region.json
```

一旦你更新了你的 region，你需要更新 region map。举例如下：

```
radosgw-admin regionmap update --name client.rgw.ceph-client
```

其中的 `client.rgw.ceph-client` 是网关用户的名字。

Note

通过 CRUSH 规则集将索引资源池 (如果可以为每一个 zone 设置) 映射到基于 SSD 的 OSD 上也能够提升 bucket 索引的性能。

[给 DNS 添加泛域名解析](#)

为了通过 S3 风格的子域名来使用 Ceph(如：bucket-name.domain-name.com)，你应该给你的 `ceph-radosgw` 守护进程所在服务器在 DNS 服务器上给它的 DNS 记录添加泛域名解析。

这个 DNS 地址也必须在 Ceph 配置文件中通过 `rgw dns name = {hostname}` 设置项来指定。

对于 `dnsmasq` 而言，通过在主机名前添加点号 (.) 来实现添加 DNS 地址设置:

```
address=/.{hostname-or-fqdn}/{host-ip-address}
```

举例如下:

```
address=/.gateway-node1/192.168.122.75
```

对于 `bind` 而言，给 DNS 记录添加一个泛域名解析。举例如下:

```
$TTL      604800

@         IN      SOA      gateway-node1. root.gateway-node1.
(
                                2           ; Serial
                                604800      ; Refresh
                                86400       ; Retry
                                2419200     ; Expire
                                604800 )    ; Negative Cache
TTL
;

@         IN      NS       gateway-node1.

@         IN      A        192.168.122.113

*         IN      CNAME    @
```

重启你的 DNS 服务，然后使用子域名 ping 你的服务器来确保你的 `ceph-radosgw` 进程能够出来子域名请求:

```
ping mybucket.{hostname}
```

举例如下:

```
ping mybucket.gateway-node1
```

开启 *DEBUGGING* (如果需要)

一旦你完成了安装过程,如果在你的配置下遇到了问题,你可以在 Ceph 配合文件的 `[global]` 小节下面添加调试选项,然后重启网关服务,进而帮助解决配置过程出现的问题。例如:

```
[global]

#append the following in the global section.

debug ms = 1

debug rgw = 20
```

使用网关

为了使用 REST 接口,首先需要为 S3 接口创建一个初始 Ceph 对象网关用户。然后,为 Swift 接口创建一个子用户。然后你需要验证创建的用户是否能够访问网关。

为 S3 访问创建 RADOSGW 用户

一个 ``radosgw`` 用户需要被新建并被分配权限。命令 `man radosgw-admin` 会提供该命令的额外信息。

为了新建用户,在 `gateway host` 上执行下面的命令:

```
sudo radosgw-admin user create --uid="testuser" --
display-name="First User"
```

命令的输出跟下面的类似:

```
{

    "user_id": "testuser",

    "display_name": "First User",

    "email": "",

    "suspended": 0,

    "max_buckets": 1000,

    "auid": 0,

    "subusers": [],

    "keys": [{

        "user": "testuser",

        "access_key": "I0PJDPICIYZ665MW88W9R",

        "secret_key":
"dxaxZ8U90SXydYzyS5ivamEP20hkLSUViiaR+ZDA"

    }],

    "swift_keys": [],

    "caps": [],

    "op_mask": "read, write, delete",

    "default_placement": "",

    "placement_tags": [],
```



```
    "bucket_quota": {  
  
        "enabled": false,  
  
        "max_size_kb": -1,  
  
        "max_objects": -1  
  
    },  
  
    "user_quota": {  
  
        "enabled": false,  
  
        "max_size_kb": -1,  
  
        "max_objects": -1  
  
    },  
  
    "temp_url_keys": []  
  
}
```

Note

其中 `keys->access_key` 和 `keys->secret_key` 的值在访问的时候需要用来做验证

Important

请检查输出的 key。有个时候 `radosgw-admin` 会在生成的 JSON 中的 `access_key` 和 `secret_key` 部分包含有转义字符 `\`，并且一些客户端不知道如何处理 JSON 中的这个字符。补救措施包括移除 JSON 中的 `\` 字符，将该字符串封装到引号中，重新生成这个 key 并确保不再包含 `\`，或者手动指定 `access_key` 和 `secret_key`。如果 `radosgw-admin` 生成的 JSON 中的同一个 key 中包含转义字符 `\` 同时包含有正斜杠 `/` 形如 `\\/`，请只移除 JSON 转义字符 `\`，不要删除正斜杠 `/`，因为在 key 中它是一个有效字符。

新建一个 SWIFT 用户

如果你想要使用这种方式访问集群，你需要新建一个 Swift 子用户。创建 Swift 用户包括两个步骤。第一步是创建用户。第二步是创建 secret key。

在`gateway host`上执行喜爱按的步骤：

新建 Swift 用户：

```
sudo radosgw-admin subuser create --uid=testuser --  
subuser=testuser:swift --access=full
```

输出类似下面这样：

```
{  
  
  "user_id": "testuser",  
  
  "display_name": "First User",  
  
  "email": "",  
  
  "suspended": 0,  
  
  "max_buckets": 1000,  
  
  "auid": 0,  
  
  "subusers": [{  
  
    "id": "testuser:swift",  
  
    "permissions": "full-control"  
  
  }],  
  
  "keys": [{
```

```
        "user": "testuser:swift",

        "access_key": "3Y1LNW4Q6X0Y53A52DET",

        "secret_key": ""

    }, {

        "user": "testuser",

        "access_key": "I0PJDPICIYZ665MW88W9R",

        "secret_key":
"dxaxZ8U90SXYdYzyS5ivamEP20hkLSUViiaR+ZDA"

    }],

    "swift_keys": [],

    "caps": [],

    "op_mask": "read, write, delete",

    "default_placement": "",

    "placement_tags": [],

    "bucket_quota": {

        "enabled": false,

        "max_size_kb": -1,

        "max_objects": -1

    },

    "user_quota": {
```

```
        "enabled": false,

        "max_size_kb": -1,

        "max_objects": -1

    },

    "temp_url_keys": []

}
```

新建 secret key:

```
sudo radosgw-admin key create --subuser=testuser:swift --
key-type=swift --gen-secret
```

输出类似下面这样:

```
{

    "user_id": "testuser",

    "display_name": "First User",

    "email": "",

    "suspended": 0,

    "max_buckets": 1000,

    "auid": 0,

    "subusers": [{

        "id": "testuser:swift",
```

```
        "permissions": "full-control"

    }],

    "keys": [{

        "user": "testuser:swift",

        "access_key": "3Y1LNW4Q6X0Y53A52DET",

        "secret_key": ""

    }, {

        "user": "testuser",

        "access_key": "I0PJDPICIYZ665MW88W9R",

        "secret_key":
"dxaxZ8U90SXydyzyS5ivamEP20hkLSUViiaR+ZDA"

    }],

    "swift_keys": [{

        "user": "testuser:swift",

        "secret_key":
"244+fz2gSqoHwR3lYtSbIyomyPHf3i7rgSJrF\//IA"

    }],

    "caps": [],

    "op_mask": "read, write, delete",

    "default_placement": "",

    "placement_tags": [],
```

```
    "bucket_quota": {  
  
        "enabled": false,  
  
        "max_size_kb": -1,  
  
        "max_objects": -1  
  
    },  
  
    "user_quota": {  
  
        "enabled": false,  
  
        "max_size_kb": -1,  
  
        "max_objects": -1  
  
    },  
  
    "temp_url_keys": []  
  
}
```

访问验证

测试 S3 访问

为了验证 S3 访问，你需要编写并运行一个 Python 测试脚本。S3 访问测试脚本将连接 `radosgw`，新建一个新的 bucket 并列出现有的 buckets。

`aws_access_key_id` 和 `aws_secret_access_key` 的值来自于命令 `radosgw_admin` 的返回值 `access_key` 和 `secret_key`。

执行下面的步骤：

1. 你需要安装 `python-boto` 包：

```
sudo yum install python-boto
```

2. 新建 Python 脚本文件:

```
vi s3test.py
```

3. 将下面的内容添加到文件中:

```
4. import boto

5. import boto.s3.connection

6.

7. access_key = 'I0PJDPCIYZ665MW88W9R'

8. secret_key =
    'dxaXZ8U90SXydYzyS5ivamEP20hkLSUViiaR+ZDA'

9. conn = boto.connect_s3(

10.         aws_access_key_id = access_key,

11.         aws_secret_access_key = secret_key,

12.         host = '{hostname}', port = {port},

13.         is_secure=False, calling_format =
    boto.s3.connection.OrdinaryCallingFormat(),

14.         )

15.

16.     bucket = conn.create_bucket('my-new-bucket')

17.     for bucket in conn.get_all_buckets():

18.         print "{name}".format(
```

```
19.                 name = bucket.name,  
  
20.                 created =  
    bucket.creation_date,
```

```
)
```

将 {hostname} 替换为你配置了网关服务的节点的主机名。比如 gateway host.
将 {port} 替换为 Civetweb 所使用的端口。

21. 运行脚本:

```
python s3test.py
```

输出类似下面的内容:

```
my-new-bucket 2015-02-16T17:09:10.000Z
```

测试 SWIFT 访问

Swift 访问的验证则可以使用 ``swift`` 的命令行客户端。可以通过命令 `man swift` 获取更多命令行选项的更多信息。

执行下面的命令安装 `swift` 客户端，在 Red Hat Enterprise Linux 上执行:

```
sudo yum install python-setuptools  
  
sudo easy_install pip  
  
sudo pip install --upgrade setuptools  
  
sudo pip install --upgrade python-swiftclient
```


在基于 Debian 的发行版上执行:

```
sudo apt-get install python-setuptools

sudo easy_install pip

sudo pip install --upgrade setuptools

sudo pip install --upgrade python-swiftclient
```

执行下面的命令验证 swift 访问:

```
swift -A http://{IP ADDRESS}:{port}/auth/1.0 -U
testuser:swift -K '{swift_secret_key}' list
```

使用网关服务器的外网 IP 地址替换其中的 {IP ADDRESS} , 使用新建 swift 用户时执行的命令 `radosgw-admin key create` 的输出替换其中的 {swift_secret_key}。使用你的 Civetweb 所使用的端口替换其中 {port} , 比如默认是 7480。如果你不替换这个端口, 它的默认值是 80. 举例如下:

```
swift -A http://10.19.143.116:7480/auth/1.0 -U
testuser:swift -K
'244+fz2gSqoHwR3lYtSbIyomyPHf3i7rgSJrF/IA' list
```

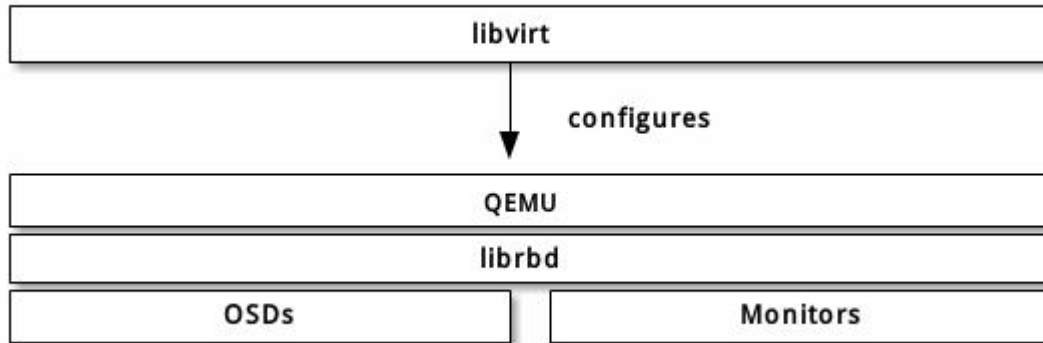
输出类似下面这样:

```
my-new-bucket
```

为块设备安装虚拟化支持

If you intend to use Ceph Block Devices and the Ceph Storage Cluster as a backend for Virtual Machines (VMs) or *Cloud Platforms* the QEMU/KVM and `libvirt` packages are important for enabling VMs and cloud platforms. Examples of VMs include: QEMU/KVM,

XEN, VMWare, LXC, VirtualBox, etc. Examples of Cloud Platforms include OpenStack, CloudStack, OpenNebula, etc.



安装 QEMU

QEMU KVM 能通过 `librbd` 库使用 Ceph 块设备，这对云平台是否能采用 Ceph 来说很重要。装好 QEMU 后，可以参考 [QEMU 和块设备](#) 把它用起来。

DEBIAN 软件包

QEMU 二进制包从 Ubuntu 12.04 Precise Pangolin 版起就被合并到了 Ubuntu 官方库。执行下面的命令安装 QEMU：

```
sudo apt-get install qemu
```

RPM 软件包

要安装 QEMU，可按如下步骤：

1. 更新软件库：

```
sudo yum update
```

2. 安装能和 Ceph 对接的 QEMU：

```
sudo yum install qemu-kvm qemu-kvm-tools qemu-img
```

3. 安装其它的 QEMU 软件包（可选的）：

```
sudo yum install qemu-guest-agent qemu-guest-agent-win32
```

构建 QEMU

To build QEMU from source, use the following procedure:

```
cd {your-development-directory}

git clone git://git.qemu.org/qemu.git

cd qemu

./configure --enable-rbd

make; make install
```

安装 LIBVIRT

To use `libvirt` with Ceph, you must have a running Ceph Storage Cluster, and you must have installed and configured QEMU. See [Using libvirt with Ceph Block Device](#) for usage.

DEBIAN 软件包

`libvirt` 软件包从 Ubuntu 12.04 Precise Pangolin 起就被并入了 Ubuntu 官方库。要在这些发行版上安装 `libvirt`，可用下面的命令：

```
sudo apt-get update && sudo apt-get install libvirt-bin
```

RPM 软件包

要通过 `libvirt` 使用 Ceph 存储集群，你必须有个正常运行的 Ceph 集群、还必须安装支持 `rbd` 格式的 QEMU。详情见 [安装 QEMU](#)。

近期的 CentOS/RHEL 发行版都集成了 `libvirt` 软件包。可执行如下命令安装 `libvirt`：

```
sudo yum install libvirt
```

构建 **LIBVIRT**

To build `libvirt` from source, clone the `libvirt` repository and use [AutoGen](#) to generate the build. Then, execute `make` and `make install` to complete the installation. For example:

```
git clone git://libvirt.org/libvirt.git

cd libvirt

./autogen.sh

make

sudo make install
```

详情见 [libvirt 安装手册](#)。

手动部署集群

把 Ceph 二进制包安装到各节点后，你也可以手动部署集群。手动过程主要是让部署脚本（如 Chef、Juju、Puppet 等）的开发者们验证程序的。

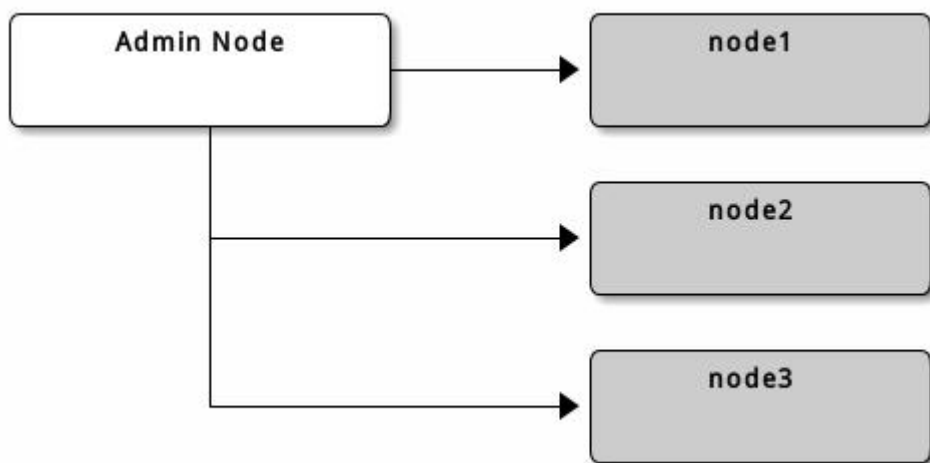
- [手动部署](#)

- [监视器的自举引导](#)
- [添加 OSD](#)
 - [精简型](#)
 - [细致型](#)
- [总结](#)

手动部署

所有 Ceph 集群都需要至少一个监视器、且 OSD 数量不小于副本数。自举引导初始监视器是部署 Ceph 存储集群的第一步，监视器的部署也为整个集群奠定了重要框架，如存储池副本数、每个 OSD 拥有的归置组数量、心跳周期、是否需认证等，其中大多数选项都有默认值，但是建设生产集群时仍需要您熟知它们。

按照[安装（快速）](#)里的相同配置，我们能配置起监视器为 `node1`，OSD 节点为 `node2`、`node3` 的集群。



监视器的自举引导

自举引导监视器（理论上是 Ceph 存储集群）需要以下几个条件：

- **惟一标识符：** `fsid` 是集群的惟一标识，它是 Ceph 作为文件系统时的文件系统标识符。现在，Ceph 还支持原生接口、块设备、和对象存储网关接口，所以 `fsid` 有点名不符实了。

- **集群名称**：每个 Ceph 集群都有自己的名字，它是个不含空格的字符串。默认名字是 `ceph`、但你可以更改；尤其是运营着多个集群时，需要用名字来区分要操作哪一个。

比如，当你以[联盟架构](#)运营多个集群时，集群名字（如 `us-west`、`us-east`）将作为标识符出现在 CLI 界面上。**注意**：要在命令行下指定某个集群，可以指定以集群名为前缀的配置文件（如 `ceph.conf`、`us-west.conf`、`us-east.conf` 等）；也可以参考 CLI 用法（`ceph --cluster {cluster-name}`）。

- **监视器名字**：同一集群内的各监视器例程都有惟一的名称，通常都用主机名作为监视器名字（我们建议每台主机只运行一个监视器、并且不要与 OSD 主机复用。短主机名可以用 `hostname -s` 获取。
- **监视器图**：自举引导初始监视器需要生成监视器图，为此，需要有 `fsid`、集群名（或用默认）、至少一个主机名及其 IP。
- **监视器密钥环**：监视器之间通过密钥通讯，所以你必须把监视器密钥加入密钥环，并在自举引导时提供。
- **管理密钥环**：要使用 `ceph` 这个命令行工具，你必须有 `client.admin` 用户，所以你要创建此用户及其密钥，并把他们加入密钥环。

前述必要条件并未提及 Ceph 配置文件的创建，然而，实践中最好创建个配置文件，并写好 `fsid`、`mon initial members` 和 `mon host` 配置。

你也可以查看或设置运行时配置。Ceph 配置文件可以只包含非默认配置，Ceph 配置文件的配置将覆盖默认值，把这些配置保存在配置文件里可简化维护。

具体过程如下：

1. 登录到初始监视器节点：

```
ssh {hostname}
```

如：

```
ssh node1
```

2. 确保保存 Ceph 配置文件的目录存在，Ceph 默认使用 `/etc/ceph`。安装 ceph 软件时，安装器也会自动创建 `/etc/ceph/` 目录。

```
3. ls /etc/ceph
```

注意：部署工具在清除集群时可能删除此目录（如 `ceph-deploy purgedata {node-name}`、`ceph-deploy purge {node-name}`）。

4. 创建 Ceph 配置文件，Ceph 默认使用 `ceph.conf`，其中的 `ceph` 是集群名字。

```
sudo vim /etc/ceph/ceph.conf
```

5. 给集群分配唯一 ID（即 `fsid`）。

```
6. uuidgen
```

7. 把此 ID 写入 Ceph 配置文件。

```
8. fsid = {UUID}
```

例如：

```
fsid = a7f64266-0894-4f1e-a635-d0aeaca0e993
```

9. 把初始监视器写入 Ceph 配置文件。

```
mon initial members = {hostname}[, {hostname}]
```

例如：

```
mon initial members = node1
```

10. 把初始监视器的 IP 地址写入 Ceph 配置文件、并保存。

```
mon host = {ip-address}[, {ip-address}]
```

例如：

```
mon host = 192.168.0.1
```

注意：你也可以写 IPv6 地址，但是必须设置 `ms bind ipv6 = true`。详情见[网络配置参考](#)。

11. 为此集群创建密钥环、并生成监视器密钥。

```
ceph-authtool --create-keyring /tmp/ceph.mon.keyring  
--gen-key -n mon. --cap mon 'allow *'
```

12. 生成管理员密钥环，生成 `client.admin` 用户并加入密钥环。

```
ceph-authtool --create-keyring  
/etc/ceph/ceph.client.admin.keyring --gen-key -n  
client.admin --set-uid=0 --cap mon 'allow *' --cap  
osd 'allow *' --cap mds 'allow'
```

13. 把 `client.admin` 密钥加入 `ceph.mon.keyring`。

```
ceph-authtool /tmp/ceph.mon.keyring --import-keyring  
/etc/ceph/ceph.client.admin.keyring
```

14. 用规划好的主机名、对应 IP 地址、和 FSID 生成一个监视器图，并保存为 `/tmp/monmap`。


```
monmaptool --create --add {hostname} {ip-address} --  
fsid {uuid} /tmp/monmap
```

例如：

```
monmaptool --create --add node1 192.168.0.1 --fsid  
a7f64266-0894-4f1e-a635-d0aeaca0e993 /tmp/monmap
```

15. 在监视器主机上分别创建数据目录。

```
sudo mkdir /var/lib/ceph/mon/{cluster-name}-  
{hostname}
```

例如：

```
sudo mkdir /var/lib/ceph/mon/ceph-node1
```

详情见[监视器配置参考——数据](#)。

16. 用监视器图和密钥环组装守护进程所需的初始数据。

```
ceph-mon [--cluster {cluster-name}] --mkfs -i  
{hostname} --monmap /tmp/monmap --keyring  
/tmp/ceph.mon.keyring
```

例如：

```
ceph-mon --mkfs -i node1 --monmap /tmp/monmap --  
keyring /tmp/ceph.mon.keyring
```

17. 仔细斟酌 Ceph 配置文件，公共的全局配置包括这些：

```
18.      [global]

19.      fsid = {cluster-id}

20.      mon initial members = {hostname}[, {hostname}]

21.      mon host = {ip-address}[, {ip-address}]

22.      public network = {network}[, {network}]

23.      cluster network = {network}[, {network}]

24.      auth cluster required = cephx

25.      auth service required = cephx

26.      auth client required = cephx

27.      osd journal size = {n}

28.      filestore xattr use omap = true

29.      osd pool default size = {n}  # Write an object
    n times.

30.      osd pool default min size = {n} # Allow
    writing n copy in a degraded state.

31.      osd pool default pg num = {n}

32.      osd pool default pgp num = {n}
```

```
osd crush chooseleaf type = {n}
```

按前述实例，`[global]` 段的配置大致如下：

```
[global]
```

```
fsid = a7f64266-0894-4f1e-a635-d0aeaca0e993

mon initial members = node1

mon host = 192.168.0.1

public network = 192.168.0.0/24

auth cluster required = cephx

auth service required = cephx

auth client required = cephx

osd journal size = 1024

filestore xattr use omap = true

osd pool default size = 2

osd pool default min size = 1

osd pool default pg num = 333

osd pool default pgp num = 333

osd crush chooseleaf type = 1
```

33. 建一个空文件 `done` , 表示监视器已创建、可以启动了 :

```
sudo touch /var/lib/ceph/mon/ceph-node1/done
```

34. 启动监视器。

在 Ubuntu 上用 Upstart :

```
sudo start ceph-mon id=node1 [cluster={cluster-name}]
```

要使此守护进程开机自启，需要创建两个空文件，像这样：

```
sudo touch /var/lib/ceph/mon/{cluster-name}-  
{hostname}/upstart
```

例如：

```
sudo touch /var/lib/ceph/mon/ceph-node1/upstart
```

在 Debian/CentOS/RHEL 上用 sysvinit：

```
sudo /etc/init.d/ceph start mon.node1
```

35. 验证下 Ceph 已经创建了默认存储池。

```
ceph osd lspools
```

你应该会看到这样的输出：

```
0 data,1 metadata,2 rbd,
```

36. 确认下集群在运行。

```
37.      ceph -s
```

你应该从输出里看到刚刚启动的监视器在正常运行，并且应该会看到一个健康错误：它表明归置组卡在了 `stuck inactive` 状态。输出大致如此：

```
cluster a7f64266-0894-4f1e-a635-d0aeaca0e993

health HEALTH_ERR 192 pgs stuck inactive; 192 pgs
stuck unclean; no osds

monmap e1: 1 mons at {node1=192.168.0.1:6789/0},
election epoch 1, quorum 0 node1

osdmap e1: 0 osds: 0 up, 0 in

pgmap v2: 192 pgs, 3 pools, 0 bytes data, 0
objects

0 kB used, 0 kB / 0 kB avail

192 creating
```

注意：一旦你添加了 OSD 并启动，归置组健康错误应该消失，详情见下一节。

添加 OSD

你的初始监视器可以正常运行后就可以添加 OSD 了。要想让集群达到 active + clean 状态，必须安装足够多的 OSD 来处理对象副本（如 `osd pooldefault size = 2` 需要至少 2 个 OSD）。在完成监视器自举引导后，集群就有了默认的 CRUSH 图，但现在此图还是空的，里面没有任何 OSD 映射到 Ceph 节点。

精简型

Ceph 软件包提供了 `ceph-disk` 工具，用于准备硬盘：可以是分区或用于 Ceph 的目录。`ceph-disk` 可通过递增索引来创建 OSD ID；还能把 OSD 加入 CRUSH 图。`ceph-disk` 的详细用法可参考 `ceph-disk -h`，此工具把后面将提到的精简型里面的步骤都自动化了。为按照精简型创建前两个 OSD，在 `node2` 和 `node3` 上执行下列命令：

1. 准备 OSD。

```
2. ssh {node-name}
```

```
sudo ceph-disk prepare --cluster {cluster-name} --  
cluster-uuid {uuid} --fs-type {ext4|xfs|btrfs}  
{data-path} [{journal-path}]
```

例如：

```
ssh node1  
  
sudo ceph-disk prepare --cluster ceph --cluster-uuid  
a7f64266-0894-4f1e-a635-d0aeaca0e993 --fs-type ext4  
/dev/hdd1
```

3. 激活 OSD：

```
sudo ceph-disk activate {data-path} [--activate-key  
{path}]
```

例如：

```
sudo ceph-disk activate /dev/hdd1
```

注：如果你的 Ceph 节点上没有 `/var/lib/ceph/bootstrap-
osd/{cluster}.keyring`，那么应该外加 `--activate-key` 参数。

细致型

要是不想借助任何辅助工具，可按下列步骤创建 OSD、将之加入集群和 CRUSH 图。按下列详细步骤可在 node2 和 node3 上增加前 2 个 OSD：

1. 登录到 OSD 主机。

```
ssh {node-name}
```

2. 给 OSD 分配 UUID。

```
3. uuidgen
```

4. 创建 OSD 。如果没有指定 UUID ，将会在 OSD 首次启动时分配一个。下列命令执行完成后将输出 OSD 号，在后续步骤里还会用到这个号。

```
ceph osd create [{uuid} [{id}]]
```

5. 在新 OSD 主机上创建默认目录。

```
6. ssh {new-osd-host}
```

```
sudo mkdir /var/lib/ceph/osd/{cluster-name}-{osd-  
number}
```

7. 如果要把 OSD 装到非系统盘的独立硬盘上，先创建文件系统、然后挂载到刚创建的目录下：

```
8. ssh {new-osd-host}
```

```
9. sudo mkfs -t {fstype} /dev/{hdd}
```

```
sudo mount -o user_xattr /dev/{hdd}  
/var/lib/ceph/osd/{cluster-name}-{osd-number}
```

10. 初始化 OSD 数据目录：

```
11.      ssh {new-osd-host}
```

```
sudo ceph-osd -i {osd-num} --mkfs --mkkey --osd-uuid  
[{uuid}]
```

加 `--mkkey` 选项运行 `ceph-osd` 之前，此目录必须是空的；另外，如果集群名字不是默认值，还要给 `ceph-osd` 指定 `--cluster` 选项。

12. 注册此 OSD 的密钥。路径内 `ceph-{osd-num}` 里的 `ceph` 其含义为 `$cluster-$id`，如果你的集群名字不是 `ceph`，请指定自己的集群名：

```
sudo ceph auth add osd.{osd-num} osd 'allow *' mon  
'allow profile osd' -i /var/lib/ceph/osd/{cluster-  
name}-{osd-num}/keyring
```

13. 把此节点加入 CRUSH 图。

```
ceph [--cluster {cluster-name}] osd crush add-bucket  
{hostname} host
```

例如：

```
ceph osd crush add-bucket node1 host
```

14. 把此 Ceph 节点放入 `default` 根下。

```
ceph osd crush move node1 root=default
```

15. 把此 OSD 加入 CRUSH 图之后，它就能接收数据了。你也可以反编译 CRUSH 图、把此 OSD 加入设备列表、对应主机作为桶加入（如果它还不在于 CRUSH 图里）、然后此设备作为主机的一条目、分配权重、重新编译、注入集群。

```
ceph [--cluster {cluster-name}] osd crush add {id-  
or-name} {weight} [{bucket-type}={bucket-name} ...]
```

例如：


```
ceph osd crush add osd.0 1.0 host=node1
```

16. 把 OSD 加入 Ceph 后，OSD 已经在配置里了。但它还没开始运行，这时处于 `down` 且 `in` 状态，要启动进程才能收数据。

在 Ubuntu 系统上用 Upstart 启动：

```
sudo start ceph-osd id={osd-num} [cluster={cluster-name}]
```

例如：

```
sudo start ceph-osd id=0  
  
sudo start ceph-osd id=1
```

在 Debian/CentOS/RHEL 上用 sysvinit 启动：

```
sudo /etc/init.d/ceph start osd.{osd-num} [--cluster {cluster-name}]
```

例如：

```
sudo /etc/init.d/ceph start osd.0  
  
sudo /etc/init.d/ceph start osd.1
```

要让守护进程开机自启，必须创建一个空文件：

```
sudo touch /var/lib/ceph/osd/{cluster-name}-{osd-num}/sysvinit
```

例如：

```
sudo touch /var/lib/ceph/osd/ceph-0/sysvinit  
  
sudo touch /var/lib/ceph/osd/ceph-1/sysvinit
```

OSD 启动后，它应该处于 `up` 且 `in` 状态。

总结

监视器和两个 OSD 开始正常运行后，你可以通过下列命令观察归置组互联过程了：

```
ceph -w
```

执行下列命令查看 OSD 树：

```
ceph osd tree
```

你应该会看到类似如下的输出：

#	id	weight	type	name	up/down	reweight
-1	2		root	default		
-2	2			host node1		
0	1			osd.0	up	1
-3	1			host node2		
1	1			osd.1	up	1

要增加（或删除）额外监视器，参见[增加/删除监视器](#)。要增加（或删除）额外 OSD，参见[增加/删除 OSD](#)。

升级软件

随着新版 Ceph 的发布，你也许想升级集群以使用新功能，升级集群前请参考升级文档。有时候，升级 Ceph 需要严格按照升级顺序进行。

- [升级 Ceph](#)
 - [概述](#)
 - [ceph-deploy 工具](#)
 - [Argonaut 到 Bobtail](#)
 - [Argonaut 到 Cuttlefish](#)
 - [Bobtail 到 Cuttlefish](#)
 - [Cuttlefish 到 Dumpling](#)
 - [Dumpling 到 Emperor](#)
 - [Dumpling 到 Firefly](#)
 - [Emperor 到 Firefly](#)
 - [升级过程](#)
 - [迁移到 ceph-deploy](#)

升级 CEPH

Ceph 的各个版本都可能有一定的步骤，升级前请参考与此版本相关的章节和[发布说明](#)文档，以确定有哪些特定于此版本的步骤。

概述

你可以在 Ceph 集群在线且提供服务时升级守护进程！某些类型的守护进程依赖其他的，如 Ceph 元数据服务器和 Ceph 对象网关依赖于 Ceph 监视器和 OSD 守护进程，所以我们建议按以下顺序升级：

1. [ceph-deploy 工具](#)
2. Ceph 监视器
3. Ceph OSD 守护进程
4. Ceph 元数据服务器
5. Ceph 对象网关

作为普适规则，我们建议一次升级一类的所有守护进程（如所有 `ceph-osd`、所有 `ceph-mon` 等），这样才能确保它们属于同一版本。

升级过程相对简单，只要参照此版本特定的章节即可。基本过程有三个：

1. 用 `ceph-deploy` 为各主机升级软件包（用 `ceph-deploy install` 命令），或者分别登录各主机**手动升级**。例如，**升级监视器**时，`ceph-deploy` 语法大致如此：

```
2. ceph-deploy install --release {release-name} ceph-  
node1[ ceph-node2]
```

```
ceph-deploy install --release firefly mon1 mon2 mon3
```

注：`ceph-deploy install` 命令会把指定节点上的旧版本升级为你所指定的版本，没有 `ceph-deploy upgrade` 这样的升级命令。

3. 登录各节点并重启各相关 Ceph 守护进程，详情参见**操纵集群**。
4. 确认集群健康状况，详情见**监控集群**。

Important

一旦升级完，就不能降级了。

CEPH-DEPLOY 工具

升级 Ceph 守护进程前，应该先升级 `ceph-deploy` 工具。

```
sudo pip install -U ceph-deploy
```

或者：

```
sudo apt-get install ceph-deploy
```

或者：

```
sudo yum install ceph-deploy python-pushy
```

ARGONAUT 到 BOBTAIL

从 Argonaut 升级到 Bobtail 时，你得注意几件事：

1. 现在认证默认是**启用的**，而以前是**禁用的**；
2. 监视器间使用新的通讯协议；
3. 通过 RBD 使用 `format2` 格式的映像前要完成所有 OSD 的升级。

确保你先更新软件库路径，如：

```
sudo rm /etc/apt/sources.list.d/ceph.list

echo deb http://download.ceph.com/debian-bobtail/
$(lsb_release -sc) main | sudo tee
/etc/apt/sources.list.d/ceph.list
```

详情见如下几段。

认证

Bobtail 版默认启用了认证，而且预置了粒度更细的认证配置选项。在先前的版本（即 v0.55 和更早的）中你可以简单地写：

```
auth supported = [cephx | none]
```

此选项仍然能被识别，但已经过时。新版支持 `cluster`、`service`、和 `client` 认证选项，如下：

```
auth cluster required = [cephx | none] # default cephx

auth service required = [cephx | none] # default cephx
```

```
auth client required = [cephx | none] # default
cephx, none
```

Important

如果你现在的集群没有用 `auth supported` 这一行启用认证，那你升级到 Bobtail 时必须显式地关闭：

```
auth cluster required = none

auth service required = none
```

这样集群认证将被关闭，但客户端默认行为未更改，它仍可以与启用了认证、但不强求的集群通讯。

Important

如果你的集群配置文件之前就有 `auth supported` 选项，那就没必要更改了。

详情见 [Ceph 认证——向后兼容性](#)。

监视器线上协议

我们建议把所有监视器都升级到 Bobtail，混用 Bobtail 和 Argonaut 将不能使用新的线上协议，因为此协议要求所有监视器的版本都在 Bobtail 及以上。只升级大多数（如三个中的两个）监视器也会导致潜在风险，即再失效一个就会丧失可用性（因为未升级的守护进程不能参与新协议构成的通讯）。我们建议各 `ceph-mon` 升级期间不要等待。

RBD 映像

Bobtail 版现在支持 `format 2` 格式的映像了！但是在所有 `ceph-osd` 都升级完之前，不要新建或使用 `format 2` 格式的 RBD 映像。注意，`format 1` 仍然是默认值。你可以用新的 `ceph osd ls` 和 `ceph tell osd.N version` 命令再次确认软件版

本，`ceph osd ls` 将给出集群内所有 OSD 的 ID 列表，把它们代入 shell 循环即可查询所有 OSD 的版本：

```
for i in $(ceph osd ls); do
    ceph tell osd.${i} version
done
```

ARGONAUT 到 CUTTLEFISH

要从 Argonaut 升级到 Cuttlefish，请仔细阅读本段、以及从 Argonaut 升到 Bobtail、还有从 Bobtail 升级到 Cuttlefish。从 Argonaut 升级到 Cuttlefish 时，**你必须先把所有监视器从 Argonaut 升级到 Bobtail v0.56.5** !!! 所有其他的守护进程都可以从 Argonaut 直接升级到 Cuttlefish，无需间接地从 Bobtail 过渡一次。

Important

确保软件仓库指定的是 Bobtail，而不是 Cuttlefish。

例如：

```
sudo rm /etc/apt/sources.list.d/ceph.list

echo deb http://download.ceph.com/debian-bobtail/
$(lsb_release -sc) main | sudo tee
/etc/apt/sources.list.d/ceph.list
```

我们建议把监视器升级到 Cuttlefish 之前，先升级到 Bobtail。混用 Bobtail 和 Argonaut 监视器将导致旧监视器不能使用新线上协议，因为此协议在 Bobtail 及更高版本上才可用。只升级大多数（如三个中的两个）监视器也会导致潜在风险，即再失效一个就会丧失可用性（因为未升级的守护进程不能参与新协议构成的通讯）。我们建议各 `ceph-mon` 升级期间不要等待。详情见[升级监视器](#)。

Note

关于 Bobtail 的认证及向后兼容性请参考[认证](#)和 [Ceph 认证——向后兼容性](#)。

把监视器从 Argonaut 升级到 Bobtail、并重启无误（可以形成法定人数）后，还必须从 Bobtail 再升级到 Cuttlefish。再次升级前，记得更改到 Cuttlefish 软件库的引用，例如：

```
sudo rm /etc/apt/sources.list.d/ceph.list

echo deb http://download.ceph.com/debian-cuttlefish/
$(lsb_release -sc) main | sudo tee
/etc/apt/sources.list.d/ceph.list
```

详情见[升级监视器](#)。

Argonaut 和 Cuttlefish 的监视器架构差异较大，详情见[监视器配置](#)和 [Joao 的博客文章](#)。完成监视器的升级后，OSD 和 MDS 守护进程可以按照常规步骤升级，详情见[升级单个 OSD](#) 和 [升级单个元数据服务器](#)。

BOBTAIL 到 CUTTLEFISH

从 Bobtail 升级到 Cuttlefish 有几点要特别注意，首先，监视器架构大变，所以你应该把所有监视器都升级到 Cuttlefish；其次，如果你的集群有多个元数据服务器，应该确保它们的名字都唯一。详情如下。

把较老的 `apt` 源替换为 Cuttlefish，如：

```
sudo rm /etc/apt/sources.list.d/ceph.list

echo deb http://download.ceph.com/debian-cuttlefish/
$(lsb_release -sc) main | sudo tee
/etc/apt/sources.list.d/ceph.list
```

监视器

Argonaut 和 Cuttlefish 的监视器架构差异较大，详情见[监视器配置](#)和 [Joao 的博客文章](#)。这意味着 v0.61 与较早版本不能相互通讯。在升级各监视器时，它会把本地数据转换为新格式存储，

一旦完成了大多数监视器的升级，它们就能用新协议形成法定人数，并且老版本的监视器会被排斥在外。正因如此，我们建议一次性升级完所有监视器。

Important

混用版本的集群不要运行太长时间。

MDS 唯一名称

现在的监视器要求集群内的 MDS 名字唯一。如果你的多个元数据服务器 ID 相同（比如都是 `mds.a`），第二个元数据服务器将把前一个显式地标记为 `failed`，多个同名 MDS 必须改得互不相同。如果你的集群只有一个元数据服务器，那你可以忽略此提醒。

CEPH-DEPLOY

现在，`ceph-deploy` 是我们推荐的集群部署工具。对于已经用 `mkcephfs` 创建、并想迁移到 `ceph-deploy` 的用户，这里提供了正确迁移的文档：[迁移到 ceph-deploy](#)。

CUTTLEFISH 到 DUMPLING

从 Cuttlefish (v0.61-v0.61.7) 开始可以滚动升级。但还有几点要特别注意：首先，你必须升级 `ceph` 这个命令行工具，因为它变动很大；其次，你必须把所有监视器升级到 `Dumpling`，因为协议有变动。

把较老的软件库源替换为 `Dumpling` 源，例如用 `apt` 执行：

```
sudo rm /etc/apt/sources.list.d/ceph.list

echo deb http://download.ceph.com/debian-dumpling/
$(lsb_release -sc) main | sudo tee
/etc/apt/sources.list.d/ceph.list
```

在 CentOS/Red Hat 发行版上可删除旧源。

```
sudo rm /etc/yum.repos.d/ceph.repo
```

然后用下列内容创建个新仓库 `ceph.repo`。

[ceph]

```
name=Ceph Packages and Backports $basearch  
  
baseurl=http://download.ceph.com/rpm/el6/$basearch  
  
enabled=1  
  
gpgcheck=1  
  
type=rpm-md  
  
gpgkey=https://download.ceph.com/keys/release.asc
```

Note

确保使用与自己发行版相匹配的 URL，对应发行版请检查 <http://download.ceph.com/rpm>。

Note

如果你可以用 `ceph-deploy` 升级软件，那你只需要把仓库加到运行 `ceph` 或 `ceph-deploy` 命令的客户端节点即可。

DUMPLING 到 EMPEROR

Dumpling (v0.64) 可滚动升级。

把较老的软件库源替换为 Emperor 源，例如用 `apt` 执行：

```
sudo rm /etc/apt/sources.list.d/ceph.list
```

```
echo deb http://download.ceph.com/debian-emperor/  
$(lsb_release -sc) main | sudo tee  
/etc/apt/sources.list.d/ceph.list
```

在 CentOS/Red Hat 发行版上可删除旧源。

```
sudo rm /etc/yum.repos.d/ceph.repo
```

然后新增软件库 `ceph.repo`，其内容如下，要用自己的发行版名字（如 `el6`、`rhel6` 等等）替换 `{disro}`。

```
[ceph]  
  
name=Ceph Packages and Backports $basearch  
  
baseurl=http://download.ceph.com/rpm-  
emperor/{distro}/$basearch  
  
enabled=1  
  
gpgcheck=1  
  
type=rpm-md  
  
gpgkey=https://download.ceph.com/keys/release.asc
```

Note

确保使用与自己发行版相匹配的 URL，对应发行版请检查 <http://download.ceph.com/rpm>。

Note

如果你可以用 `ceph-deploy` 升级软件，那你只需要把仓库加到运行 `ceph` 或 `ceph-deploy` 命令的客户端节点即可。

命令行工具

在 v0.65 中，ceph 命令行接口（CLI）工具变动很大；老的 CLI 不能与 Dumpling 通讯，也就是说，要用 ceph 命令访问 Ceph 存储集群的所有节点其 ceph-common 库必须升级。

```
sudo apt-get update && sudo apt-get install ceph-common
```

确保你已安装最新版（v0.67 或更新）。如果还没有，你也许得卸载、清除相关依赖，然后重新安装。

关于新命令行的细节在 [v0.65](#)。

监视器

Dumpling (v0.67) 版本的 ceph-mon 守护进程与 v0.66 及更早版本相比，内部协议有所变更，也就是说它不能与 v0.66 或更老的版本通讯。大多数监视器升级完后，它们就能用新协议形成法定人数了，旧版监视器将被排斥在外，正因如此，我们建议要一次性升级所有监视器（或者较快的节奏），以最小化可能的当机时间。

Important

混用版本的集群不要运行太长时间。

DUMPLING 到 FIREFLY

如果您的现有集群运行着低于 v0.67 Dumpling 的版本，请先升级到最新的 Dumpling 版，然后再升级到 v0.80 Firefly 版。

监视器

Dumpling (v0.67) 版本的 ceph-mon 守护进程与 v0.66 及更早版本相比，内部协议有所变更，也就是说它不能与 v0.66 或更老的版本通讯。大多数监视器升级完后，它们就能用新协议形成法定人数了，旧版监视器将被排斥在外，正因如此，我们建议要一次性升级所有监视器（或者较快的节奏），以最小化可能的当机时间。

Important

混用版本的集群不要运行太长时间。

CEPH 配置文件变更

我们建议升级前先把下列配置加入 `ceph.conf` 配置文件的 `[mon]` 段下：

```
mon warn on legacy crush tunables = false
```

此配置可消除因用着老 CRUSH 归置法而引起的健康告警。虽说可以在全集群范围内重均衡已有数据，但我们不建议生产集群做，因为它涉及大量数据，而且重均衡会导致严重的性能降级。

命令行工具

在 V0.65 版中，`ceph` 命令行界面（CLI）工具有重大改变，老版的 CLI 不能用于 Firefly。也就是说，在升级 Ceph 守护进程前，要用 `ceph` 命令访问存储集群的节点都必须升级 `ceph-common` 库。

在 Debian/Ubuntu 上可用此命令：

```
sudo apt-get update && sudo apt-get install ceph-common
```

在 CentOS/RHEL 上可用此命令：

```
sudo yum install ceph-common
```

确保你已安装最新版。如果还没有，你也许得卸载、清除相关依赖，然后重新安装。

关于新命令行界面的详细情况请参考 [v0.65](#)。

升级顺序

把旧版软件库改为 Firefly 的，例如用 `apt` 命令执行此命令：

```
sudo rm /etc/apt/sources.list.d/ceph.list
```

```
echo deb http://download.ceph.com/debian-firefly/  
$(lsb_release -sc) main | sudo tee  
/etc/apt/sources.list.d/ceph.list
```

对于 CentOS/Red Hat 发行版，先删除旧版软件库。

```
sudo rm /etc/yum.repos.d/ceph.repo
```

然后新增一个 `ceph.repo` 软件库条目，其内容如下，还有把 `{distro}` 替换为你的发行版名字（如 `el6`、`rhel6`、`rhel7` 等等）。

```
[ceph]  
  
name=Ceph Packages and Backports $basearch  
  
baseurl=http://download.ceph.com/rpm-  
firefly/{distro}/$basearch  
  
enabled=1  
  
gpgcheck=1  
  
type=rpm-md  
  
gpgkey=https://download.ceph.com/keys/release.asc
```

按如下顺序升级守护进程：

1. **监视器：** 如果 `ceph-mon` 守护进程晚于 `ceph-osd` 守护进程启动，那么这些监视器就不能正确注册其能力，新功能也不可用，除非再次重启。
2. **OSD**
3. **元数据服务器：** 如果 `ceph-mds` 守护进程先被重启了，它只能先等着，直到所有 OSD 都升级完，它才能完全启动。
4. **网关：** 一起升级 `radosgw` 守护进程。多片上传功能有微小的行为变化，它会阻止由新版 `radosgw` 发起、却由旧版 `radosgw` 完成的多片上传请求。

Note

确保先升级完**所有** Ceph 监视器、**而且重启完毕**，然后再升级并重启各 OSD、各元数据服务器和网关。

EMPEROR 到 FIREFLY

如果您的现有集群运行着低于 v0.67 Dumpling 的版本，请先升级到最新的 Dumpling 版，然后再升级到 v0.80 Firefly 版。详细步骤请参考 [Cuttlefish 到 Dumpling](#) 和 [Firefly 发布说明](#)。若从 Emperor 之后的版本升级，请参考 [Firefly 发布说明](#)。

CEPH 配置文件变更

我们建议升级前先把下列配置加入 `ceph.conf` 配置文件的 `[mon]` 段下：

```
mon warn on legacy crush tunables = false
```

此配置可消除因用着老 CRUSH 归置法而引起的健康告警。虽说可以在全集群范围内重均衡已有数据，但我们不建议生产集群做，因为它涉及大量数据，而且重均衡会导致严重的性能降级。

升级顺序

把旧版软件库改为 Firefly 的，例如用 `apt` 命令执行此命令：

```
sudo rm /etc/apt/sources.list.d/ceph.list

echo deb http://download.ceph.com/debian-firefly/
$(lsb_release -sc) main | sudo tee
/etc/apt/sources.list.d/ceph.list
```

对于 CentOS/Red Hat 发行版，先删除旧版软件库。

```
sudo rm /etc/yum.repos.d/ceph.repo
```

然后新增一个 `ceph.repo` 软件库条目，其内容如下，还有把 `{distro}` 替换为你的发行版名字（如 `el6`、`rhel6`、`rhel7` 等等）。

[ceph]

```
name=Ceph Packages and Backports $basearch
```

```
baseurl=http://download.ceph.com/rpm/{distro}/$basearch
```

```
enabled=1
```

```
gpgcheck=1
```

```
type=rpm-md
```

```
gpgkey=https://download.ceph.com/keys/release.asc
```

Note

确保使用与自己发行版相匹配的 URL，对应发行版请检查 <http://download.ceph.com/rpm>。

Note

如果你可以用 `ceph-deploy` 升级软件，那你只需要把仓库加到运行 `ceph` 或 `ceph-deploy` 命令的客户端节点即可。

按如下顺序升级守护进程：

1. **监视器：** 如果 `ceph-mon` 守护进程晚于 `ceph-osd` 守护进程启动，那么这些监视器就不能正确注册其能力，新功能也不可用，除非再次重启。
2. **OSD**
3. **元数据服务器：** 如果 `ceph-mds` 守护进程先被重启了，它只能先等着，直到所有 OSD 都升级完，它才能完全启动。
4. **网关：** 一起升级 `radosgw` 守护进程。多片上传功能有微小的行为变化，它会阻止由新版 `radosgw` 发起、却由旧版 `radosgw` 完成的多片上传请求。

升级过程

下面是具体升级进程。

Important

Ceph 的各版本可能有不同的步骤，所以**升级前**请参考此版本特定的升级步骤。

升级监视器

要升级监视器，执行下列步骤：

1. 升级各守护进程的二进制包。

你可以用 `ceph-deploy` 一次升级所有监视器节点，如：

```
ceph-deploy install --release {release-name} ceph-  
node1[ ceph-node2]  
  
ceph-deploy install --release hammer mon1 mon2 mon3
```

你也可以用包管理器挨个升级各节点。手动升级 Debian/Ubuntu 主机上软件包的步骤如下：

```
ssh {mon-host}  
  
sudo apt-get update && sudo apt-get install ceph
```

在 CentOS/Red Hat 主机上相应的命令如下：

```
ssh {mon-host}  
  
sudo yum update && sudo yum install ceph
```

2. 重启各监视器。Debian/Ubuntu 发行版的命令如下：

```
sudo restart ceph-mon id={hostname}
```

CentOS/Red Hat/Debian 发行版的命令如下：

```
sudo /etc/init.d/ceph restart {mon-id}
```

用 `ceph-deploy` 部署的 CentOS/Red Hat 发行版，其监视器 ID 通常是 `mon.{hostname}`。

3. 确保各监视器都重回法定人数。

```
ceph mon stat
```

再次确认你完成了所有监视器的升级。

升级单个 OSD

升级单个 OSD 守护进程的步骤如下：

1. 升级 OSD 守护进程对应的软件包。

你可以用 `ceph-deploy` 一次升级所有 OSD 守护进程，如：

```
ceph-deploy install --release {release-name} ceph-  
node1[ ceph-node2]  
  
ceph-deploy install --release hammer osd1 osd2 osd3
```

你也可以用包管理器挨个升级各节点。手动升级 Debian/Ubuntu 主机上软件包的步骤如下。

```
ssh {osd-host}
```

```
sudo apt-get update && sudo apt-get install ceph
```

在 CentOS/Red Hat 主机上相应的命令如下：

```
ssh {osd-host}  
  
sudo yum update && sudo yum install ceph
```

2. 重启 OSD，其中 `N` 是 OSD 号。对 Debian/Ubuntu，用命令：

```
sudo restart ceph-osd id=N
```

对于一主机上的多个 OSD，你可以用 Upstart 一次性全部重启。

```
sudo restart ceph-osd-all
```

对于 CentOS/Red Hat/Debian 发行版，用：

```
sudo /etc/init.d/ceph restart N
```

3. 确保升级后的 OSD 重新加入了集群：

```
ceph osd stat
```

再次确认所有 OSD 守护进程已升级完。

升级单个元数据服务器

要升级单个 Ceph 元数据服务器，挨个执行下列步骤：

1. 升级二进制包。你可以用 `ceph-deploy` 一次升级所有 MDS 节点，也可以在各节点用包管理器升级，如：

```
2. ceph-deploy install --release {release-name} ceph-  
node1
```

```
ceph-deploy install --release hammer mds1
```

在 Debian/Ubuntu 主机上可这样手动升级。

```
ssh {mon-host}  
  
sudo apt-get update && sudo apt-get install ceph-mds
```

在 CentOS/Red Hat 主机上则是：

```
ssh {mon-host}  
  
sudo yum update && sudo yum install ceph-mds
```

3. 重启元数据服务器。在 Debian/Ubuntu 上用：

```
sudo restart ceph-mds id={hostname}
```

在 CentOS/Red Hat/Debian 上用：

```
sudo /etc/init.d/ceph restart mds.{hostname}
```

用 `ceph-deploy` 部署的集群其 `{hostname}` 通常是所在主机的主机名。

4. 确保元数据服务器已启动，且运行着：

```
ceph mds stat
```

升级客户端

升级软件包并重启完集群之后，我们建议同时升级下客户端节点上的 `ceph-common` 和客户端库 (`librbd1` 和 `librados2`)。

1. 升级软件包：

```
2. ssh {client-host}
```

```
apt-get update && sudo apt-get install ceph-common  
librados2 librbd1 python-rados python-rbd
```

3. 确认升级后的版本：

```
4. ceph --version
```

如果不是最新版，你也许得卸载、清理掉依赖、然后重新安装。

迁移到 CEPH-DEPLOY

如果您的现有集群是用 `mkcephfs` 部署的（通常是 Argonaut 或 Bobtail），那么有些地方要改一下才能切换到 `ceph-deploy`。

监视器密钥环

要把 `caps mon = "allow *"` 加进监视器密钥环（如果还没加的话）。默认情况下监视器密钥环位于 `/var/lib/ceph/mon/ceph-$id/keyring`，增加 `caps` 配置后它应该类似如下：

```
[mon.]  
  
key = AQBjIHhRuHCwDRAAZjBTSJcIBIoGpdOR9ToiyQ==  
  
caps mon = "allow *"
```

添加 `caps mon = "allow *"` 将简化 `mkcephfs` 到 `ceph-deploy` 的迁移，这样 `ceph-create-keys` 就可以用 `$mon_data` 里的 `mon.` 密钥环获取它所需的权限了。

用默认路径

`mon` 和 `osd` 目录需用 `/var/lib/ceph` 之下的默认子目录。

- **OSDs:** 路径应该是 `/var/lib/ceph/osd/ceph-$id`
- **MON:** 路径应该是 `/var/lib/ceph/mon/ceph-$id`

在这些目录下都应该有一个名为 `keyring` 的密钥环文件。

CEPH 存储集群

所有 Ceph 部署都始于 **Ceph 存储集群**。基于 RADOS 的 Ceph 对象存储集群包括两类守护进程：`term:对象存储守护进程`（OSD）把存储节点上的数据存储为对象；`term:Ceph 监视器`（MON）维护集群运行图的主拷贝。一个 Ceph 集群可以包含数千个存储节点，最简系统至少需要一个监视器和两个 OSD 才能做到数据复制。

Ceph 文件系统、Ceph 对象存储、和 Ceph 块设备从 Ceph 存储集群读出和写入数据。

配置

Ceph 作为集群时可以包含数千个对象存储设备（OSD）。最简系统至少需要二个 OSD 做数据复制。要配置 OSD 集群，你得把配置写入配置文件。Ceph 对很多选项提供了默认值，你可以在配置文件里覆盖它们；另外，你可以使用命令行工具修改运行时配置。

Ceph 启动时要激活三类守护进程：

- `ceph-mon`（必备）
- `ceph-osd`（必备）
- `ceph-mds`（`cephfs` 必备）

各进程、守护进程或工具都会读取配置文件。一个进程可能需要不止一个守护进程例程的信息（即多个上下文）；一个守护进程或工具只有关于单个守护进程例程的信息（单上下文）。

Note

试用时 Ceph 可运行在单机上。

硬盘和文件系统推荐

准备硬盘

Ceph 注重数据安全，就是说，**Ceph 客户端**收到数据已写入存储器的通知时，数据确实已写入**硬盘**。使用较老的内核（版本小于 2.6.33）时，如果日志在原始硬盘上，就要禁用写缓存；较新的内核没问题。

用 `hdparm` 禁用硬盘的写缓冲功能。

```
sudo hdparm -W 0 /dev/hda 0
```

在生产环境，我们建议操作系统和 **Ceph OSD 守护进程** 数据分别放到不同的硬盘。如果必须把数据和系统放在同一硬盘里，最好给数据分配一个单独的分区。

文件系统

Ceph 的 OSD 依赖于底层文件系统的稳定性和性能。

Note

当前，我们推荐部署生产系统时使用 `xfs` 文件系统；推荐用 `btrfs` 做测试、开发和其他不太要紧的部署。我们相信，长远来看 `btrfs` 适合 Ceph 的功能需求和发展方向，但是 `xfs` 和 `ext4` 能提供当前部署所必需的稳定性。`btrfs` 开发在迅速推进，所以它的用户应该有能力经常更新到最新内核发布，而且能跟踪严重缺陷的修正进度。

OSD 守护进程有赖于底层文件系统的扩展属性（XATTR）存储各种内部对象状态和元数据。底层文件系统必须能为 XATTR 提供足够容量，`btrfs` 没有限制随文件的 `xattr` 元数据总量；`xfs` 的限制相对大（64KB），多数部署都不会有瓶颈；`ext4` 的则太小而不可用。

使用 `ext4` 文件系统时，一定要把下面的配置放于 `ceph.conf` 配置文件的 `[osd]` 段下；用 `btrfs` 和 `xfs` 时可以选择。

```
filestore xattr use omap = true
```

文件系统背景知识

XFS、btrfs 和 ext4 相比较 ext3 而言，在高伸缩性数据存储方面有几个优势。

XFS、btrfs 和 ext4 都是**日志文件系统**，这使得在崩溃、断电后恢复时更健壮，因为这些文件系统在写入数据前会先记录所有变更。

xfs 由 Silicon Graphics 开发，是一个成熟、稳定的文件系统。相反，btrfs 是相对年轻的文件系统，它致力于实现系统管理员梦寐以求的大规模数据存储基础，和其他 Linux 文件系统相比它有独一无二的功能和优势。

btrfs 是写时复制（**copy-on-write**，cow）文件系统，它支持文件创建时间戳和校验和（可校验元数据完整性）功能，所以它能探测到数据坏副本，并且用好副本修复。写时复制功能是说 btrfs 支持可写文件系统快照。btrfs 也支持透明压缩和其他功能。

btrfs 也集成了多设备管理功能，据此可以在底层支持异质硬盘存储，和数据分配策略。未来开发社区还会提供 fsck、拆分、数据加密功能，这些诱人的功能正是 Ceph 集群的理想选择。

配置 CEPH

你启动 Ceph 服务时，初始化进程会把一系列守护进程放到后台运行。**Ceph 存储集群**运行两种守护进程：

- **Ceph 监视器**（ceph-mon）
- **Ceph OSD 守护进程**（ceph-osd）

要支持 **Ceph 文件系统** 功能，它还需要运行至少一个 **Ceph 元数据服务器**（ceph-mds）；支持 **Ceph 对象存储** 功能的集群还要运行网关守护进程（radosgw）。为方便起见，各类守护进程都有一系列默认值（很多由 ceph/src/common/config_opts.h 配置），你可以用 Ceph 配置文件覆盖这些默认值。

配置文件

启动 Ceph 存储集群时，各守护进程都从同一个配置文件（即默认的 ceph.conf）里查找它自己的配置。手动部署时，你需要创建此配置文件；用部署工具（如 ceph-deploy、chef 等）创建配置文件时可以参考下面的信息。配置文件定义了：

- 集群身份

- 认证配置
- 集群成员
- 主机名
- 主机 IP 地址
- 密钥环路径
- 日志路径
- 数据路径
- 其它运行时选项

默认的 Ceph 配置文件位置相继排列如下：

1. `$CEPH_CONF`（就是 `$CEPH_CONF` 环境变量所指示的路径）；
2. `-c path/path`（就是 `-c` 命令行参数）；
3. `/etc/ceph/ceph.conf`
4. `~/.ceph/config`
5. `./ceph.conf`（就是当前所在的工作路径。

Ceph 配置文件使用 *ini* 风格的语法，以分号 (;) 和井号 (#) 开始的行是注释，如下：

```
# <--A number (#) sign precedes a comment.

; A comment may be anything.

# Comments always follow a semi-colon (;) or a pound (#)
on each line.

# The end of the line terminates a comment.

# We recommend that you provide comments in your
configuration file(s).
```

配置段落

Ceph 配置文件可用于配置存储集群内的所有守护进程、或者某一类型的所有守护进程。要配置一系列守护进程，这些配置必须位于能收到配置的段落之下，比如：

```
[global]
```

描述: `[global]` 下的配置影响 Ceph 集群里的所有守护进程。

实例: `auth supported = cephx`

`[osd]`

描述: `[osd]` 下的配置影响存储集群里的所有 `ceph-osd` 进程，并且会覆盖 `[global]` 下的同一选项。

实例: `osd journal size = 1000`

`[mon]`

描述: `[mon]` 下的配置影响集群里的所有 `ceph-mon` 进程，并且会覆盖 `[global]` 下的同一选项。

实例: `mon addr = 10.0.0.101:6789`

`[mds]`

描述: `[mds]` 下的配置影响集群里的所有 `ceph-mds` 进程，并且会覆盖 `[global]` 下的同一选项。

实例: `host = myserver01`

例:

```
[client]
```

描述: `[client]` 下的配置影响所有客户端（如挂载的 Ceph 文件系统、挂载的块设备等等）。

实例: `log file = /var/log/ceph/radosgw.log`

全局设置影响集群内所有守护进程的例程，所以 `[global]` 可用于设置适用所有守护进程的选项。但可以用这些覆盖 `[global]` 设置：

1. 在 `[osd]`、`[mon]`、`[mds]` 下更改某一类进程的配置。
2. 更改特定进程的设置，如 `[osd.1]`。

覆盖全局设置会影响所有子进程，明确剔除的例外。

典型的全局设置包括激活认证，例如：

```
[global]
```

```
#Enable authentication between hosts within the cluster.
```

```
#v 0.54 and earlier
```

```
auth supported = cephx
```

```
#v 0.55 and after
```

```
auth cluster required = cephx
```

```
auth service required = cephx

auth client required = cephx
```

你可以统一配置一类守护进程。配置写到 `[osd]`、`[mon]`、`[mds]` 下时，无须再指定某个特定例程，即可分别影响所有 OSD、监视器、元数据进程。

典型的类范畴配置包括日志尺寸、filestore 选项等，如：

```
[osd]

osd journal size = 1000
```

你也可以配置某个特定例程。一个例程由类型和及其例程 ID 确定，OSD 的例程 ID 只能是数字，但监视器和元数据服务器的 ID 可包含字母和数字。

```
[osd.1]

# settings affect osd.1 only.


[mon.a]

# settings affect mon.a only.


[mds.b]

# settings affect mds.b only.
```

如果你想配置某个 Ceph 网关客户端，可以用点（.）分隔的守护进程和例程来指定，例如：

```
[client.radosgw.instance-name]
```

```
# settings affect client.radosgw.instance-name only.
```

元变量

元变量大大简化了集群配置。Ceph 会把配置的元变量展开为具体值；元变量功能很强大，可以用在配置文件的 `[global]`、`[osd]`、`[mon]`、`[mds]` 段里，类似于 Bash 的 shell 扩展。

Ceph 支持下列元变量：

`$cluster`

描述： 展开为存储集群名字，在同一套硬件上运行多个集群时有用。

实例： `/etc/ceph/$cluster.keyring`

默认值： `ceph`

`$type`

描述： 可展开为 `mds`、`osd`、`mon` 中的一个，有赖于当前守护进程的类型。

实例： `/var/lib/ceph/$type`

`$id`

描述： 展开为守护进程标识符；`osd.0` 应为 `0`，`mds.a` 是 `a`。

实例: `/var/lib/ceph/$type/$cluster-$id`

`$host`

描述: 展开为当前守护进程的主机名。

`$name`

描述: 展开为 `$type.$id`。

实例: `/var/run/ceph/$cluster-$name.asok`

共有选项

硬件推荐段提供了一些配置 Ceph 存储集群的硬件指导。一个 **Ceph 节点** 可以运行多个进程，例如一个节点有多个硬盘，可以为每个硬盘配置一个 `ceph-osd` 守护进程。理想情况下一台主机应该只运行一类进程，例如：一台主机运行着 `ceph-osd` 进程，另一台主机运行着 `ceph-mds` 进程，`ceph-mon` 进程又在另外一台主机上。

各节点都用 `host` 选项指定主机名字，监视器还需要用 `addr` 选项指定网络地址和端口（即域名或 IP 地址）。基本配置文件可以只指定最小配置。例如：

```
[global]

mon_initial_members = ceph1

mon_host = 10.0.0.1
```

Important

`host` 选项是此节点的短名字，不是全资域名（ FQDN ），也不是 IP 地址；执行 `hostname -s` 即可得到短名字。不要给初始监视器之外的例程设置 `host` ，除非你想手动部署；**一定不能用于 `chef` 或 `ceph-deploy`**，这些工具会自动获取正确结果。

网络

关于 Ceph 网络配置的讨论见[网络配置参考](#)。

监视器

典型的 Ceph 生产集群至少部署 3 个 **监视器**来确保高可靠性，它允许一个监视器例程崩溃。奇数个监视器（ 3 个）确保 PAXOS 算法能确定一批监视器里哪个版本的**集群运行图**是最新的。

Note

一个 Ceph 集群可以只有一个监视器，但是如果它失败了，因没有监视器数据服务就会中断。

Ceph 监视器默认监听 6789 端口，例如：

```
[mon.a]

host = hostName

mon addr = 150.140.130.120:6789
```

默认情况下，Ceph 会在下面的路径存储监视器数据：

```
/var/lib/ceph/mon/$cluster-$id
```

你必须手动或通过部署工具（如 `ceph-deploy`）创建对应目录。前述元变量必须先全部展开，名为“ceph”的集群将展开为：

```
/var/lib/ceph/mon/ceph-a
```

其它细节见[监视器配置参考](#)。

认证

New in version Bobtail: 0.56

对于 v0.56 及后来版本，要在配置文件的 `[global]` 中明确启用或禁用认证。

```
auth cluster required = cephx

auth service required = cephx

auth client required = cephx
```

另外，你应该启用消息签名，详情见 [Cephx 配置参考](#)和 [Cephx 认证](#)。

Important

我们建议，升级时先明确地关闭认证，再进行升级。等升级完成后再重新启用认证。

OSDS

通常，Ceph 生产集群在一个节点上只运行一个 **Ceph OSD 守护进程**，此守护进程在一个存储驱动器上只运行一个 filestore；典型部署需指定日志尺寸。例如：

```
[osd]

osd journal size = 10000


[osd.0]

host = {hostname} #manual deployments only.
```

默认情况下，Ceph 认为你把 OSD 数据存储到了以下路径：


```
/var/lib/ceph/osd/$cluster-$id
```

你必须手动或通过部署工具（如 `ceph-deploy`）创建对应目录，名为“ceph”的集群其元变量完全展开后，前述的目录将是：

```
/var/lib/ceph/osd/ceph-0
```

你可以用 `osd data` 选项更改默认值，但我们不建议修改。用下面的命令在新 OSD 主机上创建默认目录：

```
ssh {osd-host}

sudo mkdir /var/lib/ceph/osd/ceph-{osd-number}
```

`osd data` 路径应该指向一个独立硬盘的挂载点，这个硬盘应该独立于操作系统和守护进程所在硬盘。按下列步骤准备好并挂载：

```
ssh {new-osd-host}

sudo mkfs -t {fstype} /dev/{disk}

sudo mount -o user_xattr /dev/{hdd}
/var/lib/ceph/osd/ceph-{osd-number}
```

我们推荐用 `xfs` 或 `btrfs` 文件系统，命令是 `:command:mkfs`。

配置详细步骤见 [OSD 配置参考](#)。

心跳

在运行时，OSD 守护进程会相互检查邻居 OSD、并把结果报告给 Ceph 监视器，一般不需要更改默认配置。但如果你的网络延时比较大，也许需要更改某些选项。

其它细节部分见[监视器与 OSD 交互的配置](#)。

日志、调试

有时候你可能遇到一些麻烦，需要修改日志或调试选项，请参考[调试和日志记录](#)。

CEPH.CONF 实例

[global]

```
fsid = {cluster-id}

mon initial members = {hostname}[, {hostname}]

mon host = {ip-address}[, {ip-address}]

#All clusters have a front-side public network.

#If you have two NICs, you can configure a back side
cluster

#network for OSD object replication, heart beats,
backfilling,

#recovery, etc.

public network = {network}[, {network}]

#cluster network = {network}[, {network}]

#Clusters require authentication by default.

auth cluster required = cephx

auth service required = cephx

auth client required = cephx
```

```
#Choose reasonable numbers for your journals, number of  
replicas  
  
#and placement groups.  
  
osd journal size = {n}  
  
osd pool default size = {n}  # Write an object n times.  
  
osd pool default min size = {n} # Allow writing n copy in  
a degraded state.  
  
osd pool default pg num = {n}  
  
osd pool default pgp num = {n}  
  
#Choose a reasonable crush leaf type.  
  
#0 for a 1-node cluster.  
  
#1 for a multi node cluster in a single rack  
  
#2 for a multi node, multi chassis cluster with multiple  
hosts in a chassis  
  
#3 for a multi node cluster with hosts across racks, etc.  
  
osd crush chooseleaf type = {n}
```

运行时更改

Ceph 可以在运行时更改 `ceph-osd`、`ceph-mon`、`ceph-mds` 守护进程的配置，此功能在增加/降低日志输出、启用/禁用调试设置、甚至是运行时优化的时候非常有用，下面是运行时配置方法：

```
ceph tell {daemon-type}.{id or *} injectargs --{name}
{value} [--{name} {value}]
```

用 `osd`、`mon`、`mds` 中的一个替代 `{daemon-type}`，你可以用星号（`*`）更改一类进程的所有例程配置、或者更改某一具体进程 ID（即数字或字母）的配置。例如提高名为 `osd.0` 的 `ceph-osd` 进程之调试级别的命令如下：

```
ceph tell osd.0 injectargs --debug-osd 20 --debug-ms 1
```

在 `ceph.conf` 文件里配置时用空格分隔关键词；但在命令行使用的时候要用下划线或连字符（`_` 或 `-`）分隔，例如 `debug osd` 变成 `debug-osd`。

[查看运行时配置](#)

如果你的 Ceph 存储集群在运行，而你想看一个在运行进程的配置，用下面的命令：

```
ceph daemon {daemon-type}.{id} config show | less
```

如果你现在位于 `osd.0` 所在的主机，命令将是：

```
ceph daemon osd.0 config show | less
```

[运行多个集群](#)

用 Ceph 可以实现在同一套硬件上运行多个集群，运行多个集群与同一集群内用 CRUSH 规则控制多个存储池相比提供了更高水平的隔离。独立的集群需配备独立的监视器、OSD 和元数据服务器进程。默认配置下集群名字是“ceph”，这意味着你得把配置文件保存为 `/etc/ceph` 下的 `ceph.conf`。

详情见 [ceph-deploy new](#)。.. `_ceph-deploy new: ../ceph-deploy-new`

运行多个集群时，你必须为集群命名并用这个名字保存配置文件。例如，名为 `openstack` 的集群其配置文件将是 `/etc/ceph` 下的 `openstack.conf`。

Important

集群名字里只能包含字母 a-z 和数字 0-9 。

独立的集群意味着独立数据盘和日志，它们不能在集群间共享。根据前面的[元变量](#)，`$cluster` 元变量对应集群名字（就是前面的 `openstack` ）。多处设置都用到 `$cluster` 元变量，包括：

- `keyring`
- `admin socket`
- `log file`
- `pid file`
- `mon data`
- `mon cluster log file`
- `osd data`
- `osd journal`
- `mds data`
- `rgw data`

和 `$cluster` 元变量相关的默认路径见[常规选项](#)、[OSD 选项](#)、[监视器选项](#)、[MDS 选项](#)、[RGW 选项](#)和[日志选项](#)。

创建默认目录和文件时，路径里该用集群名的地方要注意，例如：

```
sudo mkdir /var/lib/ceph/osd/openstack-0

sudo mkdir /var/lib/ceph/mon/openstack-a
```

Important

在一台主机上运行多个监视器时，你得指定不同端口。监视器默认使用 6789 端口，如果它已经被占，其它集群得指定其它端口。

要调动一个名字不是 `ceph` 的集群，要给 `ceph` 命令加 `-c {filename}.conf` 选项，例如：

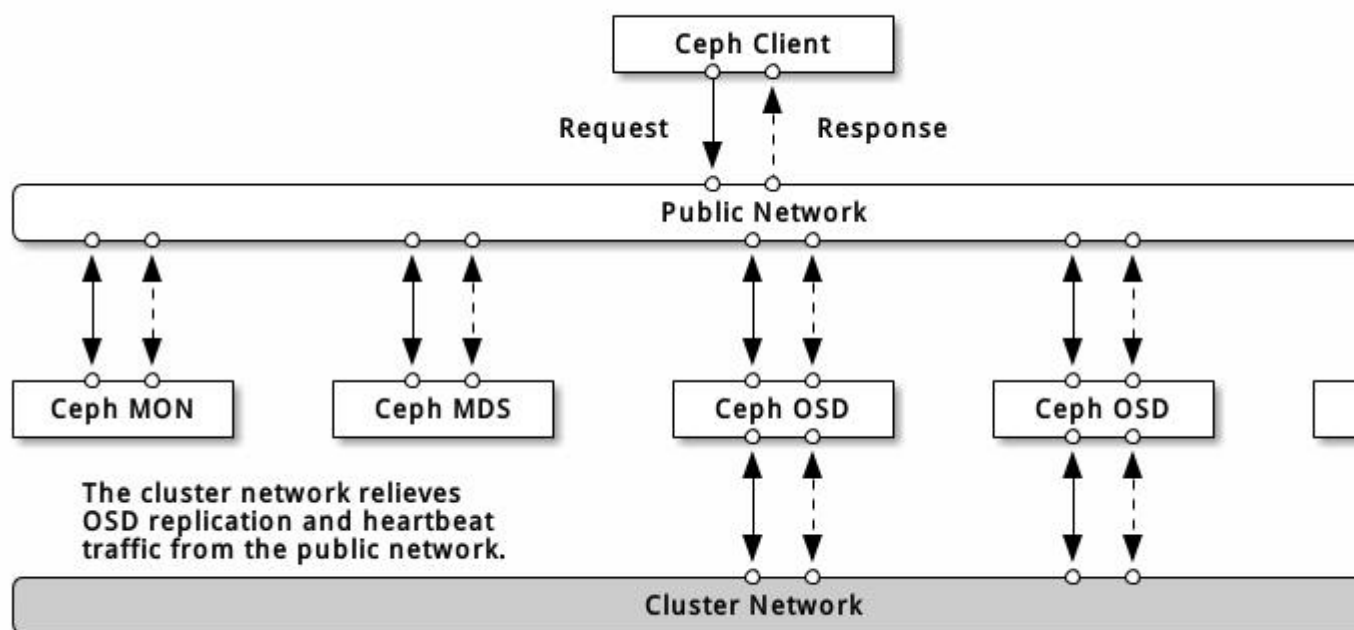
```
ceph -c {cluster-name}.conf health  
  
ceph -c openstack.conf health
```

网络配置参考

网络配置对构建高性能 **Ceph 存储集群**来说相当重要。Ceph 存储集群不会代表 **Ceph 客户端** 执行请求路由或调度，相反，Ceph 客户端（如块设备、CephFS、REST 网关）直接向 OSD 请求，然后 OSD 为客户端执行数据复制，也就是说复制和其它因素会额外增加集群网的负载。

我们的快速入门配置提供了一个简陋的 **Ceph 配置文件**，其中只设置了监视器 IP 地址和守护进程所在的主机名。如果没有配置集群网，那么 Ceph 假设你只有一个“公共网”。只用一个网可以运行 Ceph，但是在大型集群里用单独的“集群”网可显著地提升性能。

我们建议用两个网络运营 Ceph 存储集群：一个公共网（前端）和一个集群网（后端）。为此，各**节点**得配备多个网卡，见 **硬件推荐——网络**。



运营两个独立网络的考量主要有：

1. **性能：** OSD 为客户端处理数据复制，复制多份时 OSD 间的网络负载势必会影响到客户端和 Ceph 集群的通讯，包括延时增加、产生性能问题；恢复和重均衡也会显著增加公共网延时。关于 Ceph 如何复制参见[伸缩性和高可用性](#)；关于心跳流量参见[监视器与 OSD 的交互](#)。
2. **安全：** 大多数人都是良民，很少的一撮人喜欢折腾拒绝服务攻击（DoS）。当 OSD 间的流量失控时，归置组再也不能达到 `active + clean` 状态，这样用户就不能读写数据了。挫败此类攻击的一种好方法是维护一个完全独立的集群网，使之不能直连互联网；另外，请考虑用[消息签名](#)防止欺骗攻击。

防火墙

守护进程默认会[绑定](#)到 `6800:7300` 间的端口，你可以更改此范围。更改防火墙配置前先检查下 `iptables` 配置。

```
sudo iptables -L
```

一些 Linux 发行版的规则拒绝除 SSH 之外的所有网卡的所有入栈连接，例如：

```
REJECT all -- anywhere anywhere reject-with icmp-host-prohibited
```

你得先删掉公共网和集群网对应的这些规则，然后再增加安全保护规则。

监视器防火墙

监视器默认监听 6789 端口，而且监视器总是运行在公共网。按下例增加规则时，要把 {iface} 替换为公共网接口（如 eth0、eth1 等等）、{ip-address} 替换为公共网 IP、{netmask} 替换为公共网掩码。

```
sudo iptables -A INPUT -i {iface} -p tcp -s {ip-address}/{netmask} --dport 6789 -j ACCEPT
```

MDS 防火墙

元数据服务器会监听公共网 6800 以上的第一个可用端口。需要注意的是，这种行为是不确定的，所以如果你在同一主机上运行多个 OSD 或 MDS、或者在很短的时间内重启了多个守护进程，它们会绑定更高的端口号；所以说你应该预先打开整个 6800-7300 端口区间。按下例增加规则时，要把 {iface} 替换为公共网接口（如 eth0、eth1 等等）、{ip-address} 替换为公共网 IP、{netmask} 替换为公共网掩码。

例如：

```
sudo iptables -A INPUT -i {iface} -m multiport -p tcp -s {ip-address}/{netmask} --dports 6800:7300 -j ACCEPT
```

OSD 防火墙

OSD 守护进程默认**绑定**从 6800 起的第一个可用端口，需要注意的是，这种行为是不确定的，所以如果你在同一主机上运行多个 OSD 或 MDS、或者在很短的时间内重启了多个守护进程，它们会绑定更高的端口号。一主机上的各个 OSD 最多会用到 4 个端口：

1. 一个用于和客户端、监视器通讯；
2. 一个用于发送数据到其他 OSD；
3. 两个用于各个网卡上的心跳；



当某个守护进程失败并重启时没释放端口，重启后的进程就会监听新端口。你应该打开整个 6800-7300 端口区间，以应对这种可能性。

如果你分开了公共网和集群网，必须分别为之设置防火墙，因为客户端会通过公共网连接、而其他 OSD 会通过集群网连接。按下例增加规则时，要把 `{iface}` 替换为网口（如 `eth0`、`eth1` 等等）、`{ip-address}` 替换为公共网或集群网 IP、`{netmask}` 替换为公共网或集群网掩码。例如：

```
sudo iptables -A INPUT -i {iface} -m multiport -p tcp -s {ip-address}/{netmask} --dports 6800:7300 -j ACCEPT
```

Tip

如果你的元数据服务器和 OSD 在同一节点上，可以合并公共网配置。

CEPH 网络

Ceph 的网络配置要放到 `[global]` 段下。前述的 5 分钟快速入门提供了一个简陋的 [Ceph 配置文件](#)，它假设服务器和客户端都位于同一网段，Ceph 可以很好地适应这种情形。然而 Ceph 允许配置更精细的公共网，包括多 IP 和多掩码；也能用单独的集群网处理 OSD 心跳、对象复制、和恢复流量。不要混淆你配置的 IP 地址和客户端用来访问存储服务的公共网地址。典型的内网常常是 `192.168.0.0` 或 `10.0.0.0`。

Tip

如果你给公共网或集群网配置了多个 IP 地址及子网掩码，这些子网必须能互通。另外要确保在防火墙上为各 IP 和子网开放了必要的端口。

Note

Ceph 用 CIDR 法表示子网，如 10.0.0.0/24。

配置完几个网络后，可以重启集群或挨个重启守护进程。Ceph 守护进程动态地绑定端口，所以更改网络配置后无需重启整个集群。

公共网

要配置一个公共网，把下列选项加到配置文件的 `[global]` 段下。

```
[global]

...

public network = {public-network/netmask}
```

集群网

如果你声明了集群网，OSD 将把心跳、对象复制和恢复流量路由到集群网，与单个网络相比这会提升性能。要配置集群网，把下列选项加进配置文件的 `[global]` 段。

```
[global]

...

cluster network = {cluster-network/netmask}
```

为安全起见，从公共网或互联网到集群网应该是**不可达**的。

CEPH 守护进程

有一个网络配置是所有守护进程都要配的：各个守护进程都**必须**指定 `host`，Ceph 也要求指定监视器 IP 地址及端口。

Important

一些部署工具（如 `ceph-deploy`、Chef）会给你创建配置文件，如果它能胜任那就**别**设置这些值。

Tip

`host` 选项是主机的短名，不是全资域名 FQDN，也**不是** IP 地址。在命令行下输入 `hostname -s` 获取主机名。

```
[mon.a]
```

```
host = {hostname}
```

```
mon addr = {ip-address}:6789
```

```
[osd.0]
```

```
host = {hostname}
```

并非一定要给守护进程设置 IP 地址。如果你有一个静态配置，且分离了公共网和集群网，Ceph 允许你在配置文件里指定主机的 IP 地址。要给守护进程设置静态 IP，可把下列选项加到 `ceph.conf`。

```
[osd.0]
```

```
public addr = {host-public-ip-address}

cluster addr = {host-cluster-ip-address}
```

单网卡 OSD、双网络集群

一般来说，我们不建议用单网卡 OSD 主机部署两个网络。然而这事可以实现，把 `public addr` 选项配在 `[osd.n]` 段下即可强制 OSD 主机运行在公共网，其中 `n` 是其 OSD 号。另外，公共网和集群网必须互通，考虑到安全因素我们不建议这样做。

网络配置选项

网络配置选项不是必需的，Ceph 假设所有主机都运行于公共网，除非你特意配置了一个集群网。

公共网

公共网配置用于明确地为公共网定义 IP 地址和子网。你可以分配静态 IP 或用 `public addr` 覆盖 `public network` 选项。

`public network`

描述: 公共网（前端）的 IP 地址和掩码（如 `192.168.0.0/24`），置于 `[global]` 下。多个子网用逗号分隔。

类型: `{ip-address}/{netmask} [, {ip-address}/{netmask}]`

是否必需: No

默认值: N/A

`public addr`

描述: 用于公共网（前端）的 IP 地址。适用于各守护进程。

类型: IP 地址

是否必需: No

默认值: N/A

集群网

集群网配置用来声明一个集群网，并明确地定义其 IP 地址和子网。你可以配置静态 IP 或为某 OSD 守护进程配置 `cluster addr` 以覆盖 `clusternetwork` 选项。

`cluster network`

描述: 集群网（后端）的 IP 地址及掩码（如 `10.0.0.0/24`），置于 `[global]` 下。多个子网用逗号分隔。

类型: `{ip-address}/{netmask} [, {ip-address}/{netmask}]`

是否必需: No

默认值: N/A

`cluster addr`

描述: 集群网（后端） IP 地址。置于各守护进程下。

类型: Address

是否必需: No

默认值: N/A

绑定

绑定选项用于设置 OSD 和 MDS 默认使用的端口范围，默认范围是 6800:7300。确保[防火墙](#)开放了对应端口范围。

你也可以让 Ceph 守护进程绑定到 IPv6 地址。

```
ms bind port min
```

描述: OSD 或 MDS 可绑定的最小端口号。

类型: 32-bit Integer

默认值: 6800

是否必需: No

```
ms bind port max
```

描述: OSD 或 MDS 可绑定的最大端口号。

类型: 32-bit Integer

默认值: 7300

是否必需: No.

```
ms bind ipv6
```

描述: 允许 Ceph 守护进程绑定 IPv6 地址。

类型: Boolean

默认值: false

是否必需: No

主机

Ceph 配置文件里至少要写一个监视器、且每个监视器下都要配置 `mon addr` 选项；每个监视器、元数据服务器和 OSD 下都要配 `host` 选项。

```
mon addr
```

描述: `{hostname}:{port}` 条目列表，用以让客户端连接 Ceph 监视器。如果

未设置，Ceph 查找 `[mon.*]` 段。

类型: String

是否必需: No

默认值: N/A

`host`

描述: 主机名。此选项用于特定守护进程，如 `[osd.0]`。

类型: String

是否必需: Yes, for daemon instances.

默认值: `localhost`

Tip

不要用 `localhost`。在命令行下执行 `hostname -s` 获取主机名（到第一个点，不是全资域名），并用于配置文件。

Important

用第三方部署工具时不要指定 `host` 选项，它会自行获取。

TCP

Ceph 默认禁用 TCP 缓冲。

`ms tcp nodelay`

Ceph 用 `ms tcp nodelay` 使系统尽快（不缓冲）发送每个请求。禁用 `ms tcp nodelay` 可增加吞吐量，但会引进延时。如果你遇到大量小包，可以禁用 `ms tcp nodelay` 试试。

类型: Boolean

是否必需: No

默认值: `true`

`ms tcp rcvbuf`

描述: 网络套接字接收缓冲尺寸，默认禁用。

类型: 32-bit Integer

是否必需: No

默认值: 0

`ms tcp read timeout`

描述: 如果一客户端或守护进程发送请求到另一个 Ceph 守护进程，且没有断开不再使用的连接，在 `ms tcp read timeout` 指定的秒数之后它将被标记为空闲。

类型: Unsigned 64-bit Integer

是否必需: No

默认值: 900 15 minutes.

CEPHX 配置参考

(认证配置)

`cephx` 协议已默认开启。加密认证要耗费一定计算资源，但通常很低。如果您的客户端和服务网络环境相当安全，而且认证的负面效应更大，你可以关闭它，通常不推荐您这么做。

Note

如果禁用了认证，就会有篡改客户端/服务器消息这样的中间人攻击风险，这会导致灾难性后果。

关于创建用户请参考[用户管理](#)；关于 Cephx 的体系结构请参考[体系结构——高可用性认证](#)。

部署场景

There are two main scenarios for deploying a Ceph cluster, which impact how you initially configure Cephx. Most first time Ceph users use `ceph-deploy` to create a cluster (easiest). For clusters using other deployment tools (e.g., Chef, Juju, Puppet, etc.), you will need to use the manual procedures or configure your deployment tool to bootstrap your monitor(s).

CEPH-DEPLOY

When you deploy a cluster with `ceph-deploy`, you do not have to bootstrap the monitor manually or create the `client.admin` user or keyring. The steps you execute in the [存储集群入门](#) will invoke `ceph-deploy` to do that for you.

When you execute `ceph-deploy new {initial-monitor(s)}`, Ceph will create a monitor keyring for you (only used to bootstrap monitors), and it will generate an initial Ceph configuration file for you, which contains the following authentication settings, indicating that Ceph enables authentication by default:

```
auth_cluster_required = cephx
auth_service_required = cephx
auth_client_required = cephx
```

When you execute `ceph-deploy mon create-initial`, Ceph will bootstrap the initial monitor(s), retrieve a `ceph.client.admin.keyring`file containing the key for the `client.admin` user. Additionally, it will also retrieve keyrings that give `ceph-deploy` and `ceph-disk` utilities the ability to prepare and activate OSDs and metadata servers.

When you execute `ceph-deploy admin {node-name}` (**note:** Ceph must be installed first), you are pushing a Ceph configuration file and the `ceph.client.admin.keyring` to the `/etc/ceph` directory of the node. You will be able to execute Ceph administrative functions as `root` on the command line of that node.

手动部署

When you deploy a cluster manually, you have to bootstrap the monitor manually and create the `client.admin` user and keyring. To bootstrap monitors, follow the steps in [监视器的自举引导](#). The steps for monitor bootstrapping are the logical steps you must perform when using third party deployment tools like Chef, Puppet, Juju, etc.

启用和禁用 CEPHX

Enabling Cephx requires that you have deployed keys for your monitors, OSDs and metadata servers. If you are simply toggling Cephx on / off, you do not have to repeat the bootstrapping procedures.

启用 CEPHX

启用 `cephx` 后，Ceph 将在默认搜索路径（包括 `/etc/ceph/ceph.$name.keyring`）里查找密钥环。你可以在 [Ceph 配置](#) 文件的 `[global]` 段里添加 `keyring` 选项来修改，但不推荐。

在禁用了 `cephx` 的集群上执行下面的步骤来启用它，如果你（或者部署工具）已经生成了密钥，你可以跳过相关步骤。

1. 创建 `client.admin` 密钥，并为客户端保存此密钥的副本：

```
ceph auth get-or-create client.admin mon 'allow *'  
mds 'allow *' osd 'allow *' -o  
/etc/ceph/ceph.client.admin.keyring
```

警告： 此命令会覆盖任何存在的 `/etc/ceph/client.admin.keyring` 文件，如果部署工具已经完成此步骤，千万别再执行此命令。多加小心！

2. 创建监视器集群所需的密钥环、并给它们生成密钥。

```
ceph-authtool --create-keyring /tmp/ceph.mon.keyring  
--gen-key -n mon. --cap mon 'allow *'
```

3. 把监视器密钥环复制到 `ceph.mon.keyring` 文件，再把此文件复制到各监视器的 `mon data` 目录下。比如要把它复制给名为 `ceph` 集群的 `mon.a`，用此命令：

```
4. cp /tmp/ceph.mon.keyring /var/lib/ceph/mon/ceph-a/keyring
```

5. 为每个 OSD 生成密钥，`{ $id }` 是 OSD 编号：

```
ceph auth get-or-create osd.{ $id } mon 'allow rwx'
osd 'allow *' -o /var/lib/ceph/osd/ceph-{ $id }/keyring
```

6. 为每个 MDS 生成密钥，`{ $id }` 是 MDS 的标识字母：

```
ceph auth get-or-create mds.{ $id } mon 'allow rwx'
osd 'allow *' mds 'allow *' -o
/var/lib/ceph/mds/ceph-{ $id }/keyring
```

7. 把以下配置加入 [Ceph 配置文件](#)的 `[global]` 段下以启用 `cephx` 认证：

```
8. auth cluster required = cephx

9. auth service required = cephx
```

```
auth client required = cephx
```

10. 启动或重启 Ceph 集群，详情见[操纵集群](#)。

要手动自启监视器，请参考[手动部署](#)。

禁用 CEPHX

下述步骤描述了如何禁用 Cephx 。如果你的集群环境相对安全，你可以减免认证耗费的计算资源，然而我们不推荐。但是临时禁用认证会使安装、和/或排障更简单。

- 1. 把下列配置加入 [Ceph 配置文件](#) 的 `[global]` 段下即可禁用 `cephx` 认证：

```
2. auth cluster required = none

3. auth service required = none
```

```
auth client required = none
```

- 4. 启动或重启 Ceph 集群，具体参考[操纵集群](#)。

配置选项

启用事项

`auth cluster required`

描述: 如果启用了，集群守护进程（如 `ceph-mon`、`ceph-osd` 和 `ceph-mds`）间必须相互认证。可用选项有 `cephx` 或 `none`。

类型: String

是否必需: No

默认 `cephx.`

值:

`auth service required`

描述: 如果启用，客户端要访问 Ceph 服务的话，集群守护进程会要求它和集群认证。可用选项为 `cephx` 或 `none`。

类型: String

是否必需: No

默认值: `cephx`.

`auth client required`

描述: 如果启用，客户端会要求 Ceph 集群和它认证。可用选项为 `cephx` 或 `none`。

类型: String

是否必需: No

默认值: `cephx`.

密钥

如果你的集群启用了认证，`ceph` 管理命令和客户端得有密钥才能访问集群。

给 `ceph` 管理命令和客户端提供密钥的最常用方法就是把密钥环放到 `/etc/ceph`，通过 `ceph-deploy` 部署的 Cuttlefish 及更高版本，其文件名通常是 `ceph.client.admin.keyring`（或 `$cluster.client.admin.keyring`）。如果你的密钥环位于 `/etc/ceph` 下，就不需要在 Ceph 配置文件里指定 `keyring` 选项了。

我们建议把集群的密钥环复制到你执行管理命令的节点，它包含 `client.admin` 密钥。

你可以用 `ceph-deploy admin` 命令做此事，详情见‘部署管理主机’，手动复制可执行此命令：

```
sudo scp {user}@{ceph-cluster-host}:/etc/ceph/ceph.client.admin.keyring
/etc/ceph/ceph.client.admin.keyring
```

Tip

确保给客户端上的 `ceph.keyring` 设置合理的权限位（如 `chmod 644`）。

你可以用 `key` 选项把密钥写在配置文件里（别这样），或者用 `keyfile` 选项指定个路径。

`keyring`

描

述 密钥环文件的路径。

:

类

型 String

:

是否必需:
No

默认值:
`/etc/ceph/$cluster.$name.keyring, /etc/ceph/$cluster.keyring, /etc/ceph/keyring, /etc/ceph/keyring.bin`

`keyfile`

描述: 到密钥文件的路径，如一个只包含密钥的文件。

类型: String

是否必需: No

默认值: None

`key`

描述: 密钥（密钥文本），最好别这样做。

类型: String

是否必需: No

默认值: None

守护进程密钥环

管理员用户们或部署工具（如 `ceph-deploy`）生成守护进程密钥环与生成用户密钥环的方法一样。默认情况下，守护进程把密钥环保存在各自的数据目录下，默认密钥环位置、和守护进程发挥作用必需的能力展示如下：

`ceph-mon`

位置: `$mon_data/keyring`

能力: `mon 'allow *'`

`ceph-osd`

位置: `$osd_data/keyring`

能力: `mon 'allow profile osd' osd 'allow *'`

`ceph-mds`

位置: `$mds_data/keyring`

能力: `mds 'allow' mon 'allow profile mds' osd 'allow rwx'`

`radosgw`

位置: `$rgw_data/keyring`

能力: `mon 'allow rwx' osd 'allow rwx'`

Note

监视器密钥环（即 `mon.`）包含一个密钥，但没有能力，且不是集群 `auth` 数据库的一部分。

守护进程数据目录位置默认格式如下：

```
/var/lib/ceph/$type/$cluster-$id
```

例如，`osd.12` 的目录会是：

```
/var/lib/ceph/osd/ceph-12
```

你可以覆盖这些位置，但不推荐。

签名

在 Bobtail 及后续版本中，Ceph 会用开始认证时生成的会话密钥认证所有在线实体。然而 Argonaut 及之前版本不知道如何认证在线消息，为保持向后兼容性（如在同一个集群里运行 Bobtail 和 Argonaut），消息签名默认是关闭的。如果你只运行 Bobtail 和后续版本，可以让 Ceph 要求签名。

像 Ceph 认证的其他部分一样，客户端和集群间的消息签名也能做到细粒度控制；而且能启用或禁用 Ceph 守护进程间的签名。

`cephx require signatures`

描述: 若设置为 `true`，Ceph 集群会要求客户端签名所有消息，包括集群内其他守护进程间的。

类型: Boolean

是否必需: No

默认值: `false`

`cephx cluster require signatures`

描述: 若设置为 `true`，Ceph 要求集群内所有守护进程签名相互之间的消息。

类型: Boolean

是否必需: No

默认值: `false`

`cephx service require signatures`

描述: 若设置为 `true`，Ceph 要求签名所有客户端和集群间的消息。

类型: Boolean

是否必需: No

默认值: `false`

`cephx sign messages`

描述: 如果 Ceph 版本支持消息签名，Ceph 会签名所有消息以防欺骗。

类型: Boolean

默认值: `true`

生存期

`auth service ticket ttl`

描述: Ceph 存储集群发给客户端一个用于认证的票据时分配给这个票据的生存期。

类型: Double

默认
值:

60*60

向后兼容性

关于 Cuttlefish 及更低版本，参考 [Cephx](#)。

在 Ceph 0.48 及更早版本，启用 `cephx` 认证后，Ceph 仅认证客户端和守护进程间的最初通讯，不会认证后续相互发送的消息，这导致了安全隐患；Bobtail 及后续版本会用认证后生成的会话密钥来认证所有消息。

我们确定了一个向后兼容性问题，在 Argonaut v0.48（及之前版本）和 Bobtail（及后续版本）之间。测试发现，如果你混用 Argonaut（及更小版）和 Bobtail 的守护进程，Argonaut 的守护进程将对正在进行的消息不知所措，因为 Bobtail 进程坚持要求认证最初请求/响应之后的消息，导致二者无法交互。

我们已经提供了一种方法，解决了 Argonaut（及之前）和 Bobtail（及后续）系统间交互的潜在的问题。是这样解决的，默认情况下，较新系统不会再坚持要求较老系统的签名，只是简单地接收这些消息而不对其认证。这个默认行为使得两个不同版本可以交互，但我们不推荐作为长期方案。允许较新进程不认证正在进行的消息会导致安全问题，因为攻击者如果能控制你的机器、或访问你的网络，就可以宣称不能签署消息，从而禁用会话安全。

Note

即使你没有使用旧版的 Ceph，在默认配置下，攻击者也可以强制一些未签署消息被接受；虽然初始通讯认证通过了，但你失去了会话安全。

如果你确定不会使用旧版 Ceph、或者新旧服务器不能交互无所谓，那就可以排除这个安全风险；如果你这样做了，任何支持会话认证、启用了 Cephx 的 Ceph 系统都会拒绝未签名的消息。要防止新服务器和旧服务器交互，在 [Ceph 配置](#) 文件的 `[global]` 下添加下列这行，要加到启用 Cephx 之后。

```
cephx require signatures = true ; everywhere possible
```

你也可以选择只对集群内部通讯要求签名，它和面向客户端的服务是分离的：

```
cephx cluster require signatures = true      ; for cluster-internal communication

cephx service require signatures = true      ; for client-facing service
```

客户端向集群要求签名的选项还没实现。

我们推荐把所有进程迁移到较新版本，并启用前述选项选项，以增强认证安全性。

Note

Ceph 内核模块还不支持签名。

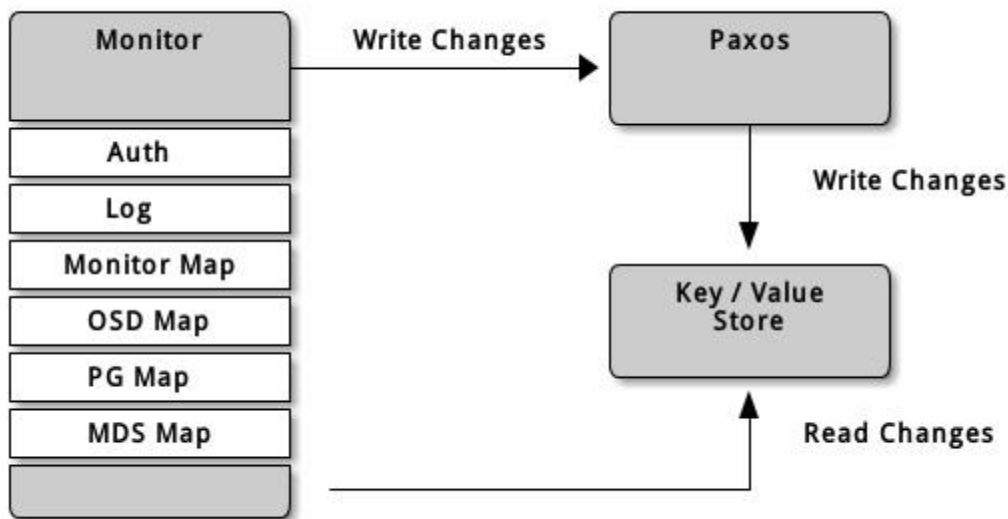
监视器配置参考

理解如何配置 [Ceph 监视器](#)是构建可靠的 [Ceph 存储集群](#)的重要方面，任何 **Ceph** 集群都需要至少一个监视器。一个监视器通常相当一致，但是你可以增加、删除、或替换集群中的监视器，详情见[增加/删除监视器](#)和[增加/删除监视器 \(ceph-deploy \)](#)。

背景

监视器们维护着集群运行图的“主副本”，就是说客户端连到一个监视器并获取当前运行图就能确定所有监视器、OSD 和元数据服务器的位置。Ceph 客户端读写 OSD 或元数据服务器前，必须先连到一个监视器，靠当前集群运行图的副本和 CRUSH 算法，客户端能计算出任何对象的位置，故此客户端有能力直接连到 OSD，这对 Ceph 的高伸缩性、高性能来说非常重要。更多信息见[伸缩性和高可用性](#)。

监视器的主要角色是维护集群运行图的主副本，它也提供认证和日志记录服务。Ceph 监视器们把监视器服务的所有更改写入一个单独的 Paxos 例程，然后 Paxos 以键/值方式存储所有变更以实现高度一致性。同步期间，Ceph 监视器能查询集群运行图的近期版本，它们通过操作键/值存储快照和迭代器（用 leveldb）来进行存储级同步。



Deprecated since version 0.58: 版

在 0.58 及更早版本中，Ceph 监视器每个服务用一个 Paxos 例程，并把运行图存储为文件。

集群运行图

集群运行图是多个图的组合，包括监视器图、OSD 图、归置组图和元数据服务器图。集群运行图追踪几个重要事件：哪些进程在集群里（`in`）；哪些进程在集群里（`in`）是 `up` 且在运行、或 `down`；归置组状态是 `active` 或 `inactive`、`clean` 或其他状态；和其他反映当前集群状态的信息，像总存储容量、和使用量。

当集群状态有明显变更时，如一 OSD 挂了、一归置组降级了等等，集群运行图会被更新以反映集群当前状态。另外，监视器也维护着集群的主要状态历史。监视器图、OSD 图、归置组图和元数据服务器图各自维护着它们的运行图版本。我们把各图的版本称为一个 `epoch`。

运营集群时，跟踪这些状态是系统管理任务的重要部分。详情见[监控集群](#)和[监控 OSD 和归置组](#)。

监视器法定人数

本文入门部分提供了一个简陋的 [Ceph 配置文件](#)，它提供了一个监视器用于测试。只用一个监视器集群可以良好地运行，然而单监视器是一个单故障点，生产集群要实现高可用性的话得配置多个监视器，这样单个监视器的失效才不会影响整个集群。

集群用多个监视器实现高可用性时，多个监视器用 [Paxos](#) 算法对主集群运行图达成一致，这里的一致要求大多数监视器都在运行且够成法定人数（如 1 个、3 之 2 在运行、5 之 3、6 之 4 等等）。

一致性

你把监视器加进 Ceph 配置文件时，得注意一些架构问题，Ceph 发现集群内的其他监视器时对其有着严格的一致性要求。尽管如此，Ceph 客户端和其他 Ceph 守护进程用配置文件发现监视器，监视器却用监视器图（monmap）相互发现而非配置文件。

一个监视器发现集群内的其他监视器时总是参考 monmap 的本地副本，用 monmap 而非 Ceph 配置文件避免了可能损坏集群的错误（如 `ceph.conf` 中指定地址或端口的拼写错误）。正因为监视器把 monmap 用于发现、并共享于客户端和其他 Ceph 守护进程间，**monmap** 可严格地保证监视器的一致性是可靠的。

严格的一致性也适用于 monmap 的更新，因为关于监视器的任何更新、关于 monmap 的变更都是通过称为 [Paxos](#) 的分布式一致性算法传递的。监视器们必须就 monmap 的每次更新达成一致，以确保法定人数里的每个监视器 monmap 版本相同，如增加、删除一个监视器。monmap 的更新是增量的，所以监视器们都有最新的一致版本，以及一系列之前版本。历史版本的存在允许一个落后的监视器跟上集群当前状态。

如果监视器通过配置文件而非 monmap 相互发现，这会引进其他风险，因为 Ceph 配置文件不是自动更新并分发的，监视器有可能不小心用了较老的配置文件，以致于不认识某监视器、放弃法定人数、或者产生一种 [Paxos](#) 不能确定当前系统状态的情形。

初始化监视器

在大多数配置和部署案例中，部署 Ceph 的工具可以帮你生成一个监视器图来初始化监视器（如 `ceph-deploy` 等），一个监视器需要 4 个选项：

- **文件系统标识符：** `fsid` 是对象存储的唯一标识符。因为你可以在一套硬件上运行多个集群，所以在初始化监视器时必须指定对象存储的唯一标识符。部署工具通常可替你完成（如 `ceph-deploy` 会调用类似 `uuidgen` 的程序），但是你也可以手动指定 `fsid`。
- **监视器标识符：** 监视器标识符是分配给集群内各监视器的唯一 ID，它是一个字母数字组合，为方便起见，标识符通常以字母顺序结尾（如 `a`、`b` 等等），可以设置于 Ceph 配置文件（如 `[mon.a]`、`[mon.b]` 等等）、部署工具、或 `ceph` 命令行工具。
- **密钥：** 监视器必须有密钥。像 `ceph-deploy` 这样的部署工具通常会自动生成，也可以手动完成。见[监视器密钥环](#)。

关于初始化的具体信息见[初始化监视器](#)。

监视器的配置

要把配置应用到整个集群，把它们放到 `[global]` 下；要用于所有监视器，置于 `[mon]` 下；要用于某监视器，指定监视器例程，如 `[mon.a]` ）。按惯例，监视器例程用字母命名。

```
[global]
```

```
[mon]
```

```
[mon.a]
```

```
[mon.b]
```

```
[mon.c]
```

最小配置

Ceph 监视器的最简配置必须包括一主机名及其监视器地址，这些配置可置于 `[mon]` 下或某个监视器下。

```
[mon]

    mon host = hostname1,hostname2,hostname3

    mon addr =
10.0.0.10:6789,10.0.0.11:6789,10.0.0.12:6789

[mon.a]

    host = hostname1

    mon addr = 10.0.0.10:6789
```

详情见[网络配置参考](#)。

Note

这里的监视器最简配置假设部署工具会自动给你生成 `fsid` 和 `mon.` 密钥。

一旦部署了 Ceph 集群，监视器 IP 地址不应该更改。然而，如果你决意要改，必须严格按照[更改监视器 IP 地址](#)来改。

集群 ID

每个 Ceph 存储集群都有一个唯一标识符 (`fsid`)。如果指定了，它应该出现在配置文件的 `[global]` 段下。部署工具通常会生成 `fsid` 并存于监视器图，所以不一定会写入配置文件，`fsid` 使得在一套硬件上运行多个集群成为可能。

`fsid`

描述: 集群 ID，一集群一个。

类型: UUID

是否必需: Yes.

默认值: 无。若未指定，部署工具会生成。

Note

如果你用部署工具就不能设置。

初始成员

我们建议在生产环境下最少部署 3 个监视器，以确保高可用性。运行多个监视器时，你可以指定为形成法定人数成员所需的初始监视器，这能减小集群上线时间。

```
[mon]
```

```
mon initial members = a,b,c
```

```
mon initial members
```

描述: 集群启动时初始监视器的 ID，若指定，Ceph 需要奇数个监视器来确定最初法定人数（如 3）。

类型: String

默认
值: None

Note

集群内的大多数监视器必须能互通以建立法定人数，你可以用此选项减小初始监视器数量来形成。

数据

Ceph 监视器有存储数据的默认路径，生产集群为实现更高性能可把监视器部署到非 OSD 节点的独立主机上。因为监视器会频繁 `fsync()`，这可能影响 OSD。

在 Ceph 0.58 及更早版本中，监视器数据以文件保存，这样人们可以用 `ls` 和 `cat` 这些普通工具检查监视器数据，然而它不能提供健壮的一致性。

在 Ceph 0.59 及后续版本中，监视器以键/值对存储数据。监视器需要 [ACID](#) 事务，数据存储的使用可防止监视器用损坏的版本进行恢复，除此之外，它允许在一个原子批量操作中进行多个修改操作。

一般来说我们不建议更改默认数据位置，如果要改，我们建议所有监视器统一配置，加到配置文件的 `[mon]` 下。

`mon data`

描述: 监视器的数据位置。

类型: String

默认值: `/var/lib/ceph/mon/$cluster-$id`

存储容量

Ceph 存储集群利用率接近最大容量时（即 `mon osd full ratio`），作为防止数据丢失的安全措施，它会阻止你读写 OSD。因此，让生产集群用满可不是好事，因为牺牲了高可用性。full ratio 默认值是 `.95` 或容量的 95%。对小型测试集群来说这是非常激进的设置。

Tip

监控集群时，要警惕和 `nearfull` 相关的警告。这意味着一些 OSD 的失败会导致临时服务中断，应该增加一些 OSD 来扩展存储容量。

在测试集群时，一个常见场景是：系统管理员从集群删除一个 OSD、接着观察重均衡；然后继续删除其他 OSD，直到集群达到占满率并锁死。我们建议，即使在测试集群里也要规划一点空闲容量用于保证高可用性。理想情况下，要做好这样的预案：一系列 OSD 失败后，短时间内不更换它们仍能恢复到 `active + clean` 状态。你也可以在 `active + degraded` 状态运行集群，但对正常使用来说并不好。

下图描述了一个简化的 Ceph 集群，它包含 33 个节点、每主机一个 OSD、每 OSD 3TB 容量，所以这个小白鼠集群有 99TB 的实际容量，其 `mon osdfull ratio` 为 `.95`。如果它只剩余 5TB 容量，集群就不允许客户端再读写数据，所以它的运行容量是 95TB，而非 99TB。

Rack 1	Rack 2	Rack 3	Rack 4	Rack 5	Rack 6
OSD 1	OSD 7	OSD 13	OSD 19	OSD 25	OSD 31
OSD 2	OSD 8	OSD 14	OSD 20	OSD 26	OSD 32
OSD 3	OSD 9	OSD 15	OSD 21	OSD 27	OSD 33
OSD 4	OSD 10	OSD 16	OSD 22	OSD 28	Spare
OSD 5	OSD 11	OSD 17	OSD 23	OSD 29	Spare
OSD 6	OSD 12	OSD 18	OSD 24	OSD 30	Spare

在这样的集群里，坏一或两个 OSD 很平常；一种罕见但可能发生的情形是一个机架的路由器或电源挂了，这会导致多个 OSD 同时离线（如 OSD 7-12），在这种情况下，你仍要力争保持集群可运行并达到 `active + clean` 状态，即使这意味着你得在短期内额外增加一些 OSD 及主机。如果集群利用率太高，在解决故障域期间也许不会丢数据，但很可能牺牲数据可用性，因为利用率超过了 `full ratio`。故此，我们建议至少要粗略地规划下容量。

找出你集群的两个数字：

1. OSD 数量。
2. 集群总容量。

用集群里 OSD 总数除以集群总容量，就能得到 OSD 平均容量；如果按预计的 OSD 数乘以这个值所得的结果计算（偏小），实际应用时将出错；最后再用集群容量乘以占满率能得到最大运行容量，然后扣除预估的 OSD 失败率；用较高的失败率（如整机架的 OSD）重复前述过程看是否接近占满率。

```
[global]
```

```
mon osd full ratio = .80
```

```
mon osd nearfull ratio = .70
```

```
mon osd full ratio
```

描述: OSD 硬盘使用率达到多少就认为它 `full`。

类型: Float

默认值: `.95`

```
mon osd nearfull ratio
```

描述: OSD 硬盘使用率达到多少就认为它 `nearfull`。

类型: Float

默认值: `.85`

Tip

如果一些 OSD 快满了，但其他的仍有足够空间，你可能配错 CRUSH 权重了。

心跳

Ceph 监视器要求各 OSD 向它报告、并接收 OSD 们的邻居状态报告，以此来掌握集群。

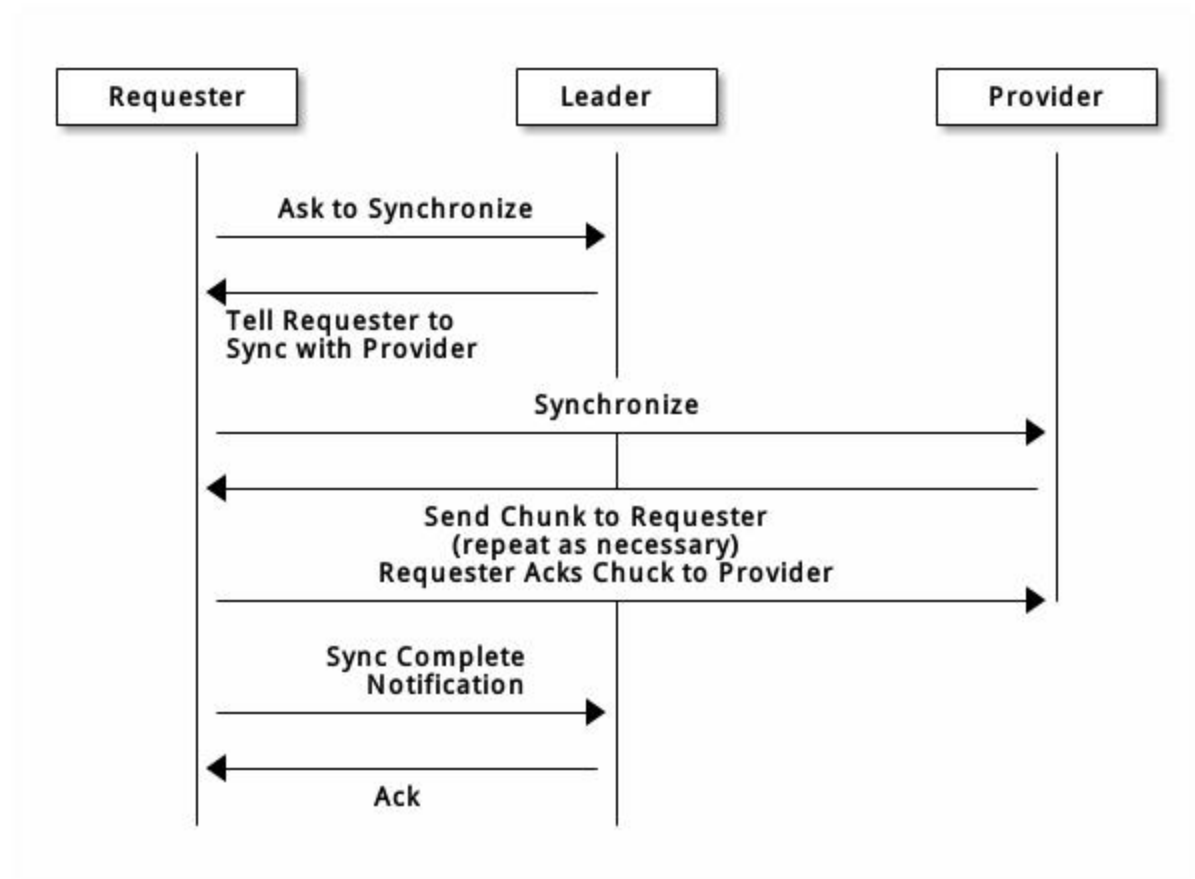
Ceph 提供了监视器与 OSD 交互的合理默认值，然而你可以按需修改，详情见[监视器与 OSD 的交互](#)。

监视器存储同步

当你用多个监视器支撑一个生产集群时，各监视器都要检查邻居是否有集群运行图的最新版本（如，邻居监视器的图有一或多个 epoch 版本高于当前监视器的最高版 epoch），过一段时间，集群里的某个监视器可能落后于其它监视器太多而不得不离开法定人数，然后同步到集群当前状态，并重回法定人数。为了同步，监视器可能承担三种中的一种角色：

1. **Leader:** *Leader* 是实现最新 Paxos 版本的第一个监视器。
2. **Provider:** *Provider* 有最新集群运行图的监视器，但不是第一个实现最新版。
3. **Requester:** *Requester* 落后于 leader，重回法定人数前，必须同步以获取关于集群的最新信息。

有了这些角色区分，leader 就可以给 provider 委派同步任务，这会避免同步请求压垮 leader、影响性能。在下面的图示中，requester 已经知道它落后于其它监视器，然后向 leader 请求同步，leader 让它去和 provider 同步。



新监视器加入集群时有必要进行同步。在运行中，监视器会不定时收到集群运行图的更新，这就意味着 leader 和 provider 角色可能在监视器间变幻。如果这事发生在同步期间（如 provider 落后于 leader ）， provider 能终结和 requester 间的同步。

一旦同步完成，Ceph 需要修复整个集群，使归置组回到 `active + clean` 状态。

`mon sync trim timeout`

描述:

类型: Double

默认值: 30.0

```
mon sync heartbeat timeout
```

描述:

类型: Double

默认值: 30.0

```
mon sync heartbeat interval
```

描述:

类型: Double

默认值: 5.0

```
mon sync backoff timeout
```

描述:

类型: Double

默认值: 30.0

```
mon sync timeout
```

描述:

类型: Double

默认值: 30.0

```
mon sync max retries
```

描述:

类型: Integer

默认值: 5

```
mon sync max payload size
```

描述: 同步载荷的最大尺寸。

类型: 32-bit Integer

默认值: 1045676

```
mon accept timeout
```

描述: leader 等待 requester(s) 接受 PAXOS 更新的时间，出于同样的目的此值也用于 PAXOS 恢复阶段。

类型: Float

默认值: 10.0

`paxos propose interval`

描述: 提议更新之前收集本时间段的更新。

类型: Double

默认值: 1.0

`paxos min wait`

描述: 经过一段不活跃时间后，收集更新的最小等待时间。

类型: Double

默认值: 0.05

`paxos trim tolerance`

描述: 修复前容忍的其他提议数量。

类型: Integer

默认值: 30

```
paxos trim disabled max versions
```

描述: 允许不修复就通过的最大版本数。

类型: Integer

默认值: 100

```
mon lease
```

描述: 监视器版本租期（秒）。

类型: Float

默认值: 5

```
mon lease renew interval
```

描述: 监视器 leader（头领）刷新其他监视器租期的间隔。

类型: Float

默认值: 3

```
mon lease ack timeout
```

描述: leader 在等到 providers（随从）确认延长租期前等待的时间。

类型: Float

默认值: 10.0

```
mon min osdmap epochs
```

描述: 一直保存的 OSD 图元素最小数量。

类型: 32-bit Integer

默认值: 500

```
mon max pgmap epochs
```

描述: 监视器应该一直保存的 PG 图元素最大数量。

类型: 32-bit Integer

默认值: 500

```
mon max log epochs
```

描述: 监视器应该保留的最大日志数量。

类型: 32-bit Integer

默认值: 500

SLURP

在 Ceph 0.58 及之前版本中，当 Paxos 服务偏差的版本数大于某值时，就会触发 *slurp* 机制，它会和法定人数 leader 建立一个连接并获取 leader 拥有的每个版本，以同步每个有偏差的服务。Ceph 0.59 及后续版本的 slurp 机制取消了，因为所有服务共享一个 Paxos 例程。

Deprecated since version 0.58.

```
paxos max join drift
```

描述: 在我们首次同步监视器数据存储前，Paxos 迭代的最大数量。

类型: Integer

默认值: 10

`mon slurp timeout`

描述: 监视器进程终止后、自举前，要等待多长时间才开始发出显式修复通告。

类型: Double

默认值: 10.0

`mon slurp bytes`

描述: 显式修复消息尺寸限制。

类型: 32-bit Integer

默认值: 256 * 1024

时钟

Ceph 的守护进程会相互传递关键消息，这些消息必须在达到超时阈值前处理掉。如果 Ceph 监视器时钟不同步，就可能出现多种异常情况。例如：

- 守护进程忽略了收到的消息（如时间戳过时了）

- 消息未及时收到时，超时触发得太快或太晚。

详情见[监视器存储同步](#)和 [Slurp](#)。

Tip

你应该在所有监视器主机上安装 NTP 以确保监视器集群的时钟同步。

时钟漂移即使尚未造成损坏也能被 NTP 感知，Ceph 的时钟漂移或时钟偏差警告即使在 NTP 同步水平合理时也会被触发。提高时钟漂移值有时候尚可容忍，然而很多因素（像载荷、网络延时、覆盖默认超时值和[监视器存储同步](#)选项）都能在不降低 Paxos 保证级别的情况下影响可接受的时钟漂移水平。

Ceph 提供了下列这些可调选项，让你自己琢磨可接受的值。

`clock offset`

描述: 时钟可以漂移多少，详情见 `Clock.cc`。

类型: Double

默认值: 0

Deprecated since version 0.58.

`mon tick interval`

描述: 监视器的心跳间隔，单位为秒。

类型: 32-bit Integer

默认值: 5

```
mon clock drift allowed
```

描述: 监视器间允许的时钟漂移量

类型: Float

默认值: .050

```
mon clock drift warn backoff
```

描述: 时钟偏移警告的退避指数。

类型: Float

默认值: 5

```
mon timecheck interval
```

描述: 和 leader 的时间偏移检查（时钟漂移检查）。单位为秒。

类型: Float

默认值: 300.0

客户端

`mon client hung interval`

描述: 客户端每 N 秒尝试一个新监视器，直到它建立连接。

类型: Double

默认值: 3.0

`mon client ping interval`

描述: 客户端每 N 秒 ping 一次监视器。

类型: Double

默认值: 10.0

`mon client max log entries per message`

描述: 某监视器为每客户端生成的最大日志条数。

类型: Integer

默认值: 1000

`mon client bytes`

描述: 内存中允许存留的客户端消息数量（字节数）。

类型: 64-bit Integer Unsigned

默认值: 100ul << 20

杂项

`mon max osd`

描述: 集群允许的最大 OSD 数量。

类型: 32-bit Integer

默认值: 10000

`mon globalid prealloc`

描述: 为集群预分配的全局 ID 数量。

类型: 32-bit Integer

默认值: 100

```
mon sync fs threshold
```

描述: 数量达到设定值时和文件系统同步，0 为禁用。

类型: 32-bit Integer

默认值: 5

```
mon subscribe interval
```

描述: 同步的刷新闻隔（秒），同步机制允许获取集群运行图和日志信息。

类型: Double

默认值: 300

```
mon stat smooth intervals
```

描述: Ceph 将平滑最后 `N` 个归置组图的统计信息。

类型: Integer

默认值: `2`

`mon probe timeout`

描述: 监视器自举无效，搜寻节点前等待的时间。

类型: Double

默认值: `2.0`

`mon daemon bytes`

描述: 给元数据服务器和 OSD 的消息使用的内存空间（字节）。

类型: 64-bit Integer Unsigned

默认值: `400ul << 20`

`mon max log entries per event`

描述: 每个事件允许的最大日志条数。

类型: Integer

默认值: 4096

监视器与 OSD 交互的配置

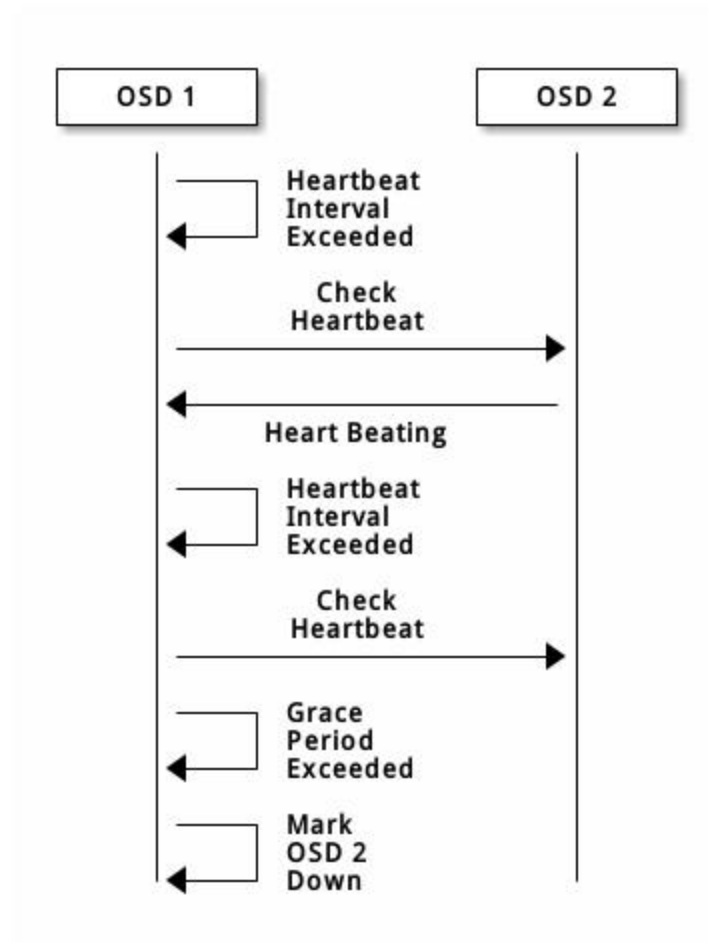
(心跳配置)

完成基本配置后就可以部署、运行 Ceph 了。执行 `ceph health` 或 `ceph -s` 命令时，[监视器](#)会报告 [Ceph 存储集群](#)的当前状态。监视器通过让各 [OSD](#) 自己报告、并接收 OSD 关于邻居状态的报告来掌握集群动态。如果监视器没收到报告，或者它只收到集群的变更报告，那它就要更新[集群运行图](#)。

关于监视器与 OSD 的交互 Ceph 提供了合理的默认值，然而你可以覆盖它们。下面几段从集群监控角度描述了 Ceph 监视器与 OSD 如何交互。

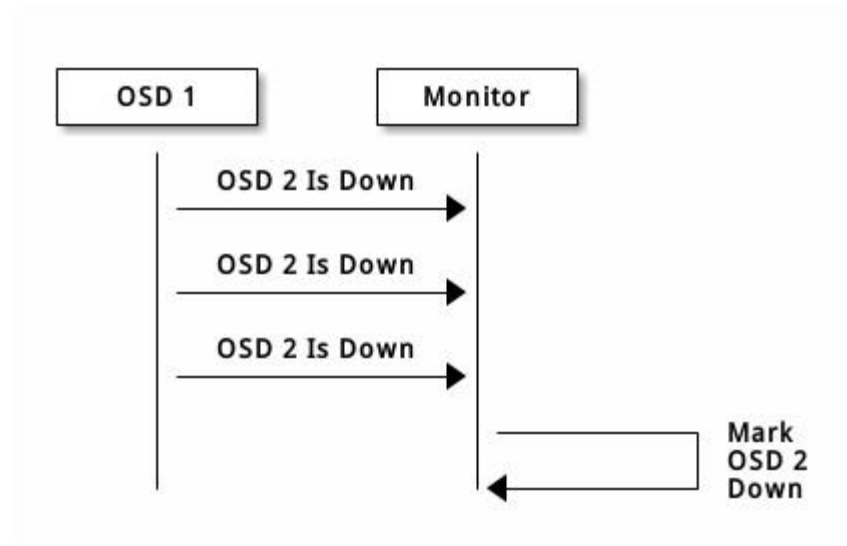
OSD 验证心跳

各 OSD 每 6 秒会与其他 OSD 进行心跳检查，用 `[osd]` 下的 `osd heartbeat interval` 可更改此间隔、或运行时更改。如果一个 OSD 20 秒都没有心跳，集群就认为它 `down` 了，用 `[osd]` 下的 `osd heartbeat grace` 可更改宽限期、或者运行时更改。



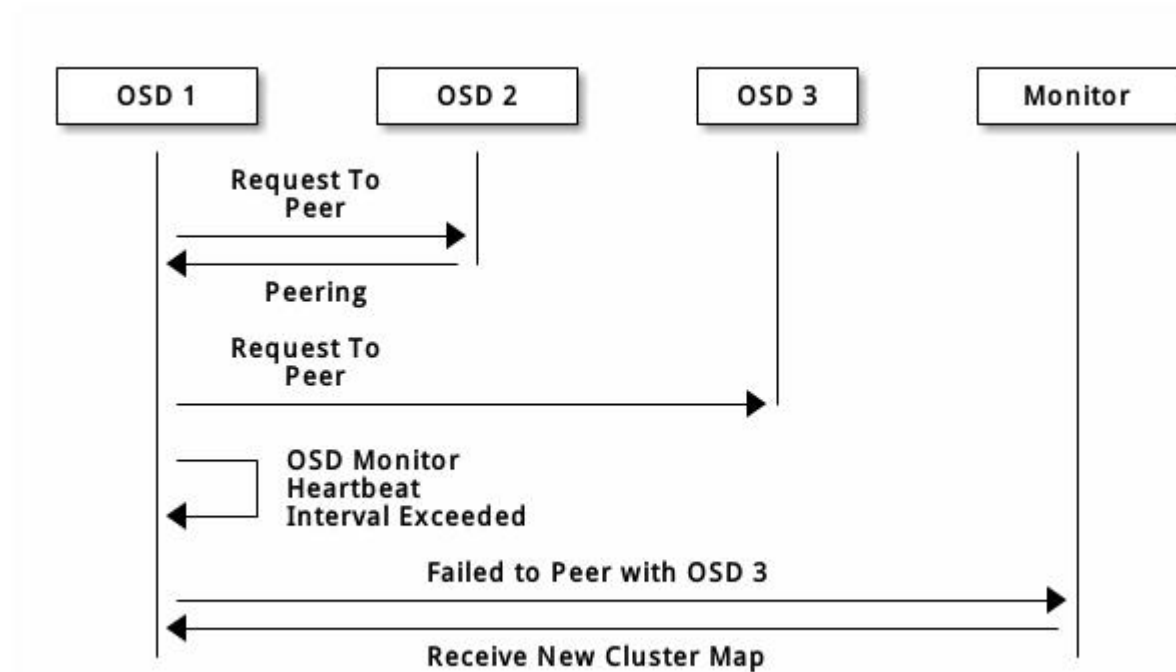
OSD 报告死亡 OSD

默认情况下，一个 OSD 必须向监视器报告三次另一个 OSD `down` 的消息，监视器才会认为那个被报告的 OSD `down` 了；配置文件里 `[mon]` 段下的 `monosd min down reports` 选项（v0.62 之前是 `osd min down reports`）可更改这个最少 `osd down` 消息次数，或者运行时设置。默认情况下，只要有一个 OSD 报告另一个 OSD 挂的消息即可，配置文件里 `[mon]` 段下的 `mon osd min down reporters` 可用来更改必需 OSD 数（v0.62 之前的 `osd min down reporters`），或者运行时更改。



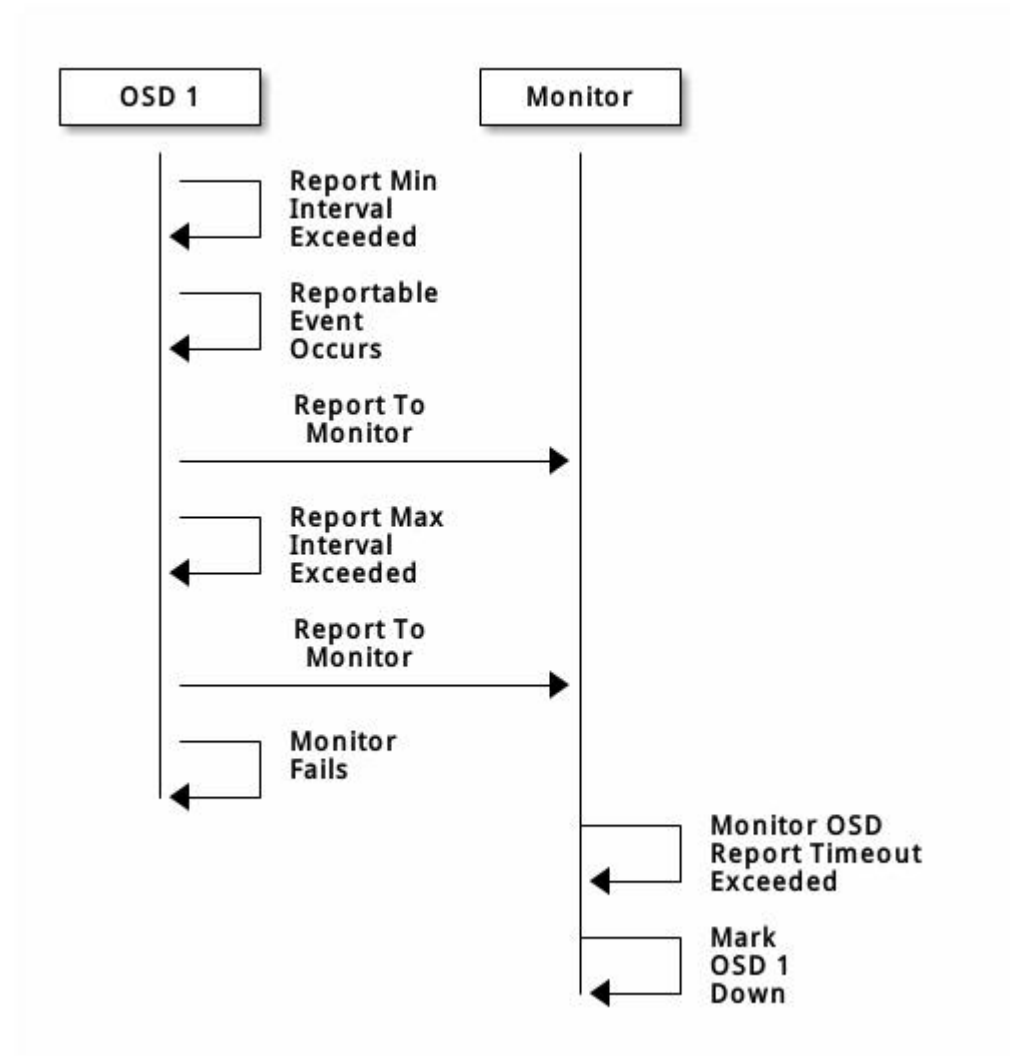
OSD 报告互联失败

如果一 OSD 守护进程不能和配置文件中定义的任何 OSD 建立连接，它会每 30 秒向监视器索要一次最新集群运行图，你可以在 `[osd]` 下设置 `osd monheartbeat interval` 来更改这个心跳间隔，或者运行时更改。



OSD 报告自己的状态

如果一 OSD 在 `mon osd report timeout` 时间内没向监视器报告过，监视器就认为它 `down` 了。在 OSD 守护进程会向监视器报告某些事件，如某次操作失败、归置组状态变更、`up_thru` 变更、或它将在 5 秒内启动。你可以设置 `[osd]` 下的 `osd mon report interval min` 来更改最小报告间隔，或在运行时更改。OSD 守护进程每 120 秒会向监视器报告其状态，不论是否有值得报告的事件。在 `[osd]` 段下设置 `osd mon reportinterval max` 可更改 OSD 报告间隔，或运行时更改。



配置选项

心跳选项应该置于配置文件的 `[global]` 段下。

监视器选项

```
mon osd min up ratio
```

描述: 在把 OSD 标记为 `down` 前, 保持处于 `up` 状态的 OSD 最小比例。

类型: Double

默认值: `.3`

```
mon osd min in ratio
```

描述: 在把 OSD 标记为 `out` 前, 保持处于 `in` 状态的 OSD 最小比例。

类型: Double

默认值: `.3`

```
mon osd laggy halflife
```

描述: 滞后量消退时间, 秒。

类型: Integer

默认值: `60*60`

```
mon osd laggy weight
```

描述: 滞后量消退时新样本的权重。

类型: Double

默认值: 0.3

```
mon osd adjust heartbeat grace
```

描述: 设置为 `true` 时，Ceph 将根据滞后量伸缩。

类型: Boolean

默认值: `true`

```
mon osd adjust down out interval
```

描述: 设置为 `true` 时，Ceph 将根据滞后量伸缩。

类型: Boolean

默认值: `true`

```
mon osd auto mark in
```

描述: Ceph 将把任何启动中的 OSD 标记为在集群中 (`in`)。

类型: Boolean

默认值: `false`

```
mon osd auto mark auto out in
```

描述: 把正在启动、且被自动标记为 `out` 状态的 OSD 标记为 `in`。

类型: Boolean

默认值: `true`

```
mon osd auto mark new in
```

描述: 把正在启动的新 OSD 标记为 `in`。

类型: Boolean

默认值: `true`

```
mon osd down out interval
```

描述: 在 OSD 停止响应多少秒后把它标记为 `down` 且 `out` 。

类型: 32-bit Integer

默认值: 300

```
mon osd downout subtree limit
```

描述: Ceph 可以把最大为多少的 `CRUSH` 单元自动标记为 `out` 。

类型: String

默认值: rack

```
mon osd report timeout
```

描述: 宣布无响应 OSD `down` 前的宽限期，秒。

类型: 32-bit Integer

默认值: 900

```
mon osd min down reporters
```

描述: 确定一 OSD 状态为 `down` 的最少报告来源 OSD 数。

类型: 32-bit Integer

默认值: `1`

```
mon osd min down reports
```

描述: 一 OSD 必须重复报告另一个 `down` 的次数。

类型: 32-bit Integer

默认值: `3`

OSD 选项

```
osd heartbeat address
```

描述: OSD 用于心跳的网络地址。

类型: Address

默认值: 此主机的地址。

```
osd heartbeat interval
```

描述: 一 OSD 探测邻居的频率，秒。

类型: 32-bit Integer

默认值: 6

```
osd heartbeat grace
```

描述: OSD 多久没心跳就会被集群认为它挂（down）了。

类型: 32-bit Integer

默认值: 20

```
osd mon heartbeat interval
```

描述: OSD 没有邻居时多久探测一次监视器。

类型: 32-bit Integer

默认值: 30

```
osd mon report interval max
```


描述: 监视器允许 OSD 报告的最大间隔，超时将认为 OSD 挂了（down）。

类型: 32-bit Integer

默认值: 120

```
osd mon report interval min
```

描述: 从一 OSD 启动或其它可报告事件发生以来，多长时间内必须向监视器报告一次。

类型: 32-bit Integer

默认值: 5

有效范围: 要小于 `osd mon report interval max`。

```
osd mon ack timeout
```

描述: OSD 等待监视器提供统计信息的时间，秒。

类型: 32-bit Integer

默认值: 30

OSD 配置参考

你可以通过配置文件调整 OSD，但靠默认值和极少的配置 OSD 守护进程就能运行。最简 OSD 配置需设置 `osd journal size` 和 `host`，其他几乎都能用默认值。

Ceph 的 OSD 守护进程用递增的数字作标识，按惯例以 0 开始，如下：

```
osd.0  
  
osd.1  
  
osd.2
```

在配置文件里，`[osd]` 段下的配置适用于所有 OSD；要添加针对特定 OSD 的选项（如 `host`），把它放到那个 OSD 段下即可，如：

```
[osd]  
  
    osd journal size = 1024  
  
[osd.0]  
  
    host = osd-host-a  
  
[osd.1]  
  
    host = osd-host-b
```

常规配置

下列选项可配置一 OSD 的唯一标识符、以及数据和日志的路径。Ceph 部署脚本通常会自动生成 UUID 。我们**不建议**更改数据和日志的默认路径，因为这样会增加后续的排障难度。

日志尺寸应该大于期望的驱动器速度和 `filestore max sync interval` 之乘积的两倍；最常见的方法是为日志驱动器（通常是 SSD ）分区并挂载好，这样 Ceph 就可以用整个分区做日志。

`osd uuid`

描述: OSD 的全局唯一标识符（UUID）。

类型: UUID

默认值: The UUID.

备注: `osd uuid` 适用于单个 OSD ， `fsid` 适用于整个集群。

`osd data`

描述: OSD 数据存储位置，你得创建并把数据盘挂载到其下。我们不推荐更改默认值。

类型: String

默认值: `/var/lib/ceph/osd/$cluster-$id`

`osd max write size`

描述: 一次写入的最大尺寸，MB。

类型: 32-bit Integer

默认值: 90

`osd client message size cap`

描述: 内存里允许的最大客户端数据消息。

类型: 64-bit Integer Unsigned

默认值: 默认为 500MB。 `500*1024L*1024L`

`osd class dir`

描述: RADOS 类插件的路径。

类型: String

默认值: `$libdir/rados-classes`

文件系统选项

Ceph 可自动创建并挂载所需的文件系统。

```
osd mkfs options {fs-type}
```

描述: 为 OSD 新建 {fs-type} 类型的文件系统时使用的选项。

类型: String

xfs 默认值: `-f -i 2048`

其余文件系统默认值: {empty string}

实例: `osd mkfs options xfs = -f -d agcount=24`

```
osd mount options {fs-type}
```

描述: 挂载 {fs-type} 类型的文件系统作为 OSD 数据目录时所用的选项。

类型: String

xfs 默认值: `rw,noatime,inode64`

其余文件
系统默认
值:

```
rw, noatime
```

实例:

```
osd mount options xfs = rw, noatime, inode64, logbufs=8
```

日志选项

默认情况下，Ceph 觉得你会把 OSD 日志存储于下列路径：

```
/var/lib/ceph/osd/$cluster-$id/journal
```

未做性能优化时，Ceph 会把日志存储在与 OSD 数据相同的硬盘上。追求高性能的 OSD 可用单独的硬盘存储日志数据，如固态硬盘能提供高性能日志。

`osd journal size` 默认值是 0，所以你得在 `ceph.conf` 里设置。日志尺寸应该是 `filestore max sync interval` 与期望吞吐量的乘积再乘以 2。

```
osd journal size = {2 * (expected throughput * filestore  
max sync interval)}
```

期望吞吐量应考虑期望的硬盘吞吐量（即持续数据传输速率）、和网络吞吐量，例如一个 7200 转硬盘的速度大致是 100MB/s。硬盘和网络吞吐量中较小的（`min()`）一个是相对合理的吞吐量，有的用户则以 10GB 日志尺寸起步，例如：

```
osd journal size = 10000
```

`osd journal`

描述: OSD 日志路径，可以是一个文件或块设备（SSD 的一个分区）的路径。如果是文件，要先创建相应目录。我们建议用 `osd data` 以外的独立驱动器。

类型: String

默认值: `/var/lib/ceph/osd/$cluster-$id/journal`

`osd journal size`

描述: 日志尺寸（MB）。如果是 0 且日志文件是块设备，它会使用整个块设备。从 v0.54 起，如果日志文件是块设备，这个选项会被忽略，且使用整个块设备。

类型: 32-bit Integer

默认值: 5120

推荐 最少 1G，应该是期望的驱动器速度和 `filestore max sync interval` 的乘积。

值:

详情见[日志配置参考](#)。

监视器和 OSD 的交互

OSD 周期性地相互检查心跳并报告给监视器。Ceph 默认配置可满足多数情况，但是如果你的网络延时大，就得用较长间隔。关于心跳的讨论参见[监视器与 OSD 交互的配置](#)。

数据归置

详情见[存储池和归置组配置参考](#)。

洗刷

除了为对象复制多个副本外，Ceph 还要洗刷归置组以确保数据完整性。这种洗刷类似对象存储层的 `fsck`，对每个归置组，Ceph 生成一个所有对象的目录，并比对每个主对象及其副本以确保没有对象丢失或错配。轻微洗刷（每天）检查对象尺寸和属性，深层洗刷（每周）会读出数据并用校验和方法确认数据完整性。

洗刷对维护数据完整性很重要，但会影响性能；你可以用下列选项来增加或减少洗刷操作。

```
osd max scrubs
```

描述: 一 OSD 的最大并发洗刷操作数。

类型: 32-bit Int

默认值: 1

```
osd scrub begin hour
```


描述: 被调度的洗刷操作在一天中可以运行的时间下限。

类型: 0 到 24 之间的整数

默认值: 0

`osd scrub end hour`

描述: 被调度的洗刷操作在一天中可以运行的时间上限。本选项与 `osd scrub begin hour` 一起定义了一个时间窗口，在此期间可以进行洗刷操作。但是，在某个归置组的洗刷间隔超过 `osd scrub max interval` 时，不管这个时间窗口是否合适都会执行。

类型: 0 到 24 之间的整数

默认值: 24

`osd scrub thread timeout`

描述: 洗刷线程最大死亡时值。

类型: 32-bit Integer

默认值: 60

```
osd scrub finalize thread timeout
```

描述: 洗刷终结线程最大超时值。

类型: 32-bit Integer

默认值: 60*10

```
osd scrub load threshold
```

描述: 最大负载，当前系统负载（`getloadavg()` 所定义的）高于此值时 Ceph 不会洗刷。默认 0.5。

类型: Float

默认值: 0.5

```
osd scrub min interval
```

描述: 集群负载低的时候，洗刷的最大间隔时间，秒。

类型: Float

默认值: 每天一次。 `60*60*24`

`osd scrub max interval`

描述: 不论集群负载如何，都要进行洗刷的时间间隔。

类型: Float

默认值: 每周一次。 `7*60*60*24`

`osd deep scrub interval`

描述: 深层洗刷的间隔（完整地读所有数据）。 `osd scrub load threshold` 不会影响此选项。

类型: Float

默认值: 每周一次。 `60*60*24*7`

`osd scrub interval randomize ratio`

描述: 在给某一归置组调度下一个洗刷作业时，给 `osd scrub min interval` 增加个随机延时，这个延时是个小于 `osd scrub mininterval * osd scrub interval randomized ratio` 的随机值。所以在实践中，这个默认设置会把洗刷操作随机地散布到允

许的时间窗口内，即 `[1, 1.5] * osd scrub min interval`。

类
型 Float
:

默
认
值 0.5
:

`osd deep scrub stride`

描述: 深层洗刷时的读取尺寸。

类型: 32-bit Integer

默认值: 512 KB. 524288

操作数

操作数选项允许你设置用于服务的线程数，如果把 `osd op threads` 设置为 0 就禁用了多线程。默认情况下，Ceph 用 30 秒超时和 30 秒抗议时间来把握 2 个线程的运行情况。你可以调整客户端操作和恢复操作的优先程度来优化恢复期间的性能。

`osd op threads`

描述: OSD 操作线程数，0 为禁用。增大数量可以增加请求处理速度。

类型: 32-bit Integer

默认值: 2

`osd client op priority`

描述: 设置客户端操作优先级，它相对于 `osd recovery op priority`。

类型: 32-bit Integer

默认值: 63

有效范围: 1-63

`osd recovery op priority`

描述: 设置恢复优先级，其值相对于 `osd client op priority`。

类型: 32-bit Integer

默认值: 10

有效范围: 1-63

`osd op thread timeout`

描述: OSD 线程超时秒数。

类型: 32-bit Integer

默认值: 30

`osd op complaint time`

描述: 一个操作进行多久后开始抱怨。

类型: Float

默认值: 30

`osd disk threads`

描述: 硬盘线程数，用于在后台执行磁盘密集型操作，像数据洗刷和快照修复。

类型: 32-bit Integer

默认值: 1

`osd disk thread ioprio class`

警告：只有 `osd disk thread ioprio class` 和 `osd disk thread ioprio priority` 同时改为非默认值时此配置才生效。OSD 用 `ioprio_set(2)` 为磁盘线程设置 I/O 调度分类（`class`），当前支持 `idle`、`be` 或 `rt`。`idle` 类意味着磁盘线程的优先级在 OSD 中是最低的，适合需延缓洗刷操作的情形，如 OSD 正忙于处理客户端操作。`be` 是默认值，将设置与其它 OSD 线程相同的优先级。`rt` 意为磁盘线程的优先级将高于其它任何 OSD 线程；适用于急需洗刷、并且即使牺牲客户端操作也要进行时。注：只能与 Linux 内核的 CFQ 调度器配合使用。

类型 String
:

默认值 空字符串
:

`osd disk thread ioprio priority`

警告：只有 `osd disk thread ioprio class` 和 `osd disk thread ioprio priority` 同时改为非默认值时此配置才生效。它通过 `ioprio_set(2)` 设置磁盘线程的 I/O 调度优先级（`priority`），优先级从最高的 0 到最低的 7。如果某主机上的所有 OSD 都在 `idle` 类中竞争 I/O 资源（即控制器堵塞了），那么你就可以用此选项把某 OSD 的磁盘线程优先级调低为 7，其它优先级为 0 的 OSD 就有可能洗刷得快一点。注：只能与 Linux 内核的 CFQ 调度器配合使用。

类型 0 到 7 间的整数，-1 禁用此功能。
:

默认值：
-1

`osd op history size`

描述: 要跟踪的最大已完成操作数量。

类型: 32-bit Unsigned Integer

默认值: 20

`osd op history duration`

描述: 要跟踪的最老已完成操作。

类型: 32-bit Unsigned Integer

默认值: 600

`osd op log threshold`

描述: 一次显示多少操作日志。

类型: 32-bit Integer

默认值: 5

回填

当集群新增或移除 OSD 时，按照 CRUSH 算法应该重新均衡集群，它会把一些归置组移出或移入多个 OSD 以回到均衡状态。归置组和对象的迁移会导致集群运营性能显著降低，为维持运营性能，Ceph 用 backfilling 来执行此迁移，它可以使得 Ceph 的回填操作优先级低于用户读写请求。

```
osd max backfills
```

描述: 单个 OSD 允许的最大回填操作数。

类型: 64-bit Unsigned Integer

默认值: 10

```
osd backfill scan min
```

描述: 集群负载低时，回填操作时扫描间隔。

类型: 32-bit Integer

默认值: 64

```
osd backfill scan max
```

描述: 回填操作时最大扫描间隔。

类型: 32-bit Integer

默认值: 512

```
osd backfill full ratio
```

描述: OSD 的占满率达到多少时拒绝接受回填请求。

类型: Float

默认值: 0.85

```
osd backfill retry interval
```

描述: 重试回填请求前等待秒数。

类型: Double

默认值: 10.0

OSD 运行图

OSD 运行图反映集群中运行的 OSD 守护进程，斗转星移，图元增加。Ceph 用一些选项来确保 OSD 运行图增大时仍运行良好。

`osd map dedup`

描述: 允许删除 OSD 图里的重复项。

类型: Boolean

默认值: `true`

`osd map cache size`

描述: 缓存的 OSD 图个数。

类型: 32-bit Integer

默认值: `500`

`osd map cache bl size`

描述: OSD 进程中，驻留内存的 OSD 图缓存尺寸。

类型: 32-bit Integer

默认值: 50

```
osd map cache bl inc size
```

描述: OSD 进程中，驻留内存的 OSD 图缓存增量尺寸。

类型: 32-bit Integer

默认值: 100

```
osd map message max
```

描述: 每个 MOSDMap 图消息允许的最大条目数量。

类型: 32-bit Integer

默认值: 100

恢复

当集群启动、或某 OSD 守护进程崩溃后重启时，此 OSD 开始与其它 OSD 们建立连接，这样才能正常工作。详情见[监控 OSD 和归置组](#)。

如果某 OSD 崩溃并重生，通常会落后于其他 OSD，也就是没有同归置组内最新版本的对象。这时，OSD 守护进程进入恢复模式并检索最新数据副本，并更新运行图。根据 OSD 挂的时间长短，OSD 的对象和归置组可能落后得厉害，另外，如果挂的是一个失效域（如一个机柜），多个 OSD 会同时重生，这样恢复时间更长、更耗资源。

为保持运营性能，Ceph 进行恢复时会限制恢复请求数、线程数、对象块尺寸，这样在降级状态下也能保持良好的性能。

```
osd recovery delay start
```

描述: 对等关系建立完毕后，Ceph 开始对象恢复前等待的时间（秒）。

类型: Float

默认值: 0

```
osd recovery max active
```

描述: 每个 OSD 一次处理的活跃恢复请求数量，增大此值能加速恢复，但它们会增加集群负载。

类型: 32-bit Integer

默认值: 15

```
osd recovery max chunk
```

描述: 一次推送的数据块的最大尺寸。

类型: 64-bit Integer Unsigned

默认值: 8 << 20

`osd recovery threads`

描述: 数据恢复时的线程数。

类型: 32-bit Integer

默认值: 1

`osd recovery thread timeout`

描述: 恢复线程最大死亡时值。

类型: 32-bit Integer

默认值: 30

`osd recover clone overlap`

描述: 在数据恢复期间保留重叠副本。应该总是 `true`。

类型: Boolean

默认值: `true`

杂项

`osd snap trim thread timeout`

描述: 快照修复线程最大死亡时值。

类型: 32-bit Integer

默认值: `60*60*1`

`osd backlog thread timeout`

描述: 积压线程最大死亡时值。

类型: 32-bit Integer

默认值: `60*60*1`

`osd default notify timeout`

描述: OSD 默认通告超时，秒。

类型: 32-bit Integer Unsigned

默认值: 30

`osd check for log corruption`

描述: 根据日志文件查找数据损坏，会耗费大量计算时间。

类型: Boolean

默认值: false

`osd remove thread timeout`

描述: OSD 删除线程的最大死亡时值。

类型: 32-bit Integer

默认值: 60*60

`osd command thread timeout`

描述: 命令线程最大超时值。

类型: 32-bit Integer

默认值: 10*60

```
osd command max records
```

描述: 限制返回的丢失对象数量。

类型: 32-bit Integer

默认值: 256

```
osd auto upgrade tmap
```

描述: 在旧对象上给 omap 使用 tmap。

类型: Boolean

默认值: true

```
osd tmapput sets users tmap
```

描述: 只在调试时使用 tmap。

类型: Boolean

默认值: `false`

`osd preserve trimmed log`

描述: 保留本该修剪掉的日志文件，但是会占用更多磁盘空间。

类型: Boolean

默认值: `false`

FILESTORE 配置参考

`filestore debug omap check`

描述: 打开对同步检查过程的调试。代价很高，仅用于调试。

类型: Boolean

是否必需: No

默认值: `0`

扩展属性

扩展属性（XATTR）是配置里的重要部分。一些文件系统对 XATTR 字节数有限制，另外在某些情况下文件系统存储 XATTR 的速度不如其他方法。下面的选项让你用独立于文件系统的存储方法，或许能提升性能。

Ceph 扩展属性用底层文件系统的 XATTR（如果没有尺寸限制）存储为 `inline xattr`。如果有限制，如 `ext4` 限制为 4KB，达到 `filestore maxinline xattr size` 或 `filestore max inline xattrs` 阈值时一些 XATTR 将存储为键/值数据库（也叫 `omap`）。

```
filestore xattr use omap
```

描述: 用 XATTR 存储对象图，采用 `ext4` 文件系统时要设置为 `true`。

类型: Boolean

是否必需: No

默认值: `false`

```
filestore max inline xattr size
```

描述: 每个对象在文件系统（如 `XFS`、`btrfs`、`ext4` 等）里存储的 XATTR 最大尺寸，应该小于文件系统支持的尺寸。

类型: Unsigned 32-bit Integer

是否必需: No

默认值: 512

```
filestore max inline xattrs
```

描述: 每个对象存储在文件系统里的 XATTR 数量。

类型: 32-bit Integer

是否必需: No

默认值: 2

同步间隔

filestore 需要周期性地静默写入、同步文件系统，这创建了一个提交点，然后就能释放相应的日志条目了。较大的同步频率可减小执行同步的时间及保存在日志里的数据量；较小的频率使得后端的文件系统能优化归并较小的数据和元数据写入，因此可能使同步更有效。

```
filestore max sync interval
```

描述: 同步 filestore 的最大间隔秒数。

类型: Double

是否必需: No

默认值: 5

`filestore min sync interval`

描述: 同步 filestore 的最小间隔秒数。

类型: Double

是否必需: No

默认值: .01

回写器

filestore 回写器强制使用 `sync file range` 来写出大块数据，这样处理有望减小最终同步的代价。实践中，禁用“filestore 回写器”有时候能提升性能。

`filestore flusher`

描述: 启用 filestore 回写器。

类型: Boolean

是否必需: No

默认值: false

Deprecated since version v.65.

```
filestore flusher max fds
```

描述: 设置回写器的最大文件描述符数量。

类型: Integer

是否必需: No

默认值: 512

Deprecated since version v.65.

```
filestore sync flush
```

描述: 启用同步回写器。

类型: Boolean

是否必需: No

默认值: `false`

Deprecated since version v.65.

```
filestore fsync flushes journal data
```

描述: 文件系统同步时也回写日志数据。

类型: Boolean

是否必需: No

默认值: `false`

队列

下面的选项能限制 filestore 队列的尺寸。

```
filestore queue max ops
```

描述: 文件存储操作接受的最大并发数，超过此设置的请求会被阻塞。

类型: Integer

是否必需： 无。对性能影响最小。

默认值： 500

`filestore queue max bytes`

描述： 一个操作的最大字节数。

类型： Integer

是否必需： No

默认值： 100 << 20

`filestore queue committing max ops`

描述： filestore 能提交的最大操作数。

类型： Integer

是否必需： No

默认值： 500

filestore queue committing max bytes

描述: filestore 器能提交的最大字节数。

类型: Integer

是否必需: No

默认值: 100 << 20

超时选项

filestore op threads

描述: 允许并行操作文件系统的最大线程数。

类型: Integer

是否必需: No

默认值: 2

filestore op thread timeout

描述: 文件系统操作线程超时值，单位为秒。

类型: Integer

是否必需: No

默认值: 60

```
filestore op thread suicide timeout
```

描述: 提交操作超时值（秒），超时后会取消。

类型: Integer

是否必需: No

默认值: 180

B-TREE 文件系统

```
filestore btrfs snap
```

描述: 对 `btrfs` `filestore` 启用快照功能。

类型: Boolean

是否必需： 不。仅适用于 `btrfs`。

默认值： `true`

`filestore btrfs clone range`

描述： 允许 `btrfs` `filestore` 克隆操作排队。

类型： `Boolean`

是否必需： 不。仅适用于 `btrfs`。

默认值： `true`

日志

`filestore journal parallel`

描述： 允许并行记日志，对 `btrfs` 默认开。

类型： `Boolean`

是否必需： `No`

默认值: `false`

`filestore journal writeahead`

描述: 允许预写日志，对 **xfs** 默认开。

类型: Boolean

是否必需: No

默认值: `false`

`filestore journal trailing`

描述: 过时了，从没用过。

类型: Boolean

是否必需: No

默认值: `false`

杂项

`filestore merge threshold`

描述: 并入父目录前，子目录内的最小文件数。注：负值表示禁用子目录合并功能。

类型: Integer

是否必需: No

默认值: 10

filestore split multiple

描述: $\text{filestore_split_multiple} * \text{abs}(\text{filestore_merge_threshold}) * 16$ 是分割为子目录前某目录内的最大文件数。

类型: Integer

是否必需: No

默认值: 2

:

filestore update to

描述: 限制 filestore 自动更新到某个指定版本。

类型: Integer

是否必需: No

默认值: 1000

filestore blackhole

描述: 丢弃任何讨论中的事务。

类型: Boolean

是否必需: No

默认值: false

filestore dump file

描述: 存储事务转储目的文件。

类型: Boolean

是否必需: No

默认值: `false`

`filestore kill at`

描述: 在第 N 次机会后注入一个失效。

类型: String

是否必需: No

默认值: `false`

`filestore fail eio`

描述: 在 IO 错误的时候失败或崩溃。

类型: Boolean

是否必需: No

默认值: `true`

KEYVALUESTORE（键值存储）配置参考

`KeyValueStore` is an alternative OSD backend compared to `FileStore`. Currently, it uses `LevelDB` as backend. `KeyValueStore` doesn't need journal device. Each operation will flush into the backend directly.

```
keyvaluestore backend
```

Description: The backend used by `KeyValueStore`.

Type: String

Required: No

Default: `leveldb`

QUEUE

The following settings provide limits on the size of the `KeyValueStore` queue.

```
keyvaluestore queue max ops
```

Description: Defines the maximum number of operations in progress the `KeyValueStore` accepts before blocking on queuing new

operations.

Type: Integer

Required: No. Minimal impact on performance.

Default: 50

```
keyvaluestore queue max bytes
```

Description: The maximum number of bytes for an operation.

Type: Integer

Required: No

Default: 100 << 20

THREAD

```
keyvaluestore op threads
```

Description: The number of `KeyValueStore` operation threads that execute in parallel.

Type: Integer

Required: No

Default: 2

keyvaluestore op thread timeout

Description: The timeout for a `KeyValueStore` operation thread (in seconds).

Type: Integer

Required: No

Default: 60

keyvaluestore op thread suicide timeout

Description: The timeout for a commit operation before canceling the commit (in seconds).

Type: Integer

Required: No

Default: 180

MISC

keyvaluestore default strip size

Description: Each object will be split into multiple key/value pairs and stored in the backend. **Note:** The size of the workload has a significant impact on performance.

Type: Integer

Required: No

Default: 4096

keyvaluestore header cache size

Description: The size of the header cache (identical to `inode` in the local filesystem). A larger cache size enhances performance.

Type: Integer

Required: No

Default: 4096

日志配置参考

Ceph 的 OSD 使用日志的原因有二：速度和一致性。

- **速度：** 日志使得 OSD 可以快速地提交小块数据的写入，Ceph 把小片、随机 IO 依次写入日志，这样，后端文件系统就有可能归并写入动作，并最终提升并发承载力。因此，使用 OSD 日志能展现出优秀的突发写性能，实际上数据还没有写入 OSD，因为文件系统把它们捕捉到了日志。
- **一致性：** Ceph 的 OSD 守护进程需要一个能保证原子操作的文件系统接口。OSD 把一个操作的描述写入日志，然后把操作应用到文件系统，这需要原子更新一个对象（例如归置组元数据）。每隔一段 `filestore max sync interval` 和 ```filestore min sync interval``` 之间的时间，OSD 停止写入、把日志同步到文件系统，这样允许 OSD 修整日志里的操作并重用空间。若失败，OSD 从上个同步点开始重放日志。

OSD 守护进程支持下面的日志选项：

`journal dio`

描述： 对日志启用径直 IO，需要 `journal block align` 设置为 `true`。

类型： Boolean

是否必需： 用 `aio` 时自动启用。

默认值： `true`

`journal aio`

Changed in version 0.61: Cuttlefish

描述: 异步写入日志时用 `libaio` 库, 需要 `journal dio` 设为 `true`。

类型: Boolean

是否必需: No.

默认值: 0.61 版之后为 `true` , 0.60 及之前为 `false` 。

`journal block align`

描述: 块对齐写, `dio` 和 `aio` 需要。

类型: Boolean

是否必需: 用 `dio` 和 `aio` 时自动启用。

默认值: `true`

`journal max write bytes`

描述: 一次写入日志的最大尺寸。

类型: Integer

是否必需: No

默认值: 10 << 20

`journal max write entries`

描述: 一次写入日志的最大数量。

类型: Integer

是否必需: No

默认值: 100

`journal queue max ops`

描述: 队列里一次允许的最大操作数量。

类型: Integer

是否必需: No

默认值: 500

`journal queue max bytes`

描述: 队列里一次允许的最大字节数。

类型: Integer

是否必需: No

默认值: `10 << 20`

`journal align min size`

描述: 对齐大于指定最小值的数据有效载荷。

类型: Integer

是否必需: No

默认值: `64 << 10`

`journal zero on create`

描述: 在创建文件系统 (`mkfs`) 期间用 0 填充整个日志。

类型: Boolean

是否必需: No

默认值: false

存储池、归置组和 CRUSH 配置参考

当你创建存储池并给它设置归置组数量时，如果你没指定 Ceph 就用默认值。**我们建议**更改某些默认值，特别是存储池的副本数和默认归置组数量，可以在运行 `pool` 命令的时候设置这些值。你也可以把配置写入 Ceph 配置文件的 `[global]` 段来覆盖默认值。

`[global]`

```
# By default, Ceph makes 3 replicas of objects. If
you want to make four

# copies of an object the default value--a primary
copy and three replica

# copies--reset the default values as shown in 'osd
pool default size'.

# If you want to allow Ceph to write a lesser number
of copies in a degraded

# state, set 'osd pool default min size' to a number
less than the

# 'osd pool default size' value.
```



```
osd pool default size = 4  # Write an object 4 times.

osd pool default min size = 1 # Allow writing one
copy in a degraded state.


# Ensure you have a realistic number of placement
groups. We recommend


# approximately 100 per OSD. E.g., total number of
OSDs multiplied by 100


# divided by the number of replicas (i.e., osd pool
default size). So for


# 10 OSDs and osd pool default size = 4, we'd
recommend approximately


# (100 * 10) / 4 = 250.


osd pool default pg num = 250

osd pool default pgp num = 250
```

```
mon max pool pg num
```

描述: 每个存储的最大归置组数量。

类型: Integer

默认值: 65536

```
mon pg create interval
```

描述: 在同一个 OSD 里创建 PG 的间隔秒数。

类型: Float

默认值: 30.0

```
mon pg stuck threshold
```

描述: 多长时间无响应的 PG 才认为它卡住了。

类型: 32-bit Integer

默认值: 300

```
osd pg bits
```

描述: 每个 OSD 的归置组位数。

类型: 32-bit Integer

默认值: 6

```
osd pgp bits
```

描述: 每个 OSD 为 PGP 留的位数。

类型: 32-bit Integer

默认值: 6

```
osd crush chooseleaf type
```

描述: 在一个 CRUSH 规则内用于 `chooseleaf` 的桶类型。用序列号而不是名字。

类型: 32-bit Integer

默认值: 1 , 通常一台主机包含一或多个 OSD 。

```
osd pool default crush replicated ruleset
```

描述: 创建多副本存储池时用哪个默认 CRUSH 规则集。

类型: 8-bit Integer

默认值: CEPH_DEFAULT_CRUSH_REPLICATED_RULESET , 也就是说, “挑选数字 ID 最小的规则集”。这样, 没有规则集 0 时也能成功创建存储池。

```
osd pool erasure code stripe width
```

描述: 设置每个已编码池内的对象条带尺寸 (单位为字节)。尺寸为 S 的各对象将存储为 N 个条带, 且各条带将分别编码/解码。

类型: Unsigned 32-bit Integer

默认值: 4096

```
osd pool default size
```

描述: 设置一存储池的对象副本数, 默认值等同于 `ceph osd pool set {pool-name} size {size}`。

类型: 32-bit Integer

默认值: 3

值:

```
osd pool default min size
```

描述:

设置存储池中已写副本的最小数量，以向客户端确认写操作。如果未达到最小值，Ceph 就不会向客户端回复已写确认。此选项可确保降级（`degraded`）模式下的最小副本数。

类型:

32-bit Integer

默认值:

0，意思是没有最小值。如果为 0，最小值是 $\text{size} - (\text{size} / 2)$ 。

```
osd pool default pg num
```

描述:

一个存储池的默认归置组数量，默认值即是 `mkpool` 的 `pg_num` 参数。

类型:

32-bit Integer

默认值: 8

```
osd pool default pgp num
```

描述:

一个存储池里，为归置使用的归置组数量，默认值等同

于 `mkpool` 的 `pgp_num` 参数。当前 PG 和 PGP 应该相同。

类型: 32-bit Integer

**默认
值:** 8

`osd pool default flags`

描述: 新存储池的默认标志。

类型: 32-bit Integer

默认值: 0

`osd max pgl`

描述: 将列出的最大归置组数量，一客户端请求量大时会影响 OSD。

类型: Unsigned 64-bit Integer

默认值: 1024

Note: 默认值应该没问题。

```
osd min pg log entries
```

描述: 清理日志文件的时候保留的归置组日志量。

类型: 32-bit Int Unsigned

默认值: 1000

```
osd default data pool replay window
```

描述: — OSD 等待客户端重播请求的时间，秒。

类型: 32-bit Integer

默认值: 45

消息传递

```
ms tcp nodelay
```

描述: 在信差的 TCP 会话上禁用 nagle 算法。

类型: Boolean

是否必需: No

默认值: true

ms initial backoff

描述: 出错时重连的初始等待时间。

类型: Double

是否必需: No

默认值: .2

ms max backoff

描述: 出错重连时等待的最大时间。

类型: Double

是否必需: No

默认值: 15.0

`ms nocrc`

描述: 禁用网络消息的 crc 校验，CPU 不足时可提升性能。

类型: Boolean

是否必需: No

默认值: `false`

`ms die on bad msg`

描述: 调试选项，不要配置。

类型: Boolean

是否必需: No

默认值: `false`

`ms dispatch throttle bytes`

描述: 等着传送的消息尺寸阈值。

类型: 64-bit Unsigned Integer

是否必需: No

默认值: 100 << 20

`ms bind ipv6`

描述: 如果想让守护进程绑定到 IPv6 地址而非 IPv4 就得启用（如果你指定了守护进程或集群 IP 就不必要了）

类型: Boolean

是否必需: No

默认值: false

`ms rwthread stack bytes`

描述: 堆栈尺寸调试选项，不要配置。

类型: 64-bit Unsigned Integer

是否必需: No

默认值: 1024 << 10

ms tcp read timeout

描述: 控制信差关闭空闲连接前的等待秒数。

类型: 64-bit Unsigned Integer

是否必需: No

默认值: 900

ms inject socket failures

描述: 调试选项，别配置。

类型: 64-bit Unsigned Integer

是否必需: No

默认值: 0

常规配置参考

`fsid`

描述: 文件系统 ID , 每集群一个。

类型: UUID

是否必需: No.

默认值: 无。通常由部署工具生成。

`admin socket`

描述: 在某个守护进程上执行管理命令的套接字，不管 Ceph 监视器团体是否已建立。

类型: String

是否必需: No

默认值: `/var/run/ceph/$cluster-$name.asok`

pid file

描述: mon、osd、mds 将它们的 PID 写入此文件，其名为 /var/run/\$cluster/\$type.\$id.pid。如名为 Ceph 的集群，其 ID 为 a 的 mon 进程将创建 /var/run/ceph/mon.a.pid。如果是正常停止的，pid file 就会被守护进程删除；如果进程未进入后台运行（即启动时加了 -f 或 -d 选项），它就不会创建 pid file。

类型: String

是否必需: No

默认值: No

chdir

描述: Ceph 进程一旦启动、运行就进入这个目录，默认推荐 /。

类型: String

是否必需: No

默认值: /

max open files

描述: 如果设置了, **Ceph 存储集群**启动时会设置操作系统级的 max open fds (即文件描述符最大允许值), 这有助于防止耗尽文件描述符。

类型: 64-bit Integer

是否必需: No

默认值: 0

fatal signal handlers

描述: 如果设置了, 将安装 SEGV、ABRT、BUS、ILL、FPE、XCPU、XFSZ、SYS 信号处理器, 用于产生有用的日志信息。

类型: Boolean

默认值: true

CEPH 部署

`ceph-deploy` 工具是一种部署 Ceph 的方法，它只依赖到服务器的 SSH 访问、`sudo` 和 Python。它可在你的工作站上运行，不需要服务器、数据库、或其它工具。如果你安装、拆卸过很多 Ceph 集群，不想要额外的工具，那 `ceph-deploy` 是理想之选。它不是个通用部署系统，只为 Ceph 用户设计，用它可以快速地设置并运行一个默认值较合理的集群，而无需头疼 [Chef](#)、Puppet 或 Juju 的安装。如果您想全面控制安全设置、分区、或目录位置，可以试试类似 Juju、Chef、Crowbar 的工具。

用 `ceph-deploy` 你可以自己开发脚本往远程主机上安装 Ceph 软件包、创建集群、增加监视器、收集（或销毁）密钥、增加 OSD 和元数据服务器、配置管理主机，甚至拆除集群。

预检

New in version 0.60.

本篇预检会协助你准备运行 `ceph-deploy` 的节点、和服务器节点，以使用无密码 `ssh` 和 `sudo`。

通过 `ceph-deploy` 部署 Ceph 前，先要在管理节点和运行 Ceph 守护进程的节点上确认一些事情。

选装一个操作系统

在各节点安装最新发布的 Debian 或 Ubuntu（如 12.04 LTS、14.04 LTS），关于 Debian 或 Ubuntu 以外的操作系统详见[操作系统推荐](#)。

安装 SSH 服务器

`ceph-deploy` 工具需要 `ssh`，所以你的服务器节点需要 SSH 服务器。

```
sudo apt-get install openssh-server
```

创建一用户

在运行 Ceph 守护进程的节点上创建一个普通用户。

Tip

我们建议选一个暴力攻击者不可能轻易猜到的用户名，如 `root`、`ceph` 之外的名字。

```
ssh user@ceph-server

sudo useradd -d /home/ceph -m ceph

sudo passwd ceph
```

`ceph-deploy` 会在节点安装软件包，所以你创建的用户需要无密码 `sudo` 权限。

Note

为安全起见，我们不推荐启用 `root` 密码。

为赋予用户所有权限，把下列加入 `/etc/sudoers.d/ceph`。

```
echo "ceph ALL = (root) NOPASSWD:ALL" | sudo tee
/etc/sudoers.d/ceph

sudo chmod 0440 /etc/sudoers.d/ceph
```

配置 SSH

配置你的管理主机，使之可通过 SSH 无密码访问各节点，口令留空。


```
ssh-keygen

Generating public/private key pair.

Enter file in which to save the key (/ceph-
client/.ssh/id_rsa):

Enter passphrase (empty for no passphrase):

Enter same passphrase again:

Your identification has been saved in /ceph-
client/.ssh/id_rsa.

Your public key has been saved in /ceph-
client/.ssh/id_rsa.pub.
```

把公钥拷贝到各节点：

`ssh-copy-id ceph@ceph-server`

修改你管理节点上的 `~/.ssh/config`，以使未指定用户名时默认用你刚刚创建的用户名。

```
Host ceph-server

    Hostname ceph-server.fqdn-or-ip-address.com

    User ceph
```

安装 CEPH-DEPLOY

执行下列命令安装 `ceph-deploy`：

```
wget -q -O- 'https://download.ceph.com/keys/release.asc'
| sudo apt-key add -
```

```
echo deb http://ceph.com/debian-dumpling/ $(lsb_release -
sc) main | sudo tee /etc/apt/sources.list.d/ceph.list

sudo apt-get update

sudo apt-get install ceph-deploy
```

确认连通性

确保你的管理节点网络连接正常、且能连接服务器节点（如确认 `iptables`、`ufw` 或其它可阻断连接的工具、流量转发等等这些组件配置正常）。

完成预检后，你就可以开始用 `ceph-deploy` 部署了。

软件包管理

安装

要在集群主机上安装 Ceph 软件包，在管理主机上打开命令行并执行下列命令：

```
ceph-deploy install {hostname [hostname] ...}
```

没提供额外选项的话 `ceph-deploy` 默认会把最新稳定版安装到集群主机，要指定某个软件包可以用下列参数：

- `--release <code-name>`
- `--testing`
- `--dev <branch-or-tag>`

例如：

```
ceph-deploy install --release cuttlefish hostname1
```

```
ceph-deploy install --testing hostname2

ceph-deploy install --dev wip-some-branch
hostname{1,2,3,4,5}
```

其余用法见：

```
ceph-deploy install -h
```

卸载

要从集群主机卸载 Ceph 软件包，在管理主机的终端下执行：

```
ceph-deploy uninstall {hostname [hostname] ...}
```

在 Debian 或 Ubuntu 系统上你也可以：

```
ceph-deploy purge {hostname [hostname] ...}
```

此工具会从指定主机上卸载 `ceph` 软件包，另外 `purge` 会删除配置文件。

创建一集群

使用 `ceph-deploy` 的第一步就是新建一个集群，新集群具备：

- 一个 Ceph 配置文件，以及
- 一个监视器密钥环。

Ceph 配置文件至少要包含：

- 它自己的文件系统 ID（`fsid`）
- 最初的监视器（们）及其主机名（们），以及

- 最初的监视器及其 IP 地址。

详情见[监视器配置参考](#)。

`ceph-deploy` 工具也创建了一个监视器密钥环并置于 `[mon.]` 内，详情见 [Cephx 手册](#)。

用法

要用 `ceph-deploy` 创建集群，用 `new` 命令、并指定几个主机作为初始监视器法定人数。

```
ceph-deploy new {host [host], ...}
```

例如：

```
ceph-deploy new mon1.foo.com  
  
ceph-deploy new mon{1,2,3}
```

`ceph-deploy` 工具会用 DNS 把主机名解析为 IP 地址。监视器将被命名为域名的第一段（如前述的 `mon1`），它会把指定主机名加入 Ceph 配置文件。其他用法见：

```
ceph-deploy new -h
```

命名集群

Ceph 集群的默认名字为 `ceph`，如果你想在同一套硬件上运行多套集群可以指定其他名字。比如，如果想优化一个集群用于块设备，另一个用作网关，你可以在同一套硬件上运行两个不同的集群，但是它们要配置不同的 `fsid` 和集群名。

```
ceph-deploy --cluster {cluster-name} new {host [host], ...}
```

例如：

```
ceph-deploy --cluster rbdcluster new ceph-mon1
```

```
ceph-deploy --cluster rbdcluster new ceph-mon{1,2,3}
```

Note

如果你运行多个集群，必须修改默认端口选项并为其打开端口，这样两个不同的集群网才不会相互冲突。

增加/删除监视器

用 `ceph-deploy` 增加和删除监视器很简单，只要一个命令就可以增加或删除一或多个监视器。以前，[增加和删除监视器](#) 涉及数个手动步骤。用 `ceph-deploy` 也预示着一个局限性，你只能在一主机上安装一个监视器。

Note

我们不建议把监视器和 OSD 置于同一主机上。

考虑到高可用性，生产集群应该至少有 3 个监视器。Ceph 用 Paxos 算法，要求法定人数里的大多数达成一致。你可以只用一个监视器形成法定人数，然而你不能用两个监视器确定大多数。大多数监视器的比例必须像：1:1、2:3、3:4、3:5、4:6 等等。

关于监视器的配置见[监视器配置参考](#)。

增加一监视器

创建集群并在监视器主机上安装 Ceph 软件包后，接着部署监视器。用 `ceph-deploy` 时，它限制一主机只能装一个监视器。

```
ceph-deploy mon create {host-name [host-name]...}
```

Note

确保你增加的监视器能在大多数成员中达成一致，否则其它步骤（像 `ceph-deploy gatherkeys`）可能会失败。

Note

在一主机上新增监视器时，如果它不是由 `ceph-deploy new` 命令所定义的，那就必须把 `public network` 加入 `ceph.conf` 配置文件。

删除一监视器

如果你想删除集群中的某个监视器，可以用 `destroy` 选项。

```
ceph-deploy mon destroy {host-name [host-name]...}
```

Note

确保你删除一监视器后，其余监视器仍能达成一致。如果不可能，删除它之前先加一个。

密钥管理

收集密钥

在准备一台主机作为 OSD 或元数据服务器时，你得收集监视器、OSD、和 MDS 的初始密钥环，可用下列命令：

```
ceph-deploy gatherkeys {monitor-host}
```

Note

检索密钥时，指定一个包含 Ceph 监视器的主机。

Note

如果建设集群时配置了多个监视器，需确保它们都已启动且在运行。如果监视器没能形成法定人数，`ceph-create-keys` 就会失败、且不会生成密钥。

销毁密钥

不再使用 `ceph-deploy`（或另建一集群）时，你应该删除管理主机上、本地目录中的密钥。可用下列命令：

```
ceph-deploy forgetkeys
```

增加/删除 OSD

新增和拆除 Ceph 的 OSD 进程相比其它两种要多几步。OSD 守护进程把数据写入磁盘和日志，所以你得相应地提供一 OSD 数据盘和日志分区路径（这是最常见的配置，但你可以按需调整）。

从 Ceph v0.60 起，Ceph 支持 `dm-crypt` 加密的硬盘，在准备 OSD 时你可以用 `--dm-crypt` 参数告诉 `ceph-deploy` 你想用加密功能。也可以用 `--dmccrypt-key-dir` 参数指定 `dm-crypt` 加密密钥的位置。

在投建一个大型集群前，你应该测试各种驱动器配置来衡量其吞吐量。详情见[数据存储](#)。

列举磁盘

执行下列命令列举一节点上的磁盘：

```
ceph-deploy disk list {node-name [node-name]...}
```

擦净磁盘

用下列命令擦净（删除分区表）磁盘，以用于 Ceph：

```
ceph-deploy disk zap {osd-server-name}:{disk-name}

ceph-deploy disk zap osdserver1:sdb
```

Important

这会删除所有数据。

准备 OSD

创建集群、安装 Ceph 软件包、收集密钥完成后你就可以准备 OSD 并把它们部署到 OSD 节点了。如果你想确认某磁盘或擦净它，参见[列举磁盘](#)和[擦净磁盘](#)。

```
ceph-deploy osd prepare {node-name}:{data-disk}[:{journal-disk}]

ceph-deploy osd prepare osdserver1:sdb:/dev/ssd

ceph-deploy osd prepare osdserver1:sdh:/dev/ssd
```

`prepare` 命令只准备 OSD。在大多数操作系统中，硬盘分区创建后，不用 `activate` 命令也会自动执行 `activate` 阶段（通过 Ceph 的 `udev` 规则）。详情见[激活 OSD](#)。

前例假定一个硬盘只会用于一个 OSD 守护进程，以及一个到 SSD 日志分区的路径。我们建议把日志存储于另外的驱动器以最优性能；你也可以指定一单独的驱动器用于日志（也许比较昂贵）、或者把日志放到 OSD 数据盘（不建议，因为它有损性能）。前例中我们把日志存储于分好区的固态硬盘。

Note

在一个节点运行多个 OSD 守护进程、且多个 OSD 守护进程共享一个日志分区时，你应该考虑整个节点的最小 CRUSH 故障域，因为如果这个 SSD 坏了，所有用其做日志的 OSD 守护进程也会失效。

激活 OSD

准备好 OSD 后，可以用下列命令激活它。

```
ceph-deploy osd activate {node-name}:{data-disk-partition}[:{journal-disk-partition}]

ceph-deploy osd activate osdserver1:/dev/sdb1:/dev/ssd1

ceph-deploy osd activate osdserver1:/dev/sdc1:/dev/ssd2
```

`activate` 命令会让 OSD 进入 `up` 且 `in` 状态，此命令所用路径和 `prepare` 相同。

创建 OSD

你可以用 `create` 命令一次完成准备 OSD、部署到 OSD 节点、并激活它。`create` 命令是依次执行 `prepare` 和 `activate` 命令的捷径。

```
ceph-deploy osd create {node-name}:{disk}[:{path/to/journal}]

ceph-deploy osd create osdserver1:sdb:/dev/ssd1
```

拆除 OSD

Note

稍后完成。手动过程见[删除 OSD](#)。

增加/拆除元数据服务器

用 `ceph-deploy` 增加和拆除元数据服务器很简单，只要一个命令就可以增加或拆除一或多个元数据服务器。

Important

你必须部署至少一个元数据服务器才能使用 CephFS 文件系统，多个元数据服务器并行运行仍处于实验阶段。不要在生产环境下运行多个元数据服务器。

增加一元数据服务器

部署完监视器和 OSD 后，还可以部署元数据服务器。

```
ceph-deploy mds create {host-name}[:{daemon-name}]
[{host-name}[:{daemon-name}] ...]
```

如果你想在同一主机上运行多个守护进程，可以为每个进程指定名字（可选）。

拆除一元数据服务器

尚未实现.....？

清除一主机

拆除 Ceph 守护进程并卸载软件包后，此主机上可能还有集群中的数据，`purge` 和 `purgedata` 命令提供了一种清理主机的简便方法。

PURGEDATA

如果只想清除 `/var/lib/ceph` 下的数据、并保留 Ceph 安装包，可以用 `purgedata` 命令：

```
ceph-deploy purgedata {hostname} [{hostname} ...]
```

PURGE

要清理掉 `/var/lib/ceph` 下的所有数据、并卸载 Ceph 软件包，用 `purge` 命令。

```
ceph-deploy purge {hostname} [{hostname} ...]
```

管理任务

用 `ceph-deploy` 建立一个集群后，你可以把客户端管理密钥和 Ceph 配置文件发给其他管理员，以便让他用 `ceph` 命令管理集群。

创建一管理主机

要允许一主机以管理员权限执行 Ceph 命令，用 `admin` 命令：

```
ceph-deploy admin {host-name} [host-name]...
```

分发配置文件

要把改过的配置文件分发给集群内各主机，可用 `config push` 命令。

```
ceph-deploy config push {host-name} [host-name]...
```

Tip

用相同前缀和递增数字命名的话，部署可以通过一个命令轻易完成，如 `ceph-deploy config hostname{1,2,3,4,5}`。

检索配置文件

要从集群内的一主机检索配置文件，用 `config pull` 命令。

```
ceph-deploy config pull {host-name [host-name]...}
```

集群运维

操纵集群

用 UPSTART 控制 CEPH

用 `ceph-deploy` 把 Ceph Cuttlefish 及更高版部署到 Ubuntu 之后，你可以用基于事件的 [Upstart](#) 来启动、关闭 *Ceph 节点* 上的守护进程。Upstart 不要求你在配置文件里定义守护进程例程。

用下列命令列出 Ceph 作业和例程：

```
sudo initctl list | grep ceph
```

详情参见 [initctl](#)。

启动所有守护进程

要启动一 Ceph 节点（任何类型）上的所有守护进程，用下列命令：

```
sudo start ceph-all
```

停止所有守护进程

要停止一 Ceph 节点（任何类型）上的所有守护进程，用下列命令：

```
sudo stop ceph-all
```

按类型启动所有守护进程

要启动一节点上的某一类守护进程，用下列命令：

```
sudo start ceph-osd-all  
  
sudo start ceph-mon-all  
  
sudo start ceph-mds-all
```

按类型停止所有守护进程

要停止一节点上的某一类守护进程，用下列命令：

```
sudo stop ceph-osd-all sudo stop ceph-mon-all sudo stop ceph-mds-all
```

启动单个进程

要启动某节点上一指定守护进程例程，用下列命令之一：

```
sudo start ceph-osd id={id}  
  
sudo start ceph-mon id={hostname}  
  
sudo start ceph-mds id={hostname}
```

例如：

```
sudo start ceph-osd id=1

sudo start ceph-mon id=ceph-server

sudo start ceph-mds id=ceph-server
```

停止单个进程

要停止某节点上一指定守护进程例程，用下列命令之一：

```
sudo stop ceph-osd id={id}

sudo stop ceph-mon id={hostname}

sudo stop ceph-mds id={hostname}
```

例如：

```
sudo stop ceph-osd id=1

sudo start ceph-mon id=ceph-server

sudo start ceph-mds id=ceph-server
```

运行 CEPH

每次用命令**启动**、**重启**、**停止** Ceph 守护进程（或整个集群）时，你必须指定至少一个选项和一个命令，还可能要指定守护进程类型或具体例程。

```
{commandline} [options] [commands] [daemons]
```

ceph 命令的选项包括：

选项	简写	描述
<code>--verbose</code>	<code>-v</code>	详细的日志。
<code>--valgrind</code>	N/A	（只适合开发者和质检人员）用 Valgrind 调试。
<code>--allhosts</code>	<code>-a</code>	在 <code>ceph.conf</code> 里配置的所有主机上执行，否则它只在本机执行。
<code>--restart</code>	N/A	核心转储后自动重启。
<code>--norestart</code>	N/A	核心转储后不自动重启。
<code>--conf</code>	<code>-c</code>	使用另外一个配置文件。

Ceph 子命令包括：

命令	描述
<code>start</code>	启动守护进程。
<code>stop</code>	停止守护进程。

命令	描述
<code>forcestop</code>	暴力停止守护进程，等价于 <code>kill -9</code>
<code>killall</code>	杀死某一类守护进程。
<code>cleanlogs</code>	清理掉日志目录。
<code>cleanalllogs</code>	清理掉日志目录内的所有文件。

至于子系统操作，`ceph` 服务能指定守护进程类型，在 `[daemons]` 处指定守护进程类型就行了，守护进程类型包括：

- `mon`
- `osd`
- `mds`

通过 SYSVINIT 机制运行 CEPH

在 CentOS、Redhat、Fedora 和 SLES 发行版上可以通过传统的 `sysvinit` 运行 Ceph，Debian/Ubuntu 的较老的版本也可以用此方法。

启动所有守护进程

要启动 Ceph 集群，执行 `ceph` 时加上 `start` 命令，语法如下：

```
sudo /etc/init.d/ceph [options] [start|restart]
[daemonType|daemonID]
```


下面是个典型实例：

```
sudo /etc/init.d/ceph -a start
```

加 `-a`（即在所有节点上执行）执行完成后 Ceph 应该开始运行了。

停止所有守护进程

要停止 Ceph 集群，执行 `ceph` 时加上 `stop` 命令，语法如下：

```
sudo /etc/init.d/ceph [options] stop [daemonType|daemonID]
```

下面是个典型实例：

```
sudo /etc/init.d/ceph -a stop
```

执行命令时一旦加了 `-a`（即在所有节点执行），整个 Ceph 集群都会关闭。

启动一类守护进程

要启动本节点上某一类的所有 Ceph 守护进程，按此语法：

```
sudo /etc/init.d/ceph start {daemon-type}

sudo /etc/init.d/ceph start osd
```

要启动非本机节点上某一类的所有 Ceph 守护进程，按此语法：

```
sudo /etc/init.d/ceph -a start {daemon-type}

sudo /etc/init.d/ceph -a start osd
```

停止一类守护进程

要停止本节点上某一类的所有 Ceph 守护进程，按此语法：

```
sudo /etc/init.d/ceph stop {daemon-type} sudo /etc/init.d/ceph stop osd
```

要启动非本机节点上某一类的所有 Ceph 守护进程，按此语法：

```
sudo /etc/init.d/ceph -a stop {daemon-type}

sudo /etc/init.d/ceph -a stop osd
```

启动单个守护进程

要启动本节点上某个 Ceph 守护进程，按此语法：

```
sudo /etc/init.d/ceph start {daemon-type}.{instance}

sudo /etc/init.d/ceph start osd.0
```

要启动另一节点上某个 Ceph 守护进程，按此语法：

```
sudo /etc/init.d/ceph -a start {daemon-type}.{instance}

sudo /etc/init.d/ceph -a start osd.0
```

停止单个守护进程

要停止本节点上某个 Ceph 守护进程，按此语法：

```
sudo /etc/init.d/ceph stop {daemon-type}.{instance}

sudo /etc/init.d/ceph stop osd.0
```

要停止另一节点上某个 Ceph 守护进程，按此语法：

```
sudo /etc/init.d/ceph -a stop {daemon-type}.{instance}

sudo /etc/init.d/ceph -a stop osd.0
```

把 CEPH 当服务运行

如果你是用 `ceph-deploy` 部署 Argonaut 或 Bobtail 的，那么 Ceph 可以作为服务运行（还可以用 `sysvinit`）。

启动所有守护进程

要启动你的 Ceph 集群，执行 `ceph` 时加上 `start` 命令，按此语法：

```
sudo service ceph [options] [start|restart]
[daemonType|daemonID]
```

下面是个典型实例：

```
sudo service ceph -a start
```

加 `-a`（即在所有节点上执行）执行完成后 Ceph 应该开始运行了。

停止所有守护进程

要停止你的 Ceph 集群，执行 `ceph` 时加上 `stop` 命令，按此语法：

```
sudo service ceph [options] stop [daemonType|daemonID]
```

例如：

```
sudo service ceph -a stop
```

加 `-a`（即在所有节点上执行）执行完成后 Ceph 应该已停止。

启动一类守护进程

要启动本节点上某一类的所有 Ceph 守护进程，按此语法：

```
sudo service ceph start {daemon-type}

sudo service ceph start osd
```

要启动所有节点上某一类的所有 Ceph 守护进程，按此语法：

```
sudo service ceph -a start {daemon-type}

sudo service ceph -a start osd
```

停止一类守护进程

要停止本节点上某一类的所有 Ceph 守护进程，按此语法：

```
sudo service ceph stop {daemon-type}

sudo service ceph stop osd
```

要停止所有节点上某一类的所有 Ceph 守护进程，按此语法：

```
sudo service ceph -a stop {daemon-type}

sudo service ceph -a stop osd
```

启动单个守护进程

要启动本节点上某个 Ceph 守护进程，按此语法：

```
sudo service ceph start {daemon-type}.{instance}

sudo service ceph start osd.0
```

要启动另一节点上某个 Ceph 守护进程，按此语法：

```
sudo service ceph -a start {daemon-type}.{instance}

sudo service ceph -a start osd.0
```

停止单个守护进程

要停止本节点上某个 Ceph 守护进程，按此语法：

```
sudo service ceph stop {daemon-type}.{instance}

sudo service ceph stop osd.0
```

要停止另一节点上某个 Ceph 守护进程，按此语法：

```
sudo service ceph -a stop {daemon-type}.{instance}

sudo service ceph -a stop osd.0
```

监控集群

集群运行起来后，你可以用 `ceph` 工具来监控，典型的监控包括检查 OSD 状态、监视器状态、归置组状态和元数据服务器状态。

交互模式

要在交互模式下运行 `ceph`，不要带参数运行 `ceph`，例如：

```
ceph

ceph> health

ceph> status

ceph> quorum_status

ceph> mon_status
```

检查集群健康状况

启动集群后、读写数据前，先检查下集群的健康状态。你可以用下面的命令检查：

```
ceph health
```

如果你的配置文件或密钥环不在默认路径下，你得指定：

```
ceph -c /path/to/conf -k /path/to/keyring health
```

集群起来的时候，你也许会碰到

像 `HEALTH_WARN XXX num placement groups stale` 这样的健康告警，等一会再检查下。集群准备好的话 `cephhealth` 会给出像 `HEALTH_OK` 一样的消息，这时候就可以开始使用集群了。

观察集群

要观察集群内正发生的事件，打开一个新终端，然后输入：

```
ceph -w
```

Ceph 会打印各种事件。例如一个包括 1 个监视器、和 2 个 OSD 的小型 Ceph 集群可能会打印出这些：

```
cluster b370a29d-9287-4ca3-ab57-3d824f65e339

health HEALTH_OK

monmap e1: 1 mons at {ceph1=10.0.0.8:6789/0}, election
epoch 2, quorum 0 ceph1

osdmap e63: 2 osds: 2 up, 2 in

pgmap v41338: 952 pgs, 20 pools, 17130 MB data, 2199
objects

    115 GB used, 167 GB / 297 GB avail

    952 active+clean

2014-06-02 15:45:21.655871 osd.0 [INF] 17.71 deep-scrub
ok

2014-06-02 15:45:47.880608 osd.1 [INF] 1.0 scrub ok

2014-06-02 15:45:48.865375 osd.1 [INF] 1.3 scrub ok

2014-06-02 15:45:50.866479 osd.1 [INF] 1.4 scrub ok

2014-06-02 15:45:01.345821 mon.0 [INF] pgmap v41339: 952
pgs: 952 active+clean; 17130 MB data, 115 GB used, 167 GB
/ 297 GB avail
```

```
2014-06-02 15:45:05.718640 mon.0 [INF] pgmap v41340: 952
pgs: 1 active+clean+scrubbing+deep, 951 active+clean;
17130 MB data, 115 GB used, 167 GB / 297 GB avail

2014-06-02 15:45:53.997726 osd.1 [INF] 1.5 scrub ok

2014-06-02 15:45:06.734270 mon.0 [INF] pgmap v41341: 952
pgs: 1 active+clean+scrubbing+deep, 951 active+clean;
17130 MB data, 115 GB used, 167 GB / 297 GB avail

2014-06-02 15:45:15.722456 mon.0 [INF] pgmap v41342: 952
pgs: 952 active+clean; 17130 MB data, 115 GB used, 167 GB
/ 297 GB avail

2014-06-02 15:46:06.836430 osd.0 [INF] 17.75 deep-scrub
ok

2014-06-02 15:45:55.720929 mon.0 [INF] pgmap v41343: 952
pgs: 1 active+clean+scrubbing+deep, 951 active+clean;
17130 MB data, 115 GB used, 167 GB / 297 GB avail
```

输出信息里包含：

- 集群唯一标识符
- 集群健康状况
- 监视器图元版本、和监视器法定人数状态
- OSD 图元版本、和 OSD 状态摘要
- 归置组图版本
- 归置组和存储池数量
- 其内存储的数据和对象数量的粗略统计，以及
- 数据总量

Ceph 如何计算数据量

used 值反映了确实已占用的原始存储空间。xxx GB / xxx GB 值则是剩余空间（较小的数）与集群总容量的比较。理论数值反映了所存储数据的原始尺寸，未计算其副本、克隆、或快照空间，所以数据存储实际占用的空间通常会超过理论数值，因为 Ceph 会自动创建数据副本，另外存储空间也可能用于克隆和快照。

检查集群的使用情况

要检查集群的数据用量及其在存储池内的分布情况，可以用 `df` 选项，它和 Linux 上的 `df` 相似。如下：

```
ceph df
```

输出的 **GLOBAL** 段展示了数据所占用集群存储空间的概要。

- **SIZE:** 集群的总容量；
- **AVAIL:** 集群的空闲空间总量；
- **RAW USED:** 已用存储空间总量；
- **% RAW USED:** 已用存储空间比率。用此值参照 `full ratio` 和 `near full \ ratio` 来确保不会用尽集群空间。详情见[存储容量](#)。

输出的 **POOLS** 段展示了存储池列表及各存储池的大致使用率。本段**没有**展示副本、克隆品和快照占用情况。例如，如果你把 1MB 的数据存储为对象，理论使用率将是 1MB，但考虑到副本数、克隆数、和快照数，实际使用率可能是 2MB 或更多。

- **NAME:** 存储池名字；
- **ID:** 存储池唯一标识符；
- **USED:** 大概数据量，单位为 KB、MB 或 GB；
- **%USED:** 各存储池的大概使用率；
- **Objects:** 各存储池内的大概对象数。

Note

POOLS 段内的数字是理论值，它们不包含副本、快照或克隆。因此，它与 **USED** 和 **%USED** 数量之和不会达到 **GLOBAL** 段中的 **RAW USED** 和 **%RAW USED** 数量。

检查集群状态

要检查集群的状态，执行下面的命令：

```
ceph status
```

或者：

```
ceph -s
```

在交互模式下，输入 `status` 然后按回车：

```
ceph> status
```

Ceph 将打印集群状态，例如一个包括 1 个监视器、和 2 个 OSD 的小型 Ceph 集群可能打印：

```
cluster b370a29d-9287-4ca3-ab57-3d824f65e339

health HEALTH_OK

monmap e1: 1 mons at {ceph1=10.0.0.8:6789/0}, election
epoch 2, quorum 0 ceph1

osdmap e63: 2 osds: 2 up, 2 in

pgmap v41332: 952 pgs, 20 pools, 17130 MB data, 2199
objects

      115 GB used, 167 GB / 297 GB avail

      1 active+clean+scrubbing+deep

      951 active+clean
```

检查 OSD 状态

你可以执行下列命令来确定 OSD 状态为 `up` 且 `in`：

```
ceph osd stat
```

或者：

```
ceph osd dump
```

你也可以根据 OSD 在 CRUSH 图里的位置来查看：

```
ceph osd tree
```

Ceph 会打印 CRUSH 的树状态、它的 OSD 例程、状态、权重：

#	id	weight	type	name	up/down	reweight
-1	3		pool	default		
-3	3		rack	mainrack		
-2	3		host	osd-host		
0	1				osd.0 up	1
1	1				osd.1 up	1
2	1				osd.2 up	1

个中详情见[监控 OSD 和归置组](#)。

检查监视器状态

如果你有多个监视器（很可能），你启动集群后、读写数据前应该检查监视器法定人数状态。运行着多个监视器时必须形成法定人数，最好周期性地检查监视器状态来确定它们在运行。

要查看监视器图，执行下面的命令：

```
ceph mon stat
```

或者：

```
ceph mon dump
```

要检查监视器的法定人数状态，执行下面的命令：

```
ceph quorum_status
```

Ceph 会返回法定人数状态，例如，包含 3 个监视器的 Ceph 集群可能返回下面的：

```
{ "election_epoch": 10,

  "quorum": [

    0,

    1,

    2],

  "monmap": { "epoch": 1,

    "fsid": "444b489c-4f16-4b75-83f0-cb8097468898",

    "modified": "2011-12-12 13:28:27.505520",

    "created": "2011-12-12 13:28:27.505520",

    "mons": [

      { "rank": 0,

        "name": "a",
```

```
        "addr": "127.0.0.1:6789\0"},
    { "rank": 1,
      "name": "b",
      "addr": "127.0.0.1:6790\0"},
    { "rank": 2,
      "name": "c",
      "addr": "127.0.0.1:6791\0"}
  ]
}
}
```

检查 MDS 状态

元数据服务器为 Ceph 文件系统提供元数据服务，元数据服务器有两种状态：up | down 和 active | inactive，执行下面的命令查看元数据服务器状态为 up 且 active：

```
ceph mds stat
```

要展示元数据集群的详细状态，执行下面的命令：

```
ceph mds dump
```

检查归置组状态

归置组把对象映射到 OSD。监控归置组时，我们希望它们的状态是 `active` 且 `clean`。个中详情见[监控 OSD 和归置组](#)。

使用管理套接字

Ceph 管理套接字允许你通过套接字接口查询守护进程，它们默认存在于 `/var/run/ceph` 下。要通过管理套接字访问某个守护进程，先登录它所在的主机、再执行下列命令：

```
ceph daemon {daemon-name}  
  
ceph daemon {path-to-socket-file}
```

比如，这是下面这两种用法是等价的：

```
ceph daemon osd.0 foo  
  
ceph daemon /var/run/ceph/ceph-osd.0.asok foo
```

用下列命令查看可用的管理套接字命令：

```
ceph daemon {daemon-name} help
```

管理套接字命令允许你在运行时查看和修改配置，见[查看运行时配置](#)。

另外，你可以在运行时直接修改配置选项（也就是说管理套接字会绕过监视器，不要求你直接登录宿主主机，不像 `ceph {daemon-type} tell {id} injectargs` 依赖监视器。

监控 OSD 和归置组

高可用性和高可靠性要求容错方法来管理软硬件。Ceph 没有单故障点，并且能在“降级”模式下继续提供服务。其数据归置引入了一个间接层，它可保证数据不会直接绑死到某一个特定 OSD 地址，这也意味着追踪系统错误的根源得深入归置组及底层的 OSD。

Tip

集群某一部分失效可能导致不能访问某个对象，但不会牵连其他对象。碰到这种问题时无需恐慌，只需按步骤检查 OSD 和归置组，然后排除故障。

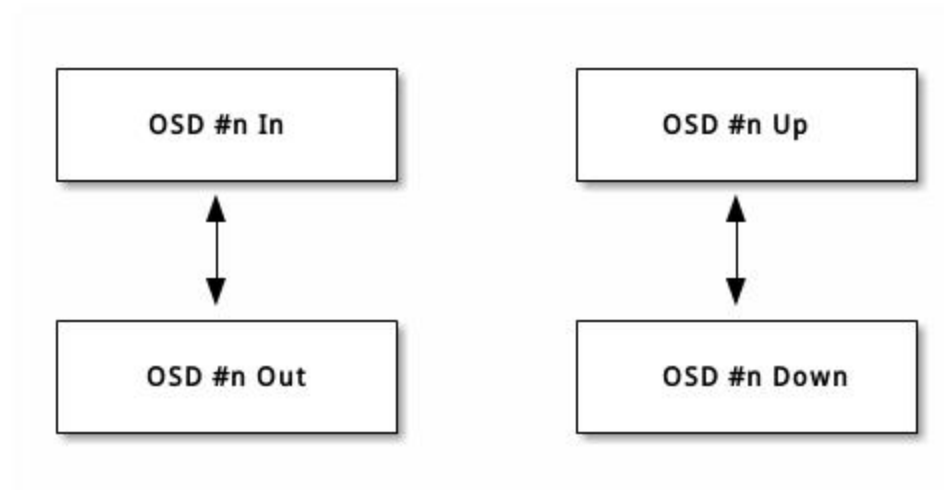
Ceph 通常能自己康复，然而如果故障持续存在，监控 OSD 和归置组有助于找出问题所在。

监控 OSD

某 OSD 的状态可以是在集群内 (in) 或集群外 (out)、也可以是活着且在运行 (up) 或挂了且不在运行 (down)。如果一个 OSD 活着，它也可以是 in (你可以读写数据) 或者 out 集群。如果它以前是 in 但最近 out 了，Ceph 会把其归置组迁移到其他 OSD。如果一 OSD out 了，CRUSH 就不会再分配归置组给它。如果它挂了 (down) 其状态也应该是 out。

Note

如果一 OSD 状态为 down 且 in，必定有问题，而且集群处于非健康状态。



如果你执行过这些命令，如 `ceph health`、`ceph -s`、或 `ceph -w`，也许注意到了，集群并非一直返回 `HEALTH OK`，别紧张。就 OSD 而言你应该明确，在一些情况下集群不会返回 `HEALTH OK`：

1. 你还没启动集群（它不会响应的）。
2. 你刚刚启动或重启完集群，而且它还没准备好，因为归置组正被创建、OSD 们正在相互建立连接。
3. 你刚刚增加或拆除一个 OSD。
4. 你刚刚修改完集群运行图。

OSD 监控的一个重要事情是，当集群启动并运行时，所有 OSD 也应该是启动（`up`）并在集群内（`in`）运行。用下列命令查看：

```
ceph osd stat
```

其结果会告诉你运行图版本（`eNNNN`）、总共有 `x` 个 OSD、`y` 个是 `up` 的、`z` 个是 `in` 的。

`eNNNN: x osds: y up, z in`

如果处于 `in` 状态的 OSD 多于 `up` 的，用下列命令看看哪些 `ceph-osd` 守护进程没在运行：

```
ceph osd tree ::
```



```
dumped osdmap tree epoch 1

# id      weight  type name          up/down reweight
-1        2      pool openstack
-3        2      rack dell-2950-rack-A
-2        2      host dell-2950-A1
0         1      osd.0  up             1
1         1      osd.1  down           1
```

Tip

精心设计的 CRUSH 分级结构可以帮你更快的定位到物理位置、加快故障排除。

若一个 OSD 状态为 `down`，启动它：

```
sudo /etc/init.d/ceph -a start osd.1
```

和 OSD 没运行或不启动相关的问题请看 [OSD 没运行](#)。

归置组集

CRUSH 要把归置组分配到 OSD 时，它先查询这个存储池的副本数设置，再把归置组分配到 OSD，这样就把各副本分配到了不同 OSD。比如，如果存储池要求归置组有 3 个副本，CRUSH 可能把它们分别分配到 `osd.1`、`osd.2`、`osd.3`。考虑到你设置于 [CRUSH 运行图](#) 中的失败域，实际上 CRUSH 找出的是伪随机位置，所以在大型集群中，你很少能看到归置组被分配到了相邻的 OSD。我们把涉及某个特定归置组副本的一组 OSD 称为 **acting set**。在一些情况下，位于 acting set 中的一个 OSD `down` 了或者不能为归置组内的对象提供服务，这些情形发生时无需惊慌，常见原因如下：

- 你增加或拆除了一 OSD 。然后 CRUSH 把那个归置组分配到了其他 OSD ， 因此改变了 Acting Set 的构成、并且用 backfill 进程启动了数据迁移；
- 一 OSD down 了、重启了、而现在正恢复（ recovering ）；
- acting set 中的一个 OSD 挂了，不能提供服务，另一个 OSD 临时接替其工作。

Ceph 靠 **up set** 处理客户端请求，它们是最终处理请求的一系列 OSD 。大多数情况下 up set 和 acting set 本质上相同，如果不同，说明可能 Ceph 在迁移数据、某 OSD 在恢复、或者哪里有问题。这种情况下，Ceph 通常表现为 HEALTH_WARN 状态，还有“stuck stale”消息。

用下列命令获取归置组列表：

```
ceph pg dump
```

要根据指定归置组号查看哪些 OSD 位于 Acting Set 或 Up Set 里，执行：

```
ceph pg map {pg-num}
```

其结果会告诉你 osdmap 版本（ eNNN ）、归置组号（ {pg-num} ）、Up Set 内的 OSD（ up[] ）、和 Acting Set 内的 OSD（ acting[] ）。

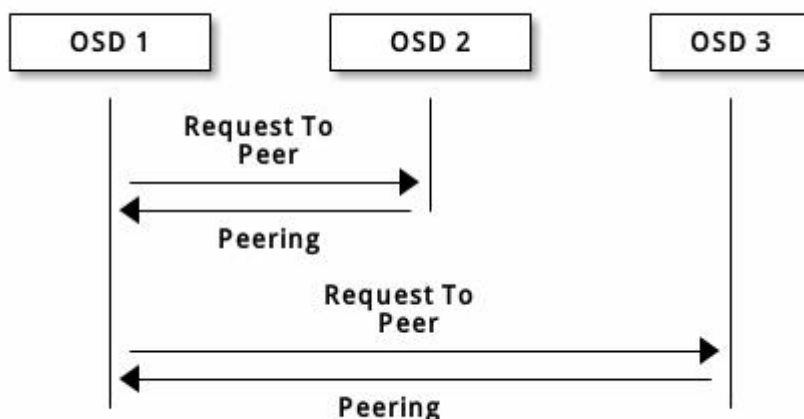
```
osdmap eNNN pg {pg-num} -> up [0,1,2] acting [0,1,2]
```

Note

如果 Up Set 和 Acting Set 不一致，这可能表明集群内部在重均衡或者有潜在问题。

节点互联

写入数据前，归置组必须处于 active、而且**应该是** clean 状态。假设一存储池的归置组有 3 个副本，为让 Ceph 确定归置组的当前状态，一归置组的主 OSD（即 acting set 内的第一个 OSD）会与第二和第三 OSD 建立连接、并就归置组的当前状态达成一致意见。



OSD 们也向监视器报告自己的状态，详情见[监视器与 OSD 交互的配置](#)。要排除连接建立问题，参见[互联失败](#)。

监控归置组状态

如果你执行过 `ceph health`、`ceph -s`、或 `ceph -w` 命令，你也许注意到了集群并非总返回 `HEALTH OK`。检查完 OSD 是否在运行后，你还应该检查归置组状态。你应该明白，在归置组建立连接时集群**不会**返回 `HEALTH OK`：

1. 刚刚创建了一个存储池，归置组还没互联好；
2. 归置组正在恢复；
3. 刚刚增加或删除了一个 OSD；
4. 刚刚修改了 CRUSH 图，并且归置组正在迁移；
5. 某一归置组的副本间的数据不一致；
6. Ceph 正在洗刷一个归置组的副本；
7. Ceph 没有足够空余容量来完成回填操作。

如果是前述原因之一导致了 Ceph 返回 `HEALTH WARN`，无需紧张。很多情况下，集群会自行恢复；有些时候你得采取些措施。归置组监控的一件重要的事情是保证集群起来并运行着，所有归置组都处于 `active` 状态、并且最好是 `clean` 状态。用下列命令查看所有归置组状态：

```
ceph pg stat
```

其结果会告诉你归置组运行图的版本号 (vNNNNNN)、归置组总数 x、有多少归置组处于某种特定状态，如 active+clean (y)。

```
vNNNNNN: x pgs: y active+clean; z bytes data, aa MB used,
bb GB / cc GB avail
```

Note

Ceph 同时报告出多种状态是正常的。

除了归置组状态之外，Ceph 也会报告数据占据的空间 (aa)、剩余空间 (bb) 和归置组总容量。这些数字在某些情况下是很重要的：

- 快达到 ``near full ratio`` 或 ``full ratio`` 时；
- 由于 CRUSH 配置错误致使你的数据没能在集群内分布。

归置组唯一标识符

归置组 ID 由存储池号 (不是存储池名字)、后面跟一个点 (.)、然后是归置组 ID，它是一个十六进制数字。用 `ceph osd lspools` 可查看存储池号及其名字，例如，默认存储池 rbd 对应的存储池号是 0。完整的归置组 ID 格式如下：

```
{pool-num}.{pg-id}
```

典型长相：

```
0.1f
```

用下列命令获取归置组列表：

```
ceph pg dump
```

你也可以让它输出到 JSON 格式，并保存到文件：

```
ceph pg dump -o {filename} --format=json
```

要查询某个归置组，用下列命令：

```
ceph pg {poolnum}.{pg-id} query
```

Ceph 会输出成 JSON 格式。

```
{  
  
  "state": "active+clean",  
  
  "up": [  
  
    1,  
  
    0  
  
  ],  
  
  "acting": [  
  
    1,  
  
    0  
  
  ],  
  
  "info": {  
  
    "pgid": "1.e",
```

```
"last_update": "4'1",

"last_complete": "4'1",

"log_tail": "0'0",

"last_backfill": "MAX",

"purged_snaps": "[]",

"history": {

  "epoch_created": 1,

  "last_epoch_started": 537,

  "last_epoch_clean": 537,

  "last_epoch_split": 534,

  "same_up_since": 536,

  "same_interval_since": 536,

  "same_primary_since": 536,

  "last_scrub": "4'1",

  "last_scrub_stamp": "2013-01-25 10:12:23.828174"

},

"stats": {

  "version": "4'1",

  "reported": "536'782",

  "state": "active+clean",
```

```
"last_fresh": "2013-01-25 10:12:23.828271",  
"last_change": "2013-01-25 10:12:23.828271",  
"last_active": "2013-01-25 10:12:23.828271",  
"last_clean": "2013-01-25 10:12:23.828271",  
"last_unstale": "2013-01-25 10:12:23.828271",  
"mapping_epoch": 535,  
"log_start": "0'0",  
"ondisk_log_start": "0'0",  
"created": 1,  
"last_epoch_clean": 1,  
"parent": "0.0",  
"parent_split_bits": 0,  
"last_scrub": "4'1",  
"last_scrub_stamp": "2013-01-25 10:12:23.828174",  
"log_size": 128,  
"ondisk_log_size": 128,  
"stat_sum": {  
    "num_bytes": 205,  
    "num_objects": 1,  
    "num_object_clones": 0,
```

```
    "num_object_copies": 0,  
  
    "num_objects_missing_on_primary": 0,  
  
    "num_objects_degraded": 0,  
  
    "num_objects_unfound": 0,  
  
    "num_read": 1,  
  
    "num_read_kb": 0,  
  
    "num_write": 3,  
  
    "num_write_kb": 1  
  
  },  
  
  "stat_cat_sum": {  
  
  },  
  
  "up": [  
  
    1,  
  
    0  
  
  ],  
  
  "acting": [  
  
    1,  
  
    0  
  
  ]  
}
```



```
    },  
  
    "empty": 0,  
  
    "dne": 0,  
  
    "incomplete": 0  
  
},  
  
"recovery_state": [  
  
    {  
  
        "name": "Started\\Primary\\Active",  
  
        "enter_time": "2013-01-23 09:35:37.594691",  
  
        "might_have_unfound": [  
  
        ],  
  
        "scrub": {  
  
            "scrub_epoch_start": "536",  
  
            "scrub_active": 0,  
  
            "scrub_block_writes": 0,  
  
            "finalizing_scrub": 0,  
  
            "scrub_waiting_on": 0,  
  
            "scrub_waiting_on_whom": [  

```

```
    ]

    }

  },

  {

    "name": "Started",

    "enter_time": "2013-01-23 09:35:31.581160"

  }

]

}
```

后续子章节详述了常见状态。

存储池在建中

创建存储池时，它会创建指定数量的归置组。Ceph 在创建一或多个归置组时会显示 `creating`；创建完后，在其归置组的 Acting Set 里的 OSD 将建立互联；一旦互联完成，归置组状态应该变为 `active+clean`，意思是 Ceph 客户端可以向归置组写入数据了。



互联建立中

Ceph 为归置组建立互联时，会让存储归置组副本的 OSD 之间就其中的对象和元数据状态**达成一致**。Ceph 完成了互联，也就意味着存储着归置组的 OSD 就其当前状态达成了一致。然而，互联过程的完成并**不能**表明各副本都有了数据的最新版本。

权威历史

Ceph **不会**向客户端确认写操作，直到 acting set 里的所有 OSD 都完成了写操作。这样处理保证了从上次成功互联起，acting set 中至少有一个成员确认了每个写操作。

有了各个已确认写操作的精确记录，Ceph 就可以构建和散布一个新的归置组权威历史——一个完整、完全有序的操作集，如果被采用，就能把一个 OSD 的归置组副本更新到最新。

活跃

Ceph 完成互联后，一归置组状态会变为 `active`。`active` 状态意味着数据已完好地保存到了主归置组和副本归置组。

整洁

某一归置组处于 `clean` 状态时，主 OSD 和副本 OSD 已成功互联，并且没有偏离的归置组。Ceph 已把归置组中的对象复制了规定次数。

已降级

当客户端向主 OSD 写入数据时，由主 OSD 负责把数据副本写入其余副本 OSD。主 OSD 把对象写入存储器后，在副本 OSD 创建完对象副本并报告给主 OSD 之前，主 OSD 会一直停留在 `degraded` 状态。

归置组状态可以处于 `active+degraded` 状态，原因在于一 OSD 即使尚未持有所有对象也可以处于 `active` 状态。如果一 OSD 挂了，Ceph 会把分配到此 OSD 的归置组都标记为 `degraded`；那个 OSD 重生后，它们必须重新互联。然而，客户端仍可以向处于 `degraded` 状态的归置组写入新对象，只要它还在 `active` 状态。

如果一 OSD 挂了，且老是处于 `degraded` 状态，Ceph 会把 `down` 的 OSD 标记为在集群外 (`out`)、并把那个 `down` 掉的 OSD 上的数据重映射到其它 OSD。从标记为 `down` 到 `out` 的时间间隔由 `mon osd down out interval` 控制，默认是 300 秒。

归置组也会被降级 (`degraded`)，因为 Ceph 找不到本应存在于此归置组中的一或多个对象，这时，你不能读写找不到的对象，但仍能访问位于降级归置组中的其它对象。

恢复中

Ceph 被设计为可容错，可抵御一定规模的软、硬件问题。当某 OSD 挂了 (`down`) 时，其内的归置组会落后于别的归置组副本；此 OSD 重生 (`up`) 时，归置组内容必须更新到当前状态；在此期间，OSD 处于 `recovering` 状态。

恢复并非总是这些小事，因为一次硬件失败可能牵连多个 OSD。比如一个机柜或房间的网络交换机失败了，这会导致多个主机上的 OSD 落后于集群的当前状态，故障恢复后每一个 OSD 都必须恢复。

Ceph 提供了几个选项来均衡资源竞争，如新服务请求、恢复数据对象和恢复归置组到当前状态。`osd recovery delay start` 选项允许一 OSD 在开始恢复进程前，先重启、重建互联、甚至处理一些重放请求；`osd recovery threads` 选项限制恢复进程的线程数，默认为 1 线程；`osd recoverythread timeout` 设置线程超时，因为多个 OSD 可能交替失败、重启和重建互联；`osd recovery max active` 选项限制一 OSD 最多同时接受多少请求，以防它压力过大而不能正常服务；`osd recovery max chunk` 选项限制恢复数据块尺寸，以防网络拥塞。

回填中

有新 OSD 加入集群时，CRUSH 会把现有集群内的部分归置组重分配给它。强制新 OSD 立即接受重分配的归置组会使之过载，用归置组回填可使这个过程在后台开始。只要回填顺利完成，新 OSD 就可以对外服务了。

在回填运转期间，你可能见到以下几种状态之一：`backfill_wait` 表明一回填操作在等待时机，尚未开始；`backfill` 表明一回填操作正在进行；`backfill_too_full` 表明需要进行回填，但是因存储空间不足而不能完成。某归置组不能回填时，其状态应该是 `incomplete`。

Ceph 提供了多个选项来解决重分配归置组给一 OSD（特别是新 OSD）时相关的负载问题。默认，`osd_max_backfills` 把双向的回填并发量都设置为 10；`osd backfill full \ ratio` 可让一 OSD 在接近占满率（默认 85%）时拒绝回填请求，如果一 OSD 拒绝了回填请求，在 `osdbackfill retry interval` 间隔之后将重试（默认 10 秒）；OSD 也能用 `osd backfill scan min` 和 `osd backfill scan max` 来管理扫描间隔（默认 64 和 512）。

被重映射

负责维护某一归置组的 Acting Set 变更时，数据要从旧集合迁移到新的。新的主 OSD 要花费一些时间才能提供服务，所以老的主 OSD 还要持续提供服务、直到归置组迁移完。数据迁移完后，运行图会包含新 acting set 里的主 OSD。

发蔫

虽然 Ceph 用心跳来保证主机和守护进程在运行，但是 `ceph-osd` 仍有可能进入 `stuck` 状态，它们没有按时报告其状态（如网络瞬断）。默认，OSD 守护进程每半秒（0.5）会一次报告其归置组、出流量、引导和失败统计状态，此频率高于心跳阈值。如果一归置组的主 OSD 所在的 acting set 没能向监视器报告、或者其它监视器已经报告了那个主 OSD 已 `down`，监视器们就会把此归置组标记为 `stale`。

启动集群时，会经常看到 `stale` 状态，直到互联完成。集群运行一阵后，如果还能看到有归置组位于 `stale` 状态，就说明那些归置组的主 OSD 挂了（`down`）、或没在向监视器报告统计信息。

找出故障归置组

如前所述，一个归置组状态不是 `active+clean` 时未必有问题。一般来说，归置组卡住时 Ceph 的自修复功能往往无能为力，卡住的状态细分为：

- **Unclean:** 归置组里有些对象的副本数未达到期望次数，它们应该在恢复中；
- **Inactive:** 归置组不能处理读写请求，因为它们在等着一个持有最新数据的 OSD 回到 `up` 状态；
- **Stale:** 归置组们处于一种未知状态，因为存储它们的 OSD 有一阵子没向监视器报告了（由 `mon osd report timeout` 配置）。

为找出卡住的归置组，执行：

```
ceph pg dump_stuck
[unclean|inactive|stale|undersized|degraded]
```

详情见[归置组子系统](#)，关于排除卡住的归置组见[排除归置组错误](#)。

定位对象

要把对象数据存入 Ceph 对象存储，一 Ceph 客户端必须：

1. 设置对象名
2. 指定一 **存储池**

Ceph 客户端索取最新集群运行图、并用 CRUSH 算法计算对象到 **归置组** 的映射，然后计算如何动态地把归置组映射到 OSD。要定位对象，只需要知道对象名和存储池名字，例如：

```
ceph osd map {poolname} {object-name}
```

练习：定位一个对象

反正是练习，我们先创建一个对象。给 `rados put` 命令指定一对象名、一个包含数据的测试文件路径、和一个存储池名字，例如：

```
rados put {object-name} {file-path} --pool=data  
  
rados put test-object-1 testfile.txt --pool=data
```

用下列命令确认 Ceph 对象存储已经包含此对象：

```
rados -p data ls
```

现在可以定位对象了：

```
ceph osd map {pool-name} {object-name}  
  
ceph osd map data test-object-1
```

Ceph 应该输出对象的位置，例如：

```
osdmap e537 pool 'data' (0) object 'test-object-1' -> pg
0.d1743484 (0.4) -> up [1,0] acting [1,0]
```

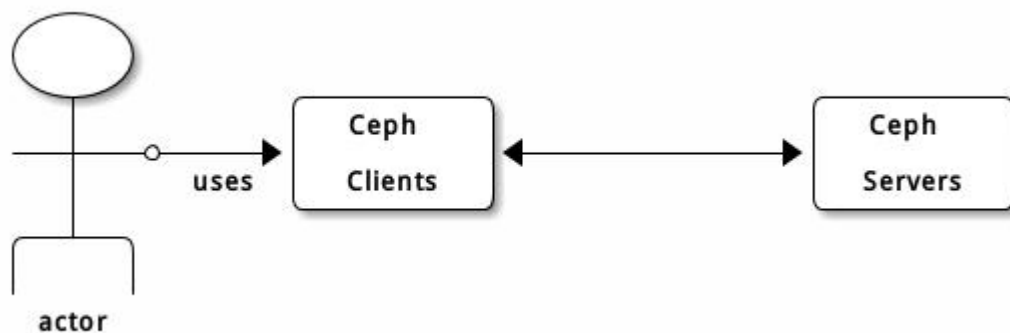
要删除测试对象，用 `rados rm` 即可，如：

```
rados rm test-object-1 --pool=data
```

随着集群的运转，对象位置会动态改变。Ceph 动态重均衡的优点之一，就是把你从人工迁移中解放了，详情见[体系结构](#)。

用户管理

本文档叙述了 **Ceph 客户端** 的用户身份，及其与 **Ceph 存储集群** 的认证和授权。用户可以是个人或系统角色（像应用程序），它们用 Ceph 客户端和 Ceph 服务器守护进程交互。



When Ceph runs with authentication and authorization enabled (enabled by default), you must specify a user name and a keyring containing the secret key of the specified user (usually via the command line). If you do not specify a user name, Ceph will use `client.admin` as the default user name. If you do not specify a keyring, Ceph will look for a keyring via the `keyring` setting in the Ceph configuration. For example, if you execute the `cephhealth` command without specifying a user or keyring:

```
ceph health
```

Ceph interprets the command like this:

```
ceph -n client.admin --  
keyring=/etc/ceph/ceph.client.admin.keyring health
```

另外你也可以用 `CEPH_ARGS` 环境变量来避免多次输入用户名和密钥。

For details on configuring the Ceph Storage Cluster to use authentication, see [Cephx 配置参考](#). For details on the architecture of Cephx, see [体系结构——高可用性认证](#).

背景

Irrespective of the type of Ceph client (e.g., Block Device, Object Storage, Filesystem, native API, etc.), Ceph stores all data as objects within [pools](#). Ceph users must have access to pools in order to read and write data. Additionally, Ceph users must have execute permissions to use Ceph's administrative commands. The following concepts will help you understand Ceph user management.

用户

A user is either an individual or a system actor such as an application. Creating users allows you to control who (or what) can access your Ceph Storage Cluster, its pools, and the data within pools.

Ceph has the notion of a `type` of user. For the purposes of user management, the type will always be `client`. Ceph identifies users in period (.) delimited form consisting of the user type and the user ID: for example, `TYPE.ID`, `client.admin`, or `client.user1`. The reason for user typing is that Ceph Monitors, OSDs, and Metadata Servers also use the Cephx protocol, but they are not clients. Distinguishing the user type helps to distinguish between client users and other users—streamlining access control, user monitoring and traceability.

Sometimes Ceph's user type may seem confusing, because the Ceph command line allows you to specify a user with or without the type, depending upon your command line usage. If you specify `--user` or `--id`, you can omit the type. So `client.user1` can be entered simply as `user1`. If you specify `--name` or `-n`, you must specify the type and name, such as `client.user1`. We recommend using the type and name as a best practice wherever possible.

Note

A Ceph Storage Cluster user is not the same as a Ceph Object Storage user or a Ceph Filesystem user. The Ceph Object Gateway uses a Ceph Storage Cluster user to communicate between the gateway daemon and the storage cluster, but the gateway has its own user management functionality for end users. The Ceph Filesystem uses POSIX semantics. The user space associated with the Ceph Filesystem is not the same as a Ceph Storage Cluster user.

授权（能力）

Ceph 用能力（capabilities, caps）这个术语来描述给认证用户的授权，这样才能使用监视器、OSD、和元数据服务器的功能。能力也用于限制对一存储池内的数据或某个名字空间的访问。Ceph 的管理用户可在创建或更新某用户时赋予他能力。

能力的语法符合下面的形式：

```
{daemon-type} 'allow {capability}' [{daemon-type} 'allow {capability}']
```

- **监视器能力**：监视器能力包括 `r`、`w`、`x` 和 `allow profile {cap}`，例如：

```
• mon 'allow rwx'
```

```
mon 'allow profile osd'
```

- **OSD 能力：**OSD 能力包括 `r`、`w`、`x`、`class-read`、`class-write` 和 `profile osd`。另外，OSD 能力还支持存储池和命名空间的配置。

```
osd 'allow {capability}' [pool={poolname}]  
[namespace={namespace-name}]
```

- **元数据服务器能力：**元数据服务器能力比较简单，只需要 `allow` 或者空白，也不会解析更多选项。

```
mds 'allow'
```

Note

Ceph 对象网关守护进程 (`radosgw`) 是 Ceph 存储集群的一种客户端，所以它没被表示成一种独立的 Ceph 存储集群守护进程类型。

下面描述了各种能力。

`allow`

描述： 在守护进程的访问设置之前，仅对 MDS 隐含 `rw`。

`r`

描述： 授予用户读权限，监视器需要它才能搜刮 CRUSH 图。

`w`

描述： 授予用户写对象的权限。

x

描述: 授予用户调用类方法的能力，即同时有读和写，且能在监视器上执行 `auth` 操作。

`class-read`

描述: 授予用户调用类读取方法的能力，`x` 的子集。

`class-write`

描述: 授予用户调用类写入方法的能力，`x` 的子集。

*

描述: 授权此用户读、写和执行某守护进程/存储池，且允许执行管理命令。

`profile osd`

描述: 授权一个用户以 **OSD** 身份连接其它 **OSD** 或监视器。授予 **OSD** 们允许其它 **OSD** 处理复制、心跳流量和状态报告。

`profile mds`

描述: 授权一个用户以 **MDS** 身份连接其它 **MDS** 或监视器。

`profile bootstrap-osd`

描述: 授权一用户自举引导一 OSD。授予部署工具，像 `ceph-disk`、`ceph-deploy` 等等，这样它们在自举引导 OSD 时就有权限增加密钥了。

```
profile bootstrap-mds
```

描述: 授权一用户自举引导一元数据服务器。授予像 `ceph-deploy` 一样的部署工具，这样它们在自举引导元数据服务器时就有权限增加密钥了。

存储池

存储池是用户存储数据的逻辑分区。在 Ceph 部署中，经常创建存储池作为逻辑分区、用以归类相似的数据。例如，用 Ceph 作为 OpenStack 的后端时，典型的部署通常会创建多个存储池，分别用于存储卷宗、映像、备份和虚拟机，以及用户（如 `client.glance`、`client.cinder` 等）。

命名空间

Objects within a pool can be associated to a namespace—a logical group of objects within the pool. A user's access to a pool can be associated with a namespace such that reads and writes by the user take place only within the namespace. Objects written to a namespace within the pool can only be accessed by users who have access to the namespace.

Note

Currently (i.e., `firefly`), namespaces are only useful for applications written on top of `librados`. Ceph clients such as block device, object storage and file system do not currently support this feature.

The rationale for namespaces is that pools can be a computationally expensive method of segregating data sets for the purposes of authorizing separate sets of users. For example, a pool should have ~100 placement groups per OSD. So an exemplary cluster with 1000 OSDs would have 100,000 placement groups for one pool. Each pool would create another

100,000 placement groups in the exemplary cluster. By contrast, writing an object to a namespace simply associates the namespace to the object name with out the computational overhead of a separate pool. Rather than creating a separate pool for a user or set of users, you may use a namespace. **Note:** Only available using `librados` at this time.

管理用户

User management functionality provides Ceph Storage Cluster administrators with the ability to create, update and delete users directly in the Ceph Storage Cluster.

When you create or delete users in the Ceph Storage Cluster, you may need to distribute keys to clients so that they can be added to keyrings. 详情见[密钥环管理](#)。

罗列用户

To list the users in your cluster, execute the following:

```
ceph auth list
```

Ceph will list out all users in your cluster. For example, in a two-node exemplary cluster, `ceph auth list` will output something that looks like this:

```
installed auth entries:

osd.0

    key: AQCvCbtToC6MDhAATtuT70Sl+DymPCfDSsyV4w==

    caps: [mon] allow profile osd

    caps: [osd] allow *

osd.1
```

```
key: AQC4CbtTCFJBChAAVq5spj0ff4eHZICxIOVZeA==

caps: [mon] allow profile osd

caps: [osd] allow *

client.admin

key: AQBHCbtT6APDHhAA5W00cBchwKQjh3dkKsyPjw==

caps: [mds] allow

caps: [mon] allow *

caps: [osd] allow *

client.bootstrap-mds

key: AQBICbtTOK9uGBAAdbE5zcIGHZL3T/u2g6EBww==

caps: [mon] allow profile bootstrap-mds

client.bootstrap-osd

key: AQBHCbtT4GxqORAADE5u7RkpCN/oo4e5W0uBtw==

caps: [mon] allow profile bootstrap-osd
```

Note that the `TYPE.ID` notation for users applies such that `osd.0` is a user of type `osd` and its ID is `0`, `client.admin` is a user of type `client` and its ID is `admin` (i.e., the default `client.admin` user). Note also that each entry has a `key: <value>` entry, and one or more `caps:` entries.

You may use the `-o {filename}` option with `ceph auth list` to save the output to a file.

获取用户

To retrieve a specific user, key and capabilities, execute the following:

```
ceph auth get {TYPE.ID}
```

For example:

```
ceph auth get client.admin
```

You may also use the `-o {filename}` option with `ceph auth get` to save the output to a file. Developers may also execute the following:

```
ceph auth export {TYPE.ID}
```

The `auth export` command is identical to `auth get`, but also prints out the internal `audit`, which isn't relevant to end users.

新增用户

Adding a user creates a username (i.e., `TYPE.ID`), a secret key and any capabilities included in the command you use to create the user.

A user's key enables the user to authenticate with the Ceph Storage Cluster. The user's capabilities authorize the user to read, write, or execute on Ceph monitors (`mon`), Ceph OSDs (`osd`) or Ceph Metadata Servers (`mds`).

There are a few ways to add a user:

- `ceph auth add`: This command is the canonical way to add a user. It will create the user, generate a key and add any specified capabilities.
- `ceph auth get-or-create`: This command is often the most convenient way to create a user, because it returns a keyfile format with the user name (in brackets) and the key. If the user already exists, this command simply returns the user name

and key in the keyfile format. You may use the `-o {filename}` option to save the output to a file.

- `ceph auth get-or-create-key`: This command is a convenient way to create a user and return the user's key (only). This is useful for clients that need the key only (e.g., libvirt). If the user already exists, this command simply returns the key. You may use the `-o {filename}` option to save the output to a file.

When creating client users, you may create a user with no capabilities. A user with no capabilities is useless beyond mere authentication, because the client cannot retrieve the cluster map from the monitor. However, you can create a user with no capabilities if you wish to defer adding capabilities later using the `ceph auth caps` command.

A typical user has at least read capabilities on the Ceph monitor and read and write capability on Ceph OSDs. Additionally, a user's OSD permissions are often restricted to accessing a particular pool.

```
ceph auth add client.john mon 'allow r' osd 'allow rw
pool=liverpool'

ceph auth get-or-create client.paul mon 'allow r' osd
'allow rw pool=liverpool'

ceph auth get-or-create client.george mon 'allow r' osd
'allow rw pool=liverpool' -o george.keyring

ceph auth get-or-create-key client.ringo mon 'allow r'
osd 'allow rw pool=liverpool' -o ringo.key
```

Important

If you provide a user with capabilities to OSDs, but you DO NOT restrict access to particular pools, the user will have access to ALL pools in the cluster!

修改用户能力

`ceph auth caps` 命令可以用来修改指定用户的能力。设置新能力时会覆盖当前能力。查看用户当前的能力可以用 `ceph auth getUSERTYPE.USERID`；增加能力时应该加上当前已经有的能力，命令格式如下：

```
ceph auth caps USERTYPE.USERID {daemon} 'allow
[r|w|x|*|...] [pool={pool-name}] [namespace={namespace-
name}]' [{daemon} 'allow [r|w|x|*|...] [pool={pool-name}]
[namespace={namespace-name}]']
```

例如：

```
ceph auth get client.john

ceph auth caps client.john mon 'allow r' osd 'allow rw
pool=liverpool'

ceph auth caps client.paul mon 'allow rw' osd 'allow rwx
pool=liverpool'

ceph auth caps client.brian-manager mon 'allow *' osd
'allow *'
```

To remove a capability, you may reset the capability. If you want the user to have no access to a particular daemon that was previously set, specify an empty string. For example:

```
ceph auth caps client.ringo mon ' ' osd ' '
```

关于能力请参考[授权（能力）](#)。

删除用户

要删除一用户，用 `ceph auth del` 命令：

```
ceph auth del {TYPE}.{ID}
```

其中 {TYPE} 是 client、osd、mon 或 mds 之一，{ID} 是用户名或守护进程的 ID。

查看用户密钥

To print a user's authentication key to standard output, execute the following:

```
ceph auth print-key {TYPE}.{ID}
```

Where {TYPE} is one of client, osd, mon, or mds, and {ID} is the user name or ID of the daemon.

Printing a user's key is useful when you need to populate client software with a user's key (e.g., libvirt).

```
mount -t ceph serverhost:/ mountpoint -o  
name=client.user,secret=`ceph auth print-key client.user`
```

导入用户

To import one or more users, use `ceph auth import` and specify a keyring:

```
ceph auth import -i /path/to/keyring
```

For example:

```
sudo ceph auth import -i /etc/ceph/ceph.keyring
```

Note

The ceph storage cluster will add new users, their keys and their capabilities and will update existing users, their keys and their capabilities.

密钥环管理

When you access Ceph via a Ceph client, the Ceph client will look for a local keyring. Ceph presets the `keyring` setting with the following four keyring names by default so you don't have to set them in your Ceph configuration file unless you want to override the defaults (not recommended):

- `/etc/ceph/$cluster.$name.keyring`
- `/etc/ceph/$cluster.keyring`
- `/etc/ceph/keyring`
- `/etc/ceph/keyring.bin`

The `$cluster` metavariable is your Ceph cluster name as defined by the name of the Ceph configuration file (i.e., `ceph.conf` means the cluster name is `ceph`; thus, `ceph.keyring`). The `$name` metavariable is the user type and user ID (e.g., `client.admin`; thus, `ceph.client.admin.keyring`).

Note

When executing commands that read or write to `/etc/ceph`, you may need to use `sudo` to execute the command as `root`.

After you create a user (e.g., `client.ringo`), you must get the key and add it to a keyring on a Ceph client so that the user can access the Ceph Storage Cluster.

The [用户管理](#) section details how to list, get, add, modify and delete users directly in the Ceph Storage Cluster. However, Ceph also provides the `ceph-authtool` utility to allow you to manage keyrings from a Ceph client.

创建密钥环

When you use the procedures in the [管理用户](#) section to create users, you need to provide user keys to the Ceph client(s) so that the Ceph client can retrieve the key for the specified

user and authenticate with the Ceph Storage Cluster. Ceph Clients access keyrings to lookup a user name and retrieve the user's key.

The `ceph-authtool` utility allows you to create a keyring. To create an empty keyring, use `--create-keyring` or `-C`. For example:

```
ceph-authtool --create-keyring /path/to/keyring
```

When creating a keyring with multiple users, we recommend using the cluster name (e.g., `$cluster.keyring`) for the keyring filename and saving it in the `/etc/ceph` directory so that the `keyring` configuration default setting will pick up the filename without requiring you to specify it in the local copy of your Ceph configuration file. For example, create `ceph.keyring` by executing the following:

```
sudo ceph-authtool -C /etc/ceph/ceph.keyring
```

When creating a keyring with a single user, we recommend using the cluster name, the user type and the user name and saving it in the `/etc/ceph` directory. For example, `ceph.client.admin.keyring` for the `client.admin` user.

To create a keyring in `/etc/ceph`, you must do so as `root`. This means the file will have `rw` permissions for the `root` user only, which is appropriate when the keyring contains administrator keys. However, if you intend to use the keyring for a particular user or group of users, ensure that you execute `chown` or `chmod` to establish appropriate keyring ownership and access.

把用户加入密钥环

当你在 Ceph 存储集群中[创建用户](#)后，你可以用[获取用户](#)里面的方法获取此用户、及其密钥、能力，并存入一个密钥环文件。

When you only want to use one user per keyring, the [获取用户](#) procedure with the `-o` option will save the output in the keyring file format. For example, to create a keyring for the `client.admin` user, execute the following:

```
sudo ceph auth get client.admin -o
/etc/ceph/ceph.client.admin.keyring
```

Notice that we use the recommended file format for an individual user.

When you want to import users to a keyring, you can use `ceph-authtool` to specify the destination keyring and the source keyring. For example:

```
sudo ceph-authtool /etc/ceph/ceph.keyring --import-
keyring /etc/ceph/ceph.client.admin.keyring
```

创建用户

Ceph provides the [创建用户](#) function to create a user directly in the Ceph Storage Cluster. However, you can also create a user, keys and capabilities directly on a Ceph client keyring. Then, you can import the user to the Ceph Storage Cluster. For example:

```
sudo ceph-authtool -n client.ringo --cap osd 'allow rwx'
--cap mon 'allow rwx' /etc/ceph/ceph.keyring
```

See [授权（能力）](#) for additional details on capabilities.

You can also create a keyring and add a new user to the keyring simultaneously. For example:

```
sudo ceph-authtool -C /etc/ceph/ceph.keyring -n
client.ringo --cap osd 'allow rwx' --cap mon 'allow rwx'
--gen-key
```

In the foregoing scenarios, the new user `client.ringo` is only in the keyring. To add the new user to the Ceph Storage Cluster, you must still add the new user to the Ceph Storage Cluster.

```
sudo ceph auth add client.ringo -i /etc/ceph/ceph.keyring
```

修改用户属性

To modify the capabilities of a user record in a keyring, specify the keyring, and the user followed by the capabilities. For example:

```
sudo ceph-authtool /etc/ceph/ceph.keyring -n client.ringo  
--cap osd 'allow rwx' --cap mon 'allow rwx'
```

To update the user to the Ceph Storage Cluster, you must update the user in the keyring to the user entry in the the Ceph Storage Cluster.

```
sudo ceph auth import -i /etc/ceph/ceph.keyring
```

See [导入用户](#) for details on updating a Ceph Storage Cluster user from a keyring.

You may also [修改用户能力](#) directly in the cluster, store the results to a keyring file; then, import the keyring into your main `ceph.keyring` file.

命令行用法

Ceph 支持用户名和密钥的下列用法：

`--id` | `--user`

Ceph 用一个类型和 ID

（如 `TYPE.ID` 或 `client.admin`、`client.user1`）来标识用户，`id`、`name`、和 `-n` 选项可用于指定用户名

描述:

（如 `admin`、`user1`、`foo` 等）的 ID 部分，你可以用 `--id` 指定用户并忽略类型，例如可用下列命令指定 `client.foo` 用户：

```
ceph --id foo --keyring /path/to/keyring
```

```
health

ceph --user foo --keyring /path/to/keyring
health
```

`--name` | `-n`

Ceph 用一个类型和 ID

（如 `TYPE.ID` 或 `client.admin`、`client.user1`）来标识用户，`--name` 和 `-n` 选项可用于指定完整的用户名，但必须指定用户类型（一般是 `client`）和用户 ID，例如：

描述:

```
ceph --name client.foo --keyring
/path/to/keyring health

ceph -n client.foo --keyring
/path/to/keyring health
```

`--keyring`

包含一或多个用户名、密钥的密钥环路径。`--secret` 选项提供了相同功能，但它不能用于 RADOS 网关，其 `--secret` 另有用途。你可以用 `ceph auth get-or-create` 获取密钥环并保存在本地，然后您就可以改用其他用户而无需重指定密钥环路径了。

描述:

```
sudo rbd map --id foo --keyring
/path/to/keyring mypool/myimage
```

局限性

cephx 协议提供 Ceph 客户端和服务端间的相互认证，并没打算认证人类用户或者应用程序。如果有访问控制需求，那必须用另外一种机制，它对于前端用户访问 Ceph 对象存储可能是特定的，其任务是确保只有此机器上可接受的用户和程序才能访问 Ceph 的对象存储。

用于认证 Ceph 客户端和服务器的密钥通常以纯文本存储在权限合适的文件里，并保存于可信主机上。

Important

密钥存储为纯文本文件有安全缺陷，但很难避免，它给了 Ceph 可用的基本认证方法，设置 Ceph 时应该注意这些缺陷。

尤其是任意用户、特别是移动机器不应该和 Ceph 直接交互，因为这种用法要求把明文认证密钥存储在不太安全的机器上，这些机器的丢失、或盗用将泄露可访问 Ceph 集群的密钥。

相比于允许潜在的欠安全机器直接访问 Ceph 对象存储，应该要求用户先登录安全有保障的可信机器，这台可信机器会给人们存储明文密钥。未来的 Ceph 版本也许会更彻底地解决这些特殊认证问题。

当前，没有任何 Ceph 认证协议保证传送中消息的私密性。所以，即使物理线路窃听者不能创建用户或修改它们，但可以听到、并理解客户端和服务端间发送过的所有数据。此外，Ceph 没有可加密用户数据的选项，当然，用户可以手动加密、然后把它们存在对象库里，但 Ceph 没有自己加密对象的功能。在 Ceph 里存储敏感数据的用户应该考虑存入 Ceph 集群前先加密。

数据归置概览

Ceph 通过 RADOS 集群动态地存储、复制和重新均衡数据对象。很多不同用户因不同目的把对象存储在不同的存储池里，而它们都坐落于无数的 OSD 之上，所以 Ceph 的运营需要些数据归置计划。Ceph 的数据归置计划概念主要有：

- **存储池（Pool）**：Ceph 在存储池内存储数据，它是对象存储的逻辑组；存储池管理着归置组数量、副本数量、和存储池规则集。要往存储池里存数据，用户必须通过认证、且权限合适，存储池可做快照。详情参见[存储池](#)。

- **归置组（ Placement Group ）：** Ceph 把对象映射到归置组（ PG ），归置组是一逻辑对象池的片段，这些对象组团后再存入到 OSD 。归置组减少了各对象存入对应 OSD 时的元数据数量，更多的归置组（如每 OSD 100 个）使得均衡更好。详情见[归置组](#)。
- **CRUSH 图（ CRUSH Map ）：** CRUSH 是重要组件，它使 Ceph 能伸缩自如而没有性能瓶颈、没有扩展限制、没有单点故障，它为 CRUSH 算法提供集群的物理拓扑，以此确定一个对象的数据及它的副本应该在哪里、怎样跨故障域存储，以提升数据安全。详情见[CRUSH 图](#)。

起初安装测试集群的时候，可以使用默认值。但开始规划一个大型 Ceph 集群，做数据归置操作的时候会涉及存储池、归置组、和 CRUSH 。如果您遇到难题，[Inktank](#) 可提供有偿技术支持。

存储池

如果你开始部署集群时没有创建存储池，Ceph 会用默认存储池存数据。存储池提供的功能：

- **自恢复力：** 你可以设置在不丢数据的前提下允许多少 OSD 失效，对多副本存储池来说，此值是一对象应达到的副本数。典型配置存储一个对象和它的一个副本（即 `size = 2` ），但你可以更改副本数；对[纠删编码的存储池](#)来说，此值是编码块数（即[纠删码配置](#)里的 `m=2` ）。
- **归置组：** 你可以设置一个存储池的归置组数量。典型配置给每个 OSD 分配大约 100 个归置组，这样，不用过多计算资源就能得到较优的均衡。配置了多个存储池时，要考虑到这些存储池和整个集群的归置组数量要合理。
- **CRUSH 规则：** 当你在存储池里存数据的时候，与此存储池相关联的 CRUSH 规则集可控制 CRUSH 算法，并以此操纵集群内对象及其副本的复制（或纠删码编码的存储池里的数据块）。你可以自定义存储池的 CRUSH 规则。
- **快照：** 用 `ceph osd pool mksnap` 创建快照的时候，实际上创建了某一特定存储池的快照。
- **设置所有者：** 你可以设置一个用户 ID 为一个存储池的所有者。

要把数据组织到存储池里，你可以列出、创建、删除存储池，也可以查看每个存储池的利用率。

列出存储池

要列出集群的存储池，命令如下：

```
ceph osd lspools
```

在新安装好的集群上，只有一个 `rbd` 存储池。

创建存储池

创建存储池前先看看[存储池、归置组和 CRUSH 配置参考](#)。你最好在配置文件里重置默认归置组数量，因为默认值并不理想。关于归置组数量请参考[设置归置组数量](#)。

例如：

```
osd pool default pg num = 100

osd pool default pgp num = 100
```

要创建一个存储池，执行：

```
ceph osd pool create {pool-name} {pg-num} [{pgp-num}]
[replicated] \

                [crush-ruleset-name] [expected-num-objects]

ceph osd pool create {pool-name} {pg-num} {pgp-num}
erasure \

                [erasure-code-profile] [crush-ruleset-name]
[expected_num_objects]
```

参数含义如下：

`{pool-name}`

描述: 存储池名称，必须唯一。

类型: String

是否必需: 必需。

{pg-num}

描述: 存储池拥有的归置组总数。关于如何计算合适的数值，请参见[归置组](#)。默认值 8 对大多数系统都不合适。

类型: 整数

是否必需: Yes

默认值: 8

{pgp-num}

描述: 用于归置的归置组总数。此值应该等于归置组总数，归置组分割的情况下除外。

类型: 整数

是否必需: 没指定的话读取默认值、或 Ceph 配置文件里的值。

默认值: 8

`{replicated|erasure}`

描述: 存储池类型，可以是**副本**（保存多份对象副本，以便从丢失的 OSD 恢复）或**纠删**（获得类似 **RAID5** 的功能）。多副本存储池需更多原始存储空间，但已实现所有 Ceph 操作；**纠删**存储池所需原始存储空间较少，但目前仅实现了部分 Ceph 操作。

类型: String

是否必需: No.

默认值: replicated

`[crush-ruleset-name]`

描述: 此存储池所用的 CRUSH 规则集名字。指定的规则集必须存在。

类型: String

是否必需: No.

默认值: 对于多副本（**replicated**）存储池来说，其默认规则集由 `osd pool default crush replicated ruleset` 配置决定，此规则集必须存在。对于用 `erasure-code` 编码的纠删码（**erasure**）存储池来说，不同的 `{pool-name}` 所使用的默认（`default`）[纠删码配置](#)是不同的，如果它不存在的话，会显式地创建它。

`[erasure-code-profile=profile]`

描述: 仅用于[纠删](#)存储池。指定[纠删码配置](#)框架，此配置必须已由 `osd erasure-code-profile set` 定义。

类型: String

是否必需: No.

创建存储池时，要设置一个合理的归置组数量（如 `100`）。也要考虑到每 OSD 的归置组总数，因为归置组很耗计算资源，所以很多存储池和很多归置组（如 50 个存储池，各包含 100 归置组）会导致性能下降。收益递减点取决于 OSD 主机的强大。

如何为存储池计算合适的归置组数量请参见[归置组](#)。

[expected-num-objects]

描述: 为这个存储池预估的对象数。设置此值（要同时把 **filestore merge threshold** 设置为负数）后，在创建存储池时就会拆分 PG 文件夹，以免运行时拆分文件夹导致延时增大。

类型: Integer

是否必需: No.

默认值: 0，创建存储池时不拆分目录。

设置存储池配额

存储池配额可设置最大字节数、和/或每存储池最大对象数。

```
ceph osd pool set-quota {pool-name} [max_objects {obj-count}] [max_bytes {bytes}]
```

例如：

```
ceph osd pool set-quota data max_objects 10000
```

要取消配额，设置为 0。

删除存储池

要删除一存储池，执行：

```
ceph osd pool delete {pool-name} [{pool-name} --yes-i-  
really-really-mean-it]
```

如果你给自建的存储池创建了定制的规则集，你不需要存储池时最好删除它。如果你曾严格地创建了用户及其权限给一个存储池，但存储池已不存在，最好也删除那些用户。

重命名存储池

要重命名一个存储池，执行：

```
ceph osd pool rename {current-pool-name} {new-pool-name}
```

如果重命名了一个存储池，且认证用户有每存储池能力，那你必须用新存储池名字更新用户的能力（即 caps ）。

Note

适用 0.48 Argonaut 及以上。

查看存储池统计信息

要查看某存储池的使用统计信息，执行命令：

```
rados df
```

拍下存储池快照

要拍下某存储池的快照，执行命令：

```
ceph osd pool mksnap {pool-name} {snap-name}
```

Note

适用 0.48 Argonaut 及以上。

删除存储池快照

要删除某存储池的一个快照，执行命令：

```
ceph osd pool rmsnap {pool-name} {snap-name}
```

Note

适用 0.48 Argonaut 及以上。

调整存储池选项值

要设置一个存储池的选项值，执行命令：

```
ceph osd pool set {pool-name} {key} {value}
```

你可以设置下列键的值：

`size`

描述: 设置存储池中的对象副本数，详情参见[设置对象副本数](#)。仅适用于副本存储池。

类型: 整数

`min_size`

描述: 设置 I/O 需要的最小副本数，详情参见[设置对象副本数](#)。仅适用于副本存储池。

类型: 整数

适用版本: 0.54 及以上。

`crash_replay_interval`

描述: 允许客户端重放确认而未提交请求的秒数。

类型: 整数

`pgp_num`

描述: 计算数据归置时使用的有效归置组数量。

类型: 整数

有效范围: 等于或小于 `pg_num`。

`crush_ruleset`

描述: 集群内映射对象归置时使用的规则集。

类型: 整数

hashpspool

描述: 给指定存储池设置/取消 HASHPSPOOL 标志。

类型: 整数

有效范围: 1 开启, 0 取消

适用版本: 0.48 及以上。

nodelete

描述: 给指定存储池设置/取消 NODELETE 标志。

类型: 整数

有效范围: 1 开启, 0 取消

适用版本: Version FIXME

nopgchange

描述: 给指定存储池设置/取消 NOPGCHANGE 标志。

类型: 整数

有效范围: 1 开启, 0 取消

适用版本: Version `FIXME`

`nosizechange`

描述: 给指定存储池设置/取消 NOSIZECHANGE 标志。

类型: 整数

有效范围: 1 开启, 0 取消

适用版本: Version `FIXME`

`write_fadvise_dontneed`

描述: 设置或取消指定存储池的 WRITE_FADVISE_DONTNEED 标志。

类型: Integer

有效范围: 1 开启, 0 取消

`noscrub`

描述: 设置或取消指定存储池的 NOSCRUB 标志。

类型: Integer

有效范围: 1 设置, 0 取消

`nodeep-scrub`

描述: 设置或取消指定存储池的 NODEEP_SCRUB 标志。

类型: Integer

有效范围: 1 开启, 0 取消

`hit_set_type`

描述: 启用缓存存储池的命中集跟踪, 详情见 [Bloom 过滤器](#)。

类型: String

有效值: bloom, explicit_hash, explicit_object

默认值: bloom，其它是用于测试的。

hit_set_count

描述: 为缓存存储池保留的命中集数量。此值越高，ceph-osd 守护进程消耗的内存越多。

类型: 整数

有效范围: 1. Agent doesn't handle > 1 yet.

hit_set_period

描述: 为缓存存储池保留的命中集有效期。此值越高，ceph-osd 消耗的内存越多。

类型: 整数

实例: 3600 1hr

hit_set_fpp

描述: `bloom` 命中集类型的假阳性概率。详情见 [Bloom 过滤器](#)。

类型: Double

有效范围: 0.0 - 1.0

默认值: `0.05`

`cache_target_dirty_ratio`

描述: 缓存存储池包含的脏对象达到多少比例时就把它回写到后端的存储池。

类型: Double

默认值: `.4`

`cache_target_dirty_high_ratio`

描述: 缓存存储池内包含的已修改（脏的）对象达到此比例时，缓存层代理就会更快地把脏对象刷回到后端存储池。

类型: Double

默认 `.6`

值:

`cache_target_full_ratio`

描述: 缓存存储池包含的干净对象达到多少比例时，缓存代理就把它们赶出缓存存储池。

类型: Double

默认值: `.8`

`target_max_bytes`

描述: 达到 `max_bytes` 阈值时 Ceph 就回写或赶出对象。

类型: 整数

实例: `1000000000000 #1-TB`

`target_max_objects`

描述: 达到 `max_objects` 阈值时 Ceph 就回写或赶出对象。

类型: 整数

实例: 1000000 #1M objects

hit_set_grade_decay_rate

描述: 在两个连续 hit_sets 间的热度衰退速率。

类型: Integer

有效范围: 0 - 100

默认值: 20

hit_set_grade_search_last_n

描述: 计算热度时，在 hit_sets 里最多计数 N 次。

类型: Integer

有效范围: 0 - hit_set_count

默认值: 1

cache_min_flush_age

描述: 达到此时间（单位为秒）时，缓存代理就把某些对象从缓存存储池刷回到存储池。

类型: 整数

实例: 600 10min

`cache_min_evict_age`

描述: 达到此时间（单位为秒）时，缓存代理就把某些对象从缓存存储池赶出。

类型: 整数

实例: 1800 30min

`fast_read`

描述: 在纠删码存储池上，如果打开了这个标志，读请求会向所有分片发送子操作读，然后等着，直到收到的分片足以解码给客户端。对 `jerasure` 和 `isa` 纠删码插件来说，只要前 `K` 个请求返回，就能立即解码、并先把这些数据发给客户端。这样有助于资源折衷，以提升性能。当前，这些标志还只能用于纠删码存储池。

类型: Boolean

默认
值: 0

`scrub_min_interval`

描述: 在负载低时，洗刷存储池的最大间隔秒数。如果是 0，就按照配置文件里的 `osd_scrub_min_interval`。

类型: Double

默认
值: 0

`scrub_max_interval`

描述: 不管集群负载如何，都要洗刷存储池的最大间隔秒数。如果是 0，就按照配置文件里的 `osd_scrub_max_interval`。

类型: Double

默认
值: 0

`deep_scrub_interval`

描述: “深度”洗刷存储池的间隔秒数。如果是 0，就按照配置文件里的 `osd_deep_scrub_interval`。

类型: Double

默认值: 0

获取存储池选项值

要获取一个存储池的选项值，执行命令：

```
ceph osd pool get {pool-name} {key}
```

你可以获取到下列选项的值：

`size`

描述: 见 [size](#)

类型: 整数

`min_size`

描述: 见 [min_size](#)

类型: 整数

适用版本: 0.54 及以上

`crash_replay_interval`

描述: 见 [crash_replay_interval](#)

类型: 整数

`pgp_num`

描述: 见 [pgp_num](#)

类型: 整数

有效范围: 小于等于 `pg_num`。

`crush_ruleset`

描述: 见 [crush_ruleset](#)

类型: 整数

`hit_set_type`

描述: 见 [hit_set_type](#)

类型: String

有效选项: bloom、explicit_hash、explicit_object

hit_set_count

描述: 见 [hit_set_count](#)

类型: 整数

hit_set_period

描述: 见 [hit_set_period](#)

类型: 整数

hit_set_fpp

描述: 见 [hit_set_fpp](#)

类型: Double

cache_target_dirty_ratio

描述: 见 [cache target dirty ratio](#)

类型: Double

`cache_target_dirty_high_ratio`

描述: 见 [cache target dirty high ratio](#)

类型: Double

`cache_target_full_ratio`

描述: 见 [cache target full ratio](#)

类型: Double

`target_max_bytes`

描述: 见 [target_max_bytes](#)

类型: 整数

`target_max_objects`

描述: 见 [target_max_objects](#)

类型: 整数

`cache_min_flush_age`

描述: 见 [cache_min_flush_age](#)

类型: 整数

`cache_min_evict_age`

描述: 见 [cache_min_evict_age](#)

类型: 整数

`fast_read`

描述: 见 [fast_read](#)

类型: Boolean

`scrub_min_interval`

描述: 见 [scrub_min_interval](#)

类型: Double

`scrub_max_interval`

描述: 见 [scrub_max_interval](#)

类型: Double

`deep_scrub_interval`

描述: 见 [deep_scrub_interval](#)

类型: Double

设置对象副本数

要设置多副本存储池的对象副本数，执行命令：

```
ceph osd pool set {poolname} size {num-replicas}
```

Important

`{num-replicas}` 包括对象自身，如果你想要对象自身及其两份拷贝共计三份，指定 3。

例如：

```
ceph osd pool set data size 3
```

你可以在每个存储池上执行这个命令。**注意**，一个处于降级模式的对象其副本数小于规定值 `pool size`，但仍可接受 I/O 请求。为保证 I/O 正常，可用 `min_size` 选项为其设置个最低副本数。例如：

```
ceph osd pool set data min_size 2
```

这确保数据存储池里任何副本数小于 `min_size` 的对象都不会收到 I/O 了。

获取对象副本数

要获取对象副本数，执行命令：

```
ceph osd dump | grep 'replicated size'
```

Ceph 会列出存储池，且高亮 `replicated size` 属性。默认情况下，Ceph 会创建一对象的两个副本（一共三个副本，或 `size` 值为 3）。

纠删码

A Ceph pool is associated to a type to sustain the loss of an OSD (i.e. a disk since most of the time there is one OSD per disk). The default choice when [creating a pool](#) is *replicated*, meaning every object is copied on multiple disks. The [Erasure Code](#) pool type can be used instead to save space.

创建样板纠删码存储池

The simplest erasure coded pool is equivalent to [RAID5](#) and requires at least three hosts:

```
$ ceph osd pool create ecpool 12 12 erasure  
  
pool 'ecpool' created  
  
$ echo ABCDEFGHI | rados --pool ecpool put NYAN -  
  
$ rados --pool ecpool get NYAN -  
  
ABCDEFGHI
```

Note

the 12 in *pool create* stands for [the number of placement groups](#).

纠删码配置

The default erasure code profile sustains the loss of a single OSD. It is equivalent to a replicated pool of size two but requires 1.5TB instead of 2TB to store 1TB of data. The default profile can be displayed with:

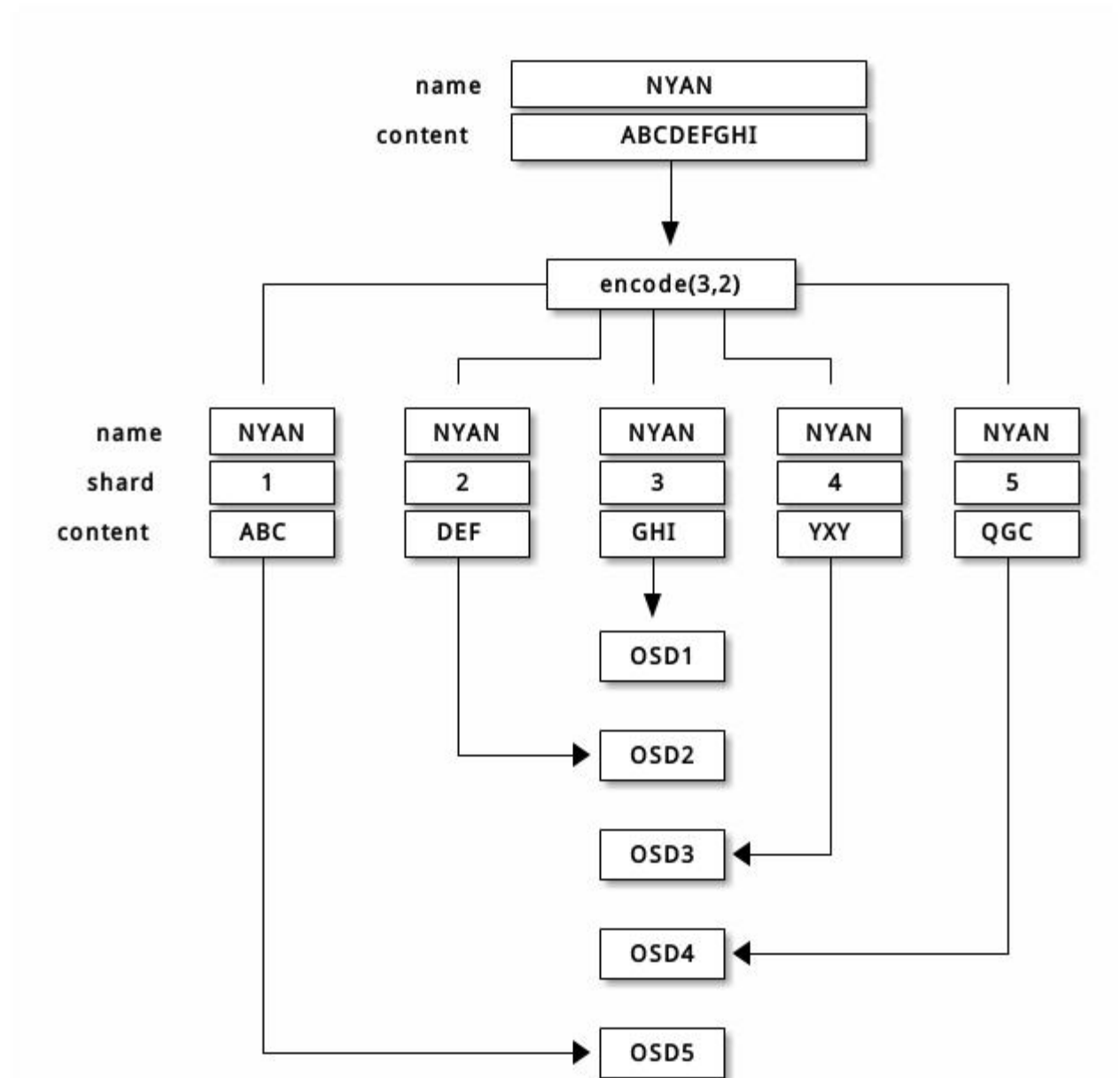
```
$ ceph osd erasure-code-profile get default  
  
directory=.libs  
  
k=2  
  
m=1  
  
plugin=jerasure  
  
ruleset-failure-domain=host  
  
technique=reed_sol_van
```

Choosing the right profile is important because it cannot be modified after the pool is created: a new pool with a different profile needs to be created and all objects from the previous pool moved to the new.

The most important parameters of the profile are K , M and *ruleset-failure-domain* because they define the storage overhead and the data durability. For instance, if the desired architecture must sustain the loss of two racks with a storage overhead of 40% overhead, the following profile can be defined:

```
$ ceph osd erasure-code-profile set myprofile \
    k=3 \
    m=2 \
    ruleset-failure-domain=rack
$ ceph osd pool create ecpool 12 12 erasure myprofile
$ echo ABCDEFGHI | rados --pool ecpool put NYAN -
$ rados --pool ecpool get NYAN -
ABCDEFGHI
```

The *NYAN* object will be divided in three ($K=3$) and two additional *chunks* will be created ($M=2$). The value of M defines how many OSD can be lost simultaneously without losing any data. The *ruleset-failure-domain=rack* will create a CRUSH ruleset that ensures no two *chunks* are stored in the same rack.



More information can be found in the [erasure code profiles](#) documentation.

纠删码存储池与缓存分级

Erasure coded pools require more resources than replicated pools and lack some functionalities such as partial writes. To overcome these limitations, it is recommended to set a [cache tier](#) before the erasure coded pool.

For instance, if the pool *hot-storage* is made of fast storage:

```
$ ceph osd tier add ecpool hot-storage  
  
$ ceph osd tier cache-mode hot-storage writeback  
  
$ ceph osd tier set-overlay ecpool hot-storage
```

will place the *hot-storage* pool as tier of *ecpool* in *writeback* mode so that every write and read to the *ecpool* are actually using the *hot-storage* and benefit from its flexibility and speed.

It is not possible to create an RBD image on an erasure coded pool because it requires partial writes. It is however possible to create an RBD image on an erasure coded pools when a replicated pool tier set a cache tier:

```
$ rbd create --size 10G ecpool/myvolume
```

更详细的文档请参阅[分级缓存](#)。

术语

chunk

when the encoding function is called, it returns chunks of the same size. Data chunks which can be concatenated to reconstruct the original object and coding chunks which can be used to rebuild a lost chunk.

K

the number of data *chunks*, i.e. the number of *chunks* in which the original object is divided. For instance if $K = 2$ a 10KB object will be divided into K objects of 5KB each.

M

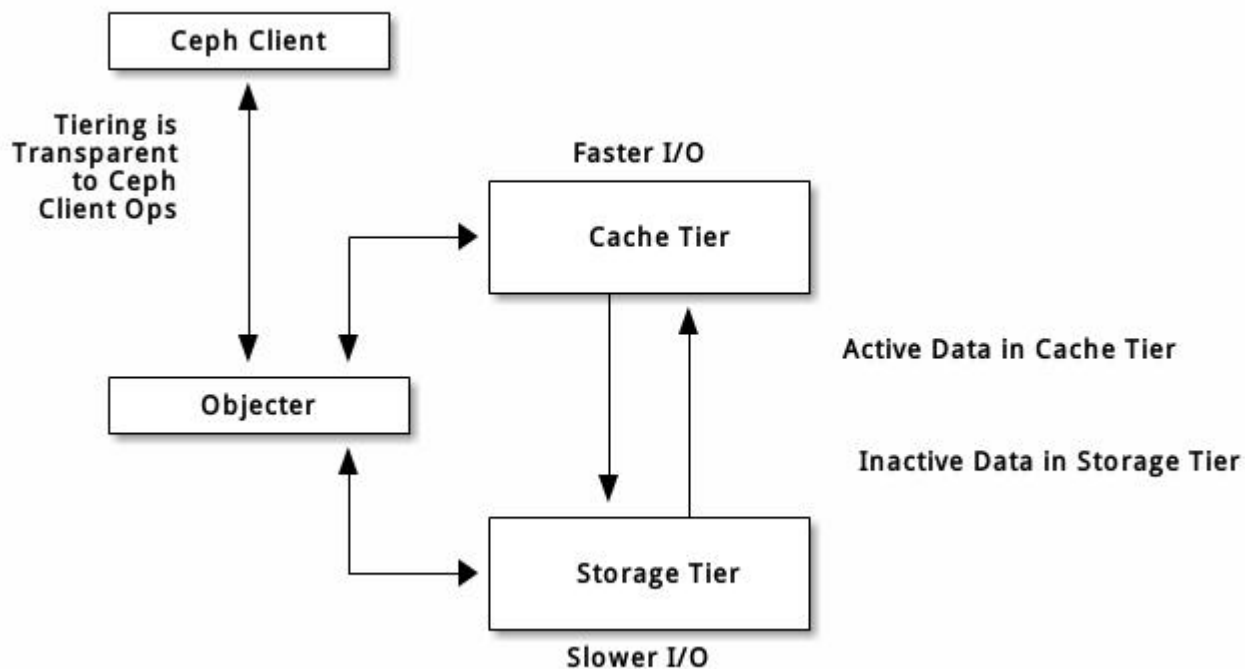
the number of coding *chunks*, i.e. the number of additional *chunks* computed by the encoding functions. If there are 2 coding *chunks*, it means 2 OSDs can be out without losing data.

内容列表

- [纠删码配置](#)
- [Jerasure 纠删码插件](#)
- [ISA 纠删码插件](#)
- [局部自修复纠删码插件](#)
- [SHEC 纠删码插件](#)

分级缓存

分级缓存可提升后端存储内某些（热点）数据的 I/O 性能。分级缓存需创建一个由高速而昂贵存储设备（如 SSD）组成的存储池、作为缓存层，以及一个相对低速/廉价设备组成的后端存储池（或纠删码编码的）、作为经济存储层。Ceph 的对象处理器决定往哪里存储对象，分级代理决定何时把缓存内的对象刷回后端存储层；所以缓存层和后端存储层对 Ceph 客户端来说是完全透明的。



缓存层代理自动处理缓存层和后端存储之间的数据迁移。然而，管理员仍可干预此迁移规则，主要有两种场景：

- **回写模式：** 管理员把缓存层配置为 `writeback` 模式时，Ceph 客户端们会把数据写入缓存层、并收到缓存层发来的 `ACK`；写入缓存层的数据会被迁移到存储层、然后从缓存层刷掉。直观地看，缓存层位于后端存储层的“前面”，当 Ceph 客户端要读取的数据位于存储层时，缓存层代理会把这些数据迁移到缓存层，然后再发往 Ceph 客户端。从此，Ceph 客户端将与缓存层进行 I/O 操作，直到数据不再被读写。此模式对于易变数据来说较理想（如照片/视频编辑、事务数据等）。
- **只读模式：** 管理员把缓存层配置为 `readonly` 模式时，Ceph 直接把数据写入后端。读取时，Ceph 把相应对象从后端复制到缓存层，根据已定义策略、脏对象会被缓存层踢出。此模式适合不变数据（如社交网络上展示的图片/视频、DNA 数据、X-Ray 照片等），因为从缓存层读出的数据可能包含过期数据，即一致性较差。对易变数据不要用 `readonly` 模式。

正因为所有 Ceph 客户端都能用缓存层，所以才有提升块设备、Ceph 对象存储、Ceph 文件系统和原生绑定的 I/O 性能的潜力。

配置存储池

要设置缓存层，你必须有两个存储池。一个作为后端存储、另一个作为缓存。

配置后端存储池

设置后端存储池通常会遇到两种场景：

- **标准存储：** 此时，Ceph 存储集群内的存储池保存了一对象的多个副本；
- **纠删存储池：** 此时，存储池用纠删码高效地存储数据，性能稍有损失。

在标准存储场景中，你可以用 CRUSH 规则集来标识失败域（如 `osd`、主机、机箱、机架、排等）。当规则集所涉及的所有驱动器规格、速度（转速和吞吐量）和类型相同时，OSD 守护进程运行得最优。创建规则集的详情见 [CRUSH 图](#)。创建好规则集后，再创建后端存储池。

在纠删码编码情景中，创建存储池时指定好参数就会自动生成合适的规则集，详情见[创建存储池](#)。

在后续例子中，我们把 `cold-storage` 当作后端存储池。

配置缓存池

缓存存储池的设置步骤大致与标准存储情景相同，但仍有不同：缓存层所用的驱动器通常都是高性能的、且安装在专用服务器上、有自己的规则集。制定规则集时，要考虑到装有高性能驱动器的主机、并忽略没有的主机。详情见[给存储池指定 OSD](#)。

在后续例子中，`hot-storage` 作为缓存存储池、`cold-storage` 作为后端存储池。

关于缓存层的配置及其默认值的详细解释请参考[存储池——调整存储池](#)。

创建缓存层

设置一缓存层需把缓存存储池挂接到后端存储池上：

```
ceph osd tier add {storagepool} {cachepool}
```

例如：

```
ceph osd tier add cold-storage hot-storage
```

用下列命令设置缓存模式：

```
ceph osd tier cache-mode {cachepool} {cache-mode}
```

例如：

```
ceph osd tier cache-mode hot-storage writeback
```

缓存层盖在后端存储层之上，所以要多一步：必须把所有客户端流量从存储池迁移到缓存存储池。用此命令把客户端流量指向缓存存储池：

```
ceph osd tier set-overlay {storagepool} {cachepool}
```


例如：

```
ceph osd tier set-overlay cold-storage hot-storage
```

配置缓存层

缓存层支持几个配置选项，可按下列语法配置：

```
ceph osd pool set {cachepool} {key} {value}
```

详情见[存储池——调整存储池](#)。

目标尺寸和类型

生产环境下，缓存层的 `hit_set_type` 还只能用 Bloom 过滤器：

```
ceph osd pool set {cachepool} hit_set_type bloom
```

例如：

```
ceph osd pool set hot-storage hit_set_type bloom
```

`hit_set_count` 和 `hit_set_period` 选项分别定义了 HitSet 覆盖的时间区间、以及保留多少个这样的 HitSet。

```
ceph osd pool set {cachepool} hit_set_count 1

ceph osd pool set {cachepool} hit_set_period 3600

ceph osd pool set {cachepool} target_max_bytes
1000000000000
```

保留一段时间以来的访问记录，这样 Ceph 就能判断一客户端在一段时间内访问了某对象一次、还是多次（存活期与热度）。

`min_read_recency_for_promote` 定义了在处理一个对象的读操作时检查多少个 HitSet，检查结果将用于决定是否异步地提升对象。它的取值应该在 0 和 `hit_set_count` 之间，如果设置为 0，对象会一直被提升；如果设置为 1，就只检查当前 HitSet，如果此对象在当前 HitSet 里就提升它，否则就不提升；设置为其它值时，就要挨个检查此数量的历史 HitSet，如果此对象出现在 `min_read_recency_for_promote` 个 HitSet 里的任意一个，那就提升它。

还有一个相似的参数用于配置写操作，它是 `min_write_recency_for_promote`。

```
ceph osd pool set {cachepool}
min_read_recency_for_promote 1

ceph osd pool set {cachepool}
min_write_recency_for_promote 1
```

Note

统计时间越

长、`min_read_recency_for_promote` 或 `min_write_recency_for_promote` 取值越高，ceph-osd 进程消耗的内存就越多，特别是代理正忙着刷回或赶出对象时，此时所有 `hit_set_count` 个 HitSet 都要载入内存。

缓存空间消长

缓存分层代理有两个主要功能：

- **刷回：** 代理找出修改过（或脏）的对象、并把它们转发给存储池做长期存储。
- **赶出：** 代理找出未修改（或干净）的对象、并把最近未用过的赶出缓存。

相对空间消长

缓存分层代理可根据缓存存储池相对大小刷回或赶出对象。当缓存池包含的已修改（或脏）对象达到一定比例时，缓存分层代理就把它们刷回到存储池。用下列命令设置

置 `cache_target_dirty_ratio`：

```
ceph osd pool set {cachepool} cache_target_dirty_ratio {0.0..1.0}
```

例如，设置为 `0.4` 时，脏对象达到缓存池容量的 40% 就开始刷回：

```
ceph osd pool set hot-storage cache_target_dirty_ratio 0.4
```

当脏对象达到其容量的一定比例时，要更快地刷回脏对象。用下列命令设置

置 `cache_target_dirty_high_ratio`：

```
ceph osd pool set {cachepool} cache_target_dirty_high_ratio {0.0..1.0}
```

例如，设置为 `0.6` 表示：脏对象达到缓存存储池容量的 60% 时，将开始更激进地刷回脏对象。显然，其值最好在 `dirty_ratio` 和 `full_ratio` 之间：

```
ceph osd pool set hot-storage cache_target_dirty_high_ratio 0.6
```

当缓存池利用率达到总容量的一定比例时，缓存分层代理会赶出部分对象以维持空闲空间。执行此命令设置 `cache_target_full_ratio`：

```
ceph osd pool set {cachepool} cache_target_full_ratio {0.0..1.0}
```

例如，设置为 `0.8` 时，干净对象占到总容量的 80% 就开始赶出缓存池：

```
ceph osd pool set hot-storage cache_target_full_ratio 0.8
```

绝对空间消长

缓存分层代理可根据总字节数或对象数量来刷回或赶出对象，用下列命令可指定最大字节数：

```
ceph osd pool set {cachepool} target_max_bytes {#bytes}
```

例如，用下列命令配置在达到 1TB 时刷回或赶出：

```
ceph osd pool set hot-storage target_max_bytes  
1000000000000
```

用下列命令指定缓存对象的最大数量：

```
ceph osd pool set {cachepool} target_max_objects  
{#objects}
```

例如，用下列命令配置对象数量达到 1M 时开始刷回或赶出：

```
ceph osd pool set hot-storage target_max_objects 1000000
```

Note

如果两个都配置了，缓存分层代理会按先到的阈值执行刷回或赶出。

缓存时长

你可以规定缓存层代理必须延迟多久才能把某个已修改（脏）对象刷回后端存储池：

```
ceph osd pool set {cachepool} cache_min_flush_age  
{#seconds}
```

例如，让已修改（或脏）对象需至少延迟 10 分钟才能刷回，执行此命令：

```
ceph osd pool set hot-storage cache_min_flush_age 600
```

你可以指定某对象在缓存层至少放置多长时间才能被赶出：

```
ceph osd pool {cache-tier} cache_min_evict_age {#seconds}
```

例如，要规定 30 分钟后才赶出对象，执行此命令：

```
ceph osd pool set hot-storage cache_min_evict_age 1800
```

拆除缓存层

回写缓存和只读缓存的去除过程不太一样。

拆除只读缓存

只读缓存不含变更数据，所以禁用它不会导致任何近期更改的数据丢失。

1. 把缓存模式改为 `none` 即可禁用。

```
ceph osd tier cache-mode {cachepool} none
```

例如：

```
ceph osd tier cache-mode hot-storage none
```

2. 去除后端存储池的缓存池。

```
ceph osd tier remove {storagepool} {cachepool}
```

例如：

```
ceph osd tier remove cold-storage hot-storage
```

拆除回写缓存

回写缓存可能含有更改过的数据，所以在禁用并去除前，必须采取些手段以免丢失缓存内近期更改的对象。

1. 把缓存模式改为 `forward`，这样新的和更改过的对象将直接刷回到后端存储池。

```
ceph osd tier cache-mode {cachepool} forward
```

例如：

```
ceph osd tier cache-mode hot-storage forward
```

2. 确保缓存池已刷回，可能要等数分钟：

```
rados -p {cachepool} ls
```

如果缓存池还有对象，你可以手动刷回，例如：

```
rados -p {cachepool} cache-flush-evict-all
```

3. 去除此盖子，这样客户端就不会被指到缓存了。

```
ceph osd tier remove-overlay {storagetier}
```

例如：

```
ceph osd tier remove-overlay cold-storage
```

4. 最后，从后端存储池剥离缓存层存储池。

```
ceph osd tier remove {storagepool} {cachepool}
```

例如：

```
ceph osd tier remove cold-storage hot-storage
```

归置组

预定义 PG_NUM

用此命令创建存储池时：

```
ceph osd pool create {pool-name} pg_num
```

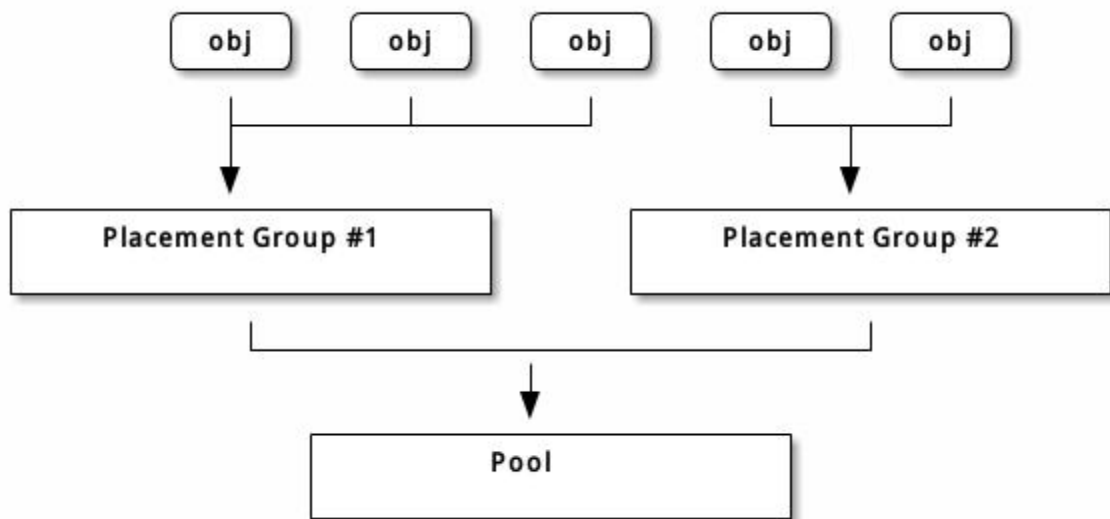
确定 `pg_num` 取值是强制性的，因为不能自动计算。下面是几个常用的值：

- 少于 5 个 OSD 时可把 `pg_num` 设置为 128
- OSD 数量在 5 到 10 个时，可把 `pg_num` 设置为 512
- OSD 数量在 10 到 50 个时，可把 `pg_num` 设置为 4096
- OSD 数量大于 50 时，你得理解权衡方法、以及如何自己计算 `pg_num` 取值
- 自己计算 `pg_num` 取值时可借助 [pgcalc](#) 工具

随着 OSD 数量的增加，正确的 pg_num 取值变得更加重要，因为它显著地影响着集群的行为、以及出错时的数据持久性（即灾难性事件导致数据丢失的概率）。

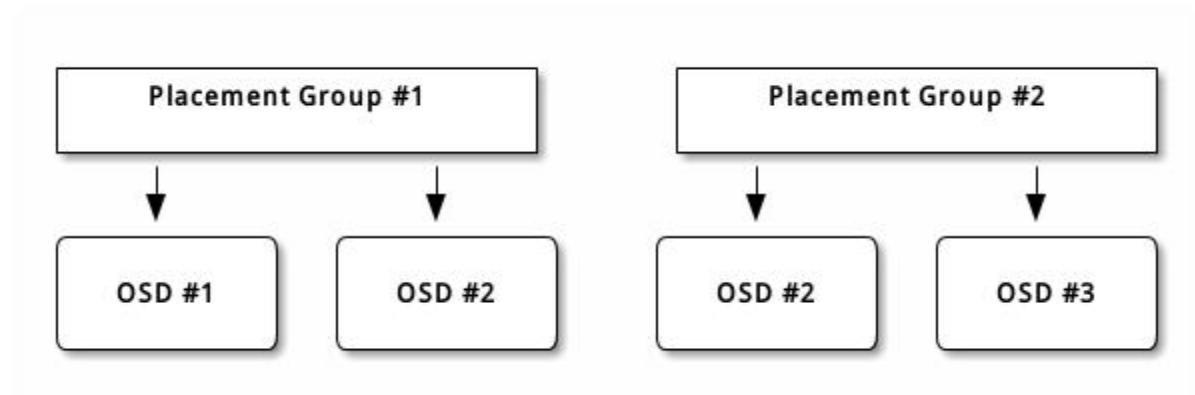
归置组是如何使用的？

存储池内的归置组（PG）把对象汇聚在一起，因为跟踪每一个对象的位置及其元数据需要大量计算——即一个拥有数百万对象的系统，不可能在对象这一级追踪位置。



Ceph 客户端会计算某一对象应该位于哪个归置组里，它是这样实现的，先给对象 ID 做哈希操作，然后再根据指定存储池里的 PG 数量、存储池 ID 做一个运算。详情见 [PG 映射到 OSD](#)。

The object's contents within a placement group are stored in a set of OSDs. For instance, in a replicated pool of size two, each placement group will store objects on two OSDs, as shown below.



Should OSD #2 fail, another will be assigned to Placement Group #1 and will be filled with copies of all objects in OSD #1. If the pool size is changed from two to three, an additional OSD will be assigned to the placement group and will receive copies of all objects in the placement group.

Placement groups do not own the OSD, they share it with other placement groups from the same pool or even other pools. If OSD #2 fails, the Placement Group #2 will also have to restore copies of objects, using OSD #3.

When the number of placement groups increases, the new placement groups will be assigned OSDs. The result of the CRUSH function will also change and some objects from the former placement groups will be copied over to the new Placement Groups and removed from the old ones.

PLACEMENT GROUPS TRADEOFFS

Data durability and even distribution among all OSDs call for more placement groups but their number should be reduced to the minimum to save CPU and memory.

DATA DURABILITY

After an OSD fails, the risk of data loss increases until the data it contained is fully recovered. Let's imagine a scenario that causes permanent data loss in a single placement group:

- The OSD fails and all copies of the object it contains are lost. For all objects within the placement group the number of replica suddenly drops from three to two.

- Ceph starts recovery for this placement group by choosing a new OSD to re-create the third copy of all objects.
- Another OSD, within the same placement group, fails before the new OSD is fully populated with the third copy. Some objects will then only have one surviving copies.
- Ceph picks yet another OSD and keeps copying objects to restore the desired number of copies.
- A third OSD, within the same placement group, fails before recovery is complete. If this OSD contained the only remaining copy of an object, it is permanently lost.

In a cluster containing 10 OSDs with 512 placement groups in a three replica pool, CRUSH will give each placement groups three OSDs. In the end, each OSDs will end up hosting $(512 * 3) / 10 = \sim 150$ Placement Groups. When the first OSD fails, the above scenario will therefore start recovery for all 150 placement groups at the same time.

The 150 placement groups being recovered are likely to be homogeneously spread over the 9 remaining OSDs. Each remaining OSD is therefore likely to send copies of objects to all others and also receive some new objects to be stored because they became part of a new placement group.

The amount of time it takes for this recovery to complete entirely depends on the architecture of the Ceph cluster. Let say each OSD is hosted by a 1TB SSD on a single machine and all of them are connected to a 10Gb/s switch and the recovery for a single OSD completes within M minutes. If there are two OSDs per machine using spinners with no SSD journal and a 1Gb/s switch, it will at least be an order of magnitude slower.

In a cluster of this size, the number of placement groups has almost no influence on data durability. It could be 128 or 8192 and the recovery would not be slower or faster.

However, growing the same Ceph cluster to 20 OSDs instead of 10 OSDs is likely to speed up recovery and therefore improve data durability significantly. Each OSD now participates in only ~ 75 placement groups instead of ~ 150 when there were only 10 OSDs and it will still require all 19 remaining OSDs to perform the same amount of object copies in order to recover. But where 10 OSDs had to copy approximately 100GB each, they now have to copy 50GB each instead. If the network was the bottleneck, recovery will happen twice as fast. In other words, recovery goes faster when the number of OSDs increases.

If this cluster grows to 40 OSDs, each of them will only host ~35 placement groups. If an OSD dies, recovery will keep going faster unless it is blocked by another bottleneck. However, if this cluster grows to 200 OSDs, each of them will only host ~7 placement groups. If an OSD dies, recovery will happen between at most of ~21 ($7 * 3$) OSDs in these placement groups: recovery will take longer than when there were 40 OSDs, meaning the number of placement groups should be increased.

不论恢复时间有多短，在此期间都可能有第二个 OSD 失败。在前述的有 10 个 OSD 的集群中，不管哪个失败了，都有大约 17 个归置组（即需恢复的大约 $150 / 9$ 个归置组）将只有一份可用副本；并且假设剩余的 8 个 OSD 中任意一个失败，两个归置组中最后的对象都有可能丢失（即正在恢复的、大约 $17 / 8$ 个仅剩一个副本的归置组）。

当集群大小变为 20 个 OSD 时，3 个 OSD 丢失导致的归置组损坏会降低。第二个 OSD 丢失会降级大约 4 个（即需恢复的归置组约为 $75 / 19$ ）而不是约 17 个归置组，并且只有当第三个 OSD 恰好是包含可用副本的四分之一 OSD 时、才会丢失数据。换句话说，假设在恢复期间丢失一个 OSD 的概率是 0.0001%，那么，在包含 10 个 OSD 的集群中丢失 OSD 的概率是 $17 * 10 * 0.0001\%$ ，而在 20 个 OSD 的集群中将是 $4 * 20 * 0.0001\%$ 。

In a nutshell, more OSDs mean faster recovery and a lower risk of cascading failures leading to the permanent loss of a Placement Group. Having 512 or 4096 Placement Groups is roughly equivalent in a cluster with less than 50 OSDs as far as data durability is concerned.

Note: It may take a long time for a new OSD added to the cluster to be populated with placement groups that were assigned to it. However there is no degradation of any object and it has no impact on the durability of the data contained in the Cluster.

OBJECT DISTRIBUTION WITHIN A POOL

Ideally objects are evenly distributed in each placement group. Since CRUSH computes the placement group for each object, but does not actually know how much data is stored in each OSD within this placement group, the ratio between the number of placement groups and the number of OSDs may influence the distribution of the data significantly.

For instance, if there was single a placement group for ten OSDs in a three replica pool, only three OSD would be used because CRUSH would have no other choice. When more placement groups are available, objects are more likely to be evenly spread among them.

CRUSH also makes every effort to evenly spread OSDs among all existing Placement Groups.

As long as there are one or two orders of magnitude more Placement Groups than OSDs, the distribution should be even. For instance, 300 placement groups for 3 OSDs, 1000 placement groups for 10 OSDs etc.

Uneven data distribution can be caused by factors other than the ratio between OSDs and placement groups. Since CRUSH does not take into account the size of the objects, a few very large objects may create an imbalance. Let say one million 4K objects totaling 4GB are evenly spread among 1000 placement groups on 10 OSDs. They will use $4GB / 10 = 400MB$ on each OSD. If one 400MB object is added to the pool, the three OSDs supporting the placement group in which the object has been placed will be filled with $400MB + 400MB = 800MB$ while the seven others will remain occupied with only 400MB.

内存、处理器和网络使用情况

各个归置组、OSD 和监视器都一直需要内存、网络、处理器，在恢复期间需求更大。为消除过载而把对象聚集成簇是归置组存在的主要原因。

最小化归置组数量可节省不少资源。

确定归置组数量

If you have more than 50 OSDs, we recommend approximately 50-100 placement groups per OSD to balance out resource usage, data durability and distribution. If you have less than 50 OSDs, choosing among the [preselection](#) above is best. For a single pool of objects, you can use the following formula to get a baseline:

$$\text{Total PGs} = \frac{(\text{OSDs} * 100)}{\text{pool size}}$$

Where **pool size** is either the number of replicas for replicated pools or the K+M sum for erasure coded pools (as returned by `ceph osd erasure-code-profile get`).

You should then check if the result makes sense with the way you designed your Ceph cluster to maximize [data durability](#), [object distribution](#) and minimize [resource usage](#).

其结果汇总后应该接近 2 的幂。汇总并非强制的，如果你想确保所有归置组内的对象数大致相等，最好检查下。

比如，一个配置了 200 个 OSD 且副本数为 3 的集群，你可以这样估算归置组数量：

```
(200 * 100)
----- = 6667. Nearest power of 2: 8192
3
```

当用了多个数据存储池来存储数据时，你得确保均衡每个存储池的归置组数量、且归置组数量分摊到每个 OSD，这样才能达到较合理的归置组总量，并因此使得每个 OSD 无需耗费过多系统资源或拖慢连接进程就能实现较小变迁。

For instance a cluster of 10 pools each with 512 placement groups on ten OSDs is a total of 5,120 placement groups spread over ten OSDs, that is 512 placement groups per OSD. That does not use too many resources. However, if 1,000 pools were created with 512 placement groups each, the OSDs will handle ~50,000 placement groups each and it would require significantly more resources and time for peering.

设置归置组数量

要设置某存储池的归置组数量，你必须在创建它时就指定好，详情见[创建存储池](#)。一存储池的归置组数量设置好之后，还可以增加（但不可以减少），下列命令可增加归置组数量：

```
ceph osd pool set {pool-name} pg_num {pg_num}
```

你增加归置组数量后、还必须增加用于归置的归置组 (`pgp_num`) 数量，这样才会开始重均衡。 `pgp_num` 数值才是 CRUSH 算法采用的用于归置的归置组数量。虽然 `pg_num` 的增加引起了归置组的分割，但是只有当用于归置的归置组 (即 `pgp_num`) 增加以后，数据才会被迁移到新归置组里。 `pgp_num` 的数值应等于 `pg_num`。可用下列命令增加用于归置的归置组数量：

```
ceph osd pool set {pool-name} pgp_num {pgp_num}
```

获取归置组数量

要获取一个存储池的归置组数量，执行命令：

```
ceph osd pool get {pool-name} pg_num
```

获取归置组统计信息

要获取集群里归置组的统计信息，执行命令：

```
ceph pg dump [--format {format}]
```

可用格式有纯文本 `plain` (默认) 和 `json`。

获取卡住的归置组统计信息

要获取所有卡在某状态的归置组统计信息，执行命令：

```
ceph pg dump_stuck  
inactive|unclean|stale|undersized|degraded [--format  
<format>] [-t|--threshold <seconds>]
```

Inactive (不活跃) 归置组不能处理读写，因为它们在等待一个有最新数据的 OSD 复活且进入集群。

Unclean（不干净）归置组含有复制数未达到期望数量的对象，它们应该在恢复中。

Stale（不新鲜）归置组处于未知状态：存储它们的 OSD 有段时间没向监视器报告了（由 `mon_osd_report_timeout` 配置）。

可用格式有 `plain`（默认）和 `json`。阈值定义的是，归置组被认为卡住前等待的最小时间（默认 300 秒）。

获取一归置组运行图

要获取一个具体归置组的归置组图，执行命令：

```
ceph pg map {pg-id}
```

例如：

```
ceph pg map 1.6c
```

Ceph 将返回归置组图、归置组、和 OSD 状态：

```
osdmap e13 pg 1.6c (1.6c) -> up [1,0] acting [1,0]
```

获取一 PG 的统计信息

要查看一个具体归置组的统计信息，执行命令：

```
ceph pg {pg-id} query
```

洗刷归置组

要洗刷一个归置组，执行命令：

```
ceph pg scrub {pg-id}
```

Ceph 检查原始的和任何复制节点，生成归置组里所有对象的目录，然后再对比，确保没有对象丢失或不匹配，并且它们的内容一致。

恢复丢失的

如果集群丢了一或多个对象，而且必须放弃搜索这些数据，你就要把未找到的对象标记为丢失（`lost`）。

如果所有可能的位置都查询过了，而仍找不到这些对象，你也许得放弃它们了。这可能是罕见的失败组合导致的，集群在写入完成前，未能得知写入是否已执行。

当前只支持 `revert` 选项，它使得回滚到对象的前一个版本（如果它是新对象）或完全忽略它。要把 unfound 对象标记为 `lost`，执行命令：

```
ceph pg {pg-id} mark_unfound_lost revert|delete
```

Important

要谨慎使用，它可能迷惑那些期望对象存在的应用程序。

CRUSH 图

CRUSH 算法通过计算数据存储位置来确定如何存储和检索。CRUSH 授权 Ceph 客户端直接连接 OSD，而非通过一个中央服务器或经纪人。数据存储、检索算法的使用，使 Ceph 避免了单点故障、性能瓶颈、和伸缩的物理限制。

CRUSH 需要一张集群的地图，且使用 CRUSH 把数据伪随机地存储、检索于整个集群的 OSD 里。CRUSH 的讨论详情参见 [CRUSH - 可控、可伸缩、分布式地归置多副本数据](#)。

CRUSH 图包含 OSD 列表、把设备汇聚为物理位置的“桶”列表、和指示 CRUSH 如何复制存储池里的数据的规则列表。由于对所安装底层物理组织的表达，CRUSH 能模型化、并因此定

位到潜在的相关失败设备源头，典型的源头有物理距离、共享电源、和共享网络，把这些信息编码到集群运行图里，CRUSH 归置策略可把对象副本分离到不同的失败域，却仍能保持期望的分布。例如，要定位同时失败的可能性，可能希望保证数据复制到的设备位于不同机架、不同托盘、不同电源、不同控制器、甚至不同物理位置。

当你写好配置文件，用 `ceph-deploy` 部署 Ceph 后，它生成了一个默认的 CRUSH 图，对于你的沙盒环境来说它很好。然而，部署一个大规模数据集群的时候，应该好好设计自己的 CRUSH 图，因为它帮你管理 Ceph 集群、提升性能、和保证数据安全性。

例如，如果一个 OSD 挂了，CRUSH 图可帮你定位此事件中 OSD 所在主机的物理数据中心、房间、行和机架，据此你可以请求在线支持或替换硬件。

类似地，CRUSH 可帮你更快地找出问题。例如，如果一个机架上的所有 OSD 同时挂了，问题可能在于机架的交换机或电源，而非 OSD 本身。

定制的 CRUSH 图也能在归置组降级时，帮你找出冗余副本所在主机的物理位置。

Note

文本框里的代码实例可能超出了边界，读或拷贝时注意滚动。

CRUSH 位置

用 CRUSH 图层次结构所表示的 OSD 位置被称为“crush 位置”，它用键/值对列表来表示。例如，一 OSD 位于某特定行、机柜、机架、和主机，且是 CRUSH 图里名为 default 树的一部分，那么其 crush 位置可表示如下：

```
root=default row=a rack=a2 chassis=a2a host=a2a1
```

注：

1. 注意键（关键词）与顺序无关；
2. 键名（= 左边）必须是 CRUSH 内的合法 `type`，默认情况下，它包含 `root`、`datacenter`、`room`、`row`、`pod`、`pdu`、`rack`、`chassis`、和 `host`，但这些类型可修改 CRUSH 图任意定义。

3. 并非所有键都需指定，例如，默认情况下 Ceph 会自动把 `ceph-osd` 守护进程的位置设置为 `root=default host=HOSTNAME`（即是 `hostname -s`）。

CEPH-CRUSH-LOCATION 挂钩

`ceph-crush-location` 工具可为某守护进程生成默认 CRUSH 位置字符串，此位置依次基于：

1. `ceph.conf` 里的 `TYPE crush location`，例如这是 OSD 守护进程的：`osd crush location`；
2. `ceph.conf` 里的 `crush location`；
3. 默认的 `root=default host=HOSTNAME`，其中主机名由 `hostname -s` 获取。

典型部署场景下，部署软件（或系统管理员）会在此主机的 `ceph.conf` 配置文件里设置 `crush location` 域来描述此机器在数据中心或集群内的位置。这样 Ceph 守护进程和客户端就能提供基于位置的服务。

完全手动管理 CRUSH 图也是可能的，在配置中把挂钩关掉即可：

```
osd crush update on start = false
```

定制位置挂钩

定制化位置挂钩可代替通用挂钩，用于控制 OSD 在分级结构中的位置（启动时，各 OSD 都确认它们的位置正确无误）：

```
osd crush location hook = /path/to/script
```

此挂钩应该接受几个参数（下述）并向标准输出打印一行 CRUSH 位置描述：

```
$ ceph-crush-location --cluster CLUSTER --id ID --type TYPE
```

其中，集群名通常是 `ceph`，`id` 是守护进程标识符（OSD 号），守护进程类型通常是 `osd`。

编辑 CRUSH 图

要编辑现有的 CRUSH 图：

1. [获取 CRUSH 图](#)；
2. [反编译 CRUSH 图](#)；
3. 至少编辑一个[设备](#)、[桶](#)、[规则](#)；
4. [重编译 CRUSH 图](#)；
5. [注入 CRUSH 图](#)。

要激活 CRUSH 图里某存储池的规则，找到通用规则集编号，然后把它指定到那个规则集。详情参见[调整存储池](#)。

获取 CRUSH 图

要获取集群的 CRUSH 图，执行命令：

```
ceph osd getcrushmap -o {compiled-crushmap-filename}
```

Ceph 将把 CRUSH 输出（-o）到你指定的文件，由于 CRUSH 图是已编译的，所以编辑前必须先反编译。

反编译 CRUSH 图

要反编译 CRUSH 图，执行命令：

```
crushtool -d {compiled-crushmap-filename} -o {decompiled-crushmap-filename}
```

Ceph 将反编译（-d）二进制 CRUSH 图，且输出（-o）到你指定的文件。

编译 CRUSH 图

要编译 CRUSH 图，执行命令：

```
crushtool -c {decompiled-crush-map-filename} -o  
{compiled-crush-map-filename}
```

Ceph 将把已编译的 CRUSH 图保存到你指定的文件。

注入 CRUSH 图

要把 CRUSH 图应用到集群，执行命令：

```
ceph osd setcrushmap -i {compiled-crushmap-filename}
```

Ceph 将把你指定的已编译 CRUSH 图输入到集群。

CRUSH 图参数

CRUSH 图主要有 4 个主要段落。

1. **设备** 由任意对象存储设备组成，即对应一个 `ceph-osd` 进程的存储器。Ceph 配置文件里的每个 OSD 都应该有一个设备。
2. **桶类型**：定义了 CRUSH 分级结构里要用的桶类型（`types`），桶由逐级汇聚的存储位置（如行、机柜、机箱、主机等等）及其权重组成。
3. **桶例程**：定义了桶类型后，还必须声明主机的桶类型、以及规划的其它故障域。
4. **规则**：由选择桶的方法组成。

如果你用我们的某个“入门手册”配起了 Ceph，应该注意到了，你并不需要创建 CRUSH 图。Ceph 部署工具生成了默认 CRUSH 运行图，它列出了你定义在 Ceph 配置文件中的 OSD 设备、并把配置文件 `[osd]` 段下定义的各 OSD 主机声明为桶。为保证数据安全和可用，你应该创建自己的 CRUSH 图，以反映出自己集群的故障域。

Note

生成的 CRUSH 图没考虑大粒度故障域，所以你修改 CRUSH 图时要考虑上，像机柜、行、数据中心。

CRUSH 图之设备

为把归置组映射到 OSD，CRUSH 图需要 OSD 列表（即配置文件所定义的 OSD 守护进程名称），所以它们首先出现在 CRUSH 图里。要在 CRUSH 图里声明一个设备，在设备列表后面新建一行，输入 `device`、之后是唯一的数字 ID、之后是相应的 `ceph-osd` 守护进程例程名字。

```
#devices  
  
device {num} {osd.name}
```

例如：

```
#devices  
  
device 0 osd.0  
  
device 1 osd.1  
  
device 2 osd.2  
  
device 3 osd.3
```

一般来说，一个 OSD 映射到一个单独的硬盘或 RAID。

CRUSH 图之桶类型

CRUSH 图里的第二个列表定义了 bucket（桶）类型，桶简化了节点和叶子层次。节点（或非叶子）桶在分级结构里一般表示物理位置，节点汇聚了其它节点或叶子，叶桶表示 `ceph-osd` 守护进程及其对应的存储媒体。

Tip

CRUSH 中用到的 bucket 意思是分级结构中的一个节点，也就是一个位置或一部分硬件。但是在 RADOS 网关接口的术语中，它又是不同的概念。

要往 CRUSH 图中增加一种 bucket 类型，在现有桶类型列表下方新增一行，输入 type、之后是惟一数字 ID 和一个桶名。按惯例，会有一个叶子桶为 type 0，然而你可以指定任何名字（如 osd、disk、drive、storage 等等）：

```
#types  
  
type {num} {bucket-name}
```

例如：

```
# types  
  
type 0 osd  
  
type 1 host  
  
type 2 chassis  
  
type 3 rack  
  
type 4 row  
  
type 5 pdu  
  
type 6 pod  
  
type 7 room  
  
type 8 datacenter  
  
type 9 region  
  
type 10 root
```

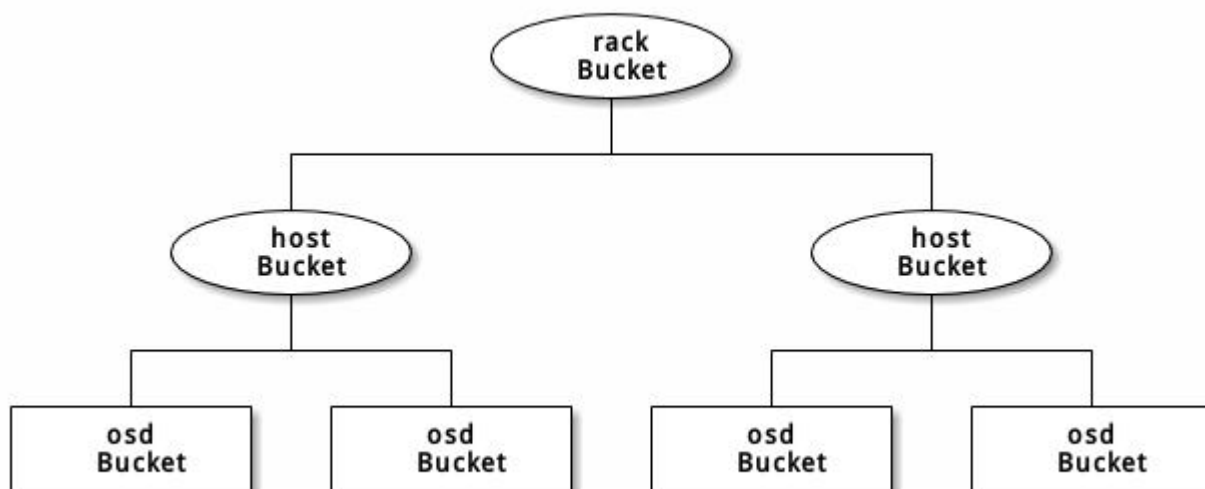
CRUSH 图之桶层次

CRUSH 算法根据各设备的权重、大致统一的概率把数据对象分布到存储设备中。CRUSH 根据你定义的集群运行图分布对象及其副本，CRUSH 图表达了可用存储设备以及包含它们的逻辑单元。

要把归置组映射到跨故障域的 OSD，一个 CRUSH 图需定义一系列分级桶类型（即现有 CRUSH 图的 `#type` 下）。创建桶分级结构的目的是按故障域隔离叶节点，像主机、机箱、机柜、电力分配单元、机群、行、房间、和数据中心。除了表示叶节点的 OSD，其它分级结构都是任意的，你可以按需定义。

我们建议 CRUSH 图内的命名符合贵公司的硬件命名规则，并且采用反映物理硬件的例程名。良好的命名可简化集群管理和故障排除，当 OSD 和/或其它硬件出问题时，管理员可轻易找到对应物理硬件。

在下例中，桶分级结构有一个名为 `osd` 的分支、和两个节点分别名为 `host` 和 `rack`。



Note

编号较高的 `rack` 桶类型汇聚编号较低的 `host` 桶类型。

位于 CRUSH 图起始部分、`#devices` 列表内是表示叶节点的存储设备，没必要声明为桶例程。位于分级结构第二低层的桶一般用于汇聚设备（即它通常是包含存储媒体的计算机，你可

以用自己喜欢的名字描述，如节点、计算机、服务器、主机、机器等等）。在高密度环境下，经常出现一机框内安装多个主机/节点的情况，因此还要考虑机框故障——比如，某一节点故障后需要拉出机框维修，这会影响多个主机/节点和其内的 OSD。

声明一个桶例程时，你必须指定其类型、唯一名称（字符串）、惟一负整数 ID（可选）、指定和各条目总容量/能力相关的权重、指定桶算法（通常是 `straw`）、和哈希（通常为 `0`，表示哈希算法 `rjenkins1`）。一个桶可以包含一到多条，这些条目可以由节点桶或叶子组成，它们可以有个权重用来反映条目的相对权重。

你可以按下列语法声明一个节点桶：

```
[bucket-type] [bucket-name] {  
  
    id [a unique negative numeric ID]  
  
    weight [the relative capacity/capability of the  
item(s)]  
  
    alg [the bucket type: uniform | list | tree |  
straw ]  
  
    hash [the hash type: 0 by default]  
  
    item [item-name] weight [weight]  
  
}
```

例如，用上面的图表，我们可以定义两个主机桶和一个机柜桶，OSD 被声明为主机桶内的条目：

```
host node1 {  
  
    id -1  
  
    alg straw  
  
    hash 0
```



```
        item osd.0 weight 1.00

        item osd.1 weight 1.00
    }

host node2 {

    id -2

    alg straw

    hash 0

    item osd.2 weight 1.00

    item osd.3 weight 1.00
}

rack rack1 {

    id -3

    alg straw

    hash 0

    item node1 weight 2.00

    item node2 weight 2.00
}
```

Note

在前述示例中，机柜桶不包含任何 OSD，它只包含低一级的主机桶、以及其内条目的权重之和。

桶类型

Ceph 支持四种桶，每种都是性能和组织简易间的折衷。如果你不确定用哪种桶，我们建议 `straw`，关于桶类型的详细讨论见 [CRUSH - 可控、可伸缩、分布式地归置多副本数据](#)，特别是 **Section 3.4**。支持的桶类型有：

1. **Uniform:** 这种桶用完全相同的权重汇聚设备。例如，公司采购或淘汰硬件时，一般都有相同的物理配置（如批发）。当存储设备权重都相同时，你可以用 `uniform` 桶类型，它允许 CRUSH 按常数把副本映射到 `uniform` 桶。权重不统一时，你应该采用其它算法。
2. **List:** 这种桶把它们的内容汇聚为链表。它基于 `RUSHP` 算法，一个列表就是一个自然、直观的**扩张集群**：对象会按一定概率被重定位到最新的设备、或者像从前一样仍保留在较老的设备上。结果是优化了新条目加入桶时的数据迁移。然而，如果从链表的中间或末尾删除了一些条目，将会导致大量没必要的挪动。所以这种桶适合**永不或极少缩减**的场景。
3. **Tree:** 它用一种二进制搜索树，在桶包含大量条目时比 `list` 桶更高效。它基于 `RUSHR` 算法，`tree` 桶把归置时间减少到了 $O(\log n)$ ，这使得它们更适合管理更大规模的设备或嵌套桶。
4. **Straw:** `list` 和 `tree` 桶用分而治之策略，给特定条目一定优先级（如位于链表开头的条目）、或避开对整个子树上所有条目的考虑。这样提升了副本归置进程的性能，但是也导致了重新组织时的次优结果，如增加、拆除、或重设某条目的权重。`straw` 桶类型允许所有条目模拟拉稻草的过程公平地相互“竞争”副本归置。

Hash

各个桶都用了一种哈希算法，当前 Ceph 仅支持 `rjenkins1`，输入 0 表示哈希算法设置为 `rjenkins1`。

调整桶的权重

Ceph 用双整形表示桶权重。权重和设备容量不同，我们建议用 1.00 作为 1TB 存储设备的相对权重，这样 0.5 的权重大概代表 500GB、3.00 大概代表 3TB。较高级桶的权重是所有枝叶桶的权重之和。

一个桶的权重是一维的，你也可以计算条目权重来反映存储设备性能。例如，如果你有很多 1TB 的硬盘，其中一些数据传输速率相对低、其他的数据传输率相对高，即使它们容量相同，也应该设置不同的权重（如给吞吐量较低的硬盘设置权重 0.8，较高的设置 1.20）。

CRUSH 图之规则

CRUSH 图支持“CRUSH 规则”概念，用以确定一个存储池里数据的归置。对大型集群来说，你可能创建很多存储池，且每个存储池都有它自己的 CRUSH 规则集和规则。默认的 CRUSH 图里，每个存储池有一条规则、一个规则集被分配到每个默认存储池。

Note

大多数情况下，你都不需要修改默认规则。新创建存储池的默认规则集是 0。

CRUSH 规则定义了归置和复制策略、或分布策略，用它可以规定 CRUSH 如何放置对象副本。例如，你也许想创建一条规则用以选择一对目的地做双路复制；另一条规则用以选择位于两个数据中心的三个目的地做三路镜像；又一条规则用 6 个设备做纠删编码。关于 CRUSH 规则的详细研究见 [CRUSH - 可控、可伸缩、分布式地归置多副本数据](#)，主要是 **Section 3.2**。

规则格式如下：

```
rule <rulename> {  
  
    ruleset <ruleset>  
  
    type [ replicated | erasure ]  
  
    min_size <min-size>  
  
    max_size <max-size>
```

```
    step take <bucket-type>

    step [choose|chooseleaf] [firstn|indep] <N>
    <bucket-type>

    step emit

}
```

ruleset

描述: 区分一条规则属于某个规则集的手段。[给存储池设置规则集](#)后激活。

目的: 规则掩码的一个组件。

类型: Integer

是否必需: Yes

默认值: 0

type

描述: 为硬盘（复制的）或 RAID 写一条规则。

目的: 规则掩码的一个组件。

类型: String

是否必需: Yes

默认值: replicated

合法取值: 当前仅支持 replicated 和 erasure

min_size

描述: 如果一个归置组副本数小于此数, CRUSH 将不应用此规则。

类型: Integer

目的: 规则掩码的一个组件。

是否必需: Yes

默认值: 1

max_size

描述: 如果一个归置组副本数大于此数, CRUSH 将不应用此规则。

类型: Integer

目的: 规则掩码的一个组件。

是否必需: Yes

默认值: 10

```
step take <bucket-name>
```

描述: 选取桶名并迭代到树底。

目的: 规则掩码的一个组件。

是否必需: Yes

实例: `step take data`

```
step choose firstn {num} type {bucket-type}
```

选取指定类型桶的数量，这个数字通常是存储池的副本数（即 `pool size`）。

描述: 如果 `{num} == 0` 选择 `pool-num-replicas` 个桶（所有可用的）；
如果 `{num} > 0 && < pool-num-replicas` 就选择那么多的桶；
如果 `{num} < 0` 它意为 `pool-num-replicas - {num}`。

目的: 规则掩码的一个组件。

先决条件:

跟在 `step take` 或 `step choose` 之后。

实例:

```
step choose firstn 1 type row
```

```
step chooseleaf firstn {num} type {bucket-type}
```

选择 `{bucket-type}` 类型的一堆桶，并从各桶的子树里选择一个叶子节点。集合内桶的数量通常是存储池的副本数（即 `pool size`）。

描述:

如果 `{num} == 0` 选择 `pool-num-replicas` 个桶（所有可用的）；
如果 `{num} > 0 && < pool-num-replicas` 就选择那么多的桶；
如果 `{num} < 0` 它意为 `pool-num-replicas - {num}`。

目的:

规则掩码的一个组件。它的使用避免了通过两步来选择一设备。

先决条件:

Follows `step take` or `step choose`.

实例:

```
step chooseleaf firstn 0 type row
```

```
step emit
```

描述:

输出当前值并清空堆栈。通常用于规则末尾，也适用于相同规则应用到不同树的情况。

目的: 规则掩码的一个组件。

先决条件:

Follows `step choose`.

实例: `step emit`

Important

把规则集编号设置到存储池，才能用一个通用规则集编号激活一或多条规则。

主亲和性

— Ceph 客户端读写数据时，总是连接 acting set 里的主 OSD（如 [2, 3, 4] 中，osd.2 是主的）。有时候某个 OSD 与其它的相比并不适合做主 OSD（比如其硬盘慢、或控制器慢），最大化硬件利用率时为防止性能瓶颈（特别是读操作），你可以调整 OSD 的主亲和性，这样 CRUSH 就尽量不把它用作 acting set 里的主 OSD 了。

```
ceph osd primary-affinity <osd-id> <weight>
```

主亲和性默认为 1（就是说此 OSD 可作为主 OSD）。此值合法范围为 0-1，其中 0 意为此 OSD 不能用作主的，1 意为 OSD 可用作主的；此权重小于 1 时，CRUSH 选择主 OSD 时选中它的可能性低。

给存储池指定 OSD

假设你想让大多数存储池坐落到使用大硬盘的 OSD 上，但是其中一些存储池映射到使用高速 SSD 的 OSD 上。在同一个 CRUSH 图内有多个独立的 CRUSH 树是可能的，定义两棵树、分别有自己的根节点——一个用于硬盘（如 root platter）、一个用于 SSD（如 root ssd），如：

```
device 0 osd.0  
  
device 1 osd.1  
  
device 2 osd.2
```



```
device 3 osd.3
```

```
device 4 osd.4
```

```
device 5 osd.5
```

```
device 6 osd.6
```

```
device 7 osd.7
```

```
host ceph-ssd-server-1 {
```

```
    id -1
```

```
    alg straw
```

```
    hash 0
```

```
    item osd.0 weight 1.00
```

```
    item osd.1 weight 1.00
```

```
}
```

```
host ceph-ssd-server-2 {
```

```
    id -2
```

```
    alg straw
```

```
    hash 0
```

```
    item osd.2 weight 1.00
```

```
    item osd.3 weight 1.00
```

```
}
```

```
host ceph-osd-platter-server-1 {
```

```
    id -3
```

```
    alg straw
```

```
    hash 0
```

```
    item osd.4 weight 1.00
```

```
    item osd.5 weight 1.00
```

```
}
```

```
host ceph-osd-platter-server-2 {
```

```
    id -4
```

```
    alg straw
```

```
    hash 0
```

```
    item osd.6 weight 1.00
```

```
    item osd.7 weight 1.00
```

```
}
```

```
root platter {
```

```
    id -5
```

```
    alg straw

    hash 0

    item ceph-osd-platter-server-1 weight 2.00

    item ceph-osd-platter-server-2 weight 2.00

}
```

```
root ssd {

    id -6

    alg straw

    hash 0

    item ceph-osd-ssd-server-1 weight 2.00

    item ceph-osd-ssd-server-2 weight 2.00

}
```

```
rule data {

    ruleset 0

    type replicated

    min_size 2

    max_size 2

    step take platter

}
```

```
        step chooseleaf firstn 0 type host

        step emit

    }
```

```
rule metadata {

    ruleset 1

    type replicated

    min_size 0

    max_size 10

    step take platter

    step chooseleaf firstn 0 type host

    step emit

}
```

```
rule rbd {

    ruleset 2

    type replicated

    min_size 0

    max_size 10

    step take platter

}
```

```
        step chooseleaf firstn 0 type host

        step emit

    }
```

```
rule platter {

    ruleset 3

    type replicated

    min_size 0

    max_size 10

    step take platter

    step chooseleaf firstn 0 type host

    step emit

}
```

```
rule ssd {

    ruleset 4

    type replicated

    min_size 0

    max_size 4

    step take ssd

}
```

```
        step chooseleaf firstn 0 type host

        step emit
    }

    rule ssd-primary {

        ruleset 5

        type replicated

        min_size 5

        max_size 10

        step take ssd

        step chooseleaf firstn 1 type host

        step emit

        step take platter

        step chooseleaf firstn -1 type host

        step emit
    }
```

然后你可以设置一个存储池，让它使用 SSD 规则：

```
ceph osd pool set <poolname> crush_ruleset 4
```

同样，用 `ssd-primary` 规则将使存储池内的各归置组用 SSD 作主 OSD，普通硬盘作副本。

增加/移动 OSD

要增加或删除在线集群里 OSD 所对应的 CRUSH 图条目，执行 `ceph osd crush set` 命令。对于 v0.48 版，执行下列：

```
ceph osd crush set {id} {name} {weight} pool={pool-name}
[ {bucket-type}={bucket-name} ... ]
```

Bobtail (v0.56) 可执行下列：

```
ceph osd crush set {id-or-name} {weight} root={pool-name}
[ {bucket-type}={bucket-name} ... ]
```

其中：

`id`

描述: OSD 的数字标识符。

类型: Integer

是否必需: Yes

实例: `0`

`name`

描述: OSD 的全名。

类型: String

是否必需: Yes

实例: `osd.0`

`weight`

描述: OSD 的 CRUSH 权重。

类型: Double

是否必需: Yes

实例: `2.0`

`root`

描述: OSD 所在树的根。

类型: Key/value pair.

是否必需: Yes

实例: `root=default`

`bucket-type`

描述: 定义 OSD 在 CRUSH 分级结构中的位置。

类型: Key/value pairs.

是否必需: No

实例: `datacenter=dc1 room=room1 row=foo rack=bar host=foo-bar-1`

下例把 `osd.0` 添加到分级结构里、或者说从前一个位置挪动一下。

```
ceph osd crush set osd.0 1.0 root=default datacenter=dc1
room=room1 row=foo rack=bar host=foo-bar-1
```

调整一 OSD 的 CRUSH 权重

要调整在线集群中一 OSD 的 CRUSH 权重，执行命令：

```
ceph osd crush reweight {name} {weight}
```

其中：

name

描述: OSD 的全名。

类型: String

是否必需: Yes

实例: `osd.0`

weight

描述: OSD 的 CRUSH 权重。

类型: Double

是否必需: Yes

实例: `2.0`

删除 OSD

要从在线集群里把一 OSD 踢出 CRUSH 图，执行命令：

```
ceph osd crush remove {name}
```

其中：

`name`

描述： OSD 全名。

类型： String

是否必需： Yes

实例： `osd.0`

增加桶

要在运行集群的 CRUSH 图中新建一个桶，用 `ceph osd crush add-bucket` 命令：

```
ceph osd crush add-bucket {bucket-name} {bucket-type}
```

其中：

`bucket-name`

描述： 桶的全名。

类型： String

是否必需： Yes

实例: `rack12`

`bucket-type`

描述: 桶的类型，它必须已存在于分级结构中。

类型: `String`

是否必需: `Yes`

实例: `rack`

下例把 `rack12` 桶加入了分级结构：

```
ceph osd crush add-bucket rack12 rack
```

移动桶

要把一个桶挪动到 CRUSH 图里的不同位置，执行命令：

```
ceph osd crush move {bucket-name} {bucket-type}={bucket-name}, [...]
```

其中：

`bucket-name`

描述: 要移动或复位的桶名。

类型: String

是否必需: Yes

实例: `foo-bar-1`

`bucket-type`

描述: 你可以指定桶在 CRUSH 分级结构里的位置。

类型: Key/value pairs.

是否必需: No

实例: `datacenter=dc1 room=room1 row=foo rack=bar host=foo-bar-1`

删除桶

要把一个桶从 CRUSH 图的分级结构中删除，可用此命令：

```
ceph osd crush remove {bucket-name}
```

Note

从 CRUSH 分级结构里删除时必须为空桶。

其中：

`bucket-name`

描述： 将要删除的桶的名字。

类型： String

是否必需： Yes

实例： `rack12`

下例从分级结构里删除了 `rack12`。

```
ceph osd crush remove rack12
```

可调选项

New in version 0.48.

在 CRUSH 最初实现时加入的几个幻数，现在看来已成问题。作为过渡方法，较新版的 CRUSH（从 0.48 起）允许调整这些值。

最近发布的 Ceph 允许在 CRUSH 图里使用可调值，然而老客户端和守护进程不会正确地调整过的 CRUSH 图交互，为应对这种情况，现在多了个功能位 `CRUSH_TUNABLES`（值 `0x40000`）和 `CRUSH_TUNABLES2` 来反映支持可调值。

如果 `ceph-mon` 或 `ceph-osd` 进程现在用的 `OSDMap` 有非遗留值，它将要求连接它的客户端和守护进程有 `CRUSH_TUNABLES` 或 `CRUSH_TUNABLES2` 功能位。

将来，新建集群的可调值其默认值会更好。这要等到此功能进入内核客户端的时间足够长，对大多数用户来说已是无痛的过渡。

过时值的影响

遗留值导致几个不当行为：

- 如果分级结构的支部只有少量设备，一些 PG 的副本数小于期望值，这通常出现在一些子结构里，`host` 节点下少数 OSD 嵌套到了其他 OSD 里。
- 大型集群里，小部分 PG 映射到的 OSD 数目小于期望值，有多层结构（如：机架行、机架、主机、OSD）时这种情况更普遍。
- 当一些 OSD 标记为 `out` 时，数据倾向于重分布到附近 OSD 而非整个分级结构。

CRUSH_TUNABLES

- `choose_local_tries`: 本地重试次数。以前是 2，最优值是 0。
- `choose_local_fallback_tries`: 以前 5，现在是 0。
- `choose_total_tries`: 选择一个条目的最大尝试次数。以前 19，后来的测试表明，对典型的集群来说 50 更合适。最相当大的集群来说，更大的值也许必要。

CRUSH_TUNABLES2

- `chooseleaf_descend_once`: 是否重递归叶子选择，或只试一次、并允许最初归置组重试。以前默认 0，最优为 1。

CRUSH_TUNABLES3

- `chooseleaf_vary_r`: 根据父节点已做过多少尝试，递归选叶是否应该以非零值 `r` 开始。原先的默认值是 0，但是用此值的话 CRUSH 有时候会找不到映射关系；较优的值（计算代价和正确性合理）是 1。然而，较老的集群里面已经有大量数据，从 0

改为 1 会导致大量的数据迁移；4 或 5 时 CRUSH 也能正确找到映射，而且数据迁移少的多。

支持 CRUSH_TUNABLES 的客户端版本

- argonaut 系列，v0.48.1 或更高版
- v0.49 或更高版
- Linux 内核版本大于 v3.6（对文件系统和 RBD 客户端都一样）

支持 CRUSH_TUNABLES2 的客户端版本

- v0.55 或更高版，包括 bobtail 系列 (v0.56.x)
- Linux 内核版本大于 v3.9（对文件系统和 RBD 客户端都一样）

支持 CRUSH_TUNABLES3 的客户端版本

- v0.78 (firefly) 或更高版
- Linux 内核版本大于 v3.15（对文件系统和 RBD 客户端都一样）

可调选项非最优时发出警告

从 v0.74 起，如果 CRUSH 可调选项不是最优值（v0.73 版里的默认值）Ceph 就会发出健康告警，有两种方法可消除这些告警：

1. 调整现有集群上的可调选项。注意，这可能会导致一些数据迁移（可能有 10% 之多）。这是推荐的办法，但是在生产集群上要注意此调整对性能带来的影响。此命令可启用较优可调选项：

```
ceph osd crush tunables optimal
```

如果切换得不太顺利（如负载太高）且切换才不久，或者有客户端兼容问题（较老的 cephfs 内核驱动或 rbd 客户端、或早于 bobtail 的 librados 客户端），你可以这样切回：


```
ceph osd crush tunables legacy
```

2. 不对 CRUSH 做任何更改也能消除报警，把下列配置加入 `ceph.conf` 的 `[mon]` 段下：

```
mon warn on legacy crush tunables = false
```

为使变更生效需重启所有监视器，或者执行下列命令：

```
ceph tell mon.* injectargs --no-mon-warn-on-legacy-crush-tunables
```

一些要点

- 调整这些值将使一些 PG 在存储节点间移位，如果 Ceph 集群已经存储了大量数据，做好移动一部分数据的准备。
- 一旦更新运行图，`ceph-osd` 和 `ceph-mon` 就会开始向新建连接要求功能位，然而，之前已经连接的客户端如果不支持新功能将行为失常。
- 如果 CRUSH 可调值更改过、然后又改回了默认值，`ceph-osd` 守护进程将不要求支持此功能，然而，OSD 连接建立进程要能检查和理解旧地图。因此，集群如果用非默认 CRUSH 值就不应该再运行版本小于 0.48.1 的 `ceph-osd`，即使最新版地图已经回滚到了遗留默认值。

调整 CRUSH

更改 crush 可调值的最简方法就是改到一个已知配置，它们有：

- `legacy`: 采用 `argonaut` 及更低版本的行为；
- `argonaut`: 采用 `argonaut` 版最初的配置；
- `bobtail`: 采用 `bobtail` 版的配置；
- `firefly`: 采用 `firefly` 版的配置；
- `optimal`: 采用当前最佳配置；
- `default`: 新建集群可采用当前默认值。

你可以在运行着的集群上选择一个配置：

```
ceph osd crush tunables {PROFILE}
```

要注意，这可能产生一些数据迁移。

调整 CRUSH —— 强硬方法

如果你能保证所有客户端都运行最新代码，你可以这样调整可调值：从集群抽取 CRUSH 图、修改值、重注入。

- 提抽取最新 CRUSH 图：

```
ceph osd getcrushmap -o /tmp/crush
```

- 调整可调参数。这些值在我们测试过的大、小型集群上都有最佳表现。在极端情况下，你需要给 `crushtool` 额外指定 `--enable-unsafe-tunables` 参数才行：

```
crushtool -i /tmp/crush --set-choose-local-tries 0 -  
-set-choose-local-fallback-tries 0 --set-choose-  
total-tries 50 -o /tmp/crush.new
```

- 重注入修改的图。

```
ceph osd setcrushmap -i /tmp/crush.new
```

遗留值

CRUSH 可调参数的遗留值可以用下面命令设置：

```
crushtool -i /tmp/crush --set-choose-local-tries 2 --set-  
choose-local-fallback-tries 5 --set-choose-total-tries 19
```

```
--set-chooseleaf-descend-once 0 --set-chooseleaf-vary-r 0  
-o /tmp/crush.legacy
```

再次申明，`--enable-unsafe-tunables` 是必需的，而且前面也提到了，回退到遗留值后慎用旧版 `ceph-osd` 进程，因为此功能位不是完全强制的。

增加/删除 OSD

如果您的集群已经在运行，你可以在运行时添加或删除 OSD 。

增加 OSD

你迟早要扩容集群，Ceph 允许在运行时增加 OSD。在 Ceph 里，一个 OSD 一般是一个 `ceph-osd` 守护进程，它运行在硬盘之上，如果你有多个硬盘，可以给每个硬盘启动一个 `ceph-osd` 守护进程。

通常，你应该监控集群容量，看是否达到了容量上限，因为达到了它的 `near full` 比率后，要增加一或多个 OSD 来扩容。

Warning

不要等空间满了再增加 OSD，空间使用率达到 `near full` 比率后，OSD 失败可能导致集群空间占满。

部署硬件

如果你通过增加主机来增加 OSD，关于 OSD 服务器硬件的配置请参见[硬件推荐](#)。要把一台 OSD 主机加入到集群，首先要安装最新版的 Linux，而且存储硬盘要做好必要的准备，详情参见[文件系统推荐](#)。

把 OSD 主机添加到集群机架上，连接好网络、确保网络通畅。详情见[网络配置参考](#)。

安装必要软件

在手动部署的集群里，你必须手动安装 Ceph 软件包，详情见[安装 Ceph（手动）](#)。你应该配置一个无密码登录 SSH 的用户，且他有 root 权限。

增加 OSD（手动）

此过程要设置一个 `ceph-osd` 守护进程，让它使用一个硬盘，且让集群把数据发布到 OSD。如果一台主机有多个硬盘，可以重复此过程，把每个硬盘配置为一个 OSD。

要添加 OSD，要依次创建数据目录、把硬盘挂载到目录、把 OSD 加入集群、然后把它加入 CRUSH 图。

往 CRUSH 图里添加 OSD 时建议设置权重，硬盘容量每年增长 40%，所以较新的 OSD 主机拥有更大的空间（即它们可以有更大的权重）。

Tip

Ceph 喜欢统一的硬件，与存储池无关。如果你要新增容量不一的驱动器，还需调整它们的权重。但是，为实现最佳性能，CRUSH 的分级结构最好按类型、容量定义。

1. 创建 OSD。如果未指定 UUID，OSD 启动时会自动生成一个。下列命令会输出 OSD 号，后续步骤你会用到。

```
ceph osd create [{uuid} [{id}]]
```

如果指定了可选参数 `{id}`，那么它将作为 OSD id。要注意，如果此数字已使用，此命令会出错。

Warning

一般来说，我们不建议指定 `{id}`。因为 ID 是按照数组分配的，跳过一些依然会浪费内存；尤其是跳过太多、或者集群很大时，会更明显。若未指定 `{id}`，将用最小可用数字。

2. 在新 OSD 主机上创建默认目录。

```
3. ssh {new-osd-host}
```

```
sudo mkdir /var/lib/ceph/osd/ceph-{osd-number}
```

4. 如果准备用于 OSD 的是单独的而非系统盘，先把它挂载到刚创建的目录下：

```
5. ssh {new-osd-host}
```

```
6. sudo mkfs -t {fstype} /dev/{drive}
```

```
sudo mount -o user_xattr /dev/{hdd}  
/var/lib/ceph/osd/ceph-{osd-number}
```

7. 初始化 OSD 数据目录。

```
8. ssh {new-osd-host}
```

```
ceph-osd -i {osd-num} --mkfs --mkkey
```

运行 `ceph-osd` 时目录必须是空的。

9. 注册 OSD 认证密钥，`ceph-{osd-num}` 路径里的 `ceph` 值应该是 `$cluster-$id`，如果你的集群名字不是 `ceph`，那就用改过的名字。

```
ceph auth add osd.{osd-num} osd 'allow *' mon 'allow  
rwx' -i /var/lib/ceph/osd/ceph-{osd-num}/keyring
```

10. 把 OSD 加入 CRUSH 图，这样它才开始收数据。用 `ceph osd crush add` 命令把 OSD 加入 CRUSH 分级结构的合适位置。如果你指定了不止一个桶，此命令会把

它加入你所指定的桶中最具体的一个，*并且*把此桶挪到你指定的其它桶之内。**重要：**如果你只指定了 root 桶，此命令会把 OSD 直接挂到 root 下面，但是 CRUSH 规则期望它位于主机内。

若用的是 v0.48 版，执行下列命令：

```
ceph osd crush add {id} {name} {weight} [{bucket-  
type}={bucket-name} ...]
```

若用的是 v0.56 及更高版，执行下列命令：

```
ceph osd crush add {id-or-name} {weight} [{bucket-  
type}={bucket-name} ...]
```

你也可以反编译 CRUSH 图、把 OSD 加入设备列表、以桶的形式加入主机（如果它没在 CRUSH 图里）、以条目形式把设备加入主机、分配权重、重编译并应用它。详情参见[增加/移动 OSD](#)。

Argonaut 0.48 版最佳实践

为降低对用户 I/O 性能的影响，加入 CRUSH 图时应该把 OSD 的初始权重设为 0，然后每次增大一点、逐步增大 CRUSH 权重。例如每次增加 0.2：

```
ceph osd crush reweight {osd-id} .2
```

迁移完成前，可以依次把权重重置为 0.4、0.6 等等，直到达到期望权重。

为降低 OSD 失败的影响，你可以设置：

```
mon osd down out interval = 0
```

它防止挂了的 OSD 自动被标记为 out，然后逐步降低其权重：

```
ceph osd reweight {osd-num} .8
```

还是等着集群完成数据迁移，然后再次调整权重，直到权重为 0。注意，这会阻止集群在发生故障时自动复制数据，所以要确保监控的及时性，以便管理员迅速介入。

注意，以上经验在 Bobtail 及后续版本已不再必要。

启动 OSD

把 OSD 加入 Ceph 后，OSD 就在配置里了。然而它还没运行，它现在的状态为 `down` 且 `out`。你必须先启动 OSD 它才能收数据。可以用管理主机上的 `service ceph`、或从 OSD 所在主机启动。

在 Debian/Ubuntu 上用 Upstart。

```
sudo start ceph-osd id={osd-num}
```

在 CentOS/RHEL 上用 sysvinit。

```
sudo /etc/init.d/ceph start osd.{osd-num}
```

一旦你启动了 OSD，其状态就变成了 `up` 且 `in`。

观察数据迁移

把新 OSD 加入 CRUSH 图后，Ceph 会重新均衡服务器，一些归置组会迁移到新 OSD 里，你可以用 `ceph` 命令观察此过程。

```
ceph -w
```

你会看到归置组状态从 `active+clean` 变为 `active, some degraded objects`（有降级的对象）、且迁移完成后回到 `active+clean` 状态。（Ctrl-c 退出）

删除 OSD（手动）

要想缩减集群尺寸或替换硬件，可在运行时删除 OSD。在 Ceph 里，一个 OSD 通常是一台主机上的一个 `ceph-osd` 守护进程、它运行在一个硬盘之上。如果一台主机上有多个数据盘，你得挨个删除其对应 `ceph-osd`。通常，操作前应该检查集群容量，看是否快达到上限了，确保删除 OSD 后不会使集群达到 `near full` 比率。

Warning

删除 OSD 时不要让集群达到 `full ratio` 值，删除 OSD 可能导致集群达到或超过 `full ratio` 值。

把 OSD 踢出集群

删除 OSD 前，它通常是 `up` 且 `in` 的，要先把它踢出集群，以使 Ceph 启动重新均衡、把数据拷贝到其他 OSD。

```
ceph osd out {osd-num}
```

观察数据迁移

一旦把 OSD 踢出（`out`）集群，Ceph 就会开始重新均衡集群、把归置组迁出将删除的 OSD。你可以用 `ceph` 工具观察此过程。

```
ceph -w
```

你会看到归置组状态从 `active+clean` 变为 `active, some degraded objects`、迁移完成后最终回到 `active+clean` 状态。（`Ctrl-c` 中止）

Note

有时候，（通常是只有几台主机的小集群，比如小型测试集群）拿出（out）某个 OSD 可能会使 CRUSH 进入临界状态，这时某些 PG 一直卡在 active+remapped 状态。如果遇到了这种情况，你应该把此 OSD 标记为 in，用这个命令：

```
``ceph osd in {osd-num}``
```

等回到最初的状态后，把它的权重设置为 0，而不是标记为 out，用此命令：

```
``ceph osd crush reweight osd.{osd-num} 0``
```

执行后，你可以观察数据迁移过程，应该可以正常结束。把某一 OSD 标记为 out 和权重改为 0 的区别在于，前者，包含此 OSD 的桶、其权重没变；而后一种情况下，桶的权重变了（降低了此 OSD 的权重）。某些情况下，reweight 命令更适合“小”集群。

停止 OSD

把 OSD 踢出集群后，它可能仍在运行，就是说其状态为 up 且 out。删除前要先停止 OSD 进程。

```
ssh {osd-host}  
  
sudo /etc/init.d/ceph stop osd.{osd-num}
```

停止 OSD 后，状态变为 down。

删除 OSD

此步骤依次把一个 OSD 移出集群 CRUSH 图、删除认证密钥、删除 OSD 图条目、删除 ceph.conf 条目。如果主机有多个硬盘，每个硬盘对应的 OSD 都得重复此步骤。

1. 删除 CRUSH 图的对应 OSD 条目，它就不再接收数据了。你也可以反编译 CRUSH 图、删除 device 列表条目、删除对应的 host 桶条目或删除 host 桶（如果它在 CRUSH 图里，而且你想删除主机），重编译 CRUSH 图并应用它。详情参见[删除 OSD](#)。

```
ceph osd crush remove {name}
```

2. 删除 OSD 认证密钥：

```
ceph auth del osd.{osd-num}
```

`ceph-{osd-num}` 路径里的 `ceph` 值是 `$cluster-$id`，如果集群名字不是 `ceph`，这里要更改。

3. 删除 OSD。

```
4. ceph osd rm {osd-num}
```

```
5. #for example
```

```
ceph osd rm 1
```

6. 登录到保存 `ceph.conf` 主拷贝的主机。

```
7. ssh {admin-host}
```

```
8. cd /etc/ceph
```

```
vim ceph.conf
```

9. 从 `ceph.conf` 配置文件里删除对应条目。

```
10.      [osd.1]
```

```
host = {hostname}
```

11. 从保存 `ceph.conf` 主拷贝的主机，把更新过的 `ceph.conf` 拷贝到集群其他主机的 `/etc/ceph` 目录下。

增加/删除监视器

你的集群启动并运行后，可以在运行时增加、或删除监视器。请参考[手动部署](#)或[监视器自举启动](#)完成初始设置。

增加监视器

Ceph 监视器是轻量级进程，它维护着集群运行图的主副本。一个集群可以只有一个监视器，我们推荐生产环境至少 3 个监视器。Ceph 使用 [Paxos](#) 算法的一个变种对各种图、以及其它对集群来说至关重要的信息达成共识。由于 Paxos 算法天生要求大部分监视器在运行，以形成法定人数（并因此达成共识）。

It is advisable to run an odd-number of monitors but not mandatory. An odd-number of monitors has a higher resiliency to failures than an even-number of monitors. For instance, on a 2 monitor deployment, no failures can be tolerated in order to maintain a quorum; with 3 monitors, one failure can be tolerated; in a 4 monitor deployment, one failure can be tolerated; with 5 monitors, two failures can be tolerated. This is why an odd-number is advisable. Summarizing, Ceph needs a majority of monitors to be running (and able to communicate with each other), but that majority can be achieved using a single monitor, or 2 out of 2 monitors, 2 out of 3, 3 out of 4, etc.

For an initial deployment of a multi-node Ceph cluster, it is advisable to deploy three monitors, increasing the number two at a time if a valid need for more than three exists.

正因为监视器是轻量级的，所以有可能在作为 OSD 的主机上同时运行它；然而，我们推荐运行于单独主机，因为与内核的 `fsync` 问题会影响性能。

Note

这里的大多数监视器之间必须能互通，这样才能形成法定人数。

部署硬件

如果你增加新监视器时要新增一台主机，关于其最低硬件配置请参见[硬件推荐](#)。要增加一个监视器主机，首先要安装最新版的 Linux（如 Ubuntu 12.04 或者 RHEL 7）。

把监视器主机安装上架，连通网络。

安装必要软件

手动部署的集群，Ceph 软件包必须手动装，详情参见[安装软件包](#)。应该配置一个用户，使之可以无密码登录 SSH、且有 root 权限。

增加监视器（手动）

本步骤创建 `ceph-mon` 数据目录、获取监视器运行图和监视器密钥环、增加一个 `ceph-mon` 守护进程。如果这导致只有 2 个监视器守护进程，你可以重演此步骤来增加一或多个监视器，直到你拥有足够多 `ceph-mon` 达到法定人数。

现在该指定监视器的标识号了。传统上，监视器曾用单个字母（`a`、`b`、`c`）命名，但你可以指定任何形式。在本文档里，要记住 `{mon-id}` 应该是你所选的标识号，不包含 `mon.` 前缀，如在 `mon.a` 中，其 `{mon-id}` 是 `a`。

1. 在新监视器主机上创建默认目录：

```
2. ssh {new-mon-host}
```

```
sudo mkdir /var/lib/ceph/mon/ceph-{mon-id}
```

3. 创建临时目录 `{tmp}`，用以保存此过程中用到的文件。此目录要不同于前面步骤创建的监视器数据目录，且完成后可删除。

```
mkdir {tmp}
```

4. 获取监视器密钥环，`{tmp}` 是密钥环文件保存路径、`{filename}` 是包含密钥的文件名。

```
ceph auth get mon. -o {tmp}/{key-filename}
```

5. 获取监视器运行图，`{tmp}` 是获取到的监视器运行图、`{filename}` 是包含监视器运行图的文件名。

```
ceph mon getmap -o {tmp}/{map-filename}
```

6. 准备第一步创建的监视器数据目录。必须指定监视器运行图路径，这样才能获得监视器法定人数和它们 `fsid` 的信息；还要指定监视器密钥环路径。

```
sudo ceph-mon -i {mon-id} --mkfs --monmap  
{tmp}/{map-filename} --keyring {tmp}/{key-filename}
```

7. 启动新监视器，它会自动加入机器。守护进程需知道绑定到哪个地址，通过 `--public-addr {ip:port}` 或在 `ceph.conf` 里的相应段设置 `mon addr` 可以指定。

```
ceph-mon -i {mon-id} --public-addr {ip:port}
```

删除监视器

从集群删除监视器时，必须认识到，Ceph 监视器用 PASOX 算法关于主集群运行图达成共识。必须有足够多的监视器才能对集群运行图达成共识。

删除监视器（手动）

本步骤从集群删除 `ceph-mon` 守护进程，如果此步骤导致只剩 2 个监视器了，你得增加或删除一个监视器，直到凑足法定人数所必需的 `ceph-mon` 数。

1. 停止监视器。

```
service ceph -a stop mon.{mon-id}
```

2. 从集群删除监视器。

```
ceph mon remove {mon-id}
```

3. 删除 `ceph.conf` 对应条目。

从不健康集群删除监视器

本步骤从不健康集群删除 `ceph-mon`，例如集群内的监视器不能形成法定人数。

1. 停止所有监视器主机上的所有 `ceph-mon` 守护进程。

```
2. ssh {mon-host}
```

```
3. service ceph stop mon || stop ceph-mon-all
```

```
# 要在所有监视器主机上执行
```

4. 找出一个活着的监视器并登录其所在主机。

```
ssh {mon-host}
```

5. 提取 monmap 副本。

```
6. ceph-mon -i {mon-id} --extract-monmap {map-path}
```

```
7. # 多数情况下都是：
```

```
ceph-mon -i `hostname` --extract-monmap /tmp/monmap
```

8. 删除不保留或有问题的监视器。例如，如果你有 3 个监视器 `mon.a`、`mon.b` 和 `mon.c`，其中仅保留 `mon.a`，按如下步骤：

```
9. monmaptool {map-path} --rm {mon-id}
```

```
10.      # 例如
```

```
11.      monmaptool /tmp/monmap --rm b
```

```
monmaptool /tmp/monmap --rm c
```

12. 把去除过监视器后剩下的运行图注入存活的监视器。比如，用下列命令把一张运行图注入 `mon.a` 监视器：

```
13.      ceph-mon -i {mon-id} --inject-monmap {map-path}
```

```
14.      # for example,
```

```
ceph-mon -i a --inject-monmap /tmp/monmap
```

15. 只启动保留下来的监视器。
16. 确认这些监视器形成了法定人数 (`ceph -s`)。
17. 你也许得把已删除监视器的数据目录 `/var/lib/ceph/mon` 备份到安全位置，如果您对其余监视器很有信心、或者有足够的冗余，也可以删除。

更改监视器的 IP 地址

Important

现有监视器不应该更改其 IP 地址。

监视器是 Ceph 集群的关键组件，它们要维护一个法定人数，这样整个系统才能正常工作。要确立法定人数，监视器得互相发现对方，Ceph 对监视器的发现要求严格。

Ceph 客户端及其它 Ceph 守护进程用 `ceph.conf` 发现监视器，然而，监视器之间用监视器运行图发现对方，而非 `ceph.conf`。例如，你看过的[增加监视器（手动）](#)，会发现创建新监视器时得获取当前集群的 monmap，因为它是 `ceph-mon -i {mon-id} \ --mkfs` 命令的必要参数。下面几段解释了 Ceph 监视器的一致性要求，和几种改 IP 的安全方法。

一致性要求

监视器发现集群内的其它监视器时总是参照 monmap 的本地副本，用 monmap 而非 `ceph.conf` 可避免因配置错误（例如在 `ceph.conf` 指定监视器地址或端口时拼写错误）而损坏集群。正因为监视器用 monmaps 相互发现、且共享于客户端和其它 Ceph 守护进程间，所以 monmap 给监视器提供了苛刻的一致性保证。

苛刻的一致性要求也适用于 monmap 的更新，因为任何有关监视器的更新、monmap 的更改都通过名为 [Paxos](#) 的分布式一致性算法运行。为保证法定人数里的所有监视器都持有同版本 monmap，所有监视器都要赞成 monmap 的每一次更新，像增加、删除监视器。monmap 的更新是增量的，这样监视器都有最近商定的版本以及一系列之前版本，这样可使一个有较老 monmap 的监视器赶上集群当前的状态。

如果监视器通过 Ceph 配置文件而非 monmap 相互发现，就会引进额外风险，因为 Ceph 配置文件不会自动更新和发布。监视器有可能用了较老的 `ceph.conf` 而导致不能识别某监视器、掉出法定人数、或者发展为一种 [Paxos](#) 不能精确确定当前系统状态的情形。总之，更改现有监视器的 IP 地址必须慎之又慎。

更改监视器 IP 地址（正确方法）

仅仅在 `ceph.conf` 里更改监视器的 IP 不足以让集群内的其它监视器接受更新。要更改一个监视器的 IP 地址，你必须以先以想用的 IP 地址增加一个监视器（见[增加监视器（手动）](#)），确保新监视器成功加入法定人数，然后删除用旧 IP 的监视器，最后更新 `ceph.conf` 以确保客户端和其它守护进程得知新监视器的 IP 地址。

例如，我们假设有 3 个监视器，如下：


```
[mon.a]

    host = host01

    addr = 10.0.0.1:6789

[mon.b]

    host = host02

    addr = 10.0.0.2:6789

[mon.c]

    host = host03

    addr = 10.0.0.3:6789
```

要把 host04 上 mon.c 的 IP 改为 10.0.0.4，按照[增加监视器（手动）](#)里的步骤增加一个新监视器 mon.d，确认它运行正常后再删除 mon.c，否则会破坏法定人数；最后依照[删除监视器（手动）](#)删除 mon.c。3 个监视器都要更改的话，每次都要重复一次。

更改监视器 IP 地址（凌乱方法）

可能有时候监视器不得不挪到不同的网络、数据中心的不同位置、甚至不同的数据中心，这是可能的，但过程有点惊险。

在这种情形下，一种方法是用所有监视器的新 IP 地址生成新 monmap，并注入到集群内的所有监视器。对大多数用户来说，这并不简单，好在它不常见。再次重申，监视器不应该更改 IP 地址。

以前面的监视器配置为例，假设你想把所有监视器的 IP 从 10.0.0.x 改为 10.1.0.x，并且两个网络互不相通，步骤如下：

1. 获取监视器运行图，其中 {tmp} 是所获取的运行图路径，{filename} 是监视器运行图的文件名。

```
ceph mon getmap -o {tmp}/{filename}
```

2. 下面是一个 monmap 内容示例：

```
3. $ monmaptool --print {tmp}/{filename}
4.
5. monmaptool: monmap file {tmp}/{filename}
6. epoch 1
7. fsid 224e376d-c5fe-4504-96bb-ea6332a19e61
8. last_changed 2012-12-17 02:46:41.591248
9. created 2012-12-17 02:46:41.591248
10.      0: 10.0.0.1:6789/0 mon.a
11.      1: 10.0.0.2:6789/0 mon.b
```

```
2: 10.0.0.3:6789/0 mon.c
```

12. 删除现有监视器。

```
13.      $ monmaptool --rm a --rm b --rm c
      {tmp}/{filename}
14.
15.      monmaptool: monmap file {tmp}/{filename}
16.      monmaptool: removing a
```

```
17.      monmaptool: removing b
```

```
18.      monmaptool: removing c
```

```
monmaptool: writing epoch 1 to {tmp}/{filename} (0  
monitors)
```

19. 添加新监视器位置。

```
20.      $ monmaptool --add a 10.1.0.1:6789 --add b  
10.1.0.2:6789 --add c 10.1.0.3:6789 {tmp}/{filename}
```

```
21.
```

```
22.      monmaptool: monmap file {tmp}/{filename}
```

```
monmaptool: writing epoch 1 to {tmp}/{filename} (3  
monitors)
```

23. 检查新内容。

```
24.      $ monmaptool --print {tmp}/{filename}
```

```
25.
```

```
26.      monmaptool: monmap file {tmp}/{filename}
```

```
27.      epoch 1
```

```
28.      fsid 224e376d-c5fe-4504-96bb-ea6332a19e61
```

```
29.      last_changed 2012-12-17 02:46:41.591248
```

```
30.      created 2012-12-17 02:46:41.591248
```

```
31.      0: 10.1.0.1:6789/0 mon.a
```

```
32.      1: 10.1.0.2:6789/0 mon.b
```

```
2: 10.1.0.3:6789/0 mon.c
```

从这里开始，假设监视器（及存储）已经被安装到了新位置。下一步把修正的 monmap 散播到新监视器，并且注入每个监视器。

1. 首先，停止所有监视器，注入必须在守护进程停止时进行。
2. 注入 monmap。

```
ceph-mon -i {mon-id} --inject-monmap {tmp}/{filename}
```

3. 重启监视器。

到这里，到新位置的迁移完成，监视器应该照常运行了。

控制命令

监视器命令

监视器命令用 `ceph` 工具发出：

```
ceph [-m monhost] {command}
```

命令格式通常是（但不总是）：

```
ceph {subsystem} {command}
```

系统命令

下列命令显示集群状态：

```
ceph -s  
  
ceph status
```

下列命令显示集群状态的运行摘要、及主要事件：

```
ceph -w
```

下列命令显示监视器法定人数状态，包括哪些监视器参与着、哪个是首领。

```
ceph quorum_status
```

下列命令查询单个监视器状态，包括是否在法定人数里。

```
ceph [-m monhost] mon_status
```

认证子系统

要添加一个 OSD 的密钥环，执行下列命令：

```
ceph auth add {osd} {--in-file|-i} {path-to-osd-keyring}
```

要列出集群的密钥及其能力，执行下列命令：

```
ceph auth list
```

归置组子系统

要显示所有归置组的统计信息，执行下列命令：

```
ceph pg dump [--format {format}]
```

可用输出格式有 `plain`（默认）和 `json`。

要显示卡在某状态的所有归置组，执行下列命令：

```
ceph pg dump_stuck  
inactive|unclean|stale|undersized|degraded [--format  
{format}] [-t|--threshold {seconds}]
```

`--format` 可以是 `plain`（默认）或 `json`

`--threshold` 定义了多久算“卡住了”（默认 300 秒）

Inactive 归置组不能处理读或写，因为它们在等待数据及时更新的 OSD 回来。

Unclean 归置组包含副本数未达期望值的对象，它们应该在恢复中。

Stale 归置组处于未知状态——归置组所托付的 OSD 有一阵没向监视器报告了（由 `mon osd report timeout` 配置）。

删除“丢失”对象，或者恢复到其先前状态，可以是前一版本、或如果刚创建就干脆删除。

```
ceph pg {pgid} mark_unfound_lost revert|delete
```

OSD 子系统

查询 OSD 子系统状态：

```
ceph osd stat
```

把最新的 OSD 运行图拷贝到一个文件，参见 [osdmapprool](#)：

```
ceph osd getmap -o file
```

从最新 OSD 运行图拷出 CRUSH 图：

```
ceph osd getcrushmap -o file
```

前述功能等价于：

```
ceph osd getmap -o /tmp/osdmap  
  
osdmapprool /tmp/osdmap --export-crush file
```

转储 OSD 运行图，`-f` 的可用格式有 `plain` 和 `json`，如未指定 `--format` 则转储为纯文本。

```
ceph osd dump [--format {format}]
```

把 OSD 运行图转储为树，每个 OSD 一行、包含权重和状态。

```
ceph osd tree [--format {format}]
```

找出某对象在哪里或应该在哪里：

```
ceph osd map <pool-name> <object-name>
```

增加或挪动一个新 OSD 条目，要给出 id/name/weight 和位置参数。

```
ceph osd crush set {id} {weight} [{loc1}] [{loc2} ...]
```

从现有 CRUSH 图删除存在的条目（ OSD ）：

```
ceph osd crush remove {name}
```

从现有 CRUSH 图删除存在的空桶：

```
ceph osd crush remove {bucket-name}
```

把有效的桶从分级结构里的一个位置挪到另一个。

```
ceph osd crush move {id} {loc1} [{loc2} ...]
```

设置 {name} 所指条目的权重为 {weight}。

```
ceph osd crush reweight {name} {weight}
```

创建集群快照。

```
ceph osd cluster_snap {name}
```

把 OSD 标记为丢失，有可能导致永久性数据丢失，慎用！

```
ceph osd lost {id} [--yes-i-really-mean-it]
```

创建新 OSD。如果未指定 ID，有可能的话将自动分配个新 ID。

```
ceph osd create [{uuid}]
```

删除指定 OSD。


```
ceph osd rm [{id}...]
```

查询 OSD 运行图里的 max_osd 参数。

```
ceph osd getmaxosd
```

导入指定 CRUSH 图。

```
ceph osd setcrushmap -i file
```

设置 OSD 运行图里的 max_osd 参数，扩展存储集群时有必要。

```
ceph osd setmaxosd
```

把 ID 为 {osd-num} 的 OSD 标记为 down。

```
ceph osd down {osd-num}
```

把 OSD {osd-num} 标记为数据分布之外（即不给分配数据）。

```
ceph osd out {osd-num}
```

把 OSD {osd-num} 标记为数据分布之内（即分配了数据）。

```
ceph osd in {osd-num}
```

列出 Ceph 集群载入的类。

```
ceph class list
```

设置或清空 OSD 运行图里的暂停标记。若设置了，不会有 IO 请求发送到任何 OSD ；用 `unpause` 清空此标记会导致重发未决的请求。

```
ceph osd pause  
  
ceph osd unpause
```

把 `{osd-num}` 的权重设置为 `{weight}`，权重相同的两个 OSD 大致会收到相同的 I/O 请求、并存储相同数量的数据。`ceph osd reweight` 命令可给 OSD 设置一个增益权重，有效值在 0 和 1 之间，它使得 CRUSH 重新归置一定数量的、本应该放到此处的数据。它不会影响 crush 图里所分配的权重，在 CRUSH 分布算法没能理想地执行时，它可作为一种纠正手段。比如，假设你的某个 OSD 使用率达到了 90%，但其它的大致都在 50%，这时你就可以试着下调此权重来补偿它。

```
ceph osd reweight {osd-num} {weight}
```

重设所有滥用 OSD 的权重，它默认会下调达到平均利用率 120% 的那些 OSD，除非你指定了阈值。

```
ceph osd reweight-by-utilization [threshold]
```

增加、删除黑名单里的地址。增加地址的时候可以指定有效期，否则有效期为 1 小时。黑名单里的地址不允许连接任何 OSD，此技术常用于防止滞后的元数据服务器“错爱” OSD 上的数据。

这些命令大多只在故障测试时有用，因为黑名单是自动维护的，无需手动干涉。

```
ceph osd blacklist add ADDRESS[:source_port] [TIME]  
  
ceph osd blacklist rm ADDRESS[:source_port]
```

创建/删除存储池快照。

```
ceph osd pool mksnap {pool-name} {snap-name}
```

```
ceph osd pool rmsnap {pool-name} {snap-name}
```

创建/删除/重命名存储池。

```
ceph osd pool create {pool-name} pg_num [pgp_num]

ceph osd pool delete {pool-name} [{pool-name} --yes-i-
really-really-mean-it]

ceph osd pool rename {old-name} {new-name}
```

更改存储池设置。

```
ceph osd pool set {pool-name} {field} {value}
```

可用的 field 值有：

- `size`: 设置存储池内数据的副本数；
- `crash_replay_interval`: 允许客户端重放确认而未提交的请求前等待的时间，秒；
- `pg_num`: 归置组数量；
- `pgp_num`: 计算归置组存放的有效数量；
- `crush_ruleset`: 用于归置映射的规则号。

获取存储池配置值。

```
ceph osd pool get {pool-name} {field}
```

可用的 field 值有：

- `pg_num`: 归置组数量；
- `pgp_num`: 计算归置组存放的有效数量；
- `lpg_num`: 本地归置组数量；
- `lpgp_num`: 用于存放本地归置组的数量。

向 OSD {osd-num} 下达一个洗刷命令，用通配符 * 把命令下达到所有 OSD。

```
ceph osd scrub {osd-num}
```

向 osdN 下达修复命令，用 * 下达到所有 OSD。

```
ceph osd repair N
```

在 osdN 上进行个简单的吞吐量测试，每次写入 BYTES_PER_WRITE、一共写入 TOTAL_BYTES。默认以 4MB 增量写入 1GB。此压力测试是非破坏性的，不会覆盖已有 OSD 数据，但可能会暂时影响同时访问此 OSD 的客户端性能。

```
ceph tell osd.N bench [NUMBER_OF_OBJECTS] [BYTES_PER_WRITE]
```

MDS 子系统

更改在运行 mds 的参数：

```
ceph tell mds.{mds-id} injectargs --{switch} {value} [--  
{switch} {value}]
```

例如：

```
ceph tell mds.0 injectargs --debug_ms 1 --debug_mds 10
```

打开了调试消息。

```
ceph mds stat
```

显示所有元数据服务器状态。

```
ceph mds fail 0
```

把活跃 MDS 标记为失败，如果有候补此命令会触发故障转移。

Todo

ceph mds 子命令缺少文档：set, dump, getmap, stop, setmap

监视器子系统

查看监视器状态：

```
ceph mon stat

2011-12-14 10:40:59.044395 mon {- [mon,stat]

2011-12-14 10:40:59.057111 mon.1 -} 'e3: 5 mons at
{a=10.1.2.3:6789/0,b=10.1.2.4:6789/0,c=10.1.2.5:6789/0,d=
10.1.2.6:6789/0,e=10.1.2.7:6789/0}, election epoch 16,
quorum 0,1,2,3' (0)
```

末尾的 `quorum` 列表列出了当前法定人数里的监视器节点。

也可以更直接地获取：

```
$ ./ceph quorum_status

2011-12-14 10:44:20.417705 mon {- [quorum_status]

2011-12-14 10:44:20.431890 mon.0 -}
```

```
'{ "election_epoch": 10,

  "quorum": [

    0,

    1,

    2],

  "monmap": { "epoch": 1,

    "fsid": "444b489c-4f16-4b75-83f0-cb8097468898",

    "modified": "2011-12-12 13:28:27.505520",

    "created": "2011-12-12 13:28:27.505520",

    "mons": [

      { "rank": 0,

        "name": "a",

        "addr": "127.0.0.1:6789\0"},

      { "rank": 1,

        "name": "b",

        "addr": "127.0.0.1:6790\0"},

      { "rank": 2,

        "name": "c",

        "addr": "127.0.0.1:6791\0"}]}}' (0)
```

如果法定人数未形成，上述命令会一直等待。

你刚刚连接的监视器的状态（用 `-m HOST:PORT` 另外指定）：

```
ceph mon_status

2011-12-14 10:45:30.644414 mon {- [mon_status]

2011-12-14 10:45:30.644632 mon.0 -}

'{ "name": "a",

  "rank": 0,

  "state": "leader",

  "election_epoch": 10,

  "quorum": [

    0,

    1,

    2],

  "outside_quorum": [],

  "monmap": { "epoch": 1,

    "fsid": "444b489c-4f16-4b75-83f0-cb8097468898",

    "modified": "2011-12-12 13:28:27.505520",

    "created": "2011-12-12 13:28:27.505520",
```

```
"mons": [  
    { "rank": 0,  
      "name": "a",  
      "addr": "127.0.0.1:6789\0"},  
    { "rank": 1,  
      "name": "b",  
      "addr": "127.0.0.1:6790\0"},  
    { "rank": 2,  
      "name": "c",  
      "addr": "127.0.0.1:6791\0"}]}' (0)
```

监视器状态转储：

```
ceph mon dump
```

```
2011-12-14 10:43:08.015333 mon {- [mon,dump]
```

```
2011-12-14 10:43:08.015567 mon.0 -} 'dumped monmap epoch  
1' (0)
```

```
epoch 1
```

```
fsid 444b489c-4f16-4b75-83f0-cb8097468898
```

```
last_changed 2011-12-12 13:28:27.505520
```

```
created 2011-12-12 13:28:27.505520
```



```
0: 127.0.0.1:6789/0 mon.a  
1: 127.0.0.1:6790/0 mon.b  
2: 127.0.0.1:6791/0 mon.c
```

监视器故障排除

When a cluster encounters monitor-related troubles there's a tendency to panic, and some times with good reason. You should keep in mind that losing a monitor, or a bunch of them, don't necessarily mean that your cluster is down, as long as a majority is up, running and with a formed quorum. Regardless of how bad the situation is, the first thing you should do is to calm down, take a breath and try answering our initial troubleshooting script.

开始排障

监视器在运行吗？

First of all, we need to make sure the monitors are running. You would be amazed by how often people forget to run the monitors, or restart them after an upgrade. There's no shame in that, but let's try not losing a couple of hours chasing an issue that is not there.

Are you able to connect to the monitor's servers?

Doesn't happen often, but sometimes people do have `iptables` rules that block accesses to monitor servers or monitor ports. Usually leftovers from monitor stress-testing that were forgotten at some point. Try ssh'ing into the server and, if that succeeds, try connecting to the monitor's port using you tool of choice (telnet, nc,...).

ceph -s 是否能运行并收到集群回复？

如果答案是肯定的，那么你的集群已启动并运行着。你可以想当然地认为如果已经形成法定人数，监视器们就只会响应 `status` 请求。

If `ceph -s` blocked however, without obtaining a reply from the cluster or showing a lot of `fault` messages, then it is likely that your monitors are either down completely or just a

portion is up – a portion that is not enough to form a quorum (keep in mind that a quorum is formed by a majority of monitors).

ceph -s 没完成是什么情况？

If you haven't gone through all the steps so far, please go back and do.

For those running on Emperor 0.72-rc1 and forward, you will be able to contact each monitor individually asking them for their status, regardless of a quorum being formed. This can be achieved using `ceph ping mon.ID`, ID being the monitor's identifier. You should perform this for each monitor in the cluster. In section [理解 mon_status](#) we will explain how to interpret the output of this command.

For the rest of you who don't tread on the bleeding edge, you will need to ssh into the server and use the monitor's admin socket. Please jump to [使用监视器的管理套接字](#).

For other specific issues, keep on reading.

使用监视器的管理套接字

通过管理套接字，你可以用 Unix 套接字文件直接与指定守护进程交互。这个文件位于你监视器的 `run` 目录下，默认配置时它位于 `/var/run/ceph/ceph-mon.ID.asok`，但你要是改过就不一定在那里了。如果你在那里没找到它，请看看 `ceph.conf` 里是否配置了其它路径、或者用下面的命令获取：

```
ceph-conf --name mon.ID --show-config-value admin_socket
```

请牢记，只有在监视器运行时管理套接字才可用。监视器正常关闭时，管理套接字会被删除；如果监视器不运行了、但管理套接字还存在，就说明监视器不是正常关闭的。不管怎样，监视器没在运行，你就不能使用管理套接字，`ceph` 命令会返回类似 `Error 111: Connection Refused` 的错误消息。

访问管理套接字很简单，就是让 `ceph` 工具使用 `asok` 文件。对于 Dumpling 之前的版本，命令是这样的：

```
ceph --admin-daemon /var/run/ceph/ceph-mon.<id>.asok  
<command>
```

对于 Dumping 及后续版本，你可以用另一个（推荐的）命令：

```
ceph daemon mon.<id> <command>
```

ceph 工具的 help 命令会显示管理套接字支持的其它命令。请仔细了解一下 config get、config show、mon_status 和 quorum_status 命令，在排除监视器故障时它们能给你些启发。

理解 MON_STATUS

mon_status can be obtained through the ceph tool when you have a formed quorum, or via the admin socket if you don't. This command will output a multitude of information about the monitor, including the same output you would get with quorum_status.

Take the following example of mon_status:

```
{ "name": "c",  
  
  "rank": 2,  
  
  "state": "peon",  
  
  "election_epoch": 38,  
  
  "quorum": [  
  
    1,  
  
    2],  
  
  "outside_quorum": [],  
  
  "extra_probe_peers": [],
```

```
"sync_provider": [],  
  
"monmap": { "epoch": 3,  
  
    "fsid": "5c4e9d53-e2e1-478a-8061-f543f8be4cf8",  
  
    "modified": "2013-10-30 04:12:01.945629",  
  
    "created": "2013-10-29 14:14:41.914786",  
  
    "mons": [  
  
        { "rank": 0,  
  
          "name": "a",  
  
          "addr": "127.0.0.1:6789\0"},  
  
        { "rank": 1,  
  
          "name": "b",  
  
          "addr": "127.0.0.1:6790\0"},  
  
        { "rank": 2,  
  
          "name": "c",  
  
          "addr": "127.0.0.1:6795\0"}]}}
```

A couple of things are obvious: we have three monitors in the monmap (*a*, *b* and *c*), the quorum is formed by only two monitors, and *c* is in the quorum as a *peon*.

Which monitor is out of the quorum?

The answer would be **a**.

Why?

Take a look at the `quorum` set. We have two monitors in this set: 1 and 2. These are not monitor names. These are monitor ranks, as established in the current monmap. We are missing the monitor with rank 0, and according to the monmap that would be `mon.a`.

By the way, how are ranks established?

Ranks are (re)calculated whenever you add or remove monitors and follow a simple rule: the **greater** the `IP:PORT` combination, the **lower** the rank is. In this case, considering that `127.0.0.1:6789` is lower than all the remaining `IP:PORT` combinations, `mon.a` has rank 0.

最常见的监视器问题

HAVE QUORUM BUT AT LEAST ONE MONITOR IS DOWN

When this happens, depending on the version of Ceph you are running, you should be seeing something similar to:

```
$ ceph health detail

[snip]

mon.a (rank 0) addr 127.0.0.1:6789/0 is down (out of
quorum)
```

How to troubleshoot this?

First, make sure `mon.a` is running.

Second, make sure you are able to connect to `mon.a`'s server from the other monitors' servers. Check the ports as well. Check `iptables` on all your monitor nodes and make sure you're not dropping/rejecting connections.

If this initial troubleshooting doesn't solve your problems, then it's time to go deeper.

First, check the problematic monitor's `mon_status` via the admin socket as explained in [使用监视器的管理套接字](#) and [理解 mon_status](#).

Considering the monitor is out of the quorum, its state should be one of `probing`, `electing` or `synchronizing`. If it happens to be either `leader` or `peon`, then the monitor believes to be in quorum, while the remaining cluster is sure it is not; or maybe it got into the quorum while we were troubleshooting the monitor, so check you `ceph -s` again just to make sure. Proceed if the monitor is not yet in the quorum.

What if the state is `probing`?

This means the monitor is still looking for the other monitors. Every time you start a monitor, the monitor will stay in this state for some time while trying to find the rest of the monitors specified in the `monmap`. The time a monitor will spend in this state can vary. For instance, when on a single-monitor cluster, the monitor will pass through the probing state almost instantaneously, since there are no other monitors around. On a multi-monitor cluster, the monitors will stay in this state until they find enough monitors to form a quorum – this means that if you have 2 out of 3 monitors down, the one remaining monitor will stay in this state indefinitely until you bring one of the other monitors up.

If you have a quorum, however, the monitor should be able to find the remaining monitors pretty fast, as long as they can be reached. If your monitor is stuck probing and you've gone through with all the communication troubleshooting, then there is a fair chance that the monitor is trying to reach the other monitors on a wrong address. `mon_status` outputs the `monmap` known to the monitor: check if the other monitor's locations match reality. If they don't, jump to [Recovering a Monitor's Broken monmap](#); if they do, then it may be related to severe clock skews amongst the monitor nodes and you should refer to [时钟偏移](#) first, but if that doesn't solve your problem then it is the time to prepare some logs and reach out to the community (please refer to [收集所需日志](#) on how to best prepare your logs).

What if state is `electing`?

This means the monitor is in the middle of an election. These should be fast to complete, but at times the monitors can get stuck electing. This is usually a sign of a clock skew among the monitor nodes; jump to [时钟偏移](#) for more infos on that. If all your clocks are properly synchronized, it is best if you prepare some logs and reach out to the community. This is not a state that is likely to persist and aside from (*really*) old bugs there isn't an obvious reason besides clock skews on why this would happen.

What if state is `synchronizing`?

This means the monitor is synchronizing with the rest of the cluster in order to join the quorum. The synchronization process is as fast as smaller your monitor store is, so if you have a big store it may take a while. Don't worry, it should be finished soon enough.

However, if you notice that the monitor jumps from `synchronizing` to `electing` and then back to `synchronizing`, then you do have a problem: the cluster state is advancing (i.e., generating new maps) way too fast for the synchronization process to keep up. This used to be a thing in early Cuttlefish, but since then the synchronization process was quite refactored and enhanced to avoid just this sort of behavior. If this happens in later versions let us know. And bring some logs (see [收集所需日志](#)).

What if state is `leader` or `peon`?

This should not happen. There is a chance this might happen however, and it has a lot to do with clock skews – see [时钟偏移](#). If you're not suffering from clock skews, then please prepare your logs (see [收集所需日志](#)) and reach out to us.

RECOVERING A MONITOR'S BROKEN MONMAP

This is how a `monmap` usually looks like, depending on the number of monitors:

```
epoch 3

fsid 5c4e9d53-e2e1-478a-8061-f543f8be4cf8

last_changed 2013-10-30 04:12:01.945629

created 2013-10-29 14:14:41.914786

0: 127.0.0.1:6789/0 mon.a

1: 127.0.0.1:6790/0 mon.b

2: 127.0.0.1:6795/0 mon.c
```

This may not be what you have however. For instance, in some versions of early Cuttlefish there was this one bug that could cause your `monmap` to be nullified. Completely filled with

zeros. This means that not even `monmaptool` would be able to read it because it would find it hard to make sense of only-zeros. Some other times, you may end up with a monitor with a severely outdated monmap, thus being unable to find the remaining monitors (e.g., say `mon.c` is down; you add a new monitor `mon.d`, then remove `mon.a`, then add a new monitor `mon.e` and remove `mon.b`; you will end up with a totally different monmap from the one `mon.c` knows).

In this sort of situations, you have two possible solutions:

Scrap the monitor and create a new one

You should only take this route if you are positive that you won't lose the information kept by that monitor; that you have other monitors and that they are running just fine so that your new monitor is able to synchronize from the remaining monitors. Keep in mind that destroying a monitor, if there are no other copies of its contents, may lead to loss of data.

Inject a monmap into the monitor

Usually the safest path. You should grab the monmap from the remaining monitors and inject it into the monitor with the corrupted/lost monmap.

These are the basic steps:

1. Is there a formed quorum? If so, grab the monmap from the quorum:

```
$ ceph mon getmap -o /tmp/monmap
```

2. No quorum? Grab the monmap directly from another monitor (this assumes the monitor you're grabbing the monmap from has id ID-FOO and has been stopped):

```
$ ceph-mon -i ID-FOO --extract-monmap /tmp/monmap
```

3. Stop the monitor you're going to inject the monmap into.
4. Inject the monmap:

```
$ ceph-mon -i ID --inject-monmap /tmp/monmap
```


5. Start the monitor

Please keep in mind that the ability to inject monmaps is a powerful feature that can cause havoc with your monitors if misused as it will overwrite the latest, existing monmap kept by the monitor.

时钟偏移

Monitors can be severely affected by significant clock skews across the monitor nodes. This usually translates into weird behavior with no obvious cause. To avoid such issues, you should run a clock synchronization tool on your monitor nodes.

What's the maximum tolerated clock skew?

By default the monitors will allow clocks to drift up to `0.05 seconds`.

Can I increase the maximum tolerated clock skew?

This value is configurable via the `mon-clock-drift-allowed` option, and although you *CAN* it doesn't mean you *SHOULD*. The clock skew mechanism is in place because clock skewed monitor may not properly behave. We, as developers and QA aficionados, are comfortable with the current default value, as it will alert the user before the monitors get out hand. Changing this value without testing it first may cause unforeseen effects on the stability of the monitors and overall cluster healthiness, although there is no risk of dataloss.

How do I know there's a clock skew?

The monitors will warn you in the form of

a `HEALTH_WARN`. `ceph health detail` should show something in the form of:

```
mon.c addr 10.10.0.1:6789/0 clock skew 0.08235s > max
0.05s (latency 0.0045s)
```

That means that `mon.c` has been flagged as suffering from a clock skew.

What should I do if there's a clock skew?

Synchronize your clocks. Running an NTP client may help. If you are already using one and you hit this sort of issues, check if you are using some NTP server remote to your network

and consider hosting your own NTP server on your network. This last option tends to reduce the amount of issues with monitor clock skews.

客户端不能连接或挂载

检查防火墙配置。有些系统安装工具把 REJECT 规则加入了 iptables，它会拒绝除 ssh 以外的所有入栈连接。如果你的监视器主机有这样的 REJECT 规则，别的客户端进来的连接将遇到超时错误而不能挂载。得先找到这条拒绝客户端连入的 iptables 规则，例如，你要找到形似以下的规则：

```
REJECT all -- anywhere anywhere reject-with icmp-host-prohibited
```

你也许还要在 Ceph 主机上增加 iptables 规则来放通 Ceph 监视器端口（即默认的 6789 端口）、和 OSD 端口（默认从 6800 到 7300）。例如：

```
iptables -A INPUT -m multiport -p tcp -s {ip-address}/{netmask} --dports 6789,6800:7300 -j ACCEPT
```

所有尝试都失败了，怎么办？

到外面寻求帮助

You can find us on IRC at #ceph and #ceph-devel at OFTC (server irc.oftc.net) and on ceph-devel@vger.kernel.org and ceph-users@lists.ceph.com. Make sure you have grabbed your logs and have them ready if someone asks: the faster the interaction and lower the latency in response, the better chances everyone's time is optimized.

收集所需日志

Monitor logs are, by default, kept in `/var/log/ceph/ceph-mon.FOO.log*`. We may want them. However, your logs may not have the necessary information. If you don't find your monitor logs at their default location, you can check where they should be by running:

```
ceph-conf --name mon.FOO --show-config-value log_file
```

The amount of information in the logs are subject to the debug levels being enforced by your configuration files. If you have not enforced a specific debug level then Ceph is using the default levels and your logs may not contain important information to track down your issue. A first step in getting relevant information into your logs will be to raise debug levels. In this case we will be interested in the information from the monitor. Similarly to what happens on other components, different parts of the monitor will output their debug information on different subsystems.

You will have to raise the debug levels of those subsystems more closely related to your issue. This may not be an easy task for someone unfamiliar with troubleshooting Ceph. For most situations, setting the following options on your monitors will be enough to pinpoint a potential source of the issue:

```
debug mon = 10  
  
debug ms = 1
```

If we find that these debug levels are not enough, there's a chance we may ask you to raise them or even define other debug subsystems to obtain info from – but at least we started off with some useful information, instead of a massively empty log without much to go on with.

我需要重启监视器来更改调试级别吗？

No. You may do it in one of two ways:

You have quorum

Either inject the debug option into the monitor you want to debug:

```
ceph tell mon.FOO injectargs --debug_mon 10/10
```

or into all monitors at once:

```
ceph tell mon.* injectargs --debug_mon 10/10
```

No quorum

Use the monitor's admin socket and directly adjust the configuration options:

```
ceph daemon mon.FOO config set debug_mon 10/10
```

Going back to default values is as easy as rerunning the above commands using the debug level `1/10` instead. You can check your current values using the admin socket and the following commands:

```
ceph daemon mon.FOO config show
```

or:

```
ceph daemon mon.FOO config get 'OPTION_NAME'
```

在某个调试级别下重现了问题，然后呢？

Ideally you would send us only the relevant portions of your logs. We realise that figuring out the corresponding portion may not be the easiest of tasks. Therefore, we won't hold it to you if you provide the full log, but common sense should be employed. If your log has hundreds of thousands of lines, it may get tricky to go through the whole thing, specially if we are not aware at which point, whatever your issue is, happened. For instance, when reproducing, keep in mind to write down current time and date and to extract the relevant portions of your logs based on that.

Finally, you should reach out to us on the mailing lists, on IRC or file a new issue on the [tracker](#).

OSD 故障排除

进行 OSD 故障前，先检查一下监视器集群和网络。如果 `ceph health` 或 `ceph -s` 返回的是健康状态，这意味着监视器们形成了法定人数。如果你还没监视器法定人数、或者监视器状态错误，要先[解决监视器问题](#)。核实下你的网络，确保它在正常运行，因为网络对 OSD 的运行和性能有显著影响。

收集 OSD 数据

开始 OSD 故障的第一步最好先收集信息，另外还有[监控 OSD](#) 时收集的，如 `ceph \ osd tree`。

CEPH 日志

如果你没改默认路径，可以在 `/var/log/ceph` 下找到日志：

```
ls /var/log/ceph
```

如果看到的日志还不够详细，可以增大日志级别。请参考[日志记录和调试](#)，看看如何保证看到大量日志时又不影响集群运行。

管理套接字

用管理套接字检索运行时信息。所有 Ceph 套接字列表：

```
ls /var/run/ceph
```

然后，执行下列命令显示可用选项，把 `{daemon-name}` 换成实际的守护进程（如 `osd.0`）：

```
ceph daemon osd.0 help
```

另外，你也可以指定一个 `{socket-file}`（如 `/var/run/ceph` 下的文件）：

```
ceph daemon {socket-file} help
```

和其它手段相比，管理接口允许你：

- 在运行时列出配置
- 列出历史操作
- 列出操作的优先队列状态
- 列出在进行的操作
- 列出性能计数器

显示剩余空间

可能是文件系统问题，用 `df` 显示文件系统剩余空间。

```
df -h
```

其它用法见 `df --help`。

I/O 统计信息

用 `iostat` 定位 I/O 相关问题。

```
iostat -x
```

诊断消息

要查看诊断信息，配合 `less`、`more`、`grep` 或 `tail` 使用 `dmesg`，例如：

```
dmesg | grep scsi
```

停止自动重均衡

你得周期性地维护集群的子系统、或解决某个失败域的问题（如一机架）。如果你不想在停机维护 OSD 时让 CRUSH 自动重均衡，提前设置 `noout`：

```
ceph osd set noout
```

在集群上设置 `noout` 后，你就可以停机维护失败域内的 OSD 了。

```
stop ceph-osd id={num}
```

Note

在定位同一故障域内的问题时，停机 OSD 内的归置组状态会变为 `degraded`。

维护结束后，重启 OSD。

```
start ceph-osd id={num}
```

最后，解除 `noout` 标志。

```
ceph osd unset noout
```

OSD 没运行

通常情况下，简单地重启 `ceph-osd` 进程就可以重回集群并恢复。

OSD 起不来

如果你重启了集群，但其中一个 OSD 起不来，依次检查：

- **配置文件**：如果你新装的 OSD 不能启动，检查下配置文件，确保它合交性（比如 `host` 而非 `hostname`，等等）；
- **检查路径**：检查你配置的路径，以及它们自己的数据和日志路径。如果你分离了 OSD 数据和日志、而配置文件和实际挂载点存在出入，启动 OSD 时就可能遇挫。如果你想把日志存储于一个块设备，应该为日志硬盘分区并为各 OSD 分别分配一分区。
- **检查最大线程数**：如果你的节点有很多 OSD，也许就会触碰到默认的最大线程数限制（如通常是 32k 个），尤其是在恢复期间。你可以用 `sysctl` 增大线程数，把最大线程数更改为支持的最大值（即 4194303），看看是否有用。例如：

```
sysctl -w kernel.pid_max=4194303
```

如果增大最大线程数解决了这个问题，你可以把此配置 `kernel.pid_max` 写入配置文件 `/etc/sysctl.conf`，使之永久生效，例如：

```
kernel.pid_max = 4194303
```

- **内核版本**：确认你在用的内核版本以及所用的发布版。Ceph 默认依赖一些第三方工具，这些工具可能有缺陷或者与特定发布版和/或内核版本冲突（如 Google `perf`tool）。检查下[操作系统推荐](#)确保你已经解决了内核相关的问题。
- **段错误**：如果有了段错误，提高日志级别（如果还没提高），再试试。如果重现了，联系 [ceph-devel](#) 并提供你的配置文件、显示器输出和日志文件内容。

如果问题仍未解决，电子邮件也无用，你可以联系 [Inktank](#) 寻求帮助。

OSD 失败

`ceph-osd` 挂掉时，监视器可通过活着的 `ceph-osd` 了解到此情况，且通过 `ceph health` 命令报告：

```
ceph health

HEALTH_WARN 1/3 in osds are down
```


而且，有 `ceph-osd` 进程标记为 `in` 且 `down` 的时候，你会得到警告，你可以用下面的命令得知哪个 `ceph-osd` 进程挂了：

```
ceph health detail

HEALTH_WARN 1/3 in osds are down

osd.0 is down since epoch 23, last address
192.168.106.220:6800/11080
```

如果有硬盘失败或其它错误使 `ceph-osd` 不能正常运行或重启，一条错误信息将会出现在日志文件 `/var/log/ceph/` 里。

如果守护进程因心跳失败、或者底层文件系统无响应而停止，查看 `dmesg` 获取硬盘或者内核错误。

如果是软件错误（失败的插入或其它意外错误），就应该回馈到 [ceph-devel](#) 邮件列表。

硬盘没剩余空间

Ceph 不允许你向满的 OSD 写入数据，以免丢失数据。在运营着的集群中，你应该能收到集群空间将满的警告。`mon osd full ratio` 默认为 0.95、或达到 95% 时它将阻止客户端写入数据。`mon osd nearfull ratio` 默认为 0.85、也就是说达到容量的 85% 时它会产生健康警告。

满载集群问题一般产生于测试 Ceph 在小型集群上如何处理 OSD 失败时。当某一节点利用率较高时，集群能够很快掩盖将满和占满率。如果你在测试小型集群上的 Ceph 如何应对 OSD 失败，应该保留足够的空间，然后试着临时降低 `mon osd full ratio` 和 `mon osd nearfull ratio` 值。

`ceph health` 会显示将满的 `ceph-osds`：

```
ceph health

HEALTH_WARN 1 nearfull osds
```

```
osd.2 is near full at 85%
```

或者：

```
ceph health

HEALTH_ERR 1 nearfull osds, 1 full osds

osd.2 is near full at 85%

osd.3 is full at 97%
```

处理这种情况的最好方法就是增加新的 `ceph-osd`，这允许集群把数据重分布到新 OSD 里。

如果因满载而导致 OSD 不能启动，你可以试着删除那个 OSD 上的一些归置组数据目录。

Important

如果你准备从填满的 OSD 中删除某个归置组，注意**不要**删除另一个 OSD 上的同名归置组，否则**你会丢数据**。**必须**在多个 OSD 上保留至少一份数据副本。

详情见[监视器配置参考](#)。

OSD 龟速或无响应

一个反复出现的问题是龟速或无响应。在深入性能问题前，你应该先确保不是其他故障。例如，确保你的网络运行正常、且 OSD 在运行，还要检查 OSD 是否被恢复流量拖住了。

Tip

较新版本的 Ceph 能更好地处理恢复，可防止恢复进程耗尽系统资源而导致 `up` 且 `in` 的 OSD 不可用或响应慢。

网络问题

Ceph 是一个分布式存储系统，所以它依赖于网络来互联 OSD 们、复制对象、恢复错误、和检查心跳。网络问题会导致 OSD 延时和打摆子，详情参见[打摆子的 OSD](#)。

确保 Ceph 进程和 Ceph 依赖的进程连接了、和/或在监听。

```
netstat -a | grep ceph

netstat -l | grep ceph

sudo netstat -p | grep ceph
```

检查网络统计信息。

```
netstat -s
```

驱动器配置

一个存储驱动器应该只用于一个 OSD。如果有其它进程共享驱动器，顺序读和顺序写吞吐量会成为瓶颈，包括日志记录、操作系统、监视器、其它 OSD 和非 Ceph 进程。

Ceph 在日志记录完成之后才会确认写操作，所以使用 ext4 或 XFS 文件系统时高速的 SSD 对降低响应延时很有吸引力。相反，btrfs 文件系统可以同时读写。

Note

给驱动器分区并不能改变总吞吐量或顺序读写限制。把日志分离到单独的分区可能有帮助，但最好是另外一块硬盘的分区。

坏扇区和碎片化硬盘

检修下硬盘是否有坏扇区和碎片。这会导致总吞吐量急剧下降。

监视器和 OSD 蜗居

监视器是普通的轻量级进程，但它们会频繁调用 `fsync()`，这会妨碍其它工作量，特别是监视器和 OSD 共享驱动器时。另外，如果你在 OSD 主机上同时运行监视器，遭遇的性能问题可能和这些相关：

- 运行较老的内核（低于 3.0）
- v0.48 版运行在老的 `glibc` 之上
- 运行的内核不支持 `syncfs(2)` 系统调用

在这些情况下，同一主机上的多个 OSD 会相互拖垮对方。它们经常导致爆炸式写入。

进程蜗居

共存于同一套硬件、并向 Ceph 写入数据的进程（像基于云的解决方案、虚拟机和其他应用程序）会导致 OSD 延时大增。一般来说，我们建议用一主机跑 Ceph、其它主机跑其它进程，实践证明把 Ceph 和其他应用程序分开可提高性能、并简化故障排除和维护。

日志记录级别

如果你为追踪某问题提高过日志级别、但结束后忘了调回去，这个 OSD 将向硬盘写入大量日志。如果你想始终保持高日志级别，可以考虑给默认日志路径挂载个硬盘，即 `/var/log/ceph/$cluster-$name.log`。

恢复节流

根据你的配置，Ceph 可以降低恢复速度来维持性能，否则它会不顾 OSD 性能而加快恢复速度。检查下 OSD 是否正在恢复。

内核版本

检查下你在用的内核版本。较老的内核也许没有移植能提高 Ceph 性能的功能。

内核与 SYNCFS 问题

试试在一主机上只运行一个 OSD，看看能否提升性能。老内核未必支持有 `syncfs(2)` 系统调用的 `glibc`。

文件系统问题

当前，我们推荐基于 xfs 或 ext4 部署集群。btrfs 有很多诱人的功能，但文件系统内的缺陷可能导致性能问题。

内存不足

我们建议为每 OSD 进程规划 1GB 内存。你也许注意到了，通常情况下 OSD 仅会用一小部分（如 100-200MB）。你也许想用这些空闲内存跑一些其他应用，如虚拟机等等，然而当 OSD 进入恢复状态时，其内存利用率激增，如果没有可用内存，此 OSD 的性能将差的多。

OLD REQUESTS 或 SLOW REQUESTS

如果某 `ceph-osd` 守护进程对一请求响应很慢，它会生成日志消息来抱怨请求耗费的时间过长。默认警告阈值是 30 秒，用 `osd op complaint time` 选项来配置。这种情况发生时，集群日志系统会收到这些消息。

很老的版本抱怨 “old requests”：

```
osd.0 192.168.106.220:6800/18813 312 : [WRN] old request
osd_op(client.5099.0:790 fatty_26485_object789 [write
0~4096] 2.5e54f643) v4 received at 2012-03-06
15:42:56.054801 currently waiting for sub ops
```

较新版本的 Ceph 抱怨 “slow requests”：

```
{date} {osd.num} [WRN] 1 slow requests, 1 included below;
oldest blocked for > 30.005692 secs

{date} {osd.num} [WRN] slow request 30.005692 seconds
old, received at {date-time}: osd_op(client.4240.0:8
```

```
benchmark_data_ceph-1_39426_object7 [write 0~4194304]
0.69848840) v4 currently waiting for subops from [610]
```

可能起因有：

- 坏驱动器（查看 `dmesg` 输出）；
- 内核文件系统缺陷（查看 `dmesg` 输出）；
- 集群过载（检查系统负载、`iostat` 等等）；
- `ceph-osd` 守护进程缺陷。

可能的解决方法：

- 从 Ceph 主机去除 VM 云解决方案；
- 升级内核；
- 升级 Ceph；
- 重启 OSD。

打摆子的 OSD

我们建议同时部署公网（前端）和集群网（后端），这样能更好地满足对象复制的容量需求。另一个优点是你可以运营一个不连接互联网的集群，以此避免拒绝估计。OSD 们互联和检查心跳时会优选集群网（后端），详情见[监视器与 OSD 的交互](#)。

然而，如果集群网（后端）失败、或出现了明显的延时，同时公网（前端）却运行良好，OSD 现在不能很好地处理这种情况。这时 OSD 们会向监视器报告邻居 `down` 了、同时报告自己是 `up` 的，我们把这种情形称为打摆子（flapping）。

如果有东西导致 OSD 打摆子（反复地被标记为 `down`，然后又 `up`），你可以强制监视器停止：

```
ceph osd set noup          # prevent OSDs from getting marked
up

ceph osd set nodown        # prevent OSDs from getting marked
down
```

这些标记记录在 osdmap 数据结构里：

```
ceph osd dump | grep flags  
  
flags no-up,no-down
```

下列命令可清除标记：

```
ceph osd unset noup  
  
ceph osd unset nodown
```

mon osd down out interval is). 还支持其它两个标记 `noin` 和 `noout`，它们分别可防止正在启动的 OSD 被标记为 `in`、或被误标记为 `out`（不管 ``mon osd down out interval`` 的值是什么）。

Note

`noup`、`noout` 和 `nodown` 从某种意义上说是临时的，一旦标记清除了，它们被阻塞的动作短时间内就会发生；相反，`noin` 标记阻止 OSD 启动后进入集群，但其它守护进程都维持原样。

归置组排障

归置组总不整洁

When you create a cluster and your cluster remains in `active`, `active+remapped` or `active+degraded` status and never achieve `anactive+clean` status, you likely have a problem with your configuration.

You may need to review settings in the [存储池、归置组和 CRUSH 配置参考](#) and make appropriate adjustments.

As a general rule, you should run your cluster with more than one OSD and a pool size greater than 1 object replica.

ONE NODE CLUSTER

Ceph no longer provides documentation for operating on a single node, because you would never deploy a system designed for distributed computing on a single node. Additionally, mounting client kernel modules on a single node containing a Ceph daemon may cause a deadlock due to issues with the Linux kernel itself (unless you use VMs for the clients). You can experiment with Ceph in a 1-node configuration, in spite of the limitations as described herein.

If you are trying to create a cluster on a single node, you must change the default of the `osd crush chooseleaf type` setting from `1` (meaning `host` or `node`) to `0` (meaning `osd`) in your Ceph configuration file before you create your monitors and OSDs. This tells Ceph that an OSD can peer with another OSD on the same host. If you are trying to set up a 1-node cluster and `osd crush chooseleaf type` is greater than `0`, Ceph will try to peer the PGs of one OSD with the PGs of another OSD on another node, chassis, rack, row, or even datacenter depending on the setting.

Tip

DO NOT mount kernel clients directly on the same node as your Ceph Storage Cluster, because kernel conflicts can arise. However, you can mount kernel clients within virtual machines (VMs) on a single node.

If you are creating OSDs using a single disk, you must create directories for the data manually first. For example:

```
mkdir /var/local/osd0 /var/local/osd1

ceph-deploy osd prepare {localhost-name}:/var/local/osd0
{localhost-name}:/var/local/osd1

ceph-deploy osd activate {localhost-name}:/var/local/osd0
{localhost-name}:/var/local/osd1
```


FEWER OSDS THAN REPLICAS

If you've brought up two OSDs to an `up` and `in` state, but you still don't see `active + clean` placement groups, you may have an `osd pooldefault size` set to greater than 2.

There are a few ways to address this situation. If you want to operate your cluster in an `active + degraded` state with two replicas, you can set the `osd pool default min size` to 2 so that you can write objects in an `active + degraded` state. You may also set the `osd pool defaultsize` setting to 2 so that you only have two stored replicas (the original and one replica), in which case the cluster should achieve an `active + clean` state.

Note

You can make the changes at runtime. If you make the changes in your Ceph configuration file, you may need to restart your cluster.

POOL SIZE = 1

If you have the `osd pool default size` set to 1, you will only have one copy of the object. OSDs rely on other OSDs to tell them which objects they should have. If a first OSD has a copy of an object and there is no second copy, then no second OSD can tell the first OSD that it should have that copy. For each placement group mapped to the first OSD (see `ceph pg dump`), you can force the first OSD to notice the placement groups it needs by running:

```
ceph pg force_create_pg <pgid>
```

CRUSH 图错误

Another candidate for placement groups remaining unclear involves errors in your CRUSH map.

卡住的归置组

有失败时归置组会进入“degraded”（降级）或“peering”（连接建立中）状态，这事时有发生，通常这些状态意味着正常的失败恢复正在进行。然而，如果一个归置组长时间处于某个这些状态就意味着有更大的问题，因此监视器在归置组卡（stuck）在非最优状态时会警告。我们具体检查：

- `inactive`（不活跃）——归置组长时间无活跃（即它不能提供读写服务了）；
- `unclean`（不干净）——归置组长时间不干净（例如它未能从前面的失败完全恢复）；
- `stale`（不新鲜）——归置组状态没有被 `ceph-osd` 更新，表明存储这个归置组的所有节点可能都挂了。

你可以摆出卡住的归置组：

```
ceph pg dump_stuck stale  
  
ceph pg dump_stuck inactive  
  
ceph pg dump_stuck unclean
```

卡在 `stale` 状态的归置组通过修复 `ceph-osd` 进程通常可以修复；卡在 `inactive` 状态的归置组通常是互联问题（参见 [归置组挂了——互联失败](#)）；卡在 `unclean` 状态的归置组通常是由于某些原因阻止了恢复的完成，像未找到的对象（参见 [未找到的对象](#)）。

归置组挂了——互联失败

在某些情况下，`ceph-osd` 连接建立进程会遇到问题，使 PG 不能活跃、可用，例如 `ceph health` 也许显示：

```
ceph health detail  
  
HEALTH_ERR 7 pgs degraded; 12 pgs down; 12 pgs peering; 1  
pgs recovering; 6 pgs stuck unclean; 114/3300 degraded  
(3.455%); 1/3 in osds are down
```

```
...

pg 0.5 is down+peering

pg 1.4 is down+peering

...

osd.1 is down since epoch 69, last address
192.168.106.220:6801/8651
```

可以查询到 PG 为何被标记为 `down` :

```
ceph pg 0.5 query

{ "state": "down+peering",

  ...

  "recovery_state": [

    { "name": "Started\\Primary\\Peering\\GetInfo",

      "enter_time": "2012-03-06 14:40:16.169679",

      "requested_info_from": []},

    { "name": "Started\\Primary\\Peering",

      "enter_time": "2012-03-06 14:40:16.169659",

      "probing_osds": [

        0,

        1],

      "blocked": "peering is blocked due to down osds",
```

```
    "down_osds_we_would_probe": [

        1],

    "peering_blocked_by": [

        { "osd": 1,

            "current_lost_at": 0,

            "comment": "starting or marking this osd
lost may let us proceed" }],

        { "name": "Started",

            "enter_time": "2012-03-06 14:40:16.169513" }

    ]

}
```

`recovery_state` 段告诉我们连接建立因 `ceph-osd` 进程挂了而被阻塞，本例是 `osd.1` 挂了，启动这个进程应该就可以恢复。

另外，如果 `osd.1` 是灾难性的失败（如硬盘损坏），我们可以告诉集群它丢失（`lost`）了，让集群尽力完成副本拷贝。

Important

集群不能保证其它数据副本是一致且最新就危险了！

让 Ceph 无论如何都继续：

```
ceph osd lost 1
```

恢复将继续。

未找到的对象

某几种失败相组合可能导致 Ceph 抱怨有找不到 (`unfound`) 的对象：

```
ceph health detail

HEALTH_WARN 1 pgs degraded; 78/3778 unfound (2.065%)

pg 2.4 is active+degraded, 78 unfound
```

这意味着存储集群知道一些对象（或者存在对象的较新副本）存在，却没有找到它们的副本。下例展示了这种情况是如何发生的，一个 PG 的数据存储在 ceph-osd 1 和 2 上：

- 1 挂了；
- 2 独自处理一些写动作；
- 1 起来了；
- 1 和 2 重新互联，1 上面丢失的对象加入队列准备恢复；
- 新对象还未拷贝完，2 挂了。

这时，1 知道这些对象存在，但是活着的 `ceph-osd` 都没有副本，这种情况下，读写这些对象的 IO 就会被阻塞，集群只能指望节点早点恢复。这时我们假设用户希望先得到一个 IO 错误。

首先，你应该确认哪些对象找不到了：

```
ceph pg 2.4 list_missing [starting offset, in json]

{ "offset": { "oid": "",

    "key": "",

    "snapid": 0,

    "hash": 0,

    "max": 0},
```

```
"num_missing": 0,

"num_unfound": 0,

"objects": [

  { "oid": "object 1",

    "key": "",

    "hash": 0,

    "max": 0 },

  ...

],

"more": 0}
```

如果在一次查询里列出的对象太多，`more` 这个字段将为 `true`，因此你可以查询更多。
(命令行工具可能隐藏了，但这里没有)

其次，你可以找出哪些 OSD 上探测到、或可能包含数据：

```
ceph pg 2.4 query

"recovery_state": [

  { "name": "Started/Primary/Active",

    "enter_time": "2012-03-06 15:15:46.713212",

    "might_have_unfound": [

      { "osd": 1,
```

```
"status": "osd is down"}]]},
```

本例中，集群知道 `osd.1` 可能有数据，但它挂了（`down`）。所有可能的状态有：

- 已经探测到了
- 在查询
- OSD 挂了
- 尚未查询

有时候集群要花一些时间来查询可能的位置。

还有一种可能性，对象存在于其它位置却未被列出，例如，集群里的一个 `ceph-osd` 停止且被剔除，然后完全恢复了；后来的失败、恢复后仍有未找到的对象，它也不会觉得早已死亡的 `ceph-osd` 上仍可能包含这些对象。（这种情况几乎不太可能发生）。

如果所有位置都查询过了仍有对象丢失，那就得放弃丢失的对象了。这仍可能是罕见的失败组合导致的，集群在写入完成前，未能得知写入是否已执行。以下命令把未找到的（`unfound`）对象标记为丢失（`lost`）。

```
ceph pg 2.5 mark_unfound_lost revert|delete
```

上述最后一个参数告诉集群应如何处理丢失的对象。

`delete` 选项将导致完全删除它们。

`revert` 选项（纠删码存储池不可用）会回滚到前一个版本或者（如果它是新对象的话）删除它。要慎用，它可能迷惑那些期望对象存在的应用程序。

无根归置组

拥有归置组拷贝的 OSD 都可以失败，在这种情况下，那一部分的对象存储不可用，监视器就不会收到那些归置组的状态更新了。为检测这种情况，监视器把任何主 OSD 失败的归置组标记为 `stale`（不新鲜），例如：

```
ceph health
```

```
HEALTH_WARN 24 pgs stale; 3/300 in osds are down
```

你能找出哪些归置组 `stale`、和存储这些归置组的最新 OSD，命令如下：

```
ceph health detail

HEALTH_WARN 24 pgs stale; 3/300 in osds are down

...

pg 2.5 is stuck stale+active+remapped, last acting [2,0]

...

osd.10 is down since epoch 23, last address
192.168.106.220:6800/11080

osd.11 is down since epoch 13, last address
192.168.106.220:6803/11539

osd.12 is down since epoch 24, last address
192.168.106.220:6806/11861
```

如果想使归置组 2.5 重新在线，例如，上面的输出告诉我们它最后由 `osd.0` 和 `osd.2` 处理，重启这些 `ceph-osd` 将恢复之（还有其它的很多 PG）。

只有几个 OSD 接收数据

如果你的集群有很多节点，但只有其中几个接收数据，[检查](#)下存储池里的归置组数量。因为归置组是映射到多个 OSD 的，这样少量的归置组将不能分布于整个集群。试着创建个新存储池，其归置组数量是 OSD 数量的若干倍。详情见[归置组](#)，存储池的默认归置组数量没多大用，你可以参考[这里](#)更改它。

不能写入数据

如果你的集群已启动，但一些 OSD 没起来，导致不能写入数据，确认下运行的 OSD 数量满足归置组要求的最低 OSD 数。如果不能满足，Ceph 就不会允许你写入数据，因为 Ceph 不能保证复制能如愿进行。详情参见[存储池、归置组和 CRUSH 配置参考](#)里的 `osd pool default min size`。

归置组不一致

If you receive an `active + clean + inconsistent` state, this may happen due to an error during scrubbing. If the inconsistency is due to disk errors, check your disks.

You can repair the inconsistent placement group by executing:

```
ceph pg repair {placement-group-ID}
```

If you receive `active + clean + inconsistent` states periodically due to clock skew, you may consider configuring your [NTP](#) daemons on your monitor hosts to act as peers. See [网络时间协议](#) and Ceph [时钟选项](#) for additional details.

纠删编码的归置组不是 ACTIVE+CLEAN

CRUSH 找不到足够多的 OSD 映射到某个 PG 时，它会显示为 `2147483647`，意思是 `ITEM_NONE` 或 `no OSD found`，例如：

```
[2, 1, 6, 0, 5, 8, 2147483647, 7, 4]
```

OSD 不够多

如果 Ceph 集群仅有 8 个 OSD，但是纠删码存储池需要 9 个，就会显示上面的错误。这时候，你仍然可以另外创建需要较少 OSD 的纠删码存储池：

```
ceph osd erasure-code-profile set myprofile k=5 m=3
```

```
ceph osd pool create erasurepool 16 16 erasure myprofile
```

或者新增一个 OSD，这个 PG 会自动用上的。

CRUSH 条件不能满足

即使集群拥有足够多的 OSD，CRUSH 规则集的强制要求仍有可能无法满足。假如有 10 个 OSD 分布于两个主机上，且 CRUSH 规则集要求相同归置组不得使用位于同一主机的两个 OSD，这样映射就会失败，因为只能找到两个 OSD，你可以从规则集里查看必要条件：

```
$ ceph osd crush rule ls

[
    "replicated_ruleset",
    "erasurepool"]

$ ceph osd crush rule dump erasurepool

{ "rule_id": 1,
  "rule_name": "erasurepool",
  "ruleset": 1,
  "type": 3,
  "min_size": 3,
  "max_size": 20,
  "steps": [
    { "op": "take",
      "item": -1,
```

```
        "item_name": "default"},

    { "op": "chooseleaf_indep",

        "num": 0,

        "type": "host"},

    { "op": "emit" } ] }
```

可以这样解决此问题，创建新存储池，其内的 PG 允许多个 OSD 位于同一主机，命令如下：

```
ceph osd erasure-code-profile set myprofile ruleset-
failure-domain=osd

ceph osd pool create erasurepool 16 16 erasure myprofile
```

CRUSH 过早中止

假设集群拥有的 OSD 足以映射到 PG（比如有 9 个 OSD 和一个纠删码存储池的集群，每个 PG 需要 9 个 OSD），CRUSH 仍然有可能在找到映射前就中止了。可以这样解决：

- 降低纠删存储池内 PG 的要求，让它使用较少的 OSD（需创建另一个存储池，因为纠删码配置不支持动态修改）。
- 向集群添加更多 OSD（无需修改纠删存储池，它会自动回到清洁状态）。
- 通过手工打造的 CRUSH 规则集，让它多试几次以找到合适的映射。
把 `set_choose_tries` 设置得高于默认值即可。

你从集群中提取出 `crushmap` 之后，应该先用 `crushtool` 校验一下是否有问题，这样你的试验就无需触及 Ceph 集群，只要在一个本地文件上测试即可：

```
$ ceph osd crush rule dump erasurepool

{ "rule_name": "erasurepool",

    "ruleset": 1,
```

```
"type": 3,  
  
"min_size": 3,  
  
"max_size": 20,  
  
"steps": [  
  
    { "op": "take",  
  
      "item": -1,  
  
      "item_name": "default"},  
  
    { "op": "chooseleaf_indep",  
  
      "num": 0,  
  
      "type": "host"},  
  
    { "op": "emit"}}]
```

```
$ ceph osd getcrushmap > crush.map
```

```
got crush map from osdmap epoch 13
```

```
$ crushtool -i crush.map --test --show-bad-mappings \
```

```
--rule 1 \
```

```
--num-rep 9 \
```

```
--min-x 1 --max-x $((1024 * 1024))
```

```
bad mapping rule 8 x 43 num_rep 9 result  
[3,2,7,1,2147483647,8,5,6,0]
```

```
bad mapping rule 8 x 79 num_rep 9 result  
[6,0,2,1,4,7,2147483647,5,8]
```

```
bad mapping rule 8 x 173 num_rep 9 result  
[0,4,6,8,2,1,3,7,2147483647]
```

其中 `--num-rep` 是纠删码 crush 规则集所需的 OSD 数量，`--rule` 是 `ceph osd crush rule dump` 命令结果中 `ruleset` 字段的值。此测试会尝试映射一百万个值（即 `[--min-x, --max-x]` 所指定的范围），且必须至少显示一个坏映射；如果它没有任何输出，说明所有映射都成功了，你可以就此打住：问题的根源不在这里。

反编译 crush 图后，你可以手动编辑其规则集：

```
$ crushtool --decompile crush.map > crush.txt
```

并把下面这行加进规则集：

```
step set_choose_tries 100
```

然后 `crush.txt` 文件内的这部分大致如此：

```
rule erasurepool {  
  
    ruleset 1  
  
    type erasure  
  
    min_size 3  
  
    max_size 20  
  
    step set_chooseleaf_tries 5  
  
    step set_choose_tries 100  
  
    step take default  
  
    step chooseleaf indep 0 type host
```

```
        step emit

    }
```

然后编译、并再次测试：

```
$ crushtool --compile crush.txt -o better-crush.map
```

所有映射都成功时，用 `crushtool` 的 `--show-choose-tries` 选项能看到成功映射的尝试次数直方图：

```
$ crushtool -i better-crush.map --test --show-bad-
mappings \

    --show-choose-tries \

    --rule 1 \

    --num-rep 9 \

    --min-x 1 --max-x $((1024 * 1024))

...

11:          42

12:          44

13:          54

14:          45

15:          35

16:          34
```

17:	30
18:	25
19:	19
20:	22
21:	20
22:	17
23:	13
24:	16
25:	13
26:	11
27:	11
28:	13
29:	11
30:	10
31:	6
32:	5
33:	10
34:	3
35:	7
36:	5

37:	2
38:	5
39:	5
40:	2
41:	5
42:	4
43:	1
44:	2
45:	2
46:	3
47:	1
48:	0
...	
102:	0
103:	1
104:	0
...	

有 42 个归置组需 11 次重试、44 个归置组需 12 次重试，以此类推。这样，重试的最高次数就是防止坏映射的最低值，也就是 `set_choose_tries` 的取值（即上面输出中的 103，因为任意归置组成功映射的重试次数都没有超过 103）。

内存剖析

Ceph 监视器、OSD、和元数据服务器可利用 `tcmalloc` 生成堆栈剖析，此功能依赖 `google-perftools`：

```
sudo apt-get install google-perftools
```

剖析器会把输出保存到 `log file` 目录（如 `/var/log/ceph`），详情见[日志记录和调试](#)。剖析器日志可用 Google 性能工具来查看，执行如下命令：

```
google-perfprof --text {path-to-daemon} {log-path/filename}
```

例如：

```
$ ceph tell osd.0 heap start_profiler

$ ceph tell osd.0 heap dump

osd.0 tcmalloc heap stats:-----
-----

MALLOC:          2632288 (    2.5 MiB) Bytes in use by
application

MALLOC: +         499712 (    0.5 MiB) Bytes in page heap
freelist

MALLOC: +         543800 (    0.5 MiB) Bytes in central
cache freelist

MALLOC: +         327680 (    0.3 MiB) Bytes in transfer
cache freelist

MALLOC: +        1239400 (    1.2 MiB) Bytes in thread
cache freelists
```

```

MALLOC: +      1142936 (    1.1 MiB) Bytes in malloc
metadata

MALLOC:  -----

MALLOC: =      6385816 (    6.1 MiB) Actual memory used
(physical + swap)

MALLOC: +              0 (    0.0 MiB) Bytes released to OS
(aka unmapped)

MALLOC:  -----

MALLOC: =      6385816 (    6.1 MiB) Virtual address
space used

MALLOC:

MALLOC:          231                Spans in use

MALLOC:          56                Thread heaps in use

MALLOC:          8192              Tcmalloc page size

-----

Call ReleaseFreeMemory() to release freelist memory to
the OS (via madvise()).

Bytes released to the OS take up virtual address space
but no physical memory.

$ google-pprof --text \

                        /usr/bin/ceph-osd \

                        /var/log/ceph/ceph-osd.0.profile.0001.heap

Total: 3.7 MB

```

```

1.9  51.1%  51.1%      1.9  51.1%
ceph::log::Log::create_entry

1.8  47.3%  98.4%      1.8  47.3%
std::string::_Rep::_S_create

0.0   0.4%  98.9%      0.0   0.6%
SimpleMessenger::add_accept_pipe

0.0   0.4%  99.2%      0.0   0.6% decode_message

...

```

再次转储堆栈会生成另外一个文件，这样便于和前面的堆栈转储相比较，看看这段时间发生了什么。例如：

```

$ google-pprof --text --base out/osd.0.profile.0001.heap \
\

ceph-osd out/osd.0.profile.0003.heap

Total: 0.2 MB

0.1  50.3%  50.3%      0.1  50.3%
ceph::log::Log::create_entry

0.1  46.6%  96.8%      0.1  46.6%
std::string::_Rep::_S_create

0.0   0.9%  97.7%      0.0  26.1% ReplicatedPG::do_op

0.0   0.8%  98.5%      0.0   0.8%
__gnu_cxx::new_allocator::allocate

```

详情见 [Google 堆栈剖析器](#)。

安装堆栈剖析器后，启动集群就可以开始使用了。你可以在运行时启用、或禁用堆栈剖析器，或确保它在持续运行。在随后的几个命令行用法中，用 `mon`、`osd` 或 `mds` 替换掉 `{daemon-type}`，用 OSD 号、监视器或元数据服务器的 ID 替换掉 `{daemon-id}`。

启动剖析器

要启动堆栈剖析器，执行如下命令：

```
ceph tell {daemon-type}.{daemon-id} heap start_profiler
```

例如：

```
ceph tell osd.1 heap start_profiler
```

另外，在启动守护进程时若设置了 `CEPH_HEAP_PROFILER_INIT=true` 环境变量，剖析器也会启动。

打印统计信息

用下列命令打印统计状态：

```
ceph tell {daemon-type}.{daemon-id} heap stats
```

例如：

```
ceph tell osd.0 heap stats
```

Note

打印状态不要求剖析器在运行，也不会把堆栈分配信息转储到文件。

转储堆栈信息

用下列命令转储堆栈信息：

```
ceph tell {daemon-type}.{daemon-id} heap dump
```

例如：

```
ceph tell mds.a heap dump
```

Note

只能在剖析器运行的时候转储堆栈信息。

释放内存

要释放由 `tcmalloc` 分配、但不是被 Ceph 守护进程使用的内存，用下列命令：

```
ceph tell {daemon-type}{daemon-id} heap release
```

例如：

```
ceph tell osd.2 heap release
```

停止剖析器

要停止堆栈剖析器，执行下列命令：

```
ceph tell {daemon-type}.{daemon-id} heap stop_profiler
```

例如：

```
ceph tell osd.0 heap stop_profiler
```

CPU 剖析

如果你从源码编译时启用了 [oprofile](#)，那就可以剖析 Ceph 的 CPU 使用情况，详情见[安装 Oprofile](#)。

初始化 OPROFILE

你首次使用 `oprofile` 时要初始化，找到对应于当前运行内核的 `vmlinux` 映像：

```
ls /boot

sudo opcontrol --init

sudo opcontrol --setup --vmlinux={path-to-image} --
separate=library --callgraph=6
```

启动 OPROFILE

执行下面的命令启动 `oprofile`：

```
opcontrol --start
```

启动 `oprofile` 后，你可以运行一些 Ceph 测试。

停止 OPROFILE

执行下面的命令停止 `oprofile`：

```
opcontrol --stop
```

查看 OPROFILE 运行结果

要查看 `cmon` 最近的结果，执行下面的命令：

```
opreport -gal ./cmon | less
```

要检索 `cmon` 最近的调用图结果，执行下面的命令：

```
opreport -cal ./cmon | less
```

Important

回顾结果后，重新剖析前应该先重置，重置 `oprofile` 动作从会话目录里删除了数据。

重置 OPROFILE

要重置 `oprofile`，执行下面的命令：

```
sudo opcontrol --reset
```

Important

你应该分析后再重置，以免混合不同的剖析结果。

CEPH 存储集群 API

Ceph 存储集群 提供了消息传递层协议，用于客户端与 *Ceph 监视器* 和 *OSD* 交互，`librados` 以库形式为 *Ceph 客户端* 提供了这个功能。所有 Ceph 客户端可以用 `librados` 或 `librados` 里封装的相同功能和对象存储交互，例

如 `librbd` 和 `libcephfs` 就利用了此功能。你可以用 `librados` 直接和 Ceph 交互（如和 Ceph 兼容的应用程序、你自己的 Ceph 接口、等等）。

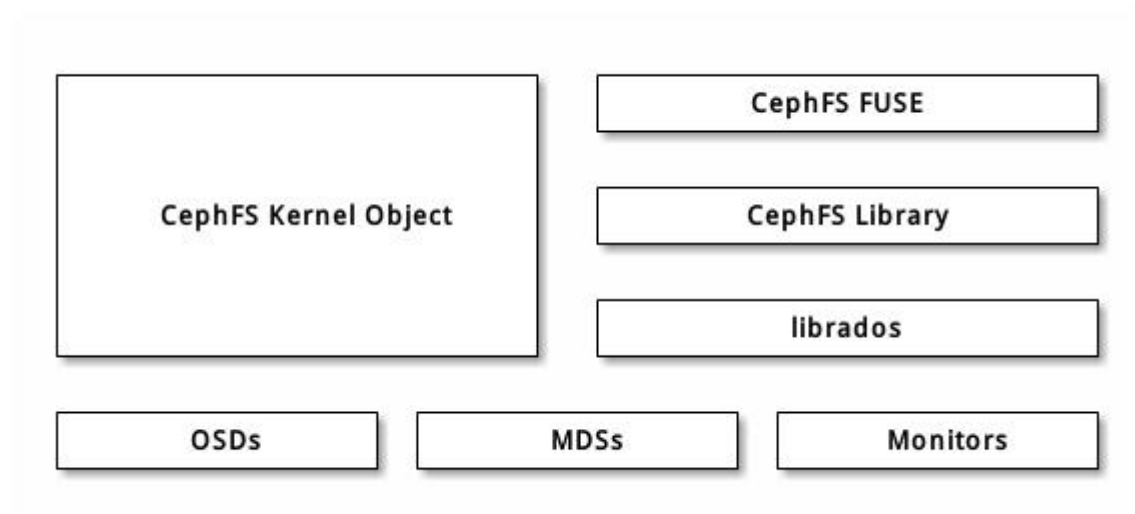
- [librados 简介](#)
 - [第一步：获取 librados](#)
 - [第二步：配置集群句柄](#)
 - [Step 3: Creating an I/O Context](#)
 - [Step 4: Closing Sessions](#)
- [librados \(C\)](#)
 - [实例：连接并写入一个对象](#)
 - [异步 IO](#)
 - [API calls](#)
- [librados \(C++\)](#)
- [librados \(Python\)](#)
 - [Installation](#)
 - [Getting Started](#)
 - [Cluster Handle API](#)
 - [Input/Output Context API](#)
 - [Object Interface](#)

CEPH 文件系统

Ceph 文件系统（Ceph FS）是个 POSIX 兼容的文件系统，它使用 Ceph 存储集群来存储数据。Ceph 文件系统与 Ceph 块设备、同时提供 S3 和 Swift API 的 Ceph 对象存储、或者原生库（librados）一样，都使用着相同的 Ceph 存储集群系统。

Important

当前，CephFS 还缺乏健壮得像 ‘fsck’ 这样的检查和修复功能。存储重要数据时需小心使用，因为灾难恢复工具还没开发完。更多的 CephFS 使用现状见 *CephFS 尝鲜*。



Ceph 文件系统要求 Ceph 存储集群内至少有一个 **Ceph 元数据服务器**。

增加/拆除元数据服务器

用 `ceph-deploy` 增加和拆除元数据服务器很简单，只要一个命令就可以增加或拆除一或多个元数据服务器。

Important

你必须部署至少一个元数据服务器才能使用 CephFS 文件系统，多个元数据服务器并行运行仍处于实验阶段。不要在生产环境下运行多个元数据服务器。

增加一元数据服务器

部署完监视器和 OSD 后，还可以部署元数据服务器。

```
ceph-deploy mds create {host-name}[:{daemon-name}]
[{host-name}[:{daemon-name}]] ...]
```

如果你想在同一主机上运行多个守护进程，可以为每个进程指定名字（可选）。

拆除一元数据服务器

尚未实现.....？

MDS 配置参考

```
mon force standby active
```

描述: 如设为 `true`，监视器会把 standby-replay 状态强设为 active。设置于 `[mon]` 或 `[global]` 下。

类型: Boolean

默认值: `true`

```
max mds
```

描述: 集群创建时的 MDS 守护进程数量，设置于 `[mon]` 或 `[global]` 下。

类型: 32-bit Integer

默认值: `1`

```
mds max file size
```

描述: 创建一新文件系统时允许的最大文件尺寸。

类型: 64-bit Integer Unsigned

默认值: 1ULL << 40

mds cache size

描述: 缓存的索引节点数。

类型: 32-bit Integer

默认值: 100000

mds cache mid

描述: 把新条目插入缓存 LRU 时的插入点（从顶端）。

类型: Float

默认值: 0.7

mds dir commit ratio

描述: 目录的脏到什么程度时做完全更新（而不是部分更新）。

类型: Float

默认值: 0.5

```
mds dir max commit size
```

描述: 一个目录更新超过多大时要拆分为较小的事务（MB）。

类型: 32-bit Integer

默认值: 90

```
mds decay halflife
```

描述: MDS 缓存热度的半衰期。

类型: Float

默认值: 5

```
mds beacon interval
```

描述: 标识消息发送给监视器的频率，秒。

类型: Float

默认值: 4

`mds beacon grace`

描述: 多久没收到标识消息就认为 MDS 落后了（并可能替换它）。

类型: Float

默认值: 15

`mds blacklist interval`

描述: OSD 运行图里的 MDS 失败后，把它留在黑名单里的时间。

类型: Float

默认值: 24.0*60.0

`mds session timeout`

描述: 客户端多久不活跃时，Ceph 就清除其能力和租期。

类型: Float

默认值: 60

`mds session autoclose`

描述: 自动关闭空闲会话前的等待时间，秒。

类型: Float

默认值: 300

`mds reconnect timeout`

描述: MDS 重启期间客户端等待时间，秒。

类型: Float

默认值: 45

`mds tick interval`

描述: MDS 执行内部周期性任务的间隔。

类型: Float

默认值: 5

```
mds dirstat min interval
```

描述: 在目录树中递归地查找目录信息的最小间隔，秒。

类型: Float

默认值: 1

```
mds scatter nudge interval
```

描述: dirstat 变更传播多快。

类型: Float

默认值: 5

```
mds client prealloc inos
```

描述: 为每个客户端会话预分配的索引节点数。

类型: 32-bit Integer

默认值: 1000

`mds early reply`

描述: 请求结果成功提交到日志前是否应该先展示给客户端。

类型: Boolean

默认值: true

`mds use tmap`

描述: 把 trivialmap 用于目录更新。

类型: Boolean

默认值: true

`mds default dir hash`

描述: 用于哈希文件在目录中分布情况的函数。

类型: 32-bit Integer

默认值: 2 (i.e., rjenkins)

`mds log`

描述: 默认为 `true` , MDS 是否要记录元数据更新 (只可在性能评测时禁用) 。

类型: Boolean

默认值: `true`

`mds log skip corrupt events`

描述: 在日志重放时 , MDS 是否应跳过损坏的日志事件。

类型: Boolean

默认值: `false`

`mds log max events`

描述: 日志里的事件达到多少时开始裁剪, 设为 -1 取消限制。

类型: 32-bit Integer

默认值: -1

`mds log max segments`

描述: 日志里的片段 (对象) 达到多少时开始裁剪, 设为 -1 取消限制。

类型: 32-bit Integer

默认值: 30

`mds log max expiring`

描述: 可同时过期的片段数。

类型: 32-bit Integer

默认值: 20

`mds log eopen size`

描述: 在一个 EOpn 事件中最大索引节点数。

类型: 32-bit Integer

默认值: 100

`mds bal sample interval`

描述: 对目录热度取样的频率（碎片粒度）。

类型: Float

默认值: 3

`mds bal replicate threshold`

描述: 达到多大热度时 Ceph 就把元数据复制到其它节点。

类型: Float

默认值: 8000

`mds bal unreplicate threshold`

描述: 热度低到多少时 Ceph 就不再把元数据复制到其它节点。

类型: Float

默认值: 0

`mds bal frag`

描述: MDS 是否应该给目录分片。

类型: Boolean

默认值: false

`mds bal split size`

描述: 目录尺寸大到多少时 MDS 就把片段拆分成更小的片段。

类型: 32-bit Integer

默认值: 10000

`mds bal split rd`

描述: 目录的最大读取热度达到多大时 Ceph 将拆分此片段。

类型: Float

默认值: 25000

```
mds bal split wr
```

描述: 目录的最大写热度达到多大时 Ceph 将拆分此片段。

类型: Float

默认值: 10000

```
mds bal split bits
```

描述: 把一个目录片段再分割成多大。

类型: 32-bit Integer

默认值: 3

```
mds bal merge size
```

描述: 目录尺寸小到多少时 Ceph 就把它合并到邻近目录片段。

类型: 32-bit Integer

默认值: 50

```
mds bal merge rd
```

描述: 读热度低于此值时 Ceph 将合并邻近目录片段。

类型: Float

默认值: 1000

```
mds bal merge wr
```

描述: 写热度低于此值时 Ceph 就合并邻近目录片段。

类型: Float

默认值: 1000

```
mds bal interval
```

描述: MDS 服务器负荷交换频率, 秒。

类型: 32-bit Integer

默认值: 10

`mds bal fragment interval`

描述: 邻近目录分片频率, 秒。

类型: 32-bit Integer

默认值: 5

`mds bal idle threshold`

描述: 热度低于此值时 Ceph 把子树迁移回父节点。

类型: Float

默认值: 0

`mds bal max`

描述: 均衡器迭代到此数量时 Ceph 就停止（仅适用于测试）。

类型: 32-bit Integer

默认值: -1

`mds bal max until`

描述: 均衡器运行多久就停止（仅适用于测试）。

类型: 32-bit Integer

默认值: -1

`mds bal mode`

计算 MDS 负载的方法。

描述: 1 = 混合；
2 = 请求速率和延时；
3 = CPU 负载。

类型: 32-bit Integer

默认值: 0

`mds bal min rebalance`

描述: 子树热度最小多少时开始迁移。

类型: Float

默认值: 0.1

```
mds bal min start
```

描述: 子树热度最小多少时 Ceph 才去搜索。

类型: Float

默认值: 0.2

```
mds bal need min
```

描述: 接受的最小目标子树片段。

类型: Float

默认值: 0.8

```
mds bal need max
```

描述: 目标子树片段的最大尺寸。

类型: Float

默认值: 1.2

```
mds bal midchunk
```

描述: 尺寸大于目标子树片段的子树，Ceph 将迁移它。

类型: Float

默认值: 0.3

```
mds bal minchunk
```

描述: 尺寸小于目标子树片段的子树，Ceph 将忽略它。

类型: Float

默认值: 0.001

```
mds bal target removal min
```

描述: Ceph 从 MDS 运行图中剔除旧数据前，均衡器至少递归多少次。

类型: 32-bit Integer

默认值: 5

```
mds bal target removal max
```

描述: Ceph 从 MDS 运行图中剔除旧数据前，均衡器最多递归多少次。

类型: 32-bit Integer

默认值: 10

```
mds replay interval
```

描述: MDS 处于 standby-replay 模式（热备）下时的日志滚动间隔。

类型: Float

默认值: 1

```
mds shutdown check
```

描述: MDS 关闭期间缓存更新间隔。

类型: 32-bit Integer

默认值: 0

`mds thrash exports`

描述: Ceph 会把子树随机地在节点间迁移。（仅用于测试）

类型: 32-bit Integer

默认值: 0

`mds thrash fragments`

描述: Ceph 会随机地分片或合并目录。

类型: 32-bit Integer

默认值: 0

`mds dump cache on map`

描述: Ceph 会把各 MDSMap 的 MDS 缓存内容转储到一文件。

类型: Boolean

默认值: false

```
mds_dump_cache_after_rejoin
```

描述: Ceph 重新加入缓存（恢复期间）后会把 MDS 缓存内容转储到一文件。

类型: Boolean

默认值: false

```
mds_verify_scatter
```

描述: Ceph 将把各种传播/聚集常量声明为 true（仅适合开发者）。

类型: Boolean

默认值: false

```
mds_debug_scatterstat
```

描述: Ceph 将把各种递归统计常量声明为 `true`（仅适合开发者）。

类型: Boolean

默认值: `false`

`mds debug frag`

描述: Ceph 将在方便时校验目录分段（仅适合开发者）。

类型: Boolean

默认值: `false`

`mds debug auth pins`

描述: 常量 debug auth pin（仅适合开发者）。

类型: Boolean

默认值: `false`

`mds debug subtrees`

描述: 常量 debug subtree (仅适合开发者)。

类型: Boolean

默认值: false

```
mds kill mdstable at
```

描述: Ceph 将向 MDSTable 代码注入 MDS 失败事件 (仅适合开发者)。

类型: 32-bit Integer

默认值: 0

```
mds kill export at
```

描述: Ceph 将向子树出口代码注入 MDS 失败事件 (仅适合开发者)。

类型: 32-bit Integer

默认值: 0

```
mds kill import at
```

描述: Ceph 将向子树入口代码注入 MDS 失败事件（仅适合开发者）。

类型: 32-bit Integer

默认值: 0

```
mds kill link at
```

描述: Ceph 将向硬链接代码注入 MDS 失败事件（仅适合开发者）。

类型: 32-bit Integer

默认值: 0

```
mds kill rename at
```

描述: Ceph 将向重命名代码注入 MDS 失败事件（仅适合开发者）。

类型: 32-bit Integer

默认值: 0

```
mds wipe sessions
```


描述: Ceph 将在启动时删除所有客户端会话（仅适合测试）。

类型: Boolean

默认值: 0

```
mds wipe ino prealloc
```

描述: Ceph 将在启动时删除索引节点号预分配元数据（仅适合测试）。

类型: Boolean

默认值: 0

```
mds skip ino
```

描述: 启动时要跳过的索引节点号数量（仅适合测试）。

类型: 32-bit Integer

默认值: 0

```
mds standby for name
```

描述: 指定一 MDS 守护进程的名字，此进程将作为它的候补。

类型: String

默认值: N/A

`mds standby for rank`

描述: 此 MDS 将作为本机架上 MDS 守护进程的候补。

类型: 32-bit Integer

默认值: -1

`mds standby replay`

描述: 决定一 `ceph-mds` 守护进程是否应该滚动并重放活跃 MDS 的日志（热备）。

类型: Boolean

默认值: false

JOURNALER

`journaler allow split entries`

描述: 允许一条目跨越条带边界。

类型: Boolean

是否必需: No

默认值: `true`

`journaler write head interval`

描述: 多频繁地更新日志头部对象？

类型: Integer

是否必需: No

默认值: `15`

`journaler prefetch periods`

描述: 重放日志时预读多少条带。

类型: Integer

是否必需: No

默认值: 10

`journal prezero periods`

描述: 把写位置前的多少条带清零。

类型: Integer

是否必需: No

默认值: 10

`journaler batch interval`

描述: 人为导致的最大额外延时。

类型: Double

是否必需: No

默认值: .001

journaler batch max

描述: 我们会延迟回写的最大字节数。

类型: 64-bit Unsigned Integer

是否必需: No

默认值: 0

CEPH-MDS – CEPH 元数据服务器守护进程

提纲

```
ceph-mds -i name [[ -hot-standby [rank] ]][[-journal_check rank]]
```

描述

ceph-mds 是 Ceph 分布式文件系统的元数据服务器守护进程。一或多个 ceph-mds 例程协作着管理文件系统的命名空间、协调到共享 OSD 集群的访问。

各 ceph-mds 守护进程例程都应该有惟一的名称，此名用于在 ceph.conf 里标识例程。

一旦守护进程启动，监视器集群会给它分配一个逻辑机架，或把它放如一个候补存储池用于接替其它崩溃的进程。一些特定选项会导致其它行为。

如果你指定了热备或日志检查选项，还必须在命令行加上机架参数、或在配置文件里指定 `mds_standby_for_[rank|name]` 之一。命令行中指定的会覆盖配置文件，指定机架会覆盖指定的名字。

选项

-f, --foreground

前台：启动后不要进入后台（在前台运行），不生成 pid 文件。通过 `ceph-run(8)` 调用时有用。

-d

调试模式：像 `-f`，但它也把所有输出日志发到了标准错误。

--setuser userorgid

启动后设置 UID。如果指定的是用户名，会查询用户记录以获取 UID 及其 GID，同时设置 GID，除非还指定了 `-setgroup` 选项。

--setgroup grouporgid

启动后设置 GID。如果指定的是组名，会查询组记录以获取 GID。

-c ceph.conf, **--conf**=ceph.conf

在启动时用 `ceph.conf` 而非默认的 `/etc/ceph/ceph.conf` 来确定监视器地址。

-m monaddress[:port]

连接到指定监视器（而非通过 `ceph.conf` 查找）。

--journal-check <rank>

尝试重放 MDS <rank> 的日志，然后退出。

--hot-standby <rank>

启动后作为 MDS <rank> 的热备。

使用范围

ceph-mds 是 Ceph 的一部分，这是个伸缩力强、开源、分布式的存储系统，更多信息参

见 <http://ceph.com/docs>。

参考

[ceph\(8\)](#), [ceph-mon\(8\)](#), [ceph-osd\(8\)](#)

创建 CEPH 文件系统

Tip

`ceph fs new` 命令是从 Ceph 0.84 起引入的，在此之前，无需手动创建文件系统，名为 `data` 和 `metadata` 的存储池默认即存在。

Ceph 命令行现在有了创建和删除文件系统的命令，但是当前一套集群只能有一个文件系统存在。

一个 Ceph 文件系统需要至少两个 RADOS 存储池，一个用于数据、一个用于元数据。配置这些存储池时需考虑：

- 为元数据存储池设置较高的副本水平，因为此存储池丢失任何数据都会导致整个文件系统失效。
- 为元数据存储池分配低延时存储器（像 SSD），因为它会直接影响到客户端的操作延时。

关于存储池的管理请参考 [存储池](#)。例如，要用默认设置为文件系统创建两个存储池，你可以用下列命令：

```
$ ceph osd pool create cephfs_data <pg_num>

$ ceph osd pool create cephfs_metadata <pg_num>
```

创建好存储池后，你就可以用 `fs new` 命令创建文件系统了：

```
$ ceph fs new <fs_name> <metadata> <data>
```

例如：

```
$ ceph fs new cephfs cephfs_metadata cephfs_data

$ ceph fs ls

name: cephfs, metadata pool: cephfs_metadata, data pools:
[cephfs_data ]
```

文件系统创建完毕后，MDS 服务器就能达到 *active* 状态了，比如在一个单 MDS 系统中：

```
$ ceph mds stat

e5: 1/1/1 up {0=a=up:active}
```

建好文件系统且 MDS 活跃后，你就可以挂载此文件系统了：

- [挂载 CephFS 文件系统](#)
- [把 CephFS 挂载为用户空间文件系统](#)

用内核驱动挂载 CEPH 文件系统

要挂载 Ceph 文件系统，如果你知道监视器 IP 地址可以用 `mount` 命令、或者用 `mount.ceph` 工具来自动解析监视器 IP 地址。例如：

```
sudo mkdir /mnt/mycephfs

sudo mount -t ceph 192.168.0.1:6789:/ /mnt/mycephfs
```


要挂载启用了 `cephx` 认证的 Ceph 文件系统，你必须指定用户名、密钥。

```
sudo mount -t ceph 192.168.0.1:6789:/ /mnt/mycephfs -o
name=admin,secret=AQATSKdNGBnwLhAAnNDKnH65FmVKpXZJVasUeQ=
```

前述用法会把密码遗留在 Bash 历史里，更安全的方法是从文件读密码。例如：

```
sudo mount -t ceph 192.168.0.1:6789:/ /mnt/mycephfs -o
name=admin,secretfile=/etc/ceph/admin.secret
```

关于 `cephx` 参见[认证](#)。

要卸载 Ceph 文件系统，可以用 `umount` 命令，例如：

```
sudo umount /mnt/mycephfs
```

Tip

执行此命令前确保你不在此文件系统目录下。

详情见 [mount.ceph](#)。

用户空间挂载 CEPH 文件系统

Ceph v0.55 及后续版本默认开启了 `cephx` 认证。从用户空间（FUSE）挂载一 Ceph 文件系统前，确保客户端主机有一份 Ceph 配置副本、和具备 Ceph 元数据服务器能力的密钥环。

1. 在客户端主机上，把监视器主机上的 Ceph 配置文件拷贝到 `/etc/ceph/` 目录下。

```
2. sudo mkdir -p /etc/ceph
```

```
sudo scp {user}@{server-machine}:/etc/ceph/ceph.conf  
/etc/ceph/ceph.conf
```

3. 在客户端主机上，把监视器主机上的 Ceph 密钥环拷贝到 `/etc/ceph` 目录下。

```
sudo scp {user}@{server-  
machine}:/etc/ceph/ceph.keyring  
/etc/ceph/ceph.keyring
```

4. 确保客户端机器上的 Ceph 配置文件和密钥环都有合适的权限位，如 `chmod 644`。

`cephx` 如何配置请参考 [CEPHX 配置参考](#)。

要把 Ceph 文件系统挂载为用户空间文件系统，可以用 `ceph-fuse` 命令，例如：

```
sudo mkdir /home/username/cephfs  
  
sudo ceph-fuse -m 192.168.0.1:6789 /home/username/cephfs
```

详情见 [ceph-fuse](#)。

用 FSTAB 挂载

如果你从文件系统表挂载，Ceph 文件系统将在启动时自动挂载。

内核驱动

要用内核驱动挂载 Ceph FS，按下列格式添加到 `/etc/fstab`：

```
{ipaddress}:{port}:/ {mount}/{mountpoint} {filesystem-  
name}  
[name=username,secret=secretkey|secretfile=/path/to/secre  
tfile],[{mount.options}]
```

例如：

```
10.10.10.10:6789:/      /mnt/ceph    ceph
name=admin,secretfile=/etc/ceph/secret.key,noatime    0
2
```

Important

启用了认证时，`name` 及 `secret` 或 `secretfile` 选项必须加。

详情见[认证](#)。

FUSE

要在用户空间挂载 Ceph 文件系统，按如下加入 `/etc/fstab`：

```
#DEVICE                                PATH
TYPE      OPTIONS

id={user-ID}[,conf={path/to/conf.conf}] /mount/path
fuse.ceph defaults 0 0
```

例如：

```
id=admin /mnt/ceph fuse.ceph defaults 0 0

id=myuser,conf=/etc/ceph/cluster.conf /mnt/ceph2
fuse.ceph defaults 0 0
```

`DEVICE` 字段是逗号分隔的一系列选项，确保填上了 ID（如 `admin`，不是 `client.admin`）。你可以用此方法把任何 `ceph-fuse` 接受的选项加上。

详情见[认证](#)。

CEPHFS – CEPH 文件系统选项工具

提纲

cephfs [*path command options*]

描述

cephfs 是个控制工具，用于查看和修改 Ceph 分布式文件系统内的文件布局和定位数据。

支持下列三个子命令：

- **show_layout** 查看一文件或目录的布局信息
- **set_layout** 设置一文件或目录的布局信息
- **show_location** 查看一文件的定位信息

选项

使用设置或查看布局或位置命令时，可用的可选参数不同。

查看选项：

-l --offset

检索（指定文件的）定位数据时指定个偏移量。

设置选项：

-u --stripe_unit

设置各条带的尺寸。

-c --stripe_count

指定条带跨越的对象数

-s --object_size

指定条带跨越的对象尺寸

-p --pool

指定目标存储池（数字，而非名字）

-o --osd

指定这些 OSD 优先作为主 OSD（已废弃、且忽略掉了）

使用限制

设置布局数据时，指定的对象尺寸必须被指定的条带单位平均分割，未指定的参数都按系统默认值。

显式地设置一文件和目录布局的含义不同。设置一文件的布局指示如何放置这一个文件，而且必须在存入数据前就设置好。删减文件也不允许你更改其布局。

设置一目录的布局设置了“默认布局”，它将用于设置此目录内后续新建文件（或子目录）的文件布局。先前已存在文件的布局不会变。

也许你注意到了布局信息允许你指定优先使用的 OSD。此功能已废弃，较新版本的 Ceph 服务器会忽略它，别用。

使用范围

cephfs 是 Ceph 的一部分，这是个伸缩力强、开源、分布式的存储系统，更多信息参见 <http://ceph.com/docs>。

参考

ceph(8)

CEPH-FUSE – 基于 FUSE 的 CEPH 客户端

提纲

ceph-fuse [-m *monaddr:port*] *mountpoint* [*fuse options*]

描述

ceph-fuse 是 Ceph 分布式文件系统的 FUSE（用户空间文件系统）客户端，它会把 Ceph 文件系统（用 -m 选项或 ceph.conf 指定）挂载到指定挂载点。

文件系统可这样卸载：

```
fusermount -u mountpoint
```

或向 **ceph-fuse** 进程发送 **SIGINT** 信号。

选项

ceph-fuse 识别不了的选项将传递给 libfuse。

-d

启动后将脱离终端、进入守护状态。

-c *ceph.conf*, **--conf**=*ceph.conf*

用指定的 *ceph.conf* 而非默认的 `/etc/ceph/ceph.conf` 来查找启动时需要的监视器地址。

-m *monaddress[:port]*

连接到指定监视器，而不是从 ceph.conf 里找。

-r *root_directory*

把文件系统内的 *root_directory* 作为根挂载，而不是整个 Ceph 文件系统树。

使用范围

ceph-fuse 是 Ceph 的一部分，这是个伸缩力强、开源、分布式的存储系统，更多信息参见 <http://ceph.com/docs>。

参考

fusermount(8), **ceph**(8)

MOUNT.CEPH – 用于挂载 CEPH 文件系统

提纲

mount.ceph *monaddr1[,monaddr2,...]:[/subdir] dir* [-o *options*]

描述

mount.ceph 是在 Linux 主机上挂载 Ceph 文件系统的简单助手。它只负责把监视器主机名解析为 IP 地址、从硬盘读取认证密钥，大多数实际工作由 Linux 内核客户端组件完成。事实上，无需认证的 Ceph 文件系统无需 **mount.ceph** 也能挂载，只要指定监视器 IP 地址即可：

```
mount -t ceph 1.2.3.4:/ mountpoint
```

各监视器地址 *monaddr* 的格式为 *host[:port]*，如果端口未指定，就用默认的 6789。

多个监视器地址用逗号分隔。要成功地挂载只需一个监视器即可，客户端将从某个能响应的监视器获知其它监视器。然而最好指定多个监视器，以免挂载时正好赶上那个监视器挂了。

如果要挂载文件系统的一子集，可指定一个子目录 *subdir*。

mount 助手程序的惯例是前两个选项分别为要挂载的设备和目标路径，其它选项必须位于这些固定参数之后。

选项

wsize

整数，最大写尺寸。默认：无（回写用较小的 wsize 和条带单元）

rsize

整数（字节），最大预读，1024 的倍数。默认：524288（512*1024）

osdtimeout

整数（秒）。默认：60

osdkeepalivetimeout

整数。默认：5

mount_timeout

整数（秒）。默认：60

osd_idle_ttl

整数（秒）。默认：60

caps_wanted_delay_min

整数，能力释放延迟时间。默认：5

caps_wanted_delay_max

整数，能力释放延迟时间。默认：60

cap_release_safety

整数。默认：自行计算

readdir_max_entries

整数。默认：1024

readdir_max_bytes

整数。默认：524288（512*1024）

write_congestion_kb

整数（kb），运行中的最大回写量，随可用内存变化。默认：根据可用内存计算

snapdirname

字符串，为快照的隐藏目录设置个名字。默认：.snap

name

使用 cephx 认证时的 RADOS 用户名。默认：guest

secret

用于 `cephx` 的密钥。这个选项不安全，因为它把密钥暴露在了命令行，用 `secretfile` 选项可避免此问题。

secretfile

用于 `cephx` 的密钥文件路径。

ip

本机 IP

noshare

创建新客户端例程，而不是和挂载同一集群的例程共享资源。

**dirst
at**

用 `cat dirname` 读取文件信息。默认： `off`

**n
o
d
i
r
s
t
a
t**

不用 `cat dirname` 读取文件信息

**r
b
y
t
e
s**

目录的 `st_size` 报告产生于目录内容的递归尺寸。默认： `on`

**n
o
r
b
y
t
e
s**

目录的 `st_size` 无需通过递归目录内容来获取。

n
o
c
r
c

写入时不做 crc 校验

n
o
a
s
y
n
c
r
e
a
d
i
r

读目录时不经过 dcache

CEPHFS 配额管理

CephFS 允许给系统内的任意目录设置配额，这个配额可以限制目录树中这一点以下的**字节数**或者**文件数**。

局限性

1. *配额是合作性的、非对抗性的。* CephFS 的配额功能依赖于挂载它的客户端的合作，在达到上限时要停止写入；无法阻止篡改过的或者对抗性的客户端，它们可以想写多少就写多少。在客户端完全不可信时，用配额防止多占空间是靠不住的。
2. *配额是不准确的。* 在达到配额限制一小段时间后，正在写入文件系统的进程才会被停止。很难避免它们超过配置的限额、多写入一些数据。会超过配额多大幅度主要取决于时间长短，而非数据量。一般来说，超出配置的限额之后 10 秒内，写入会被停掉。
3. *内核客户端还没实现配额功能。* 用户空间客户端（libcephfs、ceph-fuse）已经支持配额了，但是 Linux 内核客户端还没实现。
4. *基于路径限制挂载时必须谨慎地配置配额。* 客户端必须能够访问配置了配额的那个目录的索引节点，这样才能执行配额管理。如果某一客户端被 MDS 能力限制成了只能访问一个特定路径（如 `/home/user`），并且它们无权访问配置了配额的父目录（如 `/home`），这个客户端就不会按配额执行。所以，基于路径做访问控制时，最好在限制了客户端的那个目录（如 `/home/user`）、或者它下面的子目录上配置配额。

配置

就像 CephFS 里的其它配置一样，配额也是用虚拟扩展属性来配置的：

- `ceph.quota.max_files` – file limit
- `ceph.quota.max_bytes` – byte limit

如果这些属性出现在目录索引节点里，就意味着那里配置了配额；如果不存在，那么那个目录就没设置配额（然而父目录仍然有可能配置了）。

要设置配额：

```
# 100 MB

setfattr -n ceph.quota.max_bytes -v 100000000 /some/dir

# 10000 个文件

setfattr -n ceph.quota.max_files -v 10000 /some/dir
```

要查看设置的配额：

```
getfattr -n ceph.quota.max_bytes /some/dir  
  
getfattr -n ceph.quota.max_files /some/dir
```

需要留意的是，如果这些扩展属性的值是 0，就说明没设置配额。

要删除配额：

```
setfattr -n ceph.quota.max_bytes -v 0 /some/dir  
  
setfattr -n ceph.quota.max_files -v 0 /some/dir
```

在 CEPHFS 上使用 HADOOP

Ceph 文件系统可作为 Hadoop 文件系统（HDFS）的落地式替代品，本章描述了 Ceph 用于 Hadoop 存储的安装和配置过程。

依赖关系

- CephFS 的 Java 接口
- Hadoop 的 CephFS 插件

Important

当前要求 Hadoop 1.1.x 稳定版系列。

安装

在 CephFS 上使用 Hadoop 有三个必要条件。首先，必须有一个运行的 Ceph。建设一个 Ceph 集群和文件系统不是本章的讨论范围，请参考 Ceph 安装文档。

另外两个必要条件是 Hadoop 安装和 Ceph 文件系统 Java 软件包，包括 Java 的 CephFS Hadoop 插件。后续步骤分别是把依赖添加到 Hadoop 的环境变量 CLASSPATH、更改 Hadoop 配置让它使用 Ceph 文件系统。

CEPHFS JAVA 软件包

- CephFS Hadoop 插件 ([hadoop-cephfs.jar](#))

安装 Hadoop 时的依赖取决于你怎么部署的，一般来说，这些依赖必须安装到 Hadoop 集群的各节点、且必须能从 CLASSPATH 里找到。典型做法是把另加的 jar 文件放入 `hadoop/lib` 目录、或编辑 `hadoop-env.sh` 中的 `HADOOP_CLASSPATH` 变量。

Hadoop 集群内的各参与节点都必须安装原生 Ceph 文件系统客户端。

HADOOP 配置

本段描述了用于控制 Ceph 的 Hadoop 配置选项，这些选项应该写于 Hadoop 配置文件 `conf/core-site.xml`。

Property	Value	Notes
fs.default.name	Ceph URI	ceph://[monaddr:port]/
ceph.conf.file	Local path to ceph.conf	/etc/ceph/ceph.conf
ceph.conf.options	Comma separated list of Ceph configuration key/value pairs	opt1=val1,opt2=val2
ceph.root.dir	Mount root directory	Default value: /

Property	Value	Notes
ceph.mon.address	Monitor address	host:port
ceph.auth.id	Ceph user id	Example: admin
ceph.auth.keyfile	Ceph key file	
ceph.auth.keyring	Ceph keyring file	
ceph.object.size	Default file object size in bytes	Default value (64MB): 67108864
ceph.data.pools	List of Ceph data pools for storing file.	Default value: default Ceph pool.
ceph.localize.reads	Allow reading from file replica objects	Default value: true

对每文件定制副本数的支持

Hadoop 文件系统接口允许用户在创建文件时定制复制因子（例如每块 3 个副本）。然而对象复制因子在 Ceph 里是以每存储池为单位进行控制的，并且 Ceph 文件系统默认会包含一个预配置的存储池。因此，在 Ceph 之上运行 Hadoop 时，为支持每文件复制策略，必须创建多个有非默认复制因子的存储池，且必须配置 Hadoop 让它自动选择合适存储池。

额外的数据存储池用 `ceph.data.pools` 指定，此选项的值是逗号分隔的一溜存储池名字。此选项被忽略或为空时将使用默认存储池。例如，下面的配置会使 Hadoop 在为文件选择存储池时考虑 `pool11`、`pool12`、`pool15`：

```
<property>

  <name>ceph.data.pools</name>

  <value>pool1,pool2,pool5</value>

</property>
```

Hadoop 不会自动创建存储池。要创建一个指定复制因子的存储池，可用 `ceph osd pool create` 命令、然后用 `ceph osd pool set` 设置存储池的 `size` 属性。更多的创建、配置手册见 [RADOS 存储池文档](#)。

存储池创建、配置完毕后，必须把新存储池可用于数据存储的消息告知元数据服务，用 `ceph mds add_data_pool` 命令告知，这样存储池就可存储文件系统数据了。

首先创建存储池。本例中，我们创建 `hadoop1` 存储池，其复制因子为 1。

```
ceph osd pool create hadoop1 100

ceph osd pool set hadoop1 size 1
```

下一步，找出存储池 ID，命令为 `ceph osd dump`。例如，找出刚创建的 `hadoop1` 存储池：

```
ceph osd dump | grep hadoop1
```

输出应该类似：

```
pool 3 'hadoop1' rep size 1 min_size 1 crush_ruleset 0...
```

其中，`3` 是存储池 id。下面我们用前述 ID 把存储池注册为数据存储池，用于存储文件系统数据。

```
ceph mds add_data_pool 3
```

最后配置 Hadoop，让它在为新文件选择目标存储池时考虑此存储池。

```
<property>

    <name>ceph.data.pools</name>

    <value>hadoop1</value>

</property>
```

存储池选择规则

下面的规则描述了 Hadoop 如何根据期望复制因子从 `ceph.data.pools` 配置的一堆存储池中选择一个存储池的规则。

1. 未指定存储池时用 Ceph 的默认 `data` 存储池；
2. 复制因子相同时，定制存储池优先于 Ceph 的默认 `data` 存储池；
3. 复制因子和期望值相同的存储池会被选择；
4. 否则，选择复制因子和期望值最接近的存储池，或者复制因子最大的。

存储池选择调试

Hadoop 不确定存储池复制因子时会产生日志（如它未被配置为数据存储池），日志消息长相如下：

```
Error looking up replication of pool: <pool name>
```

未能选到复制数准确匹配的存储池时 Hadoop 也会产生日志，其长相如下：

```
selectDataPool path=<path> pool:repl=<name>:<value>
wanted=<value>
```

LIBCEPHFS (JAVADOC)

查看自动生成的 [JavaDoc pages for the CephFS Java bindings](#).

CEPHFS-JOURNAL-TOOL

目的

如果 CephFS 的日志损坏了，就需要专家介绍，把文件系统恢复到工作状态。

`cephfs-journal-tool` 工具提供的功能可帮助专家们检查、修改、或从日志中提取数据。

Warning

此工具很**危险**，因为它会直接修改文件系统的内部数据结构。做好备份、多加小心、多寻求专家建议，如果不确定，就别运行此工具。

语法

```
cephfs-journal-tool journal <inspect|import|export|reset>

cephfs-journal-tool header <get|set>

cephfs-journal-tool event <get|splice|apply> [filter]
<list|json|summary>
```

此工具有三种模式：`journal`、`header` 和 `event`，分别操作日志内的整个日志、头部、和事件。

JOURNAL 模式

这应该是你访问日志状态的起点。

- `inspect` 报告日志的健康状况，它将鉴定存储日志中丢失或被篡改的对象。要注意的是，此过程不能鉴定事件本身的不一致，它仅鉴定存在且可解码的。
- `import` 和 `export` 以一个稀疏文件格式读取和写入日志的二进制转储，用文件名作为最后一个参数。导出操作在日志损坏时可能不会可靠地运行，因为丢失了对象。
- `reset` 删减日志，丢弃其内的任意信息。

实例： JOURNAL INSPECT

```
# cephfs-journal-tool journal inspect

Overall journal integrity: DAMAGED

Objects missing:

    0x1

Corrupt regions:

    0x400000-ffffffffffffffffffff
```

实例：日志的导入和导出

```
# cephfs-journal-tool journal export myjournal.bin

journal is 4194304~80643

read 80643 bytes at offset 4194304

wrote 80643 bytes at offset 4194304 to myjournal.bin
```

注：这是个稀疏文件，你可以在保留其稀疏特性的前提下，
用 `tar cSzf myjournal.bin.tgz myjournal.bin` 进行压缩。

```
# cephfs-journal-tool journal import myjournal.bin
```

```
undump myjournal.bin

start 4194304 len 80643

writing header 200.00000000

  writing 4194304~80643

done.
```

Note

明智的做法是，在开始修改前先用 `journal export <backup file>` 命令备份日志。

头模式

- `get` 输出日志头部的当前内容。
- `set` 修改头部的一个属性。可修改属性有 `trimmed_pos`、`expire_pos` 和 `write_pos`。

实例：HEADER GET/SET

```
# cephfs-journal-tool header get

{ "magic": "ceph fs volume v011",

  "write_pos": 4274947,

  "expire_pos": 4194304,

  "trimmed_pos": 4194303,

  "layout": { "stripe_unit": 4194304,

    "stripe_count": 4194304,
```

```
"object_size": 4194304,  
  
"cas_hash": 4194304,  
  
"object_stripe_unit": 4194304,  
  
"pg_pool": 4194304}}  
  
# cephfs-journal-tool header set trimmed_pos 4194303  
  
Updating trimmed_pos 0x400000 -> 0x3ffffff  
  
Successfully updated header.
```

事件模式

事件模式下可对日志内容进行详细的检查和操作。事件模式可操作日志中的所有或过滤出的事件。

`cephfs-journal-tool event` 的参数由动作、可选过滤器参数、和输出模式组成。

```
cephfs-journal-tool event <action> [filter] <output>
```

动作：

- `get` 从日志读出事件；
- `splice` 擦除日志中的某些事件或区域；
- `apply` 从事件中提取文件系统元数据、并试着应用到元数据存储中。

过滤器选项：

- `--range <int begin>..[int end]` 过滤从 `begin`（包含）到 `end`（不包含）之间的事件；

- `--path <path substring>` 按字符串过滤相关事件，此字符串包含在与这些事件相关的元数据中；
- `--inode <int>` 按字符串过滤事件，此字符串包含在与这些事件相关的元数据中；
- `--type <type string>` 过滤此类型的事件；
- `--frag <ino>[.frag id]` 过滤与此目录片段相关的事件；
- `--dname <string>` 过滤与目录片段内的指定 **dentry** 相关的事件，只能和 `--frag` 一起使用；
- `--client <int>` 过滤来自此客户端会话 ID 的事件。

过滤器可用“与”操作组合使用，也就是最终结果为各过滤器的交集。

输出模式：

- `binary`: 把各事件写入一个二进制文件，放入 `--path` 指定的目录；
- `json`: 把所有事件组织为序列化的 **JSON** 对象列表，并写入单个文件；
- `summary`: 把人类可读的事件汇总写到标准输出；
- `list`: 写出一个人类可读的摘要列表，其中包含各事件的类型、以及此事件所影响的文件路径。

实例：事件模式

```
# cephfs-journal-tool event get json --path output.json

Wrote output to JSON file 'output.json'

# cephfs-journal-tool event get summary

Events by type:

NOOP: 2

OPEN: 2

SESSION: 2
```


SUBTREEMAP: 1

UPDATE: 43

cephfs-journal-tool event get list

0x400000 SUBTREEMAP: ()

0x400308 SESSION: ()

0x4003de UPDATE: (setattr)

/

0x40068b UPDATE: (mkdir)

diralpha

0x400d1b UPDATE: (mkdir)

diralpha/filealpha1

0x401666 UPDATE: (unlink_local)

stray0/10000000001

diralpha/filealpha1

0x40228d UPDATE: (unlink_local)

diralpha

stray0/10000000000

0x402bf9 UPDATE: (scatter_writebehind)

stray0

```
0x403150 UPDATE:  (mkdir)
```

```
    dirbravo
```

```
0x4037e0 UPDATE:  (openc)
```

```
    dirbravo/.filebravo1.swp
```

```
0x404032 UPDATE:  (openc)
```

```
    dirbravo/.filebravo1.swpx
```

```
# cephfs-journal-tool event get --path /filebravo1 list
```

```
0x40785a UPDATE:  (openc)
```

```
    dirbravo/filebravo1
```

```
0x4103ee UPDATE:  (cap update)
```

```
    dirbravo/filebravo1
```

```
# cephfs-journal-tool event splice --range
```

```
0x40f754..0x410bf1 summary
```

```
Events by type:
```

```
    OPEN: 1
```

```
    UPDATE: 2
```

```
# cephfs-journal-tool event apply --range 0x410bf1..  
summary
```

Events by type:

NOOP: 1

SESSION: 1

UPDATE: 9

cephfs-journal-tool event get --inode=1099511627776
list

0x40068b UPDATE: (mkdir)

diralpha

0x400d1b UPDATE: (mkdir)

diralpha/filealpha1

0x401666 UPDATE: (unlink_local)

stray0/10000000001

diralpha/filealpha1

0x40228d UPDATE: (unlink_local)

diralpha

stray0/10000000000

cephfs-journal-tool event get --frag=1099511627776 --
dname=filealpha1 list

0x400d1b UPDATE: (mkdir)

```
diralpha/filealpha1

0x401666 UPDATE:  (unlink_local)

stray0/10000000001

diralpha/filealpha1

# cephfs-journal-tool event get binary --path bin_events

Wrote output to binary files in directory 'bin_events'
```

文件布局

文件布局可控制如何把文件内容映射到各 Ceph RADOS 对象，你可以用*虚拟扩展属性*或 xattrs 来读、写某一文件的布局。

布局 xattrs 的名字取决于此文件是常规文件还是目录，常规文件的布局 xattrs 叫作 `ceph.file.layout`、目录的布局 xattrs 叫作 `ceph.dir.layout`。因此后续实例中若用的是 `ceph.file.layout`，处理目录时就要替换为 `dir`。

Tip

你的 Linux 发行版也许默认没提供操作 xattrs 的命令，所需软件包通常是 `attr`。

布局字段

`pool`

字符串，可指定 ID 或名字。它是文件的数据对象所在的 RADOS 存储池。

`stripe_unit`

字节数、整数。一个文件的数据块按照此尺寸（字节）像 RAID 0 一样分布。一文件所有条带单元的尺寸一样，最后一个条带单元通常不完整——即它包含文件末尾的数据、还有数据末端到固定条带单元尺寸之间的未使用“空间”。

stripe_count

整数。组成 RAID 0 “条带”数据的连续条带单元数量。

object_size

整数个字节。文件数据按此尺寸分块为 RADOS 对象。

用 GETFATTR 读取布局

读出的布局信息表示为单个字符串：

```
$ touch file

$ getfattr -n ceph.file.layout file

# file: file

ceph.file.layout="stripe_unit=4194304
stripe_count=1 object_size=4194304
pool=cephfs_data"
```

读取单个布局字段：

```
$ getfattr -n ceph.file.layout.pool file

# file: file

ceph.file.layout.pool="cephfs_data"

$ getfattr -n ceph.file.layout.stripe_unit
file

# file: file

ceph.file.layout.stripe_unit="4194304"
```

```
$ getfattr -n ceph.file.layout.stripe_count
file

# file: file

ceph.file.layout.stripe_count="1"

$ getfattr -n ceph.file.layout.object_size
file

# file: file

ceph.file.layout.object_size="4194304"
```

Note

读取布局时，存储池通常是以名字标识的。然而在极少数情况下，如存储池刚创建时，可能会输出 ID。

目录只有经过定制才会有显式的布局，如果从没更改过，那么读取其布局时就会失败：这表明它会继承父目录的显式布局设置。

```
$ mkdir dir

$ getfattr -n ceph.dir.layout dir

dir: ceph.dir.layout: No such attribute

$ setfattr -n ceph.dir.layout.stripe_count -v 2 dir

$ getfattr -n ceph.dir.layout dir

# file: dir
```

```
ceph.dir.layout="stripe_unit=4194304
stripe_count=2 object_size=4194304
pool=cephfs_data"
```

用 SETFATTR 设置布局

布局字段可用 `setfattr` 修改：

```
$ ceph osd lspools

0 rbd,1 cephfs_data,2 cephfs_metadata,

$ setfattr -n ceph.file.layout.stripe_unit -
v 1048576 file2

$ setfattr -n ceph.file.layout.stripe_count
-v 8 file2

$ setfattr -n ceph.file.layout.object_size -
v 10485760 file2

$ setfattr -n ceph.file.layout.pool -v 1
file2 # Setting pool by ID

$ setfattr -n ceph.file.layout.pool -v
cephfs_data file2 # Setting pool by name
```

布局的继承

文件会在创建时继承其父目录的布局，然而之后对父目录布局的更改不会影响其子孙。

```
$ getfattr -n ceph.dir.layout dir

# file: dir
```

```
ceph.dir.layout="stripe_unit=4194304
stripe_count=2 object_size=4194304
pool=cephfs_data"
```

证实 file1 继承了其父的布局

```
$ touch dir/file1
```

```
$ getfattr -n ceph.file.layout dir/file1
```

```
# file: dir/file1
```

```
ceph.file.layout="stripe_unit=4194304
stripe_count=2 object_size=4194304
pool=cephfs_data"
```

现在更改目录布局，然后再创建第二个文件

```
$ setfattr -n ceph.dir.layout.stripe_count -
v 4 dir
```

```
$ touch dir/file2
```

证实 file1 的布局未变

```
$ getfattr -n ceph.file.layout dir/file1
```

```
# file: dir/file1
```

```
ceph.file.layout="stripe_unit=4194304
stripe_count=2 object_size=4194304
pool=cephfs_data"
```



```
# 但 file2 继承了父目录的新布局

$ getfattr -n ceph.file.layout dir/file2

# file: dir/file2

ceph.file.layout="stripe_unit=4194304
stripe_count=4 object_size=4194304
pool=cephfs_data"
```

如果中层目录没有设置布局，那么内层目录中创建的文件也会继承此目录的布局：

```
$ getfattr -n ceph.dir.layout dir

# file: dir

ceph.dir.layout="stripe_unit=4194304
stripe_count=4 object_size=4194304
pool=cephfs_data"

$ mkdir dir/childdir

$ getfattr -n ceph.dir.layout dir/childdir

dir/childdir: ceph.dir.layout: No such
attribute

$ touch dir/childdir/grandchild

$ getfattr -n ceph.file.layout
dir/childdir/grandchild

# file: dir/childdir/grandchild
```

```
ceph.file.layout="stripe_unit=4194304
stripe_count=4 object_size=4194304
pool=cephfs_data"
```

把数据存储池加入 MDS

要把存储池当 CephFS 用，你必须把它加入元数据服务器。

```
$ ceph mds add_data_pool cephfs_data_ssd

# 现在应该能看到存储池了

$ ceph fs ls

.... data pools: [cephfs_data
cephfs_data_ssd ]
```

确保你的 cephx 密钥允许客户端访问新存储池。

CEPH 文件系统客户端的驱逐

当某个文件系统客户端不响应或者有其它异常行为时，有必要强制切断它到文件系统的访问，这个过程就叫做驱逐。

这个过程有点过头，它是为了防止异常客户端导致数据不一致。

OSD 黑名单机制

首先，把此客户端加入黑名单以防止此它在 RADOS 级再操作任何数据。你可能熟悉这个概念，它在其它存储系统里叫做隔离栏 (*fencing*)。

找出要从 MDS 会话列表中驱逐的客户端：

```
# ceph daemon mds.a session ls
```

```
[  
  
  { "id": 4117,  
  
    "num_leases": 0,  
  
    "num_caps": 1,  
  
    "state": "open",  
  
    "replay_requests": 0,  
  
    "reconnecting": false,  
  
    "inst": "client.4117 172.16.79.251:0\3271",  
  
    "client_metadata": { "entity_id": "admin",  
  
                          "hostname": "fedoravm.localdomain",  
  
                          "mount_point": "\/home\/user\/mnt"}}}]
```

本例中，‘fedoravm’客户端的地址是 172.16.79.251:0/3271，所以这样加黑名单：

```
# ceph osd blacklist add 172.16.79.251:0/3271  
  
blacklisting 172.16.79.251:0/3271 until 2014-12-09  
13:09:56.569368 (3600 sec)
```

OSD 元图屏蔽

被驱逐的客户端已在 OSD 图的中央（mon）副本中标记为进黑名单了，现在有必要确认一下本次 OSD 图更新已经传递给了后续文件系统 I/O 会涉及的所有守护进程。可以用 `osdmap barrier` 这个 MDS 管理套接字命令。

首先读出最新的 OSD 元图：

```
# ceph osd dump

epoch 12

fsid fd61ca96-53ff-4311-826c-f36b176d69ea

created 2014-12-09 12:03:38.595844

modified 2014-12-09 12:09:56.619957

...
```

本例中它是 12，让 MDS 屏蔽此元图：

```
# ceph daemon mds.a osdmap barrier 12
```

MDS 会话驱逐

最后就可以安全地驱逐此客户端的 MDS 会话了，这样它持有的所有能力就可以发行给其它客户端了。这里的 ID 是 `session ls` 输出里的 `id` 属性：

```
# ceph daemon mds.a session evict 4117
```

好了！这个客户端现在已经被驱逐了，而且它持有的资源也可以重分配给其它客户端了。

背景：OSD 元图屏蔽

此屏蔽的目的是为了确保当我们分配出这些能力后，别的客户端就有可能触碰同样的 RADOS 对象，接受分配能力的客户端们必须有足够新的 OSD 图，这样它们才不会与取消的操作（来自 ENOSPC）或进了黑名单的客户端（来自驱逐过程）竞争。

更具体些，我们设置元图屏蔽的情形有：

- 客户端驱逐（此客户端被加入黑名单了、且其它客户端必须等到有加黑之后的元图出现才能触碰同一批对象）；
- 客户端正在处理 OSD 图的完整标识（此客户端可能会在快完整的元图中取消一些 OSD 操作，这样其它客户端都必须等着，直到元图完整或周期达到才能触碰同样的对象）；
- MDS 启动，因为我们不会永久存储屏蔽图元，所以必须假设重启后总是需要最新的 OSD 图。

注意，为保持简洁性这是个全局值，其实我们可以做到按每索引节点维护此值，但我们没有这么做，因为：

- 它会复杂得多；
- 每索引节点需额外多占 4 字节的内存；
- 无论如何它都不会比大家一直都拥有最新的 OSD 图更有效，多数情况下，大家都能轻松地越过这个屏障而不是等着它。
- 我们仅在极少数情形下使用这种屏蔽，所以每索引节点这样细粒度的实现带来的好处也很少见。

元图屏蔽随其它能力消息一起传递，并且可指示消息接收器在看到这个 OSD 元图前、别再向那些 OSD 发送 RADOS 操作。主要是面向客户端（它们直接向文件写入数据）的，也适用于 MDS，因为像文件尺寸探测和文件删除这样的操作都是直接在 MDS 上进行的。

处理占满的 CEPH 文件系统

当 RADOS 集群使用率达到总容量的 `mon_osd_full_ratio`（默认 95%）时，它会被打上 OSD full 标记。在问题解决前（如扩容集群），此标记会使大多数常规 RADOS 客户端暂停所有操作。

文件系统对 full 标记还有些特殊处理，下文详述。

HAMMER 及更高版

从 hammer 发布开始，以下行为会导致占满的文件系统返回 ENOSPC 错误：

- 客户端写入数据

- 删除和裁剪以外的其它元数据操作

因为只有当数据刷回到硬盘（有时是 `write` 返回 0 之后）才可能遇到占满的情况，所以应用程序调用 `fsync` 或 `fclose`（或等价操作）进行文件处理时才可能遇到 `ENOSPC` 错误。

调用 `fsync` 能可靠地反映数据是否写入了硬盘，并且在没写入时返回错误。`fclose` 只有在缓冲的数据从上次写入以来被偶尔刷回过才会返回错误 [译者：原文可能有误] ——
`fclose` 成功并不能保证数据写入了硬盘，而且在空间占满时，`fclose` 之后如果没地方存储缓冲的数据，它们就可能被丢弃。

Warning

如果某一应用程序在占满的文件系统上行为异常，有必要检查下它调用了 `fsync()` 来确保数据已写入硬盘，然后再继续。

如果客户端运行时已经看到了 `OSD full` 标记，写数据操作可能被取消。各个客户端取消操作并释放文件能力时会更新 `osd_epoch_barrier`，以确保这些取消的操作不会妨碍 MDS 或其它客户端对这些数据对象的访问。要了解元图屏蔽机制，请参考 [Ceph 文件系统客户端的驱逐](#)。

老版本（HAMMER 之前）的行为

在 hammer 之前的 Ceph 版本中，MDS 会忽略 RADOS 集群的占满状态，而且客户端的任意数据写入都会卡死，直到集群脱离占满状态。

出现这种行为时有两种危险情形要注意：

- 如果一客户端到一个文件的写入未完成，那么它不可能把文件释放给 MDS 让它删除，这样就很难清理占满的文件系统。
- 如果客户端持续地创建大量空文件，来自 MDS 的元数据写入操作终将会耗尽 OSD 空间，这样就再也不能执行删除动作了。

故障排除

MOUNT 5 ERROR

mount 5 错误通常是 MDS 服务器滞后或崩溃导致的。要确保至少有一个 MDS 是启动且运行的，集群也要处于 `active+healthy` 状态。

MOUNT 12 ERROR

mount 12 错误显示 `cannot allocate memory`，常见于 *Ceph 客户端*和 *Ceph 存储集群*版本不匹配。用以下命令检查版本：

```
ceph -v
```

如果 Ceph 客户端版本落后于集群，试着升级它：

```
sudo apt-get update && sudo apt-get install ceph-common
```

你也许得卸载、清理和删除 `ceph-common`，然后再重新安装，以确保安装的是最新版。

灾难恢复

Danger

本章节是为专家准备的，尽可能地恢复损坏的文件系统。这些操作有可能改善你的处境，也可能更糟糕。如果你不太确定，最好别下手。

导出日志

尝试危险的操作前，先备份个日志副本，像这样：

```
cephfs-journal-tool journal export backup.bin
```

需要注意的是，此命令在日志损坏严重时也许会失效，在这种情况下，应该进行 RADOS 级的复制（<http://tracker.ceph.com/issues/9902>）。

从日志恢复 DENTRY

如果日志损坏、或因其它原因导致 MDS 不能重放它，可以这样尝试恢复文件元数据：

```
cephfs-journal-tool event recover_dentries summary
```

此命令默认会操作 rank 0 的 MDS，用 `-rank=<n>` 指定其它 rank。

在条件满足的情况下，此命令会把日志中可恢复的 inode/dentry 写入后端存储，比如这些 inode/dentry 的版本号高于后端存储中的版本。如果日志中的某一部分丢失或损坏，就会被跳过。

注意，除了写出 dentry 和 inode 之外，此命令还会更新各 MDS rank “内”的 InoTables，以把写入的 inode 标识为正在使用。在简单的案例中，此操作即可使后端存储回到完全正确的状态。

Warning

此操作不能保证后端存储的状态达到自我一致，而且在此之后有必要执行 MDS 在线洗刷。此命令不会更改日志内容，所以把能恢复的给恢复之后，应该分别截断日志。

日志截断

如果日志损坏或因故 MDS 不能重放它，你可以这样截断它：

```
cephfs-journal-tool journal reset
```

Warning

重置日志会导致元数据丢失，除非你已经用其它方法（如 `recover_dentries`）提取过了。此操作很可能会在数据存储池中留下一些孤儿对象，并导致已写过的索引节点被重分配，以致权限规则被破坏。

擦除 MDS 表

重置日志后，可能 MDS 表（`InoTable`、`SessionMap`、`SnapServer`）的内容就不再一致了。

要重置 `SessionMap`（擦掉所有会话），用此命令：

```
cephfs-table-tool all reset session
```

此命令会在所有 MDS rank “内”的表中执行。如果只想在指定 rank 中执行，把 `all` 换成对应的 MDS rank。

会话表是最有可能需要重置的表，但是如果你知道你还需要重置其它表，那就把 `session` 换成 `snap` 或者 `inode`。

MDS 图重置

一旦文件系统底层的 RADOS 状态（即元数据存储池的内容）恢复到一定程度，也许有必要更新 MDS 图以反映元数据存储池的内容。可以用下面的命令把 MDS 图重置到单个 MDS：

```
ceph fs reset <fs name> --yes-i-really-mean-it
```

运行此命令之后，MDS rank 保存在 RADOS 上的任何不为 0 的状态都会被忽略：因此这有可能导致数据丢失。

也许有人想知道 ‘`fs reset`’ 和 ‘`fs remove; fs new`’ 的不同。主要区别在于，执行删除、新建操作会使 rank 0 处于 `creating` 状态，那样会覆盖所有根索引节点、并使所有文件变成孤儿；相反，`reset` 命令会使 rank 0 处于 `active` 状态，这样下一个要认领此 rank 的 MDS 守护进程会继续、并使用已存在于 RADOS 中的数据。

元数据对象丢失的恢复

取决于丢失或被篡改的是哪种对象，你得运行几个命令生成这些对象的默认版本。

```
# 会话表

cephfs-table-tool 0 reset session

# SnapServer 快照服务器

cephfs-table-tool 0 reset snap

# InoTable 索引节点表

cephfs-table-tool 0 reset inode

# Journal 日志

cephfs-journal-tool --rank=0 journal reset

# 根索引节点 ( / 和所有 MDS 目录 )

cephfs-data-scan init
```

最后，根据数据存储池中的内容重新生成丢失文件和目录的元数据对象。这要分两步完成，首先，扫描所有对象以计算索引节点的尺寸和 mtime 元数据；其次，从每个文件的第一个对象扫描出元数据并注入元数据存储池。

```
cephfs-data-scan scan_extents <data pool>

cephfs-data-scan scan_inodes <data pool>
```

如果数据存储池内的文件很多、或者有很大的文件，这个命令就要花费很长时间。要加快处理，可以让这个工具多跑几个例程。先确定例程数量、再传递给每个例程一个数字 N，此数字应大于 0 且小于 (N - 1)，像这样：

```
# Worker 0

cephfs-data-scan scan_extents <data pool> 0 1

# Worker 1

cephfs-data-scan scan_extents <data pool> 1 1


# Worker 0

cephfs-data-scan scan_inodes <data pool> 0 1

# Worker 1

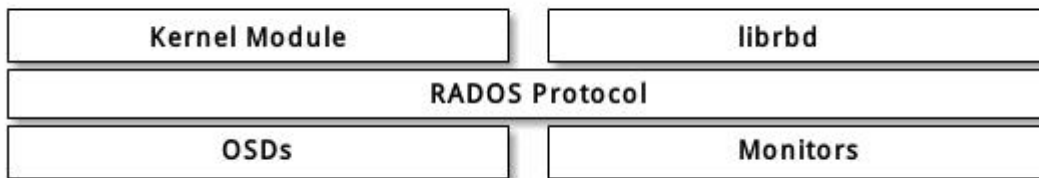
cephfs-data-scan scan_inodes <data pool> 1 1
```

切记！！所有运行 scan_extents 阶段的例程都结束后才能开始 scan_inodes 。

CEPH 块设备

块是一个字节序列（例如，一个 512 字节的数据块）。基于块的存储接口是最常见的存储数据方法，它们基于旋转介质，像硬盘、CD、软盘、甚至传统的 9 磁道磁带。无处不在的块设备接口使虚拟块设备成为与 Ceph 这样的海量存储系统交互的理想之选。

Ceph 块设备是精简配置的、大小可调且将数据条带化存储到集群内的多个 OSD。Ceph 块设备利用 RADOS 的多种能力，如快照、复制和一致性。Ceph 的 RADOS 块设备（RBD）使用内核模块或 librbd 库与 OSD 交互。



Note

内核模块可使用 Linux 页缓存。对基于 librbd 的应用程序，Ceph 可提供 [RBD 缓存](#)。

Ceph 块设备靠无限伸缩性提供了高性能，如向[内核模块](#)、或向 `abbr:KVM (kernel virtual machines)`（如 [Qemu](#)、[OpenStack](#) 和 [CloudStack](#) 等云计算系统通过 libvirt 和 Qemu 可与 Ceph 块设备集成）。你可以用同一个集群同时运行 [Ceph RADOS 网关](#)、[Ceph FS 文件系统](#)、和 Ceph 块设备。

Important

要使用 Ceph 块设备，你必须有一个在运行的 Ceph 集群。

- [命令](#)
- [内核模块](#)
- [快照](#)
- [RBD 镜像](#)
- [QEMU](#)
- [libvirt](#)
- [缓存选项](#)
- [OpenStack](#)
- [CloudStack](#)
- [rbd 手册页](#)
- [rbd-fuse 手册页](#)
- [rbd-nbd 手册页](#)
- [ceph-rbdnamer 手册页](#)
- [RBD 重放](#)

- [rbd-replay-prep 手册页](#)
- [rbd-replay 手册页](#)
- [rbd-replay-many 手册页](#)
- [librbd \(Python 接口\)](#)

块设备命令

`rbd` 命令可用于创建、罗列、内省和删除块设备映像，也可克隆映像、创建快照、回滚快照、查看快照等等。`rbd` 命令用法详情见 [RBD – 管理 RADOS 块设备映像](#)。

Important

要使用 Ceph 块设备命令，你必须有对应集群的访问权限。

创建块设备映像

要想把块设备加入某节点，你得先在 [Ceph 存储集群](#) 中创建一个映像，使用下列命令：

```
rbd create --size {megabytes} {pool-name}/{image-name}
```

例如，要在 `swimmingpool` 这个存储池中创建一个名为 `bar`、大小为 1GB 的映像，执行：

```
rbd create --size 1024 swimmingpool/bar
```

如果创建映像时不指定存储池，它将使用默认的 `rbd` 存储池。例如，下面的命令将默认在 `rbd` 存储池中创建一个大小为 1GB、名为 `foo` 的映像：

```
rbd create --size 1024 foo
```

Note

指定此存储池前必须先创建它，详情见[存储池](#)。

罗列块设备映像

要列出 `rbd` 存储池中的块设备，可以用下列命令（即 `rbd` 是默认存储池名字）：

```
rbd ls
```

用下列命令罗列某个特定存储池中的块设备，用存储池的名字替换 `{poolname}`：

```
rbd ls {poolname}
```

例如：

```
rbd ls swimmingpool
```

检索映像信息

用下列命令检索某个特定映像的信息，用映像名字替换 `{image-name}`：

```
rbd info {image-name}
```

例如：

```
rbd info foo
```

用下列命令检索某存储池内的映像的信息，用映像名字替换 `{image-name}`、用存储池名字替换 `{pool-name}`：

```
rbd info {pool-name}/{image-name}
```

例如：

```
rbd info swimmingpool/bar
```

调整块设备映像大小

Ceph 块设备映像是精简配置，只有在你开始写入数据时它们才会占用物理空间。然而，它们都有最大容量，就是你设置的 `--size` 选项。如果你想增加（或减小）Ceph 块设备映像的最大尺寸，执行下列命令：

```
rbd resize --size 2048 foo (to increase)

rbd resize --size 2048 foo --allow-shrink (to decrease)
```

删除块设备映像

可用下列命令删除块设备，用映像名字替换 `{image-name}`：

```
rbd rm {image-name}
```

例如：

```
rbd rm foo
```

用下列命令从某存储池中删除一个块设备，用要删除的映像名字替换 `{image-name}`、用存储池名字替换 `{pool-name}`：

```
rbd rm {pool-name}/{image-name}
```

例如：

```
rbd rm swimmingpool/bar
```

内核模块操作

Important

要用内核模块操作，必须有一个在运行的 Ceph 集群。

获取映像列表

要挂载块设备映像，先罗列出所有的映像。

```
rbd list
```

映射块设备

用 `rbd` 把映像名映射为内核模块。必须指定映像名、存储池名、和用户名。若 RBD 内核模块尚未加载，`rbd` 命令会自动加载。

```
sudo rbd map {pool-name}/{image-name} --id {user-name}
```

例如：

```
sudo rbd map rbd/myimage --id admin
```

如果你启用了 [cephx](#) 认证，还必须提供密钥，可以用密钥环或密钥文件指定密钥。

```
sudo rbd map rbd/myimage --id admin --keyring  
/path/to/keyring
```



```
sudo rbd map rbd/myimage --id admin --keyfile
/path/to/file
```

查看已映射块设备

可以用 `rbd` 命令的 `showmapped` 选项查看映射为内核模块的块设备映像。

```
rbd showmapped
```

取消块设备映射

要取消块设备映射，用 `rbd` 命令、指定 `unmap` 选项和设备名（即为方便起见使用的同名块设备映像）。

```
sudo rbd unmap /dev/rbd/{poolname}/{imagename}
```

例如：

```
sudo rbd unmap /dev/rbd/rbd/foo
```

快照

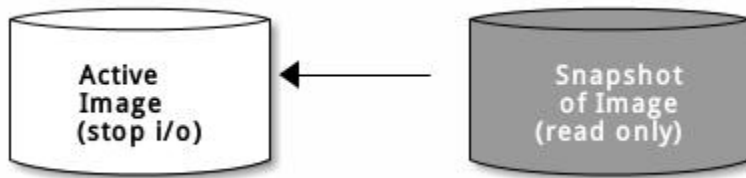
快照是映像在某一个特定时间点的一份只读副本。Ceph 块设备的一个高级特性就是你可以为映像创建快照来保留其历史。Ceph 还支持分层快照，让你快速、简便地克隆映像（如 VM 映像）。Ceph 的快照功能支持 `rbd` 命令和多种高级接口，包括 [QEMU](#)、[libvirt](#)、[OpenStack](#) 和 [CloudStack](#)。

Important

要使用 RBD 快照功能，你必须有一个在运行的 Ceph 集群。

Note

如果在做快照时映像仍在进行 I/O 操作，快照可能就获取不到该映像准确的或最新的数据，并且该快照可能不得不被克隆到一个新的可挂载的映像中。所以，我们建议在做快照前先停止 I/O 操作。如果映像内包含文件系统，在做快照前请确保文件系统处于一致的状态。要停止 I/O 操作可以使用 `fsfreeze` 命令。详情可参考 `fsfreeze(8)` 手册页。对于虚拟机，`qemu-guest-agent` 被用来在做快照时自动冻结文件系统。



CEPHX 注意事项

启用了 [cephx](#) 时（默认的），你必须指定用户名或 ID、及其对应的密钥文件，详情见[用户管理](#)。你也可以用 `CEPH_ARGS` 环境变量来避免重复输入下列参数。

```
rbd --id {user-ID} --keyring=/path/to/secret [commands]

rbd --name {username} --keyring=/path/to/secret [commands]
```

例如：

```
rbd --id admin --keyring=/etc/ceph/ceph.keyring [commands]

rbd --name client.admin --keyring=/etc/ceph/ceph.keyring
[commands]
```

Tip

把用户名和密钥写入 `CEPH_ARGS` 环境变量，这样就无需每次手动输入。

快照基础

下列过程演示了如何用 `rbd` 命令创建、罗列、和删除快照。

创建快照

用 `rbd` 命令创建快照，要指定 `snap create` 选项、存储池名和映像名。

```
rbd snap create {pool-name}/{image-name}@{snap-name}
```

例如：

```
rbd snap create rbd/foo@snapname
```

罗列快照

列出某个映像的快照，需要指定存储池名和映像名。

```
rbd snap ls {pool-name}/{image-name}
```

例如：

```
rbd snap ls rbd/foo
```

回滚快照

用 `rbd` 命令回滚到某一快照，指定 `snap rollback` 选项、存储池名、映像名和快照名。

```
rbd snap rollback {pool-name}/{image-name}@{snap-name}
```

例如：

```
rbd snap rollback rbd/foo@snapname
```

Note

把映像回滚到某一快照的意思是，用快照中的数据覆盖映像的当前版本，映像越大，此过程花费的时间就越长。从快照**克隆要快于回滚**到某快照，这也是回到先前状态的首选方法。

删除快照

要用 `rbd` 删除一快照，指定 `snap rm` 选项、存储池名、映像名和快照名。

```
rbd snap rm {pool-name}/{image-name}@{snap-name}
```

例如：

```
rbd snap rm rbd/foo@snapname
```

Note

Ceph OSDs 异步地删除数据，所以删除快照后不会立即释放磁盘空间。

清除快照

要用 `rbd` 删除某个映像的所有快照，指定 `snap purge` 选项、存储池名和映像名。

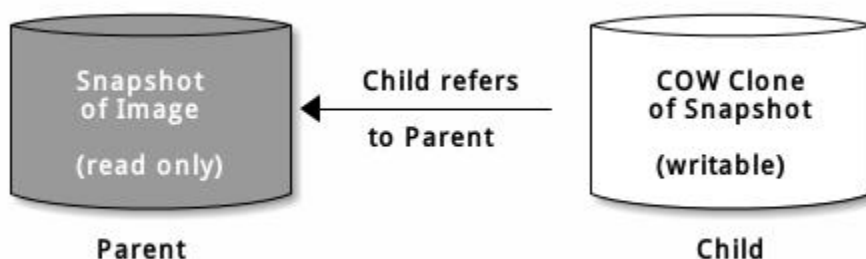
```
rbd snap purge {pool-name}/{image-name}
```

例如：

```
rbd snap purge rbd/foo
```

分层

Ceph 支持为某一设备快照创建很多个写时复制（COW）克隆。分层快照使得 Ceph 块设备客户端可以很快地创建映像。例如，你可以创建一个包含有 Linux VM 的块设备映像；然后做快照、保护快照，再创建任意多个写时复制克隆。快照是只读的，所以简化了克隆快照的语义——使得克隆很迅速。



Note

这里的术语“父”和“子”指的是一个 Ceph 块设备快照（父），和从此快照克隆出来的对应映像（子）。这些术语对下列的命令行用法来说很重要。

各个克隆出来的映像（子）都存储着对父映像的引用，这使得克隆出来的映像可以打开父映像并读取它。

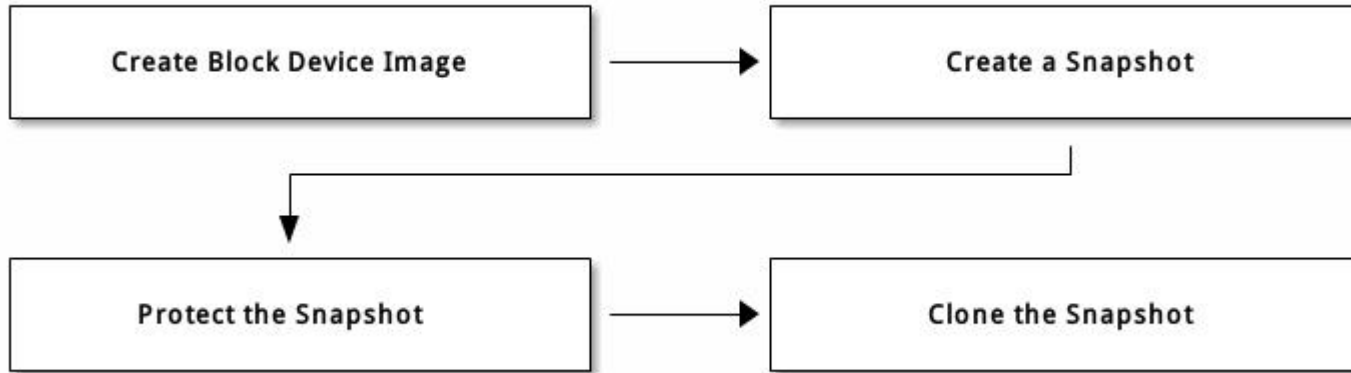
一个快照的 COW 克隆和其它任何 Ceph 块设备映像的行为完全一样。克隆出的映像没有特别的限制，你可以读出、写入、克隆、调整克隆映像的大小。然而快照的写时复制克隆引用了快照，所以你克隆快照前**必须**保护它。下图描述了此过程。

Note

Ceph 仅支持克隆 format 2 的映像（即用 `rbd create --image-format 2` 创建的）。内核客户端从 3.10 版开始支持克隆的映像。

分层入门

Ceph 块设备的分层是个简单的过程。你必须有个映像、必须为它创建快照、并且必须保护快照，执行过这些步骤后，你才能克隆快照。



克隆出的映像包含对父快照的引用，也包含存储池 ID、映像 ID 和快照 ID。包含存储池 ID 意味着你可以把一个存储池内的快照克隆到其他存储池。

1. **映像模板：** 块设备分层的一个常见用法是创建一个主映像及其快照，并作为模板以供克隆。例如，用户可以创建某一 Linux 发行版（如 Ubuntu 12.04）的映像、并对其做快照。此用户可能会周期性地更新映像、并创建新的快照（如在 `rbd snap create` 之后执行 `sudo apt-get update`、`sudo apt-get upgrade`、`sudo apt-get dist-upgrade`）。当映像成熟时，用户可以克隆任意快照。
2. **扩展模板：** 更高级的用法包括扩展映像模板，来提供比基础映像更多的信息。例如，用户可以克隆一个映像（如 VM 模板）、并安装其它软件（如数据库、内容管理系统、分析系统等等），然后为此扩展映像做快照，做好的快照可以像基础映像一样进行更新。
3. **模板存储池：** 块设备分层的一种用法是创建一个存储池，存放作为模板的主映像和那些模板的快照。然后把只读权限分给用户，这样他们就可以克隆快照了，而无需分配此存储池的写和执行权限。
4. **映像迁移/恢复：** 块设备分层的一种用法是把某一存储池内的数据迁移或恢复到另一存储池。

保护快照

克隆映像要访问父快照。如果用户不小心删除了父快照，所有克隆映像都会损坏。为防止数据丢失，在克隆前**必须**先保护快照。

```
rbd snap protect {pool-name}/{image-name}@{snapshot-name}
```

例如：

```
rbd snap protect rbd/my-image@my-snapshot
```

Note

你删除不了受保护的快照。

克隆快照

要克隆快照，你得指定父存储池、父映像名和快照，还有子存储池和子映像名。克隆前必须先保护快照。

```
rbd clone {pool-name}/{parent-image}@{snap-name} {pool-name}/{child-image-name}
```

例如：

```
rbd clone rbd/my-image@my-snapshot rbd/new-image
```

Note

你可以把某个存储池中映像的快照克隆到另一存储池。例如，你可以把某一存储池中的只读映像及其快照作为模板维护，把可写克隆置于另一存储池。

取消快照保护

删除快照前，必须先取消保护。另外，你不可以删除被克隆映像引用的快照，所以在你删除快照前，必须先拍平（flatten）此快照的各个克隆。

```
rbd snap unprotect {pool-name}/{image-name}@{snapshot-name}
```

例如：

```
rbd snap unprotect rbd/my-image@my-snapshot
```

罗列快照的子孙

用下列命令罗列某个快照的子孙：

```
rbd children {pool-name}/{image-name}@{snapshot-name}
```

例如：

```
rbd children rbd/my-image@my-snapshot
```

拍平克隆映像

克隆出来的映像仍保留了对父快照的引用。要从子克隆删除这些到父快照的引用，你可以把快照的信息复制给子克隆，也就是“拍平”它。拍平克隆映像的时间随快照尺寸增大而增加。要删除快照，必须先拍平子映像。

```
rbd flatten {pool-name}/{image-name}
```

例如：


```
rbd flatten rbd/my-image
```

Note

因为拍平的映像包含了快照的所有信息，所以拍平的映像占用的存储空间会比分层克隆要大。

RBD 镜像

可以在两个 Ceph 集群中异步备份 RBD images。该能力利用了 RBD image 的日志特性，以确保集群间的副本崩溃一致性。镜像功能需要在同伴集群（peer clusters）中的每一个对应的 pool 上进行配置，可设定自动备份某个存储池内的所有 images 或仅备份 images 的一个特定子集。用 `rbd` 命令来配置镜像功能。`rbd-mirror` 守护进程负责从远端集群拉取 image 的更新，并写入本地集群的对应 image 中。

Note

RBD 镜像功能需要 Ceph Jewel 或更新的发行版本。

Important

要使用 RBD 镜像功能，你必须有 2 个 Ceph 集群，每个集群都要运行 `rbd-mirror` 守护进程。

存储池配置

下面的程序说明了如何执行一些基本的管理工作，来用 `rbd` 命令配置镜像功能。镜像功能是在 Ceph 集群内的存储池级别上配置的。

配置存储池的步骤需要在 2 个同伴集群内都执行一遍。为清楚起见，下面的步骤假定这两个集群分别叫做“本地（local）”和“远端（remote）”，而且单主机对这 2 个集群都拥有访问权。

如何连接不同的 Ceph 集群，详情可参考 [rbd](#) 手册页。

Note

在下面的例子中，集群名称和 Ceph 配置文件的名称相同（比如 `/etc/ceph/remote.conf`）。可参考 [ceph-conf](#) 文档来配置多集群环境。

启用镜像功能

使用 `rbd` 启用某个存储池的镜像功能，需要指定 `mirror pool enable` 命令，存储池名和镜像模式：

```
rbd mirror pool enable {pool-name} {mode}
```

镜像模式可以是 `pool` 或 `image`：

- **pool**：当设定为 `pool` 模式，存储池中所有开启了日志特性的 `images` 都会被备份。
- **image**：当设定为 `image` 模式，需要对每个 `image` [显式启用](#) 镜像功能。

例如：

```
rbd --cluster local mirror pool enable image-pool pool  
rbd --cluster remote mirror pool enable image-pool pool
```

禁用镜像功能

使用 `rbd` 禁用某个存储池的镜像功能，需要指定 `mirror pool disable` 命令和存储池名：

```
rbd mirror pool disable {pool-name}
```

当采用这种方式禁用某个存储池的镜像功能时，存储池内的任一个 `image` 的镜像功能也会被禁用，即使曾显式启用过。

例如：

```
rbd --cluster local mirror pool disable image-pool  
  
rbd --cluster remote mirror pool disable image-pool
```

添加同伴集群

为了使 `rbd-mirror` 守护进程发现它的同伴集群，需要向存储池注册。使用 `rbd` 添加同伴 Ceph 集群，需要指定 `mirror pool peer add` 命令、存储池名和集群说明：

```
rbd mirror pool peer add {pool-name} {client-  
name}@{cluster-name}
```

例如：

```
rbd --cluster local mirror pool peer add image-pool  
client.remote@remote  
  
rbd --cluster remote mirror pool peer add image-pool  
client.local@local
```

移除同伴集群

使用 `rbd` 移除同伴 Ceph 集群，指定 `mirror pool peer remove` 命令、存储池名和同伴的 UUID（可通过 `rbd mirror pool info` 命令获取）：

```
rbd mirror pool peer remove {pool-name} {peer-uuid}
```

例如：

```
rbd --cluster local mirror pool peer remove image-pool  
55672766-c02b-4729-8567-f13a66893445
```

```
rbd --cluster remote mirror pool peer remove image-pool
60c0e299-b38f-4234-91f6-eed0a367be08
```

IMAGE 配置

不同于存储池配置，image 配置只需针对单个 Ceph 集群操作。

镜像 RBD image 被指定为主镜像或者副镜像。这是 image 而非存储池的特性。被指定为副镜像的 image 不能被修改。

当一个 image 首次启用镜像功能时（存储池的镜像模式设为 **pool** 且启用了该 image 的日志特性，或者通过 `rbd` 命令**显式启用**），它会自动晋升为主镜像。

启用 IMAGE 的日志支持

RBD 镜像功能使用了 RBD 日志特性，来保证 image 副本间的崩溃一致性。在备份 image 到另一个同伴集群前，必须启用日志特性。该特性可在使用 `rbd` 命令创建 image 时通过指定 `-image-feature exclusive-lock,journaling` 选项来启用。

或者，可以动态启用已有 image 的日志特性。使用 `rbd` 开启日志特性，需要指定 `feature enable` 命令，存储池名，image 名和特性名：

```
rbd feature enable {pool-name}/{image-name} {feature-name}
```

例如：

```
rbd --cluster local feature enable image-pool/image-1
journaling
```

Note

日志特性依赖于独占锁（`exclusive-lock`）特性。如果没有启用独占锁，则必须在启用日志特性之前先启用独占锁。

Tip

你可以通过在 Ceph 配置文件中增加 `rbd default features = 125` , 使得所有新建 image 默认启用日志特性。

启用 IMAGE 镜像功能

如果把某个存储池的镜像功能配置为 `image` 模式, 还需要对存储池中的每一个 image , 明确启用镜像功能。通过 `rbd` 启用某个特定 image 的镜像功能, 要指定 `mirror image enable` 命令、存储池名和 image 名 :

```
rbd mirror image enable {pool-name}/{image-name}
```

例如 :

```
rbd --cluster local mirror image enable image-pool/image-1
```

禁用 IMAGE 镜像功能

通过 `rbd` 禁用某个特定 image 的镜像功能, 要指定 `mirror image disable` 命令、存储池名和 image 名 :

```
rbd mirror image disable {pool-name}/{image-name}
```

例如 :

```
rbd --cluster local mirror image disable image-pool/image-1
```

IMAGE 的升级与降级

在需要把主名称转移到同伴 Ceph 集群这样一个故障切换场景中，应该停止所有对主 image 的访问（比如关闭 VM 的电源或移除 VM 的相关驱动），当前的主 image 降级为副，原副 image 升级为主，然后在备份集群上恢复对该 image 访问。

Note

RBD 仅提供了一些必要的工具来帮助 image 有序的故障切换。还需要一种外部机制来协调整个故障切换流程（比如在降级之前关闭 image）。

通过 `rbd` 降级主 image，需要指定 `mirror image demote` 命令、存储池名和 image 名：

```
rbd mirror image demote {pool-name}/{image-name}
```

例如：

```
rbd --cluster local mirror image demote image-pool/image-1
```

通过 `rbd` 升级副 image，需要指定 `mirror image promote` 命令、存储池名和 image 名：

```
rbd mirror image promote {pool-name}/{image-name}
```

例如：

```
rbd --cluster remote mirror image promote image-pool/image-1
```

Tip

由于主 / 副状态是对于每个 image 而言的，故可以让两个集群拆分 IO 负载来进行故障切换 / 故障自动恢复。

Note

可以使用 `--force` 选项来强制升级。当降级要求不能被正确传播到同伴 Ceph 集群的时候（比如 Ceph 集群故障，通信中断），就需要强制升级。这会导致两个集群间的脑裂，而且在调用[强制重新同步命令](#)之前，image 将不会自动同步。

强制 IMAGE 重新同步

如果 `rbd-daemon` 探测到了脑裂事件，它在此情况得到纠正之前，是不会尝试去备份受到影响的 image。为了恢复对 image 的镜像备份，首先判定[降级 image](#) 已经过时，然后向主 image 请求重新同步。通过 `rbd` 重新同步 image，需要指定 `mirror image resync` 命令、存储池名和 image 名：

```
rbd mirror image resync {pool-name}/{image-name}
```

例如：

```
rbd mirror image resync image-pool/image-1
```

Note

此条 `rbd` 命令仅标记了某 image 需要重新同步。本地集群的 `rbd-mirror` 守护进程会异步实施真正的重新同步过程。

RBD-MIRROR 守护进程

有两个 `rbd-mirror` 守护进程负责监控远端同伴集群的 image 日志，并针对本地集群进行日志重放。RBD image 日志特性会按发生的顺序记录下对该 image 的所有修改。这保证了远端 image 的崩溃一致性镜像在本地是可用的。

通过安装可选发行包 `rbdmirror` 来获取 `rbdmirror` 守护进程。

Important

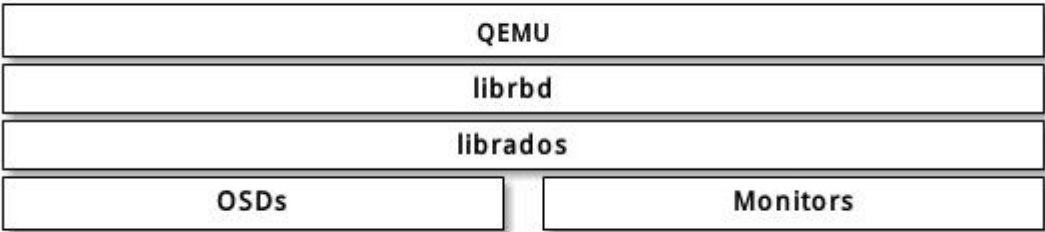
每一个 `rbdmirror` 守护进程需要同时连接本地和远程集群。

Warning

每个 Ceph 集群只能运行一个 `rbdmirror` 守护进程。将来的 Ceph 发行版将会支持对 `rbdmirror` 守护进程进行水平扩展。

QEMU 与块设备

Ceph 块设备最常见的用法之一是作为虚拟机的块设备映像。例如，用户可创建一个安装、配置好了操作系统和相关软件的“黄金标准”映像，然后对此映像做快照，最后再克隆此快照（通常很多次）。详情参见[快照](#)。能制作快照的写时复制克隆意味着 Ceph 可以快速地为虚拟机提供块设备映像，因为客户端每次启动一个新虚拟机时不必下载整个映像。



Ceph 块设备可以和 QEMU 虚拟机集成到一起，关于 QEMU 可参考 [QEMU 开源处理器仿真器](#)，其文档可参考 [QEMU 手册](#)。关于如何安装见[安装](#)。

Important

要让 QEMU 使用 Ceph 块设备，你必须有个运行着的 Ceph 集群。

用法

QEMU 命令行要求你指定 存储池名和映像名，还可以指定快照名。

QEMU 会假设 Ceph 配置文件位于默认位置（如 `/etc/ceph/$cluster.conf`），并且你是以默认的 `client.admin` 用户执行命令，除非你另外指定了其它 Ceph 配置文件路径或用户。指定用户时，QEMU 只需要 ID 部分，无需完整地指定 `TYPE:ID`，详情见‘[用户管理——用户`\\_](#)’。不要在用户 ID 前面加客户端类型（即 `client.`），否则会认证失败。还应该把 `admin` 用户、或者你用 `:id={user}` 选项所指定用户的密钥文件保存到默认路径（即 `/etc/ceph`）或本地目录内，并修正密钥环文件的所有权和权限。命令格式如下：

```
qemu-img {command} [options] rbd:{pool-name}/{image-name}[@snapshot-name][:option1=value1][:option2=value2...]
```

例如，应该这样指定 `id` 和 `conf` 选项：

```
qemu-img {command} [options] rbd:glance-pool/maipo:id=glance:conf=/etc/ceph/ceph.conf
```

Tip

配置中的值如果包含这些字符：`:`、`@`、`=`，可在此符号前加反斜线 `\` 转义。

用 QEMU 创建映像

你可以用 QEMU 创建块设备映像。必须指定 `rbd`、存储池名、要创建的映像名以及映像尺寸。

```
qemu-img create -f raw rbd:{pool-name}/{image-name} {size}
```

例如：

```
qemu-img create -f raw rbd:data/foo 10G
```

Important

`raw` 数据格式是使用 RBD 时的唯一可用 `format` 选项。从技术上讲，你可以使用 QEMU 支持的其他格式（例如 `qcow2` 或 `vmdk`），但是这样做可能会带来额外开销，而且在开启缓存（见下）模式下进行虚拟机的热迁移时会导致卷的不安全性。

用 QEMU 更改映像尺寸

你可以通过 QEMU 调整块设备大小。必须指定 `rbd`、存储池名、要调整的映像名，还有映像尺寸。

```
qemu-img resize rbd:{pool-name}/{image-name} {size}
```

例如：

```
qemu-img resize rbd:data/foo 10G
```

用 QEMU 检索映像信息

你可以用 QEMU 检索块设备映像信息。必须指定 `rbd`、存储池名和映像名。

```
qemu-img info rbd:{pool-name}/{image-name}
```

例如：

```
qemu-img info rbd:data/foo
```

通过 RBD 运行 QEMU

QEMU 能把一主机上的块设备传递给客户机，但从 QEMU 0.15 起，不需要在主机上把映像映射为块设备了。QEMU 现在能通过 `librbd` 直接把映像作为虚拟块设备访问。这样性能更好，因为它避免了额外的上下文切换，而且能利用开启 [RBD 缓存](#) 带来的好处。

你可以用 `qemu-img` 把已有的虚拟机映像转换为 Ceph 块设备映像。比如你有一个 `qcow2` 映像，可以这样转换：

```
qemu-img convert -f qcow2 -O raw debian_squeeze.qcow2
rbd:data/squeeze
```

要从那个映像启动虚拟机，执行：

```
qemu -m 1024 -drive format=raw,file=rbd:data/squeeze
```

启用 [RBD 缓存](#) 可显著提升性能。从 QEMU 1.2 起，QEMU 的缓存选项可控制 `librbd` 缓存：

```
qemu -m 1024 -drive
format=rbd,file=rbd:data/squeeze,cache=writeback
```

如果你的 QEMU 版本较老，你可以用 ‘file’ 参数更改 `librbd` 缓存配置（就像其它 Ceph 配置选项一样）：

```
qemu -m 1024 -drive
format=raw,file=rbd:data/squeeze:rbd_cache=true,cache=writeback
```

Important

如果你设置了 `rbd_cache=true`，那就必须设置 `cache=writeback`，否则有可能丢失数据。不设置 `cache=writeback`，QEMU 就不会向 `librbd` 发送回写请求。如果 QEMU 退出时未清理干净，`rbd` 之上的文件系统就有可能崩溃。

启用 DISCARD/TRIM 功能

从 Ceph 0.46 和 QEMU 1.1 起，Ceph 块设备支持 discard 操作。这意味着客户机可以发送 TRIM 请求来让 Ceph 块设备回收未使用的空间。此功能可在客户机上挂载 `ext4` 或 `XFS` 时加上 `discard` 选项。

客户机要想使用此功能，必须对块设备显式启用。为此，你必须在相关驱动器上指定 `discard_granularity`：

```
qemu -m 1024 -drive
format=raw,file=rbd:data/squeeze,id=drive1,if=none \

        -device driver=ide-
hd,drive=drive1,discard_granularity=512
```

注意此处使用 IDE 驱动器，virtio 驱动不支持 discard。

如果用的是 libvirt，需要用 `virsh edit` 编辑配置文件，加上 `xmlns:qemu` 值。然后加一个 `qemu:commandline` 块作为那个域的子域。下例展示了如何用 `qemu id=` 为两个设备设置不同的 `discard_granularity` 值。

```
<domain type='kvm'
xmlns:qemu='http://libvirt.org/schemas/domain/qemu/1.0'>

    <qemu:commandline>

        <qemu:arg value='-set' />

        <qemu:arg value='block.scsi0-0-
0.discard_granularity=4096' />

        <qemu:arg value='-set' />

        <qemu:arg value='block.scsi0-0-
1.discard_granularity=65536' />

    </qemu:commandline>
```

```
</domain>
```

QEMU 缓存选项

QEMU 的缓存选项对应下列的 Ceph **RBD 缓存** 选项。

回写：

```
rbd_cache = true
```

透写：

```
rbd_cache = true  
  
rbd_cache_max_dirty = 0
```

无：

```
rbd_cache = false
```

QEMU 的缓存选项会覆盖 Ceph 的默认选项（就是那些 Ceph 配置文件里没有的选项）。如果你在 Ceph 配置文件内设置了 **RBD 缓存** 选项，那么它们会覆盖 QEMU 缓存选项。如果你在 QEMU 命令行中设置了缓存选项，它们则会覆盖 Ceph 配置文件里的选项。

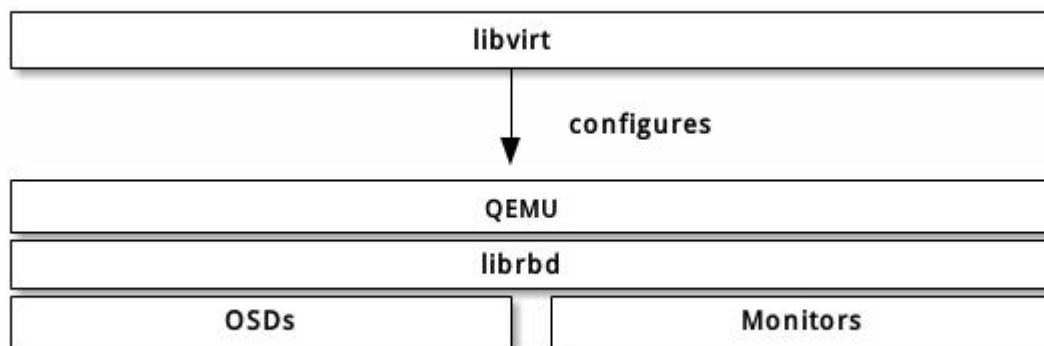
通过 LIBVIRT 使用 CEPH RBD

libvirt 库是管理程序和软件应用间的一个虚拟机抽象层。通过 libvirt，开发者和系统管理员只需要关注这些管理器的一个通用管理框架、通用 API、和通用 shell 接口（即 virsh）即可，包括：

- QEMU/KVM
- XEN

- LXC
- VirtualBox
- 等等

Ceph 块设备支持 QEMU/KVM，所以您可以通过能与 `libvirt` 交互的软件来使用 Ceph 块设备。下面的堆栈图解释了 `libvirt` 和 QEMU 如何通过 `librbd` 使用 Ceph 块设备。



`libvirt` 常见于为云解决方案提供 Ceph 块设备，像 OpenStack、CloudStack。它们用 `libvirt` 和 QEMU/KVM 交互、QEMU/KVM 再通过 `librbd` 与 Ceph 块设备交互。详情见[块设备与 OpenStack](#)和[块设备与 CloudStack](#)。关于如何安装见[安装](#)。

你也可以通过 `libvirt`、`virsh` 和 `libvirt` API 使用 Ceph 块设备，详情见[libvirt 虚拟化 API](#)。

要创建使用 Ceph 块设备的虚拟机，请参照下文中的步骤。在示范性实施例中，我们用 `libvirt-pool` 作为存储池名、`client.libvirt` 作为用户名、`new-libvirt-image` 作为映像名。你可以任意命名，但请确保在后续过程中用自己的名字替换掉对应名字。

配置 CEPH

配置 Ceph 用于 `libvirt`，执行下列步骤：

1. [创建——存储池](#)（或者用默认的）。本例用 `libvirt-pool` 作存储池名，设定了 128 个归置组。

```
ceph osd pool create libvirt-pool 128 128
```

验证存储池是否存在。

```
ceph osd lspools
```

2. [创建——Ceph 用户](#) (0.9.7 及之前的版本用 `client.admin`) , 本例用 `client.libvirt`、且权限限制到 `libvirt-pool`。

```
ceph auth get-or-create client.libvirt mon 'allow r'
osd 'allow class-read object_prefix rbd_children,
allow rwx pool=libvirt-pool'
```

验证名字是否存在。

```
ceph auth list
```

注：libvirt 访问 Ceph 时将用 libvirt 作为 ID，而不是 `client.libvirt`。关于 ID 和名字不同之处的详细解释请参考[用户管理——用户](#)和[用户管理——命令行界面](#)。

3. 用 QEMU 在 RBD 存储池中[创建映像](#)。本例中映像名为 `new-libvirt-image`、存储池为 `libvirt-pool`。

```
qemu-img create -f rbd rbd:libvirt-pool/new-libvirt-
image 2G
```

验证映像是否存在。

```
rbd -p libvirt-pool ls
```

注：你也可以用 `rbd create` 创建映像，但我们建议使用示例中的命令，来顺便确认 QEMU 可正常运行。

准备虚拟机管理器

即使没 VM 管理器你也可以用 `libvirt`，但是用 `virt-manager` 创建域更简单。

1. 安装虚拟机管理器，详情见 [KVM/VirtManager](#)。

```
sudo apt-get install virt-manager
```

2. 下载一个 OS 映像（如有必要）。
3. 启动虚拟机管理器。

```
sudo virt-manager
```

新建虚拟机

要用 `virt-manager` 创建 VM，请按下列步骤：

1. 点击 **Create New Virtual Machine** 按钮。
2. 命名新虚拟机的域，本例中我们用 `libvirt-virtual-machine`。你可以任意命名，但请在后续命令行和配置实例中替换 `libvirt-virtual-machine` 为你自己选择的名称。

```
3. libvirt-virtual-machine
```

4. 导入映像。

```
/path/to/image/recent-linux.img
```

注：请导入一个较新的映像。一些较老的映像未必能正确地重扫虚拟设备。

5. 配置并启动 VM。
6. 可以用 `virsh list` 验证 VM 域是否存在。

```
sudo virsh list
```

7. 登入 VM (root/root)
8. 在修改配置让它使用 Ceph 前，请先停止 VM。

配置 VM

配置 VM 使用 Ceph 时，切记尽量用 `virsh`。另外，`virsh` 命令通常需要 root 权限（如 `sudo`），否则不会返回正确结果或提示你需要 root 权限。`virsh` 命令参考见 [Virsh 命令参考](#)。

1. 用 `virsh edit` 打开配置文件。

```
sudo virsh edit {vm-domain-name}
```

`<devices>` 下应该有 `<disk>` 条目。

```
<devices>

    <emulator>/usr/bin/kvm</emulator>

    <disk type='file' device='disk'>

        <driver name='qemu' type='raw'/>

        <source
file='/path/to/image/recent-linux.img'/>

        <target dev='vda' bus='virtio'/>

        <address type='drive'
controller='0' bus='0' unit='0'/>
```

```
</disk>
```

用你的 OS 映像路径替换 `/path/to/image/recent-linux.img`。使用较快的 `virtio` 总线的最低内核版本是 2.6.25，参见 [Virtio](#)。

重要：要用 `sudo virsh edit` 而非文本编辑器，如果你用文本编辑器编辑了 `/etc/libvirt/qemu` 下的配置文件，`libvirt` 未必能识别出这些更改。如果 `/etc/libvirt/qemu` 下的 XML 文件和 `sudo virsh dumpxml {vm-domain-name}` 输出结果内容不同，VM 可能会运行异常。

2. 把你创建的 Ceph RBD 映像创建为 `<disk>` 条目。

```
3. <disk type='network' device='disk'>
4.     <source protocol='rbd' name='libvirt-
      pool/new-libvirt-image'>
5.         <host name='{monitor-host}'
      port='6789'>/>
6.     </source>
7.     <target dev='vda' bus='virtio'>/>
```

```
</disk>
```

用你的主机名替换 `{monitor-host}`，可能还要替换存储池和/或映像名。你可以为 Ceph monitor 添加多条 `<host>`，`dev` 属性是将出现在 VM/`dev` 目录下的逻辑设备名。可选的 `bus` 属性是要模拟的磁盘类型，有效的设定值是驱动类型，如 `ide`、`scsi`、`virtio`、`xen`、`usb` 或 `sata`。

关于 `<disk>` 标签及其子标签和属性，详见[硬盘](#)。

8. 保存文件。

9. 如果你的 Ceph 存储集群启用了 [Ceph 认证](#)（默认已启用），那么必须生成一个 secret。

```
10.      cat > secret.xml <<EOF
11.      <secret ephemeral='no' private='no'>
12.          <usage type='ceph'>
13.              <name>client.libvirt
secret</name>
14.          </usage>
15.      </secret>
```

```
EOF
```

16. 定义 secret。

```
17.      sudo virsh secret-define --file secret.xml
```

```
<uuid of secret is output here>
```

18. 获取 `client.libvirt` 密钥并把字符串保存于文件。

```
ceph auth get-key client.libvirt | sudo tee
client.libvirt.key
```

19. 设置 secret 的 UUID。

```
sudo virsh secret-set-value --secret {uuid of secret}  
--base64 $(cat client.libvirt.key) && rm  
client.libvirt.key secret.xml
```

还必须手动设置 secret，把下面的 `<auth>` 条目添加到前面的 `<disk>` 标签内（用上一命令的输出结果替换掉 `uuid` 值）。

```
sudo virsh edit {vm-domain-name}
```

然后，把 `<auth></auth>` 标签加进域配置文件：

```
...  
  
</source>  
  
<auth username='libvirt'>  
  
    <secret type='ceph' uuid='9ec59067-fdbc-  
a6c0-03ff-df165c0587b8' />  
  
</auth>  
  
<target ...
```

注：示例 ID 是 `libvirt`，不是第 2 步配置 Ceph 生成的 Ceph 名 `client.libvirt`。确保你用的是 Ceph 名的 ID 部分。如果出于某些原因你需要更换 secret，必须先执行 `sudo virsh secret-undefine {uuid}`，然后再执行 `sudo virsh secret-set-value`。

总结

一旦完成上面的配置，你就可以启动 VM 了。为确认 VM 和 Ceph 在通信，你可以执行如下过程。

1. 检查 Ceph 是否在运行：

```
ceph health
```

2. 检查 VM 是否在运行。

```
sudo virsh list
```

3. 检查 VM 是否在和 Ceph 通信，用你的 VM 域名字替换 {vm-domain-name}：

```
sudo virsh qemu-monitor-command --hmp {vm-domain-name} 'info block'
```

4. 检查一下 `<target dev='hdb' bus='ide' />` 定义的设备是否出现在 `/dev` 或 `/proc/partitions` 里。

```
5. ls dev
```

```
cat proc/partitions
```

如果看起来一切正常，你就可以在虚拟机内使用 Ceph 块设备了。

LIBRBD 选项

详情见[块设备](#)。

缓存选项

内核缓存

Ceph 块设备的内核驱动可利用 Linux 页缓存来提升性能。

Ceph 块设备的用户空间实现（即 `librbd`）不能利用 Linux 页缓存，所以它自己实现了内存缓存，名为“RBD 缓存”。RBD 缓存行为就像硬盘缓存一样端正。当 OS 发送了 barrier 或 flush 请求时，所有脏数据都会写入 OSDs。这意味着只要 VM 会正确地发送 flush 命令（即内核版本大于 2.6.32），使用回写缓存和常见物理硬盘一样安全。此缓存用最近最少使用（Least Recently Used, LRU）算法，而且在回写模式下它能合并相邻请求以提高吞吐量。

New in version 0.46.

Ceph 支持为 RBD 提供回写缓存。要启用此功能，在 `ceph.conf` 配置文件的 `[client]` 段下添加 `rbd cache = true`。`librbd` 默认不会进行任何缓存。写和读都直接到达存储集群，而且只有当数据的所有副本都写入磁盘后写操作才会返回。启用缓存后，写操作会立即返回，除非未刷写的字节数大于 `rbd cache max dirty`。这种情况下，写动作会触发回写机制并一直阻塞，直到足够多的字节数被刷写。

New in version 0.47.

Ceph 支持为 RBD 提供透写缓存。你可以设置缓存的大小，以及从回写缓存切换到透写缓存的目标和临界值。要启用透写模式，把 `rbd cache maxdirty` 置为 0。这意味着只有当数据的所有副本都写入磁盘后写操作才会返回，但可能会直接从缓存中读取数据。缓存信息保存在客户端的内存中，每个 RBD 映像有其自己的缓存。对客户端来说缓存位于本地，所以如果有其他用户访问某个映像，不会存在缓存一致性的问题。打开缓存时，在 RBD 之上不能运行 GFS 或 OCFS。

RBD 选项应该位于 `ceph.conf` 配置文件的 `[client]` 段下，可用选项有：

`rbd cache`

描述: 允许为 RADOS 块设备（RBD）提供缓存。

类型: Boolean

是否必需: No

默认值: `true`

`rbd cache size`

描述: RBD 缓存大小，单位是字节。

类型: 64-bit Integer

是否必需: No

默认值: `32 MiB`

`rbd cache max dirty`

描述: 使缓存触发回写的 `dirty` 临界值，单位是字节。若为 `0`，直接使用透写缓存。

类型: 64-bit Integer

是否必需: No

约束条件: 必须小于 `rbd cache size`。

默认值: 24 MiB

`rbd cache target dirty`

描述: 缓存开始向数据存储写回数据前的 `dirty target`。不会阻塞到缓存的写动作。

类型: 64-bit Integer

是否必需: No

约束条件: 必须小于 `rbd cache max dirty`.

默认值: 16 MiB

`rbd cache max dirty age`

描述: 写回开始前，脏数据在缓存中的暂存时间。

类型: Float

是否必需: No

默认值: 1.0

New in version 0.60.

`rbd cache writethrough until flush`

描述: 开始进入透写模式，并且在收到首个 **flush** 请求后切换到回写模式。启用该项保守但安全，以防 **rbd** 之上的虚拟机内核太老而不能发送 **flush**，例如 Linux 2.6.32 之前的 **virtio** 驱动。

类型: Boolean

是否必需: No

默认值: true

预读选项

New in version 0.86.

RBD 支持预读或预取功能，以此优化小块的顺序读。此功能通常应该由访客操作系统（在虚拟机的情况下）处理，但是引导加载程序可能不会发出高效的读请求。如果停用缓存功能，预读也会自动被禁用。

`rbd readahead trigger requests`

描述: 触发预读的顺序读请求数量。

类型: Integer

是否必需: No

默认值: 10

`rbd readahead max bytes`

描述: 预读请求的最大尺寸。如果是零，禁用预读。

类型: 64-bit Integer

是否必需: No

默认值: 512 KiB

`rbd readahead disable after bytes`

描述: 从 RBD 映像读取这么多字节后，预读功能将被禁用，直到关闭。这样访客操作系统一旦启动后就可以接管预读了。设为 0 时仍开启预读。

类型: 64-bit Integer

是否 No

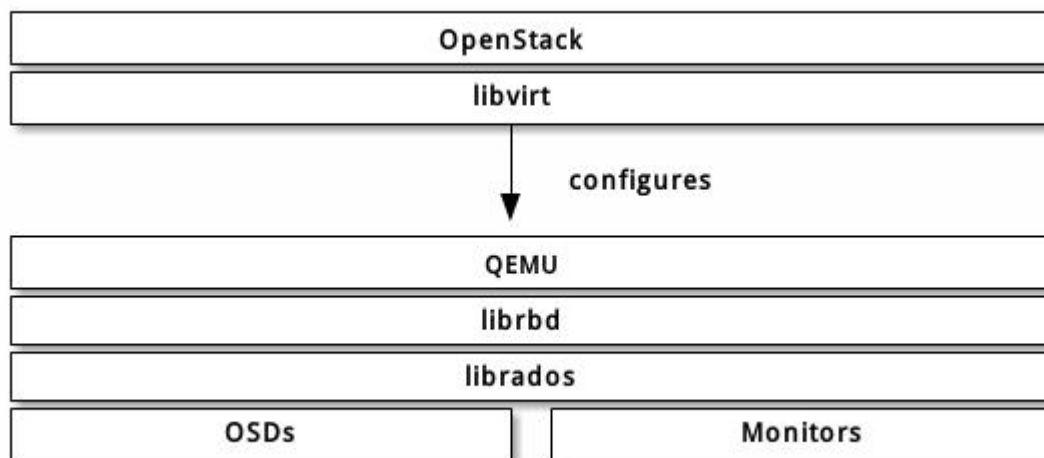
必需:

默认
值: 50 MiB

块设备与 OPENSTACK

通过 `libvirt` 你可以把 Ceph 块设备用于 OpenStack，它配置了 QEMU 到 `librbd` 的接口。Ceph 把块设备映像条带化为对象并分布到集群中，这意味着大容量的 Ceph 块设备映像其性能会比独立服务器更好。

要把 Ceph 块设备用于 OpenStack，必须先安装 QEMU、`libvirt` 和 OpenStack。我们建议用一台独立的物理主机安装 OpenStack，此主机最少需 8GB 内存和一个 4 核 CPU。下面的图表描述了 OpenStack/Ceph 技术栈。



Important

要让 OpenStack 使用 Ceph 块设备，你必须有相应的 Ceph 集群访问权限。

OpenStack 里有三个地方可以和 Ceph 块设备结合：

- **Images:** OpenStack 的 Glance 管理着 VM 的 image 。Image 相对恒定，OpenStack 把它们当作二进制文件、并以此格式下载。
- **Volumes:** Volume 是块设备， OpenStack 用它们引导虚拟机、或挂载到运行中的虚拟机上。 OpenStack 用 Cinder 服务管理 Volumes 。
- **Guest Disks:** Guest disks 是装有客户操作系统的磁盘。默认情况下，启动一台虚拟机时，它的系统盘表现为 hypervisor 文件系统的一个文件（通常位于 `/var/lib/nova/instances/<uuid>/`）。在 Openstack Havana 版本前，在 Ceph 中启动虚拟机的唯一方式是使用 Cinder 的 `boot-from-volume` 功能。不过，现在能够在 Ceph 中直接启动虚拟机而不用依赖于 Cinder，这一点是十分有益的，因为可以通过热迁移更方便地进行维护操作。另外，如果你的 hypervisor 挂掉了，也可以很方便地触发 `nova evacuate`，并且几乎可以无缝迁移虚拟机到其他地方。

你可以用 OpenStack Glance 把 image 存储到 Ceph 块设备中，还可以使用 Cinder 通过 image 的写时复制克隆来启动虚拟机。

下面将详细指导你配置 Glance 、 Cinder 和 Nova ，虽然它们不一定一起用。你可以在本地硬盘上运行 VM 、却把 image 存储于 Ceph 块设备，反之亦可。

Important

Ceph 不支持 QCOW2 格式的虚拟机磁盘，所以，如果想要在 Ceph 中启动虚拟机（临时后端或者从卷启动），Glance 镜像必须是 RAW 格式。

Tip

本文档描述了在 Openstack Havana 中使用 Ceph 块设备。更早的版本请参考 [块设备与 OpenStack \(Dumpling\)](#)。

创建存储池

默认情况下，Ceph 块设备使用 `rbd` 存储池。你可以用任何可用存储池。建议为 Cinder 和 Glance 单独创建池。确保 Ceph 集群在运行，然后创建存储池。

```
ceph osd pool create volumes 128
```

```
ceph osd pool create images 128

ceph osd pool create backups 128

ceph osd pool create vms 128
```

参考[创建存储池](#)为存储池指定归置组数量，参考[归置组](#)确定应该为存储池设定多少归置组。

配置 OPENSTACK 的 CEPH 客户端

运行着 `glance-api`、`cinder-volume`、`nova-compute` 或 `cinder-backup` 的主机被当作 Ceph 客户端，它们都需要 `ceph.conf` 文件。

```
ssh {your-openstack-server} sudo tee /etc/ceph/ceph.conf
</etc/ceph/ceph.conf
```

安装 CEPH 客户端软件包

在运行 `glance-api` 的节点上你需要 `librbd` 的 Python 绑定：

```
sudo apt-get install python-rbd

sudo yum install python-rbd
```

在 `nova-compute`、`cinder-backup` 和 `cinder-volume` 节点上，要安装 Python 绑定和客户端命令行工具：

```
sudo apt-get install ceph-common

sudo yum install ceph
```

配置 CEPH 客户端认证

如果你启用了 [cephx 认证](#)，需要分别为 Nova/Cinder 和 Glance 创建新用户。命令如下：

```
ceph auth get-or-create client.cinder mon 'allow r' osd
'allow class-read object_prefix rbd_children, allow rwx
pool=volumes, allow rwx pool=vms, allow rx pool=images'

ceph auth get-or-create client.glance mon 'allow r' osd
'allow class-read object_prefix rbd_children, allow rwx
pool=images'

ceph auth get-or-create client.cinder-backup mon 'allow
r' osd 'allow class-read object_prefix rbd_children,
allow rwx pool=backups'
```

把 `client.cinder`、`client.glance` 和 `client.cinder-backup` 的密钥环复制到适当的节点，并更改所有权：

```
ceph auth get-or-create client.glance | ssh {your-glance-
api-server} sudo tee /etc/ceph/ceph.client.glance.keyring

ssh {your-glance-api-server} sudo chown glance:glance
/etc/ceph/ceph.client.glance.keyring

ceph auth get-or-create client.cinder | ssh {your-volume-
server} sudo tee /etc/ceph/ceph.client.cinder.keyring

ssh {your-cinder-volume-server} sudo chown cinder:cinder
/etc/ceph/ceph.client.cinder.keyring

ceph auth get-or-create client.cinder-backup | ssh {your-
cinder-backup-server} sudo tee
/etc/ceph/ceph.client.cinder-backup.keyring

ssh {your-cinder-backup-server} sudo chown cinder:cinder
/etc/ceph/ceph.client.cinder-backup.keyring
```

运行 nova-compute 的节点，其进程需要密钥环文件：

```
ceph auth get-or-create client.cinder | ssh {your-nova-  
compute-server} sudo tee  
/etc/ceph/ceph.client.cinder.keyring
```

还得把 client.cinder 用户的密钥存进 libvirt。libvirt 进程从 Cinder 挂载块设备时要用它访问集群。

在运行 nova-compute 的节点上创建一个密钥的临时副本：

```
ceph auth get-key client.cinder | ssh {your-compute-node}  
tee client.cinder.key
```

然后，在计算节点上把密钥加进 libvirt、然后删除临时副本：

```
uuidgen  
  
457eb676-33da-42ec-9a8c-9293d545c337  
  
cat > secret.xml <<EOF  
  
<secret ephemeral='no' private='no'>  
  
  <uuid>457eb676-33da-42ec-9a8c-9293d545c337</uuid>  
  
  <usage type='ceph'>  
  
    <name>client.cinder secret</name>  
  
  </usage>  
  
</secret>
```

```
EOF
```

```
sudo virsh secret-define --file secret.xml
```

```
Secret 457eb676-33da-42ec-9a8c-9293d545c337 created
```

```
sudo virsh secret-set-value --secret 457eb676-33da-42ec-9a8c-9293d545c337 --base64 $(cat client.cinder.key) && rm client.cinder.key secret.xml
```

保留密钥的 uuid ，稍后配置 nova-compute 时要用。

Important

所有计算节点上的 UUID 不一定非要一样。但考虑到平台的一致性，最好使用同一个 UUID。

配置 OPENSTACK 使用 CEPH

配置 GLANCE

Glance 可使用多种后端存储 image 。要让它默认使用 Ceph 块设备，按如下配置 Glance 。

JUNO 之前的版本

编辑 /etc/glance/glance-api.conf 并把下列内容加到 [DEFAULT] 段下：

```
default_store = rbd

rbd_store_user = glance

rbd_store_pool = images

rbd_store_chunk_size = 8
```


JUNO 版

编辑 `/etc/glance/glance-api.conf` 并把下列内容加到 `[glance_store]` 段下：

```
[DEFAULT]

...

default_store = rbd

...

[glance_store]

stores = rbd

rbd_store_pool = images

rbd_store_user = glance

rbd_store_ceph_conf = /etc/ceph/ceph.conf

rbd_store_chunk_size = 8
```

关于 Glance 里可用的其它配置选项见 http://docs.openstack.org/trunk/config-reference/content/section_glance-api.conf.html。

Important

Glance 还没完全迁移到 'store'，所以我们还得在 DEFAULT 段下配置 store。

任意版 OPENSTACK

如果你想允许使用 image 的写时复制克隆，再添加下列内容到 `[DEFAULT]` 段下：

```
show_image_direct_url = True
```

注意，这会通过 Glance API 暴露后端存储位置，所以此选项启用时 endpoint 不应该被公开访问。

禁用 Glance 缓存管理，以免 image 被缓存到 `/var/lib/glance/image-cache/` 下，假设你的配置文件里有 `flavor = keystone+cachemanagement`：

```
[paste_deploy]

flavor = keystone
```

IMAGE 属性

建议配置如下 image 属性：

- `hw_scsi_model=virtio-scsi`: 添加 `virtio-scsi` 控制器以获得更好的性能、并支持 `discard` 操作；
- `hw_disk_bus=scsi`: 把所有 `cinder` 块设备都连到这个控制器；
- `hw_qemu_guest_agent=yes`: 启用 QEMU guest agent（访客代理）
- `os_require_quiesce=yes`: 通过 QEMU guest agent 发送 `fs-freeze/thaw` 调用

配置 CINDER

OpenStack 需要一个驱动和 Ceph 块设备交互。还得指定块设备所在的存储池名。编辑 OpenStack 节点上的 `/etc/cinder/cinder.conf`，添加：

```
[DEFAULT]

...

enabled_backends = ceph

...
```

```
[ceph]

volume_driver = cinder.volume.drivers.rbd.RBDDriver

rbd_pool = volumes

rbd_ceph_conf = /etc/ceph/ceph.conf

rbd_flatten_volume_from_snapshot = false

rbd_max_clone_depth = 5

rbd_store_chunk_size = 4

rados_connect_timeout = -1

glance_api_version = 2
```

如果你使用了 [cephx 认证](#)，还需要配置用户及其密钥（前述文档中存进了 `libvirt`）的 `uuid`：

```
[ceph]

...

rbd_user = cinder

rbd_secret_uuid = 457eb676-33da-42ec-9a8c-9293d545c337
```

注意如果你为 `cinder` 配置了多后端，`[DEFAULT]` 节中必须有 `glance_api_version = 2`。

配置 CINDER BACKUP

OpenStack 的 Cinder Backup 需要一个特定的守护进程，不要忘记安装它。编辑 Cinder Backup 节点的 `/etc/cinder/cinder.conf` 添加：

```
backup_driver = cinder.backup.drivers.ceph

backup_ceph_conf = /etc/ceph/ceph.conf

backup_ceph_user = cinder-backup

backup_ceph_chunk_size = 134217728

backup_ceph_pool = backups

backup_ceph_stripe_unit = 0

backup_ceph_stripe_count = 0

restore_discard_excess_bytes = true
```

配置 NOVA 来挂载 CEPH RBD 块设备

为了挂载 Cinder 块设备（块设备或者启动卷），必须告诉 Nova 挂载设备时使用的用户和 uuid。libvirt 会使用该用户来和 Ceph 集群进行连接和认证。

```
rbd_user = cinder

rbd_secret_uuid = 457eb676-33da-42ec-9a8c-9293d545c337
```

这两个标志同样用于 Nova 的临时后端。

配置 NOVA

要让所有虚拟机直接从 Ceph 启动，必须配置 Nova 的临时后端。

建议在 Ceph 配置文件里启用 RBD 缓存（从 Giant 起默认启用）。另外，启用管理套接字对于故障排查来说大有好处，给每个使用 Ceph 块设备的虚拟机分配一个套接字有助于调查性能和/或异常行为。

可以这样访问套接字：

```
ceph daemon /var/run/ceph/ceph-  
client.cinder.19195.32310016.asok help
```

编辑所有计算节点上的 Ceph 配置文件：

```
[client]  
  
    rbd cache = true  
  
    rbd cache writethrough until flush = true  
  
    admin socket = /var/run/ceph/guests/$cluster-  
$type.$id.$pid.$cctid.asok  
  
    log file = /var/log/qemu/qemu-guest-$pid.log  
  
    rbd concurrent management ops = 20
```

调整这些路径的权限：

```
mkdir -p /var/run/ceph/guests/ /var/log/qemu/  
  
chown qemu:libvirt /var/run/ceph/guests /var/log/qemu/
```

要注意，`qemu` 用户和 `libvirt` 组可能因系统不同而不同，前面的实例基于 RedHat 风格的系统。

Tip

如果你的虚拟机已经跑起来了，重启一下就能得到套接字。

HAVANA AND ICEHOUSE

Havana 和 Icehouse 需要补丁来实现写时复制克隆、修复 rbd 临时磁盘的镜像大小和热迁移中的缺陷。这些补丁可从基于 Nova [stable/havana](#) 和 [stable/icehouse](#) 的分支中获取。虽不是强制性的，但**强烈建议**使用这些补丁，以便能充分利用写时复制克隆功能的优势。

编辑所有计算节点上的 `/etc/nova/nova.conf` 文件，添加：

```
libvirt_images_type = rbd

libvirt_images_rbd_pool = vms

libvirt_images_rbd_ceph_conf = /etc/ceph/ceph.conf

libvirt_disk_cachemodes="network=writeback"

rbd_user = cinder

rbd_secret_uuid = 457eb676-33da-42ec-9a8c-9293d545c337
```

禁用文件注入也是一个好习惯。启动一个实例时，Nova 通常试图打开虚拟机的根文件系统。然后，Nova 会把比如密码、ssh 密钥等值注入到文件系统中。然而，最好依赖元数据服务和 `cloud-init`。

编辑所有计算节点上的 `/etc/nova/nova.conf` 文件，添加：

```
libvirt_inject_password = false

libvirt_inject_key = false

libvirt_inject_partition = -2
```

为确保热迁移能顺利进行，要使用如下标志：

```
libvirt_live_migration_flag="VIR_MIGRATE_UNDEFINE_SOURCE,  
VIR_MIGRATE_PEER2PEER,VIR_MIGRATE_LIVE,VIR_MIGRATE_PERSIST_DEST,VIR_MIGRATE_TUNNELLED"
```

JUNO

在 Juno 版中，Ceph 块设备移到了 `[libvirt]` 段下。编辑所有计算节点上的 `/etc/nova/nova.conf` 文件，在 `[libvirt]` 段下添加：

```
[libvirt]  
  
images_type = rbd  
  
images_rbd_pool = vms  
  
images_rbd_ceph_conf = /etc/ceph/ceph.conf  
  
rbd_user = cinder  
  
rbd_secret_uuid = 457eb676-33da-42ec-9a8c-9293d545c337  
  
disk_cachemodes="network=writeback"
```

禁用文件注入也是一个好习惯。启动一个实例时，Nova 通常试图打开虚拟机的根文件系统。然后，Nova 会把比如密码、ssh 密钥等值注入到文件系统中。然而，最好依赖元数据服务和 `cloud-init`。

编辑所有计算节点上的 `/etc/nova/nova.conf` 文件，在 `[libvirt]` 段下添加：

```
inject_password = false  
  
inject_key = false  
  
inject_partition = -2
```

为确保热迁移能顺利进行，要使用如下标志（在 `[libvirt]` 段下）：

```
live_migration_flag="VIR_MIGRATE_UNDEFINE_SOURCE,VIR_MIGRATE_PEER2PEER,VIR_MIGRATE_LIVE,VIR_MIGRATE_PERSIST_DEST,VIR_MIGRATE_TUNNELLED"
```

KILO

为虚拟机的临时根磁盘启用 discard 功能：

```
[libvirt]

...

...

hw_disk_discard = unmap # 启用 discard 功能 (注意性能)
```

重启 OPENSTACK

要激活 Ceph 块设备驱动、并把块设备存储池名载入配置，必须重启 OpenStack。在基于 Debian 的系统上去对应节点执行下列命令：

```
sudo glance-control api restart

sudo service nova-compute restart

sudo service cinder-volume restart

sudo service cinder-backup restart
```

在基于 Red Hat 的系统上执行：

```
sudo service openstack-glance-api restart

sudo service openstack-nova-compute restart
```



```
sudo service openstack-cinder-volume restart

sudo service openstack-cinder-backup restart
```

一旦 OpenStack 启动并运行正常，应该就可以创建卷并用它启动虚拟机了。

从块设备引导

你可以用 Cinder 命令行工具从弄个 image 创建卷：

```
cinder create --image-id {id of image} --display-name
{name of volume} {size of volume}
```

注意 image 必须是 RAW 格式，你可以用 [qemu-img](#) 转换格式，如：

```
qemu-img convert -f {source-format} -O {output-format}
{source-filename} {output-filename}

qemu-img convert -f qcow2 -O raw precise-cloudimg.img
precise-cloudimg.raw
```

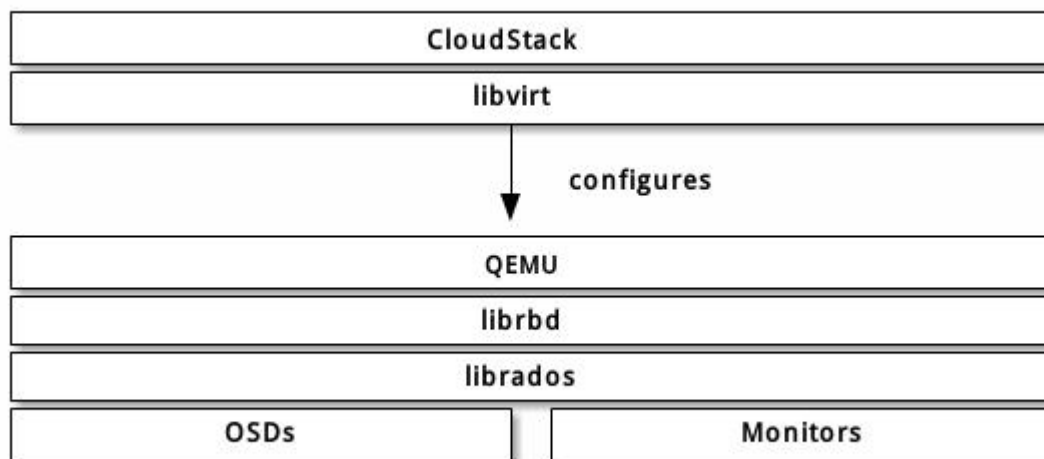
Glance 和 Cinder 都使用 Ceph 块设备，此镜像又是个写时复制克隆，就能非常快地创建一个新卷。在 OpenStack 操作面板里就能从那个启动虚拟机，步骤如下：

1. 启动新实例。
2. 选择与写时复制克隆关联的 image 。
3. 选择 'boot from volume' 。
4. 选择你刚创建的卷。

块设备与 CLOUDSTACK

CloudStack 4.0 及以上版本可以通过 `libvirt` 使用 Ceph 块设备，`libvirt` 会配置 QEMU 与 `librbd` 交互。Ceph 会把块设备映像条带化为对象并分布到整个集群，这意味着大个的 Ceph 块设备性能会优于单体服务器！

要让 CloudStack 4.0 及更高版使用 Ceph 块设备，你得先安装 QEMU、`libvirt` 和 CloudStack。我们建议在另外一台物理服务器上安装 CloudStack，此软件最低需要 4GB 内存和一个双核 CPU，但是资源越多越好。下图描述了 CloudStack/Ceph 技术栈。



Important

要让 CloudStack 使用 Ceph 块设备，你必须有 Ceph 存储集群的访问权限。

CloudStack 集成了 Ceph 的块设备作为它的后端主要存储（Primary Storage），下列指令详述了 CloudStack 主存储的安装。

Note

我们建议您安装 Ubuntu 14.04 或更高版本，这样就不用手动编译 libvirt 了。

安装、配置 QEMU 用于 CloudStack 时不需要任何特殊处理。确保你的 Ceph 存储集群在运行。安装并配置好 QEMU ；然后安装 `libvirt` v 0.9.13 或更高版本（也许得从源码手动编译）并确保它与 Ceph 结合后可正常运行。

Note

Ubuntu 14.04 和 CentOS 7.2 上的 `libvirt` 默认支持 RBD 存储池。

创建存储池

默认情况下，Ceph 块设备使用 `rbd` 存储池，建议为 CloudStack NFS 主存储新建一个存储池。确保 Ceph 集群在运行，然后创建存储池：

```
ceph osd pool create cloudstack
```

具体可参考[创建存储池](#)为存储池指定归置组数量，参考[归置组](#)确定应该为存储池分配多少归置组。

创建 CEPH 用户

为了使用 Ceph 集群，我们需要一个有正确授权的用户，来访问刚才创建的 `cloudstack` 存储池。虽然我们可以使用 `client.admin`，但还是推荐创建一个只对 `cloudstack` 存储池有访问权限的用户。

```
ceph auth get-or-create client.cloudstack mon 'allow r'
osd 'allow class-read object_prefix rbd_children, allow
rwx pool=cloudstack'
```

在后面添加主存储的步骤里，会使用到该条命令的返回信息。

更多信息见 [用户管理](#)。

添加主存储

添加 Ceph 块设备作为主存储的方法可参考 [添加主存储 \(4.2.0\)](#)，步骤包括：

1. 登录 CloudStack 界面。
2. 点击左侧导航条的 **Infrastructure**。
3. 选择要用于主存储的域。
4. 点击 **Compute** 标签。
5. 选择图表中 *Primary Storage* 节点上的 **View All**。
6. 点击 **Add Primary Storage**。
7. 依次按提示执行：
 - **Protocol** 那里选择 RBD；
 - 添加集群信息（支持 cephx）。注：不要加用户名的 `client.` 部分。
 - 把 `rbd` 加为标签。

创建硬盘方案

要新建硬盘方案，参考[新建硬盘方案 \(4.2.0\)](#)。创建一个硬盘方案以与 `rbd` 标签相配。这样 `StoragePoolAllocator` 查找合适的存储池时就会选择 `rbd` 存储池；如果硬盘方案没有与 `rbd` 标签相匹配，`StoragePoolAllocator` 就会选用你创建的存储池（比如 `cloudstack`）。

局限性

- CloudStack 只能绑定一个 `monitor`（但你可以在多个 `monitors` 上创建一个轮询域名服务）。

RBD – 管理 RADOS 块设备（RBD）映像

摘要

```
rbd [ -c ceph.conf ] [ -m monaddr ] [ -cluster cluster name ] [ -p | -pool pool ] [ -size size ] [ -order bits ] [ command ... ]
```

描述

rbd 是个操纵 rados 块设备 (RBD) 映像的工具，QEMU/KVM 通过 Linux rbd 内核驱动和 rbd 存储驱动来使用。RBD 映像是简单的块设备，它被条带化成小块对象后存储于 RADOS 对象存储集群。条带化后的对象尺寸必须是 2 的幂。

选项

-c *ceph.conf*, **--conf** *ceph.conf*

指定 *ceph.conf* 配置文件，而不是用默认的 */etc/ceph/ceph.conf* 来确定启动时需要的 monitor 地址。

-m *monaddress[:port]*

连接到指定 monitor，无需通过 *ceph.conf* 查找。

--cluster *cluster name*

使用自定义的集群名字，即不是默认的 *ceph*。

-p *pool-name*, **--pool** *pool-name*

在指定存储池下操作，大多数命令都得指定。

--no-progress

不显示进度信息（有些命令会默认输出到标准错误）。

参数

--image-format *format-id*

选择使用哪个对象布局，默认为 1。

- **format 1** - （废弃）新建 **rbd** 映像时使用最初的格式。此格式兼容所有版本的 **librbd** 和内核模块，但是不支持诸如克隆这样较新的功能。
- **format 2** - 使用第二版 **rbd** 格式，**librbd** 和 3.11 版以上内核模块才支持（除非是分拆的模块）。此格式增加了对克隆的支持，日后扩展以增加新功能也变得更加容易。

--size *size-in-M/G/T*

指定新 rbd 映像的大小，单位可以是 M/G/T。

--object-size B/K/M

指定 object 对象的大小，单位可以是 B/K/M，会向上取整为 2 的整数次幂。默认值是 4 MB，最小值是 4 KB，最大值是 32 MB。

--stripe-unit size-in-B/K/M

指定条带单元尺寸，单位可以是 B/K/M。详情见下面的条带化一节。

--stripe-count num

指定条带化要至少跨越多少对象后才能转回第一个。详情见条带化一节。

--snap snap

某些操作需要指定快照名。

--id username

指定 map 命令要用到的用户名（不含 `client.` 前缀）。

--keyring filename

指定 map 命令所需的包含特定用户密钥的文件。如果未指定，去默认密钥环位置查找。

--keyfile filename

指定 map 命令所需的包含 `--id user` 用户密钥的文件。如果同时指定了 `--keyring` 选项（常用），本选项就会被覆盖。

--shared lock-
tag

lock add 选项允许使用同一标签的多个客户端同时锁住某个映像。标签是任意字符串。当需要从多个客户端同时打开某映像时，此选项很有用，像热迁移虚拟机时、或者在集群文件系统下使用时。

--
format format

指定输出格式，默认：plain、json、xml。

--pretty-format

使 json 或 xml 格式的输出更易读。

-
o map-
options,
--
option
s map-
options

指定 map 映像时所用的选项。选项格式为逗号分隔的字符串（类似于 mount(8) 的挂载选项）。详情见下面的 map 选项小节。

--
rea
d-
only

以只读方式 map 映像，等价于 -o ro 。

-
-
i
m
a
g
e
-
f
e
a
t
u
r
e

f
e
a
t
u

指定创建 format 2 格式的 RBD 映像时，要启用的特性。可以通过重复此选项来启用多个特性。当前支持下列特性：

- layering: 支持分层
- striping: 支持条带化 v2
- exclusive-lock: 支持独占锁
- object-map: 支持对象映射（依赖 exclusive-lock ）
- fast-diff: 快速计算差异（依赖 object-map ）
- deep-flatten: 支持快照扁平化操作
- journaling: 支持记录 IO 操作（依赖独占锁）

指定该映像将被多个客户端同时使用。此选项将禁用那些依赖于独占映像所有权的功能。

-
o
b
j
e
c
t

把 diff 操作范围限定在完整的对象级别，而非对象内差异。当某一映像启用了 object-map 功能时，把 diff 操作限定到对象级别会显著地提高性能，因为通过检查驻留于内存中的对象映射就可以计算出差异，而无需针对映像内的各个对象去查询 RADOS。

命令

l
s

[
-
l

|

-
l
o
n
g
]

[
p
o
o
l
-
n
a
m
e
]

列出 `rbd_directory` 对象中的所有 `rbd` 映像。加 `-l` 选项后也显示快照，并用长格式输出，包括大小、父映像（若是克隆）、格式等等。

计算指定存储池内所有映像及其相关快照的磁盘使用量，包括预分配的和实际使用的。此命令也可用于单个映像和快照。

如果没启用 RBD 映像的 fast-diff 特性，此操作则需向多个 OSD 查询此映像涉及的每个对象。

显示指定 **rbd** 映像的信息（如映像大小和对象大小）。若映像是克隆品，会显示其父快照的信息。若指定了快照，会显示是否被保护。

新建 **rbd** 映像。必须用 **-size** 指定尺寸。**-strip-unit** 和 **-strip-count** 参数是可选项，但必须同时使用。

创建一个父快照的克隆（写时复制子映像）。若不指定 object size，会采用父映像的值。克隆映像的大小和父快照相同。参数 `-stripe-unit` 和 `-stripe-count` 是可选的，但必须同时使用。

父快照必须已被保护（见 *rbdsnap protect*）。只适用于 format 2 格式的映像。

如果映像是个克隆，就从父快照拷贝所有共享的块，使子快照独立于父快照，切断父子快照间的联系。如果没有克隆映像引用此父快照了，就可以取消保护并删除它。

只适用于 format 2 格式的映像。

列出此映像指定快照的克隆品。它会检查各存储池、并输出存储池名/映像名。

只适用于 format 2 格式的映像。

调整 rbd 映像的大小。必须指定 size 参数。--allow-shrink 选项允许缩小。

删除 rbd 映像，包括所有数据块。如果映像有快照，此命令会失效。

把映像导出到目的路径，用 - （短线） 输出到标准输出。

创建映像，并从目的路径导入数据，用 - (短线) 表示从标准输入导入。如果可能的话，导入操作会试着创建稀疏映像。如果从标准输入导入，稀疏化单位将是目标映像的数据块尺寸 (即 object size)。

参数 `-stripe-unit` 和 `-stripe-count` 是可选的，但必须同时使用。

导出映像的增量差异，用 - 表示导出到标准输出。若指定了起始快照，就只包含与此快照的差异部分；否则包含映像的所有数据部分。结束快照用 **-snap** 选项或 **@snap** 语法（见下文）指定。此映像的差异格式包含了映像尺寸变更的元数据、起始和结束快照，它高效地表达了被丢弃的或映像内的全 0 区域。

把两个连续的增量差异合并为单个差异。前一个差异的结束快照必须与后一个差异的起始快照相同。前一个差异可以是标准输入 -，合并后的差异可以是标准输出 -，这样就可以合并多个差异文件，像这样：‘`rbd merge-diff first second - | rbd merge-diff - third result`’。注意，当前此命令仅支持 `stripe_count == 1` 这样的源增量差异。

导入一个映像的增量差异并应用到当前映像。如果此差异是在起始快照基础上生成的，我们会先校验那个已存在快照再继续；如果指定了结束快照，在应用这些变更前，我们还要检查它是否不存在，完成后再创建结束快照。

打印出从指定快照点起、或从映像创建点起，映像内的变动区域。输出的各行都包含起始偏移量（按字节）、数据块长度（按字节）、还有 **zero** 或 **data**，用来指示此区域是 **0** 还是包含有其它数据。

把源映像内容复制进新建的目标映像，目标映像和源映像将有相同的大小、**object size** 和格式。

映像重命名。注：不支持跨存储池。

显示此映像持有的元数据。第一列是关键字、第二列是值。

获取关键字对应的元数据值。

设置元数据关键字的值，会显示在 *metadata-list* 中。

删除元数据关键字及其值。

为指定映像重建无效的对象映射关系。指定映像快照时，将为此快照重建无效的对象映射关系。

列出指定映像内的快照。

新建快照。需指定快照名。

把指定映像回滚到某个快照。此动作会递归整个块阵列，并把数据头内容更新到快照版本。

删除指定快照。

删除某个映像的所有快照。

保护快照，以防删除，这样才能从它克隆（见 *rbd clone* ）。做克隆前必须先保护快照，保护意味着存在有依赖于此快照的克隆。*rbd clone* 不能在未保护的快照上操作。

只适用于 format 2 格式的映像。

取消对快照的保护（撤销 *snap protect* ）。如果还有克隆出的子快照尚在，*snap unprotect* 命令会失效。（注意克隆和其父快照可能会位于不同的存储池。）

只适用于 format 2 格式的映像。

通过内核 **rbd** 模块把指定映像映射到某一块设备。

取消通过内核 **rbd** 模块的所做的块设备映射。

显示通过内核 **rbd** 模块映射的 **rbd** 映像。

通过 **rbd-nbd** 工具把指定映像映射成一个块设备。

取消通过 **rbd-nbd** 工具映射的块设备。

显示使用中的通过 `rbd-nbd` 工具映射的 `nbd` 设备。

显示映像状态，包括哪个客户端打开着它。

禁用指定镜像的某些特性，可以一次指定多个。

启用指定镜像的某些特性，可以一次指定多个。

显示加在映像上的锁，第一列是 *lock remove* 可以使用的锁名。

为映像加锁。锁标识是用户所选的任意名字。默认加的是互斥锁，这意味着如果映像已经加过锁的话此命令会失败。**-shared** 选项会改变此行为。注意，加锁操作不影响除加锁之外的任何操作，也不会保护映像、防止它被删除。

释放映像上的锁。锁标识和其持有者来自 **lock ls** 。

向某一映像生成一系列写操作，以此测量写吞吐量和延时。默认参数为 `-io-size 4096` 、`-io-threads 16` 、`-io-total 1G` 、`-io-pattern seq` 。

条带化产生的对象大小是 2 的幂。会向上取整为最接近 2 的幂的值。默认值是 4 MB，最小值是 4 KB，最大值是 32 MB。

各 [*stripe_unit*] 是一段连续的字节，相邻地存储于同一对象，放满后再去下一个对象。

我们写够 `[stripe_count]` 个长度为 `[stripe_unit]` 字节的对象后，再转回到第一个对象写另一轮条带，直到达到对象的最大尺寸。此时，我们再写下一轮 `[stripe_count]` 个对象。

RBD-FUSE – 把 RBD 映像展现为文件

摘要

rbdfuse [-p pool] [-c conffile] *mountpoint* [*fuse options*]

描述

rbdfuse 是个 rbd 映像的用户空间文件系统（ File system in USErspace , FUSE ）客户端。给定一个包含 rbd 映像的存储池，它就可以在用户空间把那些映像挂载到 **mountpoint** 下，并显示为普通文件。

用下列命令卸载文件系统：

```
fusermount -u mountpoint
```

或者向 `rbdfuse` 进程发送 `SIGINT` 信号。

选项

`rbdfuse` 无法识别的选项将传递给 `libfuse` 。

-c `ceph.conf`

用指定的 `ceph.conf` 配置文件、而非默认的 `/etc/ceph/ceph.conf` 来确定启动 monitor 时所用的地址。

-p `pool`

在 `pool` 存储池中搜索 rbd 映像，默认为 `rbd`。

使用范围

rbd-fuse 是 Ceph 的一部分，这是个大规模可伸缩、开源、分布式的存储系统，更多信息参见 <http://ceph.com/docs> 或 <http://docs.ceph.org.cn/>。

另请参阅

fusermount(8), *rbd*(8)

RBD-NBD – 把 RBD 镜像映射为 NBD 设备

摘要

rbd-nbd [-c *conf*] [--nbds_max *limit*] [--read-only] [--device *nbd device*] map *image-spec* | *snap-spec*

rbd-nbd unmap *nbd device*

rbd-nbd list-mapped

描述

rbd-nbd 是个 RADOS 块设备 (*rbd*) 映像的客户端，与 *rbd* 内核模块类似。它可以把一个 *rbd* 映像映射为 *nbd* (Network Block Device , 网络块设备) 设备，这样就可以当常规的本地块设备使用了。

选项

-c *ceph.conf*

指定 *ceph.conf* 配置文件，而不是用默认的 */etc/ceph/ceph.conf* 来确定启动时需要的 *monitor* 地址。

--nbds_max **limit**

载入 NBD 内核模块时覆盖其参数，用于限制 *nbd* 设备的数量。

映像名和快照名规则

image-spec is *[pool-name]/image-name*

snap-spec is *[pool-name]/image-name@snap-name*

pool-name 的默认值是 “*rbd*”。如果映像名里包含斜线符号（*/*），那就必须指定 *pool-name*。

使用范围

rbd-nbd 是 Ceph 的一部分，这是个大规模可伸缩、开源、分布式的存储系统，更多信息参见 <http://ceph.com/docs> 或 <http://docs.ceph.org.cn/>。

另请参阅

rbd(8)

CEPH-RBDNAMER – 用于命名 RBD 设备的 UDEV 辅助程序

摘要

ceph-rbdnamer *num*

描述

ceph-rbdnamer 把指定 RBD 设备所属的存储池和映像名打印到标准输出，以便 udev（使用类似如下的规则）设置设备的符号链接：

```
KERNEL=="rbd[0-9]*", PROGRAM="/usr/bin/ceph-rbdnamer %n",  
SYMLINK+="rbd/%c{1}/%c{2}"
```

使用范围

ceph-rbdnamer 是 Ceph 的一部分，这是个大规模可伸缩、开源、分布式的存储系统，更多信息参见 <http://ceph.com/docs> 或 <http://docs.ceph.org.cn/>。

另请参阅

[rbd\(8\)](#), [ceph\(8\)](#)

RBD 重放

RBD 重放是个用于捕捉和重放 Rados 块设备（RBD）载荷的工具集。要捕捉 RBD 载荷，必须在客户端安装 `lttng-tools`，且客户端上的 `librbd` 必须在 v0.87（Giant）版以上。

捕捉、重放需三步：

1. 捕捉一个追踪过程，确保捕捉的是 `pthread_id` 上下文：

```
2. mkdir -p traces
3. lttng create -o traces librbd
4. lttng enable-event -u 'librbd:*'
5. lttng add-context -u -t pthread_id
6. lttng start
7. # run RBD workload here
```

```
lttng stop
```

8. 用 [rbd-replay-prep](#) 处理捕捉到的追踪结果：

```
rbd-replay-prep traces/ust/uid/*/ * replay.bin
```

9. 用 `rbd-replay` 重放此追踪过程。熟知它的行为前请用只读模式：

```
rbd-replay --read-only replay.bin
```

Important

`rbd-replay` 默认会销毁数据。不要在想保留的映像上使用，除非指定了 `--read-only` 选项。

重放载荷并非只能在与捕获时相同的 RBD 映像或相同的集群上进行。为说明差异，你也许得在执行 `rbd-replay` 命令时指定 `--pool` 和 `--map-image` 选项。

RBD-REPLAY-PREP – 对捕捉到的用于重放的 RBD 工作载荷进行预处理

摘要

```
rbd-replay-prep [ -window seconds ] [ -anonymize ] trace_dir replay_file
```

描述

`rbd-replay-prep` 可处理原始 RBD 映像跟踪文件，以便用于 `rbd-replay`。

选项

--window seconds

超过 'seconds' 秒之后的请求被认为是独立的。

--anonymize

匿名化映像和快照名。

--verbose

把所有处理过的事件打印到控制台。

样例

预处理 workload1-trace 以便重放：

```
rbd-replay-prep workload1-  
trace/ust/uid/1000/64-bit workload1
```

使用范围

rbd-replay-prep 是 Ceph 的一部分，这是个大规模可伸缩、开源、分布式的存储系统，更多信息参见 <http://ceph.com/docs> 或 <http://docs.ceph.org.cn/>。

另请参阅

rbd-replay(8), *rbd*(8)

RBD-REPLAY – 重放 RBD 工作载荷

摘要

rbd-replay [*options*] *replay_file*

描述

rbd-replay 工具用于重放 RBD 载荷。

选项

-c *ceph.conf*, **--conf** *ceph.conf*

使用 `ceph.conf` 配置文件，而不是用默认的 `/etc/ceph/ceph.conf` 来确定启动时需要的 `monitor` 地址。

-p pool, --pool pool

与指定存储池交互，默认为 'rbd'。

--latency-multiplier

请求间的延时倍数，默认为 1。

--read-only

只重放非破坏性的请求。

--map-image rule

增加一条规则把跟踪文件中的映像名映射为重放集群中的映像名。规则

`image1@snap1=image2@snap2` 将把 `image1` 的快照 `snap1` 映射为 `image2` 的快照 `snap2`。

--dump-perf-counters

实验功能 关闭映像前先把性能计数器转储到标准输出。如果关闭了多个映像或者同一映像被打开、关闭多次，那么性能计数器就可能转储多次。性能计数器及其含义可能因版本而不同。

样例

尽可能快地重放 `workload1`:

```
rbid-replay --latency-multiplier=0
workload1
```

重放 `workload1`，并用 `test_image` 取代 `prod_image`:

```
rbid-replay --map-
image=prod_image=test_image workload1
```

使用范围

rbd-replay 是 Ceph 的一部分，这是个大规模可伸缩、开源、分布式的存储系统，更多信息参

见 <http://ceph.com/docs> 或 <http://docs.ceph.org.cn/>。

另请参阅

rbd-replay-prep(8), *rbd*(8)

RBD-REPLAY-MANY – 在多个客户端上重放 RBD 工作载荷

摘要

rbd-replay-many [*options*] *–original-image name host1* [*host2* [...]] *– rbd_replay_args*

描述

rbd-replay-many 工具用于在多个客户端上重放 RBD 载荷。虽然所有客户端使用相同的载荷，但他们对单独的映像进行重放。正像 librbd 的常规用法，其中各原始客户端都是有各自映像的虚拟机。

配置和重放文件不会自动复制到客户端。重放映像必须存在才能进行重放操作。

选项

–original-image *name*

指定初始被追踪映像的名字（和快照），这对正确地映射名字是必要的。

–image-prefix *prefix*

要进行重放的映像名前缀。指定 *–image-prefix=foo* 可使客户端重放 *foo-0*、*foo-1* 等等。默认为初始映像名。

–exec *program*

rbd-replay 可执行文件的路径。

--delay seconds

启动各客户端之间的延时，默认为 0。

样例

典型用法：

```
rbd-replay-many host-0 host-1 --original-  
image=image -- -c ceph.conf replay.bin
```

实际上将执行下列的命令：

```
ssh host-0 'rbd-replay' --map-image  
'image=image-0' -c ceph.conf replay.bin  
  
ssh host-1 'rbd-replay' --map-image  
'image=image-1' -c ceph.conf replay.bin
```

使用范围

rbd-replay-many 是 Ceph 的一部分，这是个大规模可伸缩、开源、分布式的存储系统，更多信息参见 <http://ceph.com/docs> 或 <http://docs.ceph.org.cn/>。

另请参阅

rbd-replay(8), *rbd*(8)

LIBRBD 的 PYTHON 接口

rbd 的 python 模块为 RBD 映像提供了类似文件的访问方法。

实例：创建一映像并写入

要使用 *rbd*，必须先连接 RADOS 并打开一个 IO 上下文：

```
cluster = rados.Rados(conffile='my_ceph.conf')

cluster.connect()

ioctx = cluster.open_ioctx('mypool')
```

然后实例化一个 `:class:rbd.RBD` 对象，用它来创建映像：

```
rbd_inst = rbd.RBD()

size = 4 * 1024**3 # 4 GiB

rbd_inst.create(ioctx, 'myimage', size)
```

要在映像上进行 I/O 操作，需实例化 `:class:rbd.Image` 对象：

```
image = rbd.Image(ioctx, 'myimage')

data = 'foo' * 200

image.write(data, 0)
```

上面的代码向映像前面写入了 600 字节的 foo 字符串。注意数据不能是 `:type:unicode`，`librados` 不知道如何处理大于 `:c:type:char` 宽度的字符。

最后，应该要关闭映像、IO 上下文和 RADOS 的连接。

```
image.close()

ioctx.close()
```

```
cluster.shutdown()
```

安全起见，每个调用都应该封装到单独的 `:finally` 块内。

```
cluster = rados.Rados(conffile='my_ceph_conf')

try:

    iocx = cluster.open_iocx('my_pool')

    try:

        rbd_inst = rbd.RBD()

        size = 4 * 1024**3 # 4 GiB

        rbd_inst.create(iocx, 'myimage', size)

        image = rbd.Image(iocx, 'myimage')

        try:

            data = 'foo' * 200

            image.write(data, 0)

        finally:

            image.close()

    finally:

        iocx.close()

finally:

    cluster.shutdown()
```

这样做有些繁琐，所以 **Rados**、**Ioctx** 和 **Image** 类可以当上下文管理器来用，它能自动关闭（见 **PEP 343**）。作为上下文管理器使用时，上面的例子可以写成：

```
with rados.Rados(conffile='my_ceph.conf') as cluster:

    with cluster.open_ioctx('mypool') as ioctx:

        rbd_inst = rbd.RBD()

        size = 4 * 1024**3 # 4 GiB

        rbd_inst.create(ioctx, 'myimage', size)

        with rbd.Image(ioctx, 'myimage') as image:

            data = 'foo' * 200

            image.write(data, 0)
```

API 参考

This module is a thin wrapper around librbd.

It currently provides all the synchronous methods of librbd that do not use callbacks.

Error codes from librbd are turned into exceptions that subclass **Error**. Almost all methods may raise **Error** (the base class of all rbd exceptions), **PermissionError** and **IOError**, in addition to those documented for the method.

class rbd.**RBD**

This class wraps librbd CRUD functions.

clone(self, p_ioctx, p_name, p_snapname, c_ioctx, c_name, features=0, order=None, stripe_unit=0, stripe_count=0)

Clone a parent rbd snapshot into a COW sparse child.

p_ioctx – the parent context that represents the parent snap
p_name – the parent image name
p_snapname – the parent image snapshot name
c_ioctx – the child context that represents the new clone
c_name – the clone (child) name
Parameters: **features** (*int*) – bitmask of features to enable; if set, must include layering
order (*int*) – the image is split into (2**order) byte objects
stripe_unit (*int*) – stripe unit in bytes (default 0 for object size)
stripe_count (*int*) – objects to stripe over before looping

Raises : **TypeError**
Raises : **InvalidArgument**
Raises : **ImageExists**
Raises : **FunctionNotSupported**
Raises : **ArgumentOutOfRange**

create(*self, ioctx, name, size, order=None, old_format=True, features=0, stripe_unit=0, stripe_count=0*)

Create an rbd image.

ioctx (**rados . Ioctx**) – the context in which to create the image
name (*str*) – what the image is called
size (*int*) – how big the image is in bytes
order (*int*) – the image is split into (2**order) byte objects
old_format (*bool*) – whether to create an old-style image that is accessible by old clients, but can't use more advanced features like layering.
Parameters: **features** (*int*) – bitmask of features to enable
stripe_unit (*int*) – stripe unit in bytes (default 0 for object size)
stripe_count (*int*) – objects to stripe over before looping

Raises : **ImageExists**
Raises : **TypeError**

Raises : **InvalidArgument**
Raises : **FunctionNotSupported**

list(*self*, *ioctx*)

List image names.

Parameters: **ioctx** (**rados . Ioctx**) – determines which RADOS pool is read

Returns: list – a list of image names

remove(*self*, *ioctx*, *name*)

Delete an RBD image. This may take a long time, since it does not return until every object that comprises the image has been deleted. Note that all snapshots must be deleted before the image can be removed. If there are snapshots left, **ImageHasSnapshots** is raised. If the image is still open, or the watch from a crashed client has not expired, **ImageBusy** is raised.

Parameters: **ioctx** (**rados . Ioctx**) – determines which RADOS pool the image is in
name (*str*) – the name of the image to remove

Raises : **ImageNotFound, ImageBusy, ImageHasSnapshots**
rename(*self*, *ioctx*, *src*, *dest*)

Rename an RBD image.

Parameters: **ioctx** (**rados . Ioctx**) – determines which RADOS pool the image is in
src (*str*) – the current name of the image
dest (*str*) – the new name of the image

Raises : **ImageNotFound, ImageExists**
version(*self*)

Get the version number of the `librbd` C library.

Returns: a tuple of (major, minor, extra) components of the librbd version

`class rbd.Image`

`Image(ioctx, name, snapshot=None, read_only=False)`

This class represents an RBD image. It is used to perform I/O on the image and interact with snapshots.

Note: Any method of this class may raise **ImageNotFound** if the image has been deleted.

break_lock(*self*, *client*, *cookie*)

Release a lock held by another rados client.

close(*self*)

Release the resources used by this image object.

After this is called, this object should not be used.

copy(*self*, *dest_ioctx*, *dest_name*, *features*=0, *order*=None, *stripe_unit*=0, *stripe_count*=0)

Copy the image to another location.

dest_ioctx (`rados.Ioctx`) – determines which pool to copy into

dest_name (*str*) – the name of the copy

features (*int*) – bitmask of features to enable; if set, must include

Parameters: layering

order (*int*) – the image is split into (2**order) byte objects

stripe_unit (*int*) – stripe unit in bytes (default 0 for object size)

stripe_count (*int*) – objects to stripe over before looping

Raises : **TypeError**

Raises : **InvalidArgument**

Raises : **ImageExists**

Raises : **FunctionNotSupported**

Raises : **ArgumentOutOfRange**

create_snap(*self*, *name*)

Create a snapshot of the image.

Parameters: **name** (*str*) – the name of the snapshot

Raises : **ImageExists**

diff_iterate(*self*, *offset*, *length*, *from_snapshot*, *iterate_cb*,
include_parent=True, *whole_object=False*)

Iterate over the changed extents of an image.

This will call *iterate_cb* with three arguments:

(*offset*, *length*, *exists*)

where the changed extent starts at *offset* bytes, continues for *length* bytes, and is full of data (if *exists* is *True*) or zeroes (if *exists* is *False*).

If *from_snapshot* is *None*, it is interpreted as the beginning of time and this generates all allocated extents.

The end version is whatever is currently selected (via *set_snap*) for the image.

iterate_cb may raise an exception, which will abort the diff and will be propagated to the caller.

Raises **InvalidArgument** if *from_snapshot* is after the currently set snapshot.

Raises **ImageNotFound** if *from_snapshot* is not the name of a snapshot of the image.

offset (*int*) – start offset in bytes

length (*int*) – size of region to report on, in bytes

from_snapshot (*str or None*) – starting snapshot name, or None

iterate_cb (*function* *accepts arguments for offset, length, and exists*) – function to call for each extent

Parameters:

include_parent (*bool*) – True if full history diff should include parent

whole_object (*bool*) – True if diff extents should cover whole object

Raises : **InvalidArgument, IOError, ImageNotFound**
 discard(*self, offset, length*)

Trim the range from the image. It will be logically filled with zeroes.

features(*self*)

Gets the features bitmask of the image.

Returns: int - the features bitmask of the image

flags(*self*)

Gets the flags bitmask of the image.

Returns: int - the flags bitmask of the image

flatten(*self*)

Flatten clone image (copy all blocks from parent to child)

flush(*self*)

Block until all writes are fully flushed if caching is enabled.

invalidate_cache(*self*)

Drop any cached data for the image.

```
is_exclusive_lock_o  
wner(self)
```

Gets the status of the image exclusive lock.

Returns: bool - true if the image is exclusively locked

```
is_protected_sna  
p(self, name)
```

Find out whether a snapshot is protected from deletion.

Parameters: **name** (*str*) – the snapshot to check

Returns: bool - whether the snapshot is protected

Raises : **IOError, ImageNotFound**

```
list_children(  
self)
```

List children of the currently set snapshot (set via set_snap()).

Returns: list - a list of (pool name, image name) tuples

```
list_locke  
rs(self)
```

List clients that have locked the image and information about the lock.

dict - contains the following keys:

Returns: **tag** - the tag associated with the lock (every additional locker must use the same tag)
exclusive - boolean indicating whether the

lock is exclusive or shared

`lockers` - a list of (client, cookie, address) tuples

list_snapshots(self)

Iterate over the snapshots of an image.

Returns: `SnapshotIterator`

lock_exclusive(self, cookie)

Take an exclusive lock on the image.

Raises : `ImageBusy` if a different client or cookie locked it `ImageExists` if the same client and cookie locked it

lock_shared(self, cookie)

e,
tag)

Take a shared lock on the image. The tag must match that of the existing lockers, if any.

Raises : **ImageBusy** if a different client or cookie locked it **ImageExists** if the same client and cookie locked it

old
format
(
se-
f
)

Find out whether the image uses the old RBD format.

Returns: bool - whether the image uses the old RBD format

over-
l

a
p
(
s
e
/
f
)

Gets the number of overlapping bytes between the image and its parent image. If open to a snapshot, returns the overlap between the snapshot and the parent image.

Returns: int - the overlap in bytes

Raises : **ImageNotFound** if the image doesn't have a parent

p
a
r
e
n
t
_
i
n
f
o
(
s
e
/
f
)

Get information about a cloned image's parent (if any)

Returns tuple -
: (pool name, image name, snapshot name) components of the parent image

Raises : **ImageNotFound** if the image doesn't have a parent

p
r
o
t
e
c
t
—
s
n
a
p
(
s
e
/
f
,

n
a
m
e
)

Mark a snapshot as protected. This means it can't be deleted until it is unprotected.

Parameters: `name` (*str*) – the snapshot to protect

Raises : `IOError`, `ImageNotFound`

Read data from the image. Raises **InvalidArgument** if part of the range specified is outside the image.

Parameters:

- offset** (*int*) – the offset to start reading at
- length** (*int*) – how many bytes to read
- advise_flags** (*int*) – advise flags for this read

Returns: str - the data read

Raises : **InvalidArgument, IOError**

Rebuilds the object map for the image HEAD or currently set snapshot

Delete a snapshot of the image.

Parameters: **name** (*str*) – the name of the snapshot

Raises : **IOError, ImageBusy**

rename a snapshot of the image.

Parameters: **srcname** (*str*) – the src name of the snapshot
 dstname (*str*) – the dst name of the snapshot

Raises : **ImageExists**

Change the size of the image.

Parameters: `size` (*int*) – the new size of the image

Revert the image to its contents at a snapshot. This is a potentially expensive operation, since it rolls back each object individually.

Parameters: **name** (*str*) – the snapshot to rollback to

Raises : **IOError**

Set the snapshot to read from. Writes will raise `ReadOnlyImage` while a snapshot is set. Pass `None` to unset the snapshot (reads come from the current image) , and allow writing again.

Parameters: **name** (*str or None*) – the snapshot to read from, or `None` to unset the snapshot

Get the size of the image. If open to a snapshot, returns the size of that snapshot.

Returns: the size of the image in bytes

Get information about the image. Currently parent pool and parent name are always - 1 and ''.

dict - contains the following keys:

`size` (int) - the size of the image in bytes

Returns: `obj_size` (int) - the size of each object that comprises the image

`num_objs` (int) - the number of objects in the image

`order` (int) - $\log_2(\text{object_size})$

`block_name_prefix` (str) - the prefix of the RADOS objects used to store the image

`parent_pool` (int) - deprecated

`parent_name` (str) - deprecated

See also `format()` and `features()`.

Returns the stripe count used for the image.

Returns the stripe unit used for the image.

Release a lock on the image that was locked by this rados client.

Mark a snapshot unprotected. This allows it to be deleted if it was protected.

Parameters: **name** (*str*) – the snapshot to unprotect

Raises : **IOError, ImageNotFound**

Updates the features bitmask of the image by enabling/disabling a single feature.
The feature must support the ability to be dynamically enabled/disabled.

features (*int*) – feature bitmask to enable/disable

Parameters: **enabled** (*bool*) – whether to enable/disable the feature

Raises : **InvalidArgument**

Write data to the image. Raises **InvalidArgument** if part of the write would fall outside the image.

Parameters: **data** (*bytes*) – the data to be written
offset (*int*) – where to start writing data
rs: **fadvise_flags** (*int*) – fadvise flags for this write

Returns: int - the number of bytes written

Raises : **IncompleteWriteError, LogicError, InvalidArgument, IOError**

```
class rbd.SnapIterator
```

SnapIterator(Image image)

Iterator over snapshot info for an image.

Yields a dictionary containing information about a snapshot.

Keys are:

- **id** (int) - numeric identifier of the snapshot
- **size** (int) - size of the image at the time of snapshot (in bytes)
- **name** (str) - name of the snapshot

CEPH 对象网关

Ceph 对象网关是一个构建在 `librados` 之上的对象存储接口，它为应用程序访问 Ceph 存储集群提供了一个 RESTful 风格的网关。**Ceph 对象存储**支持 2 种接口：

1. **兼容 S3:** 提供了对象存储接口，兼容 亚马逊 S3 RESTful 接口的一个大子集。
2. **兼容 Swift:** 提供了对象存储接口，兼容 Openstack Swift 接口的一个大子集。

Ceph 对象存储使用 Ceph 对象网关守护进程（`radosgw`），它是个与 Ceph 存储集群交互的 FastCGI 模块。因为它提供了与 OpenStack Swift 和 Amazon S3 兼容的接口，RADOS 要有它自己的用户管理。Ceph 对象网关可与 Ceph FS 客户端或 Ceph 块设备客户端共用一个存储集群。S3 和 Swift 接口共用一个通用命名空间，所以你可以用一个接口写如数据、然后用另一个接口取出数据。



Note

Ceph 对象存储**不**使用 Ceph 元数据服务器。

- [手动安装](#)
- [简单配置](#)
- [异地同步配置](#)
- [配置参考](#)
- [管理指南](#)
- [清除临时数据](#)
- [S3 API](#)
- [Swift API](#)

- [管理操作 API](#)
- [与 OpenStack Keystone 集成](#)
- [radosgw 手册页](#)
- [radosgw-admin 手册页](#)

安装 CEPH 对象网关

自从 *firefly* (v0.80) 版本开始，Ceph 对象网关运行在 Civetweb 上（已经集成进守护进程 `ceph-radosgw`），而不再是 Apache 和 FastCGI 之上。使用 Civetweb 简化了 Ceph 对象网关的安装和配置。

Note

要提供 Ceph 网关服务，你得有一个正常运行的 Ceph 集群，且网关主机能访问存储集群的公共网络。

Note

在 0.80 版本中，Ceph 对象网关不再支持 SSL。你可以设置一个支持 SSL 的反向代理服务器来将 HTTPS 请求转为 HTTP 请求发给 CivetWeb。

执行安装前的准备工作

首先进行 [环境检查](#)并在你的 Ceph 对象网关节点上执行安装前的准备工作。特别的，你需要禁用部署 Ceph 集群所用用户的 `requiretty`，同时设置 SELinux 为 `Permissive` 以及 Ceph 部署用户使用 `sudo` 时无需密码。对于 Ceph 对象网关，在生产环境下你需要开启 Civetweb 所使用的端口。

Note

Civetweb 默认运行在 `7480` 端口上。

安装 CEPH 对象网关

在你的管理节点的工作目录下，给 Ceph 对象网关节点安装 Ceph 对象所需的软件包。例如：

```
ceph-deploy install --rgw <gateway-node1> [<gateway-node2> ...]
```

`ceph-common` 包是它的一个依赖性，所以 `ceph-deploy` 也将安装这个包。`ceph` 的命令行工具就会为管理员准备好。为了让你的 Ceph 对象网关节点成为管理节点，可以在管理节点的工作目录下执行以下命令：

```
ceph-deploy admin <node-name>
```

新建网关实例

在你的管理节点的工作目录下，使用命令在 Ceph 对象网关节点上新建一个 Ceph 对象网关实例。举例如下：

```
ceph-deploy rgw create <gateway-node1>
```

在网关服务成功运行后，你可以使用未经授权的请求来访问端口 `7480`，就像这样：

```
http://client-node:7480
```

如果网关实例工作正常，你接收到的返回信息大概如下所示：

```
<?xml version="1.0" encoding="UTF-8"?>

<ListAllMyBucketsResult
  xmlns="http://s3.amazonaws.com/doc/2006-03-01/">

  <Owner>
```

```
<ID>anonymous</ID>

<DisplayName></DisplayName>

</Owner>

<Buckets>

</Buckets>

</ListAllMyBucketsResult>
```

在任何时候如果你遇到麻烦，而你也想重新来一次，执行下面的命令来清除配置：

```
ceph-deploy purge <gateway-node1> [<gateway-node2>]

ceph-deploy purgedata <gateway-node1> [<gateway-node2>]
```

如果你执行了 `purge`，你必须重新安装 Ceph。

修改默认端口

Civetweb 默认运行在端口 7480 之上。如果想修改这个默认端口（比如使用端口 80），修改你的管理节点的工作目录下的 Ceph 配置文件。添加一节，以 `[client.rgw.<gateway-node>]` 作为名字，使用你的 Ceph 对象网关节点的短主机名替换其中的 `<gateway-node>`（如，`hostname -s`）。

Note

在 0.94 版本中，Ceph 对象网关不再支持 SSL。你可以设置一个支持 SSL 的反向代理服务器来将 HTTPS 请求转为 HTTP 请求发给 CivetWeb。

比如，如果你的主机名是 `gateway-node1`，在 `[global]` 节后添加的节名如下：

```
[client.rgw.gateway-node1]  
  
rgw_frontends = "civetweb port=80"
```

Note

请确保在 `rgw_frontends` 的键值对中 `port=<port-number>` 内部没有红格。节点名 `[client.rgw.gateway-node1]` 表示接下来的 Ceph 配置文件配置了一个 Ceph 存储集群的客户端, 这个客户端的类型是 Ceph 对象网关存储 (比如, `rgw`), 同时实例名是 `gateway-node1`.

将该配置文件推送到你的 Ceph 对象网关节点(也包括其他 Ceph 节点):

```
ceph-deploy --overwrite-conf config push <gateway-node>  
[<other-nodes>]
```

为了使新配置的端口生效, 需要重启 Ceph 对象网关:

```
sudo systemctl restart ceph-radosgw.service
```

最后, 需要你选择的端口在节点的防火墙配置中是开放的 (比如, 端口 `80`)。如果它不是开放的, 将它设为开放并重启防火墙。如果你用的是 `firewalld`, 执行下面的命令:

```
sudo firewall-cmd --list-all  
  
sudo firewall-cmd --zone=public --add-port 80/tcp --  
permanent  
  
sudo firewall-cmd --reload
```

如果你使用 `iptables`, 执行:

```
sudo iptables --list
```

```
sudo iptables -I INPUT 1 -i <iface> -p tcp -s <ip-address>/<netmask> --dport 80 -j ACCEPT
```

替换其中的 `<iface>` and `<ip-address>/<netmask>` 为你的 Ceph 对象网关节点的实际值。

一旦你完成了 `iptables` 的配置，请确保你已经将本次更改持久化，这样它在你的 Ceph 对象网关节点重启也依然生效。持久化请执行：

```
sudo apt-get install iptables-persistent
```

会有一个终端界面弹出。选择 `yes` 来保存当前 IPv4 的 `iptables` 规则到文件 `/etc/iptables/rules.v4` 中，当前的 IPv6 `iptables` 规则到文件 `/etc/iptables/rules.v6` 中。

在前面一步设置的 IPv4 `iptables` 规则将会被写入文件 `"/etc/iptables/rules.v4"`，这样就实现了持久化，重启也依然生效。

如果你在安装 `iptables-persistent` 后有新增了新的 IPv4 `iptables` 规则，你需要使用 `root` 执行下面的命令：

```
iptables-save > /etc/iptables/rules.v4
```

从 APACHE 迁移到 CIVETWEB

如果你在 0.80 或以上版本的 Ceph 存储上使用 Apache 和 FastCGI 来构建 Ceph 对象网关，实际上你已经运行了 Civetweb –它随着 `ceph-radosgw` 进程启动而启动，默认情况下运行在 7480 端口，因此它不会跟基于 Apache 和 FastCGI 安装的以及其他常用的 web 服务所使用的端口冲突。迁移到使用 Civetweb 首先需要删除您的 Apache。然后，您必须从 Ceph 配置文件中删除 Apache 和 FastCGI 的设置并重置 `rgw_frontends` 到 Civetweb。

回顾使用 `ceph-deploy` 来安装 Ceph 对象网关的描述，注意到它的配置文件只有一个名为 `rgw_frontends` 的设置(这是假设你选择改变默认端口)。`ceph-deploy` 会生成对象网关的数据目录和 keyring，并将 keyring 放在 `/var/lib/ceph/radosgw/{rgw-`

instance} 目录下。从这个目录上看这个守护进程是在默认位置，然而你也可以在 Ceph 配置文件中指定不同的位置。因为你已经有密钥和数据目录，因此如果你使用的是默认路径之外的其他目录，你就需要在 Ceph 配置文件中维护这些路径。

一个典型的基于 Apache 部署的 Ceph 对象网关配置文件类似下面的样例：

在 Red Hat Enterprise Linux 系统上：

```
[client.radosgw.gateway-node1]

host = {hostname}

keyring = /etc/ceph/ceph.client.radosgw.keyring

rgw socket path = ""

log file = /var/log/radosgw/client.radosgw.gateway-
node1.log

rgw frontends = fastcgi socket\_port=9000
socket\_host=0.0.0.0

rgw print continue = false
```

在 Ubuntu 系统上：

```
[client.radosgw.gateway-node]

host = {hostname}

keyring = /etc/ceph/ceph.client.radosgw.keyring

rgw socket path =
/var/run/ceph/ceph.radosgw.gateway.fastcgi.sock

log file = /var/log/radosgw/client.radosgw.gateway-
node1.log
```

为了能够使用 Civetweb，需要做一点修改，简单地删除 Apache 特有的设置如 `rgw_socket_path` 和 `rgw_print_continue`。然后，改变 `rgw_frontends` 的设置来指向 Civetweb 而不是 Apache FastCGI 作为前端，并指定您想要使用的端口号。例如：

```
[client.radosgw.gateway-node1]

host = {hostname}

keyring = /etc/ceph/ceph.client.radosgw.keyring

log file = /var/log/radosgw/client.radosgw.gateway-
node1.log

rgw_frontends = civetweb port=80
```

最后，重启 Ceph 对象网关。在 Red Hat Enterprise Linux 请执行：

```
sudo systemctl restart ceph-radosgw.service
```

在 Ubuntu 上执行：

```
sudo service radosgw restart id=rgw.<short-hostname>
```

如果你使用的端口还没有在防护墙端放开，你还需要在你的防火墙开放该端口。

配置 BUCKET SHARDING

Ceph 对象网关在 `index_pool` 中存储 bucket 的索引数据，默认情况下是资源池 `.rgw.buckets.index`。有时用户喜欢把很多对象(几十万到上百万的对象)存放到一个 bucket 中。如果你不使用网关的管理接口来为每个 bucket 的最大对象数设置配额，那么当一旦用户存放大量的对象到一个 bucket 中时，bucket 索引的性能会呈现明显的下降。

在 0.94 版本的 Ceph 中，您可以给 bucket 索引进行分片，这样在你允许 bucket 中有大量对象时，能够有助于防止出现性能瓶颈。设置项的

“`rgw_override_bucket_index_max_shards`” 允许您设置一个 bucket 的最大分片数。它的默认值为 0，这意味着 bucket 索引分片功能在默认情况下是关闭的。

开启 bucket 的索引分片功能，只需

给 `rgw_override_bucket_index_max_shards` 设置一个大于 0 的值。

简单的配置，只需要在 Ceph 配置文件中加

入 `rgw_override_bucket_index_max_shards`。将其添加在 `[global]` 部分来设置一个系统层面生效的值。你也可以在 Ceph 配置文件中将它设置为某一个实例生效。

一旦你在 Ceph 配置文件中修改了你的 bucket 分片设置，你需要重启网关。在 Red Hat Enterprise Linux 请执行：

```
sudo systemctl restart ceph-radosgw.service
```

在 Ubuntu 上请执行：

```
sudo service radosgw restart id=rgw.<short-hostname>
```

对于异地场景的配置而言，为了灾备每一个 zone 都有一个不同的 `index_pool` 设置。为了保持这个参数在一个 region 的所有 zone 中保持一致，你可以在 region 的网关配置中指定 `rgw_override_bucket_index_max_shards`。举例如下：

```
radosgw-admin region get > region.json
```

打开 `region.json` 的文件，为每一个 zone 编辑 `bucket_index_max_shards` 的设置。保存 `region.json` 文件并重置 region。举例如下：

```
radosgw-admin region set < region.json
```

一旦你更新了你的 region，你需要更新 region map。举例如下：


```
radosgw-admin regionmap update --name client.rgw.ceph-client
```

其中的 `client.rgw.ceph-client` 是网关用户的名字。

Note

通过 CRUSH 规则集将索引资源池 (如果可以为每一个 zone 设置) 映射到基于 SSD 的 OSD 上也能够提升 bucket 索引的性能。

给 DNS 添加泛域名解析

为了通过 S3 风格的子域名来使用 Ceph(如：bucket-name.domain-name.com)，你应该给你的 ceph-radosgw 守护进程所在服务器在 DNS 服务器上给它的 DNS 记录添加泛域名解析。

这个 DNS 地址也必须在 Ceph 配置文件中通过 `rgw dns name = {hostname}` 设置项来指定。

对于 `dnsmasq` 而言，通过在主机名前添加点号 (.) 来实现添加 DNS 地址设置：

```
address=/{hostname-or-fqdn}/{host-ip-address}
```

举例如下:

```
address=/.gateway-node1/192.168.122.75
```

对于 bind 而言，给 DNS 记录添加一个泛域名解析。举例如下：

```
$TTL      604800

@          IN      SOA      gateway-node1. root.gateway-node1.
(
                                2          ; Serial
```

```

                                604800          ; Refresh
                                86400           ; Retry
                                2419200        ; Expire
                                604800 )       ; Negative Cache
TTL
;
@      IN      NS      gateway-node1.
@      IN      A       192.168.122.113
*      IN      CNAME   @

```

重启你的 DNS 服务，然后使用子域名 ping 你的服务器来确保你的 `ceph-radosgw` 进程能够出来子域名请求：

```
ping mybucket.{hostname}
```

举例如下：

```
ping mybucket.gateway-node1
```

开启 DEBUGGING (如果需要)

一旦你完成了安装过程，如果在你的配置下遇到了问题，你可以在 Ceph 配合文件的 `[global]` 小节下面添加调试选项，然后重启网关服务，进而帮助解决配置过程出现的问题。例如：

```

[global]

#append the following in the global section.

```

```
debug ms = 1

debug rgw = 20
```

使用网关

为了使用 REST 接口，首先需要为 S3 接口创建一个初始 Ceph 对象网关用户。然后，为 Swift 接口创建一个子用户。然后你需要验证创建的用户是否能够访问网关。

为 S3 访问创建 RADOSGW 用户

一个 ``radosgw`` 用户需要被新建并被分配权限。命令 `man radosgw-admin` 会提供该命令的额外信息。

为了新建用户，在 `gateway host` 上执行下面的命令：

```
sudo radosgw-admin user create --uid="testuser" --
display-name="First User"
```

命令的输出跟下面的类似：

```
{

    "user_id": "testuser",

    "display_name": "First User",

    "email": "",

    "suspended": 0,

    "max_buckets": 1000,

    "auid": 0,

    "subusers": [],
```

```
"keys": [{
    "user": "testuser",
    "access_key": "I0PJDPciYZ665MW88W9R",
    "secret_key":
"dxaxZ8U90SXydYzyS5ivamEP20hkLSUViiaR+ZDA"
}],
"swift_keys": [],
"caps": [],
"op_mask": "read, write, delete",
"default_placement": "",
"placement_tags": [],
"bucket_quota": {
    "enabled": false,
    "max_size_kb": -1,
    "max_objects": -1
},
"user_quota": {
    "enabled": false,
    "max_size_kb": -1,
    "max_objects": -1
}
```

```
    },  
  
    "temp_url_keys": []  
  
}
```

Note

其中 `keys->access_key` 和 `keys->secret_key` 的值在访问的时候需要用来做验证

Important

请检查输出的 key。有个时候 `radosgw-admin` 会在生成的 JSON 中的 `access_key` 和 `secret_key` 部分包含有转义字符 `\`，并且一些客户端不知道如何处理 JSON 中的这个字符。补救措施包括移除 JSON 中的 `\` 字符，将该字符串封装到引号中，重新生成这个 key 并确保不再包含 `\`，或者手动指定 `access_key` 和 `secret_key`。如果 `radosgw-admin` 生成的 JSON 中的同一个 key 中包含转义字符 `\` 同时包含有正斜杠 `/` 形如 `\\/`，请只移除 JSON 转义字符 `\`，不要删除正斜杠 `/`，因为在 key 中它是一个有效字符。

新建一个 SWIFT 用户

如果你想要使用这种方式访问集群，你需要新建一个 Swift 子用户。创建 Swift 用户包括两个步骤。第一步是创建用户。第二步是创建 secret key。

在 `gateway host` 上执行喜爱按的步骤：

新建 Swift 用户：

```
sudo radosgw-admin subuser create --uid=testuser --  
subuser=testuser:swift --access=full
```

输出类似下面这样：

```
{

    "user_id": "testuser",

    "display_name": "First User",

    "email": "",

    "suspended": 0,

    "max_buckets": 1000,

    "auid": 0,

    "subusers": [{

        "id": "testuser:swift",

        "permissions": "full-control"

    }],

    "keys": [{

        "user": "testuser:swift",

        "access_key": "3Y1LNW4Q6X0Y53A52DET",

        "secret_key": ""

    }, {

        "user": "testuser",

        "access_key": "I0PJDPICIYZ665MW88W9R",

        "secret_key":

"dxaxZ8U90SXdyZzyS5ivamEP20hkLSUViiaR+ZDA"
```

```
    }],  
  
    "swift_keys": [],  
  
    "caps": [],  
  
    "op_mask": "read, write, delete",  
  
    "default_placement": "",  
  
    "placement_tags": [],  
  
    "bucket_quota": {  
  
        "enabled": false,  
  
        "max_size_kb": -1,  
  
        "max_objects": -1  
  
    },  
  
    "user_quota": {  
  
        "enabled": false,  
  
        "max_size_kb": -1,  
  
        "max_objects": -1  
  
    },  
  
    "temp_url_keys": []  
  
}
```

新建 secret key:

```
sudo radosgw-admin key create --subuser=testuser:swift --  
key-type=swift --gen-secret
```

输出类似下面这样:

```
{  
  
  "user_id": "testuser",  
  
  "display_name": "First User",  
  
  "email": "",  
  
  "suspended": 0,  
  
  "max_buckets": 1000,  
  
  "auid": 0,  
  
  "subusers": [{  
  
    "id": "testuser:swift",  
  
    "permissions": "full-control"  
  
  }],  
  
  "keys": [{  
  
    "user": "testuser:swift",  
  
    "access_key": "3Y1LNW4Q6X0Y53A52DET",  
  
    "secret_key": ""  
  
  }, {  
  
    "user": "testuser",
```



```
        "access_key": "I0PJDPICIYZ665MW88W9R",

        "secret_key":
"dxaXZ8U90SXydYzyS5ivamEP20hkLSUViiaR+ZDA"

    }],

    "swift_keys": [{

        "user": "testuser:swift",

        "secret_key":
"244+fz2gSqoHwR3lYtSbIyomyPHf3i7rgSJrF\//IA"

    }],

    "caps": [],

    "op_mask": "read, write, delete",

    "default_placement": "",

    "placement_tags": [],

    "bucket_quota": {

        "enabled": false,

        "max_size_kb": -1,

        "max_objects": -1

    },

    "user_quota": {

        "enabled": false,

        "max_size_kb": -1,
```

```
        "max_objects": -1

    },

    "temp_url_keys": []

}
```

访问验证

测试 S3 访问

为了验证 S3 访问，你需要编写并运行一个 Python 测试脚本。S3 访问测试脚本将连接 `radosgw`，新建一个新的 bucket 并列出现有的 buckets。

`aws_access_key_id` 和 `aws_secret_access_key` 的值来自于命令 `radosgw_admin` 的返回值 `access_key` 和 `secret_key`。

执行下面的步骤：

1. 你需要安装 `python-boto` 包：

```
sudo yum install python-boto
```

2. 新建 Python 脚本文件：

```
vi s3test.py
```

3. 将下面的内容添加到文件中：

```
4. import boto

5. import boto.s3.connection

6.
```

```

7. access_key = 'I0PJDPICIYZ665MW88W9R'

8. secret_key =
    'dxaXZ8U90SXydYzyS5ivamEP20hkLSUViiaR+ZDA'

9. conn = boto.connect_s3(

10.         aws_access_key_id = access_key,

11.         aws_secret_access_key = secret_key,

12.         host = '{hostname}', port = {port},

13.         is_secure=False, calling_format =
            boto.s3.connection.OrdinaryCallingFormat(),

14.         )

15.

16.     bucket = conn.create_bucket('my-new-bucket')

17.     for bucket in conn.get_all_buckets():

18.         print "{name}".format(

19.             name = bucket.name,

20.             created =
                bucket.creation_date,

```

```
)
```

将 {hostname} 替换为你配置了网关服务的节点的主机名。比如 gateway host。
将 {port} 替换为 Civetweb 所使用的端口。

21. 运行脚本:

```
python s3test.py
```

输出类似下面的内容:

```
my-new-bucket 2015-02-16T17:09:10.000Z
```

测试 *SWIFT* 访问

Swift 访问的验证则可以使用 ``swift`` 的命令行客户端。可以通过命令 `man swift` 获取更多命令行选项的更多信息。

执行下面的命令安装 `swift` 客户端，在 Red Hat Enterprise Linux 上执行:

```
sudo yum install python-setuptools  
  
sudo easy_install pip  
  
sudo pip install --upgrade setuptools  
  
sudo pip install --upgrade python-swiftclient
```

在基于 Debian 的发行版上执行:

```
sudo apt-get install python-setuptools  
  
sudo easy_install pip  
  
sudo pip install --upgrade setuptools  
  
sudo pip install --upgrade python-swiftclient
```

执行下面的命令验证 `swift` 访问:

```
swift -A http://{IP ADDRESS}:{port}/auth/1.0 -U  
testuser:swift -K '{swift_secret_key}' list
```

使用网关服务器的外网 IP 地址替换其中的 {IP ADDRESS}，使用新建 swift 用户时执行的命令 `radosgw-admin key create` 的输出替换其中的 {swift_secret_key}。使用你的 Civetweb 所使用的端口替换其中 {port}，比如默认是 7480。如果你不替换这个端口，它的默认值是 80。举例如下：

```
swift -A http://10.19.143.116:7480/auth/1.0 -U  
testuser:swift -K  
'244+fz2gSqoHwR3lYtSbIyomyPHf3i7rgSJrF/IA' list
```

输出类似下面这样：

```
my-new-bucket
```

配置 CEPH 对象网关

要配置一个 Ceph 对象网关需要一个运行着的 Ceph 存储集群，以及启用了 FastCGI 模块的 Apache web 服务器。

Ceph 对象网关是 Ceph 存储集群的一个客户端，作为 Ceph 存储集群的客户端，它需要：

- 需要为网关实例配置一个名字，在本手册中我们使用 `gateway`。
- 存储集群的一个用户名，并且该用户在 `keyring` 中有合适的权限。
- 存储数据的资源池。
- 网关实例的一个数据目录。
- Ceph 配置文件中有一个实例配置入口。
- web 服务器有一个配置文件跟 FastCGI 交互。

新建用户和 KEYRING

每一个实例必须有一个用户名和 key 来跟 Ceph 存储集群通信.在下面的步骤中,我们使用管理节点来新建 keyring.然后,我们新建一个客户端用户名和 key.然后我们将这个 key 添加到 Ceph 存储集群中.最后,分发这个 key ring 到包含这个网关实例的节点.

Monitor Key CAPS

当你给 key 分配 CAPS 的时候,你必须提供读权限.然而,你也可以选择为 monitor 提供写的权限.这是一个重要的抉择.如果你给 key 分配了写权限, Ceph 对象网关将具备自动新建资源池的能力;此时,它将会根据默认的 PG 值(不是最合适的)或者你在 Ceph 配置文件中指定的 PG 数.如果你允许 Ceph 对象网关自动新建资源池,请确保你首先定义好了合理的默认 PG 值.查看[资源池配置](#)获取详细信息.

关于 Ceph 认证请参考[用户管理](#)。

1. 为每一个实例生成一个 Ceph 对象网关用户名和 key. 举一个典型实例,我们将使用 `client.radosgw` 后使用 `gateway` 作为用户名:

```
sudo ceph auth get-or-create client.radosgw.gateway
osd 'allow rwx' mon 'allow rwx' -o
/etc/ceph/ceph.client.radosgw.keyring
```

2. 分发生成的 keyring 到网关实例所在节点.

```
3. sudo scp /etc/ceph/ceph.client.radosgw.keyring
ceph@{hostname}:/home/ceph

4. ssh {hostname}
```

```
sudo mv ceph.client.radosgw.keyring
/etc/ceph/ceph.client.radosgw.keyring
```

Note

如果 `admin node` 就是 `gateway host`，那第 2 步就没必要。

创建存储池

Ceph 对象网关需要 Ceph 存储集群资源池来存储特定的网关数据。如果你新建的用户有权限，网关将会自动新建所需资源池。然而，你需要确保在你的 Ceph 配置文件中给资源池设置了合理的默认 PG 值。

Note

Ceph 对象网关有多个存储池，考虑到所有存储池会共用相同的 CRUSH 分级结构，所以 PG 数不要设置得过大，否则会影响性能。

当使用默认的 `region` 和 `zone` 时，资源池的命名规则通常省略了 `region` 和 `zone` 的名字，但是你可以使用任何你想要的命名规则。举例如下：

- `.rgw`
- `.rgw.root`
- `.rgw.control`
- `.rgw.gc`
- `.rgw.buckets`
- `.rgw.buckets.index`
- `.rgw.buckets.extra`
- `.log`
- `.intent-log`
- `.usage`
- `.users`
- `.users.email`
- `.users.swift`
- `.users.uid`

查看 [配置参考——存储池](#) 获取关于网关默认资源池的详细信息。查看 [资源池](#) 获取新建资源池的详细信息。正如前面所说，如果具备写权限，Ceph 对象网关将会自动新建资源池。如果想手动新建资源池，执行下面的命令：

```
ceph osd pool create {poolname} {pg-num} {pgp-num}
{replicated | erasure} [{erasure-code-profile}]
{ruleset-name} {ruleset-number}
```

Tip

Ceph 允许多级 CRUSH 和多种 CRUSH 规则集，这样在配置你自己的网关时就有很大的灵活性。像 `rgw.buckets.index` 这样的存储池就可以利用 SSD 来做存储池以获取高性能；后端存储也可以从更经济的纠删编码的存储中获益，还可利用缓存层提升性能。

完成上述步骤之后，执行下列的命令确认前述存储池都已经创建了：

```
rados lspools
```

为 CEPH 新增网关配置

添加 Ceph 对象网关配置到 `admin node` 节点的 Ceph 配置文件中。Ceph 对象网关配置需要你指定 Ceph 对象网关实例。然后你必须指定你安装有 Ceph 对象网关守护进程的节点的主机名，一个 keyring (为了使用 `cephx`)，FastCGI 的 socket 路径和日志文件。

对于使用 Apache 2.2 和 Apache 2.4 早期版本 (RHEL 6, Ubuntu 12.04, 14.04 etc) 的发行版，追加下面的配置到 `admin node` 的文件 `/etc/ceph/ceph.conf` 中：

```
[client.radosgw.gateway]

host = {hostname}

keyring = /etc/ceph/ceph.client.radosgw.keyring

rgw socket path = ""

log file = /var/log/radosgw/client.radosgw.gateway.log

rgw frontends = fastcgi socket_port=9000
socket_host=0.0.0.0
```



```
rgw print continue = false
```

Note

Apache 2.2 和早期 Apache 2.4 版本不使用 Unix Domain Sockets 而是用 localhost TCP.

对于使用 Apache 2.4.9 或者更新版版本的发行版 (RHEL 7, CentOS 7 等), 追加下面的配置到 admin node 的文件 `/etc/ceph/ceph.conf` 中:

```
[client.radosgw.gateway]

host = {hostname}

keyring = /etc/ceph/ceph.client.radosgw.keyring

rgw socket path =
/var/run/ceph/ceph.radosgw.gateway.fastcgi.sock

log file = /var/log/radosgw/client.radosgw.gateway.log

rgw print continue = false
```

Note

Apache 2.4.9 支持 Unix Domain Socket (UDS), 但是 Ubuntu 14.04 附带的 Apache 2.4.7 不支持 UDS, 并且默认配置使用 localhost TCP. 在“Ubuntu 14.04”中的 Apache 2.4.7 已经发现一个 bug, 并且已经申请一个补丁以支持 UDS. 查看: [Ubuntu Trusty 中支持 UDS 的补丁](#)

这里, {hostname} 是即将提供网关服务的节点的主机名 (`hostname -s` 命令的输出), 比如 gateway host. 网关实例的 `[client.radosgw.gateway]` 部分标识了 Ceph 配置文件的这个部分是用来配置 Ceph 存储集群的一个客户端的, 这个客户端的类型是 Ceph 对象网关 (比如, radosgw).

Note

配置文件的最后一行 `rgw print continue = false` 是用来避免 PUT 操作可能出现的问题。

一旦你完成了这些安装过程, 如果你所做的配置遇到了问题, 你可以在 Ceph 配置文件的 `[global]` 部分添加调试选项, 然后重启你的网关来帮忙排错, 找到可能的配置问题. 举例如下:

```
[global]

#append the following in the global section.

debug ms = 1

debug rgw = 20
```

分发更新后的 CEPH 配置文件

更新后的 Ceph 配置文件需要从 `admin node` 分发到 Ceph 集群节点上.

它包含以下步骤:

1. 将更新后 `ceph.conf` 从 `/etc/ceph/` 目录拷贝到管理节点上新建集群时的根目录中 (比如 `my-cluster` 目录). 因此 `my-cluster` 中 `ceph.conf` 将会被覆盖. 为此, 执行下面的命令:

```
ceph-deploy --overwrite-conf config pull {hostname}
```

这里, `{hostname}` 是 Ceph 管理节点的精简主机名.

2. 将更新后 `ceph.conf` 文件从管理节点推送到集群中其他所有节点, 包含 gateway host:

```
ceph-deploy --overwrite-conf config push [HOST]  
[HOST...]
```

使用其他 Ceph 节点的主机名代替上面命令中的 [HOST] [HOST...].

从管理节点拷贝 CEPH.CLIENT.ADMIN.KEYRING 到网关主机

因为 gateway host 有可能是集群之外的其他节点,所以需要
将 ceph.client.admin.keyring 从 admin node 拷贝到 gateway host. 为此,在
admin node 上执行下面的命令:

```
sudo scp /etc/ceph/ceph.client.admin.keyring  
ceph@{hostname}:/home/ceph  
  
ssh {hostname}  
  
sudo mv ceph.client.admin.keyring  
/etc/ceph/ceph.client.admin.keyring
```

Note

如果你的 admin node 就是 gateway host 则无需执行上面的步骤.

新建数据目录

部署脚本不会创建默认的 Ceph 对象网关数据目录. 因此需要为每一个 radosgw 守护进程实例新建数据目录. Ceph 配置文件中的 host 变量决定了在哪个主机上运行 radosgw 守护进程实例. 数据目录的典型命名规则是指定 radosgw 守护进程, 集群名和守护进程 ID.

执行下面的命令在 gateway host 上新建所需的数据目录:

```
sudo mkdir -p /var/lib/ceph/radosgw/ceph-radosgw.gateway
```

调整 SOCKET 目录权限

在一些发行版中, `radosgw` 守护进程是以没有什么权利的 UID 为 `apache` 的用户运行的, 但是这个 UID 必须在运行时写入 `socket` 文件的目录中具备写权限.

在 `gateway host` 上执行下面的命令来给上述 UID 授予默认 `socket` 位置的权限:

```
sudo chown apache:apache /var/run/ceph
```

改变日志文件所有者

在一些发行版中, `radosgw` 守护进程是以没有什么权利的 UID 为 `apache` 的用户运行的, 但是日志文件默认是 `root` 用户所有的. 你必须日志文件的将所有者修改 `apache` 用户, 以便 `Apache` 可以在这里写入日志文件. 为此, 执行下面的命令:

```
sudo chown apache:apache  
/var/log/radosgw/client.radosgw.gateway.log
```

启动 RADOSGW 服务

Ceph 对象网关守护进程需要启动. 为此, 在 `gateway host` 上执行下面的命令:

在基于 Debian 的发行版上:

```
sudo /etc/init.d/radosgw start
```

在基于 RPM 的发行版上:

```
sudo /etc/init.d/ceph-radosgw start
```

新建一个网关配置文件

在你安装了 Ceph 对象网关的主机上, 比如 gateway host, 新建一个 rgw.conf 文件, 对于 Debian-based 发行版将该文件放在 /etc/apache2/conf-available 目录下, 对于 RPM-based 发行版则将其放在 /etc/httpd/conf.d 目录下. 这个是 radosgw 所需的一个 Apache 配置文件. 对于 web 服务器而言这个文件必须是可读的.

按照下面的步骤执行:

1. 新建文件:

对于基于 Debian 的发行版, 执行:

```
sudo vi /etc/apache2/conf-available/rgw.conf
```

对于基于 RPM 的发行版, 执行:

```
sudo vi /etc/httpd/conf.d/rgw.conf
```

2. 对于使用 Apache 2.2 或者 Apache 2.4 早期版本的使用 localhost TCP 并且不支持 Unix Domain Socket 的发行版而言, 添加下面的内容到该文件中:

```
3. <VirtualHost *:80>

4. ServerName localhost

5. DocumentRoot /var/www/html

6.

7. ErrorLog /var/log/httpd/rgw_error.log

8. CustomLog /var/log/httpd/rgw_access.log combined

9.
```

```
10.      # LogLevel debug
11.
12.      RewriteEngine On
13.
14.      RewriteRule .* -
        [E=HTTP_AUTHORIZATION:%{HTTP:Authorization},L]
15.
16.      SetEnv proxy-nokeepalive 1
17.
18.      ProxyPass / fcgi://localhost:9000/
19.
```

```
</VirtualHost>
```

Note

For Debian-based distros

replace `/var/log/httpd/` with `/var/log/apache2.`

20. 对于使用 Apache 2.4.9 或者更新版本的支持 Unix Domain Socket 的发行版而言，添加下面的内容到该文件中 add the following contents to the file:

```
21.      <VirtualHost *:80>
22.      ServerName localhost
```

```
23.      DocumentRoot /var/www/html

24.

25.      ErrorLog /var/log/httpd/rgw_error.log

26.      CustomLog /var/log/httpd/rgw_access.log
        combined

27.

28.      # LogLevel debug

29.

30.      RewriteEngine On

31.

32.      RewriteRule .* -
        [E=HTTP_AUTHORIZATION:%{HTTP:Authorization},L]

33.

34.      SetEnv proxy-nokeepalive 1

35.

36.      ProxyPass /
        unix:///var/run/ceph/ceph.radosgw.gateway.fastcgi.sock|fcgi://localhost:9000/

37.
```

```
</VirtualHost>
```

重启 APACHE

Apache 服务需要重启来加载新的配置文件.

对于基于 Debian 的发行版, 执行:

```
sudo service apache2 restart
```

对于基于 RPM 的发行版, 执行:

```
sudo service httpd restart
```

或者:

```
sudo systemctl restart httpd
```

使用网关

为了使用 REST 接口, 首先需要为 S3 接口初始化一个 Ceph 对象网关用户. 然后为 Swift 接口新建一个子用户. 查看 [管理手册](#) 获取有关用户管理的详细信息.

为 S3 访问新建一个 RADOSGW 用户

A `radosgw` 用户需要新建并且赋予访问权限. 命令 `man radosgw-admin` 将展示提供更多额外的命令选项信息.

为了新建用户, 在 `gateway host` 上执行下面的命令:

```
sudo radosgw-admin user create --uid="testuser" --  
display-name="First User"
```

上述命令的输出结果类似下面这样:


```
{ "user_id": "testuser",  
  
  "display_name": "First User",  
  
  "email": "",  
  
  "suspended": 0,  
  
  "max_buckets": 1000,  
  
  "auid": 0,  
  
  "subusers": [],  
  
  "keys": [  
  
    { "user": "testuser",  
  
      "access_key": "I0PJDPICIYZ665MW88W9R",  
  
      "secret_key":  
      "dxaXZ8U90SXydYzyS5ivamEP20hkLSUViiaR+ZDA"}],  
  
  "swift_keys": [],  
  
  "caps": [],  
  
  "op_mask": "read, write, delete",  
  
  "default_placement": "",  
  
  "placement_tags": [],  
  
  "bucket_quota": { "enabled": false,  
  
  "max_size_kb": -1,  
  
  "max_objects": -1},
```

```
"user_quota": { "enabled": false,  
  
"max_size_kb": -1,  
  
"max_objects": -1},  
  
"temp_url_keys": []}
```

Note

keys->access_key 和 keys->secret_key 两个值在访问时是必需的，用来验证。

创建一个 SWIFT 用户

如果要通过 Swift 访问，必须创建一个 Swift 子用户。需要分两步完成，第一步是创建用户，第二步创建密钥。

在 gateway host 主机上进行如下操作：

创建 Swift 用户：

```
sudo radosgw-admin subuser create --uid=testuser --  
subuser=testuser:swift --access=full
```

此命令的输出类似如下：

```
{ "user_id": "testuser",  
  
"display_name": "First User",  
  
"email": "",  
  
"suspended": 0,  
  
"max_buckets": 1000,
```

```
"auid": 0,

"subusers": [

{ "id": "testuser:swift",

"permissions": "full-control"}],

"keys": [

{ "user": "testuser:swift",

"access_key": "3Y1LNW4Q6X0Y53A52DET",

"secret_key": ""},

{ "user": "testuser",

"access_key": "I0PJDPICYZ665MW88W9R",

"secret_key":

"dxaxZ8U90SXydyzyS5ivamEP20hkLSUViiaR+ZDA"}],

"swift_keys": [],

"caps": [],

"op_mask": "read, write, delete",

"default_placement": "",

"placement_tags": [],

"bucket_quota": { "enabled": false,

"max_size_kb": -1,

"max_objects": -1},
```

```
"user_quota": { "enabled": false,  
  
"max_size_kb": -1,  
  
"max_objects": -1},  
  
"temp_url_keys": []}
```

创建用户的密钥：

```
sudo radosgw-admin key create --subuser=testuser:swift --  
key-type=swift --gen-secret
```

此命令的输出类似如下：

```
{ "user_id": "testuser",  
  
"display_name": "First User",  
  
"email": "",  
  
"suspended": 0,  
  
"max_buckets": 1000,  
  
"auid": 0,  
  
"subusers": [  
  
{ "id": "testuser:swift",  
  
"permissions": "full-control"}],  
  
"keys": [  
  
{ "user": "testuser:swift",
```

```
"access_key": "3Y1LNW4Q6X0Y53A52DET",

"secret_key": "",

{ "user": "testuser",

"access_key": "I0PJDPICYZ665MW88W9R",

"secret_key":
"dxaXZ8U90SXydYzyS5ivamEP20hkLSUViiaR+ZDA"}}],

"swift_keys": [

{ "user": "testuser:swift",

"secret_key":
"244+fz2gSqoHwR3lYtSbIyomyPHf3i7rgSJrF\//IA"}}],

"caps": [],

"op_mask": "read, write, delete",

"default_placement": "",

"placement_tags": [],

"bucket_quota": { "enabled": false,

"max_size_kb": -1,

"max_objects": -1},

"user_quota": { "enabled": false,

"max_size_kb": -1,

"max_objects": -1},

"temp_url_keys": []}
```

访问验证

然后你得验证一下刚创建的用户是否能访问网关。

测试 S3 访问

你需要写一个 Python 测试脚本,并运行它以验证 S3 访问. S3 访问测试脚本将会连接 `radosgw`, 然后新建一个新的 bucket 再列出所有的 buckets. `aws_access_key_id` 和 `aws_secret_access_key` 的值就是前面 `radosgw_admin` 命令的返回值中的 `access_key` 和 `secret_key`.

执行下面的步骤:

1. 首先你需要安装 `python-boto` 包.

对于基于 Debian 的发行版请执行:

```
sudo apt-get install python-boto
```

对于基于 RPM 的发行版请执行:

```
sudo yum install python-boto
```

2. 新建 Python 脚本:

```
vi s3test.py
```

3. 添加下面的内容到该文件中:

```
4. import boto

5. import boto.s3.connection

6. access_key = 'I0PJDPICIY2665MW88W9R'
```

```

7. secret_key =
    'dxaXZ8U90SXydYzyS5ivamEP20hkLSUViiaR+ZDA'

8. conn = boto.connect_s3(

9.     aws_access_key_id = access_key,

10.     aws_secret_access_key = secret_key,

11.     host = '{hostname}',

12.     is_secure=False,

13.     calling_format =
        boto.s3.connection.OrdinaryCallingFormat(),

14. )

15.     bucket = conn.create_bucket('my-new-bucket')

16.     for bucket in conn.get_all_buckets():

17.         print "{name}\t{created}".format(

18.             name = bucket.name,

19.             created = bucket.creation_date,

20. )

```

将 {hostname} 替换为你配置了网关服务的主机的主机名,比如 gateway host.

21. 运行这个脚本:

```
python s3test.py
```

输出类似下面的内容:

```
my-new-bucket 2015-02-16T17:09:10.000Z
```

测试 SWIFT 访问

Swift 访问能够通过 `swift` 命令行客户端来验证. 命令 `man swift` 将提供更多、可用命令行选项的详细信息.

执行下面的命令安装 ``swift`` 客户端:

对于基于 Debian 的发行版:

```
sudo apt-get install python-setuptools

sudo easy_install pip

sudo pip install --upgrade setuptools

sudo pip install --upgrade python-swiftclient
```

对于基于 RPM 的发行版:

```
sudo yum install python-setuptools

sudo easy_install pip

sudo pip install --upgrade setuptools

sudo pip install --upgrade python-swiftclient
```

执行下面的命令验证 swift 访问:

```
swift -A http://{IP ADDRESS}/auth/1.0 -U testuser:swift -K '{swift_secret_key}' list
```


将其中的 {IP ADDRESS} 替换为网关服务器的外网访问 IP 地址,
将 {swift_secret_key} 替换为为 swift 用户所执行的 radosgw-admin keycreate 命令的输出.

举例如下:

```
swift -A http://10.19.143.116/auth/1.0 -U testuser:swift  
-K '244+fz2gSqoHwR3lYtSbIyomyPHf3i7rgSJrF/IA' list
```

输出如下:

```
my-new-bucket
```

联盟网关的配置

New in version 0.67: Dumping

Ceph 从 0.67 Dumping 起支持加入 **Ceph 对象网关** 联盟, 可加入多个 region、或一个 region 内的多个域。

- **Region:** region 是地理空间的逻辑划分, 它包含一个或多个域。一个包含多个 region 的集群必须指定一个主 region。
- **Zone:** 域是一个或多个 Ceph 对象网关例程的逻辑分组。每个 region 有一个主域处理客户端请求。

Important

你可以从二级域读取对象, 但只能写入 region 内的主域。当前, 网关程序不会禁止你写入二级域, 但是, 别那样干!

背景

当你部署一个跨地理时区的 **Ceph 对象存储** 服务时，即使 Ceph 对象网关例程运行在不同的地理时区、甚至不同的 Ceph 集群上，使用 Ceph 对象网关 region 功能和元数据同步代理都能实现全局统一的命名空间。如果你想多保留一个（或多个）数据副本，可以把一个 region 内的一或多个 Ceph 对象网关例程拆分到多个逻辑容器，然后配置 Ceph 对象网关域和数据同步代理，这样你就能多拥有一份或多份主域的数据副本了。额外的数据副本对于故障转移、备份和灾难恢复是很重要的。

如果你有低延时的网络连接（并非建议），可以把 Ceph 存储集群配置为联盟模式的单体集群。也可以在每 region 部署一套 Ceph 存储集群，然后把各存储池划分进各个域（典型部署）。如果资源充足、且冗余性要求严格，你也可以在每个域都部署一个独立的 Ceph 存储集群。

关于本指南

下面我们将演示如何用两个逻辑步骤配置联盟集群：

- **配置主 region**：这一段描述了如何配置起包含多个域的一个 region，以及如何同步主 region 内的主域和各二级域。
- **配置二级 region**：这一段描述了如何重复前一段再配置一个包含多个域的主 region，这样你就有了两个 region，而且都实现了 region 内同步。最后，你会了解到如何配置元数据同步代理，以此统一各 region 命名空间。

配置主 REGION

本段提供的典型步骤可帮你配置起包含两个域的 region，此集群将包含两个网关守护进程例程——每域一个。此 region 将作为主 region。

为主 REGION 命名

配置集群前，规划良好的 region、域和例程名字可帮你更好地管理集群。假设此 region 代表美国，那我们就引用她的标准缩写。

- United States: `us`

假设各域分别为美国的东部和西部，为保持连贯性，命名将采用 `{region name}-{zone name}` 格式，但你可以用自己喜欢的命名规则。

- United States, East Region: `us-east`
- United States, West Region: `us-west`

最后，我们假设每个域都配置了至少一个 Ceph 对象网关例程，为保持连贯性，我们将按 `{region name}-{zone name}-{instance}` 格式命名，但你可以用自己喜欢的命名规则。

- United States Region, Master Zone, Instance 1: `us-east-1`
- United States Region, Secondary Zone, Instance 1: `us-west-1`

创建存储池

你可以把整个 region 配置为一个 Ceph 存储集群，也可以把各个域都配置为一个 Ceph 存储集群。

为保持连贯性，我们将按照 `{region name}-{zone name}` 格式命名，并把它作为存储池名的前缀，但你可以用自己喜欢的命名规则。例如：

- `.us-east.rgw`
- `.us-east.rgw.root`
- `.us-east.rgw.control`
- `.us-east.rgw.gc`
- `.us-east.rgw.buckets`
- `.us-east.rgw.buckets.index`
- `.us-east.rgw.buckets.extra`
- `.us-east.log`
- `.us-east.intent-log`
- `.us-east.usage`
- `.us-east.users`
- `.us-east.users.email`
- `.us-east.users.swift`
- `.us-east.users.uid`

- `.us-west.rgw`
- `.us-west.rgw.root`
- `.us-west.rgw.control`
- `.us-west.rgw.gc`
- `.us-west.rgw.buckets`
- `.us-west.rgw.buckets.index`
- `.us-west.rgw.buckets.extra`
- `.us-west.log`
- `.us-west.intent-log`
- `.us-west.usage`
- `.us-west.users`
- `.us-west.users.email`
- `.us-west.users.swift`
- `.us-west.users.uid`

关于网关的默认存储池请参考[配置参考——存储池](#)。关于创建存储池见[存储池](#)。用下列命令创建存储池：

```
ceph osd pool create {poolname} {pg-num} {pgp-num}
{replicated | erasure} [{erasure-code-profile}] {ruleset-
name} {ruleset-number}
```

Tip

创建大量存储池时，集群回到 `active + clean` 状态可能需要较多的时间。

CRUSH 图

把整个 region 配置为单个 Ceph 存储集群时，请考虑单独为域使用 CRUSH 规则，这样就不会有重叠的故障域了。详情见 [CRUSH 图](#)。

Ceph 允许多级 CRUSH 和多种 CRUSH 规则集，这样在配置你自己的网关时就有很大的灵活性。像 `rgw.buckets.index` 这样的存储池就可以利用副本数适当的 SSD 存储池取得高性能；后端存储也可以从更经济的纠删编码的存储中获益，还可能利用缓存层提升性能。

完成这一步后，执行下列命令以确认你已经创建了前述所需的存储池：

```
rados lspools
```

创建密钥环

各例程都必须有用户名和密钥才能与 Ceph 存储集群通信。在下面几步，我们用管理节点创建密钥环，然后为各例程创建客户端用户名及其密钥，再把这些密钥加入 Ceph 存储集群，最后，把密钥环分别发布给各网关节点。

1. 创建密钥环。

```
2. sudo ceph-authtool --create-keyring  
   /etc/ceph/ceph.client.radosgw.keyring
```

```
sudo chmod +r /etc/ceph/ceph.client.radosgw.keyring
```

3. 为各例程生成 Ceph 对象网关用户名及其密钥。

```
4. sudo ceph-authtool  
   /etc/ceph/ceph.client.radosgw.keyring -n  
   client.radosgw.us-east-1 --gen-key
```

```
sudo ceph-authtool  
/etc/ceph/ceph.client.radosgw.keyring -n  
client.radosgw.us-west-1 --gen-key
```

5. 给各密钥增加能力，允许到监视器的写权限、允许创建存储池，详情见[配置参考——存储池](#)。

```
6. sudo ceph-authtool -n client.radosgw.us-east-1 --cap  
   osd 'allow rwx' --cap mon 'allow rwx'  
   /etc/ceph/ceph.client.radosgw.keyring
```

```
sudo ceph-authtool -n client.radosgw.us-west-1 --cap
osd 'allow rwx' --cap mon 'allow rwx'
/etc/ceph/ceph.client.radosgw.keyring
```

7. 创建密钥环及密钥，并授权 Ceph 对象网关访问 Ceph 存储集群之后，还需把各密钥导入存储集群。例如：

```
8. sudo ceph -k /etc/ceph/ceph.client.admin.keyring
auth add client.radosgw.us-east-1 -i
/etc/ceph/ceph.client.radosgw.keyring
```

```
sudo ceph -k /etc/ceph/ceph.client.admin.keyring
auth add client.radosgw.us-west-1 -i
/etc/ceph/ceph.client.radosgw.keyring
```

Note

按照以上步骤配置二级 region 时，需把 `us-` 替换为 `eu-`。创建主 region 和二级 region 后，你一共会拥有四个用户。

安装 APACHE 和 FASTCGI

每个运行 **Ceph 对象网关** 守护进程例程的 **Ceph 节点** 都必须安装 Apache、FastCGI、Ceph 对象网关守护进程（`radosgw`），还有 Ceph 对象网关同步代理（`radosgw-agent`）。详情见[安装 Ceph 对象网关](#)。

创建数据目录

分别为各主机上的各个守护进程例程创建数据目录。

```
ssh {us-east-1}

sudo mkdir -p /var/lib/ceph/radosgw/ceph-radosgw.us-east-
1
```

```
ssh {us-west-1}

sudo mkdir -p /var/lib/ceph/radosgw/ceph-radosgw.us-west-1
```

Note

按照以上步骤配置二级 region 时，需把 us- 替换为 eu-。创建主 region 和二级 region 后，你一共会拥有四个用户。

创建网关配置

在装了 Ceph 对象网关守护进程的主机上，为各例程创建 Ceph 对象网关配置文件，并放到 `/etc/apache2/sites-available` 目录下。典型的网关配置见下文：

```
FastCgiExternalServer /var/www/s3gw.fcgi -socket
/{path}/{socket-name}.sock

<VirtualHost *:80>

    ServerName {fqdn}

    <!--Remove the comment. Add a server alias with
    *.{fqdn} for S3 subdomains-->

    <!--ServerAlias *.{fqdn}-->

    ServerAdmin {email.address}
```

```
DocumentRoot /var/www

RewriteEngine On

RewriteRule ^/(.*) /s3gw.fcgi?%{QUERY_STRING}
[E=HTTP_AUTHORIZATION:%{HTTP:Authorization},L]


<IfModule mod_fastcgi.c>

<Directory /var/www>

    Options +ExecCGI

    AllowOverride All

    SetHandler fastcgi-script

    Order allow,deny

    Allow from all

    AuthBasicAuthoritative Off

</Directory>

</IfModule>


AllowEncodedSlashes On

ErrorLog /var/log/apache2/error.log

CustomLog /var/log/apache2/access.log combined

ServerSignature Off
```



```
</VirtualHost>
```

1. 把 `{path}/{socket-name}` 条目分别替换为套接字的路径和名字，例如 `/var/run/ceph/client.radosgw.us-east-1.sock`，确保 `ceph.conf` 里也要写入相同的套接字路径和名字。
2. 用服务器的全资域名替换 `{fqdn}` 条目。
3. 用服务器管理员的邮件地址替换 `{email.address}` 条目。
4. 如果你想用 S3 风格的子域，需添加 `ServerAlias` 配置（当然想）。
5. 把这些配置另存到一文件，如 `rgw-us-east.conf`。

在二级域上重复这些步骤，如 `rgw-us-west.conf`。

Note

按照以上步骤配置二级 region 时，需把 `us-` 替换为 `eu-`。创建主 region 和二级 region 后，你一共会拥有四个用户。

最后，如果你想启用 SSL，确保端口要设置为 SSL 端口（通常是 443），而且配置文件要包含这些：

```
SSLEngine on

SSLCertificateFile /etc/apache2/ssl/apache.crt

SSLCertificateKeyFile /etc/apache2/ssl/apache.key

SetEnv SERVER_PORT_SECURE 443
```

启用配置

启用各例程的网关配置、并禁用默认站点。

1. 启用网关站点配置。

```
sudo a2ensite {rgw-conf-filename}
```

2. 禁用默认站点。

```
sudo a2dissite default
```

Note

默认站点禁用失败会导致其他问题。

添加 FASTCGI 脚本

要启用 S3 兼容接口，各 Ceph 对象网关例程都需要一个 FastCGI 脚本。按如下过程创建此脚本。

1. 进入 `/var/www` 目录。

```
2. cd /var/www
```

3. 用编辑器打开名为 `s3gw.fcgi` 的文件。**注：**配置文件里也要指定此文件名。

```
sudo vim s3gw.fcgi
```

4. 添加 shell 脚本头，然后是 `exec`、网关的二进制文件、Ceph 配置文件路径、还有用户名（`-n` 所指，就是[创建密钥环](#)里第二步所创建的），把下面的复制进编辑器。

```
5. #!/bin/sh
```

```
exec /usr/bin/radosgw -c /etc/ceph/ceph.conf -n  
client.radosgw.{ID}
```

例如：

```
#!/bin/sh

exec /usr/bin/radosgw -c /etc/ceph/ceph.conf -n
client.radosgw.us-east-1
```

6. 保存文件。
7. 增加可执行权限。

```
sudo chmod +x s3gw.fcgi
```

在二级域上重复以上步骤。

Note

按照以上步骤配置二级 region 时，需把 `us-` 替换为 `eu-`。创建主 region 和二级 region 后，你一共会拥有四个 FastCGI 脚本。

把各例程加入 CEPH 配置文件

在管理节点上，把各例程的配置写入 Ceph 存储集群的配置文件。例如：

```
...

[client.radosgw.us-east-1]

rgw region = us

rgw region root pool = .us.rgw.root

rgw zone = us-east
```

```
rgw zone root pool = .us-east.rgw.root

keyring = /etc/ceph/ceph.client.radosgw.keyring

rgw dns name = {hostname}

rgw socket path = /var/run/ceph/$name.sock

host = {host-name}


[client.radosgw.us-west-1]

rgw region = us

rgw region root pool = .us.rgw.root

rgw zone = us-west

rgw zone root pool = .us-west.rgw.root

keyring = /etc/ceph/ceph.client.radosgw.keyring

rgw dns name = {hostname}

rgw socket path = /var/run/ceph/$name.sock

host = {host-name}
```

然后，把更新过的 Ceph 配置文件推送到各 **Ceph** 节点如：

```
ceph-deploy --overwrite-conf config push {node1} {node2}
{nodex}
```

Note

按照以上步骤配置二级 region 时，需把 region、存储池和域的名字都从 us 替换为 eu。创建主 region 和二级 region 后，你一共会有四条类似配置。

创建 REGION

1. 为 us region 创建个 region 配置文件，名为 us.json。

把下列实例的内容复制进文本编辑器，把 is_master 设置为 true，用终结点的全资域名替换 {fqdn}。这样，主域就是 us-east，另外 zones 列表中还会有 us-west。详情见[配置参考——region](#)。

```
{ "name": "us",

  "api_name": "us",

  "is_master": "true",

  "endpoints": [

    "http:///{fqdn}:80/"],

  "master_zone": "us-east",

  "zones": [

    { "name": "us-east",

      "endpoints": [

        "http:///{fqdn}:80/"],

      "log_meta": "true",

      "log_data": "true"},

    { "name": "us-west",

      "endpoints": [
```

```
        "http:\\\\{fqdn}:80\\"],

        "log_meta": "true",

        "log_data": "true"}],

    "placement_targets": [

        {

            "name": "default-placement",

            "tags": []

        }

    ],

    "default_placement": "default-placement"}
```

2. 用刚刚创建的 `us.json` 输入文件创建 `us` region 。

```
radosgw-admin region set --infile us.json --name
client.radosgw.us-east-1
```

3. 删除默认 region（如果有的话）。

```
rados -p .us.rgw.root rm region_info.default
```

4. 把 `us` region 设置为默认 region 。

```
radosgw-admin region default --rgw-region=us --name
client.radosgw.us-east-1
```

一套集群只能有一个默认 region 。

5. 更新 region 图。

```
radosgw-admin regionmap update --name  
client.radosgw.us-east-1
```

如果你把 region 配置到不同的 Ceph 存储集群上，可以加 `--name client.radosgw-us-west-1` 选项重复上述的第二、四、五步。也可以从初始网关例程导出 region 图，并按更新步骤导入。

Note

按照以上步骤配置二级 region 时，需把 `us` 替换为 `eu`。创建主 region 和二级 region 后，你一共会有两个 region。

创建域

1. 为 `us-east` 域创建名为 `us-east.json` 的配置导入文件。

把以下实例的内容复制到文本编辑器。本配置里的存储池名字用 region 名和域名作为前缀。关于网关存储池见[配置参考——存储池](#)，关于域请参考[配置参考——域](#)。

```
{ "domain_root": ".us-east.rgw",  
  
  "control_pool": ".us-east.rgw.control",  
  
  "gc_pool": ".us-east.rgw.gc",  
  
  "log_pool": ".us-east.log",  
  
  "intent_log_pool": ".us-east.intent-log",  
  
  "usage_log_pool": ".us-east.usage",  
  
  "user_keys_pool": ".us-east.users",  
  
  "user_email_pool": ".us-east.users.email",
```

```

"user_swift_pool": ".us-east.users.swift",

"user_uid_pool": ".us-east.users.uid",

"system_key": { "access_key": "", "secret_key":
""},

"placement_pools": [

    { "key": "default-placement",

      "val": { "index_pool": ".us-
east.rgw.buckets.index",

              "data_pool": ".us-east.rgw.buckets",

              "data_extra_pool": ".us-
east.rgw.buckets.extra"}

    }

]

}

```

2. 把刚创建的 `us-east` 域的配置文件 `us-east.json` 加入东部和西部存储池，需分别指定其用户名（即 `--name`）。

```

3. radosgw-admin zone set --rgw-zone=us-east --infile
us-east.json --name client.radosgw.us-east-1

```

```

radosgw-admin zone set --rgw-zone=us-east --infile
us-east.json --name client.radosgw.us-west-1

```

重复步骤一，为 `us-west` 创建配置导入文件，然后把 `us-east.json` 加入东部和西部的存储池，需分别指定其用户名（即 `--name`）。


```
radosgw-admin zone set --rgw-zone=us-west --infile
us-west.json --name client.radosgw.us-east-1

radosgw-admin zone set --rgw-zone=us-west --infile
us-west.json --name client.radosgw.us-west-1
```

4. 删除默认域（如果有的话）。

```
rados -p .rgw.root rm zone_info.default
```

5. 更新 region 图。

```
radosgw-admin regionmap update --name
client.radosgw.us-east-1
```

Note

按照以上步骤配置二级 region 时，需把 `us-` 替换为 `eu-`。在各 region 创建完主域和二级域后，你一共会有四个域。

创建域用户

Ceph 对象网关的域用户存储在域存储池中，所以配置完域之后还必须创建域用户。为各用户填充 `access_key` 和 `secret_key` 字段，然后再次更新域配置信息。

```
radosgw-admin user create --uid="us-east" --display-
name="Region-US Zone-East" --name client.radosgw.us-east-
1 --system --gen-access-key --gen-secret

radosgw-admin user create --uid="us-west" --display-
name="Region-US Zone-West" --name client.radosgw.us-west-
1 --system --gen-access-key --gen-secret
```

Note

按照以上步骤配置二级 region 时，需把 `us-` 替换为 `eu-`。在各 region 创建完主域和二级域后，你一共会有四个域用户。这些用户不同于[创建密钥环](#)那里创建的用户。

更新域配置

必须以域用户身份更新域配置，这样同步代理才能通过域的认证。

1. 打开 `us-east.json` 域配置文件，把创建域用户时输出的 `access_key` 和 `secret_key` 的内容粘贴进配置文件的 `system_key` 字段。

```
2. { "domain_root": ".us-east.rgw",
3.   "control_pool": ".us-east.rgw.control",
4.   "gc_pool": ".us-east.rgw.gc",
5.   "log_pool": ".us-east.log",
6.   "intent_log_pool": ".us-east.intent-log",
7.   "usage_log_pool": ".us-east.usage",
8.   "user_keys_pool": ".us-east.users",
9.   "user_email_pool": ".us-east.users.email",
10.    "user_swift_pool": ".us-east.users.swift",
11.    "user_uid_pool": ".us-east.users.uid",
12.    "system_key": {
13.      "access_key": "{paste-access_key-here}",
14.      "secret_key": "{paste-secret_key-here}"
```

```
15.         },
16.         "placement_pools": [
17.             { "key": "default-placement",
18.               "val": { "index_pool": ".us-
east.rgw.buckets.index",
19.                       "data_pool": ".us-
east.rgw.buckets",
20.                       "data_extra_pool": ".us-
east.rgw.buckets.extra"}
21.             }
22.         ]
23.     }
```

24. 保存 `us-east.json` 文件，然后更新域配置文件。

```
25.     radosgw-admin zone set --rgw-zone=us-east --
      infile us-east.json --name client.radosgw.us-east-1
```

```
radosgw-admin zone set --rgw-zone=us-east --infile
us-east.json --name client.radosgw.us-west-1
```

26. 重复步骤一更新 `us-west` 的域配置文件，然后更新域配置。

```
27.     radosgw-admin zone set --rgw-zone=us-west --
      infile us-west.json --name client.radosgw.us-east-1
```

```
radosgw-admin zone set --rgw-zone=us-west --infile  
us-west.json --name client.radosgw.us-west-1
```

Note

按照以上步骤配置二级 region 时，需把 us- 替换为 eu-。在各 region 创建完主域和二级域后，你一共会有四个域。

重启服务

再次发布 Ceph 配置文件后，我们建议重启 Ceph 存储集群和 Apache 例程。

对于 Ubuntu，可在各 **Ceph 节点** 上执行此命令：

```
sudo restart ceph-all
```

对于 Red Hat/CentOS 是此命令：

```
sudo /etc/init.d/ceph restart
```

确保所有组件都重载了各自的配置，对于网关例程，我们建议重启 apache2 服务，例如：

```
sudo service apache2 restart
```

启动网关例程

启动 radosgw 服务。

```
sudo /etc/init.d/radosgw start
```

如果你在同一主机上运行了多个例程，那么还必须指定用户名。

```
sudo /etc/init.d/radosgw start --name client.radosgw.us-east-1
```

打开浏览器检查各域的终结点，向其域名发起一个简单的 HTTP 请求应该会得到下面的回应：

```
<ListAllMyBucketsResult
xmlns="http://s3.amazonaws.com/doc/2006-03-01/">

  <Owner>

    <ID>anonymous</ID>

    <DisplayName/>

  </Owner>

  <Buckets/>

</ListAllMyBucketsResult>
```

配置二级 REGION

本段所述的典型步骤可帮你配置起一个拥有多个 region 的集群。配置一个跨 region 集群要求维护一个全局命名空间，这样分布于不同 region 的对象名就不会存在命名空间冲突问题了。

本段对[配置主 region](#)里的步骤有所扩展，还更改了 region 名称、修改了一些步骤，详情见下文。

为二级 REGION 命名

配置集群前，规划良好的 region、域和例程名字可帮你更好地管理集群。假设此 region 代表欧盟，那我们就引用她的标准缩写。

- European Union: eu

假设各域分别为欧盟的东部和西部，为保持连贯性，命名将采用 `{region name}-{zone name}` 格式，但你可以用自己喜欢的命名规则。

- European Union, East Region: `eu-east`
- European Union, West Region: `eu-west`

最后，我们假设每个域都配置了至少一个 Ceph 对象网关例程，为保持连贯性，我们将按 `{region name}-{zone name}-{instance}` 格式命名，但你可以用自己喜欢的命名规则。

- European Union Region, Master Zone, Instance 1: `eu-east-1`
- European Union Region, Secondary Zone, Instance 1: `eu-west-1`

二级 REGION 的配置

重复执行[配置主 region](#) 里的典型步骤，不同之处如下：

1. 按照[为二级 region 命名](#)而非[为主 region 命名](#)。
2. [创建存储池](#)，用 `eu` 取代 `us`。
3. [创建密钥环](#)及对应密钥，并用 `eu` 取代 `us`。如果你想用同样的密钥环也可以，只要确保给此 region（或 region 和域）创建好密钥就行。
4. [安装 Apache 和 FastCGI](#)。
5. [创建数据目录](#)，用 `eu` 取代 `us`。
6. [创建网关配置](#)，把套接字名称里的 `eu` 替换为 `us`。
7. [启用配置](#)。
8. [添加 FastCGI 脚本](#)，把用户名里的 `eu` 替换为 `us`。
9. [把各例程加入 Ceph 配置文件](#)，把存储池名称里的 `eu` 替换为 `us`。
10. [创建 region](#)，用 `eu` 取代 `us`。把 `is_master` 设置为 `false`，为保持一致性，在二级 region 里也要创建主 region。

```
radosgw-admin region set --infile us.json --name
client.radosgw.eu-east-1
```

11. [创建域](#)，用 `eu` 取代 `us`。一定要换成正确的用户名（即 `--name`），这样才会把域创建到正确的集群。
12. [更新域配置](#)，用 `eu` 取代 `us`。

13. 在 (所有 ?) 二级 region 里 , 创建主 region 的各个域。

```
14.      radosgw-admin zone set --rgw-zone=us-east --
        infile us-east.json --name client.radosgw.eu-east-1

15.      radosgw-admin zone set --rgw-zone=us-east --
        infile us-east.json --name client.radosgw.eu-west-1

16.      radosgw-admin zone set --rgw-zone=us-west --
        infile us-west.json --name client.radosgw.eu-east-1
```

```
radosgw-admin zone set --rgw-zone=us-west --infile
us-west.json --name client.radosgw.eu-west-1
```

17. 在主 region 里 , 创建二级 region 的各个域。

```
18.      radosgw-admin zone set --rgw-zone=eu-east --
        infile eu-east.json --name client.radosgw.us-east-1

19.      radosgw-admin zone set --rgw-zone=eu-east --
        infile eu-east.json --name client.radosgw.us-west-1

20.      radosgw-admin zone set --rgw-zone=eu-west --
        infile eu-west.json --name client.radosgw.us-east-1
```

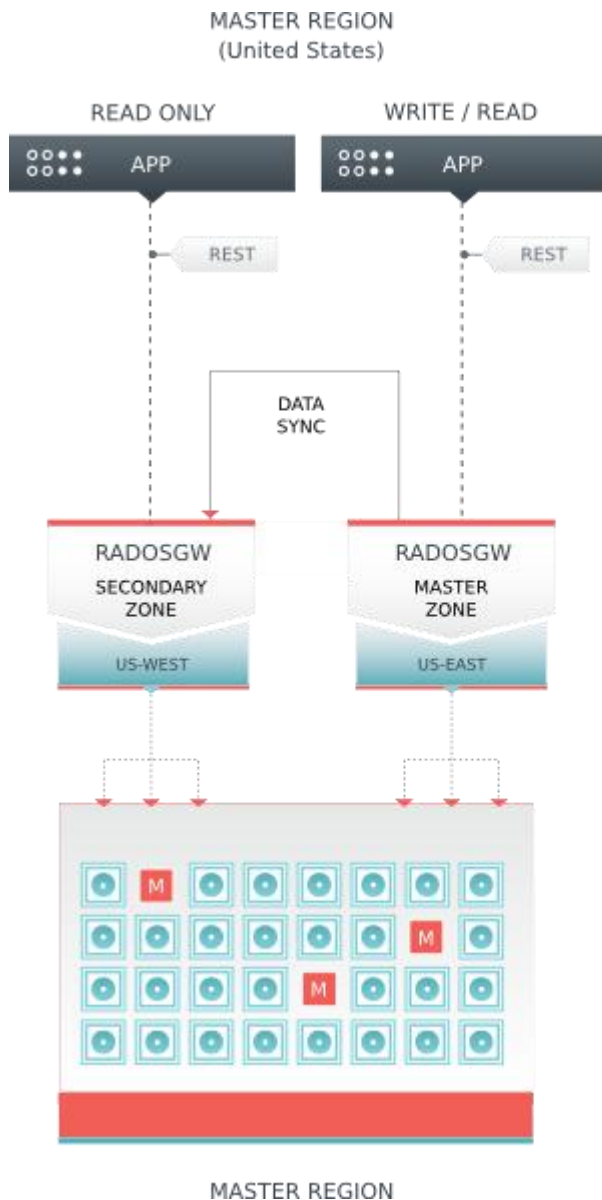
```
radosgw-admin zone set --rgw-zone=eu-west --infile
eu-west.json --name client.radosgw.us-west-1
```

21. [重启服务](#)。

22. [启动网关例程](#)。

多站点数据复制

数据同步代理会把主域数据复制到二级域。一个 region 的主域会自动选择某 region 的二级域 , 并作为它的数据源。



配置同步代理需找出源和目的地的访问密钥、私钥、以及目的 URL 和端口。

你可以用 `radosgw-admin zone list` 获取域名称列表，用 `radosgw-admin zone get` 找出此域的访问密钥和私钥。端口号可以在[创建网关配置](#)时创建的网关配置文件里找到。

一例程只需准备主机名和端口号即可（假设一 region 或域内的所有网关例程都访问同一 Ceph 存储集群）；配置文件（如 `cluster-data-sync.conf`）里的以下这些选项要填上，还要给 `log_file` 指定一个日志文件。

例如：


```
src_access_key: {source-access-key}

src_secret_key: {source-secret-key}

destination: https://zone-name.fqdn.com:port

dest_access_key: {destination-access-key}

dest_secret_key: {destination-secret-key}

log_file: {log.filename}
```

实例类似这样：

```
src_access_key: DG8RE354EFPZBICHIAF0

src_secret_key: i3U0HiRP8CXaBWrcF8bbh6CbsxGYuPPwRkixfFSb

destination: https://us-west.storage.net:80

dest_access_key: U60RFI6B08F32T2PD30G

dest_secret_key: W3HuUor7Gl1Ee93pA2pq2wFk1JMQ7hTrSDecYExl

log_file: /var/log/radosgw/radosgw-sync-us-east-west.log
```

(在哪里执行) 要启动数据同步代理，在终端内执行以下命令：

```
radosgw-agent -c region-data-sync.conf
```

同步代理运行时，你应该能从输出里看到数据片段正在同步。

```
INFO:radosgw_agent.sync:Starting incremental sync
```

```
INFO:radosgw_agent.worker:17910 is processing shard
number 0

INFO:radosgw_agent.worker:shard 0 has 0 entries after ''

INFO:radosgw_agent.worker:finished processing shard 0

INFO:radosgw_agent.worker:17910 is processing shard
number 1

INFO:radosgw_agent.sync:1/64 shards processed

INFO:radosgw_agent.worker:shard 1 has 0 entries after ''

INFO:radosgw_agent.worker:finished processing shard 1

INFO:radosgw_agent.sync:2/64 shards processed

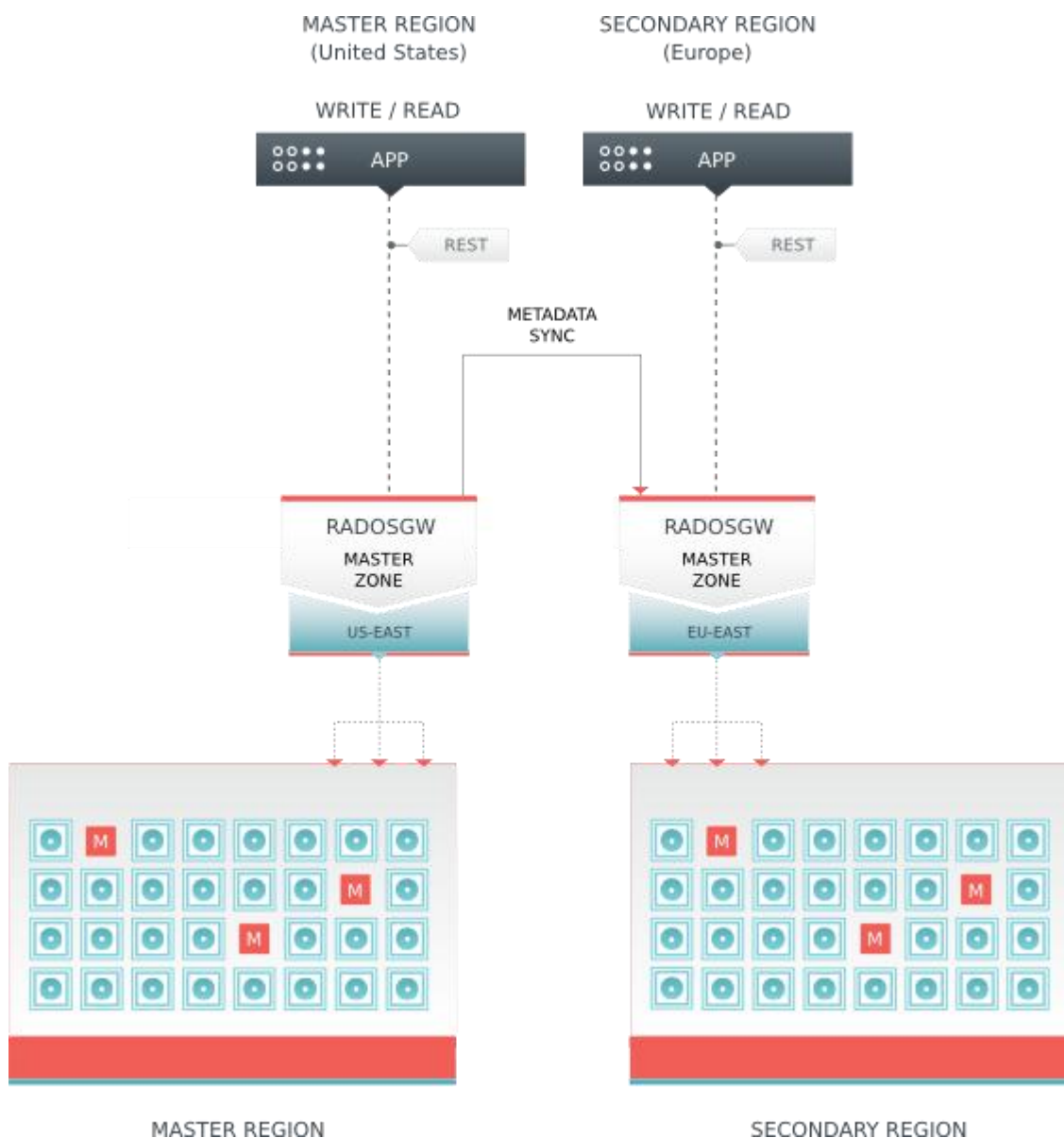
...
```

Note

每个源、目的对都要运行代理。

REGION 间元数据复制

数据同步代理会把主 region 内主域的元数据复制到二级 region 的主域。元数据包含网关用户和桶信息、但不包含桶内的对象——为确保跨集群统一的命名空间，主 region 内的主域是数据源，它会自动选择二级 region 的主域。



按照与[多站点数据复制](#)相同的步骤，并把主 region 的主域作为源域、二级 region 的主域作为二级域。启动 `radosgw-agent` 时加上 `--metadata-only` 参数，这样它就只会复制元数据。例如：

```
radosgw-agent -c inter-region-data-sync.conf --metadata-only
```

完成前述各步骤之后，你就会拥有一套包含两个 region 的集群，其内的主 region `us` 和二级 region `eu` 的命名空间是统一的。

CEPH 对象网关配置参考

下列的选项可加入 Ceph 配置文件（一般是 `ceph.conf`）的 `[client.radosgw.{instance-name}]` 段下，这些选项可能有默认值。如果你没有指定，自就会自动使用默认值。

`rgw data`

描述： 设置 Ceph 对象网关存储数据文件位置。

类型： String

默认值： `/var/lib/ceph/radosgw/$cluster-$id`

`rgw enable apis`

描述： 启用指定的 API。

类型： String

默认值： 所有 API：`s3, swift, swift_auth, admin。`

`rgw cache enabled`

描述： 是否启用 Ceph 对象网关缓存。

类型: Boolean

默认值: true

rgw cache lru size

描述: Ceph 对象网关缓存的条目限制。

类型: Integer

默认值: 10000

rgw socket path

描述: 域套接字的路径，FastCgiExternalServer 要使用此套接字。若未指定，Ceph 对象网关就不会以外部服务器的方式运行。这里的路径必须与 rgw.conf 里的路径相同。

类型: String

默认值: N/A

`rgw host`

描述: Ceph 对象网关实例所在主机，可以是 IP 地址或者主机名。

类型: String

默认值: `0.0.0.0`

`rgw port`

描述: 对象网关实例接受请求的端口。若未指定，Ceph 对象网关将运行外部 FastCGI。

类型: String

**默认
值:** None

`rgw dns name`

描述: 所服务域的 DNS 名称。请参考 region 配置里的 `hostnames` 选项。

类型: String

默认值: None

`rgw script uri`

描述: 如果请求中没带 `SCRIPT_URI` 变量, 这里的设置将作为默认值使用.

类型: String

默认值: None

`rgw request uri`

描述: 如果请求中没带 `REQUEST_URI` 变量, 这里的设置将作为默认值使用。

类型: String

默认值: None

`rgw print continue`

描述: 如果可能的话, 启用 `100-continue`。

类型: Boolean

默认值: `true`

`rgw remote addr param`

描述: 远端地址参数。例如，HTTP 字段或者 `X-Forwarded-For` 地址（如果用了反向代理）。

类型: `String`

默认值: `REMOTE_ADDR`

`rgw op thread timeout`

描述: 运行中线程的超时值。

类型: `Integer`

默认值: `600`

`rgw op thread suicide timeout`

描述: Ceph 对象网关进程的超时自杀值（`timeout`）。设置为 `0` 时禁用。

类型: Integer

默认值: 0

rgw thread pool size

描述: 线程池的尺寸。

类型: Integer

默认值: 100 threads.

rgw num rados handles

描述: Ceph 对象网关的 [RADOS 集群处理器](#) 数量。通过配置 RADOS 处理器数量可以使得各种类型的载荷都明显地提升性能，因为各个 RGW 工作线程在其短暂的活跃期内都可以分别挂靠一个 RADOS 处理器。

类型: Integer

默认值: 1

rgw num control oids

描述: 不同的 `rgw` 实例间用于缓存同步的通知对象数量。

类型: Integer

默认值: 8

`rgw init timeout`

描述: Ceph 对象网关放弃初始化前坚持的时间，秒。

类型: Integer

默认值: 30

`rgw mime types file`

描述: MIME 类型数据库文件的路径，Swift 自动探测对象类型时要用到。

类型: String

默认值: `/etc/mime.types`

`rgw gc max objs`

描述: 垃圾回收进程在一个处理周期内可处理的最大对象数。

类型: Integer

默认值: 32

```
rgw gc obj min wait
```

描述: 对象可被删除并由垃圾回收器处理前最少等待多长时间。

类型: Integer

默认值: 2 * 3600

```
rgw gc processor max time
```

描述: 两个连续的垃圾回收周期起点的最大时间间隔。

类型: Integer

默认值: 3600

```
rgw gc processor period
```

描述: 垃圾回收进程的运行周期。

类型: Integer

默认值: 3600

```
rgw s3 success create obj status
```

描述: create-obj 的另一种成功状态响应。

类型: Integer

默认值: 0

```
rgw resolve cname
```

描述: 如果主机名与 rgw dns name 不同, rgw 是否应该用请求的 hostname 字段的 DNS CNAME 记录。

类型: Boolean

默认值: false

```
rgw obj stripe size
```

描述: Ceph 对象网关的对象条带尺寸。关于条带化请参考[体系结构](#)。

类型: Integer

默认值: 4 << 20

```
rgw extended http attrs
```

描述: 为实体（用户、桶或对象）新增可设置的属性集。可以在上传实体时把这些额外属性设置在 HTTP 头的字段里、或者用 POST 方法修改；如果设置过，在此实体上执行 GET/HEAD 操作时这些属性就会以 HTTP 头的字段返回。

类型: String

默认值: None

实例: “content_foo, content_bar, x-foo-bar”

```
rgw exit timeout secs
```

描述： 等待某一进程多长时间（秒）后无条件退出。

类型： Integer

默认值： 120

```
rgw get obj window size
```

描述： 为单对象请求预留的窗口大小（字节）。

类型： Integer

默认值： 16 << 20

```
rgw get obj max req size
```

描述： 向 Ceph 存储集群发起的一次 GET 请求的最大尺寸。

类型： Integer

默认值： 4 << 20

```
rgw relaxed s3 bucket names
```

描述: 对 US region 的桶启用宽松的桶名规则。

类型: Boolean

默认值: false

```
rgw list buckets max chunk
```

描述: 列举用户桶时，每次检出的最大桶数。

类型: Integer

默认值: 1000

```
rgw override bucket index max shards
```

描述: 桶索引对象的分片数量，0 表示没有分片。我们不建议把这个值设置得太大（比如大于 1000），因为这样会增加罗列桶时的开销。

类型: Integer

默认值: 0

rgw num zone opstate shards

描述: 用于保存 region 间复制进度的最大消息片数。

类型: Integer

默认值: 128

rgw opstate ratelimit sec

描述: 各次上传后状态更新操作的最小间隔时间。0 禁用此限速。

类型: Integer

默认值: 30

rgw curl wait timeout ms

描述: 某些特定 curl 调用的超时值，毫秒。

类型: Integer

默认值: 1000

`rgw copy obj progress`

描述: 长时间复制操作时允许输出对象进度。

类型: Boolean

默认值: `true`

`rgw copy obj progress every bytes`

描述: 复制进度输出的粒度，字节数。

类型: Integer

默认值: `1024 * 1024`

`rgw admin entry`

描述: 管理 URL 请求的入口点。

类型: String

默认值: `admin`

```
rgw content length compat
```

描述: 允许兼容设置了 CONTENT_LENGTH 和 HTTP_CONTENT_LENGTH 的
FCGI 请求。

类型: Boolean

**默认
值:** `false`

REGION（域组）

Ceph 从 v0.67 版开始，通过 region 概念支持 Ceph 对象网关部署和统一的命名空间。
region 定义了位于一或多个域内的 Ceph 对象网关实例的地理位置。

region 的配置不同于一般配置过程，因为不是所有的配置都放在 Ceph 配置文件中。从 Ceph
0.67 版开始，你可以列出所有 region、获取 region 配置以及设置 region 配置。

列出所有 REGION

Ceph 集群可包含一系列 region，可用下列命令列出所有 region

```
sudo radosgw-admin regions list
```

`radosgw-admin` 命令会返回 JSON 格式的 region 列表。

```
{ "default_info": { "default_region": "default"},  
  
  "regions": [  
  
    "default"] }
```

获取 REGION-MAP

要获取各 region 的详细情况，可执行：

```
sudo radosgw-admin region-map get
```

Note

如果你得到了 `failed to read region map` 错误，先试试 `sudo radosgw-admin region-map update`。

获取单个 REGION

要查看某 region 的配置，执行：

```
radosgw-admin region get [--rgw-region=<region>]
```

`default` 这个 region 的配置大致如此：

```
{ "name": "default",  
  
  "api_name": "",  
  
  "is_master": "true",  
  
  "endpoints": [],  
  
  "hostnames": [],  
  
  "master_zone": "",  
  
  "zones": [  
    { "name": "default",
```

```
    "endpoints": [],

    "log_meta": "false",

    "log_data": "false"}

],

"placement_targets": [

    {"name": "default-placement",

     "tags": [] }],

"default_placement": "default-placement"}
```

设置一个 REGION

定义 region 需创建一个 JSON 对象、并提供必需的参数：

1. `name`: region 名字，必需。
2. `api_name`: 此 region 的 API 名字，可选。
3. `is_master`: 决定着此 region 是否为主 region，必需。**注**：只能有一个主 region。
4. `endpoints`: region 内的所有结点列表。例如，你可以用多个域名指向同一 region 区，记得在斜杠前加反斜杠进行转义（`\/`）。也可以给结点指定端口号（`fqdn:port`），可选。
5. `hostnames`: region 内所有主机名的列表。例如，这样你就可以在同一 region 内使用多个域名了。可选配置。此列表会自动包含 `rgw dns name` 配置。更改此配置后需重启所有 `radosgw` 守护进程。
6. `master_zone`: region 的主域，可选。若未指定，则选择默认域。**注**：每个 region 只能有一个主域。
7. `zones`: region 内所有域的列表。各个域都有名字（必需的）、一系列结点（可选的）、以及网关是否要记录元数据和数据操作（默认不记录）。
8. `placement_targets`: 放置目标列表（可选）。每个放置目标都包含此放置目标的名字（必需）、还有一个标签列表（可选），这样只有带这些标签的用户可以使用此放置目标（即用户信息中的 `placement_tags` 字段）。

9. `default_placement`: 对象索引及数据的默认放置目标，默认为 `default-placement`。你可以在用户信息里给各用户设置一个用户级的默认放置目标。

要配置一个 region，需创建一个包含必需字段的 JSON 对象，把它存入文件（如 `region.json`），然后执行下列命令：

```
sudo radosgw-admin region set --infile region.json
```

其中 `region.json` 是你创建的 JSON 文件。

Important

默认 region `default` 的 `is_master` 字段值默认为 `true`。如果你想新建一 region 并让它作为主 region，那你必须把 `default` region 的 `is_master` 设置为 `false`，或者干脆删除 `default` region。

最后，更新 region map。

```
sudo radosgw-admin region-map update
```

配置 REGION MAP

配置 region map 的过程包括创建含一或多个 region 的 JSON 对象，还有设置集群的主 region `master_region`。region map 内的各 region 都由键/值对组成，其中 `key` 选项等价于单独配置 region 时的 `name` 选项，`val` 是包含单个 region 完整配置的 JSON 对象。

你可以只有一个 region，其 `is_master` 设置为 `true`，而且必须在 region map 末尾设置为 `master_region`。下面的 JSON 对象是默认 region map 的实例。

```
{ "regions": [  
  { "key": "default",  
    "val": { "name": "default",
```

```
    "api_name": "",
    "is_master": "true",
    "endpoints": [],
    "hostnames": [],
    "master_zone": "",
    "zones": [
      { "name": "default",
        "endpoints": [],
        "log_meta": "false",
        "log_data": "false"}],
      "placement_targets": [
        { "name": "default-placement",
          "tags": []}],
        "default_placement": "default-placement"
      }
    ],
    "master_region": "default"
  }
```

要配置一个 region map，执行此命令：

```
sudo radosgw-admin region-map set --infile regionmap.json
```

其中 `regionmap.json` 是创建的 JSON 文件。确保你创建了 region map 里所指的那些域。最后，更新此 map。

```
sudo radosgw-admin regionmap update
```

ZONES

从 Ceph v0.67 版起，Ceph 对象网关支持 zone 概念，它是一或多个 Ceph 对象网关实例组成的逻辑组。

zone 的配置不同于典型配置过程，因为并非所有配置都位于 Ceph 配置文件内。从 0.67 版起，你可以列出所有 zone、获取 zone 配置、设置 zone 配置。

列出所有 ZONE

要列出某集群内的所有 zone，执行：

```
sudo radosgw-admin zone list
```

获取单个 ZONE

要获取某一 zone 的配置，执行：

```
sudo radosgw-admin zone get [--rgw-zone=<zone>]
```

`default` 这个默认 zone 的配置大致如此：

```
{ "domain_root": ".rgw",
```

```
"control_pool": ".rgw.control",

"gc_pool": ".rgw.gc",

"log_pool": ".log",

"intent_log_pool": ".intent-log",

"usage_log_pool": ".usage",

"user_keys_pool": ".users",

"user_email_pool": ".users.email",

"user_swift_pool": ".users.swift",

"user_uid_pool": ".users.uid",

"system_key": { "access_key": "", "secret_key": ""},

"placement_pools": [

    { "key": "default-placement",

      "val": { "index_pool": ".rgw.buckets.index",

               "data_pool": ".rgw.buckets"}

    }

]

}
```

配置 ZONE

配置 zone 时需指定一系列的 Ceph 对象网关存储池。为保持一致性，我们建议用 region 名作为存储池名字的前缀。存储池配置见[存储池](#)。

要配置起一个 zone，需创建包含存储池的 JSON 对象、并存入文件（如 `zone.json`）；然后执行下列命令，把 `{zone-name}` 替换为 zone 名称：

```
sudo radosgw-admin zone set --rgw-zone={zone-name} --
infile zone.json
```

其中，`zone.json` 是你创建的 JSON 文件。

REGION 和 ZONE 选项

你可以在 Ceph 配置文件中的各实例 `[client.radosgw.{instance-name}]` 段下设置下列选项。

New in version v.67.

`rgw zone`

描述: 网关例程所在的 **zone** 名称。

类型: String

默认值: None

New in version v.67.

`rgw region`

描述: 网关例程所在的 **region** 名。

类型: String

默认值: None

New in version v.67.

```
rgw default region info oid
```

描述: 用于保存默认 region 的 OID 。我们不建议更改此选项。

类型: String

默认值: `default.region`

存储池

Ceph zone 会映射一系列 Ceph 存储集群的存储池。

手动创建存储池与自动生成的存储池对比

如果你给 Ceph 对象网关的用户 key 分配了写权限，此网关就有能力自动创建存储池。这样虽然便捷，但 Ceph 对象存储集群的 PG 数会是默认值（此值也许不太理想）或者 Ceph 配置文件中的自定义配置。如果你想让 Ceph 对象网关自动创建存储池，确保 PG 数的默认值要合理。详情见[存储池配置](#)，关于创建存储池见[集群存储池](#)。

Ceph 对象网关的默认 zone 的默认存储池有：

- `.rgw`
- `.rgw.control`
- `.rgw.gc`

- `.log`
- `.intent-log`
- `.usage`
- `.users`
- `.users.email`
- `.users.swift`
- `.users.uid`

你应该能够清晰地判断某个 zone 会怎样访问各存储池。你可以为每个 zone 创建一系列存储池，或者让多个 zone 共用同一系列的存储池。作为最佳实践，我们建议分别位于各 region 中的主 zone 和第二 zone 都要有各自的一系列存储池。为某个域创建存储池时，建议默认存储池名以 region 名和 zone 名作为前缀，例如：

- `.region1-zone1.domain.rgw`
- `.region1-zone1.rgw.control`
- `.region1-zone1.rgw.gc`
- `.region1-zone1.log`
- `.region1-zone1.intent-log`
- `.region1-zone1.usage`
- `.region1-zone1.users`
- `.region1-zone1.users.email`
- `.region1-zone1.users.swift`
- `.region1-zone1.users.uid`

Ceph 对象网关会把 bucket 索引 (`index_pool`) 和 bucket 数据 (`data_pool`) 存储到归置存储池，这些可以重叠——也就是你可以把索引和数据存入同一存储池。索引存储池的默认归置地是 `.rgw.buckets.index`，数据存储池的默认归置地是 `.rgw.buckets`，给 zone 指定存储池的方法见 [Zones](#)。

Deprecated since version v.67.

`rgw cluster root pool`

描述: 为此实例存储 `radosgw` 元数据的存储池。从 v0.67 之后不再支持，可改用 `rgw zone root pool`。

类型: String

是否必需: No

默认值: `.rgw.root`

替代选项: `rgw zone root pool`

New in version v.67.

`rgw region root pool`

描述: 用于存储此 **region** 所有相关信息的存储池。

类型: String

默认值: `.rgw.root`

New in version v.67.

`rgw zone root pool`

描述: 用于存储此 **zone** 所有相关信息的存储池。

类型: String

默认值: `.rgw.root`

SWIFT 选项

`rgw enforce swift acls`

描述: 强制使用 Swift 的访问控制列表（ACL）选项。

类型: Boolean

默认值: `true`

`rgw swift token expiration`

描述: Swift 令牌过期时间，秒。

类型: Integer

默认值: `24 * 3600`

`rgw swift url`

描述: Ceph 对象网关 Swift 接口的 URL。

类型: String

默认值: None

```
rgw swift url prefix
```

描述: Swift API 的 URL 前缀。

默认值: `swift`

实例: <http://fqdn.com/swift>

```
rgw swift auth url
```

描述: 验证 v1 版令牌的默认 URL（如果没用 Swift 内建认证）。

类型: String

默认值: None

```
rgw swift auth entry
```

描述: Swift 认证 URL 的入口点。

类型: String

默认值: `auth`

日志记录选项

```
rgw log nonexistent bucket
```

描述: 让 Ceph 对象网关记录访问不存在的 `bucket` 的请求。

类型: Boolean

默认值: `false`

```
rgw log object name
```

描述: 对象名的记录格式。关于格式说明见 *date* 。

类型: Date

默认值: `%Y-%m-%d-%H-%i-%n`

```
rgw log object name utc
```

描述: 记录的对象名是否需包含 UTC 时间, 设置为 `false` 时将使用本地时间。

类型: Boolean

默认值: `false`

`rgw usage max shards`

描述: 使用率日志的最大可分片数量。

类型: Integer

默认值: `32`

`rgw usage max user shards`

描述: 单个用户使用率日志的最大可分片数量。

类型: Integer

默认值: `1`

`rgw enable ops log`

描述: 允许记录各次成功的 Ceph 对象网关操作。

类型: Boolean

默认值: false

```
rgw enable usage log
```

描述: 允许记录使用率日志。

类型: Boolean

默认值: false

```
rgw ops log rados
```

描述: 操作日志是否应该写入 Ceph 存储集群后端。

类型: Boolean

默认值: true

```
rgw ops log socket path
```

描述: 用于写入操作日志的 Unix 域套接字。

类型: String

默认值: None

```
rgw ops log data backlog
```

描述: 最多积攒多少操作日志数据才写入 Unix 域套接字。

类型: Integer

默认值: 5 << 20

```
rgw usage log flush threshold
```

描述: 使用率日志合并过多少条目才刷回。

类型: Integer

默认值: 1024

```
rgw usage log tick interval
```

描述: 每 `n` 秒执行一次使用率日志刷回。

类型: Integer

默认值: 30

```
rgw intent log object name
```

描述: 意图日志对象名的记录格式。格式的详细说明见 *date* 。

类型: Date

默认值: %Y-%m-%d-%i-%n

```
rgw intent log object name utc
```

描述: 意图日志对象名是否应包含 UTC 时间，设置为 `false` 时使用本地时间。

类型: Boolean

默认值: false

```
rgw data log window
```

描述: 数据日志窗口，秒。

类型: Integer

默认值: 30

```
rgw data log changes size
```

描述: 内存中保留的数据变更日志条数。

类型: Integer

默认值: 1000

```
rgw data log num shards
```

描述: 用于保存数据变更日志的分片（对象）数量。

类型: Integer

默认值: 128

```
rgw data log obj prefix
```

描述: 数据日志的对象名前缀。

类型: String

默认值: data_log

```
rgw replica log obj prefix
```

描述: 复制日志的对象名前缀。

类型: String

默认值: replica log

```
rgw md log max shards
```

描述: 用于元数据日志的最大分片数。

类型: Integer

默认值: 64

KEYSTONE 选项

```
rgw keystone url
```

描述: Keystone 服务器的 URL。

类型: String

默认值: None

```
rgw keystone admin token
```

描述: Keystone 的管理令牌（共享密钥）。

类型: String

默认值: None

```
rgw keystone accepted roles
```

描述: 要接受请求所需的角色。

类型: String

默认值: Member, admin

```
rgw keystone token cache size
```

描述: 各 Keystone 令牌缓存的最大条数。

类型: Integer

默认值: 10000

```
rgw keystone revocation interval
```

描述: 令牌有效期查验的周期，秒。

类型: Integer

默认值: 15 * 60

```
rgw keystone verify ssl
```

描述: 将 token 请求发送给 keystone 时验证 SSL 证书。

类型: Boolean

默认值: true

管理手册

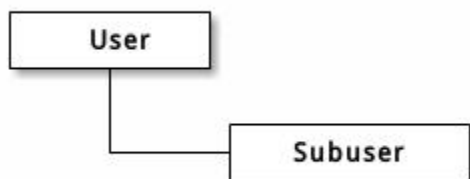
一旦你的 Ceph 对象存储服务成功启动并运行, 你就可以实现管理服务, 比如用户管理、访问控制、限额和用量跟踪等功能。

用户管理

Ceph 对象存储用户管理是指 Ceph 对象存储服务的用户(即: 这里讲的并不是 Ceph 对象网关是作为 Ceph 存储集群的一个用户)。你必须 创建一个用户, 生成 access 和 access 密钥以使得最终用户能够与 Ceph 对象网关服务进行交互。

有两种类型的用户:

- **用户:** ‘用户’ 这个表示这个是使用 **S3** 接口的一个用户。
- **子用户:** ‘子用户’ 这个词表示使用的是 **Swift** 接口的一个用户。一个子用户 是和一个用户关联的。



对于用户和子用户, 你可以执行新建、修改、查看、停用和删除操作。除了用户和 子用户的 ID, 你还需要给用户添加一个显示名称和邮件地址。你可以手动指定 access 和 secret 密钥, 或者选择自动生成。当你生成或者指定密钥的时候, 请注意, 用户的 ID 是跟 S3 类型的密钥关联而子用户的 ID 则是跟 Swift 类型的密钥关联。Swift 密钥也拥有 `read`, `write`, `readwrite` 和 `full` 这几种访问级别。

新建一个用户

执行下面的命令新建一个用户 (S3 接口):


```
radosgw-admin user create --uid={username} --display-  
name="{display-name}" [--email={email}]
```

实例如下:

```
radosgw-admin user create --uid=johndoe --display-  
name="John Doe" --email=john@example.com
```

```
{ "user_id": "johndoe",  
  
  "display_name": "John Doe",  
  
  "email": "john@example.com",  
  
  "suspended": 0,  
  
  "max_buckets": 1000,  
  
  "auid": 0,  
  
  "subusers": [],  
  
  "keys": [  
    { "user": "johndoe",  
      "access_key": "11BS02LGFB6AL6H1ADMW",  
      "secret_key":  
"vzCEkuryfn060dfee4fgQPqFrncKEIkh3ZcdOANY"}],  
  
  "swift_keys": [],  
  
  "caps": [],  
  
  "op_mask": "read, write, delete",  
  
  "default_placement": "",
```

```
"placement_tags": [],

"bucket_quota": { "enabled": false,

    "max_size_kb": -1,

    "max_objects": -1},

"user_quota": { "enabled": false,

    "max_size_kb": -1,

    "max_objects": -1},

"temp_url_keys": []}
```

新建用户的时候也会同时生成 `access_key` 和 `secret_key` 入口，以便任何兼容 S3 API 的客户端能够使用。

Important

仔细检查命令输出的密钥。有时 `radosgw-admin` 会生成一个 JSON 转义字符(`\`),但是有些客户端不知道如何处理 JSON 转义字符(`\`) 删除，将这些字符串封装在引号内，重新生成密钥 直到确定没有 JSON 转义字符存在或手动指定所需的 密钥。

新建一个子用户

为了给用户新建一个子用户 (Swift 接口)，你必须为该子用户指定用户的 ID(`--uid={username}`)，子用户的 ID 以及访问级别:

```
radosgw-admin subuser create --uid={uid} --subuser={uid}
--access=[ read | write | readwrite | full ]
```

实例如下:

```
radosgw-admin subuser create --uid=johndoe --  
subuser=johndoe:swift --access=full
```

Note

full 并不表示 readwrite, 因为它还包括访问权限策略.

```
{ "user_id": "johndoe",  
  
  "display_name": "John Doe",  
  
  "email": "john@example.com",  
  
  "suspended": 0,  
  
  "max_buckets": 1000,  
  
  "auid": 0,  
  
  "subusers": [  
    { "id": "johndoe:swift",  
      "permissions": "full-control"}],  
  
  "keys": [  
    { "user": "johndoe",  
      "access_key": "11BS02LGFB6AL6H1ADMW",  
      "secret_key":  
"vzCEkuryfn060dfec4fgQPqFrncKEIkh3ZcdOANY"}],  
  
    "swift_keys": [],  
  
    "caps": [],
```

```
"op_mask": "read, write, delete",

"default_placement": "",

"placement_tags": [],

"bucket_quota": { "enabled": false,

    "max_size_kb": -1,

    "max_objects": -1},

"user_quota": { "enabled": false,

    "max_size_kb": -1,

    "max_objects": -1},

"temp_url_keys": []}
```

获取用户信息

要获取一个用户的信息，你必须使用 `user info` 子命令并且制定一个用户 ID(`--uid={username}`)。

```
radosgw-admin user info --uid=johndoe
```

修改用户信息

要修改一个用户的信息，你必须指定用户的 ID (`--uid={username}`)，还有 你想要修改的属性值。典型的修改项主要是 `access` 和 `secret` 密钥，邮件地址，显示名称和访问级别。举例如下：

```
radosgw-admin user modify --uid=johndoe --display-
name="John E. Doe"
```

要修改子用户的信息, 使用 `subuser modify` 子命令并且执行子用户的 ID. 举例如下:

```
radosgw-admin subuser modify --uid=johndoe:swift --  
access=full
```

用户 启用/停用

当你创建了一个用户, 用户默认情况下是处于启用状态的。然而, 你可以暂停用户权 限并在以后随时重新启用它们。暂停一个用户, 使用 `user suspend` 子命令 然后哦指定用户的 ID:

```
radosgw-admin user suspend --uid=johndoe
```

要重新启用已经被停用的用户, 使用 `user enable` 子命令并指明用户的 ID.

```
radosgw-admin user enable --uid=johndoe
```

Note

停用一个用户后, 它的子用户也会一起被停用.

删除用户

删除用户时, 这个用户以及他的子用户都会被删除。当然, 如果你愿意, 可以只删除子用户。要删除用户 (及其子用户), 可使用 `user rm` 子命令并指明用户 ID :

```
radosgw-admin user rm --uid=johndoe
```

只想删除子用户时, 可使用 `subuser rm` 子命令并指明子用户 ID 。

```
radosgw-admin subuser rm --subuser=johndoe:swift
```

其它可选操作：

- **Purge Data:** 加 `--purge-data` 选项可清除与此 UID 相关的所有数据。
- **Purge Keys:** 加 `--purge-keys` 选项可清除与此 UID 相关的所有密钥。

删除子用户

在你删除子用户的同时，也失去了 Swift 接口的访问方式，但是这个用户在系统中还存在。要删除子用户，可使用 `subuser rm` 子命令并指明子用户 ID：

```
radosgw-admin subuser rm --subuser=johndoe:swift
```

其它可选操作：

- **Purge Keys:** 加 `--purge-keys` 选项可清除与此 UID 相关的所有密钥。

新建一个密钥

要为用户新建一个密钥，你需要使用 `key create` 子命令。对于用户来说，需要指明用户的 ID 以及新建的密钥类型为 `s3`。要为用户新建一个密钥，则需要指明子用户的 ID 以及密钥类型为 `swift`。实例如下：

```
radosgw-admin key create --subuser=johndoe:swift --key-type=swift --gen-secret

{ "user_id": "johndoe",

  "rados_uid": 0,

  "display_name": "John Doe",

  "email": "john@example.com",

  "suspended": 0,

  "subusers": [
```

```

    { "id": "johndoe:swift",

      "permissions": "full-control"}],

  "keys": [

    { "user": "johndoe",

      "access_key": "QFAMEDSJP5DEKJO0DDXY",

      "secret_key":
"iaSFLDVvDdQt6lkNzHyW4fPLZugBAIlg17LO0+87"}],

    "swift_keys": [

      { "user": "johndoe:swift",

        "secret_key":
"E9T2rUZNu2gxUjcwUBO8n\ /Ev4KX6\ /GprEuH4qhul"}] }

```

新建/删除 ACCESS 密钥

用户和子用户要能使用 S3 和 Swift 接口，必须有 access 密钥。在你新建用户或者子用户的时候，如果没有指明 access 和 secret 密钥，这两个密钥会自动生成。你可能需要新建 access 和/或 secret 密钥，不管是手动指定还是自动生成的方式。你也可能需要删除一个 access 和 secret。可用的选项有：

- `--secret=<key>` 指明一个 **secret** 密钥 (e.即手动生成).
- `--gen-access-key` 生成一个随机的 **access** 密钥 (新建 S3 用户的默认选项).
- `--gen-secret` 生成一个随机的 **secret** 密钥.
- `--key-type=<type>` 指定密钥类型. 这个选项的值可以是: **swift**, **s3**

要新建密钥，需要指明用户 ID.

```

radosgw-admin key create --uid=johndoe --key-type=s3 --
gen-access-key --gen-secret

```

你也可以使用指定 access 和 secret 密钥的方式.

要删除一个 access 密钥, 也需要指定用户 ID.

```
radosgw-admin key rm --uid=johndoe
```

添加/删除 管理权限

Ceph 存储集群提供了一个管理 API, 它允许用户通过 REST API 执行管理功能。默认情况下, 用户没有访问 这个 API 的权限。要启用用户的管理功能, 需要为用户 提供管理权限。

执行下面的命令为一个用户添加管理权限:

```
radosgw-admin caps add --uid={uid} --caps={caps}
```

你可以给一个用户添加对用户、bucket、元数据和用量(存储使用信息)等数据的 读、写或者所有权限。举例如下:

```
--  
caps="[users|buckets|metadata|usage|zone]=[*|read|write|read, write]"
```

实例如下:

```
radosgw-admin caps add --uid=johndoe --caps="users=*"
```

要删除某用户的管理权限, 可用下面的命令:

```
radosgw-admin caps rm --uid=johndoe --caps={caps}
```


配额管理

Ceph 对象网关允许你在用户级别、用户拥有的 bucket 级别设置配额。配额包括一个 bucket 内允许的最大对象数和最大存储容量，大小单位 是兆字节。

- **Bucket:** 选项 `--bucket` 允许你为用户的某一个 bucket 设置配额。
- **Maximum Objects:** 选项 `--max-objects` 允许 你设置最大对象数。负数表示不启用这个设置。
- **Maximum Size:** 选项 `--max-size` 允许你设置一个 最大存储用量的配额。负数表示不启用这个设置。
- **Quota Scope:** 选项 `--quota-scope` 表示这个配额 生效的范围。这个参数的值是 `bucket` 和 `user`。Bucket 配额作用于用户的某一个 bucket。而用户配额作用于一个用户。

设置用户配额

在你启用用户的配额前，你需要先设置配额参数。 例如：

```
radosgw-admin quota set --quota-scope=user --uid=<uid> [-  
-max-objects=<num objects>] [--max-size=<max size>]
```

实例如下：

```
radosgw-admin quota set --quota-scope=user --uid=johndoe  
--max-objects=1024 --max-size=1024
```

最大对象数和最大存储用量的值是负数则表示不启用指定的 配额参数。

启用/禁用用户配额

在你设置了用户配额之后，你可以启用这个配额。实例如下：

```
radosgw-admin quota enable --quota-scope=user --uid=<uid>
```

你也可以禁用已经启用了配额的用户的配额。 举例如下:

```
radosgw-admin quota-disable --quota-scope=user --uid=<uid>
```

设置 BUCKET 配额

Bucket 配额作用于用户的某一个 bucket，通过 `uid` 指定用户。这些配额设置是独立于用户之外的。:

```
radosgw-admin quota set --uid=<uid> --quota-scope=bucket [--max-objects=<num objects>] [--max-size=<max size>]
```

最大对象数和最大存储用量的值是负数则表示不启用指定的 配额参数。

启用/禁用 BUCKET 配额

在你设置了 bucket 配额之后，你可以启用这个配额。实例如下:

```
radosgw-admin quota enable --quota-scope=bucket --uid=<uid>
```

你也可以禁用已经启用了配额的 bucket 的配额。 举例如下:

```
radosgw-admin quota-disable --quota-scope=bucket --uid=<uid>
```

获取配额信息

你可以通过用户信息 API 来获取每一个用户的配额 设置。通过 CLI 接口读取用户的配额设置信息，请 执行下面的命令:

```
radosgw-admin user info --uid=<uid>
```

更新配额统计信息

配额的统计数据的同步是异步的。你也可以通过手动获取最新的配额统计数据为所有用户和所有 bucket 更新配额统计数据:

```
radosgw-admin user stats --uid=<uid> --sync-stats
```

获取用户用量统计信息

执行下面的命令获取当前用户已经消耗了配额的多少:

```
radosgw-admin user stats --uid=<uid>
```

Note

你应该在执行 `radosgw-admin user stats` 的时候带上 `--sync-stats` 参数来获取最新的数据.

读取/设置全局配额

你可以在 region map 中读取和设置配额。执行下面的命令来获取 region map:

```
radosgw-admin regionmap get > regionmap.json
```

要为整个 region 设置配额，只需要简单的修改 region map 中的配额设置。然后使用 `region set` 来更新 region map 即可:

```
radosgw-admin region set < regionmap.json
```

Note

在更新 region map 后，你必须重启网关。

用量

Ceph 对象网关会为每一个用户记录用量数据。你也可以通过指定 日期范围来跟踪用户的用量数据。

可用选项如下：

- **Start Date:** 选项 `--start-date` 允许你指定一个起始日期来过滤用量数据 (format: `yyyy-mm-dd[HH:MM:SS]`).
- **End Date:** 选项 `--end-date` 允许你指定一个截止日期来过滤用量数据 (format: `yyyy-mm-dd[HH:MM:SS]`).
- **Log Entries:** 选项 `--show-log-entries` 允许你 指明显示用量数据的时候是否要包含日志条目。 (选项值: `true` | `false`).

Note

你可以指定时间为分钟和秒，但是数据存储是以一个小时的间隔存储的。

展示用量信息

显示用量统计数据，使用 `usage show` 子命令。显示某一个特定 用户的用量数据，你必须指定该用户的 ID。你也可以指定开始日期、结 束日期以及是否显示日志条目。：

```
radosgw-admin usage show --uid=johndoe --start-date=2012-03-01 --end-date=2012-04-01
```

通过去掉用户的 ID，你也可以获取所有用户的汇总的用量信息

```
radosgw-admin usage show --show-log-entries=false
```

删除用量信息

对于大量使用的集群而言，用量日志可能会占用大量存储空间。你 可以为所有用户或者一个特定的用户删除部分用量日志。你也可以 为删除操作指定日期范围。：

```
radosgw-admin usage trim --start-date=2010-01-01 --end-date=2010-12-31

radosgw-admin usage trim --uid=johndoe

radosgw-admin usage trim --uid=johndoe --end-date=2013-12-31
```

清除临时数据

Deprecated since version 0.52.

When you delete objects (and buckets/containers), the Gateway marks the data for removal, but it is still available to users until it is purged. Since data still resides in storage until it is purged, it may take up available storage space. To ensure that data marked for deletion isn't taking up a significant amount of storage space, you should run the following command periodically:

```
radosgw-admin temp remove
```

Important

Data marked for deletion may still be read. So consider executing the foregoing command a reasonable interval after data was marked for deletion.

Tip

Consider setting up a `cron` job to purge data.

CEPH 对象网关的 S3 兼容 API

Ceph 支持 REST 风格的 API ，它与[亚马逊 S3 API](#) 的基本数据访问模型兼容。

API

- [通用](#)
- [认证](#)
- [服务操作](#)
- [Bucket 操作](#)
- [对象操作](#)
- [C++](#)
- [C#](#)
- [Java](#)
- [Perl](#)
- [PHP](#)
- [Python](#)
- [Ruby AWS::SDK 样例 \(aws-sdk gem ~>2\)](#)
- [Ruby AWS::S3 样例 \(aws-s3 gem\)](#)

功能支持情况

下面的表格列出了对亚马逊 S3 的功能支持情况：

Feature	Status	Remarks
List Buckets	Supported	
Delete Bucket	Supported	

Feature	Status	Remarks
Create Bucket	Supported	Different set of canned ACLs
Bucket Lifecycle	Not Supported	
Policy (Buckets, Objects)	Not Supported	ACLs are supported
Bucket Website	Not Supported	
Bucket ACLs (Get, Put)	Supported	Different set of canned ACLs
Bucket Location	Supported	
Bucket Notification	Not Supported	
Bucket Object Versions	Supported	
Get Bucket Info (HEAD)	Supported	
Bucket Request Payment	Not Supported	
Put Object	Supported	

Feature	Status	Remarks
Delete Object	Supported	
Get Object	Supported	
Object ACLs (Get, Put)	Supported	
Get Object Info (HEAD)	Supported	
POST Object	Supported	
Copy Object	Supported	
Multipart Uploads	Supported	(missing Copy Part)

不支持的 HEADER 字段

下列的通用请求头部字段尚不支持：

Nam	Type
x-amz-security-token	Request

Nam	Type
Server	Response
x-amz-delete-marker	Response
x-amz-id-2	Response
x-amz-version-id	Response

CEPH 对象网关 SWIFT API

Ceph 支持 REST 风格的 API ，它与 [Swift API](#) 的基本访问模型兼容。

API

- [认证](#)
- [服务操作](#)
- [容器操作](#)
- [对象操作](#)
- [临时 URL 操作](#)
- [指导手册](#)
- [Java](#)
- [Python](#)
- [Ruby](#)

功能支持

下面的表格描述了对当前 Swift 功能的支持情况：

功能	状态	备注
Authentication	支持	
Get Account Metadata	支持	
Swift ACLs	支持	支持部分 Swift ACL
List Containers	支持	
Delete Container	支持	
Create Container	支持	
Get Container Metadata	支持	
Update Container Metadata	支持	
Delete Container Metadata	支持	
List Objects	支持	
Static Website	不支持	

功能	状态	备注
Create Object	支持	
Create Large Object	支持	
Delete Object	支持	
Get Object	支持	
Copy Object	支持	
Get Object Metadata	支持	
Update Object Metadata	支持	
Expiring Objects	支持	
Object Versioning	不支持	
CORS	不支持	

管理操作

An admin API request will be done on a URI that starts with the configurable 'admin' resource entry point. Authorization for the admin API duplicates the S3 authorization mechanism. Some operations require that the user holds special administrative capabilities. The response entity type (XML or JSON) may be specified as the 'format' option in the request and defaults to JSON if not specified.

GET USAGE

请求带宽利用率信息。

caps: usage=read

语法

```
GET /{admin}/usage?format=json HTTP/1.1

Host: {fqdn}
```

请求参数

uid

Description: The user for which the information is requested. If not specified will apply to all users.

Type: String

Example: `foo_user`

Required: No

`start`

Description: Date and (optional) time that specifies the start time of the requested data.

Type: String

Example: `2012-09-25 16:00:00`

Required: No

`end`

Description: Date and (optional) time that specifies the end time of the requested data (non-inclusive).

Type: String

Example: `2012-09-25 16:00:00`

Required: No

`show-entries`

Description: Specifies whether data entries should be returned.

Type: Boolean

Example: True [False]

Required: No

`show-summary`

Description: Specifies whether data summary should be returned.

Type: Boolean

Example: True [False]

Required: No

RESPONSE ENTITIES

If successful, the response contains the requested information.

`usage`

Description: A container for the usage information.

Type: Container

entries

Description: A container for the usage entries information.

Type: Container

user

Description: A container for the user data information.

Type: Container

owner

Description: The name of the user that owns the buckets.

Type: String

bucket

Description: The bucket name.

Type: String

`time`

Description: Time lower bound for which data is being specified (rounded to the beginning of the first relevant hour).

Type: String

`epoch`

Description: The time specified in seconds since 1/1/1970.

Type: String

`categories`

Description: A container for stats categories.

Type: Container

`entry`

Description: A container for stats entry.

Type: Container

`category`

Description: Name of request category for which the stats are provided.

Type: String

`bytes_sent`

Description: Number of bytes sent by the RADOS Gateway.

Type: Integer

`bytes_received`

Description: Number of bytes received by the RADOS Gateway.

Type: Integer

`ops`

Description: Number of operations.

Type: Integer

`successful_ops`

Description: Number of successful operations.

Type: Integer

`summary`

Description: A container for stats summary.

Type: Container

`total`

Description: A container for stats summary aggregated total.

Type: Container

SPECIAL ERROR RESPONSES

TBD.

TRIM USAGE

Remove usage information. With no dates specified, removes all usage information.

caps: usage=write

SYNTAX

```
DELETE /{admin}/usage?format=json HTTP/1.1
```

```
Host: {fqdn}
```

REQUEST PARAMETERS

`uid`

Description: The user for which the information is requested. If not specified will apply to all users.

Type: String

Example: `foo_user`

Required: No

`start`

Description: Date and (optional) time that specifies the start time of the requested data.

Type: String

Example: 2012-09-25 16:00:00

Required: No

end

Description: Date and (optional) time that specifies the end time of the requested data (none inclusive).

Type: String

Example: 2012-09-25 16:00:00

Required: No

remove-all

Description: Required when uid is not specified, in order to acknowledge multi user data removal.

Type: Boolean

Example: True [False]

Required: No

SPECIAL ERROR RESPONSES

TBD.

GET USER INFO

Get user information. If no user is specified returns the list of all users along with suspension information.

caps: users=read

SYNTAX

```
GET /{admin}/user?format=json HTTP/1.1  
  
Host: {fqdn}
```

REQUEST PARAMETERS

`uid`

Description: The user for which the information is requested.

Type: String

Example: `foo_user`

Required: No

RESPONSE ENTITIES

If successful, the response contains the user information.

`user`

Description: A container for the user data information.

Type: Container

`user_id`

Description: The user id.

Type: String

Parent: `user`

`display_name`

Description: Display name for the user.

Type: String

Parent: user

suspended

Description: True if the user is suspended.

Type: Boolean

Parent: user

max_buckets

Description: The maximum number of buckets to be owned by the user.

Type: Integer

Parent: user

subusers

Description: Subusers associated with this user account.

Type: Container

Parent: user

keys

Description: S3 keys associated with this user account.

Type: Container

Parent: user

swift_keys

Description: Swift keys associated with this user account.

Type: Container

Parent: user

caps

Description: User capabilities.

Type: Container

Parent: `user`

SPECIAL ERROR RESPONSES

None.

CREATE USER

Create a new user. By Default, a S3 key pair will be created automatically and returned in the response. If only one of `access-key` or `secret-key` is provided, the omitted key will be automatically generated. By default, a generated key is added to the keyring without replacing an existing key pair. If `access-key` is specified and refers to an existing key owned by the user then it will be modified.

caps: users=write

SYNTAX

```
PUT /{admin}/user?format=json HTTP/1.1

Host: {fqdn}
```

REQUEST PARAMETERS

`uid`

Description: The user ID to be created.

Type: String

Example: `foo_user`

Required: Yes

`display-name`

Description: The display name of the user to be created.

Type: String

Example: `foo user`

Required: Yes

`email`

Description: The email address associated with the user.

Type: String

Example: `foo@bar.com`

Required: No

`key-type`

Description: Key type to be generated, options are: swift, s3 (default).

Type: String

Example: s3 [s3]

Required: No

`access-key`

Description: Specify access key.

Type: String

Example: ABCD0EF12GHIJ2K34LMN

Required: No

`secret-key`

Description: Specify secret key.

Type: String

Example: 0AbCDEFG1h2i34JklM5nop6QrSTUV+WxyzaBC7D8

Required: No

user-caps

Description: User capabilities.

Type: String

Example: usage=read, write; users=read

Required: No

generate-key

Description: Generate a new key pair and add to the existing keyring.

Type: Boolean

Example: True [True]

Required: No

max-buckets

Description: Specify the maximum number of buckets the user can own.

Type: Integer

Example: 500 [1000]

Required: No

suspended

Description: Specify whether the user should be suspended.

Type: Boolean

Example: False [False]

Required: No

RESPONSE ENTITIES

If successful, the response contains the user information.

user

Description: A container for the user data information.

Type: Container

`user_id`

Description: The user id.

Type: String

Parent: `user`

`display_name`

Description: Display name for the user.

Type: String

Parent: `user`

`suspended`

Description: True if the user is suspended.

Type: Boolean

Parent: user

max_buckets

Description: The maximum number of buckets to be owned by the user.

Type: Integer

Parent: user

subusers

Description: Subusers associated with this user account.

Type: Container

Parent: user

keys

Description: S3 keys associated with this user account.

Type: Container

Parent: `user`

`swift_keys`

Description: Swift keys associated with this user account.

Type: Container

Parent: `user`

`caps`

Description: User capabilities.

Type: Container

Parent: `user`

SPECIAL ERROR RESPONSES

`UserExists`

Description: Attempt to create existing user.

Code: 409 Conflict

InvalidAccessKey

Description: Invalid access key specified.

Code: 400 Bad Request

InvalidKeyType

Description: Invalid key type specified.

Code: 400 Bad Request

InvalidSecretKey

Description: Invalid secret key specified.

Code: 400 Bad Request

InvalidKeyType

Description: Invalid key type specified.

Code: 400 Bad Request

KeyExists

Description: Provided access key exists and belongs to another user.

Code: 409 Conflict

EmailExists

Description: Provided email address exists.

Code: 409 Conflict

InvalidCap

Description: Attempt to grant invalid admin capability.

Code: 400 Bad Request

MODIFY USER

Modify a user.

caps: users=write

SYNTAX

```
POST /{admin}/user?format=json HTTP/1.1
```

```
Host: {fqdn}
```

REQUEST PARAMETERS

`uid`

Description: The user ID to be modified.

Type: String

Example: `foo_user`

Required: Yes

`display-name`

Description: The display name of the user to be modified.

Type: String

Example: `foo user`

Required: No

`email`

Description: The email address to be associated with the user.

Type: String

Example: `foo@bar.com`

Required: No

`generate-key`

Description: Generate a new key pair and add to the existing keyring.

Type: Boolean

Example: True [False]

Required: No

`access-key`

Description: Specify access key.

Type: String

Example: ABCD0EF12GHIJ2K34LMN

Required: No

secret-key

Description: Specify secret key.

Type: String

Example: 0AbCDEFg1h2i34JklM5nop6QrSTUV+WxyzaBC7D8

Required: No

key-type

Description: Key type to be generated, options are: swift, s3 (default).

Type: String

Example: s3

Required: No

user-caps

Description: User capabilities.

Type: String

Example: `usage=read, write; users=read`

Required: No

`max-buckets`

Description: Specify the maximum number of buckets the user can own.

Type: Integer

Example: 500 [1000]

Required: No

`suspended`

Description: Specify whether the user should be suspended.

Type: Boolean

Example: False [False]

Required: No

RESPONSE ENTITIES

If successful, the response contains the user information.

`user`

Description: A container for the user data information.

Type: Container

`user_id`

Description: The user id.

Type: String

Parent: `user`

`display_name`

Description: Display name for the user.

Type: String

Parent: user

suspended

Description: True if the user is suspended.

Type: Boolean

Parent: user

max_buckets

Description: The maximum number of buckets to be owned by the user.

Type: Integer

Parent: user

subusers

Description: Subusers associated with this user account.

Type: Container

Parent: `user`

`keys`

Description: S3 keys associated with this user account.

Type: Container

Parent: `user`

`swift_keys`

Description: Swift keys associated with this user account.

Type: Container

Parent: `user`

`caps`

Description: User capabilities.

Type: Container

Parent: `user`

SPECIAL ERROR RESPONSES

`InvalidAccessKey`

Description: Invalid access key specified.

Code: 400 Bad Request

`InvalidKeyType`

Description: Invalid key type specified.

Code: 400 Bad Request

`InvalidSecretKey`

Description: Invalid secret key specified.

Code: 400 Bad Request

`KeyExists`

Description: Provided access key exists and belongs to another user.

Code: 409 Conflict

EmailExists

Description: Provided email address exists.

Code: 409 Conflict

InvalidCap

Description: Attempt to grant invalid admin capability.

Code: 400 Bad Request

REMOVE USER

Remove an existing user.

caps: users=write

SYNTAX

```
DELETE /{admin}/user?format=json HTTP/1.1
```

```
Host: {fqdn}
```

REQUEST PARAMETERS

`uid`

Description: The user ID to be removed.

Type: String

Example: `foo_user`

Required: Yes.

`purge-data`

Description: When specified the buckets and objects belonging to the user will also be removed.

Type: Boolean

Example: True

Required: No

RESPONSE ENTITIES

None

SPECIAL ERROR RESPONSES

None.

CREATE SUBUSER

Create a new subuser (primarily useful for clients using the Swift API). Note that either `gen-subuser` or `subuser` is required for a valid request. Note that in general for a subuser to be useful, it must be granted permissions by specifying `access`. As with user creation if `subuser` is specified without `secret`, then a secret key will be automatically generated.

caps: users=write

SYNTAX

```
PUT /{admin}/user?subuser&format=json HTTP/1.1

Host {fqdn}
```

REQUEST PARAMETERS

`uid`

Description: The user ID under which a subuser is to be created.

Type: String

Example: `foo_user`

Required: Yes

`subuser`

Description: Specify the subuser ID to be created.

Type: String

Example: `sub_foo`

Required: No

`secret-key`

Description: Specify secret key.

Type: String

Example: `0AbCDEFG1h2i34JklM5nop6QrSTUV+WxyzaBC7D8`

Required: No

`key-type`

Description: Key type to be generated, options are: swift (default), s3.

Type: String

Example: `swift[swift]`

Required: No

`access`

Description: Set access permissions for sub-user, should be one of `read, write, readwrite, full`.

Type: String

Example: `read`

Required: No

`generate-secret`

Description: Generate the secret key.

Type: Boolean

Example: True [False]

Required: No

RESPONSE ENTITIES

If successful, the response contains the subuser information.

`subusers`

Description: Subusers associated with the user account.

Type: Container

`id`

Description: Subuser id.

Type: String

Parent: `subusers`

`permissions`

Description: Subuser access to user account.

Type: String

Parent: subusers

SPECIAL ERROR RESPONSES

SubuserExists

Description: Specified subuser exists.

Code: 409 Conflict

InvalidKeyType

Description: Invalid key type specified.

Code: 400 Bad Request

InvalidSecretKey

Description: Invalid secret key specified.

Code: 400 Bad Request

InvalidAccess

Description: Invalid subuser access specified.

Code: 400 Bad Request

MODIFY SUBUSER

Modify an existing subuser

caps: users=write

SYNTAX

```
POST /{admin}/user?subuser&format=json HTTP/1.1  
  
Host {fqdn}
```

REQUEST PARAMETERS

`uid`

Description: The user ID under which the subuser is to be modified.

Type: String

Example: `foo_user`

Required: Yes

`subuser`

Description: The subuser ID to be modified.

Type: String

Example: `sub_foo`

Required: Yes

`generate-secret`

Description: Generate a new secret key for the subuser, replacing the existing key.

Type: Boolean

Example: True [False]

Required: No

`secret`

Description: Specify secret key.

Type: String

Example: 0AbCDEFG1h2i34JklM5nop6QrSTUV+WxyzaBC7D8

Required: No

key-type

Description: Key type to be generated, options are: swift (default), s3 .

Type: String

Example: swift [swift]

Required: No

access

Description: Set access permissions for sub-user, should be one of read, write, readwrite, full.

Type: String

Example: `read`

Required: No

RESPONSE ENTITIES

If successful, the response contains the subuser information.

`subusers`

Description: Subusers associated with the user account.

Type: Container

`id`

Description: Subuser id.

Type: String

Parent: `subusers`

`permissions`

Description: Subuser access to user account.

Type: String

Parent: subusers

SPECIAL ERROR RESPONSES

InvalidKeyType

Description: Invalid key type specified.

Code: 400 Bad Request

InvalidSecretKey

Description: Invalid secret key specified.

Code: 400 Bad Request

InvalidAccess

Description: Invalid subuser access specified.

Code: 400 Bad Request

REMOVE SUBUSER

Remove an existing subuser

caps: users=write

SYNTAX

```
DELETE /{admin}/user?subuser&format=json HTTP/1.1  
  
Host {fqdn}
```

REQUEST PARAMETERS

`uid`

Description: The user ID under which the subuser is to be removed.

Type: String

Example: `foo_user`

Required: Yes

`subuser`

Description: The subuser ID to be removed.

Type: String

Example: `sub_foo`

Required: Yes

`purge-keys`

Description: Remove keys belonging to the subuser.

Type: Boolean

Example: True [True]

Required: No

RESPONSE ENTITIES

None.

SPECIAL ERROR RESPONSES

None.

CREATE KEY

Create a new key. If a `subuser` is specified then by default created keys will be swift type.
If only one of `access-key` or `secret-key` is provided the committed key will be

automatically generated, that is if only `secret-key` is specified then `access-key` will be automatically generated. By default, a generated key is added to the keyring without replacing an existing key pair. If `access-key` is specified and refers to an existing key owned by the user then it will be modified. The response is a container listing all keys of the same type as the key created. Note that when creating a swift key, specifying the option `access-key` will have no effect. Additionally, only one swift key may be held by each user or subuser.

caps: users=write

SYNTAX

```
PUT /{admin}/user?key&format=json HTTP/1.1  
  
Host {fqdn}
```

REQUEST PARAMETERS

`uid`

Description: The user ID to receive the new key.

Type: String

Example: `foo_user`

Required: Yes

`subuser`

Description: The subuser ID to receive the new key.

Type: String

Example: `sub_foo`

Required: No

`key-type`

Description: Key type to be generated, options are: swift, s3 (default).

Type: String

Example: `s3` [`s3`]

Required: No

`access-key`

Description: Specify the access key.

Type: String

Example: AB01C2D3EF45G6H7IJ8K

Required: No

secret-key

Description: Specify the secret key.

Type: String

Example: 0ab/CdeFGhij1klmnopqRSTUv1WxyZabcDEFgHij

Required: No

generate-key

Description: Generate a new key pair and add to the existing keyring.

Type: Boolean

Example: True [True]

Required: No

RESPONSE ENTITIES

`keys`

Description: Keys of type created associated with this user account.

Type: Container

`user`

Description: The user account associated with the key.

Type: String

Parent: `keys`

`access-key`

Description: The access key.

Type: String

Parent: `keys`

`secret-key`

Description: The secret key

Type: String

Parent: `keys`

SPECIAL ERROR RESPONSES

`InvalidAccessKey`

Description: Invalid access key specified.

Code: 400 Bad Request

`InvalidKeyType`

Description: Invalid key type specified.

Code: 400 Bad Request

`InvalidSecretKey`

Description: Invalid secret key specified.

Code: 400 Bad Request

InvalidKeyType

Description: Invalid key type specified.

Code: 400 Bad Request

KeyExists

Description: Provided access key exists and belongs to another user.

Code: 409 Conflict

REMOVE KEY

Remove an existing key.

caps: users=write

SYNTAX

```
DELETE /{admin}/user?key&format=json HTTP/1.1
```

```
Host {fqdn}
```

REQUEST PARAMETERS

access-key

Description: The S3 access key belonging to the S3 key pair to remove.

Type: String

Example: AB01C2D3EF45G6H7IJ8K

Required: Yes

uid

Description: The user to remove the key from.

Type: String

Example: foo_user

Required: No

subuser

Description: The subuser to remove the key from.

Type: String

Example: `sub_foo`

Required: No

`key-type`

Description: Key type to be removed, options are: swift, s3. NOTE: Required to remove swift key.

Type: String

Example: `swift`

Required: No

SPECIAL ERROR RESPONSES

None.

RESPONSE ENTITIES

None.

GET BUCKET INFO

Get information about a subset of the existing buckets. If `uid` is specified without `bucket` then all buckets belonging to the user will be returned. If `bucket` alone is specified, information for that particular bucket will be retrieved.

caps: buckets=read

SYNTAX

```
GET /{admin}/bucket?format=json HTTP/1.1  
  
Host {fqdn}
```

REQUEST PARAMETERS

bucket

Description: The bucket to return info on.

Type: String

Example: foo_bucket

Required: No

uid

Description: The user to retrieve bucket information for.

Type: String

Example: `foo_user`

Required: No

`stats`

Description: Return bucket statistics.

Type: Boolean

Example: True [False]

Required: No

RESPONSE ENTITIES

If successful the request returns a buckets container containing the desired bucket information.

`stats`

Description: Per bucket information.

Type: Container

`buckets`

Description: Contains a list of one or more bucket containers.

Type: Container

bucket

Description: Container for single bucket information.

Type: Container

Parent: buckets

name

Description: The name of the bucket.

Type: String

Parent: bucket

pool

Description: The pool the bucket is stored in.

Type: String

Parent: bucket

id

Description: The unique bucket id.

Type: String

Parent: bucket

marker

Description: Internal bucket tag.

Type: String

Parent: bucket

owner

Description: The user id of the bucket owner.

Type: String

Parent: bucket

usage

Description: Storage usage information.

Type: Container

Parent: bucket

index

Description: Status of bucket index.

Type: String

Parent: bucket

SPECIAL ERROR RESPONSES

IndexRepairFailed

Description: Bucket index repair failed.

Code: 409 Conflict

CHECK BUCKET INDEX

Check the index of an existing bucket. NOTE: to check multipart object accounting with `check-objects`, `fix` must be set to True.

caps: buckets=write

SYNTAX

```
GET /{admin}/bucket?index&format=json HTTP/1.1

Host {fqdn}
```

REQUEST PARAMETERS

`bucket`

Description: The bucket to return info on.

Type: String

Example: `foo_bucket`

Required: Yes

`check-objects`

Description: Check multipart object accounting.

Type: Boolean

Example: True [False]

Required: No

`fix`

Description: Also fix the bucket index when checking.

Type: Boolean

Example: False [False]

Required: No

RESPONSE ENTITIES

`index`

Description: Status of bucket index.

Type: String

SPECIAL ERROR RESPONSES

`IndexRepairFailed`

Description: Bucket index repair failed.

Code: 409 Conflict

REMOVE BUCKET

Delete an existing bucket.

caps: buckets=write

SYNTAX

```
DELETE /{admin}/bucket?format=json HTTP/1.1
```

```
Host {fqdn}
```

REQUEST PARAMETERS

`bucket`

Description: The bucket to remove.

Type: String

Example: `foo_bucket`

Required: Yes

`purge-objects`

Description: Remove a buckets objects before deletion.

Type: Boolean

Example: True [False]

Required: No

RESPONSE ENTITIES

None.

SPECIAL ERROR RESPONSES

`BucketNotEmpty`

Description: Attempted to delete non-empty bucket.

Code: 409 Conflict

ObjectRemovalFailed

Description: Unable to remove objects.

Code: 409 Conflict

UNLINK BUCKET

Unlink a bucket from a specified user. Primarily useful for changing bucket ownership.

caps: buckets=write

SYNTAX

```
POST /{admin}/bucket?format=json HTTP/1.1

Host {fqdn}
```

REQUEST PARAMETERS

bucket

Description: The bucket to unlink.

Type: String

Example: `foo_bucket`

Required: Yes

`uid`

Description: The user ID to unlink the bucket from.

Type: String

Example: `foo_user`

Required: Yes

RESPONSE ENTITIES

None.

SPECIAL ERROR RESPONSES

`BucketUnlinkFailed`

Description: Unable to unlink bucket from specified user.

Code: 409 Conflict

LINK BUCKET

Link a bucket to a specified user, unlinking the bucket from any previous user.

caps: buckets=write

SYNTAX

```
PUT /{admin}/bucket?format=json HTTP/1.1

Host {fqdn}
```

REQUEST PARAMETERS

bucket

Description: The bucket to unlink.

Type: String

Example: foo_bucket

Required: Yes

uid

Description: The user ID to link the bucket to.

Type: String

Example: `foo_user`

Required: Yes

RESPONSE ENTITIES

`bucket`

Description: Container for single bucket information.

Type: Container

`name`

Description: The name of the bucket.

Type: String

Parent: `bucket`

`pool`

Description: The pool the bucket is stored in.

Type: String

Parent: bucket

id

Description: The unique bucket id.

Type: String

Parent: bucket

marker

Description: Internal bucket tag.

Type: String

Parent: bucket

owner

Description: The user id of the bucket owner.

Type: String

Parent: bucket

usage

Description: Storage usage information.

Type: Container

Parent: bucket

index

Description: Status of bucket index.

Type: String

Parent: bucket

SPECIAL ERROR RESPONSES

BucketUnlinkFailed

Description: Unable to unlink bucket from specified user.

Code: 409 Conflict

BucketLinkFailed

Description: Unable to link bucket to specified user.

Code: 409 Conflict

REMOVE OBJECT

Remove an existing object. NOTE: Does not require owner to be non-suspended.

caps: buckets=write

SYNTAX

```
DELETE /{admin}/bucket?object&format=json HTTP/1.1
```

```
Host {fqdn}
```

REQUEST PARAMETERS

bucket

Description: The bucket containing the object to be removed.

Type: String

Example: `foo_bucket`

Required: Yes

`object`

Description: The object to remove.

Type: String

Example: `foo.txt`

Required: Yes

RESPONSE ENTITIES

None.

SPECIAL ERROR RESPONSES

`NoSuchObject`

Description: Specified object does not exist.

Code: 404 Not Found

`ObjectRemovalFailed`

Description: Unable to remove objects.

Code: 409 Conflict

GET BUCKET OR OBJECT POLICY

Read the policy of an object or bucket.

caps: buckets=read

SYNTAX

```
GET /{admin}/bucket?policy&format=json HTTP/1.1

Host {fqdn}
```

REQUEST PARAMETERS

`bucket`

Description: The bucket to read the policy from.

Type: String

Example: `foo_bucket`

Required: Yes

`object`

Description: The object to read the policy from.

Type: String

Example: `foo.txt`

Required: No

RESPONSE ENTITIES

If successful, returns the object or bucket policy

`policy`

Description: Access control policy.

Type: Container

SPECIAL ERROR RESPONSES

`IncompleteBody`

Description: Either bucket was not specified for a bucket policy request or bucket and object were not specified for an object policy request.

Code: 400 Bad Request

ADD A USER CAPABILITY

Add an administrative capability to a specified user.

caps: users=write

SYNTAX

```
PUT /{admin}/user?caps&format=json HTTP/1.1  
  
Host {fqdn}
```

REQUEST PARAMETERS

`uid`

Description: The user ID to add an administrative capability to.

Type: String

Example: `foo_user`

Required: Yes

`user-caps`

Description: The administrative capability to add to the user.

Type: String

Example: `usage=read, write`

Required: Yes

RESPONSE ENTITIES

If successful, the response contains the user's capabilities.

`user`

Description: A container for the user data information.

Type: Container

Parent: `user`

`user_id`

Description: The user id.

Type: String

Parent: `user`

`caps`

Description: User capabilities.

Type: Container

Parent: `user`

SPECIAL ERROR RESPONSES

`InvalidCap`

Description: Attempt to grant invalid admin capability.

Code: 400 Bad Request

EXAMPLE REQUEST

```
PUT /{admin}/user?caps&format=json HTTP/1.1
```

```
Host: {fqdn}

Content-Type: text/plain

Authorization: {your-authorization-token}


usage=read
```

REMOVE A USER CAPABILITY

Remove an administrative capability from a specified user.

caps: users=write

SYNTAX

```
DELETE /{admin}/user?caps&format=json HTTP/1.1

Host {fqdn}
```

REQUEST PARAMETERS

uid

Description: The user ID to remove an administrative capability from.

Type: String

Example: `foo_user`

Required: Yes

`user-caps`

Description: The administrative capabilities to remove from the user.

Type: String

Example: `usage=read, write`

Required: Yes

RESPONSE ENTITIES

If successful, the response contains the user's capabilities.

`user`

Description: A container for the user data information.

Type: Container

Parent: `user`

`user_id`

Description: The user id.

Type: String

Parent: `user`

`caps`

Description: User capabilities.

Type: Container

Parent: `user`

SPECIAL ERROR RESPONSES

`InvalidCap`

Description: Attempt to remove an invalid admin capability.

Code: 400 Bad Request

`NoSuchCap`

Description: User does not possess specified capability.

Code: 404 Not Found

SPECIAL ERROR RESPONSES

None.

QUOTAS

The Admin Operations API enables you to set quotas on users and on bucket owned by users. See [Quota Management](#) for additional details. Quotas include the maximum number of objects in a bucket and the maximum storage size in megabytes.

To view quotas, the user must have a `users=read` capability. To set, modify or disable a quota, the user must have `users=write` capability. See the [Admin Guide](#) for details.

Valid parameters for quotas include:

- **Bucket:** The `bucket` option allows you to specify a quota for buckets owned by a user.
- **Maximum Objects:** The `max-objects` setting allows you to specify the maximum number of objects. A negative value disables this setting.
- **Maximum Size:** The `max-size` option allows you to specify a quota for the maximum number of bytes. A negative value disables this setting.
- **Quota Scope:** The `quota-scope` option sets the scope for the quota. The options are `bucket` and `user`.

GET USER QUOTA

To get a quota, the user must have `users` capability set with `read` permission.

```
GET /admin/user?quota&uid=<uid>&quota-type=user
```

SET USER QUOTA

To set a quota, the user must have `users` capability set with `write` permission.

```
PUT /admin/user?quota&uid=<uid>&quota-type=user
```

The content must include a JSON representation of the quota settings as encoded in the corresponding read operation.

GET BUCKET QUOTA

To get a quota, the user must have `users` capability set with `read` permission.

```
GET /admin/user?quota&uid=<uid>&quota-type=bucket
```

SET BUCKET QUOTA

To set a quota, the user must have `users` capability set with `write` permission.

```
PUT /admin/user?quota&uid=<uid>&quota-type=bucket
```

The content must include a JSON representation of the quota settings as encoded in the corresponding read operation.

STANDARD ERROR RESPONSES

`AccessDenied`

Description: Access denied.

Code: 403 Forbidden

InternalServerError

Description: Internal server error.

Code: 500 Internal Server Error

NoSuchUser

Description: User does not exist.

Code: 404 Not Found

NoSuchBucket

Description: Bucket does not exist.

Code: 404 Not Found

NoSuchKey

Description: No such access key.

Code: 404 Not Found

与 OPENSTACK KEYSTONE 对接

It is possible to integrate the Ceph Object Gateway with Keystone, the OpenStack identity service. This sets up the gateway to accept Keystone as the users authority. A user that Keystone authorizes to access the gateway will also be automatically created on the Ceph Object Gateway (if didn't exist beforehand). A token that Keystone validates will be considered as valid by the gateway.

The following configuration options are available for Keystone integration:

```
[client.radosgw.gateway]

rgw keystone url = {keystone server url:keystone server
admin port}

rgw keystone admin token = {keystone admin token}

rgw keystone accepted roles = {accepted user roles}

rgw keystone token cache size = {number of tokens to
cache}

rgw keystone revocation interval = {number of seconds
before checking revoked tickets}

rgw s3 auth use keystone = true

nss db path = {path to nss db}
```

A Ceph Object Gateway user is mapped into a Keystone `tenant`. A Keystone user has different roles assigned to it on possibly more than a single tenant. When the Ceph Object Gateway gets the ticket, it looks at the tenant, and the user roles that are assigned to that ticket, and accepts/rejects the request according to the `rgw keystone accepted roles` configurable.

KILO 之前

Keystone 自身作为对象存储服务的入口（ endpoint ），需要配置为指向 Ceph 对象网关。

```

keystone service-create --name swift --type object-store

keystone endpoint-create --service-id <id> \
    --publicurl    http://radosgw.example.com/swift/v1 \
    --internalurl  http://radosgw.example.com/swift/v1 \
    --adminurl     http://radosgw.example.com/swift/v1

```

从 KILO 起

Keystone 自身作为对象存储服务的入口（ endpoint ），需要配置为指向 Ceph 对象网关。

```

openstack service create --name=swift \
    --description="Swift Service" \
    object-store

```

Field	Value
description	Swift Service
enabled	True
id	37c4c0e79571404cb4644201a4a6e5ee

name	swift	
type	object-store	

+-----+-----+

openstack endpoint create --region RegionOne \

 --publicurl
"http://radosgw.example.com:8080/swift/v1" \

 --adminurl
"http://radosgw.example.com:8080/swift/v1" \

 --internalurl
"http://radosgw.example.com:8080/swift/v1" \

 swift

+-----+-----
-+

Field	Value

+-----+-----
-+

adminurl	http://radosgw.example.com:8080/swift/v1

id	e4249d2b60e44743a67b5e5b38c18dd3

internalurl	http://radosgw.example.com:8080/swift/v1

```
| publicurl      | http://radosgw.example.com:8080/swift/v1
|
| region         | RegionOne
|
| service_id     | 37c4c0e79571404cb4644201a4a6e5ee
|
| service_name   | swift
|
| service_type   | object-store
|
```

```
+-----+-----+
-+
```

\$ openstack endpoint show object-store

```
+-----+-----+
-+
```

```
| Field          | Value
|
```

```
+-----+-----+
-+
```

```
| adminurl       | http://radosgw.example.com:8080/swift/v1
|
| enabled        | True
|
| id             | e4249d2b60e44743a67b5e5b38c18dd3
|
```



```

| internalurl | http://radosgw.example.com:8080/swift/v1
|
| publicurl   | http://radosgw.example.com:8080/swift/v1
|
| region      | RegionOne
|
| service_id  | 37c4c0e79571404cb4644201a4a6e5ee
|
| service_name | swift
|
| service_type | object-store
|
+-----+-----+
-+

```

The keystone URL is the Keystone admin RESTful API URL. The admin token is the token that is configured internally in Keystone for admin requests.

The Ceph Object Gateway will query Keystone periodically for a list of revoked tokens. These requests are encoded and signed. Also, Keystone may be configured to provide self-signed tokens, which are also encoded and signed. The gateway needs to be able to decode and verify these signed messages, and the process requires that the gateway be set up appropriately. Currently, the Ceph Object Gateway will only be able to perform the procedure if it was compiled with `--with-nss`. Configuring the Ceph Object Gateway to work with Keystone also requires converting the OpenSSL certificates that Keystone uses for creating the requests to the nss db format, for example:

```
mkdir /var/ceph/nss
```

```
openssl x509 -in /etc/keystone/ssl/certs/ca.pem -pubkey | \
    certutil -d /var/ceph/nss -A -n ca -t
"TCu,Cu,Tuw"

openssl x509 -in /etc/keystone/ssl/certs/signing_cert.pem
-pubkey | \
    certutil -A -d /var/ceph/nss -n signing_cert -t
"P,P,P"
```

RADOSGW – RADOS REST 风格的网关

提纲

radosgw

描述

radosgw 是 RADOS 对象存储的一个 HTTP REST 网关，是 Ceph 分布式存储系统的一部分。它是用 libfcgi 实现的一个 FastCGI 模块，可联合任何支持 FastCGI 功能的网页服务器使用。

选项

-c ceph.conf, **--conf**=ceph.conf

用指定的 `ceph.conf` 配置文件而非默认的 `/etc/ceph/ceph.conf` 来确定启动时所需的监视器地址。

-m monaddress[:port]

连接到指定监视器，而非通过 `ceph.conf` 查询。

-i ID, **--id** ID

设置 radosgw 名字的 ID 部分。

-n TYPE.ID, **--name** TYPE.ID

设置网关的 rados 用户名（如 client.radosgw.gateway）。

--cluster NAME

设置集群名称（默认：ceph）

-d

在前台运行，日志记录到标准错误

-f

在前台运行，日志记录到正常位置

--rgw-socket-path=path

指定 Unix 域套接字的路径

--rgw-region=region

radosgw 所在 region

--rgw-zone=zone

radosgw 所在的区域

配置

先前的 RADOS 网关配置依

赖 Apache 和 `mod_fastcgi`；现在则

用 `mod_proxy_fcgi` 替换

了 `mod_fastcgi`，因为后者使用了非自由许可证。`mod_proxy_fcgi` 不同于传统的 FastCGI 模块，它需要 `mod_proxy` 模块所支持的 FastCGI 协议。所以，要处理 FastCGI 协议，服务器需同时有

`mod_proxy` 和 `mod_proxy_fcgi` 模块。

不

像 `mod_fastcgi`，`mod_proxy_fcgi`

不能启动应用进程。某些平台提供

了 `fcgistarter` 来实现此功能。然而，FastCGI 应用框架有可能具备外部启动或进程管理功能。

Apache 可以通过本机 TCP 连接或 Unix 域套接字使用 `mod_proxy_fcgi` 模块。不支持 Unix 域套接字的 `mod_proxy_fcgi`，像 Apache 2.2 和 2.4 的早期版本，必需通过本机 TCP 连接。

1. 更改 `/etc/ceph/ceph.conf` 文件，让 `radosgw` 使用 TCP 而非 Unix 域套接字。

```
2. [client.radosgw.gateway]

3. host = {hostname}

4. keyring =
    /etc/ceph/ceph.client.radosgw.keyring

5. rgw socket path =
    ""

6. log file =
    /var/log/ceph/client.radosgw.gateway.log

7. rgw frontends =
    fastcgi
    socket_port=9000
    socket_host=0.0.0.0
```

```
rgw print continue
= false
```

8. 把下列内容加入网关配置文件：

在 Debian/Ubuntu 上，加入 `/etc/apache2/conf-available/rgw.conf`

```
<VirtualHost *:80>

    ServerName
    localhost

    DocumentRoot
    /var/www/html

    ErrorLog
    /var/log/apache2/rgw_error.log

    CustomLog
    /var/log/apache2/rgw_access.log
    combined

    # LogLevel debug

    RewriteEngine On

    RewriteRule .* -
    [E=HTTP_AUTHORIZATI
    ON:%{HTTP:Authoriza
    tion},L]
```

```
SetEnv proxy-  
nokeepalive 1  
  
ProxyPass /  
fcgi://localhost:90  
00/  
  
</VirtualHost>
```

在 CentOS/RHEL 上，加入 `/etc/httpd/conf.d/rgw.conf`:

```
<VirtualHost *:80>  
  
ServerName  
localhost  
  
DocumentRoot  
/var/www/html  
  
ErrorLog  
/var/log/httpd/rgw_  
error.log  
  
CustomLog  
/var/log/httpd/rgw_  
access.log combined
```

```
# LogLevel debug

RewriteEngine On

RewriteRule .* -
[E=HTTP_AUTHORIZATI
ON:%{HTTP:Authoriza
tion},L]

SetEnv proxy-
nokeepalive 1

ProxyPass /
fcgi://localhost:90
00/

</VirtualHost>
```

9. 对于搭载了支持 Unix 域套接字的 Apache 2.4.9 及更高版的发行版，可使用下列配置：

```
10.      [client.rados
        gw.gateway]

11.      host =
        {hostname}
```

```
12.      keyring =  
        /etc/ceph/ceph.clie  
        nt.radosgw.keyring  
  
13.      rgw socket  
        path =  
        /var/run/ceph/ceph.  
        radosgw.gateway.fas  
        tcgi.sock  
  
14.      log file =  
        /var/log/ceph/clien  
        t.radosgw.gateway.l  
        og
```

```
rgw print continue  
= false
```

15. 把下列内容加入网关配置文件中：

在 CentOS/RHEL 上，加
入 `/etc/httpd/conf.d/rgw.
conf`:

```
<VirtualHost *:80>  
  
    ServerName  
    localhost  
  
    DocumentRoot  
    /var/www/html  
  
    ErrorLog  
    /var/log/httpd/rgw_  
    error.log
```



```
CustomLog  
/var/log/httpd/rgw_  
access.log combined
```

```
# LogLevel debug
```

```
RewriteEngine On
```

```
RewriteRule .* -  
[E=HTTP_AUTHORIZATI  
ON:%{HTTP:Authoriza  
tion},L]
```

```
SetEnv proxy-  
nokeepalive 1
```

```
ProxyPass /  
unix:///var/run/cep  
h/ceph.radosgw.gate  
way.fastcgi.sock|fc  
gi://localhost:9000  
/
```

```
</VirtualHost>
```

Ubuntu 14.04 自
带 Apache 2.4.7 , 它不支持

Unix 域套接字，所以必须配置成本机 TCP。Unix 域套接字支持存在于 Apache 2.4.9 及其后续版本中。已经有人提交了申请，要求把 UDS 支持移植到 Ubuntu 14.04 的 Apache 2.4.7。在这里：

<https://bugs.launchpad.net/ubuntu/+source/apache2/+bug/1411030>

16. 给 radosgw 生成一个密钥，用于到集群认证。

```
17.      ceph-authtool
      -C -n
      client.radosgw.gate
      way --gen-key
      /etc/ceph/keyring.r
      adosgw.gateway
```

```
ceph-authtool -n
client.radosgw.gate
way --cap mon
'allow rw' --cap
osd 'allow rwx'
/etc/ceph/keyring.r
adosgw.gateway
```

18. 把密钥导入集群。

```
ceph auth add
client.radosgw.gate
way --in-
file=keyring.radosg
w.gateway
```

19. 启动 Apache 和 radosgw 。

Debian/Ubuntu:

```
sudo  
/etc/init.d/apache2  
start  
  
sudo  
/etc/init.d/radosgw  
start
```

CentOS/RHEL:

```
sudo apachectl  
start  
  
sudo  
/etc/init.d/ceph-  
radosgw start
```

记录使用日志

radosgw 会异步地维护使用率日志，它会累积用户操作统计并周期性地刷回。可用 **radosgw-admin** 访问和管理日志。

记录的信息包括数据传输总量、操作总量、成功操作总量。这些数据是按小时记录到桶所有者名下的，除非操作是针对服务的（如罗列桶时），这时会记录到操作用户名下。

下面是个配置实例：

```
[client.radosgw.gateway]
```

```
rgw enable usage log =  
true  
  
rgw usage log tick  
interval = 30  
  
rgw usage log flush  
threshold = 1024  
  
rgw usage max shards =  
32  
  
rgw usage max user  
shards = 1
```

碎片总数决定着总共需要多少对象来保存使用日志信息。每用户碎片数确定了为单个用户保存使用信息需多少对象。tick interval 可配置刷回日志的间隔秒数，flush threshold 决定了保留的日志条数达到多少才调用异步刷回。

使用范围

radosgw 是 Ceph 的一部分，这是个伸缩力强、开源、分布式的存储系统，更多信息参见 <http://ceph.com/docs>。

参考

ceph(8) *radosgw-admin*(8)

RADOSGW-ADMIN – RADOS REST 网关的用户管理工具

提纲

radosgw-admin *command* [*options ...*]

描述

radosgw-admin 是 RADOS 网关用户管理工具，可用于创建和修改用户。

命令

radosgw-admin 工具有很多命令，可完成各种管理任务：

user create

创建一个新用户。

user modify

修改一个用户。

user info

显示用户信息，以及可能存在的子用户和密钥。

user rm

删除一个用户。

user suspend

暂停某用户。

user enable

重新允许暂停的用户。

user check

检查用户信息。

user stats

显示配额子系统统计的用户状态。

caps add

给用户分配能力。

caps rm

删除用户能力。

subuser create

新建一个子用户（适合使用 Swift API 的客户端）。

subuser modify

修改子用户。

subuser rm

删除子用户

key create

新建访问密钥。

key rm

删除访问密钥。

bucket list

罗列所有桶。

bucket link

把桶关联到指定用户。

bucket unlink

取消指定用户和桶的关联。

buck et stats

返回桶的统计信息。

b u c k e t r m

删除一个桶。

b
u
c
k
e
t

c
h
e
c
k

检查桶的索引信息。

o
b
j
e
c
t

r
m

删除一个对象。

o
b
j
e
c
t

u
n
l
i
n
k

从桶索引里去掉对象。

o
u
o
t
a

设置配额参数。

启用配额。

禁用配额。

显示 region 信息。

列出本集群配置的所有 region 。

设置 region 信息（需要输入文件）。

设置默认 region 。

显示 region-map 。

设置 region-map （需要输入文件）。

显示区域集群参数。

设置区域集群参数（需要输入文件）。

列出本集群内配置的所有区域。

增加一个已有存储池用于数据归置。

从数据归置集删除一个已有存储池。

罗列归置活跃集。

显示桶或对象相关的策略。

罗列日志对象。

显示指定对象内（或指定桶、日期、桶标识符）的日志。

删除日志对象。

查看使用率信息（可选选项有用户和数据范围）。

修剪使用率信息（可选选项有用户和数据范围）。

删除指定日期（时间可选）之前创建的临时对象。

显示过期的垃圾回收对象（加 `-include-all` 选项罗列所有条目，包括未过期的）。

手动处理垃圾。

读取元数据信息。

设置元数据信息。

删除元数据信息。

罗列元数据信息。

罗列元数据日志。

截截元数据日志。

罗列桶索引日志。

截桶索引日志（需要起始标记、结束标记）。

罗列数据日志。

截截数据日志。

罗列含状态操作（需要 client_id 、 op_id 、对象）。

设置条目状态（需指定 client_id 、 op_id 、对象、状态）。

更新某一条目的状态（需指定 `client_id`、`op_id`、对象）。

删除条目（需指定 `client_id`、`op_id`、对象）。

读取复制元数据日志条目。

删除复制元数据日志条目。

用指定的 `ceph.conf` 配置文件而非默认的 `/etc/ceph/ceph.conf` 来确定启动时所需的监视器地址。

连接到指定监视器，而非通过 `ceph.conf` 查询。

`radosgw` 用户的 ID 。

子用户名字。

用户的电子邮件地址。

配置用户的显示名称（昵称）

S3 访问密钥。

生成随机访问密钥（给 S3）。

指定密钥的密文。

生成随机密钥。

密钥类型，可用的有： swift、 S3 。

临时 URL 密钥。

给用户设置系统标识。

指定桶名

指定对象名

某些命令所需的日期

某些命令所需的起始日期

某些命令所需的终结日期

执行 `mdlog list` 时为可选项。

对 `mdlog trim`、`replica mdlog get/delete`、`replica datalog get/delete` 来说是必须的。

librados 认证所需的 `aid` 。

删除用户前先删除用户数据。

若加了此选项，删除子用户时将一起删除其所有密钥。

删除桶前先删除其内所有对象。

用 `metadata get` 检索元数据时用的密钥。

radosgw 所在的 `region` 。

radosgw 所在的区域。

除了检查桶索引，还修复它。

检查桶：根据对象的实际状态重建桶索引。

为某些操作指定输出格式：xml、json。

`user stats` 的选项，收集用户的桶索引状态、并同步到用户状态。

执行 `log show` 时，显示或不显示日志条目。

执行 `log show` 时，显示或不显示日志汇总。

让 `log show` 只显示数字字段非零的日志。

设置时指定要读取的文件。

给 `opstate set` 命令指定状态。

复制日志类型（ metadata、data、bucket ），操作复制日志时需要。

逗号分隔的一系列类目，显示使用情况时需要。

能力列表，如 “usage=read, write; user=read”。

某些特定操作需要。

为配额命令指定桶。

指定最大对象数（负数为禁用）。

指定最大尺寸（单位为字节，负数为禁用）。

配额有效范围（桶、用户）。

--

--

--

--

--

--

--

API 文档

CEPH 存储集群 API

见 [Ceph 存储集群 API](#)。

CEPH 文件系统 API

见 [libcephfs \(javadoc\)](#)。

CEPH 块设备 API

见 [librbdpy](#)。

CALAMARI APIS

见 [Calamari API](#)。

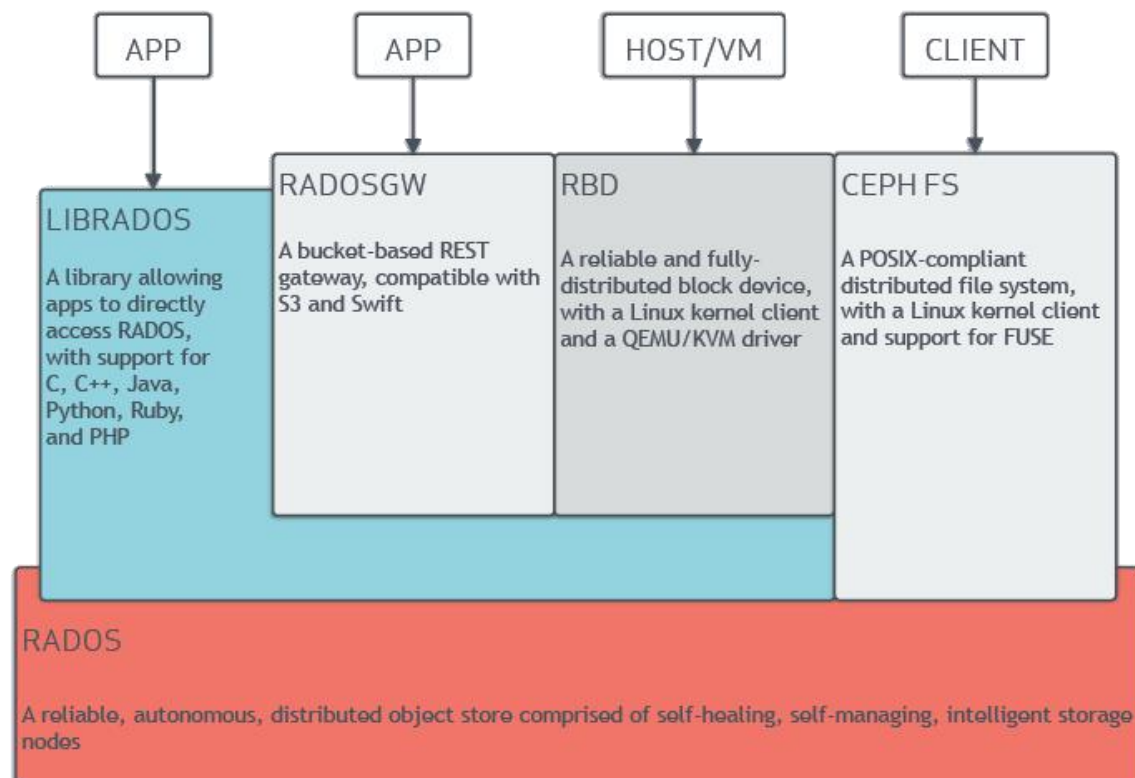
CEPH 对象存储 API

- 见 [S3-compatible API](#)。

- 见 [Swift-compatible API](#) 。
- 见 [Admin Ops API](#) 。

体系结构

Ceph 独一无二地用统一的系统提供了**对象、块、和文件存储**功能，它可靠性高、管理简便、并且是自由软件。Ceph 的强大足以改变贵公司的 IT 基础架构、和管理海量数据的能力。Ceph 可提供极大的伸缩性——供成千用户访问 PB 乃至 EB 级的数据。**Ceph 节点**以普通硬件和智能守护进程作为支撑点，**Ceph 存储集群**组织起了大量节点，它们之间靠相互通讯来复制数据、并动态地重分布数据。



CEPH 存储集群

Ceph 提供了一个可无限伸缩的 **Ceph 存储集群**，它基于 RADOS，见论文 [RADOS - A Scalable, Reliable Storage Service for Petabyte-scale Storage Clusters](#)。

Ceph 存储集群包含两种类型的守护进程：

- Ceph 监视器
- Ceph OSD 守护进程



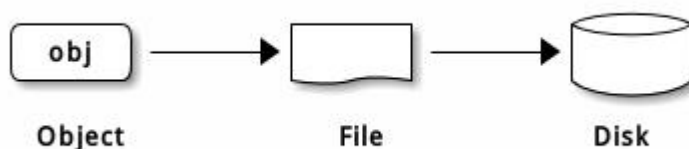
Ceph 监视器维护着集群运行图的主副本。一个监视器集群确保了当某个监视器失效时的高可用性。存储集群客户端向 Ceph 监视器索取集群运行图的最新副本。

Ceph OSD 守护进程检查自身状态、以及其它 OSD 的状态，并报告给监视器们。

存储集群的客户端和各个 Ceph OSD 守护进程使用 CRUSH 算法高效地计算数据位置，而不是依赖于一个中心化的查询表。它的高级功能包括：基于 librados 的原生存储接口、和多种基于 librados 的服务接口。

数据的存储

Ceph 存储集群从 Ceph 客户端接收数据——不管是来自 Ceph 块设备、Ceph 对象存储、Ceph 文件系统、还是基于 librados 的自定义实现——并存储为对象。每个对象是文件系统中的文件，它们存储在对象存储设备上。由 Ceph OSD 守护进程处理存储设备上的读/写操作。



Ceph OSD 在扁平的命名空间内把所有数据存储为对象（也就是没有目录层次）。对象包含一个标识符、二进制数据、和由名字/值对组成的元数据，元数据语义完全取决于 Ceph 客户端。例如，CephFS 用元数据存储文件属性，如文件所有者、创建日期、最后修改日期等等。

ID	Binary Data	Metadata	
1234	0101010101010100110101010010 0101100001010100110101010010 0101100001010100110101010010	name1 name2 nameN	value1 value2 valueN

Note

一个对象 ID 不止在本地唯一，它在整个集群内都是唯一的。

伸缩性和高可用性

在传统架构里，客户端与一个中心化的组件通信（如网关、中间件、API、前端等等），它作为一个复杂子系统的唯一入口，它引入单故障点的同时，也限制了性能和伸缩性（就是说如果中心化组件挂了，整个系统就挂了）。

Ceph 消除了集中网关，允许客户端直接和 Ceph OSD 守护进程通讯。Ceph OSD 守护进程自动在其它 Ceph 节点上创建对象副本来确保数据安全和高可用性；为保证高可用性，监视器也实现了集群化。为消除中心节点，Ceph 使用了 CRUSH 算法。

CRUSH 简介

Ceph 客户端和 OSD 守护进程都用 CRUSH 算法来计算对象的位置信息，而不是依赖于一个中心化的查询表。与以往方法相比，CRUSH 的数据管理机制更好，它很干脆地把工作分配给集群内的所有客户端和 OSD 来处理，因此具有极大的伸缩性。CRUSH 用智能数据复制确保弹性，更能适应超大规模存储。下列几段描述了 CRUSH 如何工作，更详细的机制请参阅论文：[CRUSH - 可控、可伸缩、分布式地归置多副本数据](#)。

集群运行图

Ceph 依赖于 Ceph 客户端和 OSD，因为它们知道集群的拓扑，这个拓扑由 5 张图共同描述，统称为“集群运行图”：

1. **Montior Map:** 包含集群的 `fsid`、位置、名字、地址和端口，也包括当前版本、创建时间、最近修改时间。要查看监视器图，用 `ceph mondump` 命令。

2. **OSD Map:** 包含集群 `fsid`、创建时间、最近修改时间、存储池列表、副本数量、归置组数量、OSD 列表及其状态（如 `up`、`in`）。要查看 OSD 运行图，用 `ceph osd dump` 命令。
3. **PG Map:** : ** 包含归置组版本、其时间戳、最新的 OSD 运行图版本、占满率、以及各归置组详情，像归置组 ID、`up set`、`acting set`、PG 状态（如 `active+clean`），和各存储池的数据使用情况统计。
4. **CRUSH Map:** : ** 包含存储设备列表、故障域树状结构（如设备、主机、机架、行、房间、等等）、和存储数据时如何利用此树状结构的规则。要查看 CRUSH 规则，执行 `ceph osd getcrushmap -o {filename}` 命令；然后用 `crushtool -d {comp-crushmap-filename} -o{decomp-crushmap-filename}` 反编译；然后就可以用 `cat` 或编辑器查看了。
5. **MDS Map:** 包含当前 MDS 图的版本、创建时间、最近修改时间，还包含了存储元数据的存储池、元数据服务器列表、还有哪些元数据服务器是 `up` 且 `in` 的。要查看 MDS 图，执行 `ceph mds dump`。

各运行图维护着各自运营状态的变更，Ceph 监视器维护着一份集群运行图的主拷贝，包括集群成员、状态、变更、以及 Ceph 存储集群的整体健康状况。

高可用监视器

Ceph 客户端读或写数据前必须先连接到某个 Ceph 监视器、获得最新的集群运行图副本。一个 Ceph 存储集群只需要单个监视器就能运行，但它就成了单一故障点（即如果此监视器宕机，Ceph 客户端就不能读写数据了）。

为增强可靠性和容错能力，Ceph 支持监视器集群；在一个监视器集群内，延时以及其它错误会导致一到多个监视器滞后于集群的当前状态，因此，Ceph 的各监视器例程必须就集群的当前状态达成一致。Ceph 总是使用大多数监视器（如：1、2:3、3:5、4:6 等等）和

[Paxos](#) 算法就集群的当前状态达成一致。

关于配置监视器的详情，见[监视器配置参考](#)。

高可用性认证

为识别用户并防止中间人攻击，Ceph 用 `cephx` 认证系统来认证用户和守护进程。

Note

cephx 协议不解决传输加密（如 SSL/TLS）、或者存储加密问题。

Cephx 用共享密钥来认证，即客户端和监视器集群各自都有客户端密钥的副本。这样的认证协议使参与双方不用展现密钥就能相互认证，就是说集群确信用户拥有密钥、而且用户相信集群有密钥的副本。

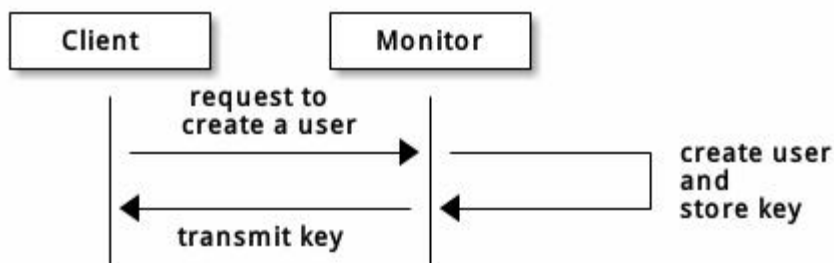
Ceph 一个主要伸缩功能就是避免了对对象存储的中央接口，这就要求 Ceph 客户端能直接和 OSD 交互。Ceph 通过 cephx 认证系统保护数据，它也认证运行 Ceph 客户端的用户，cephx 协议运行机制类似 [Kerberos](#)。

用户/参与者通过调用 Ceph 客户端来联系监视器，不像 Kerberos，每个监视器都能认证用户、发布密钥，所以使用 cephx 时不会有单点故障或瓶颈。监视器返回一个类似 Kerberos 票据的认证数据结构，它包含一个可用于获取 Ceph 服务的会话密钥，会话密钥是用户的永久私钥自加密过的，只有此用户能从 Ceph 监视器请求服务。客户端用会话密钥向监视器请求需要的服务，然后监视器给客户端一个凭证用以向实际持有数据的 OSD 认证。Ceph 的监视器和 OSD 共享相同的密钥，所以集群内任何 OSD 或元数据服务器都认可客户端从监视器获取的凭证，像 Kerberos 一样 cephx 凭证也会过期，以使攻击者不能用暗中得到的过期凭证或会话密钥。只要用户的私钥过期前没有泄露，这种认证形式就可防止中间线路攻击者以别人的 ID 发送垃圾消息、或修改用户的正常消息。

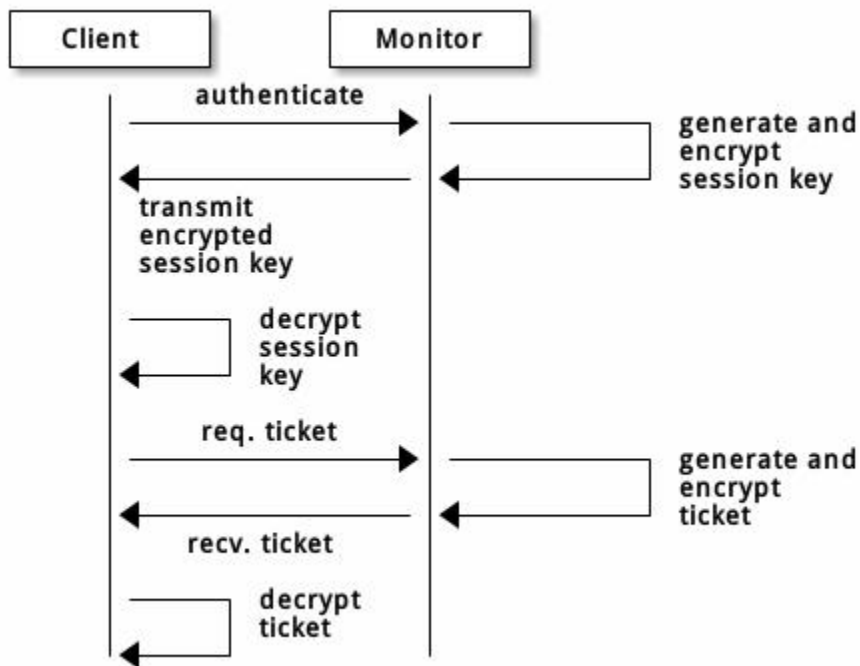
要使用 cephx，管理员必须先设置好用户。在下面的图解里，client.admin 用户从命令行调用 `ceph auth get-or-create-key` 来生成一个用户及其密钥，Ceph 的认证子系统生成了用户名和密钥、副本存到监视器然后把此用户的密钥回传给 client.admin 用户，也就是说客户端和监视器共享着相同的密钥。

Note

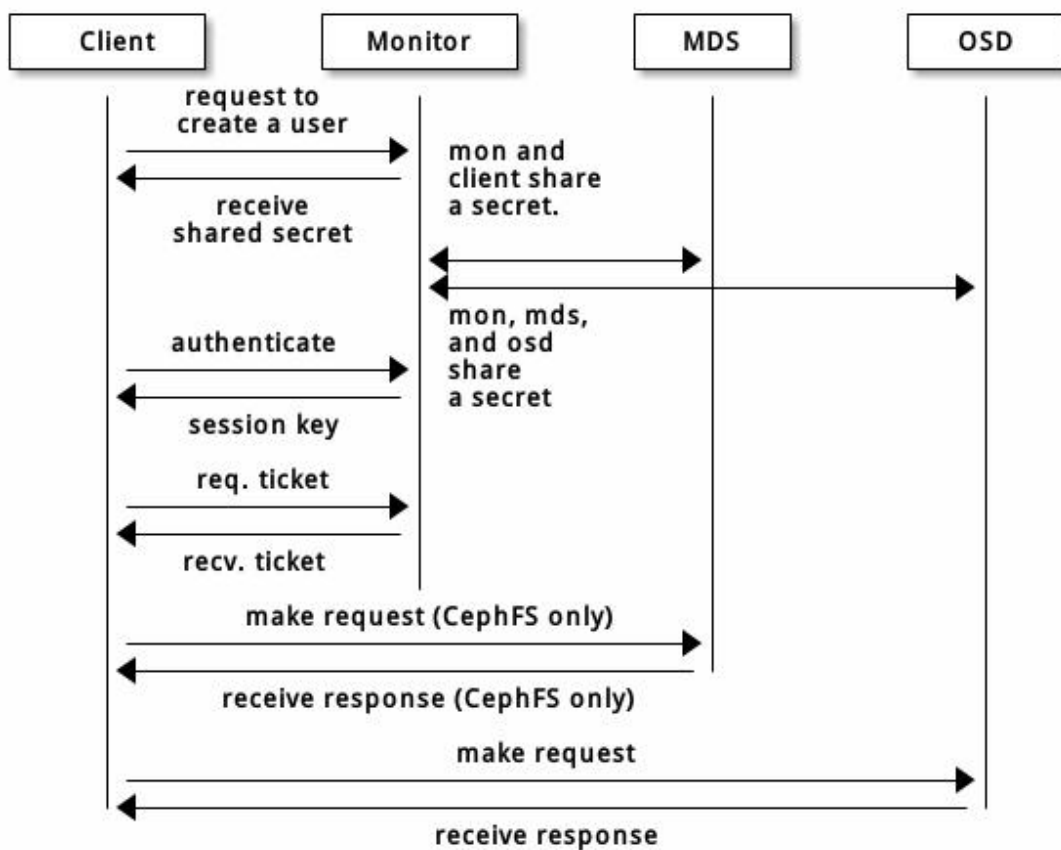
client.admin 用户必须以安全方式把此用户 ID 和密钥交给用户。



要和监视器认证，客户端得把用户名传给监视器，然后监视器生成一个会话密钥、并且用此用户的密钥加密它，然后把加密的凭证回传给客户端，客户端用共享密钥解密载荷就可获取会话密钥。会话密钥在当前会话中标识了此用户，客户端再用此会话密钥签署过的用户名请求一个凭证，监视器生成一个凭证、用用户的密钥加密它，然后回传给客户端，客户端解密此凭证，然后用它签署连接集群内 OSD 和元数据服务器的请求。



cephx 协议认证客户端机器和 Ceph 服务器间正在进行的通讯，二者间认证完成后的每条消息都用凭证签署过，监视器、OSD、元数据服务器都可用此共享的密钥来校验这些消息。



认证提供的保护位于 Ceph 客户端和服务端间，没有扩展到 Ceph 客户端之外。如果用户从远程主机访问 Ceph 客户端，Ceph 认证就不管用了，它不会影响到用户主机和客户端主机间的通讯。

关于配置细节，请参考 [Cephx 配置指南](#)；关于用户管理细节，请参考[用户管理](#)。

智能程序支撑超大规模

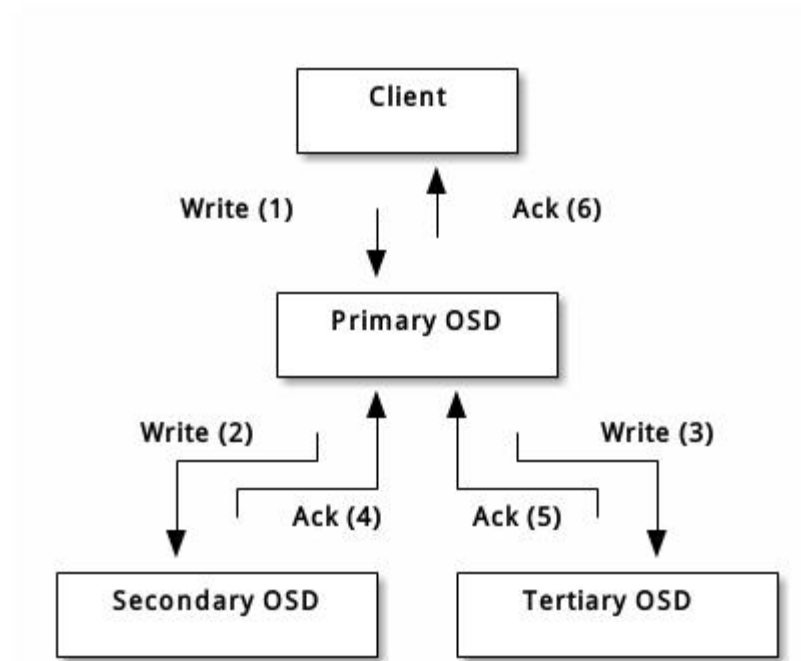
在很多集群架构中，集群成员的主要目的就是让集中式接口知道它能访问哪些节点，然后此中央接口通过一个两级调度为客户端提供服务，在 PB 到 EB 级系统中这个调度系统必将成为最大的瓶颈。

Ceph 消除了此瓶颈：其 OSD 守护进程和客户端都能感知集群，比如 Ceph 客户端、各 OSD 守护进程都知道集群内其他的 OSD 守护进程，这样 OSD 就能直接和其它 OSD 守护进程和监视器通讯。另外，Ceph 客户端也能直接和 OSD 守护进程交互。

Ceph 客户端、监视器和 OSD 守护进程可以相互直接交互，这意味着 OSD 可以利用本地节点的 CPU 和内存执行那些有可能拖垮中央服务器的任务。这种设计均衡了计算资源，带来几个好处：

1. **OSD 直接服务于客户端**：由于任何网络设备都有最大并发连接上限，规模巨大时中央化的系统其物理局限性就暴露了。Ceph 允许客户端直接和 OSD 节点联系，这在消除单故障点的同时，提升了性能和系统总容量。Ceph 客户端可按需维护和某 OSD 的会话，而不是一中央服务器。
2. **OSD 成员和状态**：Ceph OSD 加入集群后会持续报告自己的状态。在底层，OSD 状态为 up 或 down，反映它是否在运行、能否提供服务。如果一 OSD 状态为 down 且 in，表明 OSD 守护进程可能故障了；如果一 OSD 守护进程没在运行（比如崩溃了），它就不能亲自向监视器报告自己是 down 的。Ceph 监视器能周期性地 ping OSD 守护进程，以确保它们在运行，然而它也授权 OSD 进程去确认邻居 OSD 是否 down 了，并更新集群运行图、报告给监视器。这种机制意味着监视器还是轻量级进程。详情见[监控 OSD 和心跳](#)。
3. **数据清洗**：作为维护数据一致性和清洁度的一部分，OSD 能清洗归置组内的对象。就是说，Ceph OSD 能比较对象元数据与存储在其它 OSD 上的副本元数据，以捕捉 OSD 缺陷或文件系统错误（每天）。OSD 也能做深度清洗（每周），即按位比较对象中的数据，以找出轻度清洗时未发现的硬盘坏扇区。关于清洗详细配置见[数据清洗](#)。
4. **复制**：和 Ceph 客户端一样，OSD 也用 CRUSH 算法，但用于计算副本存到哪里（也用于重均衡）。一个典型的写情形是，一客户端用 CRUSH 算法算出对象应存到哪里，并把对象映射到存储池和归置组，然后查找 CRUSH 图来确定此归置组的主 OSD。

客户端把对象写入目标归置组的主 OSD，然后这个主 OSD 再用它的 CRUSH 图副本找出用于放对象副本的第二、第三个 OSD，并把数据复制到适当的归置组所对应的第二、第三 OSD（要多少副本就有多少 OSD），最终，确认数据成功存储后反馈给客户端。



有了做副本的能力，OSD 守护进程就可以减轻客户端的复制压力，同时保证了数据的高可靠性和安全性。

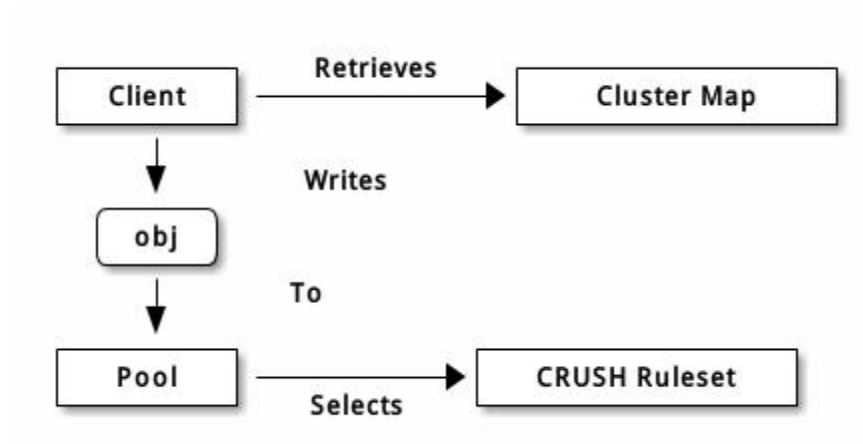
动态集群管理

在[伸缩性和高可用性](#)一节，我们解释了 Ceph 如何用 CRUSH、集群感知性和智能 OSD 守护进程来扩展和维护高可靠性。Ceph 的关键设计是自治，自修复、智能的 OSD 守护进程。让我们深入了解下 CRUSH 如何运作，如何动态实现现代云存储基础设施的数据放置、重均衡、错误恢复。

关于存储池

Ceph 存储系统支持“池”概念，它是存储对象的逻辑分区。

Ceph 客户端从监视器获取一张[集群运行图](#)，并把对象写入存储池。存储池的 `size` 或副本数、CRUSH 规则集和归置组数量决定着 Ceph 如何放置数据。



存储池至少可设置以下参数：

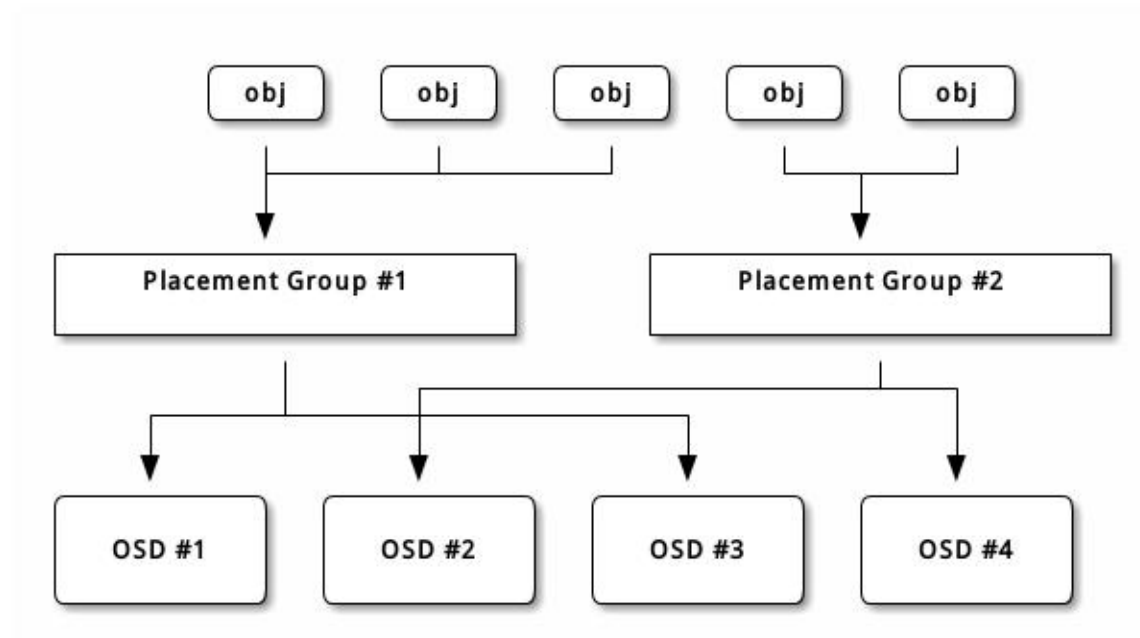
- 对象的所有权/访问权限；
- 归置组数量；以及，
- 使用的 CRUSH 规则集。

详情见[调整存储池](#)。

PG 映射到 OSD

每个存储池都有很多归置组，CRUSH 动态的把它们映射到 OSD。Ceph 客户端要存对象时，CRUSH 将把各对象映射到某个归置组。

把对象映射到归置组在 OSD 和客户端间创建了一个间接层。由于 Ceph 集群必须能增大或缩小、并动态地重均衡。如果让客户端“知道”哪个 OSD 有哪个对象，就会导致客户端和 OSD 紧耦合；相反，CRUSH 算法把对象映射到归置组、然后再把各归置组映射到一或多个 OSD，这一间接层可以让 Ceph 在 OSD 守护进程和底层设备上线时动态地重均衡。下列图表描述了 CRUSH 如何将对象映射到归置组、再把归置组映射到 OSD。



有了集群运行图副本和 CRUSH 算法，客户端就能精确地计算出到哪个 OSD 读、写某特定对象。

计算 PG ID

Ceph 客户端绑定到某监视器时，会索取最新的**集群运行图**副本，有了此图，客户端就能知道集群内的所有监视器、OSD、和元数据服务器。然而它对对象的位置一无所知。

对象位置是计算出来的。

客户端只需输入对象 ID 和存储池，此事简单：Ceph 把数据存在某存储池（如 liverpool）中。当客户端想要存命名对象（如 john、paul、george、ringo 等等）时，它用对象名，一个哈希值、存储池中的归置组数、存储池名计算归置组。Ceph 按下列步骤计算 PG ID。

1. 客户端输入存储池 ID 和对象 ID（如 pool="liverpool" 和 object-id="john"）；
2. CRUSH 拿到对象 ID 并哈希它；
3. CRUSH 用 PG 数（如 58）对哈希值取模，这就是归置组 ID；
4. CRUSH 根据存储池名取得存储池 ID（如 liverpool = 4）；
5. CRUSH 把存储池 ID 加到 PG ID（如 4.58）之前。

计算对象位置远快于查询定位，CRUSH 算法允许客户端计算对象应该存到哪里，并允许客户端连接主 OSD 来存储或检索对象。

互联和子集

在前面的章节中，我们注意到 OSD 守护进程相互检查心跳并回馈给监视器；它们的另一行为叫“互联（peering）”，这是一种把一归置组内所有对象（及其元数据）所在的 OSD 带到一致状态的过程。事实上，OSD 守护进程会向监视器[报告互联失败](#)，互联问题一般会自行恢复，然而如果问题一直持续，你也许得参照[互联失败排障](#)解决。

Note

状态达成一致并不意味着 PG 持有最新内容。

Ceph 存储集群被设计为至少存储两份对象数据（即 `size = 2`），这是保证数据安全的最小要求。为保证高可用性，Ceph 存储集群应该保存两份以上的对象副本（如 `size = 3` 且 `min size = 2`），这样才能在“degraded”状态继续运行，同时维持数据安全。

回想前面[智能程序支撑超大规模](#)中的图表，我们没明确地提 OSD 守护进程的名字（如 `osd.0`、`osd.1` 等等），而是称之为主、次、以此类推。按惯例，主 OSD 是 *acting set* 中的第一个 OSD，而且它负责协调以它为主 OSD 的各归置组的互联，也[只有它会接受](#)客户端到某归置组内对象的写入请求。

当一系列 OSD 负责一归置组时，这一系列的 OSD 就成为一个 *acting set*。一个 *acting set* 对应当前负责此归置组的一组 OSD，或者说截止到某个版本为止负责某个特定归置组的那些 OSD。

OSD 守护进程作为 *acting set* 的一部分，不一定总在 `up` 状态。当一 OSD 在 *acting set* 中是 `up` 状态时，它就是 `up set` 的一部分。`up set` 是个重要特征，因为某 OSD 失败时 Ceph 会把 PG 映射到其他 OSD。

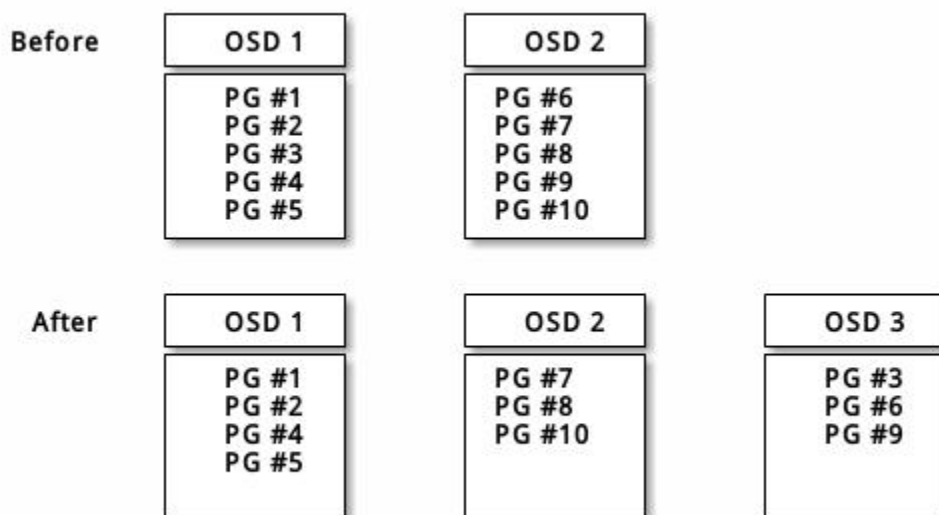
Note

在某 PG 的 *acting set* 中包含了 `osd.25`、`osd.32` 和 `osd.61`，第一个 `osd.25` 是主 OSD，如果它失败了，第二个 `osd.32` 就成为主 OSD，`osd.25` 会被移出 *up set*。

重均衡

你向 Ceph 存储集群新增一 OSD 守护进程时，集群运行图就要用新增的 OSD 更新。回想[计算 PG ID](#)，这个动作会更改集群运行图，因此也改变了对象位置，因为计算时的输入条件变

了。下面的图描述了重均衡过程（此图很粗略，因为在大型集群里变动幅度小的多），是其中的一些而不是所有 PG 都从已有 OSD（OSD 1 和 2）迁移到新 OSD（OSD 3）。即使在重均衡中，CRUSH 都是稳定的，很多归置组仍维持最初的配置，且各 OSD 都腾出了些空间，所以重均衡完成后新 OSD 上不会出现负载突增。



数据一致性

作为维护数据一致和清洁的一部分，OSD 也能清洗归置组内的对象，也就是说，OSD 会比较归置组内位于不同 OSD 的各对象副本的元数据。清洗（通常每天执行）是为捕获 OSD 缺陷和文件系统错误，OSD 也能执行深度清洗：按位比较对象内的数据；深度清洗（通常每周执行）是为捕捉那些在轻度清洗过程中未能发现的磁盘上的坏扇区。

关于数据清洗的配置见`数据清洗`。

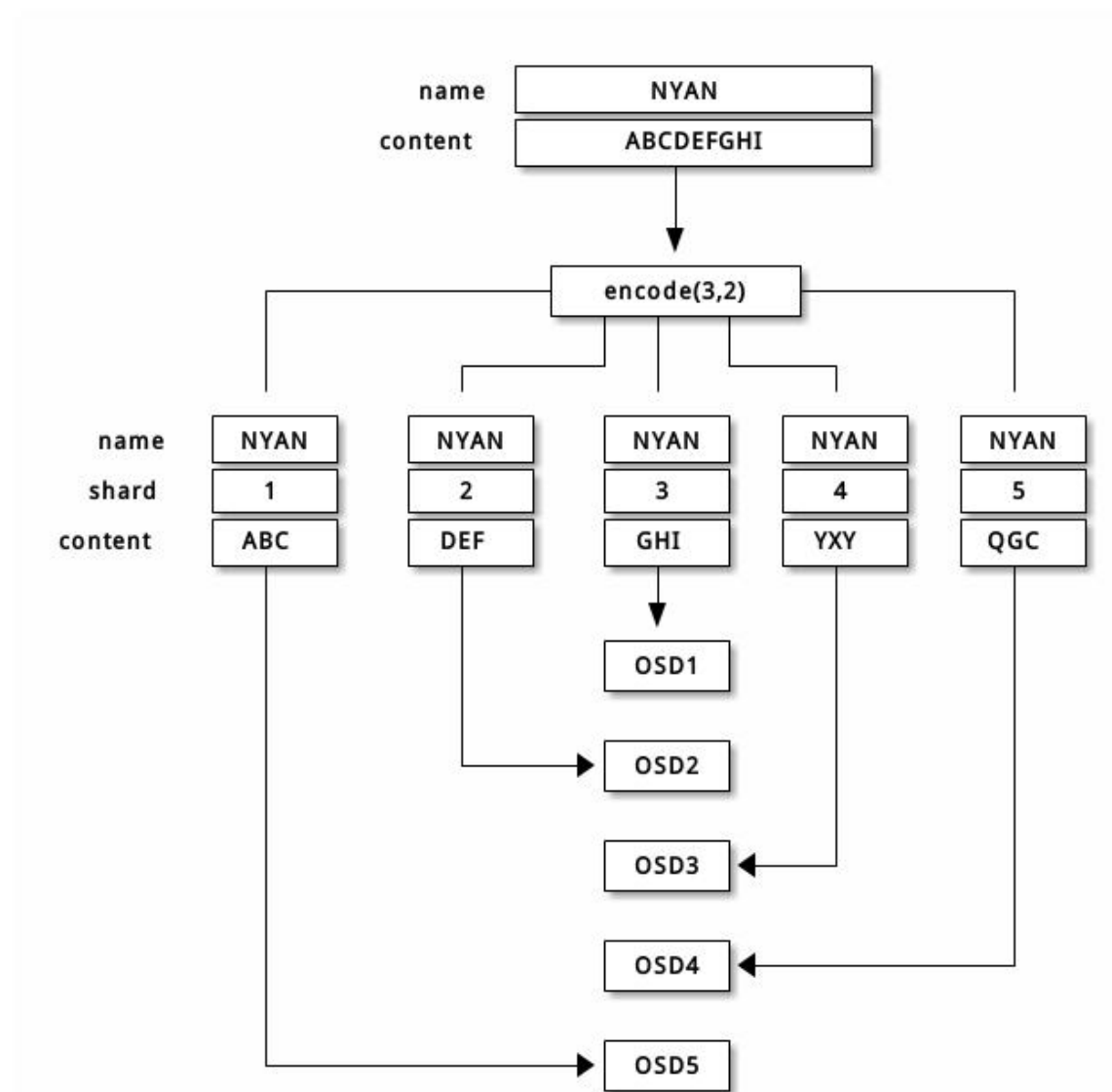
纠删编码

纠删码存储池把各对象存储为 $K+M$ 个数据块，其中有 K 个数据块和 M 个编码块。此存储池的尺寸为 $K+M$ ，这样各块被存储到位于 acting set 中的 OSD，块的位置也作为对象属性保存下来了。

比如一纠删码存储池创建时分配了五个 OSD（ $K+M = 5$ ）并容忍其中两个丢失（ $M = 2$ ）。

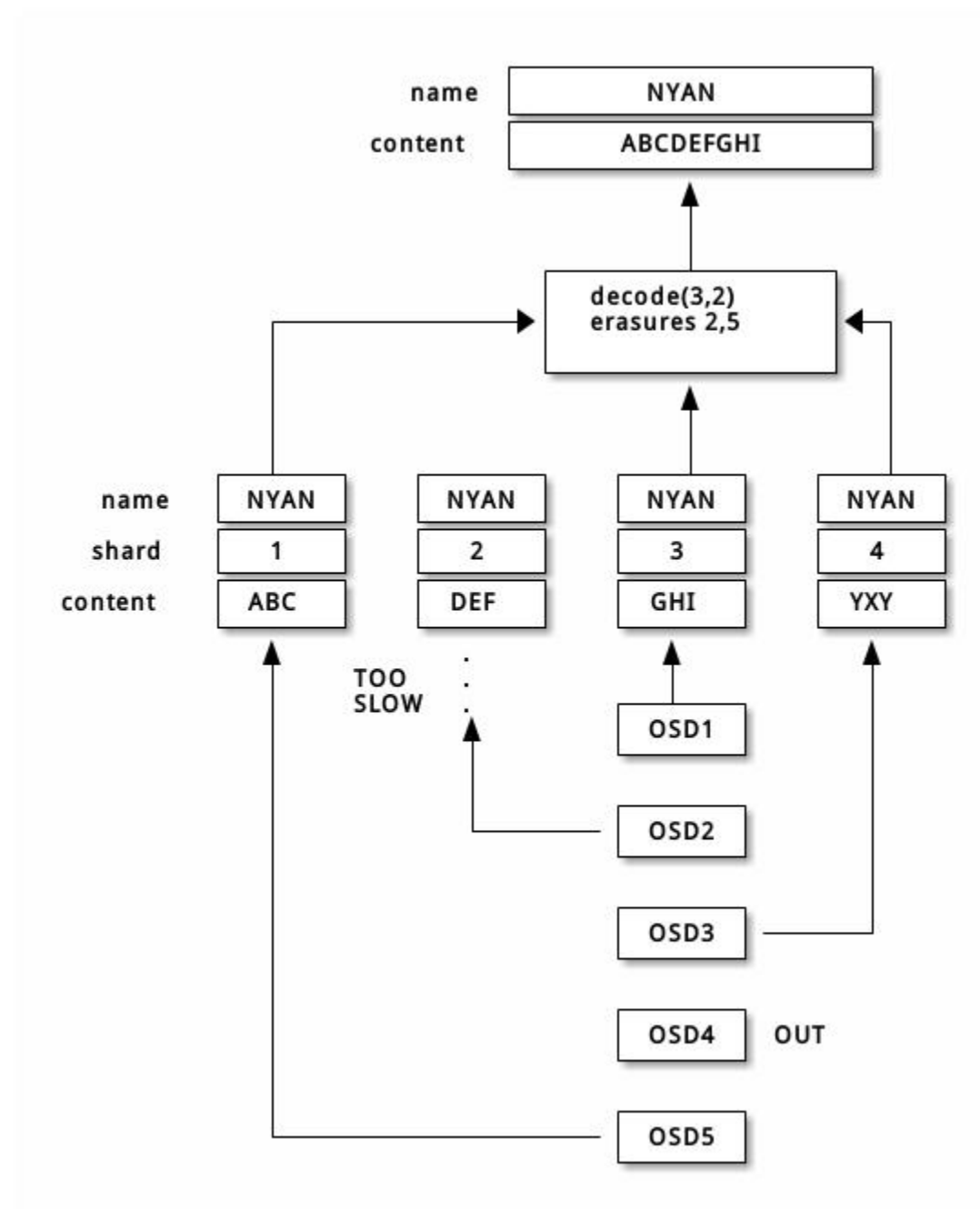
读出和写入编码块

当包含 `ABCDEFGHI` 的对象 `NYAN` 被写入存储池时，纠删编码函数把内容分割为三个数据块，只是简单地切割为三份：第一份包含 `ABC`、第二份是 `DEF`、最后是 `GHI`，若内容长度不是 `K` 的倍数则需填充；此函数还会创建两个编码块：第四个是 `YXY`、第五个是 `QGC`，各块分别存入 acting set 中的 OSD 内。这些块存储到相同名字（`NYAN`）的对象、但是位于不同的 OSD 上；分块顺序也必须保留，被存储为对象的一个属性（`shard_t`）追加到名字后面。包含 `ABC` 的块 1 存储在 **OSD5** 上、包含 `YXY` 的块 4 存储在 **OSD3** 上。



从纠删码存储池中读取 `NYAN` 对象时，解码函数会读取三个块：包含 `ABC` 的块 1，包含 `GHI` 的块 3 和包含 `YXY` 的块 4，然后重建对象的原始内容 `ABCDEFGHI`。解码函数被告

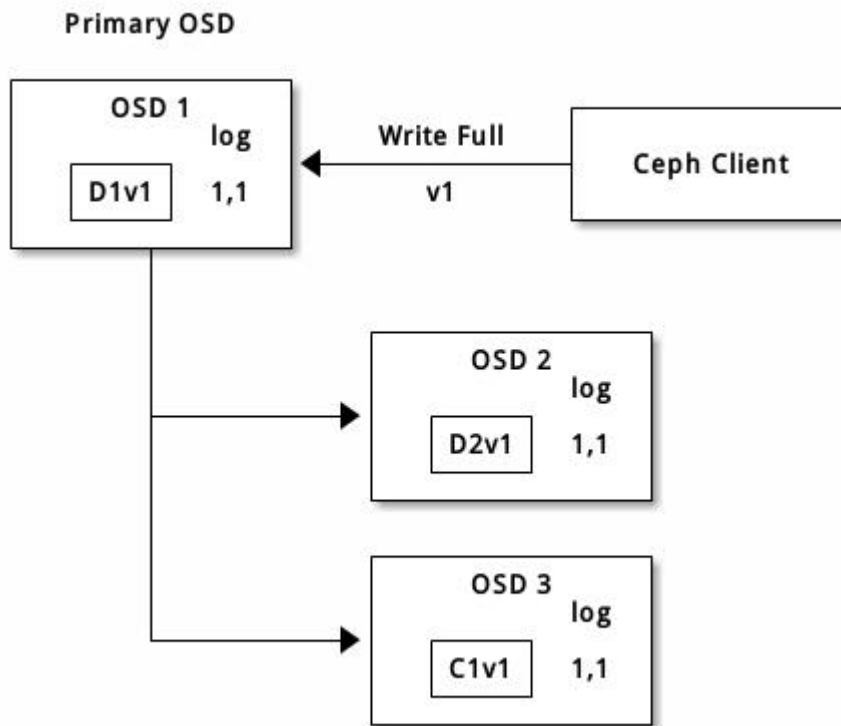
知块 2 和 5 丢失了（被称为“擦除”），块 5 不可读是因为 **OSD4** 出局了。只要有三块读出就可以成功调用解码函数。**OSD2** 是最慢的，其数据未被采纳。



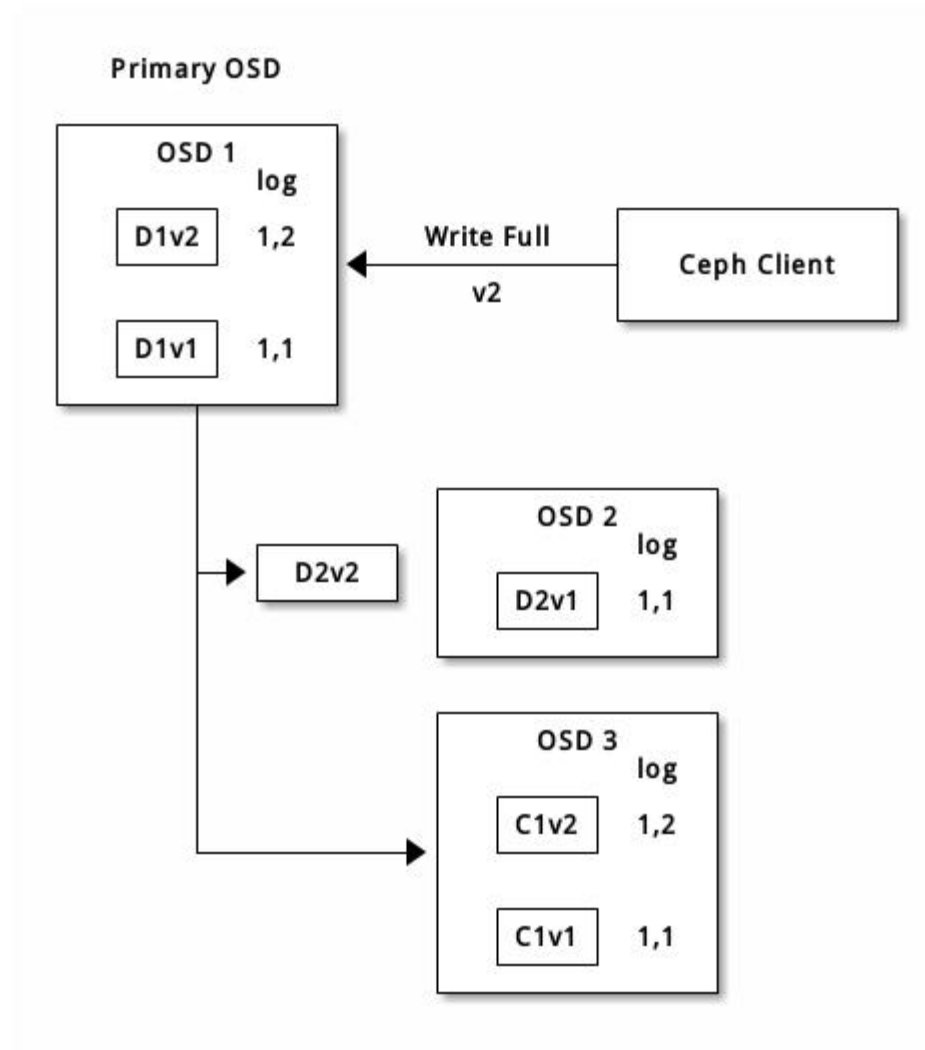
被中断的完全写

在纠删码存储池中，up set 中的主 OSD 接受所有写操作，它负责把载荷编码为 $K+M$ 个块并发送给其它 OSD。它也负责维护归置组日志的一份权威版本。

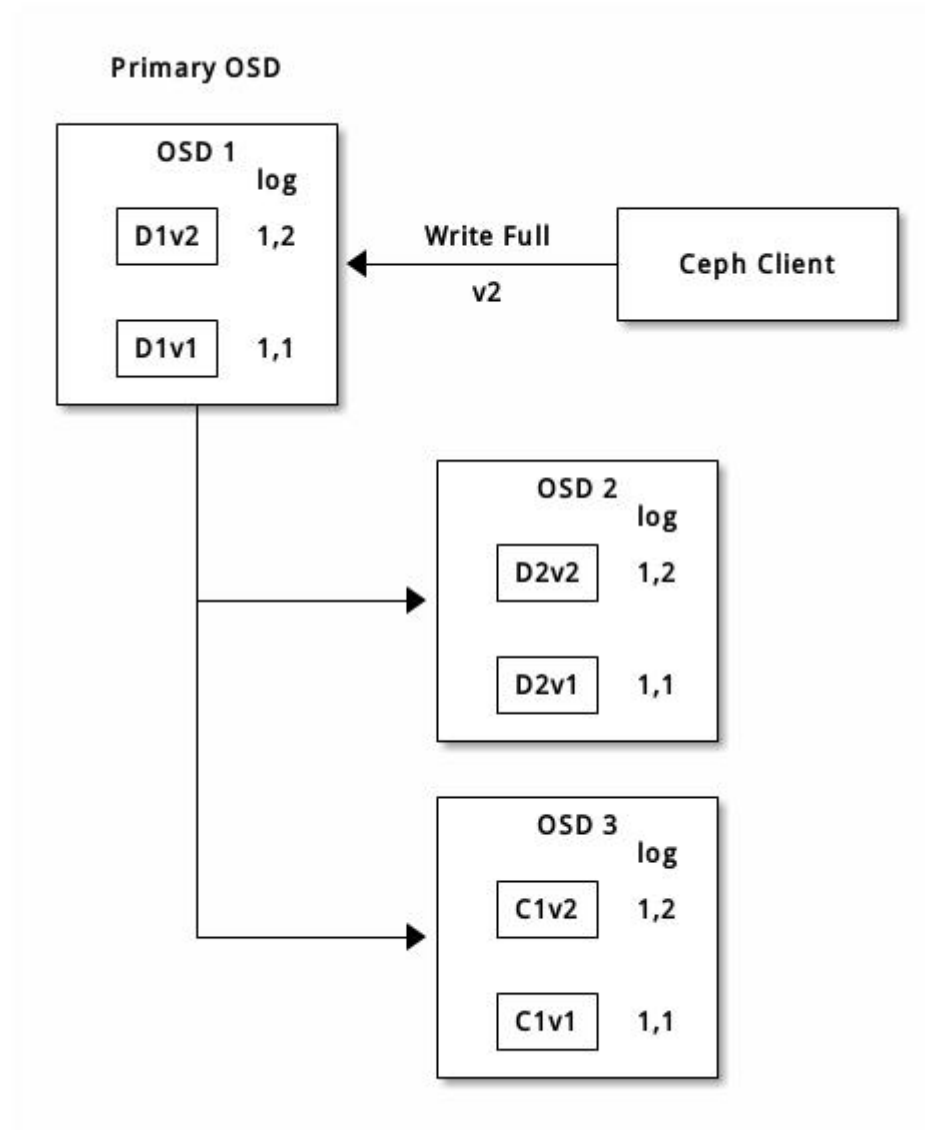
在下图中，已创建了一个参数为 $K = 2 + M = 1$ 的纠删编码归置组，存储在三个 OSD 上，两个存储 K 、一个存 M 。此归置组的 acting set 由 **OSD 1**、**OSD 2**、**OSD 3** 组成。一个对象已被编码并存进了各 OSD：块 $D1v1$ （即数据块号为 1，版本为 1）在 **OSD 1** 上、 $D2v1$ 在 **OSD 2** 上、 $C1v1$ （即编码块号为 1，版本为 1）在 **OSD 3** 上。各 OSD 上的归置组日志都相同（即 1,1，表明 epoch 为 1，版本为 1）。



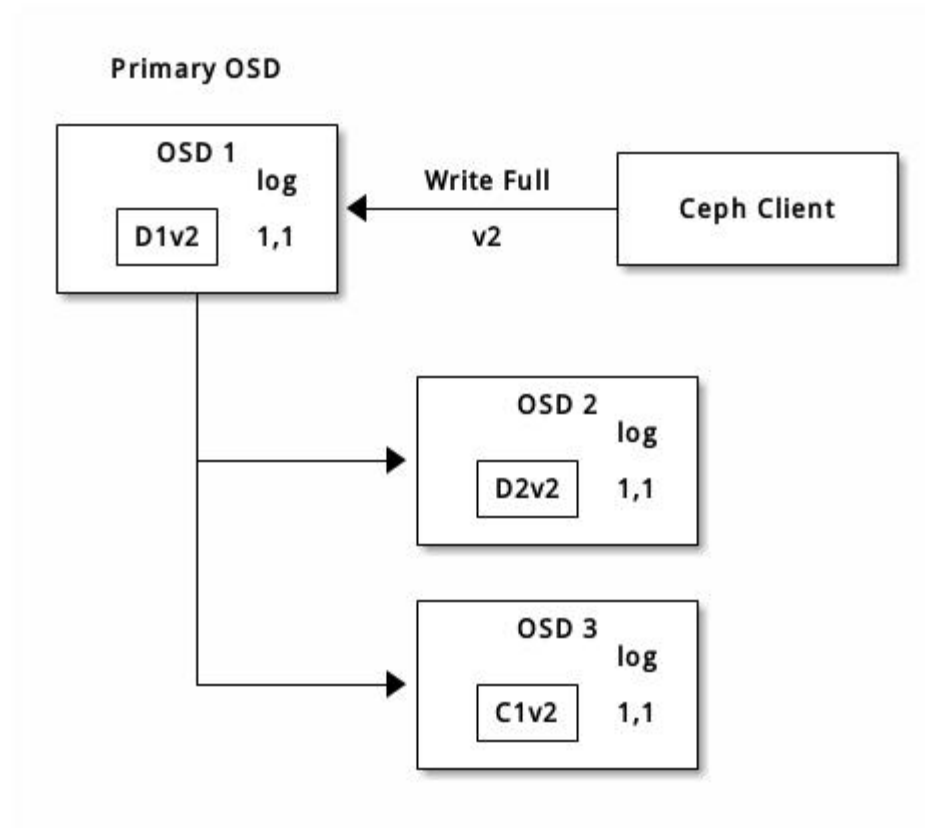
OSD 1 是主的，它从客户端收到了 **WRITE FULL** 请求，这意味着净载荷将会完全取代此对象，而非部分覆盖。此对象的版本 2（ $v2$ ）将被创建以取代版本 1（ $v1$ ）。**OSD 1** 把净载荷编码为三块： $D1v2$ （即数据块号 1、版本 2）将存入 **OSD 1**、 $D2v2$ 在 **OSD 2** 上、 $C1v2$ （即编码块号 1 版本 2）在 **OSD 3** 上，各块分别被发往目标 OSD，包括主 OSD，它除了存储块还负责处理写操作和维护归置组日志的权威版本。当某个 OSD 收到写入块的指令消息后，它也会新建一条归置组日志来反映变更，比如在 **OSD 3** 存储 $C1v2$ 时它会把 1,2（即 epoch 为 1、版本为 2）写入它自己的日志。因为 OSD 间是异步工作的，当某些块还落盘（像 $D2v2$ ），其它的可能已经被确认存在磁盘上了（像 $C1v1$ 和 $D1v1$ ）。



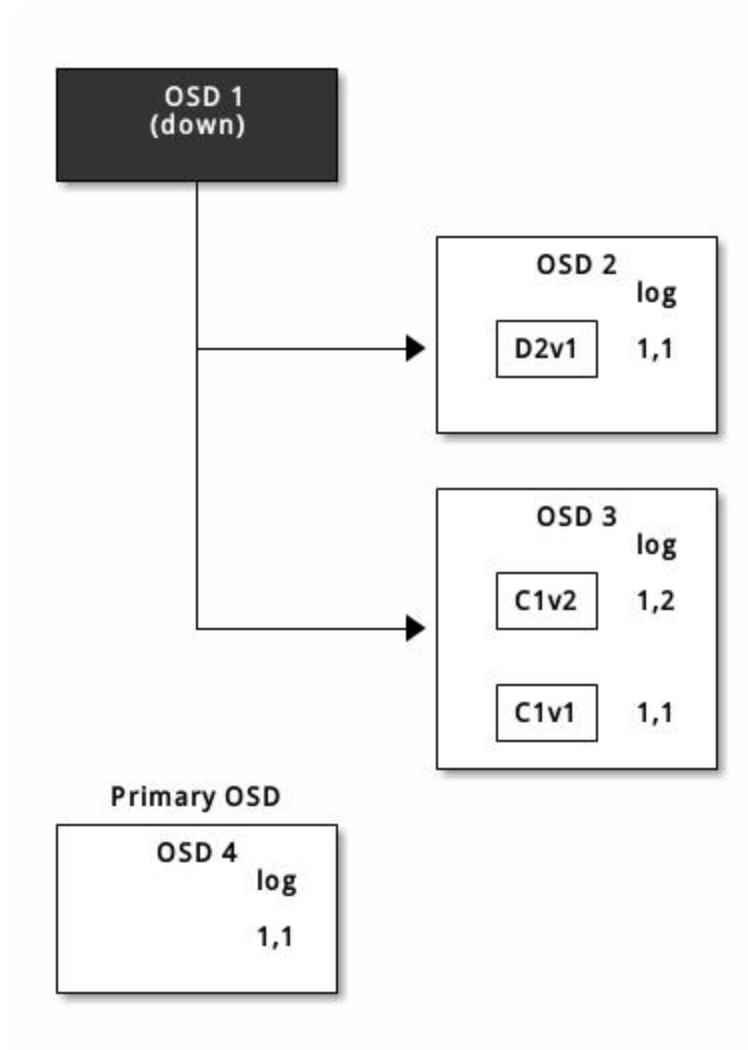
如果一切顺利，各块被证实已在 acting set 中的 OSD 上了，日志的 `last_complete` 指针就会从 `1,1` 改为指向 `1,2`。



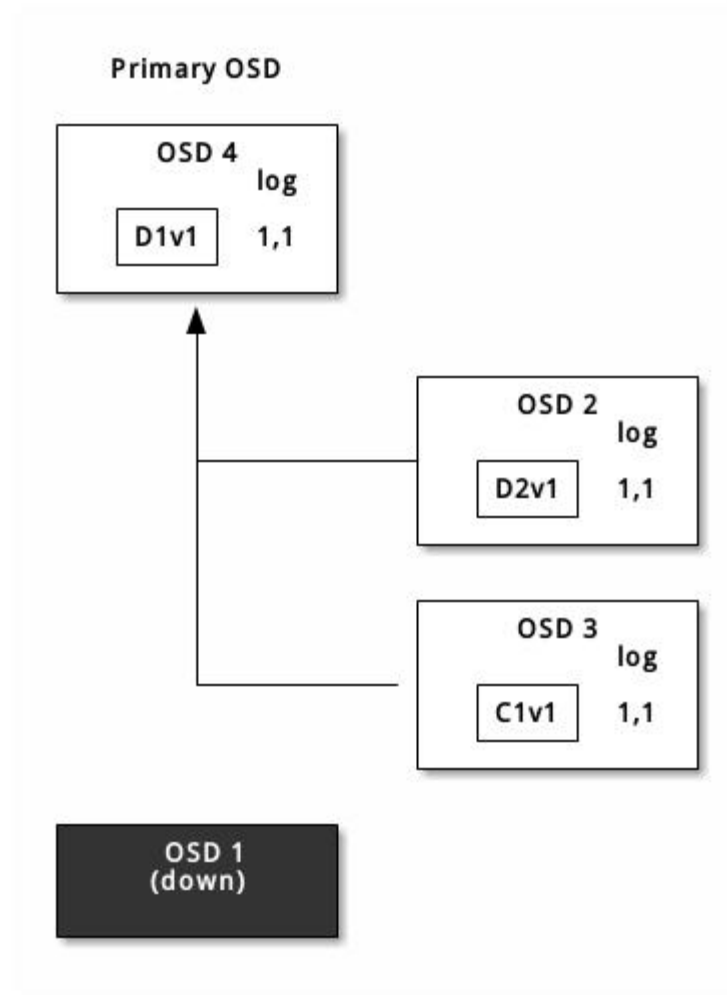
最后，用于存储对象前一版本的文件就可以删除了：OSD 1 上的 D1v1、OSD 2 上的 D2v1 和 OSD 3 上的 C1v1。



但是意外发生了，如果 **OSD 1** 挂了、同时 **D2v2** 仍写完成，此对象的版本 2 一部分已被写入了：**OSD 3** 有一块但是不足以恢复；它丢失了两块：**D1v2** 和 **D2v2**，并且纠删编码参数 $K = 2$ 、 $M = 1$ 要求至少有两块可用才能重建出第三块。**OSD 4** 成为新的主 OSD，它发现 `last_complete` 日志条目（即在此条目之前的所有对象在之前 acting set 中的 OSD 上都可用）是 **1,1** 那么它将是新权威日志的头条。



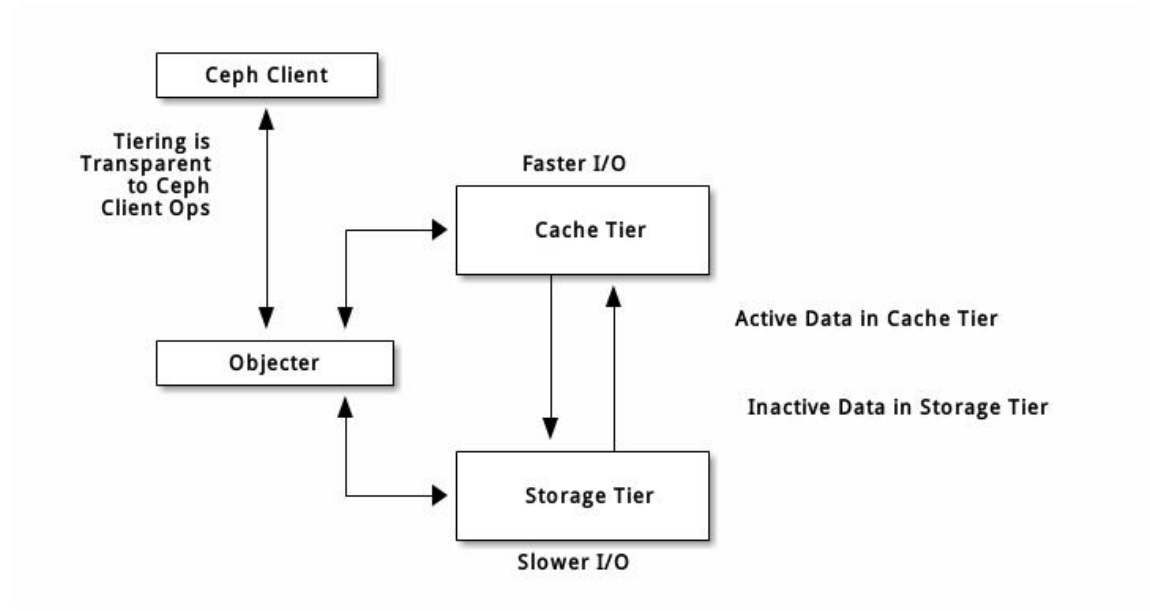
在 **OSD 3** 上发现的日志条目 1,2 与 **OSD 4** 上新的权威日志有分歧：它将被忽略、且包含 **C1v2** 块的文件也被删除。 **D1v1** 块将在清洗期间通过纠删码库的 **decode** 解码功能重建，并存储到新的主 **OSD 4** 上。



详情见[纠删码笔记](#)。

缓存分级

对于后端存储层上的部分热点数据，缓存层能向 Ceph 客户端提供更好的 IO 性能。缓存分层包含由相对高速、昂贵的存储设备（如固态硬盘）创建的存储池，并配置为 缓存层；以及一个后端存储池，可以用纠删码编码的或者相对低速、便宜的设备，作为经济存储层。Ceph 对象管理器会决定往哪里放置对象，分层代理决定何时把缓存层的对象刷回后端存储层。所以缓存层和后端存储层对 Ceph 客户端来说是完全透明的。



详情见[缓存分级](#)。

扩展 CEPH

你可以通过创建 'Ceph Classes' 共享对象类来扩展 Ceph 功能，Ceph 会动态地载入位于 `osd class dir` 目录下的 `.so` 类文件（即默认的 `$libdir/rados-classes`）。如果你实现了一个类，就可以创建新的对象方法去调用 Ceph 对象存储内的原生方法、或者公用库或自建库里的其它类方法。

写入时，Ceph 类能调用原生或类方法，对入栈数据执行任意操作、生成最终写事务，并由 Ceph 原子地应用。

读出时，Ceph 类能调用原生或类方法，对出栈数据执行任意操作、把数据返回给客户端。

Ceph 类实例

一个为内容管理系统写的类可能要实现如下功能，它要展示特定尺寸和长宽比的位图，所以入栈图片要裁剪为特定长宽比、缩放它、并嵌入个不可见的版权或水印用于保护知识产权；然后把生成的位图保存为对象。

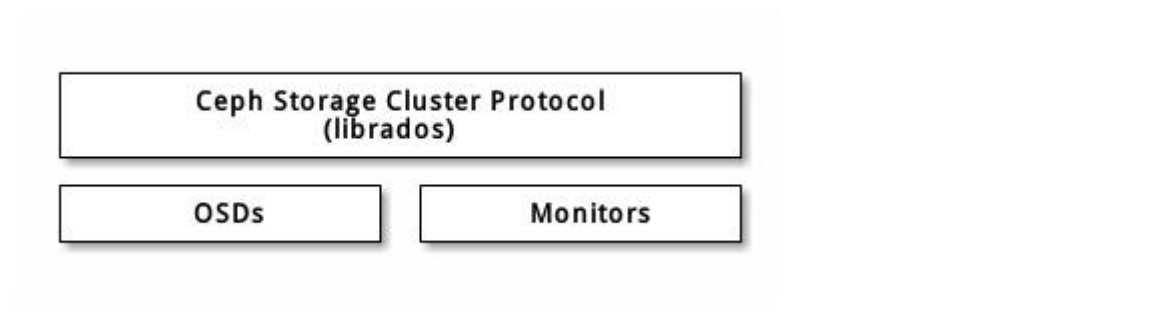
典型的实现见 `src/objclass/objclass.h`、`src/fooclass.cc`、和 `src/barclass`。

小结

Ceph 存储集群是动态的——像个生物体。尽管很多存储设备不能完全利用一台普通服务器上的 CPU 和 RAM 资源，但是 Ceph 能。从心跳到互联、到重均衡、再到错误恢复，Ceph 都把客户端（和中央网关，但在 Ceph 架构中不存在）解放了，用 OSD 的计算资源完成此工作。参考前面的[硬件推荐](#)和[网络配置参考](#)理解前述概念，就不难理解 Ceph 如何利用计算资源了。

CEPH 协议

Ceph 客户端用原生协议和存储集群交互，Ceph 把此功能封装进了 `librados` 库，这样你就能创建自己的定制客户端了，下图描述了基本架构。



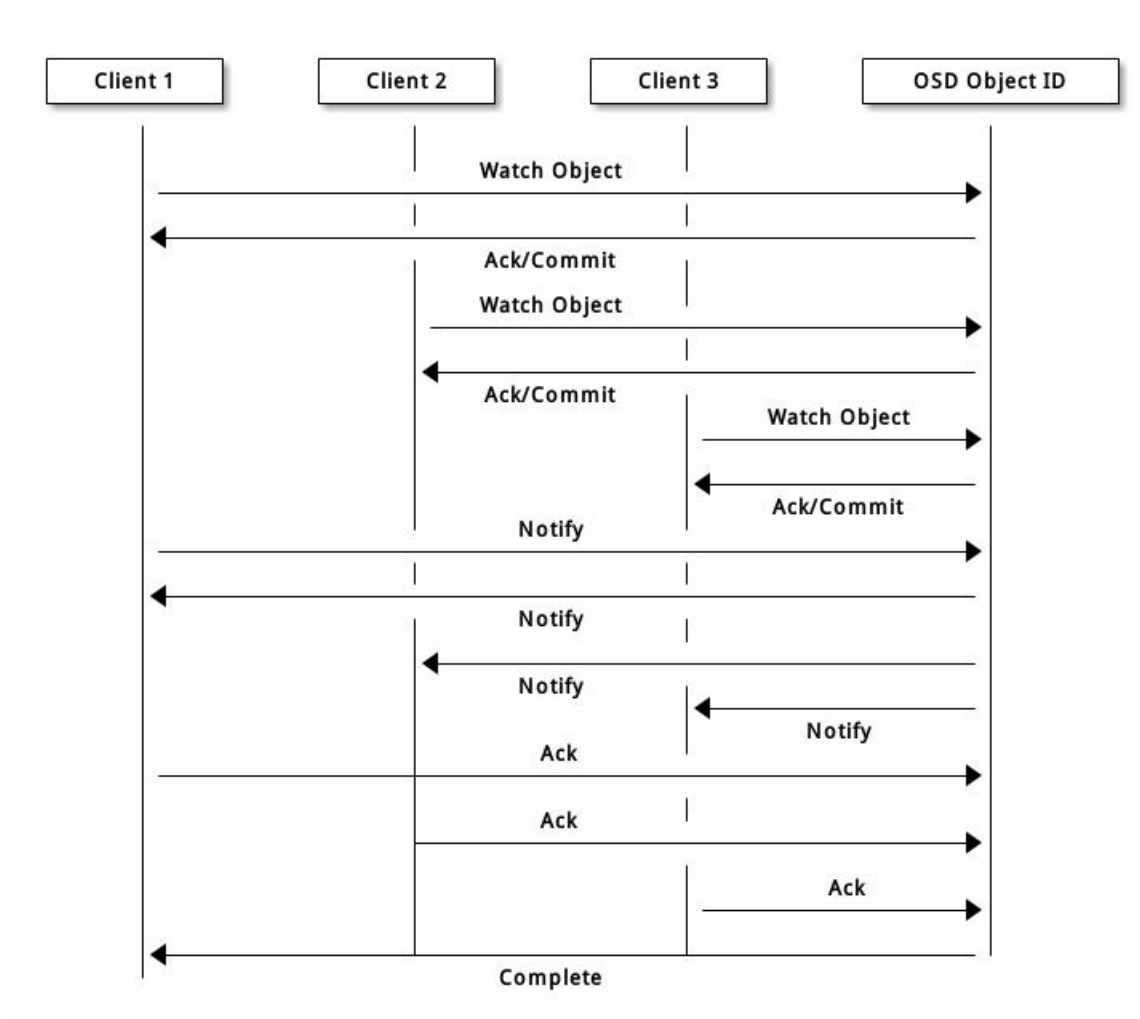
原生协议和 LIBRADOS

现代程序都需要可异步通讯的简单对象存储接口。Ceph 存储集群提供了一个有异步通讯能力的简单对象存储接口，此接口提供了直接、并行访问集群对象的功能。

- 存储池操作；
- 快照和写时复制克隆；
- 读/写对象； - 创建或删除； - 整个对象或某个字节范围； - 追加或裁截；
- 创建/设置/获取/删除扩展属性；
- 创建/设置/获取/删除键/值对；
- 混合操作和双重确认；
- 对象类。

对象监视/通知

客户端可以注册对某个对象的持续兴趣，并使到主 OSD 的会话保持打开。客户端可以发送一通知消息和载荷给所有监视者、并可收集监视者的回馈通知。这个功能使得客户端可把任意对象用作同步/通讯通道。



数据条带化

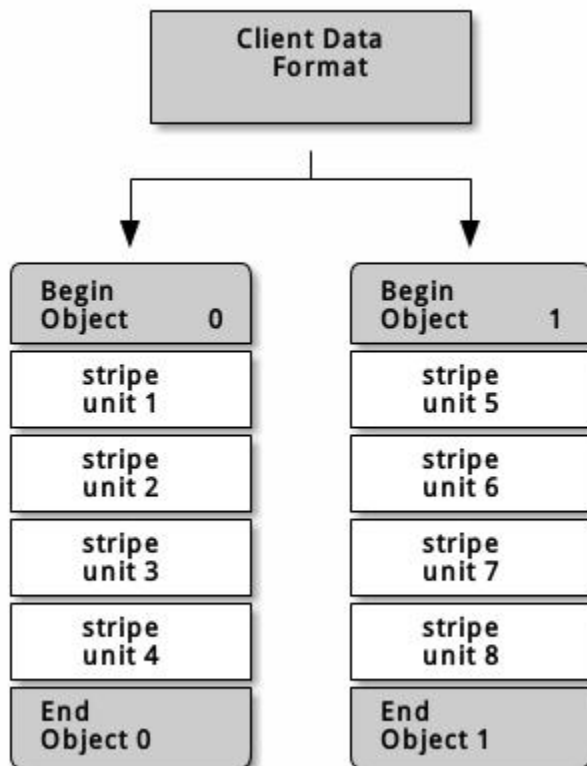
存储设备都有吞吐量限制，它会影响性能和伸缩性，所以存储系统一般都支持**条带化**（把连续的信息分片存储于多个设备）以增加吞吐量和性能。数据条带化最常见于 **RAID** 中，RAID 中最接近 Ceph 条带化方式的是 **RAID 0**、或者条带卷，Ceph 的条带化提供了像 RAID 0 一样的吞吐量、像 N 路 RAID 镜像一样的可靠性、和更快的恢复。

Ceph 提供了三种类型的客户端：块设备、文件系统和对象存储。Ceph 客户端把展现给用户的数据格式（一块设备映像、REST 风格对象、CephFS 文件系统目录）转换为可存储于 Ceph 存储集群的对象。

Tip

在 Ceph 存储集群内存储的那些对象是没条带化的。Ceph 对象存储、Ceph 块设备、和 Ceph 文件系统把他们的数据条带化到 Ceph 存储集群内的多个对象，客户端通过 `librados` 直接写入 Ceph 存储集群前必须先自己条带化（和并行 I/O）才能享受这些优势。

最简单的 Ceph 条带化形式就是一个对象的条带。Ceph 客户端把条带单元写入 Ceph 存储的对象，直到对象容量达到上限，才会再创建另一个对象存储未完的数据。这种最简单的条带化对小的块设备映像、S3、Swift 对象或 CephFS 文件来说也许足够了；然而这种简单的形式不能最大化 Ceph 在归置组间分布数据的能力，也就不能最大化性能。下图描述了条带化的最简形式：

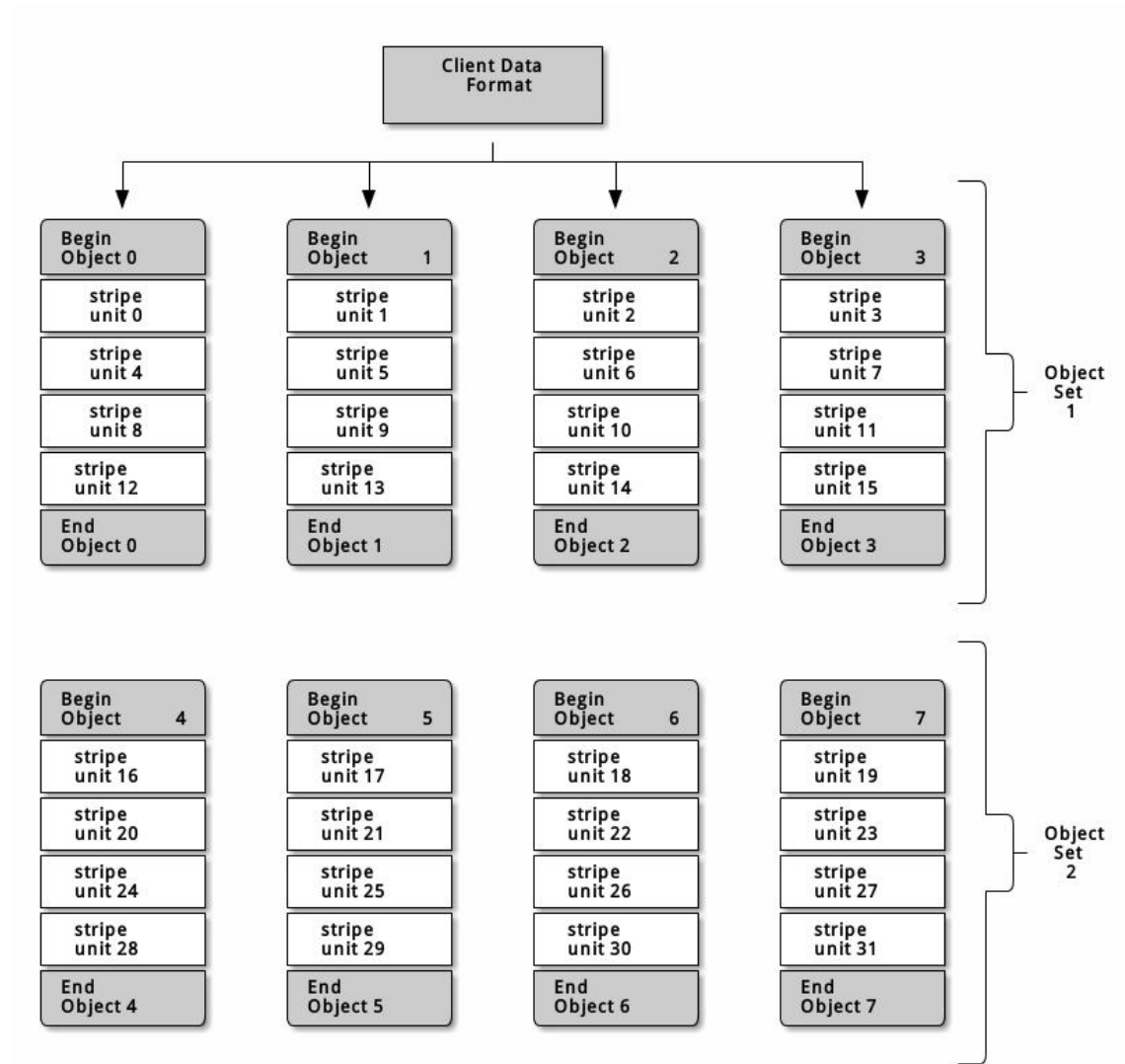


如果要处理大尺寸图像、大 S3 或 Swift 对象（如视频）、或大的 CephFS 目录，你就能看到条带化到一个对象集中的多个对象能带来显著的读/写性能提升。当客户端把条带单元并行地写入相应对象时，就会有明显的写性能，因为对象映射到了不同的归置组、并进一步映射到不同 OSD，可以并行地以最大速度写入。到单一磁盘的写入受限于磁头移动（如：6ms 寻道时间）和存储设备带宽（如：100MB/s），Ceph 把写入分布到多个对象（它们映射到了不同归置组和 OSD），这样可减少每设备寻道次数、联合多个驱动器的吞吐量，以达到更高的写（或读）速度。

Note

条带化独立于对象复制。因为 CRUSH 会在 OSD 间复制对象，数据条带是自动被复制的。

在下图中，客户端数据条带化到一个对象集（下图中的 object set 1），它包含 4 个对象，其中，第一个条带单元是 object 0 的 stripe unit 0、第四个条带是 object 3 的 stripe unit 3，写完第四个条带，客户端要确认对象集是否满了。如果对象集没满，客户端再从第一个对象起写入条带（下图中的 object 0）；如果对象集满了，客户端就得创建新对象集（下图的 object set 2），然后从新对象集中的第一个对象（下图中的 object 4）起开始写入第一个条带（stripe unit 16）。



三个重要变量决定着 Ceph 如何条带化数据：

- **对象尺寸：** Ceph 存储集群里的对象有最大可配置尺寸（如 2MB 、 4MB 等等），对象尺寸必须足够大以容纳很多条带单元、而且应该是条带单元的整数倍。
- **条带宽度：** 条带都有可配置的单元尺寸（如 64KB ）。 Ceph 客户端把数据等分成适合写入对象的条带单元，除了最后一个。条带宽度应该是对象尺寸的分片，这样对象才能包含很多条带单元。
- **条带数量：** Ceph 客户端把一系列条带单元写入由条带数量所确定的一系列对象，这一系列的对象称为一个对象集。客户端写到对象集内的最后一个对象时，再返回到第一个。

Important

把集群投入生产环境前要先测试条带化配置的性能，因为把数据条带化到对象中之后这些参数就不可更改了。

Ceph 客户端把数据等分为条带单元并映射到对象后，用 CRUSH 算法把对象映射到归置组、归置组映射到 OSD，然后才能以文件形式存储到硬盘上。

Note

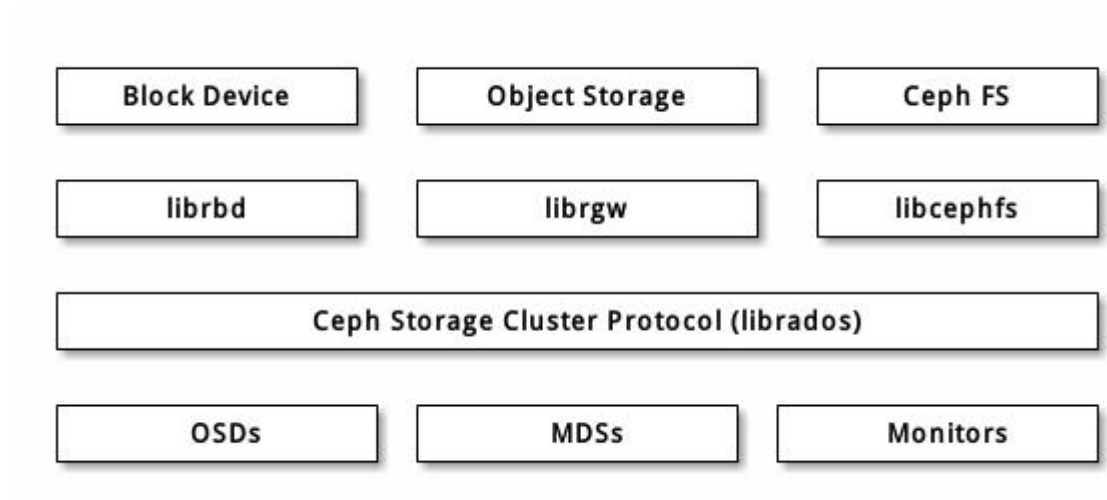
因为客户端写入单个存储池，条带化到对象的所有数据也被映射到同一存储池内的归置组，所以它们要使用相同的 CRUSH 图和相同的访问权限。

CEPH 客户端

Ceph 客户端包括数种服务接口，有：

- **块设备：** *Ceph 块设备*（也叫 RBD）服务提供了大小可调、精炼、支持快照和克隆的块设备。为提供高性能，Ceph 把块设备条带化到整个集群。Ceph 同时支持内核对象（KO）和 QEMU 管理程序直接使用“`librbd`”——避免了内核对象在虚拟系统上的开销。
- **对象存储：** *Ceph 对象存储*（也叫 RGW）服务提供了‘*RESTful 风格*’的 API，它与 Amazon S3 和 OpenStack Swift 兼容。
- **文件系统：** *Ceph 文件系统*（CephFS）服务提供了兼容 POSIX 的文件系统，可以直接 `mount` 或挂载为用户空间文件系统（FUSE）。

Ceph 能额外运行多个 OSD、MDS、和监视器来保证伸缩性和高可靠性，下图描述了高级架构。



CEPH 对象存储

Ceph 对象存储守护进程，`radosgw`，是一个 FastCGI 服务，它提供了 **RESTful 风格** 的 HTTP API 用于存储对象和元数据。它位于 Ceph 存储集群之上，有自己的数据格式，并维护着自己的用户数据库、认证、和访问控制。RADOS 网关使用统一的命名空间，也就是说，你可以用 OpenStack Swift 兼容的 API 或者 Amazon S3 兼容的 API；例如，你可以用一个程序通过 S3 兼容 API 写入数据、然后用另一个程序通过 Swift 兼容 API 读出。

S3/Swift 对象和存储集群对象比较

Ceph 对象存储用 **对象** 这个术语来描述它存储的数据。S3 和 Swift 对象不同于 Ceph 写入存储集群的对象，Ceph 对象存储系统内的对象可以映射到 Ceph 存储集群内的对象；S3 和 Swift 对象却不一定 1:1 地映射到存储集群内的对象，它有可能映射到了多个 Ceph 对象。

详情见 [Ceph 对象存储](#)。

CEPH 块设备

Ceph 块设备把一个设备映像条带化到集群内的多个对象，其中各对象映射到一个归置组并分布出去，这些归置组会分散到整个集群的 `ceph-osd` 守护进程上。

Important

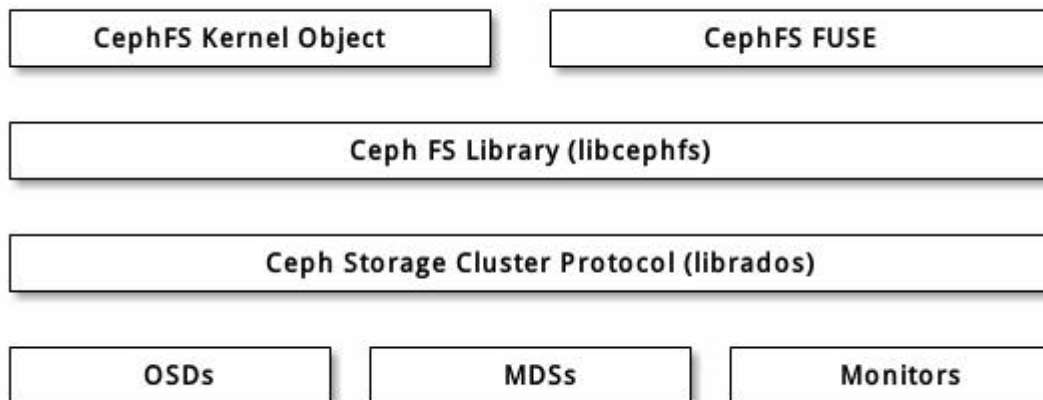
条带化会使 RBD 块设备比单台服务器运行的更好！

精简的、可快照的 Ceph 块设备对虚拟化和云计算很有吸引力。在虚拟机场景中，人们一般会用 Qemu/KVM 中的 `rbd` 网络存储驱动部署 Ceph 块设备，其中宿主机用 `librbd` 向客户机提供块设备服务；很多云计算堆栈用 `libvirt` 和管理程序集成。你可以用精简的 Ceph 块设备搭配 Qemu 和“`libvirt`”来支持 OpenStack 和 CloudStack，一起构成完整的方案。

现在“`librbd`”还不支持其它管理程序，你也可以用 Ceph 块设备内核对象向客户端提供块设备。其它虚拟化技术，像 Xen 能访问 Ceph 块设备内核对象，用命令行工具 `rbt` 实现。

CEPH 文件系统

Ceph 文件系统（Ceph FS）提供与 POSIX 兼容的文件系统服务，坐于基于对象的 Ceph 存储集群之上，其内的文件被映射到 Ceph 存储集群内的对象。客户端可以把此文件系统挂载在内核对象或用户空间文件系统（FUSE）上。



Ceph 文件系统服务包含随 Ceph 存储集群部署的元数据服务器（MDS）。MDS 的作用是把所有文件系统元数据（目录、文件所有者、访问模式等等）永久存储在相当可靠的元数据服务器中内存中。MDS（名为 `ceph-mds` 的守护进程）存在的原因是，简单的文件系统操作像列出目录（`ls`）、或进入目录（`cd`）这些操作会不必要的扰动“OSD”。所以把元数据从数据里分出来意味着 Ceph 文件系统能提供高性能服务，又能减轻存储集群负载。

Ceph FS 从数据中分离出了元数据、并存储于 MDS，文件数据存储于存储集群中的一或多个对象。Ceph 力争兼容 POSIX。`ceph-mds` 可以只运行一个，也可以分布于多台物理机器，以获得高可用性或伸缩性。

- **高可用性：** 多余的 `ceph-mds` 例程可处于 *standby*（待命）状态，随时准备替下之前处于 *active*（活跃）状态的故障 `ceph-mds`。这可以轻易做到，因为所有数据、包括日志都存储在 **RADOS** 上，这个转换过程由 `ceph-mon` 自动触发。
- **伸缩性：** 多个 `ceph-mds` 例程可以同时处于 *active* 状态，它们会把目录树拆分为子树（和单个热点目录的分片），在所有活跃服务器间高效地均衡负载。

Important

译者：虽然文档这么说，但实践中还不推荐这样做，MDS 稳定性尚不理想。多个活跃的 MDS 远没一个稳定，即便如此，您也应该先配置起几个 MDS 备用。

待命（*standby*）和活跃（*active*）MDS 可组合，例如，运行 3 个处于 *active* 状态的 `ceph-mds` 例程以实现扩展、和 1 个 *standby* 例程以实现高可用性。